ARCH LINUX 分册教材

第八册

开发与工具链

编程语言、终端、编辑器、版本控制与构建

依据 Arch Linux 中文维基全量条目编写

共 9 册 · 997 篇 · 安装 / 系统 / 网络 / 硬件 / 桌面 / 软件 / 开发 / 安全

前七册把一台 Arch 机器装好、调稳、联网、配上桌面、装满软件，读者到这里仍只是“使用者”。本册补上另一半：同一台机器如何变成工作台。读完本册，你应该能按任务挑出合适的语言运行时与构建方式，并从官方仓库而不是随手抓来的脚本里装好；能把 shell、终端模拟器与多路复用器调成一套可重复、断线可续的工作环境；能选定一款编辑器并真正用它写代码，而不是每周换一次；能用 Git 管住改动，把本地试过的源码做成 pacman 能装的包；也能在没有图形界面的机器上完成以上全部。合起来只有一句话：把 Arch 当开发平台用，而不是当成品用。

本册在九册中的位置

本册夹在“会用系统”与“守好系统”之间。向前，它依赖第二册：软件包管理给出的安装、编译与打包规矩，是本册构建话题的延长线；也依赖第三册，工具链出错时，内核、权限、服务与日志的知识决定你能否自己查下去。第四册的 SSH 是远程仓库与远程开发的地基，第七册从用户视角看过那批软件，本册换到产出者的位置重看一遍。向后交给第九册：提交签名、密钥管理、构建环境隔离与沙箱，都是本册特意留下的接口。

推荐阅读顺序

前两章（编程语言与开发）是骨架，篇幅也最重，建议按顺序读完，它们决定后面所有工具的使用场景。终端相关的几章——命令行解释器、终端模拟器、命令行本身与终端多路复用器——请当作一组连读，它们互相咬合，拆开读会反复回头。文本编辑器与 Linux 控制台属于个人口味，挑一个读到能干活，另一个留作备查。版本控制系统与 Git 网络界面不论是否写代码都值得花时间，这是少数能立刻改善日常工作的部分。册尾的专项章节（Lisp 方言、TeX、控件工具集、电子设计自动化等）按需查阅，跳读不影响主线；框架一章留给正在选型的人。

前置知识与常见误区

前置是：熟练的命令行操作（管道、重定向、权限、文本处理），第二册的软件包与打包部分——至少能看懂 PKGBUILD，分清官方仓库、AUR 与本地编译的边界；第三册关于用户、权限与服务的基础知识；再加一条能连通的 SSH 通道。这些不牢就先回去补，本册不会重复解释。

几个误区值得先说。其一，以为“搭开发环境”就是装一个 IDE：语言运行时、shell、编辑器、版本控制与构建系统其实是五件独立的事，可以自由组合，本册按部件而非按产品组织。其二，把语言自带的包管理器当作唯一入口，直接往系统目录里堆库文

件，既绕过 pacman 的记账，也会在滚动更新后留下难查的冲突；正解是让系统包与语言级环境各守一段，项目级隔离不是可选项。其三，把终端各章当成程序员专区——它们同样服务于运维、写作与日常操作，跳过是纯损失。

开发工具链

源码到可执行文件



Arch 的打包体系把「源码 → 构建 → 安装」三步显式暴露给用户

编译、打包、版本控制在 Arch 下的位置

目　录

第 1 章　编程语言（17 节）

1.1 Julia　1.2 Erlang　1.3 Crystal　1.4 AOZ Studio　1.5 Lua　1.6 PyPy　1.7 Scala　1.8 Fortran　1.9 AWK　1.10 C　1.11 编程语言　1.12 Go　1.13 Ruby　1.14 PHP　1.15 Python　1.16 Rust　1.17 语言服务器协议

第 2 章　开发（14 节）

2.1 Rbenv　2.2 Xc3sprog　2.3 LLVM　2.4 SDL　2.5 Python/虚拟环境　2.6 Node.js　2.7 核心转储　2.8 Ollama　2.9 Eclipse　2.10 Vulkan　2.11 OpenGL　2.12 Snap　2.13 Flatpak　2.14 GPGPU

第 3 章　Emacs（10 节）

3.1 Emacs/术语　3.2 Emacs/帮助　3.3 Emacs/软件包　3.4 Emacs/运行　3.5 EXWM　3.6 Emacs　3.7 Emacs/疑难解答　3.8 Emacs/提示与技巧　3.9 Emacs/概念　3.10 Emacs/使用

第 4 章　命令行解释器（8 节）

4.1 Oil Shell　4.2 PowerShell　4.3 Bash/函数　4.4 命令行解释器　4.5 Fish　4.6 Bash　4.7 在终端输出彩色　4.8 Zsh

第 5 章　文本编辑器（8 节）

5.1 Helix　5.2 Vi　5.3 Atom (文本编辑器)　5.4 Nano　5.5 Neovim　5.6 Spacemacs　5.7 Visual Studio Code　5.8 Vim

第 6 章　终端模拟器（8 节）

6.1 Mlterm　6.2 Konsole　6.3 Fbterm　6.4 Foot　6.5 Yakuake　6.6 KMSCON　6.7 Alacritty　6.8 Rxvt-unicode

第 7 章　命令行（6 节）

7.1 从终端复制文本　7.2 Readline　7.3 Man 手册　7.4 文件权限与属性　7.5 Dd　7.6 核心工具

第 8 章　Git 网络界面（4 节）

8.1 SonarQube　8.2 Gitweb　8.3 Gitea　8.4 GitLab

第 9 章　Linux 控制台（4 节）

9.1 Gpm　9.2 Getty　9.3 Linux 控制台/键盘配置　9.4 Linux 控制台

第 10 章　框架（4 节）

10.1 Faction　10.2 Mono　10.3 Electron　10.4 .NET

第 11 章　Lisp 方言（3 节）

11.1 Clojure　11.2 Scheme　11.3 Common Lisp

第 12 章　TeX（3 节）

12.1 TeXstudio　12.2 Tectonic　12.3 TeX Live

第 13 章　控件工具集（3 节）

13.1 统一 Qt 和 GTK 应用程序的外观　13.2 Qt　13.3 GTK

第 14 章　终端多路复用器（3 节）

14.1 Abduco　14.2 Kitty　14.3 Tmux

第 15 章　问题跟踪系统（3 节）

15.1 Flyspray　15.2 Bugzilla　15.3 Trac

第 16 章　版本控制系统（2 节）

16.1 Subversion　16.2 Git

第 17 章　电子设计自动化（1 节）

17.1 Xilinx Vivado

第 18 章　集成开发环境（1 节）

18.1 KDevelop

第 1 章　共 17 节　124 KB

编程语言

这一章解决的是"语言装好之后"的事：安装来源、包管理、扩展、JIT 与交叉编译、编辑器和语言服务器接入。十七节详略悬殊，不必按顺序读完。

先读 Python、Rust、PHP、Go、Ruby，这几节篇幅最长，软件包管理、Cargo、扩展与缓存都在里面，日常也最容易撞上；C、AWK 和《编程语言》一节可快速通读，看清各语言的生态位置。Julia、Erlang、Crystal、AOZ Studio、Lua、PyPy、Scala、Fortran 篇幅短，用到再查，《语言服务器协议》留到配编辑器时回看。

最容易踩的坑是混用两套包管理器：语言自带的 pip、RubyGems、Cargo、Shards 与系统仓库的升级节奏不同，装进系统目录常在升级时冲突。另一个误解是以为装完编译器就结束，IDE 支持与语言服务器往往要另配。

1.1　Julia

难度 入门 | **篇幅** 1.4 KB | **所属** 编程语言

本节将讲到

- 安装
- 带编辑器的集成

Julia 是一种高水平、高性能的针对数字计算的动态编程语言。它提供了复杂的编译器、分布式平行执行方式、数字的精准性和广泛的数学函数库。

安装

安装 julia包 包。学习和使用 Julia 请参阅上游文献。

带编辑器的集成

Vim

句法高亮和更多

julia-vim

代码查错

julialint 插件和 Lint.jl 包结合起来可以提供代码查错功能。

1.2　Erlang

难度 入门 | **篇幅** 1.8 KB | **所属** 编程语言

本节将讲到

- 安装

- 用法
- 提示和技巧

Erlang 是一种具有不可变数据和分布式计算特性的编程语言。

安装

安装 erlang包 软件包。

用法

有关更多信息，您可以阅读 Erlang 的文档。

要激活控制台，运行 ：

```
$ erl
```

您可以在其中输入如下命令：

```
Erlang/OTP 22 [erts-10.7.2] [source] [64-bit] [smp:4:4] [ds:4:4:10] [async-threads:1]
[hipe]
Eshell V10.7.2  (abort with ^G)
1> 1 + 2 .
3
2> (2 + 3) * 4 / 5 .
4.0
3>
```

提示和技巧

Emacs 模式

要设置包含的 Emacs 模式， `erlang-mode` ，请遵循文档（OTP 路径为 `/usr/lib/erlang`）。

1.3 Crystal

难度 入门 | **篇幅** 1.8 KB | **所属** 编程语言

本节将讲到

- 安装
- 用法
- Shards
- 参见

Crystal) 是一种静态的编译型编程语言，具有受 Ruby 启发的语法和全局类型推断。

安装

要安装 Cryst，安装 crystal包。要安装最新的开发版本，请安装 crystal-gitAUR。

用法

编译和运行 Crystal 程序：

```
$ crystal hello_world.cr
```

要将 Crystal 程序编译为二进制文件：

```
$ crystal build hello_world.cr
```

编译为优化的二进制文件：

```
$ crystal build --release hello_world.cr
```

有关更多选项，请参见：

```
$ crystal help
```

Shards

仓库中也提供了依赖管理器 shards。要安装它，请安装 shards包。要安装最新的开发版本，请安装 shards-gitAUR。

参见

- 官方网站
- 官方文档
- 标准库参考
- 官方 GitHub 仓库
- 在线代码评估
- #crystal-lang IRC 频道

1.4 AOZ Studio

难度 入门 | **篇幅** 1.9 KB | **所属** 编程语言

本节将讲到

- 安装
- 外部链接

最初，AMOS BASIC 是在 Amiga 上实现的 BASIC 编程语言的方言，受到 Atari ST 计算机版本 STOS 的启发。AMOS BASIC 由 Europress Software 发布，最初由 François Lionet 与 Constantin Sotiropoulos 编写。这是一个非常成功的产品，许多程序员在家庭计算的早期就开始学习在该平台上进行编程。

2019 年 4 月 4 日，AMOS2 宣布免费下载。是一种使用 JavaScript 作为解释器的独立于系统的 BASIC 语言。

安装

要安装最新的稳定版本的 AMOS2，需要从开发人员网站上获取该软件包。

外部链接

- AMOS2 网站 - AMOS2 官方网站。

1.5 Lua

难度 入门 | **篇幅** 2.1 KB | **所属** 编程语言

本节将讲到

- 安装
- JIT 编译支持
- 模块
- 参见

根据其自己的网站 lua.org：

: Lua 是一种功能强大，高效，轻巧，可嵌入的脚本语言。

这也使 Lua 非常适合配置和扩展其他程序，这也是因为它与 C 的互操作性强。 基本的 Lua 解释器的大小小于 250K。由于官方实现是用 ANSI C 编写的，因此可用于许多平台和体系结构。

安装

有多个版本可用：

- lua包——Lua 5.5
- lua54包——Lua 5.4
- lua53包——Lua 5.3
- lua52包——Lua 5.2
- lua51包——Lua 5.1

JIT 编译支持

即时编译（Just-in-time compilation）是一种在运行时而不是之前编译源代码的方法。LuaJIT 是 Lua 5.1 的直接替代品。此版本最适合更高性能的约束。

为了获得 JIT 编译支持，请安装 luajit包 软件包。

模块

LuaRocks 软件包管理器作为 luarocks包 提供。

官方存储库中也提供了一些模块。

参见

- Lua 参考手册。

1.6 PyPy

难度 入门 | **篇幅** 3.2 KB | **所属** 编程语言

本节将讲到

- 安装
- 用法
- EasyInstall

PyPy 是Python 2.7、3.9 和 3.10 解释器的替代实现。PyPy 的优势在于速度、内存使用、沙箱和无堆栈性。它与 CPython 兼容，有一些例外。PyPy 还可用于将 RPython 程序编译为 C 代码。

安装

对于 Python 2.7，安装 pypy包 软件包。对于 Python 3.10，安装 pypy3包 软件包。

PyPy 安装在 `/opt/pypy/` 或 `/opt/pypy3` 中，主 pypy 可执行文件是 `bin/pypy-c`。

用法

PyPy 的基本用法通过 `pypy` 或 `pypy3` 命令完成，其功能与 CPython 的用法类似。输入

```
$ pypy -h
```

以查看 `pypy` 选项列表。

交互式解释器

若要加载 PyPy 交互式解释器，运行

```
$ pypy
```

从文件运行程序

要在 PyPy 中从文件运行 Python 程序，运行

```
$ pypy example.py
```

创建虚拟环境

用 PyPy 创建虚拟环境：

```
$ virtualenv --python=/usr/bin/pypy venv-pypy
```

更多信息，请参见 Python/虚拟环境。

安装 pip

由于 PyPy 的 Python 软件包不作为 Arch 软件包发布，因此最方便的做法是以自己的用户身份安装所需软件：

```
$ pypy -m ensurepip --user
$ pypy -m pip install --user --upgrade pip
```

有了 ***pip***，你就可以安装任何需要的软件包，例如 `sqlalchemy`：

```
$ pypy -m pip install --user sqlalchemy
```

如果你想在全系统范围内安装软件包，只需以 root 用户身份运行前面的命令，而不使用 `--user`。请注意，这将导致软件包被安装到 `/opt/pypy` 中，而软件包管理器并不知道它们的存在。

EasyInstall

Python 库和程序可通过 EasyInstall 安装在 PyPy 中。

EasyInstall 安装

EasyInstall 并不与 PyPy 软件包一起提供，而是在安装 **pip** 时自动安装，位于 `/opt/pypy/bin/easy_install` 中。

安装 EasyInstall 软件包

要将 EasyInstall 软件包 `*package_name*` 安装到 PyPy 中，请输入

```
# /opt/pypy/bin/easy_install package_name
```

软件包位于 `/opt/pypy/site-packages`。安装的库和应用程序将位于 `/opt/pypy/bin`。通过 EasyInstall 安装在 PyPy 上的程序通常可以通过 `/opt/pypy/bin/program_name` 运行，其中 ***program_name*** 是 PyPy 程序的名称。

EasyInstall 软件包示例

以下操作将安装 Lamson 电子邮件框架：

```
# /opt/pypy/bin/easy_install lamson
```

以下命令将运行框架的 `gen -project` 命令：

```
$ /opt/pypy/bin/lamson gen -project testproject
```

1.7 Scala

难度 入门 | **篇幅** 3.4 KB | **所属** 编程语言

本节将讲到

- 安装

- 使用和 IDE
- 参见

From Wikipedia):

: *Scala is an object-functional programming and scripting language for general software applications. Scala has full support for functional programming (including currying, pattern matching, algebraic data types, lazy evaluation, tail recursion, immutability, etc.) and a very strong static type system. This allows programs written in Scala to be very concise and thus smaller in size than most general purpose programming languages. Many of Scala's design decisions were inspired by criticism over the shortcomings of Java.*

安装

从 official repositories 中安装 scala包。此外您也可以选择安装 scala-docs包 和 scala-sources包 。

因为 scala 运行在 JVM (Java Virtual Machine) 上，所以为了编译和运行 scala 程序，您也需要一个完整的 Java Runtime Environment (JRE)。

对于 scala3 (也被称作 dotty), 您可以从 AUR 中安装 scala3AUR 或者 scala-dottyAUR。 请注意 scala3 **不兼容** scala 2.13 及更早版本.

构建工具

多数构建工具可以在官方 community 仓库中安装:

- sbt包
- maven包
- gradle包

使用和 IDE

像 Python 等语言一样, 您可以在解释器中运行交互程序。

```
$ scala

Welcome to Scala version 2.*.* (OpenJDK Server VM, Java 1.*.*).
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

也可以简单地从命令行编译并运行程序。

```
$ scalac HelloWorld.scala
$ scala HelloWorld
```

许多不同的 IDE 例如 Eclipse 和 Netbeans 都提供 scala 支持。 scala-ideAUR package for example is available in the AUR. But you can also download an IDE that

is optimized for Scala and also based on Eclipse directly from the official Scala Website.

参见

- Scala Lang - 官方网站
- Scala Tutorial - 一系列小型 scala 入门教程
- Learn X=Scala in Y minutes

1.8 Fortran

难度 入门 | **篇幅** 4.1 KB | **所属** 编程语言

本节将讲到

- 编译器
- 附加软件包
- 参阅

Fortran 是一门通用型、编译式命令式编程语言，特别适用于数值计算和科学计算。

来自 Fortran 社区（fortran-lang）网站的说明：

: Fortran 是一门成熟且处于积极开发中的语言。其最新修订版本为 Fortran 2023。现有许多的开源与专有 Fortran 编译器。此外，诸如标准库和 Fortran 包管理器（Fortran Package Manager，FPM）等开源项目也正处于活跃开发阶段。

编译器

gcc-fortran包 软件包可为 GCC 启用 Fortran 前端。

其他适用于 Arch Linux 的 Fortran 编译器包括：

- **lfortran** — 基于 LLVM 构建的现代交互式 Fortran 编译器。

: https://lfortran.org || lfortranAUR

- （**LLVM**） **Flang** — 采用现代 C++ 实现的 Fortran 2018 新 LLVM 前端。

: https://flang.llvm.org/ || flangAUR

- **Flang（经典版）** — 面向 LLVM 的独立 Fortran 编译器（未来将被新版 Flang 取代）。

: https://github.com/flang-compiler/flang || flang-classicAUR

另请参阅 fortran-lang 获取开源和商业编译器列表。请查阅您所用编译器的文档以确认其符合哪些标准及提供了哪些语言扩展功能。

附加软件包

- **Fortran Standard Library** — 旨在为 Fortran 提供社区驱动且达成共识的事实"标准"库（目前处于 alpha 阶段）。

: https://stdlib.fortran-lang.org/ || fortran_stdlibAUR

- **Fortran Package Manager** — Fortran 的包管理器与构建系统（目前处于 alpha 阶段）。

: https://fpm.fortran-lang.org/ || fortran-fpmAUR、fortran-fpm-binAUR

- **fortls** — Fortran 语言服务器。

: https://fortls.fortran-lang.org/ || fortlsAUR、fortlsCNRepo

- **FORD** — 通过代码注释生成文档。

: https://github.com/Fortran-FOSS-Programmers/ford || fordAUR

- **f2c** — Fortran 转 C 代码转换器。

: https://www.netlib.org/f2c/ || f2cAUR

另请参阅 fortran-lang 软件包索引。

参阅

- FortranCon2021/Fortran-lang：Fortran 现状（YouTube 视频）
- fortran-lang 现代 Fortran 学习资源
- Fortran 维基
- J3（美国）与 WG5（国际） Fortran 标准委员会
- 左志华先生所著《Ch27-Fortran 编程》

1.9 AWK

难度 入门 | **篇幅** 4.2 KB | **所属** 编程语言

本节将讲到

- 安装
- 疑难解答
- 参见

AWK 是一种用于文本处理的小型编程语言。其名称源于作者的姓氏：Alfred Aho、Peter Weinberger 和 Brian Kernighan。该语言已经标准化，在类 Unix 系统上广泛使用。

安装

在 Arch 上，awk(1p) 命令由 gawk包 提供。gawk包 是默认安装，支持本地 Unicode，并具有大量额外功能。

注意：gawk包旨在通过动态加载所谓的*扩展*，在运行时获得额外的语言功能——捆绑在软件发行版中的扩展是开箱即用的，但单独维护的 gawkextlibAUR\-* 插件则需要手动安装。

替代方案

与许多其他核心工具一样，有几种或多或少符合要求的实现可供使用：

- **BusyBox** — BusyBox 实现的性能不是很好，但占用空间较小，适合内存紧张的环境。

: https://git.busybox.net/busybox/tree/editors/awk.c || busybox包

- **GoAWK** — 用 Go 语言实现的 AWK

: https://github.com/benhoyt/goawk || goawkAUR

- **nawk** — *AWK 编程语言*中描述的"新" AWK，又名 BWK AWK 或 One-True-AWK，现由 Arnold Robbins 和 B. W. Kernighan 共同维护，支持 UTF-8 和 csv。

: https://awk.dev/ || nawk包

- **mawk** — 性能相当出色的 AWK 实现。

: https://invisible-island.net/mawk/ || mawkAUR

疑难解答

通过 `-v` 选项为 `ARGC` 变量赋值不会在运行时保留

尽管没有文档说明，但许多实现在处理完命令行指定的 `-v` 选项的变量赋值后，似乎会在内部重置 `ARGC` 变量。因此，要在运行时（例如 `BEGIN` 代码块）获得 `ARGC` 变量的预期值，需要直接在代码块中设置该变量：

```
BEGIN {
  ARGC=1;
  ...
}
```

注意：即将更新的 POSIX 专门记录了这个问题。

参见

- nawk(1): nawk包的参考卡式手册页面
- Gawk: Effective AWK ProgrammingGNU 文档

：gawk(1) 的综合教程和典型参考文献

- Alpine Linux's community wiki article on AWK: 有一些关于 BusyBox 和其他 AWK 实现差异的说明
- AWK tech notes：语言的缺陷以及与其他现代编程语言相比在设计上的差异
- Idiomatic AWK：AWK 程序可以多么简洁

1.10 C

难度 入门 | **篇幅** 5.6 KB | **所属** 编程语言

本节将讲到

- 实用工具
- 其他的编译器
- 其他的 libc 实现
- 库
- 参见

Linux 内核和 GNU 用户空间主要由 C) 写成。

Arch Linux 使用GNU C Library（glibc包）作为 C 标准库；它是 base包 元软件包的一个依赖项。

你可以使用 GNU 工具链或者 LLVM 工具链来用 C、C++ 或 Objective-C 开发软件。

实用工具

- **Valgrind** — 用来找到程序里内存管理问题的工具。

: https://valgrind.org/ || valgrind包

- **Distcc** — 分布式编译的 GCC 前端。

: https://github.com/distcc/distcc || distcc包

- **rr** — 针对 C/C++ 的轻量的记录和定性调试工具，用的是 GDB。

: https://rr-project.org/ || rrAUR

静态代码分析

- **Cppcheck** — 静态 C/C++ 代码分析工具。

: http://cppcheck.sourceforge.net/ || cppcheck包

- **Splint)** — 静态检查 C 程序安全问题和代码错误的工具。

: https://repo.or.cz/splint-patched.git || splint包

- Clang 提供 *scan-build* 静态分析器。

其他的编译器

- **TCC** — 微型的 C 编译器，声称比 GCC 快。

: https://bellard.org/tcc/ || tcc包

- **ACK** — 阿姆斯特丹编译包。

: http://tack.sourceforge.net/ || ack-gitAUR

- **PCC** — 可移植的 C 编译器。

: http://pcc.ludd.ltu.se/ || pccAUR

- **SDCC** — 可重定向的 ANSI C 编译器。

: https://sdcc.sourceforge.net/ || sdcc包

另见 Wikipedia:List of compilers#C compilers。

其他的 libc 实现

- **dietlibc** — 为尺寸优化的 libc

: https://www.fefe.de/dietlibc/ || dietlibcAUR

- **musl** — C 标准库的轻量实现。

: https://musl.libc.org/ || musl包

库

- FFmpeg - 包含 libav：音视频库（不要与同名的 FFmpeg 分支混淆）。
- **GLib** — GNOME 的底层库，包括 GObject 和 GIO。

: https://wiki.gnome.org/Projects/GLib || glib2包

- GStreamer – 基于流水线的多媒体框架

另请参阅：

- GTK/Development#C
- Desktop notifications#C
- Libcanberra#C
- Archiving and compression#Compression libraries

- Wikipedia:Category:C (programming language) libraries_libraries)
- A list of open source C libraries

参见

- 手册页在第二区的系统调用
- 手册页在第三区的库函数
- GCC and Make – Compiling, Linking and Building C/C++ Applications
- SEI CERT C Coding Standard
- #C IRC channel

1.11 编程语言

难度 进阶 | **篇幅** 7.1 KB | **所属** 编程语言

本节将讲到

- Shell 语言
- 参阅

翻译状态：

- 本文（或部分内容）译自 Programming languages，最近一次同步于 2025-05-08，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

本页面尝试列出所有编程语言，包含适用于 Arch Linux 的已打包编译器/解释器。

- 汇编语言——binutils包 的 *as*、fasm包、nasm包、yasm包
- Ada)——Ada
- AWK
- Ballerina)——ballerinaAUR
- BASIC——freebasic包
- AOZ Studio
- Gambas
- QB64)——qb64AUR
- C——GCC、Clang
- C++——GCC、Clang
- Objective-C——GCC、Clang
- C#)——Mono、.NET Core
- Crystal
- D
- Dart)——dart包
- Erlang)——erlang包

- Elixir
- Factor)——factorAUR
- Forth)——4thAUR、gforthAUR、retroforthAUR
- Fortran——GCC 等
- Go——go包、GCC
- Java
- Groovy——groovy包
- GDL——gnudatalanguageAUR
- Haskell
- JavaScript——rhino包、v8AUR\[损坏的链接：package not found\]、quickjsAUR
- Node.js
- Julia
- Kotlin)——kotlin包
- Lisp) 方言
- Clojure
- Common Lisp
- Emacs Lisp
- Scheme
- Racket)——racket包、racket-minimal包
- newLISP)——newlispAUR
- PicoLisp)——picolispAUR
- Lua——lua包、luajit包
- ML)
- Standard ML——smlnj包、mlton包、polyml包、mosmlAUR、smlsharpAUR
- OCaml——ocaml包
- Nim)——nim包
- Octave
- Pascal)——fpc包、lazarus包
- Perl
- PHP
- Python
- QCL——qclAUR
- R
- Raku (Perl 6))——rakudoAUR
- Ruby
- Rust
- Scala
- Swift
- Tcl——tcl包

- Vala)——vala包
- Zig——zig包

Shell 语言

Bash 是 base包 元软件包的依赖项。其他相关项目请参阅命令行解释器。

参阅

- 编程语言比较
- 编程语言历史
- 编程语言列表
- 按类型分类的编程语言列表

1.12 Go

难度 进阶 | **篇幅** 8.1 KB | **所属** 编程语言

本节将讲到

- 安装
- 提示和技巧
- 故障排除
- 参见

Go 是一门受 Google 支持的开源编程语言。根据 Go 文档：

: Go 表达力强、简洁、干净、高效。它的并发机制使编写能够最大限度地利用多核和联网计算机的程序变得简单，而它新颖的类型系统则使得灵活和模块化程序构造变得可能。Go 可以快速编译成机器代码，同时还具有垃圾回收的便利性和运行时反射的强大功能。它是快速、静态类型的编译语言，同时感觉就像是动态类型的解释型语言。

安装

安装 go包 软件包，这包含标准 Go 编译器和其他开发工具。可在 *go* 命令文档（英文）中查看包含的子命令。

其他编译器

go包 软件包包含标准 Go 编译器（名为 *gc*）。还可使用下面这些编译器替代。

gccgo

gcc-go包 软件包提供了 GCC 的一个前端 gccgo。在某些情况下，*gccgo* 生成的二进制文件可能比 *gc* 更快，而且可以为更多操作系统和架构生成二进制文件。实践中，对于几乎所有的负载，*gc* 生成的二进制文件都比 *gccgo* 更快。

TinyGo

TinyGo 是一个基于 LLVM 的编译器，旨在为嵌入式系统和 WebAssembly 生成非常小的二进制文件。其由 tinygo包 软件包提供。

工具

这些软件包提供 Go 开发者工具：

- **Go tools** — 主要用于 Go 程序静态分析的各种工具和 Go 软件包。

: https://cs.opensource.google/go/x/tools || go-tools包

- **gopls** — 官方 Go 语言服务器。

: https://pkg.go.dev/golang.org/x/tools/gopls || gopls包

- **Delve** — Go 编程语言的调试器。

: https://github.com/go-delve/delve || delve包

- **go-bindata** — 一款可从任意文件生成 Go 代码的小工具。可用于在 Go 程序中嵌入二进制数据。

: https://github.com/shuLhan/go-bindata || go-bindataAUR, go-bindata-hashicorp包

- **GoReleaser** — Go 项目的发布自动化工具。

: https://goreleaser.com/ || goreleaser包

- **gox** — Go 交叉编译工具，可并行为多个平台编译。

: https://github.com/mitchellh/gox || gox包

- **ko** — Go 应用程序容器镜像生成器。

: https://github.com/ko-build/ko || ko包

- **revive** — 一个快速、可配置、可扩展、灵活、美观的 Go linter。

: https://revive.run/ || revive包

- **Staticcheck** — 先进的 Go 编程语言 linter。

: https://staticcheck.io/ || staticcheck包

- **Yaegi** — Go 解释器。包括 *yaegi* 命令行解释器/REPL。

: https://github.com/traefik/yaegi || yaegi包

安装目录

`go install` 命令会将 Go 可执行文件安装到 `GOBIN` 环境变量定义的目录或 `GOPATH` 环境变量第一个条目的 `bin` 子目录。默认情况下是 `~/go/bin`。

提示：运行 `go env` 来查看所有 Go 变量。

为方便起见，可将 bin 子目录添加到 `PATH` 中：

```
$ export PATH="$PATH:$(go env GOBIN):$(go env GOPATH)/bin"
```

更多信息，请参见如何编写 Go 代码（英文）和 `go help gopath`。

提示和技巧

编译源代码

可以编写一个 Hello World 程序，如下所示：

```
hello.go
package main

import "fmt"

func main() {
    fmt.Println("Hello, Arch!")
}
```

接着通过 *go* 工具运行它：

```
$ go run hello.go
Hello, Arch!
```

使用标准编译器编译（等价于 `go build -compiler=gc hello.go`）：

```
$ go build hello.go
```

使用 *gccgo* 编译（等价于 `go build -compiler=gccgo hello.go`）：

```
$ gccgo hello.go -o hello
```

使用 *tinygo* 编译：

```
$ tinygo build -o hello ./hello.go
```

交叉编译到其他平台

标准编译器可以很容易地交叉编译到很多平台。过程根据源代码是否通过 cgo 调用 C 代码而不同。

不使用 cgo

如果构建不需要 cgo，则只需将目标操作系统和架构指定为 `go build` 的环境变量：

```
$ GOOS=linux GOARCH=arm64 go build .
```

有关 GOOS 和 GOARCH 的有效组合，请参阅官方文档（英文）。

使用 cgo

如果构建需要 cgo，则必须通过 CC 或 CXX 环境变量提供 C/C++ 交叉编译器的路径。

假设要为 GOOS=linux 和 GOARCH=arm64 交叉编译。

首先要安装 aarch64-linux-gnu-gcc包 交叉编译器。

这是一个需要 cgo 的简单示例程序，以便测试交叉编译过程：

```
hello.go
package main

// #include <stdio.h>
// void hello() { puts("Hello, Arch!"); }
import "C"

func main() {
    C.hello()
}
```

然后，可以这样交叉编译：

```
$ GOOS=linux GOARCH=arm64 CGO_ENABLED=1 CC=/usr/bin/aarch64-linux-gnu-gcc go build
hello.go
```

可以检查生成的二进制文件的架构确实是 aarch64：

```
$ file hello
hello: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), dynamically linked,
interpreter /lib/ld-linux-aarch64.so.1,
BuildID[sha1]=b1d92ae8840a019f36cc2aee4606b6ae4a581bf1, for GNU/Linux 3.7.0, not
stripped
```

如果将 hello 复制到合适的主机，则可以运行：

```
[alarm@rpi3 ~]$ uname -a
Linux alarm 5.3.8-1-ARCH #1 SMP Tue Oct 29 19:31:23 MDT 2019 aarch64 GNU/Linux
[alarm@arpi3 ~]$ ./hello
Hello, Arch!
```

故障排除

JetBrains Go 插件

如果使用 JetBrains IDE，而 Go 插件找不到 Go SDK 路径，则可能是使用了不兼容的软件包。移除 *gcc-go* 软件包并替换为 *go*。如果已设置 GOPATH，那 IDE 现在应该可以找到位于 /usr/lib/go 的 Go SDK。

参见

- 官方网站（英文）
- 维基百科文章（英文）)
- 程序样例（包含简短的描述）（英文）
- 交互式 Go 训练教程（英文）

- Go 交叉编译（英文）
- Go IDE 和插件（英文）

1.13 Ruby

难度 进阶 | **篇幅** 8.8 KB | **所属** 编程语言

本节将讲到

- 安装
- RubyGems
- 交互式 Shell
- 另见

Ruby 是一门专注于简洁和生产力的动态解释型开源编程语言。

安装

要使用 Ruby，安装 ruby包。

要安装 IRB，安装 ruby-irb包。

多版本

如果你要在同一系统中运行多个版本 (比如 2.0.0-p0 和 1.9.3-p392），最简单的办法就是使用 RVM（英语：RVM）、chrubyAUR、rbenv 或者 asdf-vmAUR。

文档

要通过 `ri` 命令行工具访问文档，安装 ruby-rdoc包（工具）和 ruby-docs包（文档），随后便可以使用 `ri Array`、`ri Array.pop` 等命令查询文档（类似于手册页）。

JRuby

Ruby 的 Java 实现 JRuby 可通过 jruby包 安装。

标准库

Ruby 的标准库包含一系列 Ruby 模块（关于模块的更多信息请参见 #RubyGems）。ruby包 并未包含所有 Ruby 用户预期在标准 Ruby 系统中存在的标准模块，因此某些 Ruby 代码可能无法直接运行。关于标准库模块集的详细信息可参考：

https://stdgems.org/

在 #RubyGems 章节中讨论了多种安装模块的方法。若要通过 pacman 系统级安装标准模块，可以安装 ruby-stdlib包。注意 JRuby 无需此操作，因为 jruby包 软件包已包含标准模块。

RubyGems

RubyGems 是 Ruby 模块（称为 gem）的包管理器，和 pacman 与 Arch Linux 的关系大致相当，可通过 rubygems包 安装（ruby包 依赖）。

配置

在 Arch Linux 中，默认情况下运行 `gem` 时，gem 会按用户安装到 `~/.local/share/gem/ruby/` 目录下，而非安装到系统级的 `/usr/lib/ruby/gems/` 目录。这被视为 Arch 上管理 gem 的最佳实践，因为若采用系统级安装可能会与 Pacman 安装的 gem 产生冲突。

推荐的手动配置方法是设定 `$GEM_HOME` 环境变量，并将其添加到 `$PATH` 环境变量中，以便执行 RubyGems 的二进制文件：

```
~/.profile
export GEM_HOME="$(gem env user_gemhome)"
export PATH="$PATH:$GEM_HOME/bin"
```

此配置使得无需输入完整路径即可运行可执行 gem，而库 gem 无需修改路径即可正常使用。

注意：保存修改后，请重启终端以使配置生效。

用法

查看已安装的 gem 列表:

```
$ gem list
```

获取某个 gem 的详细信息：

```
$ gem spec "gem_name"
```

默认情况下，`gem list` 和 `gem spec` 开启了 `--local` 选项，导致 gem 只会在本地系统里进行搜索。这可以用 `--remote` 参数来覆盖。因此，可以这样搜索 mysql2 gem：

```
$ gem list --remote mysql2
```

安装某个 gem：

```
$ gem install mysql2
```

可以通过不安装本地文档加快一点安装速度：

```
$ gem install mysql2 --no-document
```

注意：这可以通过配置以下 `~/.gemrc` 文件成为默认选项：

```
~/.gemrc
gem: --no-document
```

更新所有已安装的 gem：

```
$ gem update
```

按用户或以系统级安装 gem

在 Arch Linux 下运行 `gem` 时，gem 默认按用户安装（即安装到 `~/.gem/ruby/`），而非以系统级安装（即安装到 `/usr/lib/ruby/gems/`）。这被认为是 Arch 上管理 gem 的最好方式，因为系统级安装可能会和 pacman 安装的 gem 冲突。

可以通过以 root 身份运行 `gem` 命令并附加 `--no-user-install` 标志以系统级安装 gem。若要将此标志设为默认行为，可在 `/etc/gemrc`（系统级配置）或 `~/.gemrc`（用户级配置，优先级更高）中将原有的 `--user-install` 参数替换为 `--no-user-install`。

Bundler 通过将 gem 打包到应用程序中，在一定程度上解决了这些问题。请参阅下面关于使用 Bundler 的部分。

Bundler

Bundler 用于指定应用程序所依赖的 gem，并可以同时指定所需版本。完成依赖声明后，Bundler 将安装所有必需的 gem（包含完整的依赖树）并记录安装结果以便后续检查。默认情况下，Bundler 将 gem 安装到统一位置，但也支持直接安装到应用程序目录中。当运行应用程序时，Bundler 可确保使用正确的 gem 版本（即使系统中存在同一 gem 的多个版本），这需要以下配置：

- 应用程序必须通过 `bundle exec` 命令启动
- 需在应用程序的主执行文件中添加所需样板代码

安装 Bundler：

```
$ gem install bundler
```

新建一个 bundle：

```
$ bundle init
```

然后编辑当前目录下的 `Gemfile`（由 `bundle init` 创建）以添加所需的 gem：

```
Gemfile
gem "rails", "3.2.9"
gem "mysql2"
```

运行下面的命令安装所有必需的 gem 到 `GEM_HOME`：

```
$ bundle install
```

或者，运行下面的命令把 gem 安装到工作路径的 `.bundle`：

```
$ bundle config set --local path '.bundle'
```

注意：命令 `bundle install --path .bundle` 已过时。但如果你的 Bundler 或 Ruby 版本较低导致上条命令无效，可以选择尝试使用这条命令。

别忘了编辑主执行文件：

```
#!/usr/bin/env ruby
```

```
# "This will automatically discover your Gemfile, and make all of the gems in
# your Gemfile available to Ruby." https://bundler.io/rationale.html
require 'bundler/setup'

...
```

最后运行你的程序:

```
$ bundle exec 主执行文件.rb
```

使用 pacman 管理 gem

除了使用 `gem` 管理 gem，也可以通过 pacman 或 AUR 助手进行管理。Ruby 软件包遵循命名约定：`ruby-*gem名称*`。

此方法具有以下优势：

- gem 会随系统其他软件一同更新
- 安装的 gem 可在系统范围内使用，而非仅限于安装它们的用户

注意：另有工具可通过自动生成指定 gem 的 PKGBUILD 文件，实现 gem 与 pacman 的集成，参见创建软件包#PKGBUILD 生成器。

Quarry

Quarry 工具用于通过 RubyGems 生成二进制 Arch Linux 仓库，可作为手动从 AUR 构建软件包的简化替代方案。其源代码托管于 Github。

Quarry 仓库由 Arch 开发者 anatolik 维护，地址为 https://pkgbuild.com/~anatolik/quarry/ ，包含了许多流行的 gem，并可根据请求添加新 gem。

启用方法请参阅非官方用户仓库#quarry。

随后可安装所需的 gem 包，软件包名为 `ruby-*gem名称*`（如 ruby-rails包）。

一般性问题可至 https://bbs.archlinux.org/viewtopic.php?id=182729 询问。

交互式 Shell

Pry

Pry 是 Ruby 标准 IRB Shell 的强大替代品，具备语法高亮、灵活的插件架构、运行时代码调用及源代码与文档浏览功能。

安装与启动方法：

```
$ gem install pry
$ pry
```

另见

- Ruby on Rails

- Ruby - https://www.ruby-lang.org/
- Bundler - https://bundler.io/
- why's (poignant) Guide to Ruby_Guide_to_Ruby)
- Learn Ruby The Hard Way

1.14 PHP

难度 进阶 | **篇幅** 16.5 KB | **所属** 编程语言

本节将讲到

- 安装
- 运行
- 配置
- 扩展
- 缓存

翻译状态：

- 本文（或部分内容）译自 PHP，最近一次同步于 2025-01-24，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

PHP是一种广泛使用的通用脚本语言，特别适合于 Web 开发，可嵌入到 HTML 中。

安装

安装软件包 php包。

由于某些应用程序无法使用最新版本的 PHP，您可以并行安装 php-legacy包。该软件包提供了最老的仍在积极支持的 PHP 分支，并且必须在需要它的应用程序中进行配置。有关详细说明，请参阅 Nextcloud#Migrating to php-legacy。

您可以在 AUR 中找到较旧和固定版本的 PHP，包括 php56AUR、php74AUR、php80AUR、php81AUR 和 php82AUR。这些是使用 openSUSE Build Service 构建的二进制版本。

运行

虽然PHP可以独立运行，但它通常与Web服务器一起使用。这需要安装额外的软件包并编辑配置文件。常见的设置请参考以下内容：

- Apache HTTP Server#PHP
- nginx#PHP implementation
- lighttpd#PHP
- Hiawatha#CGI

要想像纯CGI那样运行PHP,需要安装 php-cgi包 。

配置

主要的PHP配置文件位于 `/etc/php/php.ini`，并且有详细的文档说明。

- 建议在`/etc/php/php.ini` 中设置所在时区(list of timezones) 。如下:

```
date.timezone = Europe/Berlin
```

- 如果你想调试PHP时显示错误，在`/etc/php/php.ini`中将 `display_errors` 设为 `On`：

```
display_errors=On
```

- open_basedir 限制 PHP 可以访问的目录，可以增加安全性，但是会影响程序的正常执行。从 PHP 7.0 开始，和上游一样默认不再设置，要使用的用户请手动设置。符号链接会被解析，所以无法通过符号链接跳过限制。某些软件的 Arch 软件包，例如 `nextcloud` 和 `phpmyadmin` 安装在 `/usr/share/webapps`，然后在 `/etc/webapps` 中创建了配置文件的符号链接。设置 `open_basedir` 时请加入这两个目录。例如：

```
open_basedir
= /srv/http/:/var/www/:/home/:/tmp/:/var/tmp/:/var/cache/:/usr/share/pear/:/usr/share/w
ebapps/:/etc/webapps/
```

扩展

一些常用的PHP扩展也可以在官方库发现：

```
$ pacman -Ss php-
```

提示：不要编辑`/etc/php/php.ini`，扩展的启停可在 `/etc/php/conf.d` 中设置，如: (e.g. `/etc/php/conf.d/imagick.ini`)

已经安装的扩展位于 `/usr/lib/php/modules` 目录。

例如要启用 `ext-iconv` 扩展，在 `/etc/php/conf.d/extensions.ini` 中配置:

```
extension=iconv
```

要安装 PHP 的扩展，可以在 AUR 中搜索 php-\ *或 php56-*\, 例如 php-imagick包, php-redis包 php56-mcryptAUR。

gd

欲使用 php-gd包 在 `/etc/php/php.ini`中取消下列内容的注释:

```
extension=gd
```

imagemagick

安装php-imagick包。这个软件包会创建 `/etc/php/conf.d/imagick.ini` 配置文件。

要让 imagemagick 支持 SVG,需要安装 librsvg包。

多线程

要使用 POSIX 多线程，需要 parallel 扩展 。用 `pecl` 安装 parallel (https://pecl.php.net/package/parallel) 扩展，需要 PHP 在编译时启用线程安全选项`--enable-maintainer-zts`. 当前最简单的方式是用需要的选项重新编译.

可在 PHP pthreads extension 页面找到指令介绍。

PCNTL

利用 PCNTL 可以在服务器上直接创建进程。虽然这可能是你想要的，但是这样也会让 PHP 有能力把机器搞的一团糟。所以 PHP 不能和其他扩展一样加载，要启用此扩展，需要重新编译PHP。ArchLinux 的 PHP 已经加入 "--enable-pcntl"选项，默认已经启用。

MySQL/MariaDB

根据 MariaDB 页面安装并配置 MySQL/MariaDB.

取消 `/etc/php/php.ini` 中下面行前面的注释 :

```
extension=pdo_mysql.so
extension=mysqli.so
```

警告：PHP 7.0 中 删除了 `mysql.so`。

可以给网络脚本最低的 MySQL 用户权限，可以编辑 `/etc/my.cnf.d/server.cnf`，在 'mysqld *段落添加* `skip-networking`，*这样 MyQSL 服务器就仅允许通过 localhost 本地访问。请参考 MariaDB#Enable access locally only via Unix sockets。设置之后需要重启 MySQL。*

Redis

安装并配置 Redis，然后安装 php-redis包.

在 `/etc/php/conf.d/redis.ini` 中取消 redis 扩展的注释。同时在 `/etc/php/conf.d/igbinary.ini` 中启用(取消注释) igbinary 扩展。

PostgreSQL

安装并配置 PostgreSQL，然后安装 php-pgsql包 软件包并取消 `/etc/php/php.ini` 中下面几行的注释:

```
extension=pdo_pgsql
extension=pgsql
```

Sqlite

安装并配置 SQLite，然后安装 php-sqlite包 软件包并取消 `/etc/php/php.ini` 中下面几行的注释:

```
extension=pdo_sqlite
extension=sqlite3
```

XDebug

用 XDebug 可以很容易的通过修改的 var_dump() 函数进行调试、评测、追踪。

安装 xdebug包 并取消 `/etc/php/conf.d/xdebug.ini` 中如下行前面的注释：

```
zend_extension=xdebug.so
```

你可以通过在同一文件中添加 xdebug.mode 行来配置 XDebug 的行为。默认情况下，它设置为 `xdebug.mode=develop`。

对于 Xdebug 3，默认端口是 9003，要更改它，请设置 `xdebug.remote_port=9000`。

Snuffleupagus

安装 php-snuffleupagus包，取消 `/etc/php/conf.d/snuffleupagus.ini` 中的两行注释，并将 `snuffleupagus.rules` 文件的路径放在第二行：

```
extension=snuffleupagus.so
sp.configuration_file=/etc/php/conf.d/snuffleupagus.rules
```

缓存

PHP有两种缓存：*opcode*/*bytecode* 缓存和*userland*/*user data* 缓存，这两种缓存都大幅度提升性能，因此最好开启。

- Zend OPCache仅提供*opcode*缓存。
- APCu仅提供*userland*缓存

OPCache

OPCache随PHP发布，因此在PHP configuration file中开启或添加此行即可：

```
/etc/php/php.ini
zend_extension=opcache
```

你可在官网 找到其他设置以及建议设置。

警告：如果你使用推荐设置，要确保你一仔细看过说明，某些情况下可能导致如下错误：`zend_mm_heap corrupted`。

APCu

通过 php-apcu包 软件包安装 APCu, 然后在 `/etc/php/conf.d/apcu.ini` 中取消下面行的注释：

```
extension=apcu.so
```

作者 建议进行一些设置\[失效链接 2021-05-17 ⓘ\]:

- `apc.enabled=1` 和 `apc.shm_size=32M` 并不是必须的，因为这已经是 默认值\[失效链接 2021-05-17 ⓘ\];
- `apc.ttl=7200` 看上去 很有效\[失效链接 2021-05-17 ⓘ\];
- 最后, `apc.enable_cli=1`, 虽然手册上 不建议启用，有些软件比如 ownCloud 需要这个选项.

提示：可以将设置加入 APCu 自己的 `/etc/php/conf.d/apcu.ini` 或直接加到住配置文件，只需要注意不要同时加入。

开发工具

- **Visual Studio Code** — 支持 PHP 等多种语言的开发编辑器。

: https://code.visualstudio.com/ || visual-studio-code-binAUR

- **Aptana Studio** — 用于PHP和网页开发的IDE。没有PHP调试器。

: http://www.aptana.com/products/studio3.html || aptana-studioAUR

- **Eclipse PDT** — Eclipse的PHP变种。

: https://www.eclipse.org/pdt/ || eclipse-php-binAUR

- **Komodo** — 集成了PHP+HTML+JavaScript的IDE。

: http://komodoide.com/ || komodo-ideAUR, 仅编辑器：komodo-editAUR

- **NetBeans** — 用于多种语言的IDE，包括PHP。包含调试、重构、代码模板、自动补全、XML特性、网页设计和其他开发功能。

: https://netbeans.org/ || netbeans包

- **JetBrains PhpStorm** — 商业的、跨平台PHP IDE，基于JetBrains的IntelliJ IDEA平台。可以从JetBrains获取免费的教育许可。

: https://www.jetbrains.com/phpstorm/ || phpstormAUR, 30天试用：phpstorm-eapAUR

命令行工具

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：使用 Template:App。（在Talk:PHP讨论）

Composer

Composer 是 PHP 的依赖管理工具。 可以通过 composer包 包进行安装。

要允许当前用户全局安装包（例如 `$ composer global require "package/name"`），您可能需要通过使用环境变量指定默认位置：

```
PATH="$HOME/.config/composer/vendor/bin:$PATH"
```

与 php-legacy 一起使用

某些应用程序可能需要 php-legacy包，但默认情况下，`composer` 使用最新版本的 PHP 运行。因此，为了使用旧版 PHP，必须在脚本、Makefile 和其他适用位置中将 `composer` 替换为 `php-legacy /usr/bin/composer`。例如，在构建 Nextcloud 应用时就需要这样做。

Box

Box 是一个用于构建和管理 Phars 的应用程序。 可以通过 php-boxAUR 包进行安装。

PDepend

PHP Depend (pdepend) 是一个用于 PHP 的软件度量工具。 可以通过 pdependAUR 包进行安装。

PHP Coding Standards Fixer

PHP Coding Standards Fixer 是一个用于修复代码以符合 PSR-1 和 PSR-2 编码标准的工具。 可以通过 php-cs-fixerAUR 包进行安装。

PHP-CodeSniffer

PHP CodeSniffer 可以对 PHP、JavaScript 和 CSS 文件进行标记化，并检测违反定义的编码标准的行为。 可以通过 php-codesnifferAUR 包进行安装。

phpcov

phpcov 是 PHP_CodeCoverage 库的命令行前端。 可以通过 phpcovAUR 包进行安装。

phpDox

phpDox 是 PHP 项目的文档生成器。这包括但不限于 API 文档。 可以通过 phpdoxAUR 包进行安装。

PHPLoc

PHPLoc 是一个用于快速测量 PHP 项目大小的工具。 可以通过 phplocAUR 包进行安装。

PhpMetrics

PhpMetrics 提供了关于 PHP 项目的各种度量。 可以通过 phpmetricsAUR 包进行安装。

PHPUnit

PHPUnit 是一个面向程序员的 PHP 测试框架。 可以通过 phpunitAUR 包进行安装。

Producer

Producer 是一个命令行质量保证工具，用于验证并发布您的 PHP 库包。 可以通过 producerAUR 包进行安装。

故障排除

PHP Fatal error: Class 'ZipArchive' not found

确保启用了 zip 扩展。

```
/etc/php/php.ini
extension=zip
```

/etc/php/php.ini 未被解析

如果你的 `php.ini` 未被解析，ini 文件的名称会根据所使用的 SAPI 命名。例如，如果你使用的是 uwsgi，文件将被称为 `/etc/php/php-uwsgi.ini`。如果你使用的是 cli，文件则是 `/etc/php/php-cli.ini`。

PHP Warning: PHP Startup: <module>: Unable to initialize module

当运行 `php` 时，此错误表明上述模块已过时。在 Arch Linux 中这种情况很少发生，因为维护者确保核心 PHP 和所有模块仅在兼容的版本中可用。

这种情况可能与从 AUR 编译的模块一起发生。你通常可以通过查看 `/usr/lib/php/modules/` 目录中的文件日期来确认这一点。

要修复此问题，请为你的模块找到一个兼容的更新，通常可以通过使用其常用名称在 AUR 中查找。

如果适用，请将过时的 AUR 包标记为 *outdated*。

参见

- PHP 官方网站
- Arch Linux PHP 传统分支公告

1.15 Python

难度 进阶 | **篇幅** 17.0 KB | **所属** 编程语言

本节将讲到

- 安装
- 软件包管理
- GUI 绑定
- 提示与技巧
- 问题解决

翻译状态：

- 本文（或部分内容）译自 Python，最近一次同步于 2024-11-30，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

来自“什么是 Python？”：

: Python 是一种解释型、交互式、面向对象的编程语言。 它包含了模块、异常、动态类型、高层级动态数据类型以及类等特性。 在面向对象编程以外它还支持多种编程范式，例如过程式和函数式编程等。 Python 结合了超强的功能和极清晰的语法。 它带有许多系统调用和库以及多种窗口系统的接口，并且能用 C 或 C++ 来进行扩展。 它还可用作需要可编程接口的应用程序的扩展语言。 最后，Python 非常易于移植：它可以在包括 Linux 和 macOS 在内的许多 Unix 变种以及 Windows 上运行。

安装

安装 python包。

其它版本

Python 的旧版本和预发布版本可以通过 AUR 获得，这对于无法在当前版本上运行的旧应用程序、打算在其他版本上运行的程序，或者仅仅出于好奇，可能会很有用：

Feature 版本（接受新特性）和**Prerelease 版本**（不接受新特性）

- Python 3.15: python315AUR （Prerelease 版本）

Bugfix 版本（也叫维护版本或稳定版本）

- Python 3.14：python314-freethreadedAUR（自由线程——移除了全局解释器锁(GIL)）
- Python 3.13: python313AUR
- Python 3.13：python313-freethreadedAUR（自由线程）

Security 版本（只接受安全修复）

- Python 3.12：python312AUR

- Python 3.11：python311AUR
- Python 3.10：python310AUR

End-of-life 版本（不再维护）

- Python 3.9：python39AUR
- Python 3.8：python38AUR（PEP-569）
- Python 3.7：python37AUR（PEP-537）
- Python 3.6：python36AUR（PEP-494）
- Python 2.7：python2AUR（Python 2 的日落）

为实现多版本共存，每个这样的软件包都会安装一个以版本号命名的独立二进制文件，例如 Python 3.10 的二进制文件名为 `python3.10`。pyenv包 可用于轻松安装和切换多个 Python 版本。

在 AUR 上搜索 `python<版本号（不带小数点）>` 可以找到用于旧版本的额外模块/库，例如搜索 `python310` 可以找到 3.10 版本的模块。

可以从 https://www.python.org/downloads/ 下载任意版本的源代码。

替代实现

python包 包会安装 CPython， 是 Python 的参考实现，而同时还有多种其它的实现。这些实现通常基于旧版本的 Python 而并不完全与 CPython 兼容。

Arch Linux 官方仓库与 AUR 包含的实现：

- **PyPy** — 使用 Python 编写的 Python 实现，较 CPython 而言具有速度和内存用量上的优势。

: https://www.pypy.org || pypy包、pypy3包

- **Jython** — 使用 Java 编写的 Python 实现，可以用于将 Python 脚本嵌入到 Java 程序中，或是在 Python 程序中使用 Java 库。

: https://www.jython.org/ || jython包

- **micropython** — 用于微控制器的 Python 实现，包含了 Python 标准库的一小块子集，并针对在微控制器或是受限环境中运行进行了优化。

: https://micropython.org/ || micropythonAUR

- **IronPython** — 与 .NET 紧密集成的 Python 实现，可以调用 .NET 库，同时允许 .NET 程序使用 Python 库。

: https://ironpython.net || ironpython-gitAUR

Python 还有其它多种实现。一些（例如 Cinder）在大型科技公司内部得到使用。还有一些在历史上具有重要意义，但因最流行的实现已有改进而不再维护。

替代 shell

python包 软件包包含一个交互式 Python shell/REPL，可以通过 `python` 命令启动，还可以使用以下 shell：

- **bpython** — 为 Python 解释器提供美观界面。

: https://bpython-interpreter.org/ || bpython包

- **IPython** — 功能强大的交互式 Python Shell。

: https://ipython.org/ || ipython包

- **Jupyter** — 由 IPython 提供支持的基于 Web 的计算应用程序。

: https://jupyter.org/ || jupyterlab包、jupyter-notebook包

- **ptpython** — 基于 prompt-toolkit 构建的高级 Python REPL。

: https://github.com/prompt-toolkit/ptpython || ptpythonAUR

软件包管理

在 Arch Linux 上，有多种方法可以安装 Python 软件包。

Arch 官方仓库

大量流行的软件包可以在官方仓库和 AUR 中找到。这是安装系统范围内软件包的首选方式，也是 Arch Linux 官方支持的唯一方法。

第三方软件包

开发者在使用 Python 时，可能需要使用 Arch 仓库中没有的软件包或特定版本。推荐的做法是为每个项目使用单独的虚拟环境隔离环境，避免与 `/usr` 中的系统软件包发生冲突。在虚拟环境中安装软件包可以使用以下工具：

- **pip(1)** — Python 官方的软件包安装工具。

: https://pip.pypa.io/ || python-pip包

- **pipx** — 专门的软件包安装工具，仅用于安装具有 CLI 入口点的软件包（而非库包）。

: https://pypa.github.io/pipx/ || python-pipx包

- **Poetry** — 简化 Python 依赖管理和打包的工具。Poetry 是一个集开发、构建、发布和依赖管理于一体的工具。

: https://python-poetry.org/ || python-poetry包

- **Conda** — Conda 提供适用于任何语言的软件包、依赖和环境管理。Conda 最初为 Python 而创建，广泛用于科学计算、数据科学和机器学习。Conda 是 miniforge 社区 Python 发行版以及 Anaconda 和 Miniconda Python 发行版的软件包管理器。

: https://docs.conda.io || python-condaAUR

- **uv** — 使用 Rust 编写的极快的 Python 软件包安装器和解析器。设计为常见 pip 和 pip-tools 工作流程的直接替代品。

: https://docs.astral.sh/uv/ || uv包

`pip`、`pipx`、`poetry` 和 `uv` 从 Python 软件包索引（PyPI）和其他索引中安装软件包。Conda 和 Miniconda 使用 Anaconda 仓库。

作为虚拟环境的替代方案，可以使用 `pip install --user` 将软件包安装到用户方案，而不是 `/usr`。这种方法按用户而非项目分隔软件包，但虚拟环境通常是更好的选择。

软件包管理的官方最佳实践见 Python 打包用户指南（Python Packaging User Guide）。

注意：还有一些工具将 `pip` 与 `pacman` 集成，通过为指定的 PyPI 软件包自动生成 PKGBUILD 文件来实现，详见创建软件包#PKGBUILD 生成器。

提示：pipenv 提供用于管理 Pipfile、`pip` 和 virtualenv 的统一的 CLI，可以通过 python-pipenv包 获取。

历史说明

之前，`easy_install`（python-setuptools包 的一部分）用于安装以 Egg 格式分发的软件包。`easy_install` 和 Egg 已被 pip 和 Wheel 替代。详见 pip vs easy_install 和软件包格式（Package Formats）。

早期版本的 `pip` 可以在系统范围内安装第三方软件包，但这导致了 PEP668 中提到的诸多问题。现在，系统范围的环境被标记为外部管理环境（externally managed environment），`pip` 不再允许系统范围的安装。

GUI 绑定

以下是可用的控件工具包（GUI 工具包）绑定：

- **Tkinter** — Tk GUI 工具包的标准 Python 接口。

: https://docs.python.org/3/library/tkinter.html || python包

- **Qt for Python (PySide2)** — Qt5 的官方 Python 绑定。

: https://www.qt.io/qt-for-python || pyside2AUR、pyside2-toolsAUR

- **Qt for Python (PySide6)** — Qt6 的官方 Python 绑定。

: https://www.qt.io/qt-for-python || pyside6包、pyside6-tools包

- **pyQt** — Qt 的 Python 绑定集合。

: https://riverbankcomputing.com/software/pyqt/intro || python-pyqt5包、python-pyqt6包

- **PyGObject** — 用于基于 GObject 的库（如 GTK、GStreamer、WebKitGTK、GLib 和 GIO）的 Python 绑定。

: https://pygobject.readthedocs.io/ || python-gobject包

- **wxPython** — 封装 wxWidgets 的跨平台 Python GUI 工具包。

: https://wxpython.org/ || python-wxpython包

若要在 Python 中使用它们，可能还需要安装相关的控件工具包软件包（例如，必须同时安装 tk包 才能使用 Tkinter）。

提示与技巧

虚拟环境

Python 提供了创建隔离的**虚拟环境**的工具，可以在其中安装软件包，而不与其他虚拟环境或系统软件包发生冲突。虚拟环境还可以在同一系统上运行不同版本的 Python。

详见 Python/虚拟环境。

在 Python Shell 中实现 Tab 补全功能

Tab 自动补全在交互式 shell 中默认可用。需要注意，readline 补全器只能补全全局命名空间中的名称。python-jedi包 提供了更丰富的 Tab 自动补全体验 。

问题解决

更新 Python 后找不到模块

在升级 python包 到新版本（例如从 3.10 升级到 3.11）后，基于 Python 的应用程序可能会输出 `No module named *模块名称*` 错误，表示已安装的依赖项 *模块名称* 不可用。

这种情况发生在某个依赖项不支持当前的 Python 版本，或者根本没有安装。Python 软件包在安装时会放置在版本隔离的 site-packages 目录中（如果是全局安装，路径为 `/usr/lib/python*X.Y*/site-packages`，如果是用户安装，路径为 `~/.local/lib/python*X.Y*/site-packages/`，其中 `*X.Y*` 是类似“3.11”这样的版本号）。因此，每当 Python 版本进行小版本升级时，使用之前版本构建的基于 Python 的软件包必须重新构建，以便能在新版本中正常使用。

需要注意，用户有责任重新构建非官方软件包，包括从 AUR 安装的基于 Python 的软件包。请参阅 AUR#更新包和\[\[常见问题#Q)_执行`pacman -Syu`时，终端显示某个共享库需要升级，但依赖它的程序没有升级，我该怎么做呢？\]\]。

另见

官方

- 官方 Python 文档（可以通过 python-docs包 安装以离线阅读。）
- 官方 Python 教程

第三方

- Automate the Boring Stuff with Python——知识共享书籍
- Awesome Python——精选 Python 资源列表
- A Byte of Python——知识共享书籍
- Cracking Codes With Python——免费在线书籍
- Crash into Python——免费教程
- Python Debugging With Pdb——`pdb`（Python 官方调试器）教程
- Dive Into Python——知识共享书籍
- Fluent Python——商业书籍
- Introducing Python——商业书籍
- Invent Your Own Computer Games with Python——免费在线书籍
- Learn Python——免费交互式教程
- Learn Python the Hard Way——商业书籍

- Pythonspot Python Tutorials——免费在线教程
- Think Python——知识共享书籍

1.16 Rust

难度 进阶 | **篇幅** 17.8 KB | **所属** 编程语言

本节将讲到

- 语言核心
- 安装
- 交叉编译
- Cargo
- IDE 支持

翻译状态：

- 本文（或部分内容）译自 Rust，最近一次同步于 2025-10-03，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

根据维基百科)：

: Rust 是由 Mozilla 主导开发的通用、编译型编程语言。设计准则为“安全、并发、实用”，支持函数式、并发式、过程式以及面向对象的程序设计风格。Rust 的设计目标之一，是要使设计大型的互联网客户端和服务器的任务变得更容易。因此更加强调安全性、存储器配置、以及并发处理等方面的特性。

语言核心

Rust 核心库

Rust 核心库是 Rust 标准库的基础，无需依赖。它直接与 LLVM 原语对接，使 Rust 可以与平台和硬件无关。正是这种与 LLVM 的集成，使 Rust 比 Clang 编译的同样功能的 C 语言程序性能更高，使得用 libcore 设计的 Rust 软件比 C 语言的更底层。为嵌入式平台设计软件的开发者可通过 `#![no_std]` 放弃使用标准库，只使用语言自带的核心库，以获得更小的二进制文件和更好的性能。然而，由于大多数库都需要标准库，使用 `#![no_std]` 意味着来自 Rust 社区的软件支持会受限。

Rust 标准库

Rust 标准库提供了方便的高级抽象，用于开发大多数可移植的 Rust 软件。标准库包含 `Vec` 和 `String` 类型、大量的语言原语方法、大量的标准宏、I/O 和多线程支持、使用 `Box` 实现的堆分配，以及更多核心库不具有的高级特性。

发布周期

与 Firefox 类似，Rust 使用每六周的发布周期。在每个新版本中，核心库和标准库都会得到改进，以支持更多的平台、提高性能，以及为稳定版本增加新特性。

安装

安装 Rust 主要有两种方式：

- 直接安装。如果你只使用 Rust 来运行或安装软件，推荐使用此方式。
- Rustup 安装。如果你打算进行 Rust 编程，推荐使用此方式。

直接安装

要从 Arch Linux 官方软件库中安装最新的稳定版 Rust，安装 rust包。这将安装 `rustc` 编译器和 Cargo。

除此之外还有开发版的 Rust 编译器 rust-nightly-binAUR 和 rust-gitAUR 可用，分别用于预编译的通用二进制文件、编译器与系统库的构建。

通过 rustup 安装

为开发软件，官方推荐的 Rust 安装方法是使用 rustup 工具链管理器安装。

使用 rustup 工具链管理器而不是仓库中独立的 Rust 软件包的优点，主要是能为多个目标平台（Windows、macOS、Android）和架构（x86、x86_64、arm）安装多个工具链（稳定版、测试版、开发版）。需要注意的是，安装 rustup 并不会自动安装 Rust 工具链，使用任何方法更新 rustup 也不会自动提供 Rust 的最新工具链版本。有关工具链的更多信息，请参见#用法或 rustup 工具链文档。

Rustup 的安装有两种方式，一种是通过 Arch Linux 自带的 pacman，另一种是通过 Rust 官方安装脚本。

Arch Linux 软件包

rustup包 在 Arch Linux 软件仓库中可用。注意，以这种方式安装时，`rustup self update` 将不起作用，只能使用 pacman 更新 rustup。然而，这一变化并不适用于其他 rustup 功能，例如用于更新 Rust 工具链的 `rustup update`。

使用此软件包的优点是 Rust 可执行文件都在 `/usr/bin` 目录内，而不是 `~/.cargo/bin`，因此避免了在 `PATH` 中添加其他目录的麻烦。

注意：rustup包 软件包默认不会安装工具链。它提供了 `/usr/bin/rustup` 与 `/usr/bin/rustc`、`/usr/bin/cargo` 等普通二进制文件之间的符号链接。如上所述，要使上述 Rust 命令能工作，用户仍需手动安装工具链。

为安装工具链，需要指定使用的版本号或使用稳定版（stable）、测试版（beta）、开发版（nightly）：

```
$ rustup default stable
```

构建 Rust 时更新 LLVM

由于 Rust 采用引导策略进行构建，因此在构建前必须先安装一个已经可用的 Rust 软件包。当构建使用的 llvm包 版本高于官方仓库中的版本时，需要提供用于构建仓库中 Rust 版本的旧版 llvm-libs包 动态链接库才能进行构建。

示例：官方仓库提供 llvm-18.1.8，需要针对 llvm-19.1.6 构建。

引导步骤需要来自 llvm-libs-18.1.8 的 `/usr/lib/libLLVM.so.18.1` 文件。该文件可以通过手动放置到构建根目录，或通过安装诸如 llvm15-libs包 之类的软件包来提供。

上游安装脚本

Rustup 也可通过 rustup 官方网页下载并手动安装。

使用下面命令下载：`curl --proto '=https' --tlsv1.3 -sSf https://sh.rustup.rs -o rust.sh`，查看脚本：`less ./rust.sh`，运行脚本以安装 rustup：`./rust.sh`。此脚本仅对当前登录 Shell 的配置文件进行 PATH 修改。在注销并重新登录之前需要`source ~/.cargo/env`。在此之后升级 rustup 需执行 `rustup self update`。

该脚本自动安装并激活默认工具链（rust包 所使用的工具链），因此无需手动安装工具链即可使用 Rust。

警告：正如 Rust 文档所建议的，执行 `curl *some-url* | sh` 会带来安全风险，因为它执行的是未知的、甚至可能在下载过程中被破坏的代码。因此建议在执行脚本之前，手动下载并检查。

注意：在运行 `rustup` 时请确保 `~/.cargo/bin` 在 `PATH` 内。

用法

可能需要手动安装工具链，例如稳定版（stable）、测试版（beta）、开发版（nightly）或特定版本（1.58.0）。如果想要使用或测试另一个工具链，也需要如下操作：

```
$ rustup toolchain install toolchain
```

现在可通过 `rustup run *toolchain* *command*` 来执行 Rust 命令。然而，如果要直接使用这些命令，则需要激活该工具链：

```
$ rustup default toolchain
```

使用 `rustc -V` 检查目前安装的 Rust 版本:

```
$ rustc -V
rustc 1.58.0 (02072b482 2022-01-11)
```

注意：Rust 自身并不会进行链接操作，因此需要确保已安装链接器。可使用 gcc包，否则 Rust 会提示以下错误：`error: linker cc not found.`

注意：

Rustup 不会自动更新提供的工具链。如果用户希望使用最新版本的 Rust、crates 以及其他相关包，他们可能需要偶尔使用 `rustup update` 来更新他们的工具链。有关更多信息，请参见 官方 rustup 文档。

测试安装结果

通过构建一个小程序来测试 Rust 是否已正确安装，源码如下所示：

```
~/hello.rs
fn main() {
    println!("Hello, World!");
}
```

可使用 `rustc` 编译，然后运行：

```
$ rustc hello.rs && ./hello
Hello, World!
```

交叉编译

使用 rustup

使用 rustup 可轻松地交叉编译。rustup 支持许多交叉编译目标，完整列表可通过执行 `rustup target list` 查看。

例如，使用 Windows 稳定通道安装 Rust，使用 GNU 编译器，需要执行以下命令：

```
$ rustup toolchain install stable-x86_64-pc-windows-gnu
```

此操作只会为目标架构安装 Rust 及其工具，但在交叉编译时可能还需要其他工具。

Windows

本节中，`$ARCH` 是目标架构（`x86_64` 或 `i686`）。以下内容将解释如何使用 rustup 进行交叉编译。

1. 安装 mingw-w64-gcc包
2. 为目标架构安装 Rust 标准库：`rustup target add $ARCH-pc-windows-gnu`
3. 最后，为了使 cargo 能找到 MinGW-w64 gcc/ar，需要在 `~/.cargo/config.toml` 中添加如下内容：

```
~/.cargo/config.toml
[target.$ARCH-pc-windows-gnu]
linker = "/usr/bin/$ARCH-w64-mingw32-gcc"
ar = "/usr/bin/$ARCH-w64-mingw32-ar"
```

最后，通过向 cargo 传递 `--target $ARCH-pc-windows-gnu` 参数，即可为 Windows 交叉编译：

```
$ # Build
$ cargo build --release --target "$ARCH-pc-windows-gnu"
$ # Run unit tests under wine
$ cargo test --target "$ARCH-pc-windows-gnu"
```

目前无法使用 MinGW 6 和通过 rustup 安装的工具链构建可执行文件。要修复此问题，请执行

```
for lib in crt2.o dllcrt2.o libmsvcrt.a; do cp -v /usr/x86_64-w64-mingw32/lib/$lib $HOME/.rustup/toolchains/$CHANNEL-x86_64-unknown-linux-gnu/lib/rustlib/x86_64-pc-windows-gnu/lib/; done
```

其中 `CHANNEL` 为更新通道：稳定版（`stable`）、测试版（`beta`）或开发版（`nightly`）。

非官方软件包

非官方仓库 archlinuxcn 有 rust-nightly 和 i686, ARM, ARMv7, Windows 32 & 64 的 Rust 标准库，因此只需安装对应的标准库即可享受交叉编译。然而，ARM 工具链只能自行寻找。对于 32 位 Windows 目标，需要获取 mingw-w64-gcc包 提供的 libgcc_s_dw2-1.dll 来编译运行。

Cargo

Cargo，Rust 的包管理器，是 rust包 的一部分。作为 rust-nightly-binAUR 的一部分，Cargo 开发版（nightly）在 AUR 上可用。如果使用 rustup包 则无需再安装 Cargo。

Cargo 使 Rust 项目能声明各种依赖关系，并确保始终能得到可重现构建。推荐阅读官方指南。

用法

使用 Cargo 创建新项目：

```
$ cargo new hello_world
```

此操作将创建包含默认 Cargo.toml 文件的目录，该文件用于构建可执行文件。

注意：Cargo 将 `Cargo.toml` 作为包含编译项目所需所有元数据的列表。

```
Cargo.toml
[package]
name = "hello_world"
version = "0.1.0"
edition = "2021"

[dependencies]
```

为本机 CPU 平台优化

通过在 `~/.cargo/config.toml` 中添加一个标志，Cargo 将始终为本机 CPU 进行编译优化。请注意，由此产生的二进制文件无法在其他计算机上运行。如果未来更换 CPU，您甚至可能无法在自己的系统上运行这些二进制文件。

找出安装时默认使用的目标平台：

```
$ rustup toolchain list
stable-x86_64-unknown-linux-gnu (default)
```

这说明目前在 `x86_64-unknown-linux-gnu` 平台上使用稳定版（`stable`）Rust。

将 Cargo 设置为始终为本机 CPU 进行编译优化：

```
~/.cargo/config.toml
[target.x86_64-unknown-linux-gnu]
rustflags = ["-C", "target-cpu=native"]
```

sccache

使用 sccache（sccache包）可以大大减少编译时间。这将在本地维护编译器的工作缓存，从而无需重新编译自上次编译以来不变的代码。

为启用 sccache，可使用 `RUSTC_WRAPPER` 环境变量：

```
$ export RUSTC_WRAPPER=sccache
$ cargo build
```

或者

```
$ RUSTC_WRAPPER=sccache cargo build
```

或者也可以将以下内容添加到 `~/.cargo/config.toml`：

```
~/.cargo/config.toml
[build]
rustc-wrapper = "sccache"
```

IDE 支持

工具

Rust 项目的推荐工具请参阅 https://www.rust-lang.org/zh-CN/tools 。

rust-analyzer

rust-analyzer 是官方的 Rust 语言服务器协议（LSP）实现，已取代 RLS。

安装 rust-analyzer包 软件包即可使用，最新 Git 版本可通过安装 rust-analyzer-gitAUR 获取。另外，如果已安装 rustup包，可使用以下命令安装 rust-analyzer：

```
$ rustup component add rust-analyzer
```

rust-analyzer 需要标准库的源代码。如果源代码不存在，rust-analyzer 将尝试使用 rustup 自动安装。要使用 rustup 手动安装源代码，请执行以下命令：

```
$ rustup component add rust-src
```

Clippy

Clippy 借助编译器插件支持实现了许多额外提示，能检测出更多错误和非惯用的 Rust 语法。rust包 软件包中包含了 Clippy。

使用 rustup 安装 Clippy：

```
$ rustup component add clippy
```

Rustfmt

Rustfmt 是一个根据官方代码风格规范来格式化 Rust 代码的工具。

rust包 软件包中包含了 Rustfmt。执行以下命令，用 rustup 来安装 Rustfmt：

```
$ rustup component add rustfmt
```

编辑器

Emacs

Emacs 对 Rust 的支持可通过官方的 rust-mode 插件获取。

Emacs 支持使用 EglotGNU 文档 、lsp-bridge 等包连接 rust-analyzer。

GNOME Builder

GNOME Builder 对 Rust 的支持通过语言服务器协议（LSP）实现。默认使用 rust-analyzer，需要同时安装 Rust 源代码。

Helix

Helix 编辑器使用 Rust 编写，包含了 Rust 语言服务器协议（LSP）。Helix 的灵感来自 Neovim 和 Kakoune。

Kate

Kate 对 Rust 的支持通过语言服务器协议（LSP）实现。默认使用 rust-analyzer，需要同时安装 Rust 源代码。

IntelliJ IDEA

IntelliJ IDEA 有 Rust 插件。此插件同样适用于 CLion。

如果使用 rustup，请使用 rustup 下载源代码（`rustup component add rust-src`），然后将工具链所在位置指定为 `~/.rustup/toolchains/<your toolchain>/bin`。

如果使用 Arch Linux 官方软件库中的 Rust，请将工具链所在位置指定为 `/usr/bin`，将标准库所在位置指定为 `/usr/lib/rustlib/src/rust/library/`。

Jetbrains RustRover

Jetbrains 也正在开发一款专为 Rust 设计的编辑器。可以在其官方网站或 AUR（rustroverAUR 和 rustrover-eapAUR） 中找到并下载。

Visual Studio Code

Visual Studio Code 对 Rust 的支持可通过 rust-analyzer 与 rust-lang.rust-analyzer 扩展获取。

Vim

Vim 对 Rust 的支持可通过官方的 rust.vim 插件获取，支持文件检测、语法高亮、格式化和 Syntastic 语法检查插件。许多代码补全引擎都支持 Rust，例如 coc（通过 coc-rust-analyzer 插件）和 YouCompleteMe。

另见

- Rust 编程语言官网
- Rust 文档
- 标准库 API
- Rust 官方文档中文教程
- 《Rust 程序设计语言》
- 《通过例子学 Rust》
- 《Rust 参考手册》
- Rust 维基百科词条
- Rust 语言术语中英文对照表
- Rust Telegram 中文讨论群
- 带少量注释的 Rust 示例（英文）
- Cargo 可用的库（单元包/crate）（英文）
- Rust 本周动态（英文）
- Rust 编程语言博客（英文）
- Rust 用户论坛（英文）
- Rust 内部人员论坛（英文）

1.17 语言服务器协议

难度 进阶 | **篇幅** 19.2 KB | **所属** 编程语言

本节将讲到

- 参见

语言服务器协议（LSP）定义了编辑器或者 IDE 与语言服务器之间使用的协议，它提供了如自动补全、转到定义、查找所有引用等功能。

语言服务器可以安装以下软件包。如果你所使用的编程语言不在该列表中，请在 Arch 的官方仓库或在 AUR 中寻找。

注意：多数情况下，你每一种语言只需要一个语言服务器即可。但是一些语言服务器可能不包含某些功能，此时你可能就需要安装多个语言服务器。更多信息请参阅上游文档。

语言	语言服务器	语言服务器软件包名
Ada/SPARK	als	ada_language_serverAUR
Angular	angularls	
Ansible	ansible-language-server	ansible-language-serverAUR
Arduino	arduino-language-server	arduino-language-server包
Assembly	asm-lsp	asm-lspAUR
Astrojs	astrojs-language-server	astrojs-language-serverAUR
Autotools	autotools-language-server	autotools-language-serverAUR
AWK	awk-language-server	awk-language-serverAUR
Bash	bashls	bash-language-server包
Bazel	bazelrc-lsp	bazelrc-lspAUR
Bicep	bicep	bicep-langserver-binAUR
C3	c3-lsp	c3-lspAUR
C#	omnisharp-roslyn	omnisharp-roslynAUR
C, C++, Objective-C	ccls, clangd	ccls包, clang包
Cabal	hls	haskell-language-server包
Clojure	clojure_lsp	clojure-lsp-binAUR
CMake	cmake, neocmakelsp	cmake-language-serverAUR, neocmakelspAUR
CodeQL	codeqlls	codeqlAUR
Crystal	crystalline	crystallineAUR
CSS, LESS, SASS	cssls, tailwindcss	vscode-css-languageserver包, tailwindcss-language-server包
D	serve-d	serve-dAUR
Dart	dartls	dart包
Dhall	dhall-lang	dhall-lsp-server包
Dockerfile, Dockercompose	dockerls, compose-language-service	dockerfile-language-server包, nodejs-compose-language-serviceAUR
DOT	dot-language-server	dot-language-server包
Elixir	elixirls	elixir-lsAUR
Elm	elmls	elm-language-serverAUR
Erlang	elp, erlangls	elpAUR, erlang_lsAUR
F#	fsautocomplete, fsharp-language-server	fsharp-language-serverAUR, fsautocomplete-binAUR
Fennel	fennel-ls	fennel-lsAUR
Fish	fish-lsp	fish-lspAUR
Flow	flow	flowAUR
Fortran	fortls, fortran-language-server	fortlsAUR, fortran-language-serverAUR
GDScript	gdscript	godot包
General purpose	diagnosticls, efm	diagnostic-languageserverAUR, efm-langserverAUR
Gleam	gleam	gleam包
GOlang	gopls	gopls包
GraphQL	graphql	graphql-lspAUR
Groovy	groovyls	groovy-language-server-gitAUR
Haskell	hls	haskell-language-server包

语言	语言服务器	语言服务器软件包名
Haxe	haxe_language_server	
HTML	html	vscode-html-languageserver包
Hyprlang	hyprls	hyprlsAUR
Janet	janet-lsp	
Java	java_language_server, jdtls	java-language-serverAUR, jdtlsAUR
JavaScript	denols	deno包
JSON	jsonls	vscode-json-languageserver包
Julia	julials	
Jupyter	jupyterlab-lsp	jupyterlab-lsp包
Just	just-lsp	just-lsp包
KDL		
Kotlin	kotlin_language_server	kotlin-language-serverAUR
Kubernetes		
Lean 3	lean3ls	lean-language-serverAUR
Lean 4		
Lua	lua_ls, emmylua_ls	lua-language-server包, emmylua-ls-binAUR
Makefile	autotools-language-server	autotools-language-serverAUR
Markdown	marksman, markdown-oxide, rumdl	marksman包, markdown-oxide包, rumdl包
Meson	muon analyze lsp, mesonlsp	muon-mesonAUR, mesonlspAUR
Mojo	mojo-lsp-server	mojo-gitAUR
Natural language	typos-lsp, codebook	typos-lsp包, codebook-lsp包
Nginx	nginx-language-server	
Nickel	nls	nickel-language-server包
Nim	nimls	nimlspAUR
nix	nil, nixd	nil-gitAUR, nixdAUR
Nushell	nu --lsp	nushell包
OCaml, Reason	ocamllsp	ocaml-lsp-gitAUR
Odin	ols	odinls-binAUR
OmniSharp	omnisharp	omnisharp-roslynAUR
Perl	perlls	perl-perl-languageserverAUR
PHP	phpactor, intelephense	phpactorAUR, nodejs-intelephenseAUR
Pkl	pkl-lsp	
Pony	pony-lsp	ponyc包
PowerShell	powershell_es	powershell-editor-services-binAUR
PureScript	purescriptls	purescript-language-serverAUR
Python	pylsp, basedpyright, pyright, jedi_language_server, pyrefly, ruff, ty	python-lsp-server包, pyright包, basedpyrightAUR, jedi-language-server包, pyreflyAUR, ruff包, ty包
QML	qmlls	qt6-declarative包
R	r_language_server	r-languageserverAUR
Racket	racket_langserver	
ReScript	rescriptls	rescript-lsAUR
Roc	roc_language_server	roc-nightly-binAUR
RST		
Ruby	solargraph, sorbet, ruby-lsp	ruby-solargraphAUR, ruby-sorbet-runtime包, ruby-lsp包
Rust	rust_analyzer	rust-analyzer包
Scala	metals	metalsAUR
Solidity	solc --lsp	solidityAUR
SQL	sqls, sql-language-server	sqlsAUR, sql-language-serverAUR
Standard ML	Millet	milletAUR
Svelte	svelte	

语言	语言服务器	语言服务器软件包名
		svelte-language-server包
Swift	sourcekit	swift-languageAUR
Systemd	systemd-lsp	systemd-lsp包
SystemVerilog	svls	svlsAUR
Tcl		
Terraform	terraformls	terraform-lsAUR
TOML	taplo, tombi	taplo-cli包, tombi包
(La)TeX	texlab	texlab包
TypeScript	tsserver, denols	typescript-language-server包, deno包
Typst	tinymist	tinymist包
V	v-analyzer	v-analyzer-binAUR
Vala	vala_ls	vala-language-serverAUR
Vimscript	vimls	vim-language-serverAUR
Vue	volar	vue-language-server包
WebAssembly	wasm-language-tools	wasm-language-toolsAUR
XML	LemMinX	lemminxAUR
YAML	yamlls	yaml-language-server包
Zig	zls	zls包
Zsh		

参见

- Wikipedia:Language Server Protocol
- neovim 的 LSP 配置综合列表
- langserver 一个由社区驱动的关于语言服务器协议集成的知识库
- A community-driven source of knowledge for Language Server Protocol implementations

开发

开发环境的坑很少在"装不上"，而在装好之后：同一台机器常并存多套语言运行时、图形驱动与打包方式。本章把语言版本管理、编译与图形工具链、软件分发与运行时这些互不相干的话题并列。

建议按序读 Vulkan、OpenGL、GPGPU：显卡栈从安装、验证到使用的整条链；Python 虚拟环境与 Node.js 宜按序读完。Snap、Flatpak、Ollama、Eclipse、Rbenv、Xc3sprog 彼此独立，用不上可跳过。最易踩的坑：换驱动后不重新验证，以及核心转储默认策略与快照式打包叠加时行为与直觉不符。

2.1　Rbenv

难度 入门 | **篇幅** 1.4 KB | **所属** 开发

本节将讲到

- 安装
- 插件
- 参见

翻译状态：

- 本文（或部分内容）译自 Rbenv，最近一次同步于 2025-03-17，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

rbenv (Simple Ruby Version Management，简易 Ruby 版本管理) 可以让你在多个 Ruby 版本之间轻松切换。它简单、不张扬，沿袭了 UNIX 传统的单用途工具，只做好一件事。

另一个用于相同目的的工具是 RVM。

安装

安装 rbenv包 软件包。

插件

rbenv 可以通过插件系统进行扩展，rbenv wiki 上有一个实用插件列表。ruby-build 插件尤其有用，它允许你使用 `rbenv install` 命令安装 Ruby 版本。你可以从 AUR 安装 ruby-build包。

参见

- 官方网站

2.2 Xc3sprog

难度 入门 | **篇幅** 1.5 KB | **所属** 开发

本节将讲到

- 安装
- 设备

xc3sprog 是一套实用程序套件，用于使用 Xilinx 并行电缆和其他 JTAG 适配器对 Xilinx FPGA，CPLD 和 EEPROM 进行编程

安装

从 AUR 安装 xc3sprog-svnAUR 包。

设备

Xilinx USB JTAG

最初有 `USBID=03fd:000f`，经过适当的初始化后变成 `03fd:0008`。

- 从 AUR 安装 fxloadAUR。
- 从 Xilinx ISE 提取 xusb_xlp.hex
- 创建 `/etc/udev/rules.d/99-xilinx.rules` 文件

```
 SUBSYSTEM=="usb", ACTION=="add", ATTR{idVendor}=="03fd", ATTR{idProduct}=="000f", RUN+="/usr/bin/fxload -v -t fx2 -I /path/to/xusb_xlp.hex -D $tempnode"
```

- 使用 `udevadm control --reload` 重新加载 udev 规则并重新插入 JTAG
- 测试连接使用 `xc3sprog -c xpc -j`

2.3 LLVM

难度 入门 | **篇幅** 2.3 KB | **所属** 开发

本节将讲到

- 工具链
- 参见

来自维基百科：

: LLVM 编译器基础设施项目是用于开发编译器前端和后端的“模块化且可重用的编译器和工具链技术集合”。

工具链

- **Clang** — LLVM 的 C 语言家族前端。

: https://clang.llvm.org/ || clang包

- **lld** — 来自 LLVM 项目的高性能链接器。

: https://lld.llvm.org/ || lld包

- **LLDB)** — 用于 C、Objective-C 和 C++ 的高性能调试器。

: https://lldb.llvm.org/ || lldb包

- **llvm** — 编译器基础设施及附加工具集

: https://lld.llvm.org/ || llvm包

- **libc++** — LLVM C++ 标准库，完全支持 C++11 和 C++14。

: https://libcxx.llvm.org/ || libc++包

- **libclc** — OpenCL C 编程语言的标准库实现

: https://libclc.llvm.org/ || libclc包

- **Polly** — 基于 LLVM 的高级循环优化与数据局部性优化框架

: https://polly.llvm.org/ || polly包

参见

- https://llvm.org/

2.4 SDL

难度 入门 | **篇幅** 2.6 KB | **所属** 开发

本节将讲到

- 安装
- 文档
- 参见

SDL（Simple DirectMedia Layer）是一个跨平台软件开发库，旨在为计算机多媒体硬件组件提供一个硬件抽象层。软件开发者可用它编写高性能计算机游戏和其他多媒体应用程序。SDL 常常被误解为一个游戏引擎。不过，此库适合直接用于构建游戏，或被构建在其之上的引擎间接使用。

安装

安装 sdl3包 软件包。

如果您需要 sdl2AUR 或 sdl12-compat包，请相应安装，不过建议迁移到 SDL3。

SDL2 应用程序也可以使用 sdl2-compat包。SDL2 兼容层在幕后使用 SDL3。

与 SDL3 类似，SDL2 也是模块化的，不过模块都在不同的软件包中。SDL 2 的模块包包括 sdl2_image包、sdl2_mixer包、sdl2_ttf包 等，SDL1.2 的模块包包括 sdl_image包、sdl_mixer包、sdl_ttf包 等。

文档

官方 SDL Wiki 提供了使用 SDL3 库所需的各种资源和文档。

官方还提供了关于将程序从 SDL2 迁移到 SDL3 的文档。

此外，SDL3 Examples 还提供了一些小型示例程序。

参见

- SDL 发布
- SDL 论坛
- SDL 主页
- SDL 邮件列表
- SDL 语言绑定
- SDL Github
- Wikipedia:Simple DirectMedia Layer

2.5 Python/虚拟环境

难度 进阶 | **篇幅** 6.5 KB | **所属** 开发

本节将讲到

- 概述
- 安装
- 使用方法
- Python 版本
- virtualenvwrapper

虚拟环境工具可为 Python 应用程序创建一个隔离的工作空间。 虚拟环境的优势在于，可在本地安装模块、导出工作环境，并在其中执行 Python 程序。

概述

虚拟环境是安装了一些二进制文件和 shell 脚本的目录。 二进制文件通常是用于执行脚本的 *python* 和用于安装模块的 *pip*。 虚拟环境中也包含了用于激活环境的 shell 脚本（bash、csh 和 fish 均可用）。 虚拟环境本质上是模拟了一个安装了 Python 和所有所需的模块的完整系统，同时不会影响到任何运行此应用程序的系统。

Pipenv 于 2017 年发布，提供了上述所有工具的管理功能——管理 python 解释器所在的虚拟环境、包/模块和依赖关系及其激活和 Pipfiles 中可重现的版本锁定。

安装

Python 3.3+ 版本提供了一个叫做 *venv* 的模块。 对于使用早于此版本的 Python 的应用程序，必须使用 *virtualenv*。

软件包

从下面的软件包中选一个进行安装即可开始使用 Python 虚拟环境：

- Python 3.3+：python包
- Python 3：python-virtualenv包

Pipenv：

- Python 3：python-pipenv包

使用方法

上述三种工具的工作流程基本相似。

创建虚拟环境

在项目目录中使用 *venv* 或 *virtualenv* 创建一个虚拟环境。 请确保将虚拟环境目录从版本控制系统中排除——一份 `pip freeze` 的拷贝即可重建这个环境。

提示：下面的示例中，*envname* 是虚拟环境名称也是目录名称。

venv

注意：这种方式将替换 *pyvenv* 脚本（此脚本已在 python包 3.8 版本中被移除）。

此工具由 python包（3.3+）提供：

```
$ python -m venv envname
```

virtualenv

python-virtualenv包 提供了可用于 Python 3 的 *virtualenv*。

```
$ virtualenv envname
```

激活虚拟环境

使用目录中提供的 shell 脚本以激活或停用虚拟环境：

```
$ source envname/bin/activate
```

虚拟环境激活后，命令行解释器的提示符前会显示类似下面的虚拟环境名称：

```
(envname) $
```

在虚拟环境中可正常使用 *pip* 安装模块和运行脚本。

运行如下的命令即可退出虚拟环境（此命令由 `bin/activate` 提供）：

```
(envname) $ deactivate
```

Python 版本

默认情况下，虚拟环境由系统上安装的 Python 创建。 其中的 *bin/python* 是一个链接到系统上所安装的 *python* 的符号链接。 用 `ls` 查看可得到类似如下的输出结果：

```
$ ls -l envname/bin/python
lrwxrwxrwx 1 foo foo 15 Jan 29 18:48 envname/bin/python -> /usr/bin/python
```

对于 *virtualenv*，可以使用 `-p` 或 `--python` 选项在虚拟环境中使用其他版本的 Python：

```
$ virtualenv -p 3.8 envname
$ ls -l envname/bin/python
lrwxrwxrwx 1 foo foo 18 Jan 29 18:48 envname/bin/python -> /usr/bin/python3.8
```

也可以使用 pypy3包：

```
$ virtualenv -p pypy3 envname
```

virtualenvwrapper

virtualenvwrapper 提供了更自然的命令行交互（一些用于创建、激活以及删除虚拟环境的命令）以管理虚拟环境。 该软件包是对 python-virtualenv包 的包装。

安装

安装 python-virtualenvwrapperAUR 软件包，并在 `~/.bashrc` 中添加下面的配置：

```
~/.bashrc
export WORKON_HOME=~/.virtualenvs
source /usr/bin/virtualenvwrapper.sh
```

`source /usr/bin/virtualenvwrapper.sh` 会影响 shell 的启动速度。 使用 `source /usr/bin/virtualenvwrapper_lazy.sh` 可解决此问题，在这种情况下，只有当 virtualenvwrapper 被首次调用时才会加载。

重新打开 shell 以应用更改。 `WORKON_HOME` 目录会自动创建。

基本用法

使用方法和扩展功能请参见 virtualenvwrapper 文档。

创建虚拟环境（除了 `-a`、`-i`、`-r` 和 `-h`，其余的命令行参数都会被直接传递给 *virtualenv*，这里同样可以使用 `-p` 来指定 Python 版本）：

```
$ mkvirtualenv envname
```

激活虚拟环境：

```
$ workon envname
```

在虚拟环境中安装包（模块），以 Django 为例：

```
(envname) $ pip install django
```

停用虚拟环境：

```
(envname) $ deactivate
```

Pipenv

pipenv 提供单个、囊括了上述工具的所有功能的程序，实现了更好的命令行交互管理。

安装

安装 python-pipenv包 软件包。

基本用法

所有命令都可在项目目录中执行，pipenv 还会检测并定位 virtualenv，使用特定的虚拟解释器执行等。

更多信息请参见：、 和 。

另请参见

- Python venv
- PyPI 页面上的 virtualenv
- virtualenvwrapper 文档

2.6 Node.js

难度 进阶 | **篇幅** 6.7 KB | **所属** 开发

本节将讲到

- 安装
- Node Packaged Modules
- 问题处理
- 其他资源

Node.js 是一个 javascript 运行环境，并附带有功能丰富的库.使用 Google's V8 引擎在浏览器外执行代码。 由于 Node.js 是事件驱动、非阻塞 I/O，非常适合于实时 web 应用。

安装

安装 nodejs包。还可以安装长期支持版本：

- nodejs-lts-krypton包 - 24.X
- nodejs-lts-jod包 - 22.X
- nodejs-lts-iron包 - 20.X

多版本需求

如果需要使用多个 nodejs包 版本，可以使用 NVM (Node Version Manager)。nvm包 可以很方便的安装多个版本，并在版本间快速切换。命令很简单：

将下面命令加入 shell 的启动文件：

```
# Set up Node Version Manager
source /usr/share/nvm/init-nvm.sh
```

项目的 GitHub 页面包含使用文档，命令很简单：

```
$ nvm install 8.0
Downloading and installing node v8.0.0...
[..]

$ nvm use 8.0
Now using node v8.0.0 (npm v5.0.0)
```

使用 nvm包 时，可以用 pacman 的 `--assume-installed nodejs=<version>` 参数避免安装系统提供的版本。

如果希望在目录存在 `.nvmrc` 时自动执行 `nvm use`，将此配置添加到 shell 初始化文件。

Node Packaged Modules

npm 是官方的 node.js 包管理器，通过 npm包 进行安装。

使用 npm 管理包

安装软件包

任何包可以用以下命令安装：

```
$ npm install packageName
```

这个命令会将包安装在当前目录下 `node_modules` 目录内，可执行命令（如果有）安装在 `node_modules/.bin` 目录下.

作为系统级的全局安装使用 `-g` 选项:

```
# npm -g install packageName
```

默认情形下这个命令会将包安装至 `/usr/lib/node_modules/npm`，需要管理员权限.

用户级别安装

作为个人用户级的安装您可以使用一个本地目录来配置 `npm` 。请配置 `npm_config_prefix` 用户环境变量。npm 和 yarn 都会使用此环境变量。

```
~/.profile
PATH="$HOME/.node_modules/bin:$PATH"
export npm_config_prefix=~/.node_modules
```

不要忘记重新登录或读取新配置。

也可以在 `npm install` 时指定 `--prefix` 参数，但是不建议使用这个方式，因为需要每次安装全局软件包时都记得使用此参数。

```
$ npm -g install packageName --prefix ~/.node_modules
```

还有一个方式是设置 `$HOME/.npmrc` 中的 `prefix`。这和设置 `.profile` 中的 `npm_config_prefix="$HOME/.local"` 具有同样的效果，但是仅应用于 *npm*。

```
$ npm set prefix="$HOME/.local"
```

更新包

更新包只需要执行

```
 $ npm update packageName
```

对于全局环境安装的包 (`-g`)

```
 # npm update -g packageName
```

注意：请记住全局安装的包需要管理员权限，除非使用 `prefix` 设置到用户可写目录。

更新所有包

有时您只希望更新所有包，去掉包名将试图更新所有包。

```
 $ npm update
```

或者添加 `-g` 标记更新全局环境安装的包

```
 # npm update -g
```

删除包

删除使用 `-g` 标记安装的包只须：

```
# npm -g uninstall packageName
```

注意：请记住全局安装的包需要管理员权限

若删除个人用户目录下的包去掉标记执行：

```
 $ npm uninstall packageName
```

列出所有包

若要显示已安装的包的树形视图执行：

```
$ npm -g list
```

仅显示顶层树：

```
$ npm list --depth=0
```

要显示需要更新的过期软件包：

```
$ npm outdated
```

使用 pacman 管理包

一些 node.js 包可以在 Arch User Repository 找到，命名为 `nodejs-packageName` 格式。要向 AUR 中添加 node.js 软件包，请参考 Node.js 打包准则。

问题处理

npm help 不显示文档

如果 `npm help *topic*` 没有显示 *topic* 的文档，请使用 `man npm-*topic*`。例如:

```
$ npm help install
Top hits for "install"
...
$ man npm-install
... shows the documentation for the npm install subcommand
```

这是 Arch npm 软件包的一个 bug。

node-gyp python 错误

如果出现 **gyp WARN EACCES user "root" does not have permission to access the ... dir**，可以使用 *\--unsafe-perm* 选项:

```
# npm install --unsafe-perm -g node-inspector
```

无法找到模块错误

从 npm 5.x.x. 开始，package-lock.json 会和 package.json 文件一起创建，如果两个文件引用了不同的版本，会出现冲突。临时解决方案是：

```
$ rm package-lock.json
$ npm install
```

nmp 5.1.0 或以上版本已经解决了此问题，请参考: missing dependencies

其他资源

更多关于 nodejs包 和官方包管理器 npm 的使用信息您也许需要查询下列额外资源。

- NodeJs Documentation Node 文档和教程。
- NodeJS Community
- API Documentation 官方 `npm` API 文档
- IRC channel #node.js on irc.libera.chat

中文社区

- v2ex NodeJS 节点开发者作品发布与开发探讨
- cnodejs.org Node.js 专业中文社区

2.7 核心转储

难度 进阶 | **篇幅** 8.4 KB | **所属** 开发

本节将讲到

- 禁用自动核心转储
- 生成核心转储

- 管理核心转储
- 分析核心转储
- 另见

核心转储是进程意外终止时包含进程地址空间（内存）的文件。核心转储可以按需生成（例如由 debugger 生成），也可以在进程终止时自动生成。核心转储由内核在程序崩溃时触发，并可能传递给辅助程序（如 systemd-coredump(8)）进行进一步处理。普通用户通常不会使用核心转储，但开发人员可以将其用作程序崩溃时的状态快照，尤其是在故障难以可靠重现的情况下。

警告：核心转储可能包含敏感数据（如密码或加密密钥），因此只能与受信方共享。

禁用自动核心转储

出于多种原因，用户可能希望禁用自动核心转储：

- 性能：为占用内存大的进程生成核心转储会浪费系统资源并延迟内存清理。
- 磁盘空间：如果不压缩，占用内存大的进程的核心转储占用的磁盘空间可能等于甚至大于该进程的内存占用空间。
- 安全性：核心转储虽然通常只有 root 可读，但敏感数据（如密码或加密密钥）会在崩溃后写入磁盘。

使用 sysctl

可以将 `kernel.core_pattern` 设置为空值以使 sysctl 不处理核心转储。请创建以下文件：

```
/etc/sysctl.d/50-coredump.conf
kernel.core_pattern=|/bin/false
```

要立即应用设置，请执行以下命令：

```
# sysctl -p /etc/sysctl.d/50-coredump.conf
```

使用 systemd

`/usr/lib/sysctl.d/50-coredump.conf` 文件定义了 systemd 的默认行为，该文件将`kernel.core_pattern` 设置为调用 `systemd-coredump`，并在 `/var/lib/systemd/coredump` 为所有进程生成核心转储。请在 `/etc/systemd/coredump.conf.d/` 目录中创建包含以下内容的配置以覆盖 `systemd-coredump` 行为（参见 coredump.conf(5) § DESCRIPTION, ）：

```
/etc/systemd/coredump.conf.d/custom.conf
[Coredump]
Storage=none
ProcessSizeMax=0
```

注意：记得包含 `[Coredump]` 部分名称，否则该选项将被忽略。

之后用 daemon-reload 重新加载 systemd 配置文件。

参见 systemd-coredump(8) § Disabling coredump processing。

使用 PAM 限制

**本文或本章节的事实准确性存在争议。**

原因： limits.conf#core 建议设置软限制，以便使用 `ulimit -c unlimited` 临时启用核心转储。（在 Talk:核心转储 中讨论）

通过 PAM 登录的用户，其最大核心转储大小由 limits.conf 定义的。用户将其设置为零将完全禁用核心转储。

```
/etc/security/limits.conf
* hard core 0
```

使用 ulimit

像是*bash*，*zsh*这样的命令行解释器中提供了内置的 *ulimit* 命令，这条命令可用于报告或设置 shell 和由 shell 启动的进程的资源限制。详见 bash(1) § SHELL BUILTIN COMMANDS 或 zshbuiltins(1) 。

在当前shell禁用核心转储：

```
$ ulimit -c 0
```

如果系统设置为使用 `kernel.core_pattern` 将核心转储通过管道传递到诸如 *systemd-coredump* 之类的程序中，那么 Linux 内核本身会忽略 *ulimit* 设置（参见 core(5)）。因此，是否遵循该设置取决于接收转储的程序（*systemd-coredump* 仍会遵循此设置）。

对于未使用 *ulimit* 设置的崩溃进程的程序，可以使用 `dumpable` 和 prctl(2) 来禁用特定进程的核心转储处理。

生成核心转储

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Partially duplicates Debugging/Getting traces#Attaching to an existing process and misses the info on ptrace scope.（在Talk:核心转储讨论）

要生成任意进程的核心转储，首先要安装 gdb包 软件包。然后查找正在运行的进程的 PID，例如使用“pgrep”：

```
$ pgrep -f firefox
2071 firefox
```

关联进程：

```
$ gdb -p 2071
```

在 `(gdb)` 提示下：

```
(gdb) generate-core-file
Saved corefile core.2071
(gdb) quit
```

现在您有了一个名为 `core.2071` 的 coredump 文件。

核心转储文件去哪儿了?

`kernel.core_pattern` 内核模式 sysctl 决定自动核心转储的位置。默认情况下，核心转储将被发送到 *systemd-coredump*，可以使用 `/etc/systemd/coredump.conf` 来定义其行为。默认下，所有的核心转储文件储存在 `/var/lib/systemd/coredump`（因为配置了 `Storage=external`），它们被 `zstd` 压缩（因为配置了 `Compress=yes`）。此外，还可以配置存储的各种大小限制。

注意：`/usr/lib/sysctl.d/50-coredump.conf` 中配置了 `kernel.core_pattern` 的默认值，如果遵循默认的 sysctl.d(5) 规则，该文件可能会被屏蔽或覆盖。

要从日志中获取核心转储，请参见 coredumpctl(1)。

管理核心转储

使用 *coredumpctl* 查找相应的转储文件。请注意，普通用户无需特殊权限即可运行 *coredumpctl* 来管理其进程的核心转储。

```
# coredumpctl list
```

清理核心转储

存储在 `/var/lib/systemd/coredump/` 中的核心转储文件会被 `systemd-tmpfiles --clean` 自动清理，该命令由 `systemd-tmpfiles-clean.timer` 每天触发。核心转储文件的默认保留时间为至少 3 天，具体配置可通过运行 `systemd-tmpfiles --cat-config` 查看。

分析核心转储

首先，您需要找到需要相关转储。可通过指定 `PID`、可执行文件名称、可执行文件路径或 journalctl 谓词来实现（详见 coredumpctl(1) 和 journalctl(1)）。要查看核心转储的详细信息：

```
# coredumpctl info match
```

注意 “Signal” 行，它有助于确定崩溃原因。分析时，通常使用调试器（默认为 gdb(1)）检查回溯：

```
# coredumpctl debug match
```

启动 *gdb* 后，使用 `bt` 命令打印完整的回溯：

```
(gdb) thread apply all backtrace full
```

在许多情况下，输出将包含问号作为缺失调试符号的占位符。如何获取这些符号，请参阅调试/获取跟踪数据。

另见

- american fuzzy lop - 一个自动测试内核和程序的工具
- Filesystem fuzzing - LWN 有关测试文件系统错误的文章

2.8 Ollama

难度 进阶 | **篇幅** 9.8 KB | **所属** 开发

本节将讲到

- 安装
- 使用
- 配置
- 故障和修复
- 另请参阅

翻译状态：

- 本文（或部分内容）译自 Ollama，最近一次同步于 2026-07-29，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

Ollama 可以让用户在离线情况下，使用本地版大语言模型。

安装

安装 ollama包 软件包，提供了后台服务、命令行工具以及使用CPU推断的能力。

使用显卡推理:

- 安装 ollama-vulkan包，以使用 Vulkan 推理。
- 安装 ollama-cuda包，以使用 CUDA 推理。
- 安装 ollama-rocm包，以使用 ROCm 推理。

提示：对于搭载来自 RDNA3 的 AMD 显卡，ollama-vulkan包 通常比 ollama-rocm包 可以更快且使用更少的电量，并且可以避免 ROCm 的不支持显卡问题（另见 #ROCm 没有使用我的 AMD 显卡）。集成显卡自动被跳过；设定 `OLLAMA_IGPU_ENABLE=1` 来允许他们。

接下来，启用/启动 `ollama.service`。然后验证 Ollama 是否成功启动：

```
$ ollama --version
```

如果输出提示 `Warning: could not connect to a running Ollama instance`，则 Ollama 服务没有成功启动；否则，Ollama 服务启动成功并且进入 Ollama 程序的命令行界面，等待用户输入命令。

然后，验证您可以运行模型。下面的命令将会下载最新的 DeepSeek-R1 蒸馏模型 1.5b 参数版 并返回一个 Ollama 的提示符允许您和这个模型对话。

```
$ ollama run deepseek-r1:1.5b
>>> Send a message (/? for help)
```

使用

Ollama 的可执行程序没有提供一个搜索界面。没有类似 `ollama search` 的命令。需要寻找一个模型，您需要访问他们的 搜索页面。

运行模型：

```
$ ollama run model
```

停止模型：

```
$ ollama stop model
```

拉取模型：

```
$ ollama pull model
```

删除模型：

```
$ ollama rm model
```

查看本地可用模型：

```
$ ollama list
```

配置

全系统配置

Ollama 通过环境变量来进行配置。设定它们在对应服务的附加配置片段，然后重启 `ollama.service`：

```
/etc/systemd/system/ollama.service.d/environment.conf
[Service]
Environment="VARIABLE=value"
```

常用变量：

变量名	解释
`OLLAMA_FLASH_ATTENTION=1`	更快地提示词处理和更小的键值缓存；键值缓存量化条件。
`OLLAMA_KV_CACHE_TYPE`	键值缓存量化，需要注意: - `f16` （默认） - `q8_0`（`f16` 的一半 VRAM，轻微的质量下降） - `q4_0`（`q8_0` 的一半 VRAM，明显的质量下降）

查看 `ollama serve --help` 和 上游 FAQ 以了解其余变量。

注意：当安装有 ollama包 时，Vulkan 支持默认启用；`OLLAMA_VALKAN=1` 就不需要了。

警告：Ollama API 没有验证措施。若使用 `OLLAMA_HOST=0.0.0.0`，这这个网络上的任何人都可以使用模型————使用防火墙来限制访问如果这个网络不可信。

每个模型参数

要运行一个带有自定义设定的模型，需要把它们使用一个 Modelfile 来拷入一个新模型。

```
Modelfile
FROM deepseek-r1:1.5b
PARAMETER num_gpu 29
PARAMETER num_ctx 32768
```

变量名	解释
`FROM`	一个被 `ollama list` 列出的已拉取模型。
`PARAMETER num_gpu`	Number of model layers offloaded to the GPU. Speed rises with each extra layer until VRAM fills. One layer too many spills into GTT (system RAM over PCIe) and speed collapses, even though the model still reports "100% GPU". Benchmark a few values - the optimum is the most layers that fit just under *free* VRAM. For example, on a 16 GiB RX 9070 XT running `gemma4:26b` (Q4_K_M) at 32k context, 29 of 31 layers hit ~88 tok/s with most of the VRAM available; 30 layers dropped to ~75 tok/s (spill) and the auto-offload default gave ~37. With a browser holding ~5 GiB of VRAM the optimum fell to ~24 layers. On an 8 GiB GPU only ~13 layers of this model fit and generation is mostly CPU-bound - there, prefer a model that fits in VRAM entirely (a 7-9B model at Q4_K_M takes ~4-5 GiB) at 8-16k context.

之后，使用以下命令创建这个模型：

```
$ ollama create deepseek-r1-tuned -f Modelfile
```

`ollama create` 不会复制本体————这些存储是由内容定位的，所以一个调整过的变种只会增加一个小的 manifest。在 API `option` 字段中，被个请求也可以传入同样的参数。

Ollama 不会保存原 Modelfile；这些参数作为附加层存在模型存储的地方（`/var/lib/ollama`，manifest 在 `manifests/registry.ollama.ai/library/*name*/*tag*/`）。要更改一个创建好的模型，重写他的 Modelfile，编辑它并再次执行 `ollama create` 使用相同的名字（仅有小的 manifest 被重写）

```
$ ollama show --modelfile deepseek-r1-tuned > Modelfile
$ ollama create deepseek-r1-tuned -f Modelfile
```

参见 Modelfile 参考 以了解其余指令和参数。

故障和修复

ROCm 没有使用我的 AMD 显卡

您可能通过类似 amdgpu_top包 去监控您的显卡时在运行 Ollama 服务时，发现显卡根本没有被使用。

在没有配置的情况下，ROCm 忽略不支持的显卡，这导致所有东西都会在CPU下进行。

注意：检验支持显卡通过检查 ROCm 系统要求。

为了解决这个问题，针对 `ollama.service` 创建一个附加配置片段：

```
/etc/systemd/system/ollama.service.d/override_gfx_version.conf
[Service]
Environment="HSA_OVERRIDE_GFX_VERSION=X.Y.Z"
```

其中 `X.Y.Z` 由您系统的 GFX 版本所决定。

为了设定使用 GFX 的哪个版本来使用，首先确保 rocminfo包 已安装。它应该被安装在您的系统里作为一个 rocblas包 的非直接的依赖，这由 hipblas包 所需要，这并由 ollama包 所需要。

接下来，查询您系统的真正 GFX 版本：

```
$ /opt/rocm/bin/rocminfo | grep amdhsa
```

您需要记住输出的 `gfx` 后边的数字，因为这是您系统真正的 GFX 版本。这些数字按以下的方式解读：

- 如果数字为4位数，它们可以释作 `XX.Y.Z` ，及前两个数字为 `X` 部分。
- 如果数字为3位数，三位数字可被解释为 `X.Y.Z` 。

然后，查找所有安装的 rocblas包 内核：

```
$ find /opt/rocm/lib/rocblas/library -name 'Kernels.so-*'
```

您需要设定 `X.Y.Z` 为其中一个列出的可用版本。以下是总结好的的规则：

1. 对于 `X` 部分，它必须严格等于实际版本。
2. 对于 `Y` 部分，可以不同，但是它必须不得超过实际版本。
3. 对于 `Z` 部分，可以不同，但是它必须不得超过实际版本。

在设定好正确的 `X.Y.Z` 后，执行 daemon-reload 并 重启 `ollama.service`。

现在，您可以照常地运行您的模型。您可能希望再次通过类似 amdgpu_top包 去监控集成显卡的使用情况。

卸载 Ollama 后模型仍存在

您可以手动删掉这些模型文件，这些文件存在于 `/var/lib/ollama/blobs`。

另请参阅

- Ollama Blog
- Ollama Docs

2.9 Eclipse

难度 进阶 | **篇幅** 10.5 KB | **所属** 开发

本节将讲到

- 安装
- 插件
- 启用 javadoc 集成
- 疑难问题
- 参阅

Eclipse 是一个开源的社区项目，它致力于提供一个通用的开发平台。Eclipse 项目最广为人知的是它的跨平台集成开发环境(IDE). Arch Linux 软件包 (以及本文) 只针对于此 IDE。

Eclipse IDE 主要是用 Java 写成，但是可以用来用数种语言开发应用程序，包括 Java, C/C++, PHP 和 Perl. 此 IDE 也可以提供 subversion 支持(见下文) 以及任务管理。

安装

从下面列表中选择一个进行安装:

- eclipse-jee-binAUR - Java EE 开发
- eclipse-java-binAUR - Java 开发
- eclipse-cpp-binAUR - C/C++ 开发
- eclipse-php-binAUR - PHP 开发
- eclipse-javascriptAUR\[损坏的链接：package not found\] - JavaScript 和网页开发

目前不支持同时安装多个版本 FS#45577: 请选择最接近您需求的软件包，其它功能可以通过#插件进行安装。

插件

大部分插件只需 **pacman** 就可安装 (更多信息见 Eclipse 插件包参考). 这样同时能保证它们是最新的。另外，你也可以使用 #Eclipse Marketplace 或内建的#插件管理器。

添加默认更新站点

确保您已配置好针对当前版本的 Eclipse 的更新站点，这样它就能自动安装插件依赖。Eclipse 最新版本是 Luna ，它的更新站点是:

https://download.eclipse.org/releases/luna. 转到 帮助 > 安装新软件 > 添加，填写名称后可以轻松地找到更新站点 - 比如，Luna 软件源 - 并填上网址。

Eclipse Marketplace

注意：确保您遵循了#添加默认更新站点部分。

要使用 Eclipse Marketplace, 首先要安装: 转到 帮助 > 安装新软件 > 切换到默认更新站点 > 通用工具 > Marketplace 客户端。重启 Eclipse, 之后在 帮助 > Eclipse Marketplace 可以找到。

插件管理器

注意：确保您遵循了#添加默认更新站点部分。

使用 Eclipse 的插件管理器以从官方源下载并安装插件: 这种情况下你需要在插件网站上找到需要的源，然后转到*帮助* > *安装新软件...*, 在 *Work with* 栏目里输入源地址，从下面的列表里选择要安装的插件并遵循教程。

注意：

- 如果你使用 Eclipse 的插件管理器，建议您以 root 身份运行 Eclipse: 这种方法插件会安装到 `/usr/lib/eclipse/plugins/`; 如果你以普通用户安装，它们会被存储在 `~/.eclipse/` 下的一个和版本号相关的文件夹里，并且升级 Eclipse 之后插件再也无法被识别。
- 日常工作时不要以 root 身份运行 Eclipse.

通过插件管理器升级

运行 Eclipse 并执行*帮助* > *检查更新*。如果你如上以 root 身份安装，那么需要以 root 身份来升级。

对于准备更新的插件，你应该确保已经启用它们的更新源*窗口* > *首选项* > *安装/升级* > *可用软件站点*: 你可在各自的网站上找到插件的更新源。要添加/编辑/移除... 源只需使用*可用软件站点*面板右部的按钮。对于 Eclipse 4.5 (Mars), 检查你是否启用了:

```
https://download.eclipse.org/releases/mars
```

要接受更新提示转到*窗口* > *首选项* > *安装/升级* > *自动更新*. 如果你想接受以 root 身份安装插件的更新提示，你需要以 root 身份运行 Eclipse. 转到*窗口* > *首选项* > *安装/升级* > *可用软件站点*, 选择插件相关的源并*导出*它们，然后以普通用户运行 Eclipse 并在同样的面板里*导入*它们。

插件列表

- **AVR** — AVR 微控制器插件。

: https://avr-eclipse.sourceforge.net/wiki/index.php/The_AVR_Eclipse_Plugin || eclipse-avrAUR

- **Aptana** — HTML5/CSS3/JavaScript/Ruby/Rails/PHP/Pydev/Django support. Also available as standalone application.

: http://www.aptana.com/ || aptana-studioAUR

- **IvyDE** — IvyDE 依赖管理器

: https://ant.apache.org/ivy/ivyde/ || eclipse-ivydeAUR\[损坏的链接：package not found\]

- **Markdown** — Eclipse 的 Markdown 编辑插件。

: https://www.winterwell.com/software/markdown-editor.php || eclipse-markdownAUR

- **PyDev** — Python 支持。

: https://www.pydev.org/ || eclipse-pydevAUR

- **Subclipse** — Subversion 支持。

: https://github.com/subclipse/subclipse || eclipse-subclipseAUR

- **Subversive** — 另一 Subversion 支持。

: https://www.eclipse.org/subversive/ || eclipse-subversiveAUR

- **TestNG** — TestNG 支持。

: https://testng.org/doc/eclipse.html || eclipse-testngAUR

- **TeXlipse** — LaTeX 支持。

: http://texlipse.sourceforge.net/ || texlipseAUR\[损坏的链接：package not found\]

- **Checkstyle** — Eclipse Checkstyle support.

: https://checkstyle.org/eclipse-cs/ || eclipse-checkstyleAUR

启用 javadoc 集成

将鼠标指针放在标准 Java 方法的时候想看API条目?

在线版本

如果你的机器有不间断的互联网连接，您可以使用在线文档:

1. 转到 *窗口* > *首选项*, 然后转到 *Java* > *已安装的 JRE*.
2. 应该会有一个类型为 "Standard VM" 的 "java". 选择它并点击 *Edit*.
3. 选择 "JRE system libraries:" 下的 `/opt/java/jre/lib/rt.jar` 条目然后点击 *Javadoc Location...*.

4. 在 "Javadoc location path:" 文本栏里键入 "https://docs.oracle.com/javase/7/docs/api/".

离线版本

你可安装 openjdk8-doc包 软件包把文档存储到本地。Eclipse 能够自动找到 javadocs. 如果不起效，针对 rt.jar 把 Javadoc 设置到 `file:/usr/share/doc/java8-openjdk/api`.

疑难问题

Ctrl+X 关闭了 Eclipse

是这个 bug 的一部分。只要看看 `~/workspace/.metadata/.plugins/org.eclipse.e4.workbench/workbench.xmi` 并删除错误的 `Ctrl+X` 组合。通常它是第一个。

暗色主题

Luna 提供了暗色主题，可在 首选项 > 外观 再选择 'Dark' 主题启用。

暗色主题使用它自己的颜色而不是 GTK 主题的，如果你想完全使用 GTK 颜色设定的话，从此处删除或移动所有 .css 文件到备份子文件夹: /usr/share/eclipse/plugins/org.eclipse.ui.themes_1.0.0.xxxx/css/

使用 Gnome 3.6 Adwaita 主题时"提示"显示为深色背景色

如下取消 `/usr/share/themes/Adwaita/gtk-2.0/gtkrc` 倒数第二行的注释

```
#widget "gtk-tooltip*"  style "tooltips"
```

相关 bug:

- https://bugzilla.gnome.org/show_bug.cgi?id=688285
- https://bugs.eclipse.org/bugs/show_bug.cgi?id=381010 (WONTFIX)

禁止 GTK+ 3

当GTK 3 有很多bug并且不稳定时（比如，代码框不断闪烁），你可以禁止GTK 3。在启动 eclipse 时使用 SWT_GTK3=0 environment variable ：

```
SWT_GTK3=0 eclipse
```

另一个解决方案是添加下面两行到`/usr/lib/eclipse/eclipse.ini`。

```
--launcher.GTK_version
2
```

这两行必须加在下面两行之前：

```
--launcher.appendVmargs
```

切换按钮的选择/未选择状态是一样的

如下取消 `/usr/share/themes/Adwaita/gtk-2.0/gtkrc` 最后一行的注释

```
#widget "*swt*toolbar*" style "null"
```

要应用修改后的主题，使用 `gnome-tweak-tool` 来选择另一主题之后切回 Adwaita.

相关 bug:

- https://bugzilla.gnome.org/show_bug.cgi?id=687519

改变默认窗口标题字号

使用 Eclipse 配置并不能改变窗口字号，你必须编辑对应的 .css 文件。注意，当你更新 Eclipse 后必须再执行一遍。 它们位于

```
/usr/share/eclipse/plugins/org.eclipse.platform_4.3.<your version number>/css
```

Open the appropriate file with your text editor, ie e4_default_gtk.css if you are using the "GTK theme". 寻找 .MPartStack, 并把字号改成你想要的大小

```
.MPartStack {
	font-size: 9;
	swt-simple: false;
	swt-mru-visible: false;
}
```

参阅

- How to use Subversion with Eclipse

2.10 Vulkan

难度 进阶 | **篇幅** 11.0 KB | **所属** 开发

本节将讲到

- 安装
- 验证
- 切换
- 软件渲染
- Vulkan 硬件数据库

来自 wikipedia:Vulkan (API)):

: Vulkan是一种低开销、跨平台的3D图形和计算API。它于2016年首次发布，是 OpenGL 的继任者。

在 Vulkan的网站 了解更多信息。

安装

提示：在混合图形技术（NVIDIA Optimus/AMD Dynamic Switchable Graphics）上:

- Vulkan 目前没有被 Bumblebee 官方支持，但可以工作在 primus_vk包上。
- Radeon Vulkan 驱动程序现在支持 PRIME 。

要运行 Vulkan 程序，你需要安装 vulkan-icd-loader包 软件包（如果想运行32位程序，还需要从multilib仓库安装 lib32-vulkan-icd-loader包）以及显卡的驱动程序。有几个软件包可以提供 *vulkan-driver* 和 *lib32-vulkan-driver*：

- AMD: vulkan-radeon包 (或 lib32-vulkan-radeon包)
- Intel: vulkan-intel包 (或 lib32-vulkan-intel包)
- NVIDIA: 有两种实现：
- nvidia-utils包 (或 lib32-nvidia-utils包) - NVIDIA 专有
- vulkan-nouveau包(或lib32-vulkan-nouveau包) - NVK (Mesa 项目的一部分)

> **提示**：启用 NVK 还需要额外的系统配置，有关详细信息，请参阅 Nouveau#使用 Mesa NVK Vulkan 驱动程序。

以下为软件光栅化器，可让您在不支持 Vulkan 的设备上使用。

- **Lavapipe**: vulkan-swrast包 (32位程序支持需安装lib32-vulkan-swrast包)
- **SwiftShader**: swiftshader-gitAUR

对于 Vulkan 程序开发，可以安装 vulkan-devel包组 软件包组。或者，也可以单独安装 vulkan-headers包，并根据需要选择安装 vulkan-validation-layers包、vulkan-man-pagesAUR 和 vulkan-tools包（其中包含 vulkaninfo 和 vkcube 工具）。

验证

要查看系统上当前安装了哪些 Vulkan 实现，请使用以下命令：

```
$ ls /usr/share/vulkan/icd.d/
```

为确保 Vulkan 在你的硬件上工作，安装 vulkan-tools包 并使用 `vulkaninfo` 命令调出你的系统信息。如果你得到了显卡信息，就说明 Vulkan 在正常工作。

```
$ vulkaninfo
```

访问 https://linuxconfig.org/install-and-test-vulkan-on-linux 以获取更多信息。

切换

在设备之间切换

在具有多个 GPU 的系统上，您可能需要强制使用特定 GPU。此功能需要安装 vulkan-mesa-implicit-layers包 软件包。通过将环境变量 `MESA_VK_DEVICE_SELECT` 设置为 `*vendorID:deviceID*`，可以选择所需的 GPU。

要列出可用的 GPU，请运行：

```
$ MESA_VK_DEVICE_SELECT=list vulkaninfo
```

在指定值的末尾添加 ！可以强制使用所选 GPU。有关更多信息，请参阅 \[https\:\://docs.mesa3d.org/envvars.html#vulkan-mesa-device-select-layer-environment-variables Vulkan Mesa 设备选择层环境变量\]。

软件渲染

您可以安装名为lavapipe 的 Vulkan 光栅化软件，例如用于调试硬件问题：vulkan-swrast包 。(或者它的32位版本lib32-vulkan-swrast包).

以下示例展示了如何通过设置必要的环境变量运行 *vulkaninfo*，以强制 Vulkan 和 OpenGL 完全使用软件渲染（其中 `__GLX_VENDOR_LIBRARY_NAME=mesa` 可确保该命令同样适用于使用 PRIME 的用户）：

```
$ LIBGL_ALWAYS_SOFTWARE=1 __GLX_VENDOR_LIBRARY_NAME=mesa
VK_DRIVER_FILES=/usr/share/vulkan/icd.d/lvp_icd.json vulkaninfo
```

Vulkan 硬件数据库

硬件数据库提供了用户报告的 GPU/驱动程序组。使用 vulkan-caps-viewer-waylandAUR 或 vulkan-caps-viewer-x11AUR 可以提供自己的信息。

疑难解答

NVIDIA - vulkan 无法工作并无法初始化

环境变量

无效或相互冲突的环境变量值可能导致 Vulkan 无法正常工作，而不恰当的设置可能导致配备多个 GPU 的系统使用非预期的 GPU。正确设置这些变量还有助于在不需要使用辅助 GPU 时使其保持关闭状态。

GPU 切换

如果系统配备多个 GPU，而 Vulkan 无法检测或使用其中某个 GPU，请确保该 GPU 未被 BIOS/UEFI 或内核禁用。有关不同 GPU 切换方式的概述，请参阅 NVIDIA Optimus。

以下示例展示了使用 optimus-manager-gitAUR 检查当前状态的命令：

```
$ optimus-manager --status
Optimus Manager (Client) version 1.4

Current GPU mode : nvidia
GPU mode requested for next login : no change
GPU at startup : integrated
Temporary config path: no
```

GSP 固件

参阅 NVIDIA/故障排除#GSP 固件。

找不到显示 GPU 的设备。是否安装了 intel-mesa 驱动程序?

尝试将 intel_icd 与 primus_vk_wrapper 两者都配置进 VK_ICD_FILENAMES 列表

```
export
VK_ICD_FILENAMES=/usr/share/vulkan/icd.d/intel_icd.x86_64.json:/usr/share/vulkan/icd.d/
nv_vulkan_wrapper.json
```

在 vulkaninfo 之后出现 AMDGPU - ERROR_INITIALIZATION_FAILED

如果在 GCN1 或 GCN2 系列的 AMD 卡上运行 `vulkaninfo` 后，你会收到如下错误消息：

```
ERROR at /build/vulkan-tools/src/Vulkan-
Tools-1.2.135/vulkaninfo/vulkaninfo.h:240:vkEnumerateInstanceExtensionProperties failed
with ERROR_INITIALIZATION_FAILED
```

然后检查你是否已正确启用对此型号显卡的支持（AMDGPU#开启 Southern Islands (SI) 和 Sea Islands (CIK) 支持_和_Sea_Islands_(CIK)_支持)）。

可以使用 `lspci -k` 检查 gpu 驱动程序是否已正确加载，在运行该命令后检查内核驱动中关于 gpu 的信息，它应该是 `amdgpu`。

```
$ lspci -k
...
01:00.0 VGA compatible controller: Advanced Micro Devices, Inc. [AMD/ATI] Curacao PRO
[Radeon R7 370 / R9 270/370 OEM]
       Subsystem: Gigabyte Technology Co., Ltd Device 226c
       Kernel driver in use: amdgpu
       Kernel modules: radeon, amdgpu
...
```

关于这个问题的一些论坛主题：

Vulkan 应用程序启动缓慢（延迟 30 秒）

启动时，Vulkan 会尝试初始化由 环境变量 `VK_DRIVER_FILES` 指定的可安装客户端驱动程序（Installable Client Driver，ICD）。该变量指向一个文件（例如 NVIDIA 的 `/usr/share/vulkan/icd.d/nvidia_icd.json`），其中包含 Vulkan GPU 驱动程序路径的相关信息。如果该变量指向了错误的 ICD JSON 文件，某些应用程序（例如基于 Chromium/Electron 的应用程序）可能会在启动时出现延迟，直到达到 30 秒的超时时间。

当 `nvidia_icd.json` 文件存在时，Vulkan 会尝试先于 Radeon 驱动程序加载 NVIDIA 驱动程序。因此，对于安装了 nvidia-utils包 软件包（例如为了使用 cuda包）的 Radeon GPU 系统，可能需要手动设置环境变量 `VK_DRIVER_FILES=/usr/share/vulkan/icd.d/radeon_icd.json`。

在混合图形技术配置中，即使已经关闭独立 GPU（dGPU），Vulkan 仍可能尝试加载 NVIDIA 驱动程序。在这种情况下，应将 `VK_DRIVER_FILES` 设置为集成

GPU（iGPU）驱动程序对应的 ICD 文件。也可以完全取消设置该变量，这种方法似乎同样有效，但其行为未定义，因此不推荐使用。

2.11 OpenGL

难度 进阶 | **篇幅** 11.0 KB | **所属** 开发

本节将讲到

- 安装
- 验证安装
- 在驱动程序之间切换
- Vulkan 上的 OpenGL (Zink)
- 故障排除

摘自 Wikipedia:OpenGL:

: OpenGL (Open Graphics Library) 是一个跨语言、跨平台的应用程序编程接口（API），用于渲染 2D 和 3D 矢量图形。

可在Khronos上了解更多信息。

OpenGL 的开发已于 2017 年停止，取而代之的是 Vulkan，这是“下一代”API，可在较新的硬件上提供更高的性能。

安装

要运行使用 OpenGL 的应用程序，您需要为您的硬件（GPU 或 CPU）安装正确的驱动程序。

提示：对于 NVIDIA，建议将专有驱动程序用在比 Kepler (NVE0/GK*XXX*) 系列更新的显卡上，并且总体上性能更好。

注意：

- 英特尔的GenN硬件不是指CPU的一代，而是指GPU的一代，这与CPU的一代不同。
- 要查找 AMD（和 ATI）GPU 系列，请查阅Wikipedia:List of AMD graphics processing units#Features overview。
- 要查找 NVIDIA GPU 的代号，请查看 Nouveau 项目中的代码列表。
- 与 *Fedora Linux* 的社区软件仓库不同，*Arch Linux* 不会为受专利保护的视频和音频编解码器提供“freeworld”软件包变体；标准 Mesa 软件包在编译时已启用所有相关编解码器，与 *Fedora* 的“freeworld”软件包相同。

Mesa 是一个开源的 OpenGL 实现，它不断更新以支持最新的 OpenGL 规范。它有一系列用于 Intel 图形处理器、AMDGPU（以前称为 ATI）和 NVIDIA GPU 的开源驱动程序。Mesa 还提供软件光栅器，例如 llvmpipe。

有两个 Mesa 软件包可选，每个软件包都有一组不同的驱动程序：

- mesa包 mesa 是最新的 Mesa 软件包，其中包括用于较新硬件的大多数现代驱动程序：
- `r300`：适用于AMD的Radeon R300, R400, 和 R500 GPU。
- `r600`：适用于 AMD 的 Radeon R600 GPU，最高支持Northern Islands架构。由 AMD 官方支持。
- `radeonsi`：适用于 AMD 的 Southern Island GPU 及更高版本。由 AMD 官方支持。
- `nouveau`：Nouveau是用于 NVIDIA GPU 的开源驱动程序。
- `virtio_gpu`：一个适用于 virtio 的虚拟 GPU 驱动程序，可与基于 QEMU 的 VMM（如 KVM 或 Xen）一起使用。
- `vmwgfx`：适用于 VMware 虚拟 GPU。
- `i915`：适用于英特尔的第 3 代硬件。
- `crocus`：适用于英特尔的第 4 代至第 7 代硬件。
- `iris`：适用于英特尔的第 8 代硬件及更高版本。由英特尔正式支持。
- `zink`：用于在 Vulkan 上运行 OpenGL 的 Gallium 驱动程序。
- `d3d12`：用于仅支持 D3D12（即 WSL）的设备上的 OpenGL 3.3 支持。
- `softpipe`：软件光栅器和参考 Gallium 驱动程序。
- `llvmpipe`：软件光栅化器，它使用 LLVM 生成 x86 JIT 代码，并且是多线程的。
- mesa-amber包 是传统的 Mesa 软件包，其中包括用于旧硬件的经典（非 Gallium3D）驱动程序：
- `i830`：适用于英特尔的第 2 代硬件。与 `i965` 相同的二进制文件。
- `i915`：适用于英特尔的第 3 代硬件。与 `i965` . 相同的二进制文件。
- `i965`：适用于英特尔的第 4 代至第 11 代硬件。由英特尔正式支持。
- `radeon`：适用于 AMD 的 Radeon R100 GPU。与 `r200` 相同的二进制文件。
- `r200`：适用于 AMD 的 Radeon R200 GPU。
- `nouveau_vieux`：适用于 NVIDIA NV04(Fahrenheit) 至 NV20(Kelvin) GPU。
- `swrast`：旧版软件光栅器

: > 注意：使用 Mesa 时，系统会自动选择正确的驱动程序，因此安装软件包后无需配置。

- nvidia-utils包 是用于NVIDIA GPU 的专有驱动程序，其中包括关于 OpenGL 的实现。

验证安装

要验证您的 OpenGL 安装，您可以使用 mesa-utils包 `eglinfo` ，它应该显示如下输出（当然，根据您的设置，其值不同）：

```
$ eglinfo -B
Wayland platform:
EGL API version: 1.5
EGL vendor string: Mesa Project
EGL version string: 1.5
EGL client APIs: OpenGL OpenGL_ES
OpenGL core profile vendor: Intel
OpenGL core profile renderer: Mesa Intel(R) UHD Graphics (CML GT2)
OpenGL core profile version: 4.6 (Core Profile) Mesa 25.1.3-arch1.3
OpenGL core profile shading language version: 4.60
OpenGL compatibility profile vendor: Intel
OpenGL compatibility profile renderer: Mesa Intel(R) UHD Graphics (CML GT2)
OpenGL compatibility profile version: 4.6 (Compatibility Profile) Mesa 25.1.3-arch1.3
OpenGL compatibility profile shading language version: 4.60
OpenGL ES profile vendor: Intel
OpenGL ES profile renderer: Mesa Intel(R) UHD Graphics (CML GT2)
OpenGL ES profile version: OpenGL ES 3.2 Mesa 25.1.3-arch1.3
OpenGL ES profile shading language version: OpenGL ES GLSL ES 3.20

X11 platform:
EGL API version: 1.5
EGL vendor string: Mesa Project
EGL version string: 1.5
EGL client APIs: OpenGL OpenGL_ES
OpenGL core profile vendor: Intel
OpenGL core profile renderer: Mesa Intel(R) UHD Graphics (CML GT2)
OpenGL core profile version: 4.6 (Core Profile) Mesa 25.1.3-arch1.3
OpenGL core profile shading language version: 4.60
OpenGL compatibility profile vendor: Intel
OpenGL compatibility profile renderer: Mesa Intel(R) UHD Graphics (CML GT2)
OpenGL compatibility profile version: 4.6 (Compatibility Profile) Mesa 25.1.3-arch1.3
OpenGL compatibility profile shading language version: 4.60
OpenGL ES profile vendor: Intel
OpenGL ES profile renderer: Mesa Intel(R) UHD Graphics (CML GT2)
OpenGL ES profile version: OpenGL ES 3.2 Mesa 25.1.3-arch1.3
OpenGL ES profile shading language version: OpenGL ES GLSL ES 3.20
```

在 X11 平台上，也可以使用`glxinfo`。

在同一个软件包中，您还可以使用 `eglgears` 或 `glxgears_x11` （在 X11 上）或 `eglgears_wayland` （在 Wayland 上）作为基本 OpenGL 测试。运行程序时，您应该会看到 3 个旋转齿轮。

在驱动程序之间切换

有关混合图形技术，请参阅 PRIME。

注意：根据这篇 Reddit 帖子 ，您可以使用来自不同供应商的 2 个 GPU 同时使用 PRIME，而不会出现任何问题。

Mesa

您可以使用以下环境变量覆盖用于应用程序的驱动程序：

```
MESA_LOADER_DRIVER_OVERRIDE=driver
```

默认情况下，Mesa 在 `/lib/dri/` 中搜索驱动程序。您可以使用以下命令查看已安装的驱动程序列表。

```
$ ls /lib/dri/
```

`*driver*_dri.so` 中的 `*driver*` 是驱动程序的实际名称。如果 Mesa 无法找到指定的驱动程序，就会退回到 `llvmpipe`。

您还可以通过设置以下环境变量来使用 OpenGL 软件光栅器：

```
LIBGL_ALWAYS_SOFTWARE=true
GALLIUM_DRIVER=driver
```

`*driver*` 可以是 `softpipe`, `llvmpipe` 或 `swr`。

提示：在大多数用例中， `llvmpipe` 与 `swr` 比 `softpipe` 更快。

Vulkan 上的 OpenGL (Zink)

来自 Mesa 文档:

: Zink 驱动程序是一个 Gallium 驱动程序，它能发出 Vulkan API 调用，而不是针对特定的 GPU 架构。它可用于在仅支持 Vulkan 的设备上获得完整的桌面 OpenGL 支持。

如果您在默认 OpenGL 驱动程序中遇到问题（RadeonSI、Iris 等中的错误），您可以尝试使用 Zink 驱动程序。

根据此 Phoronix 基准测试, 与 RadeonSI 相比，某些应用程序的平均 FPS 可能更低。

请注意，在使用 AMD 或 Intel DDX 驱动程序（分别为 xf86-video-amdgpu包 和 xf86-video-intel包）的 X 系统上，Zink 已无法开箱即用。上游开发者建议使用通用的 modesetting(4) DDX 驱动程序。 或者，也可以通过设置以下环境变量来绕过此问题：

```
$ LIBGL_KOPPER_DRI2=1 MESA_LOADER_DRIVER_OVERRIDE=zink
```

要在 NVIDIA 驱动程序上使用 Zink，请使用以下命令或类似命令：

```
$ env __GLX_VENDOR_LIBRARY_NAME=mesa
__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/50_mesa.json
MESA_LOADER_DRIVER_OVERRIDE=zink GALLIUM_DRIVER=zink LIBGL_KOPPER_DRI2=1 application
```

故障排除

联想设备的 Mesa GPU 图形错误

Mesa 使用 CPU（llvmpipe）进行软件渲染，导致部分图形界面软件崩溃。可以通过进入 BIOS 设置，将显卡模式从独立显卡模式（Discrete Graphics）切换为动态显卡模式（Dynamic Graphics）来解决此问题（如果使用其他品牌的计算机，应选择允许在不同 GPU 之间切换的选项，而不是禁用集成显卡）。如果未安装主 GPU 的驱动程序，却期望集成显卡正常工作，则可能出现此问题。

2.12 Snap

难度 进阶 | **篇幅** 11.7 KB | **所属** 开发

本节将讲到

- 安装
- 配置
- 使用
- 提示与技巧
- Troubleshooting

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

Snap 是一个软件部署和包管理系统。这些包被称为“Snap 软件包”（“snaps”），可以通过“snapd”来管理“Snap 软件包”。它适用于一系列 Linux 发行版，因此允许与发行版无关的上游软件开发。Snap 的开发商 Canonical 管理 Snap Store 服务。用户可以通过该服务安装 Snap 软件包。

snapd 是一个用于管理 Snap 软件包的 REST API 守护进程。用户可以通过使用 *snap* 客户端与它进行交互，该客户端是 snapd 包的一部分。

用户可以使用 AppArmor 限制 Snap 软件包。现在内核中默认启用了该功能。请查阅相关维基页面，了解在系统中启用 AppArmor 的步骤。

警告：

- 如果您的系统中未启用AppArmor，则所有快照都将以 *devel* 模式运行，这意味着它们将与从Arch Linux存储库安装的应用程序一样，可以不受限制地访问您的系统。
- 运行不受信任的代码永远是不安全的，沙盒也无法改变这一点。

安装

安装 snapdAUR 包组。

提示：`snapd` 在 `/etc/profile.d/snapd.sh` 中安装了一个脚本，记录了snapd软件包和桌面软件的二进制安装文件路径。重启后才会使改变生效。

如果你在你的系统中启用了 AppArmor，snapd 是支持这种安全模型的，请根据 AppArmor#Installation 进行安装。

如果你在使用 *AppArmor*，请确保启用并打开了`apparmor.service`和`snapd.apparmor.service`服务。

配置

请启用/开启`snapd.socket`服务以使`snapd`守护线程正常运行，这样才能正常使用*snap*。

使用

*用**snap**工具来管理snap软件包。*

查找软件包

用下面的命令可以在Ubuntu Store中查询你想要安装的snap软件包：

```
$ snap find searchterm
```

安装软件包

一但你找到想要安装的snap软件包了，用下面的命令来安装：

```
# snap install snapname
```

这将需要root权限。目前还不能为每个用户安装snap。为了使其可以被系统使用，snap会把下载的内容放到`/var/lib/snapd/snaps`目录并挂载到`/var/lib/snapd/snap/*snapname*`。

同时这也会为每个snap软件包创建挂载单元，并将它们作为符号链接添加到 `/etc/systemd/system/multi-user.target.wants/`下，以便在系统启动时使所有snap软件包可用。完成后，可以使用下面的命令在已安装snap软件包的列表中找到这个snap软件包以及其版本号、修订号和开发者：

```
$ snap list
```

你也可以用下面的命令从本地侧载snap软件包：

```
# snap install --dangerous /path/to/snap
```

Updating

使用以下命令来更新 snap 软件包：

```
# snap refresh
```

snap 的 `refresh.timer` 会自动更新 snap 软件包。

使用以下命令来查看下次或上次的更新时间：

```
# snap refresh --time
```

要设置不同的更新时间，例如每天两次：

```
# snap set core refresh.timer=0:00~24:00/2
```

请阅读 system options documentation page 来详细了解如个自定义更新时间。

移除软件包

下面的命令可以用来移除snap软件包：

```
# snap remove snapname
```

提示与技巧

旧版 Snap 包

某些 Snap 包（例如 Julia 和 Pycharm）使用旧版隔离模式。然而，旧版隔离模式需要 `/snap` 目录，但该目录不符合 FHS 标准。snapdAUR 软件包默认不提供此目录，但用户可以手动创建 `/var/lib/snapd/snap` 与 `/snap` 之间的符号链接，以允许安装旧版 Snap 软件包：

```
# ln -s /var/lib/snapd/snap /snap
```

隔离机制

当使用 AppArmor 时，*snapd* 会为 Snap 软件包生成与 Ubuntu 上相同的配置文件。AppArmor 解析器会忽略主线内核尚未支持的规则。

要验证基本隔离机制是否正常工作，请先安装 *hello-world* snap 包。然后运行以下命令：

```
$ hello-world.evil
Hello Evil World!
This example demonstrates the app confinement
You should see a permission denied error next
/snap/hello-world/27/bin/evil: 9: /snap/hello-world/27/bin/evil: cannot
create /var/tmp/myevil.txt: Permission denied
```

AppArmor 应该会拒绝此次访问并将其记录到日志中：

```
# dmesg
...
[  +0.000003] audit: type=1327 audit(1540469583.966:257):
proctitle=2F62696E2F7368002F736E61702F68656C6C6F2D776F726C642F32372F62696E2F6576696C
[ +12.268939] audit: type=1400 audit(1540469596.236:258): apparmor="DENIED"
operation="open" profile="snap.hello-world.evil" name="/var/tmp/myevil.txt" pid=10835
comm="evil" requested_mask="wc" denied_mask="wc" fsuid=1000 ouid=1000
[  +0.000006] audit: type=1300 audit(1540469596.236:258): arch=c000003e syscall=2
success=no exit=-13 a0=55d991ba6bc8 a1=241 a2=1b6 a3=55d991ba6be0 items=0 ppid=31349
pid=10835 auid=1000 uid=1000 gid=1000 euid=1000 suid=1000 fsuid=1000 egid=1000
sgid=1000 fsgid=1000 tty=pts2 ses=3 comm="evil" exe="/bin/dash" subj==snap.hello-
world.evil (enforce)
...
```

如果您没有看到拒绝访问的记录，请验证配置文件是否已加载：

```
# aa-status | grep snap.hello-world
   snap.hello-world.env
   snap.hello-world.evil
   snap.hello-world.hello-world
   snap.hello-world.sh
```

此外，您可以根据 snapd 检查系统中可用的沙箱功能：

```
$ snap debug sandbox-features
apparmor:              kernel:caps kernel:domain kernel:file kernel:mount
kernel:namespaces kernel:network_v8 kernel:policy kernel:ptrace kernel:query
kernel:rlimit kernel:signal parser:unsafe policy:default support-level:partial
confinement-options:  devmode
dbus:                  mediated-bus-access
kmod:                  mediated-modprobe
mount:                 freezer-cgroup-v1 layouts mount-namespace per-snap-persistency
per-snap-profiles per-snap-updates per-snap-user-profiles stale-base-invalidation
```

```
seccomp:              bpf-argument-filtering kernel:allow kernel:errno
kernel:kill_process kernel:kill_thread kernel:log kernel:trace kernel:trap
```

Hide the snap folder

See XDG Base Directory#Hiding unwanted directories to hide the `snap` folder.

Sudo

With `secure_path` being enabled in sudo by default,
the `/var/lib/snapd/snap/bin` directory is no longer present in the default `$PATH` environment variable of the process started by sudo. Commands such as `sudo lxc list` will fail, as the `lxc` symbolic link can no longer be found by the shell process.

This can be addressed on per user basis by adding the following snippet to `/etc/sudoers.d/90_snap`:

```
# Add snap binaries installation dir to PATH
Defaults:<your-user-name>
secure_path="/usr/local/sbin:/usr/local/bin:/usr/bin:/var/lib/snapd/snap/bin"
```

Where `<your-user-name>` must be replaced with the desired `$USER`.

Troubleshooting

Text unreadable

If you are seeing squares instead of readable characters, you need to clear the font cache:

```
# rm -f /var/cache/fontconfig/*
$ rm -f ~/.cache/fontconfig/*
# fc-cache -r -v
```

Snapctl also stores internal caches for each individual snap, which need to be cleared seperately. First, find them by running:

```
$ find ~/snap/ -wholename '*/.cache/fontconfig'
... /home/darth_vader/snap/mailspring/common/.cache/fontconfig
... /home/darth_vader/snap/authy/common/.cache/fontconfig
... /home/darth_vader/snap/icedrive/common/.cache/fontconfig
... /home/darth_vader/snap/discord/common/.cache/fontconfig
... /home/darth_vader/snap/bitwarden/common/.cache/fontconfig
```

Then either remove them individually or use this simple loop.

Finally, Restart your session.

Error: cannot mount squashfs

Snap packages use the SquashFS file system. In the event of an error similar to the following:

```
error: system does not fully support snapd: cannot mount squashfs image using
"squashfs"
```

you may verify that the SquashFS kernel module is loaded with

```
$ lsmod
Module                  Size  Used by
squashfs               xxxxx  x
...
```

提示：If you have recently installed the snapd package to your system, you may need to reboot Arch Linux before installing Snap packages.

Error: /user.slice/user-1000.slice/session-1.scope is not a snap cgroup

You need to set your **DBUS_SESSION_BUS_ADDRESS** environment variable like so:

```
export DBUS_SESSION_BUS_ADDRESS="unix:path=$XDG_RUNTIME_DIR/bus"
```

To make this change permanent and also available in your GUI session, consider adding this line to your **~/.xprofile** file.

For more information and full discussion about this issue, please read here

Graphical management

**本文或本章节的事实准确性存在争议。**

原因： snap plugin for gnome-software包 is not built by default（在 Talk:Snap#Snap support in Gnome Software 3.38 中讨论）

Both Gnome Software Center and KDE Discover can provide native snap support. For KDE Discover install discover-snapAUR package.

The Snap Store can be installed via snap

```
# snap install snap-store
```

Support

Arch Linux related mailing lists and other official Arch Linux support channels are not an appropriate place to request help with snaps on Arch Linux. An appropriate place to ask for support is the Snapcraft forum.

See also

- Official site
- GitHub repository
- ArsTechnica article (2016-06) about Ubuntu snaps becoming available for Arch and other distributions

2.13 Flatpak

难度 高级 | **篇幅** 26.8 KB | **所属** 开发

本节将讲到

- 安装
- 管理仓库
- 管理运行时和应用
- 创建自定义基本运行时
- 排障

引自项目 README：“Flatpak 是一个 Linux 桌面程序的构建、分发和沙箱化运行系统。”

引自 flatpak(1)：

: Flatpak 是一个用来管理应用和应用所使用的运行时的工具。在 Flatpak 模型中，应用的构建和分发不依赖其主系统，并且它们在运行时一定程度上独立于主系统（'沙箱化'）。 : Flatpak 使用 OSTree 以分发和部署数据。它使用的仓库是 OSTree 仓库并且可以用 ostree 的工具来操作，已安装的运行时和应用都是 OSTree 检出。

警告：

- flathub 上的很多 Flatpak 应用默认并未有效地沙箱化。在未检查 flatpak 相关权限清单是否有沙箱逃逸问题前，请勿依赖提供的检查隔离功能。
- 运行不受信任的软件永不安全，沙箱化也无法改变这一点。

安装

安装 flatpak包 软件包。如果需要构建 flatpak，请一并安装 `flatpak-builder`。

桌面环境整合

对于与桌面环境交互的 flatpak 应用（例如使用应用打开 URL、共享屏幕等），先确保已配置了 XDG 桌面门户。取决于桌面环境的实现不同，在应用调用功能前会出现一个确认窗口。

应用管理

- **Discover** — Flatpak 的 KDE 前端，可用于搜索和安装应用、游戏和工具，是 plasma包组 的一部分。

: https://apps.kde.org/discover/ || discover包

- **GNOME Software** — Flatpak 的 GNOME 前端，可用于安装和更新应用及系统扩展，是 gnome包组 的一部分。

: https://apps.gnome.org/Software/ || gnome-software包

权限管理

- **Flatpak 权限管理 KCM** — KDE 配置模块，可以修改已安装 Flatpak 应用的权限，是 plasma包组 的一部分。

: https://invent.kde.org/plasma/flatpak-kcm || flatpak-kcm包

- **Flatseal** — 用于检查并修改 Flatpak 应用权限的图形化工具。

: https://github.com/tchx84/Flatseal || flatsealAUR

- **malcontent（家长控制）** — 实现了控制非管理员账户能访问的内容类型的功能，是 gnome包组 的一部分。

: https://gitlab.freedesktop.org/pwithnall/malcontent || malcontent包

管理仓库

注意：默认情况下，每个 flatpak 命令都是全局可用的，具体来说，软件包是安装给当前机器的所有用户的，并且要求用户提供 root 密码。如果想针对单个用户安装软件包或操作仓库，你可以添给每个命令加 `--user` 参数，此时不需要提供超级权限。例如，你想添加一个仅自己可见的仓库，你应该执行 `flatpak remote-add --user *name* *location*`。

添加一个仓库

要添加一个远程 flatpak 仓库，执行：

```
$ flatpak remote-add name location
```

这其中 *name* 是新远程仓库的名字，而 *location* 是仓库的路径或 URL。

安装 flatpak 时默认会全局添加官方 Flathub 仓库。如果要添加用户独立的官方仓库，可以执行：

```
$ flatpak remote-add --if-not-exists --user flathub
https://dl.flathub.org/repo/flathub.flatpakrepo
```

官方仓库的镜像站

对于中国大陆用户，有上海交通大学的 Flathub 缓存可供选择。

使用方法：

```
sudo flatpak remote-modify flathub --url=https://mirror.sjtu.edu.cn/flathub
```

如果出现错误可尝试：

```
wget https://mirror.sjtu.edu.cn/flathub/flathub.gpg
sudo flatpak remote-modify --gpg-import=flathub.gpg flathub
```

注意：由于重分发授权问题，NVIDIA 驱动、JetBrains 系列等软件需要从官方服务器下载，无法使用镜像站加速。

删除一个仓库

要删除一个远程 flatpak 仓库，执行：

```
$ flatpak remote-delete name
```

这其中 *name* 是要删除的远程仓库的名字。

仓库列表

要列出所有已添加的仓库列表，执行：

```
$ flatpak remotes
```

管理运行时和应用

注意：如果仓库是用户独立的，可以在 flatpak 命令添加 `--user` 选项。例如，如果要安装一个只有你可见的应用，可以执行 `flatpak install --user *软件包名称*`。

搜索远程仓库的运行时和应用

在能搜索新添加的远程仓库中的运行时和应用之前，我们需要获取其软件应用流的数据：

```
$ flatpak update
Looking for updates...
Updating appstream data for remote name
```

之后我们可以通过 `flatpak search *packagename*` 命令来实现搜索，例如在配置完成的远程仓库 `flathub` 中搜索 `libreoffice` 软件包：

```
$ flatpak search libreoffice
Application ID              Version Branch Remotes Description
org.libreoffice.LibreOffice         stable flathub The LibreOffice productivity suite
```

列出所有可用的运行时和应用

要列出远程仓库 *remote* 中所有可用的运行时和应用，执行：

```
$ flatpak remote-ls remote
```

安装一个运行时或应用

要安装一个运行时或应用，执行：

```
$ flatpak install remote name
```

这其中 *remote* 是远程仓库的名字，而 *name* 是待安装的应用或运行时的名字。

提示：您可以使用部分标识符 `flatpak install *partial-name*` (例如 `flatpak install libreoffice`)。

列出所有已安装的运行时和应用

要列出所有已安装的运行时和应用，执行：

```
$ flatpak list
```

运行应用

二进制文件位于 `/var/lib/flatpak/exports/bin` 目录下，已自动通过 `/etc/profile.d/flatpak-bindir.sh` 添加到 $PATH 中，可能需要重新登录才会应用更改。

Flatpak 应用也可以通过命令行运行：

```
$ flatpak run name
```

升级一个运行时或应用

列出所有可更新的运行时和应用：

```
$ flatpak remote-ls --updates
```

要升级一个名为 *name* 的运行时或应用，执行：

```
$ flatpak update name
```

要更新所有应用和运行时，执行：

```
$ flatpak update
```

通过 systemd 自动更新

警告：不建议通过 systemd 托管更新，应用可能会在用户不知道的情况下获取到新权限，相关案例可参考该文。

要自动更新运行时和应用，先创建以下文件：

```
/etc/systemd/system/flatpak-update.service
[Unit]
Description=Update Flatpak
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
ExecStart=/usr/bin/flatpak update --noninteractive --assumeyes

[Install]
WantedBy=multi-user.target
/etc/systemd/system/flatpak-update.timer
[Unit]
Description=Update Flatpak

[Timer]
OnBootSec=2m
OnActiveSec=2m
OnUnitInactiveSec=24h
OnUnitActiveSec=24h
AccuracySec=1h
RandomizedDelaySec=10m

[Install]
WantedBy=timers.target
```

然后执行 daemon-reload，并启用/启动 `flatpak-update.timer` 单元。

注意：

- 该部分适用于默认下的系统全局 flatpak 环境。对于用户独立的 flatpak 环境：
- 在 `/etc/systemd/user/` 目录下创建以上文件
- 在 `flatpak-update.service` 中的 `ExecStart` 一行添加 `--user` 选项
- 搭配 `--user` 选项执行 `systemctl` 命令

卸载一个运行时或应用

要卸载一个名为 *name* 运行时或应用，执行：

```
$ flatpak uninstall name
```

要在卸载时从 `~/.var/app` 清除应用数据，并清除权限记录，执行：

```
$ flatpak uninstall --delete-data name
```

提示：你可以通过 `flatpak uninstall --unused` 卸载不再使用的 flatpak 依赖（即无应用/运行时依赖的孤立包）。

降级一个运行时或应用

要降级一个运行时或应用，需要先获取到对应的提交（commit）ID：

```
$ flatpak remote-info --log remote name
```

其中 *remote* 是仓库名称（例如 flathub），*name* 是应用或运行时的名称。接下来是部署提交：

```
$ flatpak update --commit=commit name
```

其中 *commit* 是目标版本对应的的提交，*name* 与上文一致。

该操作也可以将软件包选择性地升级到非最新的一个版本。

要禁止 `flatpak update` 升级该软件包，请参考#禁止运行时或应用更新。

禁止运行时或应用更新

要禁止自动或手动升级运行时或软件包，请使用 `flatpak mask` 命令：

```
$ flatpak mask name
```

这也同时会禁止选择性升级和降级。

如果要重新启用升级功能，请使用 `flatpak mask --remove`：

```
$ flatpak mask --remove name
```

添加 Flatpak .desktop 文件到您的菜单

Flatpak 期望窗口管理器按照 XDG_DATA_DIRS 环境变量来查找应用，该变量通过 `/etc/profile.d/flatpak.sh` 脚本进行配置。在更新系统环境后，有可能需要重新登录系统。如果启动器不支持 `XDG_DATA_DIRS`，可以编辑目录扫描列表并添加以下两个文件夹：

```
~/.local/share/flatpak/exports/share/applications
/var/lib/flatpak/exports/share/applications
```

目前已知此方法在 Awesome 中是必须的。

覆盖应用的沙箱权限

如果你发现应用的预定义权限过松或过紧，你可以使用 `flatpak override` 命令进行修改。

例如：

```
$ flatpak override --nofilesystem=home name
```

这会禁止应用访问你的用户主目录。

所有可以赋予权限的类型，如设备、文件系统或 socket ，都有对应的命令行选项以允许或禁止确切的权限。例如，可以使用 `--device=*device_name*` 允许访问特定设备，而 `--nodevice=*device_name*` 会禁用设备访问权限。

对于所有可操作的权限类型，详见：flatpak-override(1)

下面的命令可将所有更改过的权限重设为默认：

```
$ flatpak override --reset name
```

如果更偏好图形界面，可以使用 *Flatseal*；在 KDE Plasma 上，*Flatpak 权限管理 KCM* 在系统设置中提供了类似的图形界面：*System Settings* > *Applications* > *Flatpak Permission Settings*

创建自定义基本运行时

警告：如果要将你的应用作为 Flatpak 发布，不要使用基于 Arch 的运行时。请参考官方文档使用通用运行时将你的软件正确地整合到 Flatpak 生态中。

注意：

- 鉴于在创建应用和运行时期间软件并没有沙箱化，你可能会想使用一个单独的非受信、非特权用户对不信任的软件进行打包。
- 在分发软件包时，你可能会有法律义务提供一些包内软件的源码。你可能会想使用 ABS 从源码构建软件包。

除了使用默认的 `org.freedesktop.BasePlatform` 和 `org.freedesktop.BaseSdk` 运行时外，你还可以在自用的情况下，通过 pacman 为 Flatpak 创建一个基于 Arch 的自定义运行时和基本 SDK，然后用其构建并打包软件。

除 flatpak包 外，你还需要安装 fakeroot包，对于 pacman 钩子支持还需安装 fakechroot包。

首先创建一个目录，用于构建运行时或应用：

```
$ mkdir myflatpakbuilddir
$ cd myflatpakbuilddir
```

然后就可以准备一个用于构建运行时基本平台的目录，其中的文件将被打包到沙箱中的 `/usr` 目录。因此，你会需要创建一些软链接，以使 Arch 下的 `/usr/share` 等目录仍可通过正常路径访问：

```
$ mkdir -p myruntime/files/var/lib/pacman
$ touch myruntime/files/.ref
$ ln -s /usr/usr/share myruntime/files/share
$ ln -s /usr/usr/include myruntime/files/include
$ ln -s /usr/usr/local myruntime/files/local
```

使主机下的字体在 Arch 运行时中可用：

```
$ mkdir -p myruntime/files/usr/share/fonts
$ ln -s /run/host/fonts myruntime/files/usr/share/fonts/flatpakhostfonts
```

在安装软件包到运行时前，需要修改一下你的 `pacman.conf` 文件。

将 `/etc/pacman.conf` 复制到构建目录中，然后进行以下修改：

- 移除 `CheckSpace` 选项，以使 pacman 不会因检查硬盘空间产生找不到根文件系统的报错。
- 移除不需要的自定义仓库，以及只有主机需要的 `IgnorePkg`，`IgnoreGroup`，`NoUpgrade` 和 `NoExtract` 设置。

接下来为运行时安装软件包：

```
$ fakechroot fakeroot pacman -Syu --root myruntime/files --dbpath
myruntime/files/var/lib/pacman --config pacman.conf base
$ mv pacman.conf myruntime/files/etc/pacman.conf
```

编辑 `*myruntime*/files/etc/locale.gen`，配置需要使用的 locales，然后生成 locale：

```
$ fakechroot chroot myruntime/files locale-gen
```

可以通过为基础运行时添加构建软件包和运行 pacman 所需的应用来创建基础 SDK：

```
$ cp -r myruntime mysdk
$ fakechroot fakeroot pacman -S --root mysdk/files --dbpath mysdk/files/var/lib/pacman
--config mysdk/files/etc/pacman.conf base-devel fakeroot fakechroot --needed
```

添加运行时和 SDK 的元数据：

```
myruntime/metadata
[Runtime]
name=org.mydomain.BasePlatform
runtime=org.mydomain.BasePlatform/x86_64/2016-06-26
sdk=org.mydomain.BaseSdk/x86_64/2016-06-26
mysdk/metadata
[Runtime]
name=org.mydomain.BaseSdk
runtime=org.mydomain.BasePlatform/x86_64/2016-06-26
sdk=org.mydomain.BaseSdk/x86_64/2016-06-26
```

将基础运行时和 SDK 添加到位于当前目录的本地仓库，你还可以为仓库添加例如“My Arch base runtime”和“My Arch base SDK”等提交（commit）信息：

```
$ ostree init --mode archive-z2 --repo=.
$ EDITOR="nano -w" ostree commit -b runtime/org.mydomain.BasePlatform/x86_64/2016-06-26
--tree=dir=myruntime
$ EDITOR="nano -w" ostree commit -b runtime/org.mydomain.BaseSdk/x86_64/2016-06-26 --
tree=dir=mysdk
$ ostree summary -u
```

安装运行时和 SDK：

```
$ flatpak remote-add --user --no-gpg-verify myarchos file://$(pwd)
$ flatpak install --user myarchos org.mydomain.BasePlatform 2016-06-26
$ flatpak install --user myarchos org.mydomain.BaseSdk 2016-06-26
```

使用 pacman 创建应用

除了构建应用的通用方式外，我们还可以创建通常 Arch 软件包的容器化版本。注意，在创建应用时 `/usr` 是只读的，因此在构建应用时无法使用 Arch 软件包。有两种方式可以通过 pacman 创建软件：

- 使用 pacman 创建包含所有依赖的运行时

- 然后根据正常方式自行构建应用，或是将 pacman 搭配为 Flatpak 定制的 PKGBUILD，其中在 `configure` 脚本中使用了 `--prefix=/app`。

或是

- 使用 pacman 创建一个运行时，其中包含通过 pacman 安装完成的应用
- 然后使用一个虚拟应用进行启动

对于后一种方式，以 gedit包 为例，首先通过 pacman 创建一个运行时，该运行时已为 pacman 进行了初始化和准备：

```
$ flatpak build-init -w geditruntime org.mydomain.geditruntime org.mydomain.BaseSdk
org.mydomain.BasePlatform 2016-06-26
$ flatpak build geditruntime sed -i "s/^#Server/Server/g" /etc/pacman.d/mirrorlist
$ flatpak build geditruntime ln -s /usr/var/lib /var/lib
$ flatpak build geditruntime fakeroot pacman-key --init
$ flatpak build geditruntime fakeroot pacman-key --populate
```

然后安装软件包，这一步需要联网：

```
$ flatpak build --share=network geditruntime fakechroot fakeroot pacman --root /usr -S
gedit
```

可以在完成运行时前测试一下（此时还没有完全沙箱化）：

```
$ flatpak build --socket=x11 geditruntime gedit
```

接下来完成运行时，并将其导出到新的本地仓库中。pacman 的 GnuPG 带有权限，可能会产生些问题，因此需要先将其移除：

```
$ flatpak build geditruntime rm -r /etc/pacman.d/gnupg
$ flatpak build-finish geditruntime
$ sed -i "s/\[Application\]/
\[Runtime\]/;s/runtime=org.mydomain.BasePlatform/runtime=org.mydomain.geditruntime/"
geditruntime/metadata
$ flatpak build-export -r geditrepo geditruntime
```

然后创建一个虚拟应用：

```
$ flatpak build-init geditapp org.gnome.gedit org.mydomain.BaseSdk
org.mydomain.geditruntime
```

接下来完成虚拟应用，你可以在完成应用时追加额外选项来调整应用权限，具体可用选项请参考 Flatpak 文档和 GNOME manifest files。你还可以在完成构建后和导出前按需修改 `geditapp/metadata`。在完成元数据修改后，将应用导出到仓库：

```
$ flatpak build-finish geditapp --socket=x11 [possibly other options] --command=gedit
$ flatpak build-export geditrepo geditapp
```

安装应用和运行时：

```
$ flatpak --user remote-add --no-gpg-verify geditrepo geditrepo
$ flatpak install --user geditrepo org.mydomain.geditruntime
$ flatpak install --user geditrepo org.gnome.gedit
$ flatpak run org.gnome.gedit
```

排障

Flatpak 无法在 linux-hardened 内核上运行

linux-hardened包 内核将 `kernel.unprivileged_userns_clone` 设为 `0`，使得只有特权用户才能创建新的用户命名空间。

其中一个解决方案是安装 bubblewrap-suid包，该包提供了启用 `setuid` 的 bwrap(1)，使得 bubblewrap 可以自行提升权限并创建新命名空间。

或者，也可以通过 sysctl(8) 将 `kernel.unprivileged_userns_clone` 设为 `1`，以允许非特权用户创建新的用户命名空间。

```
# sysctl kernel.unprivileged_userns_clone=1
```

警告：该操作将给安全性带来负面影响，具体信息请参考安全#沙盒程序。

要将该更改持久化，可以在 sysctl.d(5) 中添加一个配置文件：

```
/etc/sysctl.d/flatpak.conf
kernel.unprivileged_userns_clone=1
```

更多信息请参考 Bubblewrap#安装。

Failed to connect to Wayland display

如果应用未能正常启动，并在使用 `flatpak run` 时出现类似如下报错：

```
Failed to connect to Wayland display: No such file or directory
```

可能是因为像 `ELECTRON_OZONE_PLATFORM_HINT="auto"` 等选项使 Flatpak 应用调用了 Wayland，同时又未将 Wayland 权限授予应用导致的。

可以使用 Flatseal 等工具将 `socket=wayland` 的访问添加到白名单解决。

xdg-desktop-portal 无法启动

If you are starting X with manually-configured run commands, ensure you are including all essential components of the reference \xinitrc\. One of which sources a script which runs an update of the environment used for D-Bus session services.

```
systemctl --user import-environment DISPLAY XAUTHORITY
if command -v dbus-update-activation-environment >/dev/null 2>&1; then
   dbus-update-activation-environment DISPLAY XAUTHORITY
fi
```

Flatpak 应用没有使用系统默认主题

根据 flatpak 文档，当前不存在理想的为 flatpak 软件应用系统主题的方法 。最简单的方式是使用 Flathub 中可用的主题。然而，有办法可以将主题应用到 flatpak 软件中，该方法的自动化版本为 stylepak-gitAUR。

Flatpak 应用没有使用系统安装的 Fcitx5 皮肤

如果系统从 X11 会话切换至 Wayland 会话，由于环境变量 `QT_IM_MODULE` 和 `GTK_IM_MODULE` 仍被设置为 `fcitx`，可能导致 Flatpak 应用无法加载 Fcitx 5 的主题皮肤。

解决方法：通过 `flatpak override` 命令为 Flatpak 应用取消设置这两个环境变量。

```
$ flatpak override --env=QT_IM_MODULE= --user
$ flatpak override --env=GTK_IM_MODULE= --user
```

若要应用于全局系统环境，请去掉 `--user` 参数并使用 root 权限执行：

```
# flatpak override --env=QT_IM_MODULE=
# flatpak override --env=GTK_IM_MODULE=
```

执行完毕后，需关闭所有 Flatpak 应用并重新打开以使更改生效。

详见 Github 相关讨论：Theme not being applied when used in Flatpak applications

"File not found" error when Open local HTML pages in Firefox

By default, the Flatpak version of Firefox will display a "File not found" error page when opening a local HTML. This is because permission must be granted to the app for accessing the folder containing the file.

However, note that when granting permission to access the entire Home folder, Firefox will then check for an existing profile in `~/.mozilla` and load it instead of those previously in use from the sandboxed folder `~/.var/app/org.mozilla.firefox/cache/mozilla/`. If your previous session's tabs and browsing history is missing after changing a permission (e.g. with Flatseal), either modify the permission to exclude access to `~/.mozilla`, or consider copying the profile from `~/.var/app/org.mozilla.firefox/cache/mozilla/` to `~/.mozilla`.

Links fail to open on wlroots-based compositors

Flatpak applications that attempt to open URIs make use of the `org.freedesktop.portal.OpenURI.OpenURI` D-Bus interface exposed by xdg-desktop-portal. The xdg-desktop-portal-wlr backend does not support this call and therefore you will need an additional backend to fill the gap, for example xdg-desktop-portal-gtk包.

Applications do not use the correct cursor theme

There is no single standard to set the cursor properly. Some programs only need read access to the cursors directory, others also rely on other mechanisms. For GTK applications, ensure that xdg-desktop-portal-gtk包 is installed.

Otherwise, the following overrides should work for most common desktop applications.

```
$ flatpak -u override --filesystem=/usr/share/icons/:ro
$ flatpak -u override --filesystem=/home/$USER/.icons/:ro
$ flatpak -u override --filesystem=xdg-config/gtk-3.0:ro
$ flatpak -u override --env=XCURSOR_PATH=~/.icons
```

In some cases you may also need to override the environment variables `XCURSOR_THEME` and `XCURSOR_SIZE`:

```
$ flatpak -u override --env=XCURSOR_THEME=Adwaita
$ flatpak -u override --env=XCURSOR_SIZE=24
```

See this discussion for additional details.

Apparently it is not possible anymore to enable access to applications to directories under `/usr/`. The following hints at this when launching a program:

```
$ flatpak run com.spotify.Client
F: Not sharing "/usr/share/icons" with sandbox: Path "/usr" is reserved by Flatpak
```

One possible workaround would be to manually copy your icon theme from `/usr/share/icons` to `/home/$USER/.icons/`.

Flatpak Qt applications do not use Gnome Adwaita dark theme

If you switched your theme to Adwaita-dark and Flatpak Qt applications still use the light version, install the required KStyle:

```
# flatpak install flathub org.kde.KStyle.Adwaita
```

Permission denied error when running Flatpak applications

Flatpak applications will not run if the mount point that contains the folder in which the application is stored, typically `/var/lib/flatpak/` for system wide installations, and `~/.local/share/flatpak/` for user-specific installations, is mounted with the `noexec` option.

With `noexec` set you will get errors such as this:

```
$ bwrap: execvp ldconfig: Permission denied
$ error: ldconfig failed, exit status 256
```

参阅

- 官方网站
- 官方 Github 维基
- 维基百科页面
- Gnome SandboxedApps
- KDE 测试运行时及应用

2.14 GPGPU

难度 高级 | **篇幅** 27.6 KB | **所属** 开发

本节将讲到

- OpenCL
- SYCL
- CUDA

- ROCm
- 支持GPGPU加速的软件

通用图形处理单元计算（GPGPU）API定义包括厂商无关的OpenCL、SYCL、HIP、OpenMP和Vulkan计算着色器;以及Nvidia的CUDA。每个API可以有多种硬件或软件的实现：GPU、CPU、NPU、FPGA，或者仅仅是不同的GPGPU API（shim/transpiler）。

OpenCL

OpenCL（开放计算语言）是由非营利联盟Khronos Group开发的一种开放、免版税的并行编程规范。

OpenCL 规范描述了一种基于 C 的编程语言、一个必须存在的通用环境，以及一个 C API，使程序员能够调用该环境。还有绑定允许其他语言调用该环境，以及编写 OpenCL 计算内核的替代语言。

提示：clinfo 工具可用于列出 OpenCL 平台、设备及 ICD 加载器属性。

OpenCL 环境至少包含以下一种：

- *libOpenCL.so* 的副本，ICD加载器呈现完整的OpenCL API接口。
- 由ICD加载器加载的平台相关驱动程序。

ICD加载器（libOpenCL.so）

系统通常拥有多个支持 OpenCL 的硬件，因此存在多个 OpenCL 运行时实现。一个名为“OpenCL ICD loader”的组件应是一个平台无关的库，通过 OpenCL API 提供加载设备特定驱动程序的手段。 大多数 OpenCL 厂商都提供自己的 OpenCL ICD 加载器实现，这些都能与其他厂商的 OpenCL 实现兼容。遗憾的是，大多数厂商并未提供完全最新的ICD加载器，因此Arch Linux决定从一个独立项目ocl-icd包中提供该库，该项目提供了当前OpenCL API的可运行实现。截至2025年，当前的OpenCL版本为3.0。

其他ICD加载器库则作为各厂商SDK的一部分安装。如果你想确保使用 ocl-icd包 中的 ICD 加载器，可以在 `/etc/ld.so.conf.d` 创建一个文件，为动态程序加载器的搜索目录添加`/usr/lib`：

```
/etc/ld.so.conf.d/00-usrlib.conf
/usr/lib
```

这是必要的，因为所有SDK都会把运行时的库目录添加到`ld.so.conf.d`文件的搜索路径中。

包含各种 OpenCL ICD 的可用软件包有：

- ocl-icd包：推荐，最新
- intel-compute-runtime包：Intel提供的OpenCL实现。

注意：ICD 加载器的供应商在功能上无关紧要：它们应当与厂商无关，只要实现正确，就可以互换使用。供应商的作用仅用于区分它们之间的区别。

管理实现

要查看当前系统中哪些OpenCL实现正在运行，请使用以下命令：

ls /etc/OpenCL/vendors

要了解 OpenCL 平台及系统上可用设备的所有可能（已知）属性，请安装 clinfo包。

你可以用ocl-icd-choose指定应用应接收哪些实现。例如：

ocl-icd-choose amdocl64.icd:mesa.icd davinci-resolve-checker

`ocl-icd-choose` 是一个简单的包装脚本，用于设置`OCL_ICD_FILENAMES`环境变量。如果`OCL_ICD_VENDORS`能像说明书建议的那样处理单个ICD文件，那就不需要它了;这是一个已知的bug，被追踪为OCL-dev/ocl-icd#7。

运行时

要**执行**使用 OpenCL 的程序，需要安装一个可由 *libOpenCL.so* 加载的运行时。

通用GPU上的OpenCL

对于Mesa支持*的任何*显卡，你可以通过安装opencl-mesa包来使用rusticl。可以通过使用环境变量 `RUSTICL_ENABLE=*driver*` 来启用，其中 `*driver*` 是 Gallium 驱动程序，如 `radeonsi` 或 `iris`。它能在最广泛的硬件上运行，但不一定能提供最佳性能。如果你在设置其他驱动时遇到困难，可以试试用它：启用只需要非常少的配置。

OpenCL 在 AMD/ATI GPU 上

- rocm-opencl-runtime包：AMD ROCm GPU 计算栈的一部分，官方支持少量 GPU 型号（其他显卡可能支持非官方或部分支持）。要支持比Vega更早的显卡，你需要设置运行时变量`ROC_ENABLE_PRE_VEGA=1`。这与在 Ubuntu 的 amdgpu-install 中指定 `opencl=rocr` 类似，但不完全等同，因为该包的 ROCM 版本与 Ubuntu 安装程序版本不同。
- opencl-legacy-amdgpu-proAUR：从AMD Ubuntu版本重新包装的Legacy Orca OpenCL。相当于在 Ubuntu 的 amdgpu-install 中指定 `opencl=legacy`。

- opencl-amdAUR、opencl-amd-devAUR：ROCm组件从AMD的Ubuntu版本重新打包而来。相当于在 Ubuntu 的 amdgpu-install 中指定 `opencl=rocr,legacy`。

OpenCL 图像支持

最新的ROCm版本现已包含用于GPGPU加速软件如Darktable的OpenCL图像支持。配备AMDGPU开源图形驱动的ROCm仅需完成。AMDGPU PRO不是必需的。

```
/opt/rocm/bin/clinfo | grep -i "image support"
Image support     Yes
```

NVIDIA

- opencl-nvidia包：官方NVIDIA运行库

Intel

- intel-compute-runtime包：也被称为Neo OpenCL运行库，开源实现，支持Intel HD Graphics GPU第八代(Broadwell)及更新的硬件。
- opencl-clover-mesa包或opencl-rusticl-mesa包：mesa驱动通过clover/rusticl提供的OpenCL支持
- beignetAUR：Intel HD Graphics GPU第七代(Ivy Bridge)及更新硬件的开源支持，因被NEO OpenCL替代而被弃用，依然是传统硬件平台的推荐解决方案（如Ivy Bridge，Haswell）
- intel-openclAUR：Intel HD Graphics GPU第七代(Ivy Bridge)及更新硬件的闭源支持，因被NEO OpenCL替代而被弃用，依然是传统硬件平台的推荐解决方案（如Ivy Bridge，Haswell）
- intel-opencl-runtimeAUR：Intel酷睿及志强处理器的实现，同样支持非Intel CPU

Others

- pocl包：基于LLVM的OpenCL实现（与硬件无关）

还有一些编译器和翻译器能够让OpenCL应用在Vulkan上运行。

- clspv-gitAUR：Clspv是一个OpenCL子集到Vulkan计算着色器的原型编译器
- clvk-gitAUR：clvk是一个在Vulkan上的OpenCL 3.0原型实现，使用clspy作为编译器
- xrt-binAUR：Xilinx FPGA xrt运行库
- fpga-runtime-for-opencl:FPGA运行库

32位运行库

要运行使用OpenCL的32位程序，需要安装一个兼容硬件的32位运行库。

提示：clinfo包工具只能显示64位OpenCL平台，设备和ICD加载器属性。要用于32位，你需要编译32位clinfo或使用archlinux32项目中的 clinfo。

AMD/ATI

- lib32-opencl-clover-mesa包 或 lib32-opencl-rusticl-mesa包：AMD/ATI Radeon mesa 32位驱动的OpenCL支持

NVIDIA

- lib32-opencl-nvidia包：NVIDIA（32位）的OpenCL实现

ICD加载器 (libOpenCL.so)

OpenCL ICD加载器应该是一个平台无关的库，提供了通过OpenCL API加载设备相关驱动的方法。 大多数OpenCL供应商提供了自己的OpenCL ICD加载器实现，这些加载器也应该全部与其它供应商的实现兼容。但不幸的是，大多供应商没有提供完全最新的ICD加载器，因此Arch Linux决定提供一个来自独立项目的加载器库(ocl-icd包)，该库目前提供了当前OpenCL API的实现。

其它ICD加载器库文件安装在各个供应商的SDK中。如果你想确保使用ocl-icd包的ICD加载器，你可以在`/etc/ld.so.conf.d` 创建配置文件，添加`/usr/lib`到动态程序加载器的搜索目录中：

```
/etc/ld.so.conf.d/00-usrlib.conf
/usr/lib
```

因为所有SKD通过`ld.so.conf.d`添加自己的运行库库目录到搜索路径。

包含各种OpenCL ICD的包有：

- ocl-icd包：推荐，几乎是最新的
- intel-openclAUR：由Intel提供的OpenCL 2.0，因被intel-compute-runtime包替代而废弃

注意：上文提及的ICD加载器的供应商仅用于区分各个供应商提供的加载器，除此之外没有任何关联。ICD加载器是与供应商无关的，也应该是可以相互替换的（只要这些加载器的实现是正确的）。

开发

要**开发**OpenCL，需要最小的额外包：

- ocl-icd包：OpenCL ICD加载器实现，符合最新OpenCL标准
- opencl-headers包：OpenCL C/C++ API头文件

供应商提供了大量工具和支持库文件：

- intel-opencl-sdkAUR：Intel OpenCL SDK（旧版，新版OpenCL SDK包含在INDE 和Intel Media Server Studio中）
- amdapp-sdkAUR：安装在`/opt/AMDAPP`，除了SKD文件，还包含一一些代码示例（`/opt/AMDAPP/SDK/samples/`）。还提供了`clinfo`工具，用于显示系统中的OpenCL平台和设备及其详细信息。SDK还附带了OpenCL CPU驱动，无需其它驱动即可在CPU设备执行OpenCL（不论CPU的供应商）
- cuda包：Nvidia的GPU SDK包含了OpenCL 3.0的支持

实现

使用下列命令查看当前你的系统上的OpenCL实现：

```
$ ls /etc/OpenCL/vendors
```

要确认系统可用的OpenCL平台和设备上所有可能（已知）的特性，安装clinfo包。

你可以指定你的程序使用特定实现，通过ocl-icd-chooseAUR完成：

```
$ ocl-icd-choose amdocl64.icd:mesa.icd davinci-resolve-checker
```

Rusticl

**本文或本章节的事实准确性存在争议。**

原因： 可能不能为OCL_ICD_VENDORS指定单个icd文件，尽管手册这样说。参见 https://github.com/OCL-dev/ocl-icd/issues/7#issuecomment-1522941979（在 Talk:GPGPU 中讨论）

Rusticl是opencl-rusticl-mesa包提供的一个由Rust编写的新的OpenCL实现。可以通过设置环境变量`RUSTICL_ENABLE=*driver*`启用该实现，命令中的`*driver*`是Gallium驱动，如`iris`或`radeonsi`。

另外，如果OpenCL应用无法检测到Rusticl，使用以下的环境变量：

```
OCL_ICD_VENDORS=/etc/OpenCL/vendors/rusticl.icd
```

语言绑定

- **JavaScript/HTML5**: WebCL
- Python: python-pyopencl包
- D: cl4d or DCompute
- Java: Aparapi or JOCL (a part of JogAmp)
- Mono/.NET: Open Toolkit
- Go: OpenCL bindings for Go
- **Racket**: Racket has a native interface on PLaneT that can be installed via raco.
- Rust: ocl
- Julia: OpenCL.jl

SYCL

根据Wikipedia:SYCL：

: SYCL 是一种用于在各种硬件加速器上提高编程生产力的高级编程语言模型。这是一种单源特定于域的嵌入式语言，基于C++17。

: SYCL 是一个免版税的跨平台抽象层，它建立在受 OpenCL 启发的底层概念、可移植性和效率之上，它使异构处理器的代码能够使用完全标准的C++以“单一来源”风格编写。SYCL 支持单源开发，其中C++模板函数可以包含主机和设备代码，以构建使用硬件加速器的复杂算法，然后在不同类型的数据上始终使用这些源代码。

: 虽然 SYCL 标准最初是 OpenCL 工作组的高级编程模型子组，最初是为与 OpenCL 和 SPIR 一起使用而开发的，自2019年9月20日，SYCL从 OpenCL工作中独立，成为一个Khronos Group 工作组，从 SYCL 2020 开始，SYCL 已被推广为能够针对其他系统的更通用的异构架构。现在可以通过一个通用后端，针对任何加速 API，同时实现 API 的完全互操作性，比如简化编程工作的同时，用已有的本地库来达到最大性能。例如，Open SYCL 通过 AMD 的 HIP 来利用 ROCm 和 CUDA。

实现

- computecppAUR：Codeplay的私有SYCL 1.2.1实现。支持SPIR，SPIR-V和实验性支持PTX (NVIDIA)。（2023年9月1日结束支持，将并入intel llvm Source）。
- trisycl-gitAUR：Xilinx的开源实现。
- hipsycl-cuda-gitAUR和hipsycl-rocm-gitAUR：基于AMD HIP而不是OpenCL构建的开放实现，能够在AMD和NVIDIA GPU上运行。
- intel-oneapi-dpcpp-cpp包: Intel的数据并行C++：oneAPI的SYCL实现。

检查SPIR支持

大多数SYCL实现能够编译加速器代码到SPIR或SPIR-V。两者都是由Khronos设计的中间语言，并被OpenCL驱动使用。要检查是否支持SPIR或SPIR-V，使用clinfo包：

```
$ clinfo | grep -i spir
Platform Extensions                                cl_khr_icd
cl_khr_global_int32_base_atomics cl_khr_global_int32_extended_atomics
cl_khr_local_int32_base_atomics cl_khr_local_int32_extended_atomics
cl_khr_byte_addressable_store cl_khr_depth_images cl_khr_3d_image_writes
cl_intel_exec_by_local_thread cl_khr_spir cl_khr_fp64 cl_khr_image2d_from_buffer
cl_intel_vec_len_hint
  IL version                                       SPIR-V_1.0
  SPIR versions                                    1.2
```

ComputeCpp也提供了一个显示相关系统信息的工具：

```
$ computecpp_info
Device 0:
```

```
  Device is supported                     : UNTESTED - Untested OS
  CL_DEVICE_NAME                          : Intel(R) Core(TM) i7-4770K CPU @ 3.50GHz
  CL_DEVICE_VENDOR                        : Intel(R) Corporation
  CL_DRIVER_VERSION                       : 18.1.0.0920
  CL_DEVICE_TYPE                          : CL_DEVICE_TYPE_CPU
```

已知至少部分支持SPIR或SPIR-V的驱动包括intel-compute-runtime包，intel-opencl-runtimeAUR，pocl包和amdgpu-pro-openclAUR\[损坏的链接：package not found\].

开发

SYCL需要C++11环境，以下是一些开源库：

- ComputeCpp SDK：包含示例代码，用于ComputeCpp的cmake包集成
- SYCL-DNN：神经网络性能基元
- SYCL-BLAS：线性代数性能基元
- VisionCpp：计算视觉库
- SYCL Parallel STL：C++17并行算法的GPU实现

CUDA

CUDA（Compute Unified Device Architecture，计算统一设备体系结构）是NVIDIA的私有闭源并行计算架构。需要NVIDIA GPU，和一些组件：

- 必须：
- 私有NVIDIA内核模块
- CUDA"驱动"和"运行时"库
- 可选：
- 额外的库: CUBLAS，CUFFT，CUSPARSE等
- CUDA工具包，包括`nvcc`编译器
- CUDA SDK包含了许多示例代码，CUDA和OpenCL示例程序

内核模块和CUDA"驱动"库包含在nvidia-open包和opencl-nvidia包中。"运行时"库及其它CUDA工具包在cuda包中。`cuda-gdb`需要安装ncurses5-compat-libsAUR，见FS#46598。

开发

cuda包中所有组件安装在`/opt/cuda`。在`/etc/profile.d/cuda.sh`里设置了相关环境变量，这样所有支持CUDA的构建系统都能找到CUDA。

要确认CUDA是否安装成功，可以编译CUDA示例代码。一种检查安装的方法是运行`deviceQuery`示例。 One way to check the installation is to run the `deviceQuery` sample.

语言绑定

- **Fortran**: PGI CUDA Fortran Compiler
- Haskell: accelerate package列出了可用的CUDA后端
- Java: JCuda
- Mathematica: CUDAlink
- Mono/.NET: CUDAfy.NET, managedCuda
- Perl: KappaCUDA, CUDA-Minimal
- Python: python-pycuda包
- Ruby: rbcuda
- Rust: cuda-sys (绑定) 或 RustaCUDA (高级封装)

ROCm

ROCm （Radeon开发计算平台）是AMD的开源并行计算架构。尽管其需要AMD GPU，一些ROCm工具却是硬件无关的。参见ROCm for Arch Linux repository。

- rocm-hip-sdk包：为AMD平台开发使用HIP的应用和库
- rocm-opencl-sdk包：为AMD平台开发基于OpenCL的应用

HIP

异构计算可移植接口（Heterogeneous Interface for Portability，HIP）是AMD的专用GPU编程环境，用于设计GPU硬件的高性能内核。HIP作为C++运行库API和编程语言，允许开发者为不同平台创建可移植应用。

- rocm-hip-runtime包：基本运行库，用于在AMD平台运行HIP应用
- hip-runtime-amd包：ROCm中用于AMDGPU的异构接口，支持polaris架构（RX 500系列）到RDNA 2架构（RX 6000系列）
- miopen-hip包: AMD开源深度学习库，使用HIP后端
- hip-runtime-nvidiaAUR：ROCm中用于NVIDIA GPU的异构接口

提示：关于Blender使用HIP的更多信息，参见这里。

OpenMP

openmp-extrasAUR提供了AOMP - 一个开源基于Clang/LLVM的编译器，提供OpenMP API在AMD GPU的支持。

OpenCL

rocm-opencl-runtime包是ROCm框架的一部分，提供了OpenCL运行库。

OpenCL图像支持

ROCm最新版本提供了OpenCL 图像支持，可用于GPGPU加速软件如Darktable。仅需AMDGPU开源驱动和ROCm，不需要AMDGPU PRO。

```
$ /opt/rocm/bin/clinfo | grep -i "image support"
Image support                                   Yes
```

疑难解答

首先确认你的GPU出现在`/opt/rocm/bin/rocminfo`的输出里。如果没有，这可能意味着ROCm不支持你的GPU或者ROCm软件在编译时未做相关配置。

PyTorch

要让PyTorch支持ROCm,安装python-pytorch-rocm包。

```
$ python -c 'import torch; print(torch.cuda.is_available())'
True
```

上面的命令如果没有返回`True`，那么python-pytorch-rocm可能在编译时没有支持你的GPU或者你的依赖存在冲突，你可以使用`ldd /usr/lib/libtorch.so`进行检查，输出结果应该不会有任何`.so`文件的丢失或多个不同版本的相同`.so`文件。

支持GPGPU加速的软件

- Bitcoin
- Blender – Nvidia GPU支持CUDA，AMD GPU支持HIP。详情参见这里
- BOINC
- FFmpeg – 详情参见这里
- Folding@home
- GIMP – 实验性支持 – 更多信息参见这里
- HandBrake
- Hashcat
- LibreOffice Calc – 更多信息参见这里.
- mpv - 参见 mpv#Hardware video acceleration.
- clinfo包 – 确认系统可用的OpenCL平台和设备上所有可能（已知）的特性
- cuda_memtestAUR – 一个GPU内存测试。尽管名字有cuda，它也支持OpenCL
- darktable包 – OpenCL特性需要至少1 GB现存和*图形支持*（使用clinfo命令检查）
- DaVinci Resolve - 非线性视频编辑器，同时支持OpenCL和CUDA
- imagemagick包
- lc0AUR - 用于搜索神经网络（支持tensorflow，OpenCL，CUDA和openblas）
- opencv包
- pyritAUR
- python-pytorch-cuda包 - CUDA后端的PyTorch
- tensorflow-cuda包 - TensorFlow的CUDA版本
- tensorflow-computecppAUR - TensorFlow的SYCL版本

- whisper.cpp-clblasAUR 和 whisper.cpp-cublasAUR - OpenAI Whisper 模型 的 C/C++ 移植(使用 OpenCL 和 CUDA 优化)
- xmrig包 - High Perf CryptoNote CPU and GPU (OpenCL, CUDA) miner

另见

- OpenCL官方主页
- SYCL官方主页
- SPIR官方主页
- CUDA工具包主页
- Intel SDK用于OpenCL应用
- ComputeCpp官方主页
- 不同GPU的OpenCL框架支持列表

Emacs

Emacs 装起来容易，用起来才费功夫：它自成一套按键与命令模型。本章把安装、配置、运行、排错与技巧串成一条线，共十节。

《Emacs》主页讲安装与配置，《Emacs/使用》讲进入退出与基本编辑，两节是必读主干，按此先后走；要让 Emacs 常驻，再读《Emacs/运行》的守护进程与 systemd 单元。《Emacs/概念》讲屏幕组织、按键与命令，按 C-x、M-x 接不上时回头补；《Emacs/术语》《Emacs/帮助》留作回查，《Emacs/提示与技巧》按需取用，EXWM 是把 Emacs 当窗口管理器，不作此用可跳，《Emacs/疑难解答》等出问题再翻。最常见的坑是照抄别人的配置：镜像源没改、use-package 没装，启动即报无法打开加载文件；守护进程或 gccemacs 与 systemd 单元没理顺，服务起不来；无彩色终端下启动还会撞上 WINDOW_FOREGROUND 报错。

3.1　Emacs/术语

难度 入门 | **篇幅** 0.3 KB | **所属** Emacs

下表主要记录了本维基 Emacs 有关文章采用的术语及其英文。

术语	英文
帧	frame
缓冲区	buffer
帧缓冲区	frame buffer
模式	mode
框架	framework
变音符号	accent
点	point
按键；关键字	Key

3.2　Emacs/帮助

难度 入门 | **篇幅** 1.7 KB | **所属** Emacs

本节将讲到

- 手册

Emacs 有一个内置教程，点击启动屏幕上的第一个链接，然后从菜单中选择 *Help>Emacs Tutorial*，或者按 `C-h t`，即可进入该教程。

要以英语以外的语言（如**简体中文**）阅读教程，请使用命令 `Alt x`，并输入 `help-with-tutorial-spec-language`。

Emacs 在设计上是自文档化的。因此，我们可以利用大量信息来确定特定命令的名称或其按键绑定。使用 `C-h C-h` 查看所有上下文帮助绑定。

Emacs 还包含一套参考资料卡，对初学者和专家都很有用，请参见 `/usr/share/emacs/*version*/etc/refcards/` 。

手册

如果你真的想掌握 Emacs，最推荐的文档来源仍然是官方手册：

- Emacs：完整的 Emacs 用户手册。
- Emacs FAQ。
- Emacs Lisp Intro：如果你以前从未使用过任何编程语言，那么就来学习 Emacs Lisp 入门吧。
- Elisp：如果您已经熟悉一种编程语言。

您可以从 GNU.org 或直接从 Emacs 本身通过内嵌的 *info* 阅读器以 PDF 格式访问它们： `C-h i` 。按 `m` 键选择图书。

一些用户喜欢使用 *info* 阅读书籍，因为它有方便的快捷键，段落可根据窗口宽度调整，字体可根据当前屏幕分辨率调整。有些人则认为 *info* 对眼睛的刺激较小。最后，你可以轻松地将书中的内容复制到任何 Emacs 缓冲区，甚至可以直接执行示例中的 Lisp 代码片段。

您可能想阅读 **Info** 书籍以了解更多：`C-h i m info <RET>` 。 在 *Info mode* 下按 `?` 可快速查看快捷键列表。

3.3 Emacs/软件包

难度 入门 | **篇幅** 3.9 KB | **所属** Emacs

本节将讲到

- 软件镜像源
- use-package

Emacs 的功能可以通过第三方软件包进行扩展。官方支持使用内置的软件包管理器 `package.el` ，不过 Emacs 社区成员还编写了其他一些软件包管理器。`package.el` 依靠变量 `package-archives` 查找软件包。默认情况下，这包括 Emacs Lisp Package Archive (ELPA)。`M-x list-packages` 将创建一个缓冲区，列出 Emacs 知道的所有软件包。手册（(info "(emacs) Packages") ）包含更多信息。

可以添加第三方软件包存档。其中使用最广泛的是 MELPA。

许多流行的扩展以软件包的形式出现在官方软件仓库中，更多的扩展则通过AUR提供。这些软件包的名称通常带有 *emacs-* 前缀（例如 emacs-lua-mode包），但也不总是如此（例如 auctexAUR）。

有些软件包可能需要修改配置文件才能激活，以便在 Emacs 会话中使用其功能。例如，如果安装了 auctexAUR，则需要在配置文件中添加

```
(load "auctex.el" nil t t)
(load "preview-latex.el" nil t t)
```

到配置文件中。其他软件包会在源代码的注释部分或 README 中告诉你如何激活它们。

软件镜像源

中国大陆用户可能会需要 ELPA 镜像站以加速软件包下载。

校园网联合镜像站列出了国内已知许多 ELPA 镜像。

对于大多数镜像源，需要设置 `package-archives`。比如用北京外国语大学开源软件镜像站提供的 GNU ELPA 和 MELPA 镜像：

```
(setq package-archives '(("gnu"    . "https://mirrors.bfsu.edu.cn/elpa/gnu/")
                         ("nongnu" . "https://mirrors.bfsu.edu.cn/elpa/nongnu/")
                         ("melpa"  . "https://mirrors.bfsu.edu.cn/elpa/melpa/")))
(package-initialize) ;; 您的配置文件里可能已经有了这一行
```

浏览各镜像站帮助页面以获取详细信息。

use-package

自 Emacs 29.1 起，**use-package** 已随 Emacs 一起提供。*use-package* 宏允许你以声明的方式在启动文件中设置软件包自定义。它可以帮你处理许多原本需要大量重复模板代码的事情。它可以帮助进行常见的自定义，如绑定键、设置钩子、自定义用户选项和 face、自动加载等。它还能帮助你保持 Emacs 的启动速度，即使你使用了很多（甚至上百个）软件包。

请注意，use-package 并不是软件包管理器。尽管 use-package 具有与 Emacs 软件包管理器接口的实用功能，但它的主要用途是帮助配置和加载软件包，而不是管理软件包的下载、升级和安装。

更多信息，请参阅 Info 手册 “(use-package) Top”（在线阅读：https://elpa.gnu.org/devel/doc/use-package.html ）。

3.4 Emacs/运行

难度 入门 | **篇幅** 4.9 KB | **所属** Emacs

本节将讲到

- 无彩色
- 作为守护进程
- 作为 systemd 单元

同 Vim、Neovim 一样，在启动 emacs 之前，您应该知道如何关闭它（尤其是在终端中运行时）：使用 `Ctrl+xCtrl+c` 键序。

要启动 Emacs，请运行：

```
$ emacs
```

如果 X11 或 Wayland 图形界面可用，此命令将会启动 Emacs 的图形用户界面。在这种情况下，建议运行以下命令以使其在后台运行：

```
$ emacs &
```

或在纯控制台中使用 Emacs（出于空间原因，菜单通常不会显示，可以按 `F10` 调出菜单。）：

```
$ emacs -nw
```

或在命令行中快速加载（无需 .emacs）和编辑：

```
$ emacs -Q -nw
```

如果安装的是 nox 版本，`emacs` 和 `emacs -nw` 将启相同作用。

还可以提供文件名，以便立即打开该文件：

```
$ emacs filename.txt
```

无彩色

默认情况下，Emacs 启动时的颜色主题显示超链接为深蓝色。要在文本终端上启动 Emacs 而不使用任何颜色主题或方案，请执行下列操作：

```
$ emacs -nw --color=no
```

这将使所有文本显示为终端的前景色——通常是黑底白字或白底黑字。

作为守护进程

为了避免每次 Emacs 启动时都重新加载 Emacs 配置文件，可以将 Emacs 作为守护进程运行：

```
$ emacs --daemon
```

然后，您可以连接到守护进程通过运行：

```
$ emacsclient -nc
```

`-c` 将创建一个新帧缓冲区（如果希望在终端中使用，请使用 `-t`}），并且不会占用终端 `-n --no-wait`）。

许多窗口管理器、混成器以及桌面环境都实现了 XDG Autostart 规范，您可以利用此特性在桌面环境等启动时自动运行 `emacs --daemon`。

需要注意的是，有些程序（如 Mutt 或 Git，用于提交信息）会等待编辑器结束，因此不能使用 `-n` 参数。 如果你的默认编辑器设置为使用该参数，则必须为这些程序指定替代编辑器（*例如* `emacsclient -a "" -t`）。

作为 systemd 单元

Emacs 26.1 中包含一个 systemd 单元。该单元与 Emacs 一起安装，但必须在安装 Emacs 后作为用户单元（而非全系统单元）启用：

```
$ systemctl --user enable --now emacs
```

服务启动后，Emacs 就准备就绪了。

如果想在 Wayland 上通过 `emacsclient` 启动图形 emacs 框架，则需要特定的附加配置片段（除非运行的是 emacs-wayland包），如 EmacsWiki 所示。

请注意，systemd 用户单元不会从登录 shell（如 `~/.bash_profile`）继承环境变量。更多信息请参见 Systemd/用户#环境变量。

如果将 emacs 作为守护进程启动，可能需要将 `VISUAL` 和 `EDITOR` 环境变量设置为 `emacsclient`，这样启动编辑器的程序就会使用 emacsclient，而不是启动编辑器的完整实例。使用外部编辑器的程序包括电子邮件程序（用于编辑邮件）、Git（用于编辑提交信息）和 less（`v` 命令用于编辑显示的文件）。请勿使用 `-n`（`--nowait`）选项，因为程序通常希望编辑器退出时编辑已经完成。

还建议更改 Emacs 的任何图形用户界面开始菜单条目（或类似条目、桌面项），使其指向 emacsclient 而不是 emacs，这样就可以使用 emacs 守护进程而不是启动一个新的 emacs 进程。

3.5 EXWM

难度 入门 | **篇幅** 5.7 KB | **所属** Emacs

本节将讲到

- 安装
- 配置
- 嵌入 LXDE
- 疑难解答
- 参见

翻译状态：

- 本文（或部分内容）译自 EXWM，最近一次同步于 2025-02-14，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

EXWM 是一款基于 Emacs 的窗口管理器。

安装

确保已安装 emacs包。还需要 xorg-xinit包。

在 Emacs 中安装 EXWM： `M-x package-install RET exwm RET`。

编辑 xinitrc 并添加：

```
exec emacs
```

在 emacs 初始化（init）文件中添加

```
(require 'exwm)
(require 'exwm-config)
(exwm-config-example)
```

以使用默认设置。如果想使用自己的设置，请使用 `(exwm-enable)` 而不是 `(exwm-config-example)`（也无需 `(require 'exwm-config)`）。

还可以在服务器模式下启动 emacs，并通过命令行启动 EXWM。请参见 https://github.com/ch11ng/exwm/issues/284 。

配置

EXWM 是一个完整的 X 窗口管理器，因此 Emacs 可以管理浏览器、vlc 等 X 窗口。您可以使用所有正常的 Emacs 窗口命令来控制窗口的位置。在 X 窗口（即非“正常”的 Emacs 缓冲区）中，有些命令会被 EXWM 捕捉，而不会传递给程序。这些键值存储在 `exwm-input-prefix-keys` 中。此外，您也可以通过自定义 `exwm-input-global-keys` 来设置全局命令。如果您不使用自定义功能，而希望在 elisp 中设置 `exwm-input-global-keys`，请注意可能需要重启 EXWM（并在启用 exwm 之前设置 `exwm-input-global-keys`）。或者，你也可以尝试使用 "or emacs" blog 中的 `cset` 宏，它可以在不重启 EXWM 的情况下重新定义 `exwm-input-global-keys`。要使用 s-& 作为启动程序（如 firefox）的快捷键，可以这样做：

```
(setq exwm-input-global-keys `(,(kbd "s-&") .
                               (lambda (command)
                                 (interactive (list (read-shell-command "$ ")))
                                 (start-process-shell-command command nil command))))
```

多显示器

EXWM 可以通过（可选）`exwm-randr` 软件包处理多显示器问题。在调用 `(exwm-enable)` 之前，您需要安装 xrandr 并在 emacs 配置文件中启用 exwm-randr。您需

要调整 “DP-1” 和 “DP-2” 的值，使其与您电脑使用的值一致；在命令行中调用 `xrandr` 不带参数，即可查看可用输出。

```
(require 'exwm-randr)
(setq exwm-randr-workspace-output-plist '(1 "DP-1"))
(add-hook 'exwm-randr-screen-change-hook
          (lambda ()
            (start-process-shell-command
             "xrandr" nil "xrandr --output DP-1 --right-of DP-2 --auto")))
(exwm-randr-enable)
```

系统托盘

EXWM 支持系统托盘，但默认情况下并未启用。要启用它，请在 dotemacs 文件中的 `(exwm-enable)` 前添加以下内容：

```
(require 'exwm-systemtray)
(exwm-systemtray-mode 1)
```

之后可能需要调整高度；可以使用 `exwm-systemtray-height` 变量进行调整。

嵌入 LXDE

EXWM 可以代替 openbox，让您仍然可以使用 LXDE 会话管理工具。

在这样做之前，请确保已为 emacs 设置好运行 EXWM 的启动文件（见上文）

lxsession 使用`~/.config/lxsession/LXDE/desktop.conf`中定义的窗口管理器。(默认情况下使用Openbox）。如果该文件不存在，则会在 `/etc/xdg/lxsession/LXDE/desktop.conf` 中搜索。

用 emacs 替换任一文件中的 `openbox-lxde`：

```
[Session]
window_manager=emacs
```

lxsession-logout

您可以在 emacs 中创建以下函数，以便在 LXDE 会话中干净利落地注销、关机或重启：

```
(defun exwm-logout ()
  (interactive)
  (recentf-save-list)
  (save-some-buffers)
  (start-process-shell-command "logout" nil "lxsession-logout"))
```

该功能会将最近的历史记录存储到磁盘中，提示您保存、丢弃或更改未保存缓冲区中的更改，然后启动注销管理器。你可以将此功能绑定到 emacs 中的任意键上。

疑难解答

火狐浏览器屏幕撕裂

在某些程序中，特别是 Firefox 中，您可能会遇到屏幕撕裂现象。您可以试试

- 在首选项 > 高级 > 使用平滑滚动中关闭平滑滚动。
- 安装（并激活）Compton 或其他合成管理器：Xorg#合成。

令人困惑的缓冲区名称

您可能会看到缓冲区名称被命名为 "*EXWM*"。EXWM 允许缓冲区自行命名。要允许缓冲区自行命名，请在 dotemacs 中添加以下内容：

```
;; Make buffer name more meaningful
(add-hook 'exwm-update-class-hook
          (lambda ()
          (exwm-workspace-rename-buffer exwm-class-name)))
```

参见

- EXWM 维基

3.6 Emacs

难度 进阶 | **篇幅** 9.4 KB | **所属** Emacs

本节将讲到

- 页面
- 行文与翻译说明
- 安装
- 配置
- 替代品

GNU Emacs 基本界面，来源：自由软件基金会

GNU Emacs 是一个可扩展、可定制、自由的文本编辑器，其核心是 Emacs Lisp 解释器，Emacs Lisp 是 Lisp 编程语言的一种方言，具有支持文本编辑的扩展功能。

GNU Emacs 的特性包括：

- 内容感知编辑模式，包括语法着色，适用于多种文件类型。可以使用 EglotGNU 文档

、lsp-bridge 等包添加语言服务器支持。

- 完整的内置文档，包括针对新用户的教程。以及正在进行的简体中文翻译。
- 完全支持几乎所有人类文字的 Unicode。
- 可使用 Emacs Lisp 代码或图形界面进行高度定制。
- 除文本编辑功能外，还具有项目计划器、邮件和新闻阅读器、调试器界面、日历、IRC 客户端等内置多种功能。

- 用于下载和安装扩展程序的打包系统。
- 各种语言的活跃的社区。

页面

Arch Linux 中文维基对 Emacs 有关介绍分为以下几部分：

- 章节 #安装
- 章节 #配置
- 章节 #替代品
- 章节 #参见
- 子页面 /运行
- 子页面 /使用
- 子页面 /提示与技巧
- 子页面 /疑难解答
- 子页面 /软件包
- 子页面 /帮助

此外还有编者的 #行文与翻译说明及术语表。

行文与翻译说明

术语

详见 /术语。

翻译说明

翻译状态：

- 本文（或部分内容）译自 Emacs，最近一次同步于 2025-02-02，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

安装

安装下列软件包之一：

- emacs包——稳定版，
- emacs-nox包——无图形用户界面；仅在终端模拟器上运行，
- emacs-wayland包——启用了 PGTK，
- emacs-lucidAUR——用 Lucid (Athena) 工具包代替 GTK；少了些 Bug，少了些光彩。该界面现在是 Emacs Android 端的默认界面。

如果希望 Emacs 拼写检查正常工作，还需要安装 aspell包 和 aspell 语言，如 aspell-en包

还可以搜索其他变体。例如，emacs-gitAUR 提供了 GNU Emacs 的开发分支。

配置

Emacs 的主要特点之一是可扩展性和易于配置。Emacs 有一个内置的自定义引擎。你可以使用 `M-x customize`，它将显示一个自定义选项列表。关于如何使用该界面，请参阅 “Easy Customization” info 节点：`(info "(emacs) Easy Customization")`。你可以只在一个 Emacs 会话中设置自定义选项，也可以将其保存到配置文件中，以便在不同的 Emacs 会话中保存。请注意，如果您选择 *Apply and Save*，自定义界面就会这样做。

Emacs 启动时，通常会尝试从“初始化文件”（简称 “init 文件”）中加载 Lisp 程序。如果该文件存在，它将指定如何为您初始化 Emacs。Emacs 会在 `~/.emacs`、`~/.emacs.el`、`~/.emacs.d/init.el` 或 `~/.config/emacs/init.el` 中查找初始化文件。更多信息，请参见 info 节点 “Init File”：`(info "(emacs) Init File")`

替代品

有许多“小型”文本编辑器，至少表面上与 GNU Emacs 类似。下面是其中一些：

- **mg** — 用 C 语言编写的 Emacs 轻量级实现。注意它不支持 UTF-8。

: https://github.com/hboetes/mg || mg包

- **Zile** — “Zile Is Lossy Emacs” 是一个轻量级克隆，编写时尽可能与 Emacs 相似。Zile 不支持 UTF-8。

: https://www.gnu.org/software/zile/ || zileAUR

- **uemacs** — Linus Torvalds 定制的 “Micro-emacs” 版本。

: https://git.kernel.org/pub/scm/editors/uemacs/uemacs.git || uemacs-gitAUR

- **emacs-ng** — Emacs 的新方法——包括 TypeScript、线程、异步 I/O 和 WebRender。

: https://github.com/emacs-ng/emacs-ng || emacs-ng-gitAUR

- **qemacs** — “Quick Emacs”，基于 Emacs Lisp 解释器，但代码量较小，用户界面也有所简化。

: https://bellard.org/qemacs/ || qemacsAUR

- **sxemacs** — xemacs 的分叉。

: https://www.sxemacs.org/ || sxemacsAUR

- **jove** — Jonathan 自己的 Emacs 版本是一个类似 Emacs 的编辑器，但没有 Lisp。

: https://github.com/jonmacs/jove || joveAUR

Emacs 发行版

Emacs “发行版”是 Emacs 软件包和自定义软件的集合。它们比你自己定制 emacs 更容易安装和使用（但却不那么个性化）。

- Spacemacs——社区驱动的 Emacs 发行版——最好的编辑器既不是 Emacs 也不是 Vim，而是 Emacs 与 Vim！
- Doom Emacs——An Emacs framework for the stubborn martian hacker.为顽固的火星黑客设计的 Emacs 框架。
- Witchmacs——最可爱的 Emacs 发行版。
- Yukimacs——从头开始构建的 GNU/Emacs 配置（无 Spacemacs 或 Doom Emacs），灵感主要来自 Uncle Dave 的 Emacs 和 Witchmacs。

更多：Starter Kits

参见

- GNU Emacs 主页
- emacs 手册GNU 文档
- Emacs Wiki（英文）
- Emacs China
- WikEmacs ——一个可读性更强但不够完整的 Emacs 维基（英文）
- Emacs 及其快捷键的实用介绍（英文）
- The Church of Emacs (通过 Google drive) (humor)（英文）
- 官方参考卡（英文）
- EXWM，Emacs X 窗口管理器
- 《Pavinberg's Emacs Book》：专业 Emacs 入门
- Emacs-for-Noobs

3.7 Emacs/疑难解答

难度 进阶 | **篇幅** 9.8 KB | **所属** Emacs

本节将讲到

- Emacs 启动失败，提示错误信息 'Undefined color: "WINDOW_FOREGROUND"'
- 使用 Gccemacs 时 Emacs systemd 服务无法启动
- 彩色输出问题
- 在 X 窗口中显示字符的问题
- 启动缓慢

本文主要讲解了在使用 Emacs 的过程中可能遇到的问题及其解决方案。

Emacs 启动失败，提示错误信息 'Undefined color: "WINDOW_FOREGROUND"'

您需要安装 mcpp包 软件包或 gcc包 软件包。*xrdb* 使用 C 预处理器 *cpp* 对 X 资源进行预处理。如果系统中未安装 C 预处理器，*xrdb* 会自动跳过运行 C 预处理器，并且符号 WINDOW_FOREGROUND 不会扩展为十六进制颜色代码。

使用 Gccemacs 时 Emacs systemd 服务无法启动

使用 Gccemacs（分支 `emacs-native-comp` 或 `pgtk-nativecomp`）并尝试为其启动 systemd 服务时，可能会记录一条提示未找到 `.eln` 文件的错误信息：

```
$ journalctl --user -xb -g eln
emacs: emacs: ../native-lisp/28.0.50-x86_64-pc-linux-gnu-
fc9c33938bf279333039c28064d363e8/lisp-
mode-410874a46ab8852855281f021ca61fe7-1738806322de892570d69dfc55b437c2.eln: cannot open
shared object file: No such file or directory
```

作为一种解决方法，编辑 `emacs.service`中 \[Service\] 部分的 `WorkingDirectory` 行。

```
emacs.service
[Service]
...
WorkingDirectory=/usr/lib/emacs/28.0.50/x86_64-pc-linux-gnu/
...
```

注意：必须在可能存在的 `Restart` 参数**之前**定义 `WorkingDirectory` 参数。

彩色输出问题

默认情况下，Emacs shell 会显示用于打印颜色的原始转义序列。换句话说，它会显示奇怪的符号来代替所需的彩色输出。

在 `~/.emacs` 中加入以下内容后，问题就迎刃而解了：

```
(add-hook 'shell-mode-hook 'ansi-color-for-comint-mode-on)
```

在 X 窗口中显示字符的问题

如果在 X 窗口中启动 emacs 时，主窗口中的所有字符都是带黑边的白框（即在尝试查看未安装正确字体的字符时看到的白框），则需要安装 xorg-fonts-75dpi包 和/或 xorg-fonts-100dpi包 并重启 X 窗口。

启动缓慢

提示：为了减少启动时间，用户可以考虑运行 Emacs #作为 Systemd 单元。

启动时间慢通常由以下两种情况之一造成。

要确定是哪种情况，请使用以下命令运行 Emacs：

```
$ emacs -q
```

- 错误，尤其是在 `/etc/hosts` 中的错误，往往会导致 Emacs 启动时出现 5 秒以上的延迟。有关信息，请参阅网络配置指南中的"设置主机名"。
- 您可能需要监控从计算机发送的任何网络数据包（使用 Wireshark 等程序），查看是否有任何奇怪的行为。
- 查找原因的一个简单方法是注释掉（即在行前加上";"）`~/.emacs` 中的可疑部分（或 `~/.emacs.d/init.el` (或 `~/.emacs.d/init.el`）中的可疑部分，然后再次启动 Emacs 看看是否有任何变化。请注意，使用 “require” 和 “load” 可能会减慢启动速度，尤其是在使用较大的扩展时。原则上，只有在以下情况下才可使用它们：Emacs 启动后需要，或只为扩展提供“自动加载”功能。否则，请直接使用 “autoload” 函数。例如

```
    (require 'anything)
```

你可以使用

```
    (autoload 'anything "anything" "Select anything" t)
```

无法打开加载文件： ...

导致此错误的最常见原因是 “load-path” 变量未包含扩展所在目录的路径。要解决这个问题，可在尝试加载扩展之前将适当的路径添加到要搜索的列表中：

```
 (add-to-list 'load-path "/path/to/directory/")
```

如果尝试使用扩展包，而 Emacs 的配置前缀又不是 `/usr`，则需要更新加载路径。在软件包提供的说明之前，将以下内容放入 `~/.emacs` 中：

```
 (add-to-list 'load-path "/usr/share/emacs/site-lisp")
```

如果手动编译 Emacs，请注意默认前缀为 `/usr/local`。

坏掉的变音符号键问题：“<dead-acute> is undefined”

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：文风需要调整。（在Talk:Emacs/疑难解答讨论）

在谷歌上搜索这个错误，我们找到了这个链接：https://lists.gnu.org/archive/html/help-gnu-emacs/2009-05/msg00167.html ，解释了这个问题。使用变音符号键的正常方法无法达到预期效果。如果尝试对 “fiancé” 这样的单词使用变音符号，就会产生上述信息。

解决方法是在启动文件 `~/.emacs` 中加入下面一行：

```
  (require 'iso-transl)
```

不，这不是一个 bug，而是新 Emacs 版本的一项功能。通过阅读邮件列表中的相关信息，我们发现了它：

: 因为需要选择 iso-transl 或 iso-acc，所以似乎什么都不会自动加载。这两种输入法似乎都提供了 C-x 8 或 Alt-<accent> 前缀，但你我现在做的只是按一个死键（^、´、\`、~、¨）来输入变音符号，然后再按另一个键来“组成”变音符号。在此过程中没有使用 Alt 键！根据文档，它似乎适用于 8 位编码，因此在 UTF-8 中应该没什么用。这个错误出现时我就报告过，但这个错误似乎被归类为一个功能……也许这只是因为文件是自动加载的，虽然很没用。

C-M-% 和其他一些绑定在 emacs nox 中不起作用

这是因为终端比 Xorg 更受局限。不过，某些终端可能比其他终端处理更多的绑定。两种解决方案：

- 要么使用图形版本，
- 要么将绑定更改为受支持的绑定。

示例：

```
.emacs
(global-set-key (kbd "C-M-y") 'query-replace-regexp)
```

Emacs 挂起

由于 Emacs 的单线程特性，许多操作都会阻塞。这种情况可能有几种。例如，Emacs 可能正在等待你的输入（例如，你在一帧中打开了 minibuffer，却在另一帧中尝试工作）。或者，Emacs 正在运行的代码需要一段时间才能完成。又或者，你遇到了一个错误。有几种方法可以在不杀死 Emacs 进程的情况下解锁 Emacs。

- 试试按 `C-g`。根据 Emacs 的操作，您可能需要按多次。
- 尝试按 `ESC ESC ESC`。
- 在另一个终端运行 `killall -SIGUSR2 emacs`

Emacs-nox 输出变得凌乱

在终端中工作时，颜色、缩进或任何与输出相关的内容都可能变得疯狂。这（很可能？）是因为 Emacs 在某一时刻发送了一个特殊字符，而该字符可能与当前终端冲突。 如果出现这种情况，你可以执行 `M-x redraw-display`，这将重绘终端显示。如果这个问题经常出现，你可能需要将命令绑定到某个按键上，例如：`M-x redraw-display`

```
(global-set-key (kbd "<f7>") 'redraw-display)
```

在您的 `.emacs` 文件中。

图形化 Emacs 没有这个问题。

在 emacs 终端中显示奇怪的转义数字 (utf-8)

将这些值导出到 `.bashrc` 或 `.zshrc` 中：

```
$ ~/.bashrc or ~/.zshrc
export LANG\='en_US.UTF-8'
export LC_ALL\="en_US.UTF-8"
export TERM\=xterm-256color
```

这可能会导致错误，因为在 Linux 发行版中，正确值使用的是小写 utf（例如 `en_US.utf-8`。

要查看所有可用的本地语言，请使用 `locale -a`。

在 tmux 的 emacs 中，Shift + 方向键不起作用

在 tmux 配置中启用 xterm 键：

```
~/.tmux.conf
setw -g xterm-keys on
```

因为这会破坏其他组合键，所以请在 emacs 配置中加入以下内容。

```
~/.emacs
(defadvice terminal-init-screen
  ;; The advice is named `tmux', and is run before `terminal-init-screen' runs.
  (before tmux activate)
  ;; Docstring. This describes the advice and is made available inside emacs;
  ;; for example when doing C-h f terminal-init-screen RET
  "Apply xterm keymap, allowing use of keys passed through tmux."
  ;; This is the elisp code that is run before `terminal-init-screen'.
  (if (getenv "TMUX")
    (let ((map (copy-keymap xterm-function-map)))
    (set-keymap-parent map (keymap-parent input-decode-map))
(set-keymap-parent input-decode-map map))))
```

详见 tmux FAQ。

KDE 中不正确的窗口大小调整

KDE 用户可能会发现 Emacs 窗口不能正确调整大小，而是调整后的部分是透明的，鼠标点击会被发送到底层窗口。要纠正这种行为，请将 KDE 的 GTK3 主题改为 oxygen-gtk 以外的其他主题。例如，使用 gtk3包 附带的 Emacs 主题。

要强制 Emacs 在 KDE 中完全最大化，点击标题栏中的 Emacs 图标，然后选择 *More Actions* > *Special Window Settings*。然后在 *Size & Position* 选项卡中选择 *Obey geometry restrictions*，在下拉菜单中选择 *Force*，并从右侧的单选按钮中选择 *No*。

3.8 Emacs/提示与技巧

难度 进阶 | **篇幅** 11.6 KB | **所属** Emacs

本节将讲到

- TRAMP
- 将 Emacs 用作 git 合并工具
- 将大写锁定键用作 Control 键
- 复用 emacs 和 emacsclient
- 多种配置

本文主要讲解了 Emacs 的使用技巧和高效方法，包括各方面的实用提示。

TRAMP

TRAMP（Transparent Remote Access, Multiple Protocols）是一个扩展名，顾名思义，它可以通过多种协议对远程文件进行透明访问。当提示输入文件名时，输入一个特定的表单就会调用 TRAMP。举几个例子：

以根权限打开 `/etc/hosts` 前提示输入根密码：

```
C-x C-f /sudo::/etc/hosts
```

以“你”的身份通过 SSH 连接到 “remotehost”，并打开文件 `~/example.txt`：

```
C-x C-f /ssh:you@remotehost:~/example.txt
```

TRAMP 的路径形式通常为"/\[协议\]:\[\[用户@\]主机\]:<文件>"。

以“你”的身份连接“myhost”，并使用 *sudo* 编辑 `/etc/hosts`：

```
/ssh:you@remotehost|sudo:remotehost:/etc/hosts
```

TRAMP 支持的功能远不止上面的例子。欲了解更多信息，请参阅随 Emacs 一起发布的 TRAMP info 手册。

将 Emacs 用作 git 合并工具

Git 默认支持使用 Emacs 的 *Emerge* 模式作为合并工具。不过，你可能更喜欢 *Ediff* 模式。遗憾的是，由于技术原因，git 并不支持这种模式。但还是有办法在调用 emacs 时评估一些 elisp 代码来使用它。

```
.gitconfig
[mergetool.ediff]
    cmd = emacs --eval \" (progn (defun ediff-write-merge-buffer () (let ((file ediff-
merge-store-file)) (set-buffer ediff-buffer-C) (write-region (point-min) (point-max)
file) (message \\\"Merge buffer saved in: %s\\\" file) (set-buffer-modified-p nil)
(sit-for 1))) (setq ediff-quit-hook 'kill-emacs ediff-quit-merge-hook 'ediff-write-
merge-buffer) (ediff-merge-files-with-ancestor \\\"$LOCAL\\\" \\\"$REMOTE\\\" \\
\"$BASE\\\" nil \\\"$MERGED\\\"))\"

[merge]
       tool = ediff
```

注意，命令必须在一行中。 在上例中，我们启动了一个新的 Emacs 实例。你可能想使用 emacsclient 更快地启动 Emacs，但不建议这样做，因为 Ediff 调用并不干净：它可能会扰乱你当前的 Emacs 会话。

如果想立即启动，可以使用 `-q` 参数。如果想快速启动 Emacs，同时至少保留部分配置，可以使用：

```
emacs -q -l ~/.emacs-light
```

其中的轻量配置文件只加载 Ediff 所需的内容。

有关此技巧和 Ediff 问题的更多详情，请参阅 kerneltrap.org 和 stackoverflow。

将大写锁定键用作 Control 键

有些用户喜欢这种行为，以避免出现所谓的 “emacs 小指”。 在图形用户界面桌面（Xorg 或 Wayland）、终端甚至控制台中实现这一功能的好方法是使用 keyd包。安装此软件包并创建此配置文件：

```
/etc/keyd/main.conf
[ids]
*

[main]
capslock = overload(control, noop)
```

然后启用/启动 keyd 服务。

这样大写锁定键就会启 Control 键之作用。

复用 emacs 和 emacsclient

在同一个 `emacs-session` 中打开新文件，需要使用 `emacsclient`。如果会话存在，`emacs` 命令本身也可以进行封装，以更智能地打开文件。

要启动会话，需要使用 `start-server`。该代码段将在 emacs 的第一个会话中创建服务器。将其添加到 `emacs` 配置文件中。

```
.emacs or .emacs.d/init.el
(require 'server)
(unless (server-running-p)
  (server-start))
```

Shell 别名方法并不能满足要求，因为你还需要传递变量或启动自己的独立会话。在 shell 的 `.bashrc` 或任何 rc 文件中添加此选项。这样，如果参数被传递，`emacs` 命令就会像 emacsclient 一样运行。

```
function emacs {
    if [[ $# -eq 0 ]]; then
        /usr/bin/emacs # "emacs" is function, will cause recursion
        return
    fi
    args=($*)
    for ((i=0; i <= ${#args}; i++)); do
        local a=${args[i]}
        # NOTE: -c for creating new frame
        if [[ ${a:0:1} == '-' && ${a} != '-c' && ${a} != '--' ]]; then
            /usr/bin/emacs ${args[*]}
            return
        fi
    done
    setsid emacsclient -n -a /usr/bin/emacs ${args[*]}
}
```

如果要在新会话中运行，只需执行 `emacs *file* -`。

多种配置

您可以使用多种配置，并告诉 Emacs 加载其中一种或另一种。

例如，让我们定义两种配置文件。

```
.emacs
(load "~/.emacs.d/main" nil t)
(load "~/.emacs.d/functions" nil t)
(load "~/.emacs.d/modes" nil t)
(load "~/.emacs.d/plugins" nil t)
(load "~/.emacs.d/theme" nil t)
```

这是我们针对守护进程加载的全部配置。但 *plugins* 文件很大，加载速度也很慢。如果我们想生成一个不需要 *plugins* 功能的新 Emacs 实例，那么从长远来看，每次加载都会很麻烦。

```
.emacs-light
(load "~/.emacs.d/main" nil t)
(load "~/.emacs.d/functions" nil t)
(load "~/.emacs.d/modes" nil t)
(load "~/.emacs.d/theme" nil t)
```

现在我们用：

```
emacs -q -l ~/.emacs-light
```

您可以创建一个别名（如 `emacs-light`），以便调用。

局部变量和自定义变量

您可以在配置文件中定义变量，以后可以在本地对文件进行修改。

```
(defcustom my-compiler "gcc" "Some documentation")
```

现在，在任何文件中都可以通过两种方式定义局部变量，详见手册。

- 使用 `M-x add-file-local-variable-prop-line` 可以在开头添加一行注释，类似于：

```
// -*- my-compiler:g++; mode:c++ -*-
```

- 或者使用 `M-x add-file-local-variable` 在文件末尾添加行：

```
// Local Variables:
// my-compiler: g++
// mode: c++
// End:
```

请注意，要使这些值生效，您需要调用 `M-x revert-buffer`。

默认情况下，自定义变量被认为是不安全的。如果你试图打开一个包含重新定义自定义变量的局部变量的文件，Emacs 会要求你确认。

您可以将变量声明为安全变量，从而消除 Emacs 的确认提示。您需要指定一个谓词，任何新值都必须经过该谓词的验证，这样才能被认为是安全的。

```
(defcustom my-compiler "gcc" "Some documentation" :safe 'stringp)
```

在上例中，如果您试图设置字符串以外的内容，Emacs 会认为这是不安全的。

自定义色彩和主题

可以使用 *face* 工具轻松定制颜色。

```
(set-face-background  'region                 "color-17")
(set-face-foreground  'region                 "white")
(set-face-bold-p      'font-lock-builtin-face t )
```

您可以让 Emacs 告诉您点所在的 face 的名称。为此，请使用 *customize-face* 功能。该功能会告诉你如何设置颜色、粗体、下划线等。

控制台中的 Emacs 可以处理 256 种颜色，但必须使用合适的终端。例如，URxvt 支持 256 种颜色。你可以使用 *list-colors-display* 查看支持颜色的完整列表。这在很大程度上取决于终端。

参见：

- https://www.emacswiki.org/emacs/ColorThemes
- https://www.gnu.org/software/emacs/manual/html_node/emacs/Custom-Themes.html

SyncTeX 支持

Emacs 是一款功能强大的 LaTeX 编辑器。这主要是因为你可以调整或创建最适合自己需要的 LaTeX 模式。

不过，也可能会遇到一些挑战，比如 SyncTeX 支持。首先，你需要确保你的 TeX 发行版支持 SyncTeX。如果你手动安装了 TeX Live，可能需要安装 *synctex* 软件包。

```
# umask 022 && tlmgr install synctex
```

SyncTeX 支持与阅读器有关。在此，我们将以 Zathura 为例，因此如果您想使用其他 PDF 阅读器，则需要对代码进行调整。

```
(defcustom tex-my-viewer "zathura --fork -s -x \"emacsclient --eval '(progn (switch-to-buffer  (file-name-nondirectory \"'\"'\"%{input}\"'\"'\")) (goto-line %{line}))'\""
  "PDF Viewer for TeX documents. You may want to fork the viewer
so that it detects when the same document is launched twice, and
persists when Emacs gets closed.

Simple command:

  zathura --fork

We can use

  emacsclient --eval '(progn (switch-to-buffer  (file-name-nondirectory \"%{input}\")) (goto-line %{line}))'

to reverse-search a pdf using SyncTeX. Note that the quotes and double-quotes matter and must be escaped appropriately."
:safe 'stringp)
```

在这里，我们定义了自定义变量。如果您使用的是 AucTeX 或 Emacs 默认的 LaTeX 模式，则必须相应地设置查看器。

现在用 Emacs 打开一个 LaTeX 源文件，编译文档，然后启动查看器。Zathura 进程会自动创建。如果按下 `Ctrl+Left click` 键，Emacs 就会将点放在相应的位置。

systemd 文件的语法高亮显示

您可以使用 systemd-mode。

或者，你也可以在 init 文件中添加以下内容，让 emacs 为 systemd 文件（服务、定时器等）着色：

```
(add-to-list 'auto-mode-alist '("\\.service\\'" . conf-unix-mode))
(add-to-list 'auto-mode-alist '("\\.timer\\'" . conf-unix-mode))
(add-to-list 'auto-mode-alist '("\\.target\\'" . conf-unix-mode))
(add-to-list 'auto-mode-alist '("\\.mount\\'" . conf-unix-mode))
(add-to-list 'auto-mode-alist '("\\.automount\\'" . conf-unix-mode))
(add-to-list 'auto-mode-alist '("\\.slice\\'" . conf-unix-mode))
(add-to-list 'auto-mode-alist '("\\.socket\\'" . conf-unix-mode))
(add-to-list 'auto-mode-alist '("\\.path\\'" . conf-unix-mode))
(add-to-list 'auto-mode-alist '("\\.netdev\\'" . conf-unix-mode))
(add-to-list 'auto-mode-alist '("\\.network\\'" . conf-unix-mode))
(add-to-list 'auto-mode-alist '("\\.link\\'" . conf-unix-mode))
```

为 emacs-nox 提供剪贴板支持

要在 emacs-nox 中使用 Xorg 剪贴板，请安装。xclip包 并在 `~/.emacs` 中添加以下函数：

```
;; use xclip to copy/paste in emacs-nox
(unless window-system
  (when (getenv "DISPLAY")
    (defun xclip-cut-function (text &optional push)
      (with-temp-buffer
       (insert text)
```

```
       (call-process-region (point-min) (point-max) "xclip" nil 0 nil "-i" "-selection"
"clipboard")))
    (defun xclip-paste-function()
      (let ((xclip-output (shell-command-to-string "xclip -o -selection clipboard")))
       (unless (string= (car kill-ring) xclip-output)
         xclip-output )))
    (setq interprogram-cut-function 'xclip-cut-function)
    (setq interprogram-paste-function 'xclip-paste-function)
    ))
```

提示：您还可以通过添加以下内容来启用终端鼠标支持：

```
;; xterm mouse support
(require 'mouse)
(xterm-mouse-mode t)
```

参见 mwheel.el\[失效链接 2024-11-05 ⓘ\]。

3.9 Emacs/概念

难度 进阶 | **篇幅** 19.0 KB | **所属** Emacs

本节将讲到

- 屏幕组织
- 用户输入类型
- 按键
- 鼠标输入
- 按键与命令

本文介绍 Emacs 的基本概念。

屏幕组织

在图形显示器上，例如在使用 X 窗口系统的 GNU/Linux 上，Emacs 占用一个图形窗口。在文本终端上，Emacs 占用整个终端屏幕。我们将使用*帧*一词来指 Emacs 占用的图形窗口或终端屏幕。Emacs 在这两种帧上的表现非常相似。通常一开始只有一个帧，但也可以根据需要创建其他帧。

每个帧都由几个不同的区域组成。帧顶部是菜单栏，可以通过一系列菜单访问命令。在图形显示屏上，菜单栏的正下方是*工具栏*，这是一排图标，点击后可执行编辑命令。帧的最下方是回显区，这里显示信息，当 Emacs 要求输入信息时，你也可以在这里输入信息。

帧的主要区域，即*工具栏*（如果有的话）下方和回显区上方，称为*窗口*。因此，在本手册中，我们将在这个意义上使用*“窗口“*一词。图形显示系统通常使用含义不同的“窗口”一词，但如上所述，我们将这些图形窗口称为*“帧”*。

Emacs 窗口是显示*缓冲区*的地方，缓冲区就是你正在编辑或查看的文本或其他图形。在图形显示器上，窗口的一侧有一个滚动条，可以用来滚动缓冲区。窗口的最后一行

是模式行。它显示缓冲区中的各种信息，如是否有未保存的更改、正在使用的编辑模式、当前行号等。

启动 Emacs 时，帧中通常只有一个窗口。不过，您可以水平或垂直细分这个窗口，创建多个窗口，每个窗口都可以独立显示一个缓冲区。

在任何时候，一个窗口都是选定窗口。在图形显示屏上，选定窗口显示一个较突出的光标（通常为实心和闪烁）；其他窗口显示一个不太突出的光标（通常为空心方框）。在文本终端上，只有一个光标，显示在选定窗口中。选定窗口中显示的缓冲区称为当前缓冲区，编辑工作就在这里进行。大多数 Emacs 命令都隐式地应用于当前缓冲区；未选中窗口中显示的文本主要供参考。如果在图形显示屏上使用多个帧，则选择特定帧就会选择该帧中的窗口。

点

所选窗口中的光标显示大多数编辑命令生效的位置，称为*点*。许多 Emacs 命令会将点移动到缓冲区中的不同位置；例如，在所需位置单击鼠标键 1（通常是*左键*），就可以将点移动到该位置。

默认情况下，所选窗口中的光标绘制为*实心块*，似乎位于一个字符上，但您应该将 point 视为**两个字符之间**；它位于光标下的字符之**前**。例如，如果您的文本是 “Arch Linux 中社区”，光标在“社”上，那么点就在“中”和“社”之间。如果在该位置插入字符，结果就是“Arch Linux 中文社区”，点位于“文”和“社”之间。因此，光标仍和以前一样位于“社”之上。

回显区

帧最下方的一行是回显区。它用于显示各种用途的少量文字。

回显区之所以叫回显区，是因为它的作用之一就是回显，即在输入多字符命令时显示其中的字符。单字符命令不会有回显。多字符命令如果在输入过程中停顿超过一秒，就会产生回显。然后，Emacs 会回显显示该命令到目前为止的所有字符，以提示您输入其余字符。一旦开始回显，命令的其余部分就会在你输入时立即回显。这种行为旨在为自信的用户提供快速响应，同时为犹豫不决的用户提供最大程度的反馈。

当命令无法执行时，回显区还用于显示错误信息。错误信息可能伴有蜂鸣声或屏幕闪烁。

有些命令会在回显区显示提示信息，告诉您该命令执行了什么操作，或为您提供一些特定信息。这些信息与错误信息不同，不会伴有蜂鸣声或闪光。例如，`C-x =`（按住 *Ctrl* 键并键入 *x*，然后放开 *Ctrl* 键并键入 \=）会显示一条信息，说明点的字符、它在

缓冲区中的位置以及它在窗口中的当前列。耗时较长的命令在运行过程中通常会显示以“...”结尾的信息（有时还会以百分比的形式显示运行进度），并在完成后添加“done”（已完成）。

回显区的消息保存在一个名为 \Messages** 的特殊缓冲区中。如果错过了屏幕上短暂出现的信息，可以切换到 \Messages** 缓冲区再次查看。\Messages** 缓冲区有一定的行数限制，由变量 `message-log-max` 指定。(我们也没有解释变量；有关变量的更多信息，请参阅变量）。超过这个限制，每当在末尾添加一行新的信息时，就会从开头删除一行。

回显区还用于显示 **minibuffer**，这是一个特殊窗口，您可以在此输入命令参数，如要编辑的文件名。使用微型缓冲区时，回显 区显示的文本以提示字符串开头，活动光标显示在微型缓冲区内，**minibuffer** 暂时被视为选定窗口。您可以通过键入 C-g 退出 **minibuffer**。

模式行

每个窗口的底部都有一个模式行，用于描述当前缓冲区中正在发生的事情。当只有一个窗口时，模式行会出现在回显区的正上方；它是帧中倒数第二行。在图形显示屏上，模式行以 3D 方框的形式显示。Emacs 通常还会用不同于未选定窗口的颜色来绘制选定窗口的模式线，以使其更加突出。

模式线中显示的文本格式如下：

```
cs:ch-fr  buf      pos line   (major minor)
```

在文本终端上，该文本后面会有一连串延伸至窗口右边缘的破折号。在图形显示屏上，这些破折号会被省略。

cs 字符串及其后的冒号字符描述了当前缓冲区使用的字符集和换行约定。通常，Emacs 会自动为您处理这些设置，但有时掌握这些信息也很有用。

cs 描述缓冲区中文本的字符集。如果是破折号（'-'），则表示没有特殊的字符集处理（行结束约定可能除外，下一段将介绍）。\= 表示不进行任何转换，通常用于包含非文本数据的文件。其他字符表示各种编码系统，例如'1'表示 ISO Latin-1。

在文本终端上，cs 前面还有两个字符，分别描述键盘输入和终端输出的编码系统。此外，如果使用的是输入法，cs 前面还会有一个字符串，用来标识输入法。

cs 后面的字符通常是冒号。如果显示的字符串不同，则表示文件编码的行尾约定不尽相同。通常，文件中的文本行以换行符分隔，但有时也使用其他两种约定。MS-DOS 惯例使用一个回车符，后面跟一个换行符；在编辑此类文件时，冒号会根据操作系统

的不同变为反斜线（'\\'）或'(DOS)'。旧版 Macintosh 系统采用的另一种习惯是使用回车符代替换行符；编辑此类文件时，冒号会变为正斜线（'/'）或'(Mac)'。在某些系统上，对于使用换行符作为行分隔符的文件，Emacs 会显示"(Unix)"而不是冒号。

在为 emacsclient 创建的帧中，下一个字符是"@"。对于作为守护进程运行的 Emacs 进程的帧来说，这种指示很典型。

如果窗口中显示的缓冲区与磁盘上相应文件的内容相同，即缓冲区未修改，则显示两个破折号（'--'）。如果缓冲区已修改，则显示两颗星（'\\'）。对于只读缓冲区，如果缓冲区已修改，则显示"%*"，否则显示"%%"。

ch 后面的字符通常是破折号（'-'）。但是，如果当前缓冲区的默认目录在远程计算机上，则会显示"@"。

fr 给出所选帧的名称。它只出现在文本终端上。初始帧名为 "F1"。

buf 是窗口中显示的缓冲区名称。通常与正在编辑的文件名相同。请参阅使用多个缓冲区。

pos 显示窗口顶部上方或底部下方是否有附加文本。如果缓冲区较小，窗口中可以看到全部缓冲区，则 pos 为"全部"。否则，如果您看到的是缓冲区的开头，则是 "Top"；如果您看到的是缓冲区的结尾，则是 "Bot"；或者是 "nn%"，其中 nn 是窗口顶部以上缓冲区的百分比。使用 "Size Indication"（大小指示）模式，还可以显示缓冲区的大小。

行是字符 "L"，后跟在点处的行号。(打开列号模式，也可以显示当前列号。请参阅可选模式行功能）。

major 是缓冲区中使用的主要模式名称。主要模式是缓冲区的主要编辑模式，如文本模式、Lisp 模式、C 模式等。请参阅主要模式。某些主要模式会在主要模式名称后显示附加信息。例如，编译缓冲区和 Shell 缓冲区会显示子进程的状态。

minor （次要）模式是一些已启用的 minor 模式的列表，它们是在主要模式基础上提供附加功能的可选编辑模式。

有些功能虽然不是真正的 minor 模式，但只要开启，就会与次要模式一起列出。Narrow 表示显示的缓冲区的编辑范围仅限于部分文本。Def 表示当前正在定义键盘宏（请参阅键盘宏）。

此外，如果 Emacs 处于递归编辑层内，模式周围的括号会出现方括号（'\[...\]'）。如果 Emacs 处于另一个递归编辑层中的一个递归编辑层，则会出现双方括号，依此类

推。由于递归编辑级别会对 Emacs 产生全局影响，因此每个窗口的模式行中都会出现此类方括号。

您可以更改模式行的外观及其内容格式。请参阅可选模式行功能。此外，模式行对鼠标敏感；单击模式行的不同部分可执行各种命令。此外，将鼠标指针悬停在模式行的鼠标敏感区域上会显示工具提示，其中包含通过单击模式行可以调用的命令信息。

菜单栏

每个 Emacs 帧的顶部通常都有一个菜单栏，您可以用它来执行常用操作。无需在此一一列举，因为你可以更容易地看到它们。

在支持鼠标的显示器上，你可以用鼠标从菜单栏中选择命令。菜单项右边的箭头表示它指向一个辅助菜单或子菜单。菜单项末尾的"..."表示该命令在实际执行之前会提示你进一步输入。

菜单栏中的某些命令也有普通的按键绑定；如果有，按键绑定会显示在项目本身之后。要查看某个菜单项的完整命令名称和文档，请键入 C-h k，然后用鼠标以通常的方式选择菜单栏（请参阅 "按键文档"）。

您也可以不使用鼠标，而是按 `F10` 键调用第一个菜单栏项目（运行命令 `menu-bar-open`）。然后可以使用箭头键或 `C-b`、`C-f`（左/右）、`C-p` 和 `C-n`（上/下）浏览菜单。要激活选定的菜单项，请按 `RET`；要取消菜单导航，请按 `C-g` 或 `ESC ESC ESC`。(不过请注意，如果 Emacs 是使用图形用户界面工具包创建的，那么菜单的绘制和控制都是由工具包完成的，取消菜单导航的按键顺序可能与上述描述不同）。

在文本终端上，可以选择访问回显区的菜单栏菜单。为此，请将变量 `tty-menu-open-use-tmm` 设置为**非零值**。然后键入 `F10` 将运行 `tmm-menubar` 命令，而不是下拉菜单。(您也可以键入 ` M-` `，它将始终调用 `*tmm-menubar*`。）`*tmm-menubar*` 可让您用键盘选择菜单项。回显区会出现一个临时选项。您可以使用上下箭头键在菜单中移动到不同的项目，然后键入 RET` 来选择项目。每个菜单项也由一个字母或数字（通常是项目名称中某个单词的首字母）指定。该字母或数字与项目名称之间用"==>"隔开。您可以键入该项目的字母或数字来选择该项。

用户输入类型

GNU Emacs 主要是为使用键盘而设计的。虽然可以使用鼠标通过菜单栏和工具栏发出编辑命令，但效率通常不如键盘。

Emacs 中的键盘输入是基于 ASCII 的重度扩展版本。简单字符，如 "a"、"B"、"3"、"="和空格符（表示为 SPC），可通过键入相应的键来输入。控制字

符，如 RET、TAB、DEL、ESC、F1、Home 和 LEFT，以及非英语键盘上的某些字符，也可以通过这种方式输入。

Emacs 还能识别使用修改键输入的控制字符。例如，按住 Ctrl 键的同时按 a，就可以输入 Control-a；我们将其简称为 C-a。同样，Meta-a 或简称 M-a 是按住 Alt 键并按 a 键输入的。修改键也可用于非字母数字字符，例如 C-F1 或 M-LEFT。

您还可以使用以 ESC 开头的双字符序列输入 Meta 字符。因此，输入 ESC a 可以输入 M-a。输入 ESC C-a 可以输入 C-M-a（同时按住 Ctrl 和 Alt，然后按 a）。与 Meta 不同，ESC 是作为一个单独的字符输入的。输入下一个字符时不要按住 ESC，而是按下 ESC 并松开，然后输入下一个字符。在某些 Meta 键无法正常工作的文本终端上，这一功能非常有用。

Emacs 还支持另外 3 个修改键，请参阅修改键。

Emacs 广泛支持使用鼠标按钮、鼠标滚轮和其他指向设备（如触摸板和触摸屏）。

在图形显示器上，窗口管理器可能会阻止某些键盘输入，包括 M-TAB、M-SPC、C-M-d 和 C-M-l。如果遇到这种问题，可以自定义窗口管理器，使其不阻塞这些键，或者重新绑定受影响的 Emacs 命令。

简单字符和控制字符以及某些非键盘输入（如鼠标点击）统称为**输入事件**。

按键

有些 Emacs 命令只需一个输入事件即可调用，例如 C-f 在缓冲区中向前移动一个字符。其他命令需要两个或多个输入事件才能调用，例如 C-x C-f 和 C-x 4 C-f。

按键序列（简称**按键**）是由一个或多个输入事件组成的序列，作为一个单元是有意义的。如果一个按键序列调用了一个命令，我们就称它为一个*完整按键*；例如，C-f、C-x C-f 和 C-x 4 C-f 都是*完整按键*。如果一个键序的长度不足以调用命令，我们就称它为*前缀键*；从前面的例子中，我们可以看到 C-x 和 C-x 4 都是*前缀键*。每个按键序列要么是完整按键，要么是前缀按键。

前缀按键与后面的输入事件组合成一个更长的按键序列。例如，C-x 是一个前缀键，因此单独键入 C-x 并不会调用命令；相反，Emacs 会等待进一步的输入（如果暂停超过一秒，它就会回显 C-x 键以提示输入；参见回显区）。C-x 与下一个输入事件组合成一个双事件按键序列，它本身可以是一个前缀键（如 C-x 4），也可以是一个完整的按键（如 C-x C-f）。按键序列的长度没有限制，但在实际应用中很少超过三个或四个输入事件。

您不能在一个完整键上添加输入事件。例如，由于 C-f 是一个完整的键，所以 C-f C-k 这个双事件序列是两个键序列，而不是一个。

默认情况下，Emacs 的前缀键是C-c、C-h、C-x、C-x RET、C-x @、C-x a、C-x n、C-x r、C-x t、C-x v、C-x 4、C-x 5、C-x 6、ESC 和 M-g（F1 和 F2 是 C-h 和 C-x 6 的别名）。例如，如果删除了 C-x 4 的前缀定义，那么 C-x 4 C-f 就会成为一个无效的键序。请参阅自定义按键绑定。

在前缀键后输入帮助字符（C-h 或 F1），会显示以该前缀开头的命令列表。唯一的例外是 ESC：ESC C-h 等同于 C-M-h，后者的功能完全不同。不过，您可以使用 F1 显示以 ESC 开头的命令列表。

鼠标输入

默认情况下，Emacs 支持所有正常的鼠标操作，例如通过单击鼠标左键设置光标，以及通过拖动鼠标指针选择区域。所有鼠标操作都可以用来绑定命令，绑定方式与绑定键盘事件相同（请参阅#按键）。本节概述了如何在 Emacs 中使用鼠标。

单击鼠标左键时，Emacs 会收到一个 **mouse-1** 事件。要查看该事件绑定了什么命令，可以键入 C-h c，然后按下鼠标左键。同样，鼠标中键是 **mouse-2**，鼠标右键是 **mouse-3**。如果你使用的是带滚轮的鼠标，滚轮事件通常会绑定到 **wheel-down** 或 **wheel-up**，或者 **mouse-4** 和 **mouse-5**，但这取决于操作系统的配置。

一般来说，传统 X 系统和终端会报告 **mouse-4** 和 **mouse-5**，而所有其他系统都会报告 **wheel-down** 和 **wheel-up**。

有些鼠标还有水平滚轮，触摸板通常也支持水平滚动。这些事件在所有系统中都会报告为 **wheel-left** 和 **wheel-right**，但终端和传统 X 系统除外，在这些系统中报告为 **mouse-6** 和 **mouse-7**。

您还可以将键盘修饰符与鼠标事件结合起来，例如，您可以绑定一个特殊命令，当您按住 *Meta* 键并使用鼠标中键时触发该命令。在这种情况下，事件名称将是 **M-mouse-2**。

在某些系统中，还可以绑定用于处理触摸屏事件的命令。在这种情况下，事件名称为触摸屏更新（*touchscreen-update*）和触摸屏结束（*touchscreen-end*）。

按键与命令

本文中有很多段落告诉你特定按键的作用。但 Emacs 并不直接为按键赋予含义。相反，Emacs 将含义分配给已命名的命令，然后通过将按键与命令绑定来赋予按键含义。

每个命令都有一个由程序员选择的名称。名称通常由几个英文单词组成，中间用破折号隔开，例如 *next-line* 或 *forward-word*。在内部，每个命令都是一种特殊的 Lisp 函数，与命令相关的操作通过运行函数来执行。请参阅《Emacs Lisp 参考手册》中的"什么是函数"。

键与命令之间的绑定记录在称为键映射的表中。

当我们说 "C-n 垂直向下移动一行" 时，我们忽略了一个微妙的区别，这个区别在普通使用中无关紧要，但对于 Emacs 的定制却至关重要。命令 *next-line* 是垂直向下移动一行。`C-n` 具有这种效果是因为它绑定了 *next-line*。如果将 `C-n` 与 *forward-word* 命令重新绑定，`C-n` 将向前移动一个字。

在本文中，我们经常把 `C-n` 等按键说成是命令，尽管严格来说，按键是与命令绑定的。通常情况下，我们在提到运行命令的按键后，会在括号内注明真正执行命令的命令名称。例如，我们会说 "命令 C-n（next-line）将点垂直向下移动"，意思是命令 *next-line* 将点垂直向下移动，而按键 `C-n` 通常与之绑定。

既然是讨论自定义，我们就应该介绍一下变量。通常情况下，命令的说明中会说："要改变这一点，请设置变量 mumble-foo"。变量是用来存储数值的名称。本文中记录的大多数变量都是用于自定义的：Emacs 的某些命令或其他部分会检查变量，并根据你设置的值做出不同的行为。在你对自定义变量感兴趣之前，可以忽略有关变量的信息。然后再阅读有关变量的基本信息，这样有关特定变量的信息就会变得有意义。

3.10 Emacs/使用

难度 高级 | **篇幅** 36.4 KB | **所属** Emacs

本节将讲到

- 进入 Emacs
- 退出 Emacs
- 基本编辑命令

本文介绍 Emacs 的基础使用。

阅读本文前，请先阅读 Emacs/概念。

进入 Emacs

**本文或本章节可能需要合并到Emacs/运行。**

附注： 内容有重复。（在 Talk:Emacs/使用 中讨论）

调用 Emacs 的通常方法是使用 shell 命令 `emacs`。在 GUI 终端上运行 Unix shell 的终端窗口中，可以使用 `emacs &` 在后台运行 Emacs；这样，Emacs 就不会占用终端窗口，因此你可以用它来运行其他 shell 命令。

Emacs 启动时，初始帧会显示一个名为 \GNU Emacs** 的特殊缓冲区。这个启动界面包含 Emacs 的相关信息，以及对初学者有用的常用任务链接。例如，激活 *Emacs Tutorial* 链接可打开 Emacs 教程；其作用与 `C-h t`（`help-with-tutorial`）命令相同。要激活链接，可以将点移动到该链接上并键入 `RET`，或者用 `mouse-1`（鼠标左键）单击该链接。

使用命令行参数，可以让 Emacs 在启动时立即访问一个或多个文件。例如，`emacs 文本.txt` 启动 Emacs 时，缓冲区会显示文件 *文本.txt* 的内容。这一功能主要是为了与其他文本编辑器兼容，因为其他编辑器是为了从 shell 启动进行短时间编辑而设计的。如果以这种方式调用 Emacs，初始帧会被分成两个窗口，一个显示指定文件，另一个显示启动屏幕。

一般来说，每次编辑文件都重新启动 Emacs 既**不必要又浪费**。建议使用 Emacs 的方法是在登录后只启动一次，并在同一个 Emacs 会话中完成所有编辑工作。如果这样使用 Emacs，Emacs 会话就会积累一些有价值的上下文，如 *kill ring*、*寄存器*、*撤销历史*和*标记环数据*，这些都会使编辑更加方便。本文稍后将介绍这些功能。

要在 Emacs 运行时从其他程序编辑文件，可以使用 emacsclient 辅助程序在现有的 Emacs 会话中打开文件。

Emacs 还接受其他命令行参数，这些参数会告诉它加载某些 Lisp 文件、在何处放置初始框架等。参见 Emacs 调用的命令行参数。

如果变量 `inhibit-startup-screen` 不为零，则 Emacs **不显示**启动屏幕。在这种情况下，如果在命令行中指定了一个或多个文件，Emacs 会简单地显示这些文件；否则，它会显示一个名为 `*scratch*` 的缓冲区，该缓冲区可用于交互式地评估 Emacs Lisp 表达式。你可以使用自定义工具或编辑初始化文件来设置变量 `inhibit-startup-screen`。

通过将变量 `initial-buffer-choice` 设置为命名文件或目录的字符串，也可以强制 Emacs 在启动时显示文件或目录。`initial-buffer-choice` 的值也可以是一个

函数（无参数），它应该返回一个缓冲区，然后显示出来。如果 `initial-buffer-choice` 的值**不是零**，那么如果你在命令行中指定了任何文件，Emacs 仍会访问它们，但最初不会显示它们。

退出 Emacs

同 Vim/Neovim、Helix 等编辑器一样，初学者可能不知如何退出。

C-x C-c：杀死 Emacs（save-buffers-kill-terminal）。 **C-z**：在文本终端上，挂起 Emacs；在图形显示屏上，图标化（或*最小化*）所选帧（挂起帧）。 **系统 shell 内运行 `killall emacs`**：强制终止 Emacs 进程，可以在 Emacs 卡死时使用。

杀死 Emacs 意味着终止 Emacs 程序。为此，请键入 `C-x C-c`（`save-buffers-kill-terminal`）。使用两个字符的键序是为了**防止意外键入**。如果输入 `C-x C-c` 时有任何修改过的文件访问缓冲区，Emacs 会首先保存这些缓冲区。如果没有全部保存，则会再次请求确认，因为未保存的更改将丢失。如果有子进程仍在运行，Emacs 也会要求确认，因为杀死 Emacs 也会杀死这些子进程。

如果将 Emacs 用作服务器，`C-x C-c` 的行为会比较特殊。如果你在客户端框架中输入它，它会关闭客户端连接。请参阅将 Emacs 用作服务器。

Emacs 可以选择在杀死会话时记录某些会话信息，例如当时正在访问的文件。下次启动 Emacs 时就可以使用这些信息。请参阅保存 Emacs 会话。

如果变量 `confirm-kill-emacs` 的值不是零，`C-x C-c` 会假定其值是一个谓词函数，并调用该函数。如果函数调用的结果不是零，会话就会被杀死，否则 Emacs 会继续运行。用来作为 `confirm-kill-emacs` 值的一个方便的函数是 `yes-or-no-p` 函数。`confirm-kill-emacs` 的默认值为 **nil**。

如果变量 `confirm-kill-processes` 的值为 nil，则 `C-x C-c` 在杀死 Emacs 启动的子进程之前不会请求确认。默认值为 **t**。

要进一步自定义 Emacs 退出时发生的情况，请参阅《GNU Emacs Lisp 参考手册》中的杀死 Emacs。

要在不提示保存的情况下杀死 Emacs，请键入 `M-x kill-emacs`。

`C-z` 运行命令 `suspend-frame`。在图形显示屏上，该命令会最小化（或图标化）所选的 Emacs 框架，使其隐藏起来，以便稍后恢复（具体如何隐藏取决于窗口系统）。在文本终端上，`C-z` 命令会暂停 Emacs，暂时停止程序并将控制权交还给父进

程（通常是 shell）；在大多数 shell 中，暂停 Emacs 后可以使用 shell 命令 `%emacs` 恢复 Emacs。

文本终端通常会侦听某些特殊字符，其含义是杀死或暂停正在运行的程序。在 Emacs 中，这种终端功能是关闭的。在 Emacs 中，`C-z` 和 `C-x C-c` 作为按键的含义是受多个操作系统中使用 `C-z` 和 `C-c` 作为停止或杀死程序的字符的启发，但这是它们与操作系统的唯一关系。你可以自定义这些键来运行你选择的任何命令。

基本编辑命令

在此，我们将讲解如何输入文本、进行修改并将文本保存到文件中的基础知识。如果您对这些内容不熟悉，建议您首先运行 Emacs 边做边学教程，键入 `C-h t`（`help-with-tutorial`）。

插入文本

您可以通过键入相关键来插入普通图形字符（如 "a"、"B"、"3" 和 "="）。这样就可以将字符添加到点的缓冲区中。插入后，点将向前移动，因此点将保持在插入文本之后。参见点。

要结束一行并开始新一行，键入 `RET`（换行）。(`RET` 键在键盘上可能标有 Return、Enter 或有趣的向左箭头，但在本手册中我们将其称为 `RET`，中国人常称其**回车**）。该命令在缓冲区中插入一个换行符，然后根据主要模式缩进。如果点位于行尾，效果是在其后创建一个新的空行并缩进新行；如果点位于行中间，则在该位置分行。要关闭自动缩进功能，可以禁用 "Electric Indent" 模式（请参阅 "缩进的便利功能"），或者键入 C-j，只插入新行，而不进行任何自动缩进。

正如我们在本手册后面所解释的，你可以通过开启次要模式来改变 Emacs 处理文本插入的方式。例如，名为 "Auto Fill" 的次要模式会在行太长时自动分行。名为 "Overwrite" 模式的次要模式会使插入的字符替换（覆盖）现有文本，而不是将其推到右边。请参阅次要模式。

只有图形字符可以通过键入相关的按键来插入；其他按键作为编辑命令，不会自行插入。例如，默认情况下 DEL 运行 delete-backward-char 命令（某些模式将其绑定到不同的命令）；但它不会插入字面意义上的 "DEL "字符（ASCII 字符代码 127）。

要插入一个非图形字符或键盘不支持的字符，首先要输入 C-q（quoted-insert）来引用该字符。使用 C-q 有两种方法：

- C-q 后接任何非图形字符（甚至是 C-g）都会插入该字符。例如，C-q DEL 插入一个字面意义上的 "DEL" 字符。

- C-q 后跟一串八进制数字，插入指定八进制字符代码的字符。您可以使用任意八进制数字；任何非数字都会终止序列。如果终止字符是 RET，则 RET 仅用于终止序列。任何其他非数字都会终止序列，然后作为正常输入，如 C-q 1 0 1 B 插入 "AB"。

: 在普通的非二进制覆写模式下，八进制序列的使用是禁用的，这样可以方便地插入一个数字，而不是用它来覆写。

要使用十进制或十六进制代替八进制，可将变量 read-quoted-char-radix 设置为 10 或 16。如果弧度为 16，字母 a 至 f 就会像数字一样作为字符代码的一部分。大小写将被忽略。

通过以 C-x 8 开头的命令，可以插入一些常见的 Unicode 字符。例如，C-x 8 \[插入"`"，它是 Unicode 代码点 U+2018 左单引号标记，有时也称为左单引号 "弯引号 "或 "卷曲引号"。同样，C-x 8 \]、C-x 8 { 和 C-x 8 } 分别插入"'"、""和""。此外，Alt 键的作用类似于 C-x 8（除非后面有 RET）；例如，A-\[的作用类似于 C-x 8 \[并插入。*要查看哪些字符具有 C-x 8 速记功能，请键入 C-x 8 C-h。*

或者，也可以使用 C-x 8 RET（插入-字符）命令。该命令使用微型缓冲区提示输入字符的 Unicode 名称或码位。如果输入的是名称，命令将提供补全（参见补全）。如果输入码位，应为十六进制数（Unicode 的约定），或具有指定弧度的数字，例如 #o23072（八进制）；参见《Emacs Lisp 参考手册》中的整数基础知识。然后，命令将相应的字符插入缓冲区。

例如，以下命令都插入了相同的字符：

```
C-x 8 RET left single quotation mark RET
C-x 8 RET left sin TAB RET
C-x 8 RET 2018 RET
C-x 8 [
A-[  (if the Alt key works)
`    (in Electric Quote mode)
```

C-q 或 C-x 8 ...的数字参数指定插入字符的份数（参见数字参数）。

作为 C-x 8 的替代方法，可以通过键入 C-u C-x \\ iso-transl RET 选择相应的暂存输入法，然后通过键入 C-x \\ \[临时激活该暂存输入法，将插入相同的字符 ' （参见暂存输入法）。

此外，在某些情况下，如果输入使用重音符号和撇号的引号 "like this"，即使没有 C-x 8 命令，也会转换为使用单引号的 "like this "形式。同样，使用双重音符号和撇号键入引号 "like this"，也会转换为使用双引号的 "like this "形式。参见引号。

更改点的位置

除了插入字符外，还必须知道如何移动点（请参阅 "点"）。键盘命令 C-f、C-b、C-n 和 C-p 可分别向右、向左、向下和向上移动点。您也可以使用大多数键盘上的方向键移动点： 不过，许多 Emacs 用户发现，使用方向键比使用控制键要慢，因为你需要将手移到键盘上这些键所在的区域。

你也可以点击鼠标左键，将点移动到所点击的位置。Emacs 还提供了多种额外的键盘命令，可以以更复杂的方式移动点。

C-f : Move forward one character (forward-char). **RIGHT** : This command (right-char) behaves like C-f, except when point is in a right-to-left paragraph (see Bidirectional Editing). **C-b** : Move backward one character (backward-char). **LEFT** : This command (left-char) behaves like C-b, except if the current paragraph is right-to-left (see Bidirectional Editing). **C-n DOWN** : Move down one screen line (next-line). This command attempts to keep the horizontal position unchanged, so if you start in the middle of one line, you move to the middle of the next. **C-p UP** : Move up one screen line (previous-line). This command preserves position within the line, like C-n. **C-a Home** : Move to the beginning of the line (move-beginning-of-line). **C-e End** : Move to the end of the line (move-end-of-line). **M-f** : Move forward one word (forward-word). See Words. **C-RIGHT M-RIGHT** : This command (right-word) behaves like M-f, except it moves backward by one word if the current paragraph is right-to-left. See Bidirectional Editing. **M-b** : Move backward one word (backward-word). See Words. **C-LEFT M-LEFT** : This command (left-word) behaves like M-b, except it moves forward by one word if the current paragraph is right-to-left. See Bidirectional Editing. **M-r** : Without moving the text on the screen, reposition point on the left margin of the center-most text line of the window; on subsequent consecutive invocations, move point to the left margin of the top-most line, the bottom-most line, and so forth, in cyclic order (move-to-window-line-top-bottom). : A numeric argument says which screen line to place point on, counting downward from the top of the window (zero means the top line). A negative argument counts lines up from the bottom (−1 means the bottom line). See Numeric Arguments, for more information on numeric arguments. **M-<** : Move to the top of the buffer (beginning-of-buffer). With numeric argument n, move to n/10 of the way from the top. On graphical displays, C-HOME does the same. **M->** : Move to the end of the buffer (end-of-buffer). On graphical displays, C-END does the same. **C-v PageDown next** : Scroll the display one screen forward, and move point onscreen if necessary (scroll-up-command). See Scrolling. **M-v PageUp prior** : Scroll one screen backward, and move point onscreen if necessary (scroll-down-command). See Scrolling. **M-g c** : Read a number n and move point to buffer position n. Position 1 is the beginning of the buffer. If point is on or just after a number in the buffer, that is the default for n. Just type RET in the minibuffer to use it. You can also specify n by giving M-g c a numeric prefix argument. **M-g M-g M-g g** : Read a number n and move point to the beginning of line number n (goto-line). Line 1 is the beginning of the buffer. If point is on or just after a

number in the buffer, that is the default for n. Just type RET in the minibuffer to use it. You can also specify n by giving M-g M-g a numeric prefix argument. See Creating and Selecting Buffers, for the behavior of M-g M-g when you give it a plain prefix argument. Alternatively, you can use the command goto-line-relative to move point to the line relative to the accessible portion of the narrowed buffer. : goto-line has its own history list (see Minibuffer History). You can have either a single list shared between all buffers (the default) or a separate list for each buffer, by customizing the user option goto-line-history-local. **M-g TAB** : Read a number n and move to column n in the current line. Column 0 is the leftmost column. If called with a prefix argument, move to the column number specified by the argument's numeric value. **C-x C-n** : Use the current column of point as the semipermanent goal column (set-goal-column) in the current buffer. When a semipermanent goal column is in effect, C-n, C-p, <prior> and <next> always try to move to this column, or as close as possible to it, after moving vertically. The goal column remains in effect until canceled. **C-u C-x C-n** : Cancel the goal column. Henceforth, C-n and C-p try to preserve the horizontal position, as usual.

When a line of text in the buffer is longer than the width of the window, Emacs usually displays it on two or more screen lines, a.k.a. visual lines. For convenience, C-n and C-p move point by screen lines, as do the equivalent keys down and up. You can force these commands to move according to logical lines (i.e., according to the text lines in the buffer) by setting the variable line-move-visual to nil; if a logical line occupies multiple screen lines, the cursor then skips over the additional screen lines. For details, see Continuation Lines. See Variables, for how to set variables such as line-move-visual.

Unlike C-n and C-p, most of the Emacs commands that work on lines work on logical lines. For instance, C-a (move-beginning-of-line) and C-e (move-end-of-line) respectively move to the beginning and end of the logical line. Whenever we encounter commands that work on screen lines, such as C-n and C-p, we will point these out.

When line-move-visual is nil, you can also set the variable track-eol to a non-nil value. Then C-n and C-p, when starting at the end of the logical line, move to the end of the next logical line. Normally, track-eol is nil.

C-n normally stops at the end of the buffer when you use it on the last line in the buffer. However, if you set the variable next-line-add-newlines to a non-nil value, C-n on the last line of a buffer creates an additional line at the end and moves down into it.

擦除文本

DEL BACKSPACE : Delete the character before point, or the region if it is active (delete-backward-char). **Delete** : Delete the character or grapheme cluster after point, or the region if it is active (delete-forward-char). **C-d** : Delete the character after point (delete-char). **C-k** : Kill to the end of the line (kill-line). **M-d** : Kill forward to

the end of the next word (kill-word). **M-DEL M-BACKSPACE** : Kill back to the beginning of the previous word (backward-kill-word).

The DEL (delete-backward-char) command removes the character before point, moving the cursor and the characters after it backwards. If point was at the beginning of a line, this deletes the preceding newline, joining this line to the previous one.

If, however, the region is active, DEL instead deletes the text in the region. See The Mark and the Region, for a description of the region.

On most keyboards, DEL is labeled BACKSPACE, but we refer to it as DEL in this manual. (Do not confuse DEL with the Delete key; we will discuss Delete momentarily.) On some text terminals, Emacs may not recognize the DEL key properly. See If DEL Fails to Delete, if you encounter this problem.

The Delete (delete-forward-char) command deletes in the opposite direction: it deletes the character after point, i.e., the character under the cursor. If point was at the end of a line, this joins the following line onto this one. Like DEL, it deletes the text in the region if the region is active (see The Mark and the Region). If the character after point is composed with following characters and displayed as a single display unit, a so-called grapheme cluster representing the entire sequence, Delete deletes the entire sequence in one go. This is in contrast to DEL which always deletes a single character, even if the character is composed.

C-d (delete-char) deletes the character after point, similar to Delete, but regardless of whether the region is active.

See Deletion, for more detailed information about the above deletion commands.

C-k (kill-line) erases (kills) a line at a time. If you type C-k at the beginning or middle of a line, it kills all the text up to the end of the line. If you type C-k at the end of a line, it joins that line with the following line.

See Killing and Moving Text, for more information about C-k and related commands.

撤销更改

C-/ C-x u C-_ : Undo one entry of the undo records—usually, one command worth (undo). (The first key might be unavailable on text-mode displays.)

Emacs records a list of changes made in the buffer text, so you can undo recent changes. This is done using the undo command, which is bound to C-/ (as well as C-x u and C-_). Normally, this command undoes the last change, moving point back to where it was before the change. The undo command applies only to changes in the buffer; you can’t use it to undo cursor motion.

On a terminal that supports the Control modifier on all other keys, the easiest way to invoke undo is with C-/, since that doesn’t need the Shift modifier. On terminals which allow only the ASCII control characters, C-/ does not exist, but for many of

them C-/ still works because it actually sends C-_ to Emacs, while many others allow you to omit the Shift modifier when you type C-_ (in effect pressing C--), making that the most convenient way to invoke undo.

Although each editing command usually makes a separate entry in the undo records, very simple commands may be grouped together. Sometimes, an entry may cover just part of a complex command.

If you repeat C-/ (or its aliases), each repetition undoes another, earlier change, back to the limit of the undo information available. If all recorded changes have already been undone, the undo command displays an error message and does nothing.

To learn more about the undo command, see Undo.

文件

Text that you insert in an Emacs buffer lasts only as long as the Emacs session. To keep any text permanently, you must put it in a file.

Suppose there is a file named test.emacs in your home directory. To begin editing this file in Emacs, type

```
C-x C-f test.emacs RET
```

Here the file name is given as an argument to the command C-x C-f (find-file). That command uses the minibuffer to read the argument, and you type RET to terminate the argument (see The Minibuffer).

Emacs obeys this command by visiting the file: it creates a buffer, copies the contents of the file into the buffer, and then displays the buffer for editing. If you alter the text, you can save the new text in the file by typing C-x C-s (save-buffer). This copies the altered buffer contents back into the file test.emacs, making them permanent. Until you save, the changed text exists only inside Emacs, and the file test.emacs is unaltered.

To create a file, just visit it with C-x C-f as if it already existed. This creates an empty buffer, in which you can insert the text you want to put in the file. Emacs actually creates the file the first time you save this buffer with C-x C-s.

To learn more about using files in Emacs, see File Handling.

帮助

If you forget what a key does, you can find out by typing C-h k (describe-key), followed by the key of interest; for example, C-h k C-n tells you what C-n does.

The prefix key C-h stands for “help”. The key F1 serves as an alias for C-h. Apart from C-h k, there are many other help commands providing different kinds of help.

See Help, for details.

空白行

Here are special commands and techniques for inserting and deleting blank lines.

C-o : Insert a blank line after the cursor (open-line). **C-x C-o** : Delete all but one of many consecutive blank lines (delete-blank-lines).

We have seen how RET (newline) starts a new line of text. However, it may be easier to see what you are doing if you first make a blank line and then insert the desired text into it. This is easy to do using the key C-o (open-line), which inserts a newline after point but leaves point in front of the newline. After C-o, type the text for the new line.

You can make several blank lines by typing C-o several times, or by giving it a numeric argument specifying how many blank lines to make. See Numeric Arguments, for how. If you have a fill prefix, the C-o command inserts the fill prefix on the new line, if typed at the beginning of a line. See The Fill Prefix.

The easy way to get rid of extra blank lines is with the command C-x C-o (delete-blank-lines). If point lies within a run of several blank lines, C-x C-o deletes all but one of them. If point is on a single blank line, C-x C-o deletes it. If point is on a nonblank line, C-x C-o deletes all following blank lines, if any exists.

延续线

Sometimes, a line of text in the buffer—a logical line—is too long to fit in the window, and Emacs displays it as two or more screen lines, or visual lines. This is called line wrapping or continuation, and the long logical line is called a continued line. On a graphical display, Emacs indicates line wrapping with small bent arrows in the left and right window fringes. On a text terminal, Emacs indicates line wrapping by displaying a ‘\\’ character at the right margin.

Most commands that act on lines act on logical lines, not screen lines. For instance, C-k kills a logical line. As described earlier, C-n (next-line) and C-p (previous-line) are special exceptions: they move point down and up, respectively, by one screen line (see Changing the Location of Point).

Emacs can optionally truncate long logical lines instead of continuing them. This means that every logical line occupies a single screen line; if it is longer than the width of the window, the rest of the line is not displayed. On a graphical display, a truncated line is indicated by a small straight arrow in the right fringe; on a text terminal, it is indicated by a ‘$’ character in the right margin. See Line Truncation.

By default, continued lines are wrapped at the right window edge. Since the wrapping may occur in the middle of a word, continued lines can be difficult to read. The usual solution is to break your lines before they get too long, by inserting newlines. If you prefer, you can make Emacs insert a newline automatically when a line gets too long, by using Auto Fill mode. See Filling Text.

Sometimes, you may need to edit files containing many long logical lines, and it may not be practical to break them all up by adding newlines. In that case, you can use Visual Line mode, which enables word wrapping: instead of wrapping long lines exactly at the right window edge, Emacs wraps them at the word boundaries (i.e., space or tab characters) nearest to the right window edge. Visual Line mode also redefines editing commands such as C-a, C-n, and C-k to operate on screen lines rather than logical lines. See Visual Line Mode.

光标位置信息

Here are commands to get information about the size and position of parts of the buffer, and to count words and lines.

M-x what-line : Display the line number of point. **M-x line-number-mode M-x column-number-mode** : Toggle automatic display of the current line number or column number. See Optional Mode Line Features. If you want to have a line number displayed before each line, see Customization of Display. **M-=** : Display the number of lines, sentences, words, and characters that are present in the region (count-words-region). See The Mark and the Region, for information about the region. **M-x count-words** : Display the number of lines, sentences, words, and characters that are present in the buffer. If the region is active (see The Mark and the Region), display the numbers for the region instead. **C-x =** : Display the character code of character after point, character position of point, and column of point (what-cursor-position). **M-x hl-line-mode** : Enable or disable highlighting of the current line. See Displaying the Cursor. **M-x size-indication-mode** : Toggle automatic display of the size of the buffer. See Optional Mode Line Features.

M-x what-line displays the current line number in the echo area. This command is usually redundant because the current line number is shown in the mode line (see The Mode Line). However, if you narrow the buffer, the mode line shows the line number relative to the accessible portion (see Narrowing). By contrast, what-line displays both the line number relative to the narrowed region and the line number relative to the whole buffer.

M-= (count-words-region) displays a message reporting the number of lines, sentences, words, and characters in the region (see The Mark and the Region, for an explanation of the region). With a prefix argument, C-u M-=, the command displays a count for the entire buffer.

The command M-x count-words does the same job, but with a different calling convention. It displays a count for the region if the region is active, and for the buffer otherwise.

The command C-x = (what-cursor-position) shows information about the current cursor position and the buffer contents at that position. It displays a line in the echo area that looks like this:

```
Char: c (99, #o143, #x63) point=28062 of 36168 (78%) column=53
```

After ‘Char:’, this shows the character in the buffer at point. The text inside the parenthesis shows the corresponding decimal, octal and hex character codes; for more information about how C-x = displays character information, see Introduction to International Character Sets. After ‘point=’ is the position of point as a character count (the first character in the buffer is position 1, the second character is position 2, and so on). The number after that is the total number of characters in the buffer, and the number in parenthesis expresses the position as a percentage of the total. After ‘column=’ is the horizontal position of point, in columns counting from the left edge of the window.

If the user option what-cursor-show-names is non-nil, the name of the character, as defined by the Unicode Character Database, is shown as well. The part in parentheses would then become:

```
(99, #o143, #x63, LATIN SMALL LETTER C)
```

If the buffer has been narrowed, making some of the text at the beginning and the end temporarily inaccessible, C-x = displays additional text describing the currently accessible range. For example, it might display this:

```
Char: C (67, #o103, #x43) point=252 of 889 (28%) <231-599> column=0
```

where the two extra numbers give the smallest and largest character position that point is allowed to assume. The characters between those two positions are the accessible ones. See Narrowing.

Related, but different feature is display-line-numbers-mode (see Customization of Display).

数值参数

In the terminology of mathematics and computing, argument means “data provided to a function or operation”. You can give any Emacs command a numeric argument (also called a prefix argument). Some commands interpret the argument as a repetition count. For example, giving C-f an argument of ten causes it to move point forward by ten characters instead of one. With these commands, no argument is equivalent to an argument of one, and negative arguments cause them to move or act in the opposite direction.

The easiest way to specify a numeric argument is to type a digit and/or a minus sign while holding down the Meta key. For example,

```
M-5 C-n
```

moves down five lines. The keys M-1, M-2, and so on, as well as M--, are bound to commands (digit-argument and negative-argument) that set up an argument for the next command. M-- without digits normally means −1.

If you enter more than one digit, you need not hold down the Meta key for the second and subsequent digits. Thus, to move down fifty lines, type

```
M-5 0 C-n
```

Note that this does not insert five copies of ‘0’ and move down one line, as you might expect—the ‘0’ is treated as part of the prefix argument.

(What if you do want to insert five copies of ‘0’? Type M-5 C-u 0. Here, C-u terminates the prefix argument, so that the next keystroke begins the command that you want to execute. Note that this meaning of C-u applies only to this case. For the usual role of C-u, see below.)

Instead of typing M-1, M-2, and so on, another way to specify a numeric argument is to type C-u (universal-argument) followed by some digits, or (for a negative argument) a minus sign followed by digits. A minus sign without digits normally means −1.

C-u alone has the special meaning of “four times”: it multiplies the argument for the next command by four. C-u C-u multiplies it by sixteen. Thus, C-u C-u C-f moves forward sixteen characters. Other useful combinations are C-u C-n, C-u C-u C-n (move down a good fraction of a screen), C-u C-u C-o (make sixteen blank lines), and C-u C-k (kill four lines).

You can use a numeric argument before a self-inserting character to insert multiple copies of it. This is straightforward when the character is not a digit; for example, C-u 6 4 a inserts 64 copies of the character ‘a’. But this does not work for inserting digits; C-u 6 4 1 specifies an argument of 641. You can separate the argument from the digit to insert with another C-u; for example, C-u 6 4 C-u 1 does insert 64 copies of the character ‘1’.

Some commands care whether there is an argument, but ignore its value. For example, the command M-q (fill-paragraph) fills text; with an argument, it justifies the text as well. (See Filling Text, for more information on M-q.) For these commands, it is enough to specify the argument with a single C-u.

Some commands use the value of the argument as a repeat count but do something special when there is no argument. For example, the command C-k (kill-line) with argument n kills n lines, including their terminating newlines. But C-k with no argument is special: it kills the text up to the next newline, or, if point is right at the end of the line, it kills the newline itself. Thus, two C-k commands with no arguments can kill a nonblank line, just like C-k with an argument of one. (See Killing and Moving Text, for more information on C-k.)

A few commands treat a plain C-u differently from an ordinary argument. A few others may treat an argument of just a minus sign differently from an argument of −1. These unusual cases are described when they come up; they exist to make an individual command more convenient, and they are documented in that command’s documentation string.

We use the term prefix argument to emphasize that you type such arguments before the command, and to distinguish them from minibuffer arguments (see The Minibuffer), which are entered after invoking the command.

On graphical displays, C-0, C-1, etc. act the same as M-0, M-1, etc.

重复命令

Many simple commands, such as those invoked with a single key or with M-x command-name RET, can be repeated by invoking them with a numeric argument that serves as a repeat count (see Numeric Arguments). However, if the command you want to repeat prompts for input, or uses a numeric argument in another way, that method won't work.

The command C-x z (repeat) provides another way to repeat an Emacs command many times. This command repeats the previous Emacs command, whatever that was. Repeating a command uses the same arguments that were used before; it does not read new arguments each time.

To repeat the command more than once, type additional z's: each z repeats the command one more time. Repetition ends when you type a character other than z or press a mouse button.

For example, suppose you type C-u 2 0 C-d to delete 20 characters. You can repeat that command (including its argument) three additional times, to delete a total of 80 characters, by typing C-x z z z. The first C-x z repeats the command once, and each subsequent z repeats it once again.

You can also activate repeat-mode which allows repeating commands bound to sequences of two or more keys by typing a single character. For example, after typing C-x u (undo, see Undo) to undo the most recent edits, you can undo many more edits by typing u u u…. Similarly, type C-x o o o… instead of C-x o C-x o C-x o… to switch to the window several windows away. This works by entering a transient repeating mode after you type the full key sequence that invokes the command; the single-key shortcuts are shown in the echo area.

Only some commands support repetition in repeat-mode; type M-x describe-repeat-maps RET to see which ones.

The single-character shortcuts enabled by the transient repeating mode do not need to be identical: for example, after typing C-x {, either { or } or ^ or v, or any series that mixes these characters in any order, will resize the selected window in respective ways. Similarly, after M-g n or M-g p, typing any sequence of n and/or p in any mix will repeat next-error and previous-error to navigate in a *compilation*\ or *grep*\ buffer (see Compilation Mode).

Typing any key other than those defined to repeat the previous command exits the transient repeating mode, and then the key you typed is executed normally. You can also define a key which will exit the transient repeating mode without executing the

key which caused the exit. To this end, customize the user option repeat-exit-key to name a key; one natural value is RET. Finally, it’s possible to break the repetition chain automatically after some amount of idle time: customize the user option repeat-exit-timeout to specify the idle time in seconds after which this transient repetition mode will be turned off automatically.

命令行解释器

本章解决命令行解释器的替换：有哪些可选、怎么改默认、怎么卸载，登录 Shell 与配置文件的关系；随后是 Bash、Zsh、Fish 的配置与技巧，以及终端彩色输出。

必读《命令行解释器》，其余各篇默认你已会换默认 Shell、知道配置在哪；再读《Bash》，其调用与别名可通读。换用 Zsh 或 Fish 的直接进对应长文，启动文件与配置不可跳，技巧可略读；《Bash/函数》《在终端输出彩色》按需查，《Oil Shell》《PowerShell》是短条目。最常踩的坑是改完默认 Shell 就收工：问题多半在登录 Shell 与配置文件，写错等于没换。

4.1　Oil Shell

难度 入门 | **篇幅** 1.9 KB | **所属** 命令行解释器

本节将讲到

- 安装
- 卸载
- 疑难解答
- 参见

Oil Shell (OSH) 是一个兼容 Bash 的 UNIX 命令行 shell。OSH 可以在大多数类 UNIX 的操作系统上运行，包括 GNU/Linux。它是用 Python (v2.7) 编写的，但有本地可执行程序。OSH 所支持的 Bash 方言被称为 OSH 语言。

安装

安装 oil包 包。

冒烟测试

通过在终端中运行以下命令，确保 OSH 已正确安装：

```
$ osh
```

这将启动一个 OSH 会话并显示一个 shell 提示：

```
osh$
```

识别已安装的二进制文件并尝试在 OSH 会话中调用它以确认输出正确。

例如：

```
osh$ ls
...
```

使 OSH 成为您的默认 shell

参见命令行解释器#更改默认 Shell。

卸载

参见命令行解释器#卸载 Shell。

疑难解答

可以在 Github 上报告可重现的错误/问题。 提交报告时，请包括 OSH 在详细模式下运行时的输出。启用详细模式需要执行以下命令：

```
$> export OVM_VERBOSE=1
```

参见

- Oil Github
- Oil 项目主页
- Oil 推特提要
- Oil Subreddit

4.2 PowerShell

难度 入门 | **篇幅** 3.2 KB | **所属** 命令行解释器

本节将讲到

- 安装
- 遥测
- 通过 SSH 进行 PowerShell 远程管理
- 参见

翻译状态：

- 本文（或部分内容）译自 PowerShell，最近一次同步于 2026-7-26，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

PowerShell 是微软开发的跨平台任务自动化框架和脚本语言。它专为系统管理员和高级用户设计，通过命令行 shell 和脚本语言促进管理任务的自动化，从而实现对 Microsoft Windows 和其他操作系统的高效管理。

安装

安装 powershellAUR 或者 powershell-binAUR。

PowerShell Core 也可以通过 .NET SDK 安装（如果已经配置）。或者，也可以通过 Snap 包管理器来安装 PowerShell。在 Snapcraft 中，它以 powershell 的名称被打包。

安装完成后，可以通过 `pwsh` 命令来启动 PowerShell。强烈建议通过 `Update-Help` 命令来从互联网上获取 PowerShell 的帮助文件。

遥测

默认情况下，PowerShell 会收集遥测数据。可以通过设置以下环境变量来禁用它：

```
POWERSHELL_TELEMETRY_OPTOUT=1
```

通过 SSH 进行 PowerShell 远程管理

PowerShell 具有通过 `PSSession` cmdlet 内置的远程管理系统。该系统可以运行于 SSH 之上。为了为此配置 Linux SSH 主机，OpenSSH 服务器配置文件 `sshd_config` 需要包含以下行：

```
/etc/ssh/sshd_config
Subsystem powershell /usr/bin/pwsh -sshs
```

之后就可以使用 `New-PSSession`、`Enter-PSSession` 和 `Invoke-Command` cmdlet 来控制主机。这也可以在 Linux 到 Windows 或反之亦然之间进行。有关使用示例，请参阅 Microsoft 文档。

参见

- Pleau, Rob. "Powershell + Arch Linux = AWESOME!". Retrieved 2024-02-13.
- Diaz, Kevin. "Powershell Remote Management From Linux". Retrieved 2024-02-13.
- "PowerShell remoting over SSH". Retrieved 2025-03-09.
- Getting Started with PowerShell 3.0 series - Microsoft Learn. Retrieved 2025-07-23.

4.3 Bash/函数

难度 进阶 | **篇幅** 5.9 KB | **所属** 命令行解释器

本节将讲到

- 显示错误代码
- 动态编译和执行 C 源代码
- 提取
- cd 与 ls 合二为一
- 简单的记录者

Bash 也支持函数。将函数添加到 `~/.bashrc`，或从 `~/.bashrc` source 的单独文件。更多 Bash 函数示例可以在 BBS#30155 中找到。

显示错误代码

要设置 `trap` 以拦截上次运行的程序的非零返回代码，请执行以下操作：

```
EC() {
       echo -e '\e[1;33m'code $?'\e[m\n'
}
trap EC ERR
```

动态编译和执行 C 源代码

下面的函数将编译（在 `/tmp/` 目录中）并动态执行 C) 源参数（并且执行过程将不带参数）。最后，在程序终止后，将删除已编译的文件。

```
# 动态编译和执行 C 源代码
csource() {
       [[ $1 ]]     || { echo "缺少操作数" >&2; return 1; }
       [[ -r $1 ]] || { printf "文件 %s 不存在或不可读\n" "$1" >&2; return 1; }
       local output_path=${TMPDIR:-/tmp}/${1##*/};
       gcc "$1" -o "$output_path" && "$output_path";
       rm "$output_path";
       return 0;
}
```

提取

下面的函数将提取各种压缩文件类型。将其与语法 `extract <file1> <file2> ...` 一起使用

```
extract() {
    local c e i

    (($#)) || return

    for i; do
        c=''
        e=1

        if [[ ! -r $i ]]; then
            echo "$0: 文件不可读: \`$i'" >&2
            continue
        fi

        case $i in
            *.t@(gz|lz|xz|b@(2|z?(2))|a@(z|r?(.@(Z|bz?(2)|gz|lzma|xz|zst))))
                   c=(bsdtar xvf);;
            *.7z)  c=(7z x);;
            *.Z)   c=(uncompress);;
            *.bz2) c=(bunzip2);;
            *.exe) c=(cabextract);;
            *.gz)  c=(gunzip);;
            *.rar) c=(unrar x);;
            *.xz)  c=(unxz);;
            *.zip) c=(unzip);;
            *.zst) c=(unzstd);;
            *)     echo "$0: 无法识别的文件扩展名: \`$i'" >&2
                   continue;;
        esac

        command "${c[@]}" "$i"
        ((e = e || $?))
    done
    return "$e"
}
```

注意：确保 `extglob` 已启用：`shopt -s extglob`，方法是将其添加到 `~/.bashrc` 中（参见 gregswiki:glob#Options which change globbing behavior）。

另一种方法是安装专门的软件包，请参阅归档与压缩#便利工具。

cd 与 ls 合二为一

通常更改到一个目录之后会使用 `ls` 命令来列出它的内容。因此，让第二个函数同时执行这两个操作是很有帮助的。

在这个例子中，我们将它命名为 `cl`（change list），如果指定的目录不存在，则显示一条错误消息。

```
cl() {
	local dir="$1"
	local dir="${dir:=$HOME}"
	if [[ -d "$dir" ]]; then
		cd "$dir" >/dev/null; ls
	else
		echo "bash: cl: $dir: 找不到目录"
	fi
}
```

当然，可以修改 *ls* 命令以满足您的需要，例如 `ls -hall --color=auto`。

简单的记录者

```
note () {
    # 若文件不存在，则创建它
    if [[ ! -f $HOME/.notes ]]; then
        touch "$HOME/.notes"
    fi

    if ! (($#)); then
        # 无参数，印出文件
        cat "$HOME/.notes"
    elif [[ "$1" == "-c" ]]; then
        # 清除文件
        printf "%s" > "$HOME/.notes"
    else
        # 添加所有参数到文件
        printf "%s\n" "$*" >> "$HOME/.notes"
    fi
}
```

简单的任务实用程序

灵感来自#简单的记录者

```
todo() {
    if [[ ! -f $HOME/.todo ]]; then
        touch "$HOME/.todo"
    fi

    if ! (($#)); then
        cat "$HOME/.todo"
    elif [[ "$1" == "-l" ]]; then
        nl -b a "$HOME/.todo"
    elif [[ "$1" == "-c" ]]; then
        > $HOME/.todo
    elif [[ "$1" == "-r" ]]; then
        nl -b a "$HOME/.todo"
        eval printf %.0s- '{1..'"${COLUMNS:-$(tput cols)}"\}; echo
        read -p "键入要删除的数字： " number
```

```
        sed -i ${number}d $HOME/.todo "$HOME/.todo"
    else
        printf "%s\n" "$*" >> "$HOME/.todo"
    fi
}
```

计算器

```
calc() {
    echo "scale=3;$@" | bc -l
}
```

Kingbash

Kingbash - 菜单驱动的自动完成（参见 BBS#101010）。

从 AUR 安装 kingbash-gb-gitAUR，然后将以下内容插入你的 `~/.bashrc`：

```
function kingbash.fn() {
    echo -n "KingBash> $READLINE_LINE" #其中 "KingBash> " 看起来最像你的 PS1，至少在长度上是这样的。
    OUTPUT=$(/usr/bin/kingbash "$(compgen -ab -A function)")
    READLINE_POINT=$(echo "$OUTPUT" | tail -n 1)
    READLINE_LINE=$(echo "$OUTPUT" | head -n -1)
    echo -ne "\r\e[2K"
}
bind -x '"\t":kingbash.fn'
```

IP 信息

有关 bash 中的 IP 地址或主机名的详细信息，通过 https://ipinfo.io 查询：

```
ipif() {
    if grep -P "(([1-9]\d{0,2})\.){3}(?2)" <<< "$1"; then
        curl ipinfo.io/"$1"
    else
       ipawk=($(host "$1" | awk '/address/ { print $NF }'))
       curl ipinfo.io/${ipawk}
    fi
    echo
}
```

注意：Bash 仅限于扩展正则表达式；此示例在 *grep* 中使用 perl 正则表达式。

4.4 命令行解释器

难度 进阶 | **篇幅** 13.8 KB | **所属** 命令行解释器

本节将讲到

- Shell 列表
- 更改默认 Shell
- 卸载 Shell
- 登录 Shell
- 配置文件

翻译状态：

- 本文（或部分内容）译自 Command-line shell，最近一次同步于 2025-10-24，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。

- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

根据 Wikipedia:

: Unix shell 是为 Unix 和类 Unix 系统提供传统用户界面的命令行解释器或 shell，通过执行用户输入的命令文本，或包含命令的文本脚本来指导计算机的运行。

Shell 列表

或多或少符合 POSIX 的 Shell 列在 #符合 POSIX 下，语法不同的 Shell 列在 #其他 Shell 下。

符合 POSIX

可以从 `/usr/bin/sh` 链接到这些 Shell。当使用 `sh` 名称调用 Bash、mkshAUR 和 zsh 时，它们会自动变得更符合 POSIX。

- **Bash** — Bash 扩展了 Bourne shell，支持命令行历史和补全、索引和关联数组、整数算术、进程替换、here document、正则表达式匹配和大括号展开。

: https://www.gnu.org/software/bash/ || bash包

- **Dash** — Almquist SHell (ash) 的 NetBSD 版本的后代。快速且符合 POSIX 的 Shell，以尽可能小为目标。

: http://gondor.apana.org.au/~herbert/dash/ || dash包

- **KornShell（ksh）** — KornShell 语言是完整、强大的高级编程语言，通常比其他高级语言更容易快捷。这使得它特别适合于原型开发。Ksh 拥有 Bourne shell 和 C shell 的最佳特性，以及独有的许多新特性。因此，无论是系统交互，还是编程，ksh 都可以大大提高工作效率和质量。Ksh 程序更容易编写，并且比用 C 等低级语言编写的程序更简洁和可读。

: http://www.kornshell.com || ksh包

- **nsh** — 类似于 Fish 的命令行 shell，但兼容 POSIX。

: https://github.com/nuta/nsh || nsh-gitAUR

- **Oil Shell（OSH）** — Oil Shell 是兼容 Bash 的 UNIX 命令行 Shell。OSH 可以在大多数类 UNIX 操作系统上运行，包括 GNU/Linux。它是用 Python (v2.7) 编写的，但附带了一个原生可执行文件。OSH 使用的 Bash 方言称为 OSH 语言。

: https://oils.pub/ || oil包

- **Yash** — Yet another shell（另一个 shell）是用 C99（ISO/IEC 9899:1999）编写的符合 POSIX 的命令行 shell。Yash 旨在成为最符合 POSIX 的 shell，同时支持日常交互和脚本使用。

: https://magicant.github.io/yash/ || yashAUR

- **Zsh** — 为交互式使用而设计的 Shell，同时也是一门强大的脚本语言。Bash、ksh 和 tcsh 的许多有用特性都被合并到 Zsh 中，还添加了许多原创特性。介绍性文档详细介绍了 Zsh 的一些特性。

: https://www.zsh.org/ || zsh包

提示：可以用 shellcheck包 lint POSIX 和 Bash 脚本。

其他 Shell

- **C shell (tcsh)** — 命令语言解释器，既可用作交互式登录 shell，也可用作 shell 脚本命令处理器。其包含命令行编辑器，可编程单词补全， 拼写修正，历史机制，任务控制和类 C 句法。

: https://www.tcsh.org || tcsh包

- **Closh** — 基于 Clojure 的类 Bash shell。

: https://github.com/dundalek/closh || closhAUR

- **Elvish** — Elvish 是一个现代、富有表现力的 shell。可以通过管道传输内部结构值。这一功能可以避免大量复杂的文本处理代码。它是一种表现力丰富的编程语言，具有异常、命名空间和匿名函数等功能。它具有强大的读入行功能，可在键入时检查语法，默认情况下还具有语法高亮功能。

: https://elv.sh || elvish包

- **fish** — 智能、用户友好型命令行 shell。Fish 采用全色命令行句法高亮，也对命令及其参数、文件存在性和历史提供高亮和补全。其也支持对历史和命令的输入实时补全。Fish 还能够解析系统 man 手册页面，以决定命令的有效参数，从而实现高亮和补全。可以采用 `Alt+Up` 对上一命令进行修改。fish 守护程序（fishd）对历史和通用且持续的环境变量，在所有 fish 实例间进行同步。而且，fish 具有极度简洁的编程句法和类似 ruby 的控制流。参见教程以获得更多信息。

: https://fishshell.com/ || fish包

- **ion** — Ion 是一个现代系统 shell，具有简洁但强大的句法。全部使用 Rust 编写，这极大提升了 shell 的整体质量和安全性，消除了类似于 ShellShock 的漏洞的可能，也简化了开发。利用其功能，Ion 还能提供超越 Dash 的性能水平。尽管它伴随且主要为了 RedoxOS 开发，在其它 *nix 平台上也完全可用。参见用户手册以获得更多信息.

: https://gitlab.redox-os.org/redox-os/ion/ || ion-gitAUR

- **Murex** — 改进功能和增强用户体验的命令行 shell。

: https://murex.rocks/ || murexAUR

- **nushell** — Nu 从函数式编程语言、现代命令行接口工具中汲取灵感。其不将文件和服务视为纯文本本流，而是将输入视为有结构之物。

: https://www.nushell.sh || nushell包

- **Oh** — Go 编写的 Unix shell。其主旨类似其他 Unix shell，但细节有所不同。在没有牺牲 shell 的交互功能的条件下，Oh 扩展了 shell 的编程语言功能。

: https://github.com/michaelmacinnis/oh || oh-gitAUR

- **PowerShell** — PowerShell 是一种面向对象的编程语言和交互式命令行 shell。原先为 Windows 编写且被其独占。后来被开源、移植到 Linux 和 MacOS 上。

: https://github.com/PowerShell/PowerShell || powershellAUR

- **rc** — Plan 9 的命令解释器，功能与 UNIX 的 Bourne shell 类似，也增加了一些小功能，语法也不那么独树一帜。

: https://doc.cat-v.org/plan_9/4th_edition/papers/rc || 9base包

- **xonsh** — 由 Python 驱动的 shell，附加 Bash 和 IPython 中常用的 shell 原语。

: https://xon.sh/ || xonsh包

更改默认 Shell

在安装上面的 Shell 之后，运行其可执行文件即可在当前 Shell 中直接执行。但是，如果想在登录时直接使用，则需要更改默认 Shell。

列出所有已安装的 shell：

```
$ chsh -l
```

为当前用户设置默认 Shell：

```
$ chsh -s /full/path/to/shell
```

如果使用 systemd-homed，请运行：

```
$ homectl update --shell=/full/path/to/shell 用户名
```

其中 */full/path/to/shell* 是 `chsh -l` 提供的完整路径。

注意：新用户的默认 shell 是 Bash。要切换回 Bash，以上命令 shell 路径采用 `/usr/bin/bash` 即可。

如果现在注销并再次登录，就将使用新设置的 Shell。

提示：*chsh* 会参考 `/etc/shells`。如果最近安装的 Shell 不在列表中，可以手动将其添加到此文件中。

卸载 Shell

警告：不遵循以下步骤可能会导致用户无法访问能用的 Shell。

移除 Shell 软件包前先更改默认 Shell。

或者，修改用户数据库（英语：user database）。

对于每个设置了除 *bash* 之外的 Shell 作为登录 Shell 的用户（如果有必要，也包括 root 用户）执行这一操作。完成后就可以移除软件包。

登录 Shell

登录 Shell 是一种调用模式，在这种模式下，Shell 读取一次性初始化文件，例如系统范围的 `/etc/profile`、用户的 `~/.profile` 或其他特定于 Shell 的文件。这些文件将会设置初始环境，且会被所有从 Shell 启动的进程（包括其他非登录 Shell 或图形程序）继承。因此，其只会在会话开始时读取一次，例如，在控制台或通过 SSH 登录、使用 `--login` 参数通过 sudo 或 su 切换用户、手动调用登录 Shell（例如，通过 `bash --login`）。

可在#配置文件及其所包含的链接中查看各种初始化文件的概述。参见 Stackexchange 上的登录 Shell 和非登录 Shell 之间的区别？（英文）和为什么要用“登录”Shell 而不是“非登录”Shell？（英文）了解更多有关登录 Shell 的信息.

配置文件

要在登录时或在控制台中自动启动程序，可以使用 Shell 启动文件或目录。阅读 Shell 的文档或其对应文章，例如 Bash#配置文件或 Zsh#启动/结束文件（英语：Zsh#Startup/Shutdown files）。

Wikipedia:Unix shell#Configuration files 中有对各种 Shell 配置文件的比较。

/etc/profile

在登录时，所有兼容 Bourne Shell 的 Shell 都会读取 `/etc/profile`，而 `/etc/profile` 又会读取 `/etc/profile.d/` 中任何可读的 `*.sh` 文件：这些脚本不需要解释器指令，也不需要具有可执行权限。它们用于设置环境并定义特定于应用程序的设置。

标准化

可以使（某些）Shell 配置文件遵循相同的命名约定，并支持一些 Shell 之间的共同的配置。

请参阅相关文章和相关存储库，也请参阅xsh。

输入与输出

另见 GregsWiki 及 I/O 重定向。

- 重定向在命令执行前截断文件：因此使用

```
$ command file > file
```

命令不会达到预期效果。的确，有部分命令（例如 sed） 提供就地编辑文件的选项，但大部分不会。这时，可用 moreutils包 的 sponge(1) 命令。

- 因为 *cat* 不是 shell 内置的，所以，多数情况下，您可能发现使用重定向)更方便。例如，在脚本中或在苛求性能的情况下。事实上 `< *file*` 等价于 `cat *file*`。
- 遵循 POSIX 的 shell 支持 Here Document:

```
$ cat << EOF
one
two
three
EOF
```

- Shell 管道)默认在标准输出 stdout 上操作。要在标准错误 stderr(3) 上输出，可重定向 *stderr* 至 *stdout*：`*command* 2>&1 | *othercommand*`；或者，对于 Bash 4：`*command* |& *othercommand*`。
- 许多 GNU 核心工具接受文件作为参数。因此，例如 `grep *pattern* < *file*` 可替代为 `grep *pattern* *file*`。

参见

- Linux 中 Shell 的演变（英文）（位于 IBM developerWorks）
- terminal.sexy（英文）——终端配色方案设计器
- Hyperpolyglot（英文）——Shell 语法的并排比较
- UNIX Power Tools（英文）——一般命令行工具用法

- commandlinefu.com（英文）——命令行代码片段分享
- 应用程序列表/工具#终端模拟器

4.5 Fish

难度 进阶 | **篇幅** 14.3 KB | **所属** 命令行解释器

本节将讲到

- 安装
- 在系统中配置 fish
- 配置
- 提示与技巧
- 另请参阅

fish是“***friendly interactive shell***”的缩写，是一个“交互式的、对用户友好的命令行shell”。

fish 被有意设计成不完全与 POSIX 兼容。fish 的作者们认为 POSIX 中存在一些缺陷和矛盾，并通过 fish 简化的或不同的语法解决这些问题。因此，即使简单的 POSIX 兼容的脚本也可能需要较多的修改——甚至完全重写——才能在 fish 中运行。

安装

安装 fish包 软件包。或者安装开发版的 fish-gitAUR。

安装完成后，输入 `fish` 即可进入 fish shell。

要查看帮助文档，在 fish 中输入 `help`，文档将会在浏览器中打开。建议用户至少读完文档中“Syntax overview”这一节内容，因为 fish 的语法和其他 shell 的并不一样。

在系统中配置 fish

需要考虑的是：是把 fish 设置为默认 shell，也就是用户登录时直接进入的 shell，还是通过当前的默认 shell 启动 fish，并只在交互模式下使用 fish。这里，我们假设当前的默认 shell 是 Bash。

- 把 fish 设为**默认shell**：这种方式需要用户对 fish 的运行机制及其脚本语言有些基本的了解。用户需要把当前的初始化脚本和环境变量移动到 fish 的环境中。#将 fish 设为默认 shell 中有具体的步骤。
- 只把 fish 用作**交互式shell**：这种方法“破坏性”较小。Bash 会照常运行所有的初始化脚本，并在此基础上启动 fish。#仅将 fish 用作交互式 shell 中有具体的步骤。

将 fish 设为默认 shell

如果你决定把 fish 设为默认的 shell，首先你需要将你的用户的 shell 改为`/usr/bin/fish`。参照 Command-line shell#Changing your default shell 中的步骤。

下一步是把 Bash 的几个初始化脚本中的操作和配置用 fish 的方法和工具重写，这些脚本分别有`/etc/profile`、`~/.bash_profile`、`/etc/bash.bashrc`和`~/.bashrc`。

需要特别关注的是，在登录到 fish 中后，你需要尽快检查、调整环境变量`$PATH`的内容。在 fish 中，`$PATH`是一个*全局环境变量*：*全局*表示作用范围包含所有函数，并且会在重启的时候被清除；*环境变量*则意味着它对 fish 的所有子进程都是可见的。相较于修改直接修改 `$fish_user_path`，现在更推荐在配置文件中使用 fish_add_path 来设置路径，比如

```
$ fish_add_path -p /路径_1 /路径_2 /路径_3
```

会将这三个路径添加到搜索路径的头部。

仅将 fish 用作交互式 shell

如果不将 fish 设为默认 shell，就能照常运行 Bash 的初始化脚本。这能够保证用户当前的环境变量不受影响并且在 fish 中也能使用，因为 fish 是作为 Bash 的子进程运行的。下面是几种只把 fish 用作交互式 shell 的方法。

通过 .bashrc 启动 fish

保持默认 shell 为 Bash 不变，然后添加`exec fish`到合适的 Bash 配置文件中，比如`.bashrc`。用这种方法，Bash 会正常执行`/etc/profile`和`/etc/profile.d`中的所有配置文件。相对于后面的几种方法，这种是最通用的，因为这种方法在本机计算机和 SSH 远程计算机上都能使用。

提示：

- 在这种配置下，如要使用 Bash 而不是接着启动 fish，需要使用`bash --norc`，以防止Bash加载`~/.bashrc`之后执行`exec fish`。
- 要让类似于`bash -c 'echo test'`中的命令在 Bash 中执行而不是启动 fish，你可以将`exec fish`换为`if [ -z "$BASH_EXECUTION_STRING" ]; then exec fish; fi`。
- 为让fish知晓其是否是一个登录shell，您可以在 `~/.bashrc` 检测登录 shell 状态，并给 fish 加上 `--login` 选项。 fish 的 shell 命令 `status` 可用于显示状态。
- 在父进程不是 fish 的情况下才起动 fish：这种方法可以在更方便地切换到 Bash 的同时继续使用 `~/.bashrc` 中的配置。

```
if [[ $(ps --no-header --pid=$PPID --format=comm) != "fish" && -z
${BASH_EXECUTION_STRING} && ${SHLVL} == 1 ]]
```

```
then
shopt -q login_shell && LOGIN_OPTION='--login' || LOGIN_OPTION=''
exec fish $LOGIN_OPTION
fi
```

使用终端模拟器的选项

另一种方法是在启动终端模拟器时通过添加命令行选项来启动fish。对于大部分的终端，这个选项是 `-e`。比如，要打开 gnome-terminal 并运行fish，可以将 gnome-terminal 的快捷方式改为：

```
gnome-terminal -e fish
```

对于不支持设置 shell 的终端模拟器，比如lilyterm-gitAUR，则需要这样：

```
SHELL=/usr/bin/fish lilyterm
```

另外，有些终端可以在设置或配置文件中将 fish 设为默认 shell。

使用终端复用器的选项

要将 fish 设为 tmux 启动的默认 shell，在`~/.tmux.conf`中加入这行：

```
set-option -g default-shell "/usr/bin/fish"
```

当 *tmux* 启动时，就会自动进入 fish。

配置

fish的配置脚本保存在`~/.config/fish/config.fish`。这个文件与 `.bashrc`相似，当每次启动时，fish会执行这个文件中的命令。需要注意的是，如果需要设置长期保留的变量，应该将这个变量设为一个*全局变量*，而不是将它定义在这些配置脚本中。

用户定义的函数保存在`~/.config/fish/functions`中，文件名为`*`函数名`*.fish`。

网页界面

可以在 fish 的交互式网页配置页面中修改配色、提示符、函数、变量、历史记录和快捷键：

```
$ fish_config
```

在 IPv6 被完全禁止的环境下，网页配置界面或许不能启动。参见 和 IPv6#Disable IPv6。

命令补全

fish 可以通过解析 man 的数据生成自动补全规则，这些自动补全规则被保存在`~/.config/fish/generated_completions/`。可以运行下面的命令自动更新补齐规则。

```
$ fish_update_completions
```

你也可以在 `~/.config/fish/completions/` 目录下放置自己定义的补全规则。`/usr/share/fish/completions/` 中有更多的例子。

对 Arch Linux 来说， *pacman*, *pacman-key*, *makepkg*, *pbget*, *pacmatic* 这些命令的自动补全规则已经在内置在 fish 中了，因为 fish 在开发的时候就包含了很多命令的补全规则。fish 的内存管理足够智能，不会因为命令补全产生负面影响。

提示与技巧

命令替换

fish 没有 Bash 式的历史记录替换功能（如 `sudo !!`）。fish 的开发者在 fish faq 中建议使用交互式的方式调用历史记录：上方向键可以按整条命令回顾历史，而`Alt+上`可以回顾命令的单个参数。`Alt+S` 会在整条命令的最前面加上 `sudo`。

不过，fish wiki) 中介绍了一些替代方法。虽然这些方法提供的历史替换功能并不完整，但可用的有 `!!`（替换上一条命令）和 `!$`（替换上一条命令的最后一个参数）。

取消问候语

默认情况下，每次启动时 fish 都会打印问候语。如要不显示问候语，可以在 fish 中运行：

```
$ set -U fish_greeting
```

让 su 默认启动 fish

如果 *su* 目标用户的默认 shell 是 Bash，但是你想使用 fish 作为 shell 的话，可以添加一个 `su` 函数覆盖默认的 *su* 命令，在切换用户的时候自动使用 fish：

```
function su
    /bin/su --shell=/usr/bin/fish $argv
end
```

登录 fish 时自动启动 X

把下面配置添加到 `~/.config/fish/config.fish`，即可在登录 tty1 的时候自动启动 X。

```
# Start X at login
if status --is-login
    if test -z "$DISPLAY" -a $XDG_VTNR = 1
        exec startx -- -keeptty
    end
end
```

如果在使用 fish 作为交互 shell，需要把例子中的 `is-login` 换为 `is-interactive`。

在提示符中增加 git 状态

当你处在一个 git 目录中时，如果你想在 fish 的提示符中显示分支和修改的相关信息，可以仿照以下的例子编写 `fish_prompt` 函数：

```
~/.config/fish/functions/fish_prompt.fish
function fish_prompt
  set -l last_status $status

  if not set -q __fish_git_prompt_show_informative_status
    set -g __fish_git_prompt_show_informative_status 1
  end
  if not set -q __fish_git_prompt_color_branch
    set -g __fish_git_prompt_color_branch brmagenta
  end
  if not set -q __fish_git_prompt_showupstream
    set -g __fish_git_prompt_showupstream "informative"
  end
  if not set -q __fish_git_prompt_showdirtystate
    set -g __fish_git_prompt_showdirtystate "yes"
  end
  if not set -q __fish_git_prompt_color_stagedstate
    set -g __fish_git_prompt_color_stagedstate yellow
  end
  if not set -q __fish_git_prompt_color_invalidstate
    set -g __fish_git_prompt_color_invalidstate red
  end
  if not set -q __fish_git_prompt_color_cleanstate
    set -g __fish_git_prompt_color_cleanstate brgreen
  end

  printf '%s%s %s%s%s%s ' (set_color $fish_color_host) (prompt_hostname) (set_color
$fish_color_cwd) (prompt_pwd) (set_color normal) (__fish_git_prompt)

  if not test $last_status -eq 0
    set_color $fish_color_error
  end
  echo -n '$ '
  set_color normal
end
```

然而，fish 已经抛弃了这种方式，参见 fish-shell git。作为替代，fish 现在内置了 Informative Git Prompt，该功能可以通过 `fish_config` 启用。

在 SSH 中用彩色显示主机名

如果需要在通过 SSH 登录到 fish 时用不同的颜色标记提示符中的主机名，可以将以下内容添加到函数 `fish_prompt` 中，或者添加到 fish 的配置文件中。这里以红色为例：

```
~/.config/fish/functions/fish_prompt.fish
...
if set -q SSH_TTY
  set -g fish_color_host brred
end
...
```

Note that the default fish prompt colors the hostname yellow when connected through SSH; additional modification to the prompt is not required for this functionality.

ssh-agent 问题

在fish中，`eval (ssh-agent)`会因为变量的设置方式而报错。一个变通的方案是使用 csh 风格的选项`-c`：

```
$ eval (ssh-agent -c)
```

"command not found" 事件函数

fish 包含了一个响应“命令未找到”（"command not found"）事件的函数`fish_command_not_found`。当遇到一个当前系统不存在的命令时，这个函数会使用 pkgfile包 在官方仓库中查找哪个软件包拥有这条命令，而如果 pkgfile 不存在则回落使用 `pacman -F` 作为替代。

从 fish 3.2.2 开始，因为 `pacman -F` 严重的卡顿问题，所以`fish_command_not_found` 不会再使用它作为回落。

可以写一个 `fish_command_not_found` 函数来替换掉 fish 自带的。比如在“命令未找到”时只打印错误消息，这样就不会有卡顿现象：

```
$ function fish_command_not_found
      __fish_default_command_not_found_handler $argv
  end
```

然后保存这个函数：

```
$ funcsave fish_command_not_found
```

从 jobs 中删除进程

当退出 *fish* 的时候，所有后台进程也会终止。为了让一个任务即使在 fish 退出了之后也继续运行，需要先输入 `disown` 命令再退出。比如说，这个例子在后台起动 Firefox 然后 `disown` 它：

```
$ firefox &
$ disown
$ exit
```

这样即使退出了 fish，Firefox 也会继续运行。请查阅 *fish* 的 disown(1) 了解更多细节。

快速设置别名

如果想要快速设置一个持久化的别名（即使 fish 退出也不会失效），可以使用这种方法：

```
$ alias FooAliasName "foo --option"
$ funcsave FooAliasName
```

而在 fish 3.0 之后的版本中，`alias` 增加了 `-s`/`--save` 选项。

```
$ alias -s FooAliasName "foo --option"
```

别名会自动保存，这样就不需要再执行 `funcsave`。如果想查看或者编辑所有的函数，可以运行 `fish_config`，然后在网页界面的 *Function* 标签下进行配置。

如要获取更多文档，查阅 alias - create a function — fish-shell。

Source /etc/profile on login

与 `bash` 不同，`fish` 在 tty 登录时不会加载 `/etc/profile`。如果你需要加载这个文件以获取在 `/etc/profile.d` 目录中追加和声明的环境变量，可以将以下内容添加到你的配置文件中：

```
~/.config/fish/config.fish
# 使用 bash 加载 /etc/profile
if status is-login
exec bash -c "test -e /etc/profile && source /etc/profile;\
exec fish"
end
```

这样，你可以在登录时使用 `fish` 作为你的登录 shell，同时仍然拥有通常在 `bash` 登录会话中所有的环境变量。

另请参阅

- https://fishshell.com/ - 主页
- https://fishshell.com/docs/current/ - 文档
- https://hyperpolyglot.org/unix-shells - Shell 语法对照表（Shell 的“罗塞塔石碑“）
- https://github.com/fish-shell/fish-shell - fish 的 GitHub 仓库

4.6 Bash

难度 进阶 | **篇幅** 17.0 KB | **所属** 命令行解释器

本节将讲到

- 调用
- 命令行
- 别名
- 提示与技巧
- 错误排除

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：文章内许多链接需要调整。（在Talk:Bash讨论）

Bash (Bourne-again Shell) 是一个来自 GNU 的命令行解释器/编程语言。它的名字是向它的前身——很早以前的 Bourne shell 致敬。Bash可以运行在大部分类 UNIX 操作系统中，包括 GNU/Linux。

Bash是Arch Linux的默认命令行解释器。

调用

Bash 的运行方式会取决于 Bash 被调用的方式。下面是一些不同模式的描述。

如果 Bash 以TTY中的`login`、SSH 守护进程、或者其它类似的方式派生出来，我们称之为登录 (login) shell。你可以使用命令行选项 `-l` 或 `--login` 来使用这种模式。

如果 Bash 的标准输入输出和标准错误输出都连接到终端(比如说，一个终端模拟器)，并且在启动的时候既没有使用 `-c` 选项和非选项参数(比如说，`bash **script**`)，我们称之为交互 (interactive) shell。所有的交互式 shell 都会执行`/etc/bash.bashrc` 和 `~/.bashrc` 文件中的配置，而登录shell还会执行`/etc/profile` 和 `~/.bash_profile` 中的配置。

注意：在 Arch Linux 中 `/bin/sh` (过去是 Bourne shell 的执行文件名) 是 `bash` 的符号链接。如果 Bash 通过 `sh` 方式调用，它会尽量模拟历史上 `sh` 的启动行为，包括 POSIX 兼容能力。

通过在Bash启动时使用 `--posix` 命令行参数或者在启动后执行 '`set -o posix`' 来使Bash在增强的POSIX标准下运行。

配置文件

Bash会在启动时按照不同的启动方式执行一系列启动文件。详细描述请参考GNU Bash指南中`/usr/share/doc/bash/bashref.html` (Bash启动文件)这一节。

非登录

`/etc/profile`	加载全部储存在 `/etc/profile.d/*.sh` 和 `/etc/bash.bashrc` 中的配置。	是	否
`~/.bash_profile`	针对每个用户，紧接 `/etc/profile` 执行。如果这个文件不存在，会顺序检查 `~/.bash_login` 和 `~/.profile` 文件。框架文件 `/etc/skel/.bash_profile` 同时会引用 `~/.bashrc`。	是	否
`~/.bash_logout`	针对每个用户，退出登录 shell 后。	是	否
`/etc/bash.bash_logout`	取决于 `-DSYS_BASH_LOGOUT="/etc/bash.bash_logout"` 编译标记。退出登录 shell 后。	是	否
`/etc/bash.bashrc`	取决于编译标志 `-DSYS_BASHRC="/etc/bash.bashrc"`。加载 `/usr/share/bash-completion/bash_completion` 配置。	否	是
`~/.bashrc`	针对每个用户，在 `/etc/bash.bashrc` 后加载。	否	是

注意：

- 如果以 `--login` 调用，登录 shell 可能不是交互式的。

- 如果可以交互，*非登录* shell **不会** 加载 `~/.bash_profile`。它会继承调用他们的父进程（可能是一个登录 shell）的环境参数。更多信息，请参考GregsWiki:ProcessManagement#On processes, environments and inheritance。

Shell 与环境变量

Bash的行为和通过它启动的程序会被许多环境变量影响。环境变量用于储存有用的值，比如命令搜索路径，或者默认浏览器。当一个新的 shell 或者脚本被启动时，这个 shell 会继承它的父进程的环境变量，从而这个 shell 会带有内部 shell 变量。

这些内部 shell 变量可以以此导出以变成环境变量：

```
VARIABLE=content
export VARIABLE
```

或者

```
export VARIABLE=content
```

环境变量依照惯例放置在`~/.profile`或者`/etc/profile`中，这样其他兼容 Bourne shell 的 shell 也可以使用。

关于更详尽的内容，您可以参考环境变量。

命令行

Bash 的命令行由一个叫做 Readline 的分离库来管理。Readline 提供了emacs 和 vi 风格的快捷键用于操作命令行，比如说，以单词为基准前后移动、删除等。管理输入历史也是 Readline 的职责。它还允许你创造宏。

Tab 键补全

Tab 键补全，提供在按下 `tab` 键后自动补全命令的功能（这个功能默认启用）。

Tab 按下的次数

可能最多需要按三次 tab 才能显示所有的补全选项。如果希望减少这个数值，请参考更快的补全操作。

常用命令的选项补全

通常来讲，Bash 中按下 tab 只会补全命令、文件名和变量。安装 bash-completion包包，并加载 `/usr/share/bash-completion/bash_completion` 文件中的配置（这个文件应该已经在Arch的/etc/bash.bashrc中加载了），可以提供更多针对常见命令的选项的 tab 补全。安装这个包后，常规的补全（比如说 `ls file.* Tab Tab`）可能会表现得不同。但是，您可以通过`compopt -o bashdefault *program*`命令来重新启用。(更多细节，请参考 and 。)

自定义命令补全

注意：使用 `complete` 功能可能与 bash-completion包 冲突。

通常来讲，Bash 中按下 tab 只会补全命令后的文件名。通过`complete -c`命令，Bash 可以规定某些命令后的补全形式为命令，比如：

```
~/.bashrc
complete -c man which
```

或通过`-cf`命令，规定补全形式为命令和文件，比如：

```
complete -cf sudo
```

更多补全形式，请参考手册bash(1) § Programmable Completion。

历史

历史补全

您可以绑定上下键来在 Bash 的历史中查找（请参考 Readline#历史 and Readline 启动文件语法)：

```
~/.bashrc
 bind '"\e[A": history-search-backward'
 bind '"\e[B": history-search-forward'
```

或者所有 Readline 程序：

```
~/.inputrc
"\e[A": history-search-backward
"\e[B": history-search-forward
```

更近的历史记录

`HISTCONTROL`变量可以避免历史记录记录特定的命令。

不记录连续重复的命令：

```
~/.bashrc
export HISTCONTROL=ignoredups
```

历史记录中重复命令（无论是否连续）只记录最后一条：

```
~/.bashrc
export HISTCONTROL=erasedups
```

不记录空格开头的命令：

```
~/.bashrc
export HISTCONTROL=ignorespace
```

不记录连续重复的命令、空格开头的命令：

```
~/.bashrc
export HISTCONTROL=ignoreboth
```

重复命令只记录最后一条，且不记录空格开头的命令：

```
~/.bashrc
export HISTCONTROL="erasedups:ignorespace"
```

更多选项，请参考手册bash(1) § HISTCONTROL。

禁用历史记录

临时禁用历史记录：

```
$ set +o history
```

现在输入的命令将不会存入`$HISTFILE`。

比如说，你现在可以执行 `printf secret | sha256sum` 来生成密码文件的散列值，或是隐藏您使用GPG的历史，如执行`gpg -eaF secret-pubkey.asc`命令。这些秘密不会被写入磁盘。

开启历史记录：

```
$ set -o history
```

禁用所有的 Bash 历史：

```
~/.bashrc or /etc/profile
export HISTSIZE=0
```

为了保险（这会永远清除所有的历史记录）：

```
$ wipe -i -l2 -x4 -p4 "$HISTFILE"
$ ln -sv /dev/null "$HISTFILE"
```

模仿 Zsh 的帮助功能

Zsh 可以在光标指向命令的时候按 `Alt+h` 来调用这个命令的手册。 相同的行为可以通过这个 Readline 绑定在 Bash 中开启：

```
~/.bashrc
run-help() { help "$READLINE_LINE" 2>/dev/null || man "$READLINE_LINE"; }
bind -m vi-insert -x '"\eh": run-help'
bind -m emacs -x     '"\eh": run-help'
```

这个操作假设你使用（默认的）Emacs 编辑模式。

Share bash history across machines

atuin包 replaces your existing shell history with an SQLite database, and records additional context for your commands. Additionally, it provides optional and fully encrypted synchronization of your history between machines, via an Atuin server.

Enable bash history timestamps (`export HISTTIMEFORMAT="%F %T "`) before syncing. Atuin works well with tools like blesh-gitAUR and cmd-wrapped to provide an enhanced terminal experience across machines.

别名

别名（alias）是可以让您用另一个字符串来替换一个字符串的命令。这个命令常常被用来缩短系统命令，或者用来将默认参数加入到常用命令中。

针对用户的别名可以保存在`~/.bashrc`, 或任意`~/.bashrc`中加载的脚本。系统级的别名(这些会影响所有用户)存放在`/etc/bash.bashrc`。别名示例可参考 。

关于函数，请参考函数。

提示与技巧

自定义提示符

参见自定义提示符。

语法高亮与自动提示

ble.sh (Bash Line Editor), packed as blesh-gitAUR, is a command line editor written in pure Bash, which is an alternative to GNU Readline. It has many enhanced features like syntax highlighting, autosuggestions, menu-completion, abbreviations, Vim editing mode, and hook functions. Other interesting features include status line, history share, right prompt, transient prompt, and xterm title.

After installing it, source it in an interactive session.

```
~/.bashrc
source /usr/share/blesh/ble.sh
```

Configurations are explained in depth in the ~/.blerc file and at the wiki. The stable bleshAUR package is also available.

找不到命令

pkgfile 提供了一个"找不到命令"的钩子，可以在输入未知命令后自动查找官方的软件包。

你需要加载这个钩子来启用它，如下：

```
~/.bashrc
source /usr/share/doc/pkgfile/command-not-found.bash
```

现在，运行一个不可用的命令将会显示如下信息：

```
$ abiword
abiword may be found in the following packages:
  extra/abiword 3.0.1-2     /usr/bin/abiword
```

注意：需要先更新 pkgfile 的数据库才能运作。更多细节，请参考pkgfile#安装。

在终端内禁用Ctrl+Z

你可以像这样包装你的命令，来关闭 `Ctrl+Z` 功能（暂停/关闭程序）。通过在这个脚本中包装命令

```
#!/bin/bash
trap "" 20
adom
```

这时如果你在玩 adomAUR 要按 `Shift+Z` 组合键时不小心按下了 `Ctrl+Z` 组合键，你的游戏就不会停止运行了，因为我们已经禁用了`Ctrl+Z`。

登出后清空屏幕

当登出虚拟终端时，清空屏幕：

```
~/.bash_logout
clear
reset
```

输入路径自动添加"cd"

Bash 可以自动在输入的一个路径前添加 `cd` 。比如说：

```
$ /etc
bash: /etc: Is a directory
```

但是如果在 `.bashrc` 文件里添加一行:

```
~/.bashrc
...
shopt -s autocd
...
```

你会得到：

```
[user@host ~]$ /etc
cd /etc
[user@host etc]$
```

自动跳转

Python脚本autojump-gitAUR 允许在用户访问最多的路径中搜索文件系统。

安装完后，加载 `/etc/profile.d/autojump.bash` 来启动这项功能。

zoxide包是`autojump`的功能性能增强版，可直接替代。

防止覆盖文件

在当前的会话中，防止 shell 输出重定向覆盖一个已有的文件：

```
$ set -o noclobber
```

这和`set -C`命令是一样的。

如果想让该用户一直生效：

```
~/.bashrc
...
set -o noclobber
```

在设定 `noclobber` 的情况下强制覆盖文件：

```
$ echo "output" >| file.txt
```

使用目录堆栈切换目录

`pushd`与`popd`以堆栈的方式控制目录切换，利于“撤销”目录切换动作。

```
 [user@host ~] pushd /tmp/dir1
 [user@host /tmp/dir1] pushd /var/lib
 [user@host/var/lib] popd
 [user@host/tmp/dir1] popd
 [user@host ~]
```

参见bash(1) § DIRSTACK。

错误排除

修正窗口大小调整时的换行

如果您调整了终端模拟器的大小，Bash 可能并没有得到大小重调的信号，你键入的文本就不会正确的换行，并且与已输入内容重叠。启用 `checkwinsize` 选项可以在每一个命令后检查窗口的大小，并按需更新 `LINES` 和 `COLUMNS` 的值来调整。

```
~/.bashrc
shopt -s checkwinsize
```

设置 ignoreeof 后 shell 仍然退出

如果您设置了 `ignoreeof` 选项，但是如果重复按下 `ctrl-D` shell 仍然会退出。因为这个选项只允许忽略 10 次连续的EOF记号（即 `Ctrl+D`）。

如果需要将这个次数调的更高，需要使用 IGNOREEOF 变量。

比如：

```
export IGNOREEOF=100
```

分析代码以检查错误

包 shellcheck包 可以分析bash脚本（以及其他脚本），显示可能存在的问题，并对优化代码质量提出意见。

shellcheck.net 网站也提供了基于此程序的相同功能。

更多信息 （英语）

- Wikipedia:Bash (Unix shell))
- Bash 参考手册，或是 `/usr/share/doc/bash/bashref.html`
- Readline 启动文件语法
- The Bourne-Again Shell - *开源软件架构*第三章
- PS1 生成器 - 通过直观的界面生成你的 .bashrc/PS1 脚本
- 有用的 .bashrc 命令

教程

- Greg's Wiki
- GregsWiki:BashGuide
- GregsWiki:BashFAQ
- Quote Tutorial

社区

- 一个 Bash 的活跃友好的IRC频道

例子

- 怎样修改 xterm 的标题

4.7 在终端输出彩色

难度 高级 | **篇幅** 29.8 KB | **所属** 命令行解释器

本节将讲到

- Background
- Applications
- Wrappers
- Shells
- Terminal emulators

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Page is partly a collection of information on different applications, partly begin to lay theory to color output process. Please contribute your knowledge, if you can.（在Talk:在终端输出彩色讨论）

翻译状态：

- 本文（或部分内容）译自 Color_output_in_console，最近一次同步于 202510-10，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： Not translated yet.（在 Talk:在终端输出彩色# 中讨论）

This page was created to consolidate colorization of CLI outputs.

Background

Escape sequences

ANSI escape sequences 定义了一种把附加信息追加在终端输出，并且颜色也是“附加信息”的一部分的方法。从最开始的八种颜色到24位全真式色彩，终端颜色在迅速的扩展。

基本颜色编码具有8种亮度颜色和8种这些颜色更亮的版本。现代终端模拟器，包括 Linux 控制台本身，可以指定翻译为更精确的 RGB 值。这种模式大多数仿真终端都是支持的。

With the advent of 256-color displays came the 256-color escape. The 256 colors are the 16 basic colors, the 216 RGB colors (laid out in a 6x6x6 cube), and 24 levels

of greyscale. Except for the first 16 colors, the scheme is usually not customizable as it has a well-defined mapping to RGB. This mode is supported by most terminal emulators. (A minority of emulators use a similar but incompatible encoding with only 88 colors. You are very unlikely to use them in practice, but they will appear in the terminfo database.)

Less commonly supported is the truecolor mode, allowing one to use 16.7 million (224) colors in RGB (each value ranging from 0 to 255).

Termcap and terminfo

Termcap and terminfo, part of ncurses包, are databases that provide information on the escape sequences terminals (usually specified by the `TERM` env-var) understand. The tput(1) and infocmp(1) commands can be used to access them from command-line.

Applications

diff

diffutils from version 3.4 includes the `--color` option (GNU mailing list).

```
$ alias diff='diff --color=auto'
```

grep

The `--color=auto` option enables color highlighting. Color codes are emitted only on standard output; not in pipes or redirection.

Color output in *grep* is also useful with regexp tasks.

Use an alias to permanently enable this option:

```
alias grep='grep --color=auto'
```

The `GREP_COLORS` variable is used to define colors, and it configures various parts of highlighting. To change the colors, find the needed ANSI escape sequence and apply it. See grep(1) § GREP COLORS for more information.

The `-n` option includes file line numbers in the output.

ip

ip(8) command from iproute2包 supports colors with `-c`/`-color` option. When using the `auto` parameter, colored output will be enabled only when stdout is a terminal.

iproute2包 can be built with `auto` as the default, but such a change has been rejected by the Arch package maintainer.

You can use an alias to enable colored output, e.g.:

```
alias ip='ip -color=auto'
```

less

Environment variables

As with the #man case, we can tell less to emit colors when it is meaning to make bold text and other formatting effects.

Add the following lines to your shell configuration file:

```
export LESS='-R --use-color -Dd+r$Du+b$'
```

It will set red for bold and blue for underlined.

For more information about the `--use-color` and `-D` options, see less(1) § D or .

Reading from stdin

注意：It is recommended to add colored output through #Environment variables to your `~/.bashrc` or `~/.zshrc`, as the below is based on `export LESS='-R'`

When you run a command and pipe its standard output (*stdout*) to *less* for a paged view (e.g. `pacman -Qe | less`), you may find that the output is no longer colored. This is usually because the program tries to detect if its *stdout* is an interactive terminal, in which case it prints colored text, and otherwise prints uncolored text. This is good behaviour when you want to redirect *stdout* to a file, e.g. `pacman -Qe > pkglst-backup.txt`, but less suited when you want to view output in `less`.

Some programs provide an option to disable the interactive tty detection:

```
# dmesg --color=always | less
```

In case that the program does not provide any similar option, it is possible to trick the program into thinking its *stdout* is an interactive terminal with the following utilities:

- **ColorThis** — Force colored output of a program by running it within a (group of) pty, support forwarding stdin.

: https://github.com/Sasasu/ColorThis || colorthis-gitAUR

- **stdoutisatty** — A small program and a `LD_PRELOAD`\-able library that catches the isatty(3) function call.

: https://github.com/lilydjwg/stdoutisatty. || stdoutisatty包 : Example: `stdoutisatty *program* | less`

- **unbuffer** — A tclsh script comes with expect, it invokes desired program within a pty.

: unbuffer(1) || expect包 : Example: `unbuffer *program* | less`

Alternatively, using zpty module from zsh:

```
~/.zshrc
zmodload zsh/zpty

pty() {
        zpty pty-${UID} ${1+$@}
        if [[ ! -t 1 ]];then
```

```
                setopt local_traps
                trap '' INT
        fi
        zpty -r pty-${UID}
        zpty -d pty-${UID}
}

ptyless() {
        pty $@ | less
}
```

Usage:

```
$ ptyless program
```

To pipe it to other pager (less in this example):

```
$ pty program | less
```

ls

The `--color=auto` option enables color highlighting. Color codes are emitted only on standard output; not in pipes or redirection.

Use an alias to permanently enable this option:

```
alias ls='ls --color=auto'
```

The `LS_COLORS` variable is used to define colors, and it configures various parts of highlighting. Use the dircolors(1) command to set it.

An advanced alternative to *dircolors* that ships with many themes is the vivid包 package, see `vivid --help` for usage.

注意：Using the `--color` option may incur a noticeable performance penalty when *ls* is run in a directory with very many entries. The default settings require *ls* to stat(1) every single file it lists. However, if you would like most of the file-type coloring but can live without the other coloring options (e.g. executable, orphan, sticky, other-writable, capability), use *dircolors* to set the `LS_COLORS` environment variable like this:

```
eval $(dircolors -p | perl -pe 's/^((CAP|S[ET]|O[TR]|M|E)\w+).*/$1 00/' | dircolors -)
```

See ls(1) for more information.

man

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：The *grotty* utility is mentioned only in the intro - where is it actually used? Also, there are more subsections than just `less` and `most`.（在Talk:在终端输出彩色讨论）

There is a real color facility in grotty(1), but it is strongly discouraged for man pages. Here we fake a colored `man` by hacking two main pagers, `less` and `most`: we replace the sequences for bold, standout, and underline with spiced ones that contain color.

Using bat

bat包 can be used as a colorizing pager for man, by setting the `MANPAGER` environment variable as documented here.

Using less

See #less for a more detailed description.

```
export MANPAGER="less -R --use-color -Dd+r -Du+b"
export MANROFFOPT="-P -c"
```

For Fish you could accomplish this with:

```
~/.config/fish/config.fish
set -xU MANPAGER 'less -R --use-color -Dd+r -Du+b'
set -xU MANROFFOPT '-P -c'
```

Remember to source your config or restart your shell to make the changes take effect.

Using most

The basic function of 'most' is similar to `less` and `more`, but it has a smaller feature set. Configuring most to use colors is easier than using less, but additional configuration is necessary to make most behave like less. Install the most包 package.

Edit `/etc/man_db.conf`, uncomment the pager definition and change it to:

```
DEFINE     pager     most -s
```

Test the new setup by typing:

```
$ man whatever_man_page
```

Modifying the color values requires editing `~/.mostrc` (creating the file if it is not present) or editing `/etc/most.conf` for system-wide changes. Example `~/.mostrc`:

```
% Color settings
color normal lightgray black
color status yellow blue
color underline yellow black
color overstrike brightblue black
```

Using X resources

A quick way to add color to manual pages viewed on xterm包/`uxterm` or rxvt-unicode包 is to modify `~/.Xresources`.

xterm

```
*VT100.colorBDMode:     true
*VT100.colorBD:         red
*VT100.colorULMode:     true
*VT100.colorUL:         cyan
```

which *replaces* the decorations with the colors. Also add:

```
*VT100.veryBoldColors: 6
```

if you want colors and decorations (bold or underline) *at the same time*. See xterm(1) § veryBoldColors for more information.

rxvt-unicode

```
URxvt.colorIT:      #87af5f
URxvt.colorBD:      #d7d7d7
URxvt.colorUL:      #87afd7
```

Run:

```
$ xrdb -load ~/.Xresources
```

Launch a new `xterm/uxterm` or `rxvt-unicode` and you should see colorful man pages.

This combination puts colors to **bold** and underlined words in `xterm/uxterm` or to **bold**, underlined, and *italicized* text in `rxvt-unicode`. You can play with different combinations of these attributes. See the sources (archived) of this item.

pacman

Pacman has a color option. Uncomment the `Color` line in `/etc/pacman.conf`.

Wrappers

**此页面或章节适合移动到 \[\[\]\]。**

附注： Some of these could be made into sections of their own, or moved to existing sections such as #diff（在 Talk:在终端输出彩色 讨论）

Universal wrappers

(most of them outdated, but still functioning)

They go with multiple preconfigured presets that can be changed, and new ones can be created/contributed.

警告：Wrappers replace output of commands with escape sequences. Some shell scripts and programs which use the output of standard shell utilities may not work correctly.

- **rainbow** — Colorize commands output or STDIN using patterns.
- Presets: df, diff, env, host, ifconfig, java-stack-trace, jboss, jonas, md5sum, mvn2, mvn3, ping, tomcat, top, traceroute.

: https://github.com/nicoulaj/rainbow || rainbowAUR

- **grc** — Yet another colouriser for beautifying your logfiles or output of commands.
- Presets: cat, cvs, df, diff, dig, gcc, g++, ls, ifconfig, make, mount, mtr, netstat, ping, ps, tail, traceroute, wdiff, blkid, du, dnf, docker, docker-machine, env, id, ip, iostat, last, lsattr, lsblk, lspci, lsmod, lsof, getfacl, getsebool, ulimit, uptime, nmap, fdisk, findmnt, free, semanage, sar, ss, sysctl, systemctl, stat, showmount, tune2fs and tcpdump.

: https://github.com/garabik/grc || grc包

- **cope** — A colourful wrapper for terminal programs.
- Presets: acpi, arp, cc, df, dprofpp, fdisk, free, g++, gcc, id, ifconfig, ls, lspci, lsusb, make, md5sum, mpc, netstat, nm, nmap, nocope, ping, pmap, ps, readelf, route, screen, sha1sum, sha224sum, sha256sum, sha384sum, sha512sum, shasum, socklist, stat, strace, tcpdump, tracepath, traceroute, w, wget, who, xrandr.

: https://github.com/yogan/cope || cope-gitAUR

- **cw** — A non-intrusive real-time ANSI color wrapper for common unix-based commands. Wraps file包 which can cause issues.
- Presets: arp, arping, auth.log@, blockdev, cal, cksum, clock, configure, cpuinfo@, crontab@, cw-pipe, cw-test.cgi, date, df, diff, dig, dmesg, du, env, figlet, file, find, finger, free, fstab@, fuser, g++, gcc, group@, groups, hdparm, hexdump, host, hosts@, id, ifconfig, inittab@, iptables, last, lastlog, lsattr, lsmod, lsof, ltrace-color, make, md5sum, meminfo@, messages@, mount, mpg123, netstat, nfsstat, nmap, nslookup, objdump, passwd@, ping, pmap, pmap_dump, praliases, profile@, protocols@, ps, pstree, quota, quotastats, resolv.conf@, route, routel, sdiff, services@, showmount, smbstatus, stat, strace-color, sysctl, syslog, tar, tcpdump, tracepath, traceroute, umount, uname, uptime, users, vmstat, w, wc, whereis, who, xferlog.

: https://cwrapper.sourceforge.net/ || cwAUR

- **ccze** — A fast log colorizer written in C, intended to be a drop-in replacement for colorize

: https://github.com/cornet/ccze/ || cczeAUR

Libraries for colorizing an output

- **libtextstyle** — A C library for styling text output to terminals

: https://www.gnu.org/software/gettext/libtextstyle/manual/index.html || gettext包

- **ruby-rainbow** — Rainbow is extension to ruby's String class adding support for colorizing text on ANSI terminal

: https://rubygems.org/gems/rainbow/ || ruby-rainbow包

- **python-blessings** — A thin, practical wrapper around terminal coloring, styling, and positioning

: https://github.com/erikrose/blessings || python-blessingsAUR

- **lolcat** — Ruby program that makes the output colorful like a rainbow

: https://github.com/busyloop/lolcat/ || lolcat包

Application specific

Compilers

- **colorgcc** — A Perl wrapper to colorize the output of compilers with warning/error messages matching the gcc output format

: https://github.com/colorgcc/colorgcc || colorgcc包

diff

Diff has built-in color output, which is reasonable to use. But the following wrappers can be used:

- **colordiff** — Perl script for *diff* highlighting.

: https://www.colordiff.org/ || colordiff包

- **cwdiff** — *(w)diff* wrapper with directories support and highlighting.

: https://github.com/junghans/cwdiff || cwdiffAUR

- **git-delta** — A syntax-highlighting pager for git and diff output.

: https://github.com/dandavison/delta || git-delta包

cat

- **bat** — Cat clone with syntax highlighting and git integration.

: https://github.com/sharkdp/bat || bat包

less

source-highlight

You can enable code syntax coloring in *less*. First, install source-highlight包, then add these lines to your shell configuration file:

```
~/.bashrc
export LESSOPEN="| /usr/bin/source-highlight-esc.sh %s"
export LESS='-R '
```

lesspipe

Frequent users of the command line interface might want to install lesspipe包.

Users may now list the compressed files inside of an archive using their pager:

```
$ less compressed_file.tar.gz
==> use tar_file:contained_file to view a file in the archive
-rw------- username/group  695 2008-01-04 19:24 compressed_file/content1
-rw------- username/group   43 2007-11-07 11:17 compressed_file/content2
compressed_file.tar.gz (END)
```

lesspipe also grants *less* the ability of interfacing with files other than archives, serving as an alternative for the specific command associated for that file-type (such as viewing HTML via python-html2text包).

Re-login after installing *lesspipe* in order to activate it, or source `/etc/profile.d/lesspipe.sh`.

Make

- **colormake** — A simple wrapper around make to make its output more readable.

: https://github.com/pagekite/Colormake/ || colormakeAUR

Ping

- **prettyping** — Add some great features to ping monitoring. A wrapper around the standard ping tool with the objective of making the output prettier, more colorful, more compact, and easier to read.

: https://denilson.sa.nom.br/prettyping/ || prettyping包

Shells

bash

See Bash/Prompt customization#Colors.

Fish

See Fish#Web interface.

xonsh

See Customizing the Prompt.

zsh

See Zsh#Colors.

Terminal emulators

Virtual console

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Lacks clarity on what "the colors" are, i.e. in #Virtual console they are about the *representations* of the 16 base colors (RGB values for yellow, red, blue, etc.), while in #Login screen they are about the base colors themselves. See also console_codes(4) and User:Isacdaavid/Linux_Console（在Talk:在终端输出彩色讨论）

The colors in the Linux virtual console running on the framebuffer can be changed. This is done by writing the escape code `\\e]PXRRGGBB`, where `X` is the hexadecimal index of the color from 0-F, and `RRGGBB` is a traditional hexadecimal RGB code.

For example, to reuse existing colors defined in `~/.Xresources`, add the following to the shell initialization file (such as `~/.bashrc`):

```
if [ "$TERM" = "linux" ]; then
    _SEDCMD='s/.*\*color\([0-9]\{1,\}\).*#\([0-9a-fA-F]\{6\}\).*/\1 \2/p'
    for i in $(sed -n "$_SEDCMD" $HOME/.Xresources | awk '$1 < 16 {printf "\\e]P%X%s",
$1, $2}'); do
        echo -en "$i"
    done
    clear
fi
```

Login screen

The below is a colored example of the virtual console login screen in `/etc/issue`. Create a backup of the original file with `mv /etc/issue /etc/issue.bak` as root, and create a new `/etc/issue`:

```
\e[H\e[2J
                                                                \e[1;30m| \e[34m\r \s
      \e[36;1m/\\\\                              \e[37m||      \e[36m| =
\e[30m|
     \e[36m/  \\\\                             \e[37m||      \e[36m|
\e[30m| \e[32m\t
    \e[1;36m/ \e[0;36m.. \e[1m\\\\   \e[37m//==\\\\\\ ||/= /==\\\\ ||/=\\\\  \e[36m| |
|/\\\\ |  | \\\\ /  \e[30m| \e[32m\d
   \e[0;36m/ .  . \\\\ \e[37m||    || ||  ||     ||  ||  \e[36m| | |  | |  |   X
\e[1;30m|
  \e[0;36m/  .  .  \\\\ \e[37m\\\\\\==/| ||   \\\\==/ ||  ||  \e[36m| | |  |\  \\/|  /
\\\\ \e[1;30m| \e[31m\U
 \e[0;36m/ ..    .. \\\\   \e[0;37mA simple, lightweight linux distribution.
\e[1;30m|
\e[0;36m/_'        `_\\\\                                          \e[1;30m|
\e[35m\l \e[0mon \e[1;33m\n
\e[0m
```

See also:

- https://bbs.archlinux.org/viewtopic.php?pid=386429#p386429
- https://www.linuxfromscratch.org/blfs/view/svn/postlfs/logon.html

X window system

Most Xorg terminals, including xterm and urxvt, support at least 16 basic colors. The colors 0-7 are the 'normal' colors. Colors 8-15 are their 'bright' counterparts, used for highlighting. These colors can be modified through X resources, or through specific terminal settings. For example:

```
~/.Xresources
! Black + DarkGrey
*color0:  #000000
*color8:  #555753
! DarkRed + Red
*color1:  #ff6565
*color9:  #ff8d8d
! DarkGreen + Green
*color2:  #93d44f
*color10: #c8e7a8
! DarkYellow + Yellow
*color3:  #eab93d
*color11: #ffc123
! DarkBlue + Blue
*color4:  #204a87
*color12: #3465a4
! DarkMagenta + Magenta
*color5:  #ce5c00
*color13: #f57900
!DarkCyan + Cyan (both not tango)
*color6:  #89b6e2
*color14: #46a4ff
! LightGrey + White
*color7:  #cccccc
*color15: #ffffff
```

警告：Color resources such as `foreground` and `background` can be read by other applications (such as emacs). This can be avoided by specifiying the class name, for example `XTerm.foreground`.

See also:

- #Using X resources for how to color bold and underlined text automatically.
- Color Themes - Extensive list of terminal color themes by Phraktured.
- Xcolors by dkeg (see files with paths matching `theme/dkeg - *theme*` in the repository)
- base16 color schemes

Setting foreground and background colors with tput

Example to set bold, yellow text, with blue background:

```
$ tput bold; tput setaf 3; tput setab 4
```

<table class="wikitable"><caption>tput command line option cheatsheet</caption><tbody><tr><th>tput command</th><th>Description</th></tr><tr><td><code>tput bold</code></td><td>Bold text</td></tr><tr><td><code>tput setaf [CODE]</code></td><td>Set foreground color</td></tr><tr><td><code>tput setab [CODE]</code></td><td>Set background color</td></tr><tr><td><code>tput rev</code></td><td>Inverse colors</td></tr><tr><td><code>tput sgr0</code></td><td>Reset everything</td></tr></tbody></table>

<table class="wikitable sortable"><caption>tput color codes</caption><tbody><tr><th>tput color [CODE]</th><th>Color</th></tr><tr><td>0</td><td>Black</td></tr><tr><td>1</td><td>Red</td></tr><tr><td>2</td><td>Green</td></tr><tr><td>3</td><td>Yellow</td></tr><tr><td>4</td><td>Blue</td></tr><tr><td>5</td><td>Magenta</td></tr><tr><td>6</td><td>Cyan</td></tr><tr><td>7</td><td>White</td></tr></tbody></table>

Example to print 256 colors across the screen:

```
$ (x=`tput op` y=`printf %76s`;for i in {0..256};do o=00$i;echo -e ${o:${#o}-3:3} `tput setaf $i;tput setab $i`${y// /=}$x;done)
```

Display tput escape codes

**本文或本章节可能需要合并到 Bash/Prompt_customization。**

附注： More context on *tput* is provided in that article（在 Talk:在终端输出彩色 中讨论）

Replace `tput op` with whatever tput you want to trace. `op` is the default foreground and background color.

```
$ ( strace -s5000 -e write tput op 2>&2 2>&1 ) | tee -a /dev/stderr | grep -o '"[^"]*"'
033[\033[1;34m"\33[39;49m"\033[00m
```

Enumerate supported colors

The following command will let you discover all the terminals you have terminfo support for, and the number of colors each terminal supports. The possible values are: 8, 15, 16, 52, 64, 88 and 256.

```
$ for T in `find /usr/share/terminfo -type f -printf '%f '`;do echo "$T `tput -T $T colors`";done|sort -nk2
Eterm-88color 88
rxvt-88color 88
xterm+88color 88
```

```
xterm-88color 88
Eterm-256color 256
gnome-256color 256
konsole-256color 256
putty-256color 256
rxvt-256color 256
screen-256color 256
screen-256color-bce 256
screen-256color-bce-s 256
screen-256color-s 256
xterm+256color 256
xterm-256color 256
```

Color scheme scripts

See for scripts which display a chart of your current terminal scheme.

True color support

Some terminals support the full range of 16 million colors (RGB, each with 8 bit resolution): xterm, konsole, st, etc. The corresponding TERM values `xterm-direct`, `konsole-direct`, `st-direct`, etc. are supported starting with ncurses version 6.1 . For more info about terminal emulators and applications that support true color, see .

Note that the Linux kernel supports the SGR (Select Graphic Rendition) escape sequences for true-color, but it is pointless to use it, because the driver maps the 24-bit color specifications to a 256-colors color map in the kernel (see the functions `rgb_foreground`, `rgb_background`). For this reason, there is no terminfo entry `linux-direct`.

See also

- lolcat clone in x64 assembly

4.8 Zsh

难度 高级 | **篇幅** 42.0 KB | **所属** 命令行解释器

本节将讲到

- 安装
- 启动/关闭文件
- 配置 Zsh
- 提示与技巧
- Remove duplicate entries

Zsh 是一款功能强大的命令行解释器（shell），既可以作为交互式终端来使用，也可以作为脚本语言解释器来使用。它在兼容 POSIX 的 sh 的同时（默认不兼容，仅在使用 `emulate sh` 时兼容），还改进了 Tab 补全和通配符等功能。

Zsh FAQ 说明了使用 Zsh 的更多原因。

安装

在开始安装之前，可以查看一下当前正在使用的 Shell 软件：

```
$ echo $SHELL
```

安装 zsh包 包，也可以同时安装 zsh-completions包 包以获取更多命令补全规则。

初始配置

在终端里面输入以下命令，确保已经正确安装了 Zsh：

```
$ zsh
```

运行后你应该会看到*zsh-newuser-install*（新用户安装向导），它可以协助你完成一些基础配置。如果要跳过这一步骤，可以按 q 退出。如果你没有看到*zsh-newuser-install*，可以手动运行以下命令：

```
$ autoload -Uz zsh-newuser-install
$ zsh-newuser-install -f
```

注意：确保你的终端尺寸至少为 72×15 大小，否则*zsh-newuser-install*将无法运行。

将 Zsh 设为默认 Shell 软件

将 Shell 改为 `/usr/bin/zsh`。参见命令行解释器#更改默认_Shell。

提示：如果要替换 bash包，用户可能希望能够将 `~/.bashrc` 文件的某些代码转移到 `~/.zshrc` 文件（例如：命令提示符和别名），以及将 `~/.bash_profile` 文件的代码转移到 `~/.zprofile` 文件（例如：启动 X Window System 的代码）。

启动/关闭文件

提示：有关交互式 shell 和登录 shell 的解释以及在启动文件中放置内容的建议，请参阅 A User's Guide to the Z-Shell（Z-Shell 用户指南，英文）。

注意：

- 如果未设置`$ZDOTDIR`，则使用`$HOME`。
- 如果在任何文件中取消设置`RCS`选项，那么之后的配置文件将不会被读取。
- 如果在任何文件中取消设置`GLOBAL_RCS`选项，那么之后的全局配置文件（`/etc/zsh/*`）将不会被读取。

在启动时，Zsh 会按以下顺序读取这些文件（如果存在）：

- `/etc/zsh/zshenv` 用于设置所有用户的环境变量。不应包含会产生输出或假设 shell 连接到 TTY 的命令。此文件一旦存在就会被读取，且无法跳过。
- `$ZDOTDIR/.zshenv` 用于设置用户的环境变量。不应包含会产生输出或假设 shell 连接到 TTY 的命令。此文件一旦存在就会被读取。

- `/etc/zsh/zprofile` 在启动为**登录 shell** 时为所有用户执行的命令。在 Arch Linux 中，此文件默认包含一行引入C1的代码。在删除该行之前，请参阅下方的警告！
- `/etc/profile` 此文件应由所有符合 POSIX 标准的 shell 在登录时引入，用于设置`$PATH`及其他环境变量，以及应用程序相关的设置（`/etc/profile.d/*.sh`）。
- `$ZDOTDIR/.zprofile` 在启动为**登录 shell** 时执行的用户命令。通常用于自动启动图形会话和设置会话范围的环境变量。
- `/etc/zsh/zshrc` 用于为所有用户设置交互式 shell 的配置并执行命令，会在启动为**交互式 shell** 时读取。
- `$ZDOTDIR/.zshrc` 用于为用户设置交互式 shell 配置并执行命令，会在启动为**交互式 shell** 时读取。
- `/etc/zsh/zlogin` 在初始进程结束时为所有用户执行的命令，会在启动为**登录 shell** 时读取。
- `$ZDOTDIR/.zlogin` 在初始进程结束时执行的用户命令，会在启动为**登录 shell** 时读取。通常用于自动启动命令行工具，不应用于启动图形会话，因为此时会话可能包含仅适用于交互式 shell 的配置。
- `$ZDOTDIR/.zlogout` 在**登录 shell** 退出时执行的用户命令。
- `/etc/zsh/zlogout` 在**登录 shell** 退出时为所有用户执行的命令。

请参阅图示说明（英文）。

注意：`$HOME/.profile`不是Zsh的启动文件的一部分，除非 zsh 以`sh`或`ksh`模式运行并作为登录 shell 启动，否则不会被 zsh 引入。有关 sh 和 ksh（英语：ksh） 兼容模式的更多详情，请参阅zsh(1) § COMPATIBILITY。

警告：不要删除`/etc/zsh/zprofile`中的默认代码，否则会破坏某些在`/etc/profile.d/`目录下提供脚本的软件包的完整性。

配置 Zsh

尽管 Zsh 开箱即用，但它的默认设置不符合大多数用户的使用习惯。但由于 Zsh 有着大量自定义选项，配置 Zsh 可能是一项艰巨而耗时的体验。有关自动配置，请参阅#第三方扩展。

简单的 .zshrc

下面是一个示例配置文件。它提供了一组不错的默认选项，并给出了许多可以自定义 Zsh 的方法的示例。要使用此配置，请将其保存为 `.zshrc`。

提示：运行 `source ~/.zshrc` 可立即应用更改，无需注销后重新登录。

```
~/.zshrc
autoload -Uz compinit promptinit
compinit
promptinit

# 将默认命令行提示设置为 walters 主题
prompt walters
```

有关命令提示主题系统的更多详细信息，请参阅#提示主题。

配置 $PATH

Zsh 将 `PATH` 变量绑定到 `path` 数组。这样，您只需修改 `path` 数组即可操作 `PATH`。有关详细信息，请参阅 A User's Guide to the Z-Shell（Z-Shell 用户指南，英文）。

往`PATH`变量中添加`~/.local/bin/`：

```
~/.zshenv
typeset -U path PATH
path=(~/.local/bin $path)
export PATH
```

命令补全

也许 Zsh 最引人注目的特性就是它先进的自动补全功能。至少，请在 `.zshrc` 中启用自动完成功能。在 `~/.zshrc` 的最后加入下面的配置开启自动补全：

```
~/.zshrc
autoload -U compinit
compinit
```

上面的补全配置包括 ssh/scp/sftp 命令中的主机名补全。但是要让这个特性正常工作，不能启用 ssh 的主机名散列（hash，即 ssh 客户端配置中的 `HashKnownHosts` 选项）。

添加下面的配置可以启动使用方向键控制的自动补全：

```
~/.zshrc
zstyle ':completion:*' menu select
```

按两次 `Tab` 启动菜单

为了实现在特权命令中（例如，当你以 sudo 开始输入一个命令时，补全脚本也会尝试确定你可以用 sudo 完成的选项）对特权环境的自动补全，添加：

```
~/.zshrc
zstyle ':completion::complete:*' gain-privileges 1
```

警告：这允许 Zsh 补全脚本以 sudo 权限运行命令。如果您使用不受信任的自动补全脚本，则不应启用此功能。

注意：这种特殊的上下文感知补全仅适用于少数命令。

自定义补全

可以自己编写自定义补全，参考 zshcompsys(1) 手册页。

请注意，官方文档可能难以阅读，可以考虑尝试更简单的 zsh-completion-howto 教程（英文），以便轻松入门。

消除历史记录中的重复条目

```
~/.zshrc
setopt HIST_IGNORE_DUPS
```

假如目前的历史记录中已经有重复条目，可以运行下面的命令清除

```
$ sort -t ";" -k 2 -u ~/.zsh_history | sort -o ~/.zsh_history
```

快捷键绑定

Zsh 不使用 readline，而是使用其自身且更为强大的 Zsh 行编辑器（Zsh Line Editor，ZLE）。它不会读取 `/etc/inputrc` 或 `~/.inputrc` 文件。阅读 A closer look at the zsh line editor and creating custom widgets 以了解 ZLE 配置。

ZLE 有 Emacs 模式和 vi 模式。如果 `VISUAL` 和 `EDITOR` 环境变量之一包含字符串 `vi`，则会使用 vi 模式，否则默认为 Emacs 模式。可以分别使用 `bindkey -e` 或 `bindkey -v` 显式指定 Emacs 模式和 vi 模式。在 vi 模式下按 Esc 键的延迟默认是 0.4 秒，你可以通过设置 `export KEYTIMEOUT=5` 将其缩短至 0.05 秒。

按键绑定是通过将与按键对应的一个转义序列映射到一个 ZLE 小部件来分配的。可用的小部件及其作用描述和默认的按键绑定可以在 zshzle(1) § STANDARD WIDGETS 和 zshcontrib(1) § ZLE FUNCTIONS 中找到。

在 Zsh 中设置按键绑定的推荐方法是使用来自 terminfo(5) 的字符串功能。例如：

```
~/.zshrc
# 创建兼容 zkbd 的哈希表；
# 要向此哈希表添加其他键，请参阅：man 5 terminfo
typeset -g -A key

key[Home]="${terminfo[khome]}"
key[End]="${terminfo[kend]}"
key[Insert]="${terminfo[kich1]}"
key[Backspace]="${terminfo[kbs]}"
key[Delete]="${terminfo[kdch1]}"
key[Up]="${terminfo[kcuu1]}"
key[Down]="${terminfo[kcud1]}"
key[Left]="${terminfo[kcub1]}"
key[Right]="${terminfo[kcuf1]}"
key[PageUp]="${terminfo[kpp]}"
key[PageDown]="${terminfo[knp]}"
key[Shift-Tab]="${terminfo[kcbt]}"

# 相应地设置键
[[ -n "${key[Home]}"      ]] && bindkey -- "${key[Home]}"      beginning-of-line
[[ -n "${key[End]}"       ]] && bindkey -- "${key[End]}"       end-of-line
[[ -n "${key[Insert]}"    ]] && bindkey -- "${key[Insert]}"    overwrite-mode
[[ -n "${key[Backspace]}" ]] && bindkey -- "${key[Backspace]}" backward-delete-char
[[ -n "${key[Delete]}"    ]] && bindkey -- "${key[Delete]}"    delete-char
[[ -n "${key[Up]}"        ]] && bindkey -- "${key[Up]}"        up-line-or-history
[[ -n "${key[Down]}"      ]] && bindkey -- "${key[Down]}"      down-line-or-history
[[ -n "${key[Left]}"      ]] && bindkey -- "${key[Left]}"      backward-char
[[ -n "${key[Right]}"     ]] && bindkey -- "${key[Right]}"     forward-char
```

```
[[ -n "${key[PageUp]}"    ]] && bindkey -- "${key[PageUp]}"     beginning-of-buffer-or-history
[[ -n "${key[PageDown]}"  ]] && bindkey -- "${key[PageDown]}"   end-of-buffer-or-history
[[ -n "${key[Shift-Tab]}" ]] && bindkey -- "${key[Shift-Tab]}"  reverse-menu-complete

# 最后，确保终端处于应用程序模式，此时 ZLE 处于
# 活动状态。只有这样，$terminfo 中的值才有效。
if (( ${+terminfo[smkx]} && ${+terminfo[rmkx]} )); then
       autoload -Uz add-zle-hook-widget
       function zle_application_mode_start { echoti smkx }
       function zle_application_mode_stop { echoti rmkx }
       add-zle-hook-widget -Uz zle-line-init zle_application_mode_start
       add-zle-hook-widget -Uz zle-line-finish zle_application_mode_stop
fi
```

查找历史记录

您需要设置 `key` 数组并确保 ZLE 进入应用程序模式以使用以下说明。请参阅 #快捷键绑定。

要启用历史搜索，请将这些行添加到 `.zshrc` 文件中：

```
~/.zshrc
autoload -Uz up-line-or-beginning-search down-line-or-beginning-search
zle -N up-line-or-beginning-search
zle -N down-line-or-beginning-search

[[ -n "${key[Up]}"   ]] && bindkey -- "${key[Up]}"   up-line-or-beginning-search
[[ -n "${key[Down]}" ]] && bindkey -- "${key[Down]}" down-line-or-beginning-search
```

这样，当按下 `Up` 或 `Down` 键时，只会显示与当前光标位置之前匹配的过去命令。

Shift、Alt、Ctrl 和 Meta 修饰键

xterm 兼容终端可以使用 user_caps(5) 中的扩展键定义，即 `Shift`、`Alt`、`Ctrl` 和 `Meta` 与 `Up`、`Down`、`Left`、`Right`、`PageUp`、`PageDown`、`Home`、`End` 或 `Del` 的组合。请参阅 zkbd 源代码以获取修饰键和组合键的推荐名称列表。

例如，使用 `Ctrl+Left` 移动到上一个单词的开头，使用 `Ctrl+Right` 移动到下一个单词的开头：

```
~/.zshrc
key[Control-Left]="${terminfo[kLFT5]}"
key[Control-Right]="${terminfo[kRIT5]}"

[[ -n "${key[Control-Left]}" ]] && bindkey -- "${key[Control-Left]}" behind-word
[[ -n "${key[Control-Right]}" ]] && bindkey -- "${key[Control-Right]}" front-word
```

另一种方法

该方法会在启动应用之前，将你的输入保存在一行当中。

```
~/.zshrc
ncmpcppShow() { ncmpcpp <$TTY; zle redisplay; }
zle -N ncmpcppShow
bindkey '^[\' ncmpcppShow
```

命令提示

Zsh 提供了使用提示主题的选项，而对于对主题不满意（或想要扩展其实用性）的用户，可以构建自定义提示。

提示主题

提示主题是快速轻松地在 Zsh 中设置彩色提示符的方法。有关提示主题以及如何编写自己的主题，请参阅 zshcontrib(1) § PROMPT THEMES。

要使用主题，请确保提示主题系统在 `.zshrc` 中设置为自动加载。这可以通过添加以下几行实现：

```
~/.zshrc
autoload -Uz promptinit
promptinit
```

然后你可以运行下面的命令查看可用的提示主题：

```
$ prompt -l
```

例如，要使用 `walters` 主题，请输入：

```
$ prompt walters
```

要预览所有可用的主题，请使用以下命令：

```
$ prompt -p
```

手动安装提示主题

可以手动安装主题而无需外部配置管理器工具。对于本地安装，首先创建一个文件夹并将其添加到 `fpath` 数组，例如：

```
$ mkdir ~/.zprompts
$ fpath=("$HOME/.zprompts" "$fpath[@]")
```

现在在此文件夹中创建主题文件的符号链接：

```
$ ln -s mytheme.zsh ~/.zprompts/prompt_mytheme_setup
```

如果您希望全局安装主题，请执行以下操作：

```
# ln -s mytheme.zsh /usr/share/zsh/functions/Prompts/prompt_mytheme_setup
```

现在您应该能够使用以下方法激活它：

```
$ prompt mytheme
```

如果一切正常，可以相应地编辑 `.zshrc`。

添加提示主题而不为每个主题单独创建一个文件

除了通过自己的文件添加提示主题外，还可以从另一个文件（如 `.zshrc`）中添加主题，例如：

```
~/.zshrc
# 加载 promptinit
autoload -Uz promptinit && promptinit

# 定义主题
prompt_mytheme_setup() {
```

```
PS1="%~%# "
}

# 将主题添加到 promptsys
prompt_themes+=( mytheme )

# 加载主题
prompt mytheme
```

自定义命令提示符

除了所有 shell 都通用的主要左侧提示符 `PS1`（`PROMPT`、`prompt`）之外，Zsh 还支持右侧提示符 `RPS1`（`RPROMPT`），这两个变量可用于自定义。

其他特殊用途的提示符，例如 `PS2`（`PROMPT2`）、`PS3`（`PROMPT3`）、`PS4`（`PROMPT4`）、`RPS1`（`RPROMPT`）、`RPS2`（`RPROMPT2`）和 `SPROMPT`，在 zshparam(1) § PARAMETERS USED BY THE SHELL 中有说明。

所有提示符都可以使用提示符转义进行自定义。可用的提示转义列在 zshmisc(1) § EXPANSION OF PROMPT SEQUENCES 中。

彩色

Zsh 设置颜色的方式与 Bash（英语：Bash/Prompt customization） 不同；您无需使用 terminfo(5) 中的大量 ANSI 转义序列或终端功能。Zsh 提供了方便的提示转义来设置前景色、背景色和其他视觉效果；请参阅 zshmisc(1) § Visual effects 以获取它们的列表及其说明。

颜色可以使用十进制整数、八种最广泛支持的颜色之一的名称或十六进制格式的 # 后跟 RGB 三元组来指定。有关更多详细信息，请参阅 zshzle(1) § CHARACTER HIGHLIGHTING 中 fg=colour 的描述。

命令	描述
`%F{color} [...] %f`	和前面介绍的 $fg 是一样的，但是更简洁。还可以在 F 前面添加数字。
`$fg_no_bold[color]`	设置文本为非粗体同时设定文本颜色
`$fg_bold[color]`	设置文本为粗体同时设定文本颜色
`$reset_color`	重置文本颜色（改为默认颜色）。不会重置粗体设定。使用 `%b` 来重置粗体设定。可以使用 `%f` 来简化配置。
`%K{color} [...] %k`	设置背景颜色。和非粗体文本颜色一样。任何单一数字前缀会设置背景为黑色。

大多数终端支持以下英文颜色名称作为代号：

中文名称	英文名称/代号	编号
黑色	black	0
红色	red	1
绿色	green	2
黄色	yellow	3
	blue	4

中文名称	英文名称/代号	编号
蓝色		
洋红色	magenta	5
青色	cyan	6
白色	white	7

注意： 粗体文本不一定会和普通文本使用同一种颜色。例如, `$fg['yellow']` 会使用暗一点的黄色, 而 `$fg_bold['yellow']` 可能会使用亮一点的黄色。（译者注：具体是由你的终端模拟器配置决定的）

与 xterm 256 色兼容的终端模拟器的颜色编号 0–255 可在 xterm-256color 图表中找到。

正确设置 TERM 环境变量后，可以使用 `echoti colors` 从 terminfo(5) 数据库中找到终端支持的最大颜色数。对于 24 位颜色，还需使用 `print $COLORTERM` 检查 COLORTERM 环境变量。如果它返回 `24bit` 或 `truecolor`，则即使 terminfo 显示的数字较小，您的终端也支持 16777216（224）种颜色。

注意：

- 终端模拟器及其使用的配色方案之间的颜色 0–15 可能有所不同。
- 许多终端仿真器使用更亮的颜色显示粗体。

提示：

- 可以使用命令 `print -P *"prompt escapes"*` 测试提示转义，例如：

```
$ print -P '%B%F{red}co%F{green}lo%F{blue}rs%f%b'
```

- 如果您使用 24 位颜色，您可能需要在不支持它们的终端中加载 `zsh/nearcolor` 模块。例如：

```
[[ "$COLORTERM" == (24bit|truecolor) || "${terminfo[colors]}" -eq '16777216' ]] || zmodload zsh/nearcolor
```

有关 `zsh/nearcolor` 模块的详细信息，请参阅 zshmodules(1) § THE ZSH/NEARCOLOR MODULE。

示例

简单无色提示的示例：

```
PROMPT='%n@%m %~ %# '
```

显示方式：

用户名@主机 ~ %

带颜色的左右双面提示示例：

```
PROMPT='%F{green}%n%f@%F{magenta}%m%f %F{blue}%B%~%b%f %# '
RPROMPT='[%F{yellow}%?%f]'
```

显示方式如下：

用户名@主机 ~ % \[0\]

要使用 16-255 范围内的颜色和 24 位真彩色，您可以分别使用分配给所需颜色的 0 到 255 之间的数字及其十六进制颜色代码：

```
PROMPT='%F{2}%n%f@%F{5}%m%f %F{4}%B%~%b%f %# '
RPROMPT='[%F{3}%?%f]'
PROMPT='%F{#c0c0c0}%n%f@%F{#008000}%m%f %F{#800080}%B%~%b%f %# '
RPROMPT='[%F{#0000ff}%?%f]'
```

示例 .zshrc 文件

- 要获得与每月 ISO 版本（默认使用 Zsh）相同的设置，请安装 grml-zsh-config 包。它包括来自 grml 的许多调整和高级优化。
- https://github.com/MrElendig/dotfiles-alice/blob/master/.zshrc - 基本设置，带有动态提示和窗口标题/硬信息。
- https://github.com/slashbeast/conf-mgmt/blob/master/roles/home_files/files/DOTzshrc - 具有多种功能的 zshrc，请务必查看其中的评论。值得注意的功能：确认功能以确保用户想要运行关机、重启或休眠，在提示中支持 GIT（无需 vcsinfo），带菜单的制表符完成，将当前执行的命令打印到窗口的标题栏中等等。

请参阅 dotfiles#用户仓库了解更多。

提示与技巧

登录时自动启动 X

请参阅 xinit#登录时自动启动 X。

程序异常退出后恢复终端设置

许多程序会更改终端状态，并且通常不会在异常退出时恢复终端设置（例如崩溃或遇到 SIGINT 时）。

这通常可以通过执行 reset(1) 来解决：

```
$ reset
```

以下部分介绍了避免手动重置终端的方法。

ttyctl 命令

ttyctl 命令可用于“冻结/解冻”终端。要在启动时冻结交互式 shell，请使用以下命令：

```
~/.zshrc
ttyctl -f
```

使用转义序列重置终端

替代画线字符集可能会以 ttyctl 无法阻止的方式搞砸终端。

一个简单的解决方案是从 `precmd` 钩子函数输出重置终端的转义序列，以便在每次绘制提示之前执行它们。例如，使用转义序列 `\e[0m\e(B\e)0\017\e[?5l\e7\e[0;0r\e8`:

```
~/.zshrc
autoload -Uz add-zsh-hook
```

```
function reset_broken_terminal () {
printf '%b' '\e[0m\e(B\e)0\017\e[?5l\e7\e[0;0r\e8'
}

add-zsh-hook -Uz precmd reset_broken_terminal
```

要测试它是否有效，请运行：

```
$ print '\e(0\e)B'
```

记住最近的目录

目录栈（dirstack）

Zsh 可以配置 DIRSTACK 相关变量来加速 cd 访问常用目录。在你的配置文件中添加下面的配置：

```
.zshrc
DIRSTACKFILE="$HOME/.cache/zsh/dirs"
if [[ -f $DIRSTACKFILE ]] && [[ $#dirstack -eq 0 ]]; then
  dirstack=( ${(f)"$(< $DIRSTACKFILE)"} )
  [[ -d $dirstack ]] && cd $dirstack
fi
chpwd() {
  print -l $PWD ${(u)dirstack} >$DIRSTACKFILE
}

DIRSTACKSIZE=20

setopt autopushd pushdsilent pushdtohome

## Remove duplicate entries
setopt pushdignoredups

## This reverts the +/- operators.
setopt pushdminus
```

现在你可以使用

```
dirs -v
```

来打印目录栈（dirstack）。使用 `cd -<NUM>` 来跳转到以前访问过的目录。你还可以在连字符后面使用自动补全，非常方便。

注意：如果您打开了多个 zsh 会话并尝试 `cd`，则此操作将不起作用，因为两个会话在写入同一个文件时会发生冲突。

cdr

cdr 允许您从自动维护的列表中将工作目录更改为上一个工作目录。它将所有条目存储在跨会话维护的文件中，并且（默认情况下）在当前会话中的终端仿真器之间维护。

有关设置说明，请参阅 zshcontrib(1) § REMEMBERING RECENT DIRECTORIES。

zoxide

zoxide包 是一个更智能的 cd 命令，只需按几下键，您就可以导航到任何地方。它会记住您经常使用的目录，并使用评分机制来猜测您想要去的地方。

帮助命令

与 Bash 不同，Zsh 没有内置的 `help` 命令，而是提供 `run-help`。默认情况下，`run-help` 是 `man` 的别名，可以通过将其添加到命令前面来手动执行，也可以使用键盘快捷键 `Alt+h` 或 `Esc h` 为当前输入的命令调用它。

由于默认情况下它只是 man 的别名，因此它仅适用于外部命令。为了改进其功能，使其适用于 shell 内置命令和其他 shell 功能，您需要使用 `run-help` 函数。有关 `run-help` 及其辅助函数的更多信息，请参阅 zshcontrib(1)。

首先加载 `run-help` 函数，然后删除现有的 `run-help` 别名。为了方便起见，可以将 `help` 别名为 `run-help`。例如，将以下内容添加到您的 `zshrc`：

```
autoload -Uz run-help
(( ${+aliases[run-help]} )) && unalias run-help
alias help=run-help
```

必须单独启用助手功能：

```
autoload -Uz run-help-git run-help-ip run-help-openssl run-help-p4 run-help-sudo run-help-svk run-help-svn
```

例如，`run-help git commit` 命令现在将打开 man page git-commit(1)，而不是 git(1)。

持续 rehash

通常情况下，compinit 不会自动找到 `$PATH` 中的新可执行文件。例如，在安装了一个新包之后，`/usr/bin/` 目录下的文件不会立即或自动包含在补全列表中。因此，为了使这些新的可执行文件包含进来，可以运行以下命令：

```
$ rehash
```

可以让这个 rehash 自动发生。 只需在你的 `zshrc` 文件中包含以下内容：

```
~/.zshrc
zstyle ':completion:*' rehash true
```

必要时 rehash

如上所述，但是可以通过配置 pacman 使用钩子来自动请求 `rehash`，这不会像上面的持续 rehash 那样带来性能损失。要启用此功能，需要创建 `/etc/pacman.d/hooks` 目录和一个 `/var/cache/zsh` 目录，然后创建一个钩子文件：

```
/etc/pacman.d/hooks/zsh.hook
[Trigger]
Operation = Install
Operation = Upgrade
Operation = Remove
Type = Path
Target = usr/bin/*
[Action]
Depends = zsh
When = PostTransaction
```

```
Exec = /usr/bin/install -Dm644 /dev/null /var/cache/zsh/pacman
```

这样做的目的是保持文件 `/var/cache/zsh/pacman` 的修改日期与上次安装、升级或移除包的时间一致。然后，需要通过在你的 `~/.zshrc` 文件中添加以下内容来促使 `zsh` 在其命令缓存过期时 rehash：

```
~/.zshrc
zshcache_time="$(date +%s%N)"

autoload -Uz add-zsh-hook

rehash_precmd() {
  if [[ -a /var/cache/zsh/pacman ]]; then
    local paccache_time="$(date -r /var/cache/zsh/pacman +%s%N)"
    if (( zshcache_time < paccache_time )); then
      rehash
      zshcache_time="$paccache_time"
    fi
  fi
}

add-zsh-hook -Uz precmd rehash_precmd
```

如果在 `/var/cache/zsh/pacman` 被更新之前触发了 `precmd` 钩子，则补全可能不会生效，直到启动新的提示符为止。运行一个空命令（例如按 `enter`）应该是足够的。

使用 SIGUSR1 的替代必要时 rehash 方法

如上所述，然而钩子文件看起来是这样的：

```
/etc/pacman.d/hooks/zsh-rehash.hook
[Trigger]
Operation = Install
Operation = Upgrade
Operation = Remove
Type = Path
Target = usr/bin/*

[Action]
Depends = zsh
Depends = procps-ng
When = PostTransaction
Exec = /usr/bin/pkill zsh --signal=USR1
```

警告：这会向所有正在运行的 `zsh` 实例发送 SIGUSR1。请注意，默认情况下 SIGUSR1 的行为是终止进程，所以当你第一次配置这个设置时，如果没有加载下面的 trap，所有用户的（包括登录 shell）所有正在运行的 `zsh` 实例将终止。

```
~/.zshrc
TRAPUSR1() { rehash }
```

上面的 ***trap** 函数*（function trap）可以用 ***trap** 列表*（list trap） `trap 'rehash' USR1` 替换。参见 zshmisc(1) § Trap Functions 了解不同类型的 trap 之间的区别。

这种方法将即时对所有 `zsh` 实例进行 `rehash`，无需按下回车键来触发 `precmd`。

ncurses 应用的快捷键绑定

将 ncurses 应用程序绑定到按键会导致无法交互。使用 `BUFFER` 变量使其工作。以下示例允许用户使用 `Alt+\` 打开 ncmpcpp:

```
~/.zshrc
ncmpcppShow() {
BUFFER="ncmpcpp"
zle accept-line
}
zle -N ncmpcppShow
bindkey '^[\' ncmpcppShow
```

另一种方法，它将保留您在调用应用程序之前在行中输入的所有内容:

```
~/.zshrc
ncmpcppShow() {
ncmpcpp <$TTY
zle redisplay
}
zle -N ncmpcppShow
bindkey '^[\' ncmpcppShow
```

文件管理器的快捷键绑定

图形化文件管理器中使用快捷键可能很实用（译者注：你也可以在 Zsh 中自定义快捷键达到这样的效果）。第一个使用 `Alt+Left` 让用户撤销最近的 cd 操作，第二个使用 `Alt+Up` 让用户进入上层目录。这两个快捷键同时也会显示目录中的内容。

```
~/.zshrc
cdUndoKey() {
  popd      > /dev/null
  zle       reset-prompt
  echo
  ls
  echo
}

cdParentKey() {
  pushd .. > /dev/null
  zle      reset-prompt
  echo
  ls
  echo
}

zle -N                  cdParentKey
zle -N                  cdUndoKey
bindkey '^[[1;3A'       cdParentKey
bindkey '^[[1;3D'       cdUndoKey
```

xterm 标题

如果您的终端仿真器支持，您可以从 Zsh 设置其标题。这允许动态更改标题以显示有关 shell 状态的相关信息，例如显示用户名和当前目录或当前正在执行的命令。

xterm 标题使用 xterm 控制序列操作系统命令 `\e]2;\a` 或 `\e]2;\e\\` 设置。例如：

```
$ print -n '\e]2;My xterm title\a'
```

将标题设置为

```
My xterm title
```

获得动态标题的一种简单方法是在 `precmd` 和 `preexec` 钩子函数中设置标题。有关可用钩子函数及其说明的列表，请参阅 zshmisc(1) § Hook Functions。

通过使用 `print -P`，您还可以利用 Zsh 的提示转义。

提示：

- 标题打印可以拆分为多个命令，只要它们是连续的即可。
- GNU Screen 将 xterm 标题发送到硬状态 (`%h`)。如果您想使用 Screen 的 字符串转义（例如用于颜色），您应该使用 `\e_\e\\` 转义序列设置硬状态。否则，如果在 `\e]2;\a` 中使用字符串转义，终端仿真器将获得乱码标题，因为它无法解释 Screen 的字符串转义。

注意：

- 打印变量时不要使用 `print` 的 `-P` 选项，以防止它们被解析为提示转义符。
- 打印变量时使用 `q` 参数扩展标志，以防止它们被解析为转义序列。

```
~/.zshrc
autoload -Uz add-zsh-hook

function xterm_title_precmd () {
	print -Pn -- '\e]2;%n@%m %~\a'
	[[ "$TERM" == 'screen'* ]] && print -Pn -- '\e_\005{2}%n\005{-}@\005{5}%m\005{-}
\005{+b 4}%~\005{-}\e\\'
}

function xterm_title_preexec () {
	print -Pn -- '\e]2;%n@%m %~ %# ' && print -n -- "${(q)1}\a"
	[[ "$TERM" == 'screen'* ]] && { print -Pn -- '\e_\005{2}%n\005{-}
@\005{5}%m\005{-} \005{+b 4}%~\005{-} %# ' && print -n -- "${(q)1}\e\\"; }
}

if [[ "$TERM" == (Eterm*|alacritty*|aterm*|foot*|gnome*|konsole*|kterm*|putty*|rxvt*|
screen*|wezterm*|tmux*|xterm*) ]]; then
	add-zsh-hook -Uz precmd xterm_title_precmd
	add-zsh-hook -Uz preexec xterm_title_preexec
fi
```

终端仿真器选项卡标题

一些终端仿真器和多路复用器支持设置选项卡的标题。转义序列取决于终端：

终端	转义序列	描述
GNU Screen	`\ek\e\\`	Screen 的窗口标题 (`%t`)。
Konsole	`\e]30;\a`	Konsole 的选项卡标题。

Shell 环境检测

请参阅有关 shell 环境检测的存储库以了解检测 shell 环境的测试。这包括登录/交互式 shell、Xorg 会话、TTY 和 SSH 会话。

/dev/tcp 等效项：ztcp

使用 `zsh/net/tcp` 模块：

```
$ zmodload zsh/net/tcp
```

您现在可以建立 TCP 连接：

```
$ ztcp example.com 80
```

更多详细信息请参阅 zshmodules(1) § THE_ZSH/NET/TCP_MODULE 和 zshtcpsys(1)。

在部分命令行上退出 shell 的快捷方式

默认情况下，如果命令行已填满，`Ctrl+d` 将不会关闭您的 shell，这可以修复它：

```
.zshrc
exit_zsh() { exit }
zle -N exit_zsh
bindkey '^D' exit_zsh
```

pacman -F“未找到命令”处理程序

pacman 包含搜索包含文件的包的功能。以下命令未找到处理程序将在执行未知命令时直接使用 pacman 搜索匹配的包。

```
~/.zshrc
...
function command_not_found_handler {
    local purple='\e[1;35m' bright='\e[0;1m' green='\e[1;32m' reset='\e[0m'
    printf 'zsh: 未找到命令：%s\n' "$1"
    local entries=(
        ${(f)"$(/usr/bin/pacman -F --machinereadable -- "/usr/bin/$1")"}
    )
    if (( ${#entries[@]} ))
    then
        printf "${bright}$1${reset} 可能位于以下软件包中：\n"
        local pkg
        for entry in "${entries[@]}"
        do
            # (存储库 软件包 版本 文件)
            local fields=(
                ${(0)entry}
            )
            if [[ "$pkg" != "${fields}" ]]
            then
                printf "${purple}%s/${bright}%s ${green}%s${reset}\n" "${fields}"
"${fields}" "${fields}"
            fi
            printf '    /%s\n' "${fields}"
            pkg="${fields}"
        done
    fi
    return 127
}
...
```

注意：pacman 的文件数据库与普通同步数据库是分开的，需要使用 `pacman -Fy` 来获取。详情请参阅 pacman#搜索包含特定文件的包。

有关使用 *pkgfile* 的替代方法，请参阅 #pkgfile“未找到命令”处理程序。

使用键绑定清除后缓冲区

默认情况下，清除屏幕键绑定不会清除大多数终端仿真器上的后缓冲区（您需要向上滚动才能看到的部分）。此问题的一个可能解决方案如下：

```
~/.zshrc
...
function clear-screen-and-scrollback() {
```

```
    printf '\x1Bc'
    zle clear-screen
}

zle -N clear-screen-and-scrollback
bindkey '^L' clear-screen-and-scrollback
...
```

第三方扩展

配置框架

注意：框架引入了一定程度的抽象和复杂性。它们可以并且经常会引入未定义的行为。如果 shell 出现故障，则“第一个”调试步骤应该是恢复到普通 shell。

- **oh-my-posh** — Oh My Posh 是任何 shell 的自定义提示引擎，它能够使用函数或变量调整提示字符串。

: https://github.com/JanDeDobbeleer/oh-my-posh || oh-my-poshAUR

- **oh-my-zsh** — 一个流行的、社区驱动的框架，用于管理您的 Zsh 配置。它捆绑了大量有用的函数、帮助程序、插件和主题。

: https://github.com/ohmyzsh/ohmyzsh || oh-my-zsh-gitAUR

- **Prezto** — Zsh 的配置框架。它带有模块，通过合理的默认值、别名、函数、自动完成和提示主题丰富了命令行界面环境。

: https://github.com/sorin-ionescu/prezto || prezto-gitAUR

- **ZIM** — 具有极快速度和模块化扩展的配置框架。Zim 非常易于定制，并且带有一组丰富的模块和功能，而不会影响速度或功能。

: https://github.com/zimfw/zimfw || zsh-zim-gitAUR

插件管理器

- **Antidote** — 功能齐全的 Zsh 旧版 Antibody 插件管理器实现。

: https://github.com/mattmc3/antidote || zsh-antidoteAUR

- **zinit（原“zplugin”)** — 灵活的 Zsh 插件管理器，具有干净的 fpath、报告、完成管理、涡轮模式 REVIVED

: https://github.com/zdharma-continuum/zinit || zinitAUR

- **zi（原“zplugin”）** — zplugin 的替代分支，旨在扩展原始项目，而不是像 zinit 那样保存和维护原始项目。

: https://github.com/z-shell/zi || 未被打包？在 AUR 里搜索

- **sheldon** — 快速、可配置的 shell 插件管理器，用 Rust 编写

: https://github.com/rossmacarthur/sheldon || sheldon包

- **Antigen** — Zsh 的插件管理器，灵感来自 oh-my-zsh 和 vundle。 ABANDONED

: https://github.com/zsh-users/antigen || antigen-gitAUR

- **zgen** — Zsh 的轻量级和简单插件管理器。 ABANDONED

: https://github.com/tarjoilija/zgen || zgen-gitAUR

- **zplug** — Zsh 的下一代插件管理器。 ABANDONED

: https://github.com/zplug/zplug || zplugAUR

仿 Fish 命令高亮

: 译者注：Fish 是一款比较新的终端软件

Fish 提供了非常强大的命令高亮。如果你希望在 zsh 中使用类似的功能，你可以安装 zsh-syntax-highlighting包、zsh-autosuggestions包，然后在你的 zshrc 中 source 下面的两个脚本或其中之一：

```
source /usr/share/zsh/plugins/zsh-syntax-highlighting/zsh-syntax-highlighting.zsh
source /usr/share/zsh/plugins/zsh-autosuggestions/zsh-autosuggestions.zsh
```

pkgfile"未找到命令"处理程序

pkgfile 包含一个 Zsh 脚本文件，该文件提供了一个 `command_not_found_handler` 函数，该函数将在输入无法识别的命令时自动搜索 pkgfile 数据库。

您需要 source 该脚本才能启用它。例如：

```
~/.zshrc
source /usr/share/doc/pkgfile/command-not-found.zsh
```

注意：可能需要更新 pkgfile 数据库才能正常工作。有关详细信息，请参阅 pkgfile#安装。

有关使用 pacman 本机功能的替代方法，请参阅 #pacman -F"未找到命令"处理程序。

卸载

在卸载 zsh包 之前请先更换默认终端。

警告：如果不遵循下面的步骤可能会导致用户无法访问任何终端

运行下面的命令：

```
$ chsh -s /bin/bash user
```

每一个使用 *zsh* 作为默认终端的用户都需要执行一遍条命令。当完成之后就可以把 zsh包 软件包删除了。

当然你也也可以以 root 身份修改 `/etc/passwd` 文件，来批量更改用户的默认终端。

警告：强烈建议使用 `vipw` 来修改 `/etc/passwd`，因为它可以帮助你消灭格式错误

例如将下面的配置中的 /bin/zsh

```
username:x:1000:1000:Full Name,,,:/home/username:/bin/zsh
```

改成 /bin/bash

```
username:x:1000:1000:Full Name,,,:/home/username:/bin/bash
```

另请参见

- zhwp:Zsh
- An Introduction to the Z Shell
- A User's Guide to ZSH
- The Z Shell Manual（可用的不同格式）
- Zsh FAQ
- zsh-lovers(1)（可通过zsh-lovers包使用）
- Gentoo: Zsh/Guide
- Bash2Zsh Reference Card

文本编辑器

改一个配置项、写几行脚本，都绕不开文本编辑器。本章解决的是装哪一个、怎么配得顺手、出错怎么查：从随手改配置的轻量工具，到模态编辑，再到带插件生态的图形编辑器，各节都给出安装、配置与排错的路子。

必读的是 Vim 与 Neovim，篇幅最长、内容最全；想立刻改配置，先看 Nano。Helix、Vi、Atom 篇幅短，可略读或按需跳；Spacemacs 与 Visual Studio Code 按个人习惯取舍。最常见的误解是装好即可用——配置、插件与排错才是主体，多数节单列了问题处理；也别把 Vi 与 Vim 当成同一个编辑器。

5.1　Helix

难度 入门 | **篇幅** 1.7 KB | **所属** 文本编辑器

本节将讲到

- 安装
- 配置
- 参见

翻译状态：

- 本文（或部分内容）译自 Helix，最近一次同步于 2024-10-02，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Helix 是一个用 Rust 编写的模态（modal）文本编辑器，灵感来自于 Neovim 和 Kakoune。它对传统的 Vim 工作流程进行了与 Kakoune 类似的修改，如基于选择的编辑和多光标支持。Helix 捆绑并启用了许多开箱即用的功能，目前还没有插件系统。但它可以添加自定义语言支持。因此，与 Vim 和类似的编辑器相比，Helix 更容易设置，但可定制性较差。

安装

安装 helix包 软件包，或 helix-gitAUR 开发版本。

配置

Helix 会读取一个可选的 `~/.config/helix/config.toml` 配置文件。可用选项列表请参见官方文档此页。

参见

- Github 仓库
- 官方文档

5.2 Vi

难度 入门 | **篇幅** 2.6 KB | **所属** 文本编辑器

本节将讲到

- 安装
- Vi 风格的软件
- 参见

来自维基百科：

: vi 是一款面向屏幕的文本编辑器，最初为 Unix 操作系统创建。《单一 Unix 规范》和 POSIX 描述（并标准化）了 vi 及其衍生程序的可移植行为子集，以及这些程序中支持的 ex 编辑器语言。

安装

vi 已不再提供。作为代替方案，你可以使用新一点的文本编辑器，例如 vim包 和 neovim包。还有一个兼容包 ex-vi-compat包 同时提供了ex 和 vi（兼容模式下的 vim）两种文本编辑器。另外， vi-vim-symlinkAUR 提供了从 vi 到 vim 的简单链接。

Vi 风格的软件

Vi 的键绑定和模式已在其他各种软件中重新创建：

- 文本编辑器，其中最受欢迎的是 Vim
- 文件管理器，如 ranger 和 Vifm
- Readline 输入库（由 Bash 使用）
- shell，如 Zsh
- 网页浏览器，如 Luakit，qutebrowser 和 vimb包；对于 Firefox 和 Chromium，有可用的浏览器扩展。
- 大多数平铺窗口管理器都可以配置为进行 vi 样式控制

参见

- vi(1)

5.3 Atom (文本编辑器)

难度 入门 | **篇幅** 3.3 KB | **所属** 文本编辑器

本节将讲到

- 安装
- 插件
- 问题处理



本文所述教程或工具由于已经过时而被弃用，请使用其他替代方案： 其他文本编辑器 (在Talk:Atom (文本编辑器))讨论)

Atom 是一个由 GitHub 开发的开源文本编辑器，采用 MIT 证书授权方式。它主要用 CoffeeScript 和 Javascript 编写，并使用 Node.js 作为运行时环境。超过4,000个插件和1,000种主题使它具有很强的扩展性。它使用其内建的 apm 软件包管理器管理软件包和主题。

安装

以下软件包都可用于安装Atom:

- atomAUR
- atom-editor-gitAUR

插件

它的插件可以在Atom软件中或者使用apm命令完成安装，正确的apm命令语法是：

```
$ apm install package_name1 package_name2 package_name3 ...
```

一些包已经被预装到Atom中，未预装包中值得注意的有：

- build 使Atom可以编译源代码。
- git-plus 允许开发者在Atom中管理Git库。
- markdown-writer 将Atom变为一个有效的Markdown编辑器。
- script 使Atom可以基于文件名运行脚本。

问题处理

环境变量设置未被使用

你可能会遇到一些因为软件包使用环境变量而引起的问题，像go-plus (`$GOPATH not found`)。而且，问题只有在通过文件管理器打开Atom时才会出现（这是由DBUS引发的，因而不会使用在 `.bashrc` 中定义的环境变量）。

你可以通过Systemd/用户#环境变量为DBUS-spawned进程创建可用的环境变量

关于这个问题的更多内容，请参考 Environment variables#Per user.

无法删除文件

Electron 程序默认使用 `gvfs-trash` 删除文件，不使用 GNOME 的用户可以使用 `ELECTRON_TRASH` 环境变量设置删除工具。

例如要在 Plasma 中删除文件:

```
$ ELECTRON_TRASH=kioclient5 atom
```

目前 Electron 支持 `kioclient5`, `kioclient`, `trash-cli` 和 `gvfs-trash` (默认)。 更多信息请参考 Github 页面.

启动时黑屏

在某些显卡，例如 VirtualBox 客户系统中，只有使用 `--disable-gpu` 禁用硬件加速的显卡，或者编辑配置文件 `.atom/config.cson` 并在`editor`中增加或更改 `useHardwareAcceleration: false`，Atom 才会渲染窗口。

无拼写检查

请确保 hunspell 和需要的字典已经安装.

5.4 Nano

难度 入门 | **篇幅** 4.9 KB | **所属** 文本编辑器

本节将讲到

- 安装
- 配置
- 使用
- 提示和技巧
- 问题解决

GNU nano （或 nano）是一个基于控制台的文本编辑器，旨在提供一个简单的界面和直观的命令选项。 *nano* 支持的功能包括语法高亮、DOS/Mac 文件格式转换、拼写检查和UTF-8编码。 用空缓冲区打开的*nano*通常占用少于4MB的驻留内存。

安装

Nano 对应的软件包是 nano包。

配置

Nano的外观、感觉和功能通常由命令行参数或者配置文件 `~/.config/nano/nanorc`控制。

程序安装时会同时安装一个示例配置文件，位于`/etc/nanorc`。 要自定义配置，首先复制一份配置文件到`~/.config/nano/nanorc`：

```
$ cp /etc/nanorc ~/.config/nano/nanorc
```

通过设置~/.config/nano/nanorc文件中的参数控制nano的设置。

提示：nanorc(5)列出nano的全部可用配置选项。

注意：命令行参数会优先覆盖配置文件~/.config/nano/nanorc中的参数。

语法高亮

Nano包含预定义的语法高亮规则，位于文件/usr/share/nano/*.nanorc 和 /usr/share/nano/extra/*.nanorc。 添加以下配置到 ~/.config/nano/nanorc或者/etc/nanorc使语法高亮生效：

```
include "/usr/share/nano/*.nanorc"
include "/usr/share/nano/extra/*.nanorc"
```

可以在 AUR（nano-syntax-highlighting-gitAUR）找到默认语法高亮规则的增强扩展。 参考，用于Forth)突出显示。

提示：您也可以安装 nano-syntax-highlighting-gitCNRepo。

PKGBUILD

- 将 https://paste.xinu.at/4ss/ （类似 svntogit-server）保存到 /etc/nano/pkgbuild.nanorc，引用它：

```
include "/etc/nano/pkgbuild.nanorc"
```

- 还可以选择安装 nano-syntax-highlighting-gitAUR

挂起

Nano与大部分交互程序不同，默认情况下关闭挂起功能。 取消/etc/nanorc中set suspend行的注释以启用挂起功能。 允许用按键Ctrl+z使nano挂起到后台。

文本换行

*Nano*与大部分文本编辑器不同，默认文本自动换行。 要关闭自动换行，在 ~/.config/nano/nanorc添加以下参数：

```
set nowrap
```

使用

快捷键提示可以在*nano*中看到。 Nano中可以用Ctrl+g打开在线帮助，nano Command Manual可以查看详细说明和帮助。

特殊按键

Nano在屏幕底部两行显示功能快捷键。

表示方式如下：

- ^表示按住键盘上的Ctrl
- M-表示按住键盘上的*Meta*（通常是Alt）或Esc

提示：Feature Toggles列出nano全部可用快捷键。

提示和技巧

用nano替换vi

要用nano替换vi作为控制台默认文本编辑器，例如用于visudo，设置VISUAL和EDITOR 环境变量，示例：

```
export VISUAL=nano
export EDITOR=nano
```

问题解决

快捷键绑定冲突

部分窗口管理器会与nano的快捷键冲突，例如Alt+Enter。 删除或重新绑定快捷键例如Super（用mutter包、muffin包和marco包修改dconf包），然后重新启动窗口管理器。

参考

- nano (text editor)) - Wikipedia入口
- GNU nano Homepage - 官方网页
- GNU nano Bugs 报告bug
- Nano语法高亮文件扩展

5.5 Neovim

难度 进阶 | **篇幅** 9.9 KB | **所属** 文本编辑器

本节将讲到

- 安装
- 配置
- 提示与技巧
- 问题解决
- 参见

翻译状态：

- 本文（或部分内容）译自 Neovim，最近一次同步于 2024-02-04，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Neovim 是 Vim 的分支，旨在改进代码库，从而使得 API 更容易实现，并改善用户体验和插件实现。Neovim 启发了 Helix 等编辑器。

安装

安装 neovim包 或 neovim-gitAUR（开发版本）软件包，两者强烈建议使用 Lua 作为主要的配置语言。要使 Neovim 能够访问系统剪贴板，可能需要安装 xclip包（X11）或 wl-clipboard包（Wayland）。

注意：在 Neovim 中，一些功能委托给外部“providers”（提供者）。对于 Python 提供者，使用 python-pynvim包。 对于剪贴板提供者，参见 provider-clipboard 或 `:help provider-clipboard` Neovim 命令。

也可以安装 GUI 和其他相关项目，它们中的大多数都在官方软件仓库或 AUR 中：

- **neovim-qt包** — Fast, lightweight, and customizable Qt GUI. Provides a modern interface, including support for multiple tabs, split windows, and customizable themes.

: https://github.com/equalsraf/neovim-qt || neovim-qt包

- **neovim-gtkAUR** — GTK GUI. Provides a modern, customizable interface, including support for split windows, multiple tabs, and customizable themes.

: https://github.com/Lyude/neovim-gtk || neovim-gtkAUR

- **uivonim-gitAUR** — (Inactive) Simple and lightweight GTK GUI. Provides a minimalistic interface, including support for split windows and customizable themes.

: uivonim-gitAUR || uivonim-gitAUR

- **neovide包** — Rust GUI.

: https://github.com/neovide/neovide || neovide包

- **neoray-gitAUR** — Go GUI.

: https://github.com/hismailbulut/neoray || neoray-gitAUR

- **gnvimAUR** — GTK GUI.

: https://github.com/vhakulinen/gnvim || gnvimAUR

- **fvimAUR** — F# GUI.

: https://github.com/yatli/fvim || fvimAUR

配置

Neovim 的用户配置位于 `$XDG_CONFIG_HOME/nvim/init.vim`（默认是 `~/.config/nvim/init.vim`）。全局配置文件位于 `$XDG_CONFIG_DIRS/nvim/sysinit.vim`（默认是 `/etc/xdg/nvim/sysinit.vim`）。如果不存在，则会载入 `/usr/share/nvim/sysinit.vim`，后者不建议由用户编辑。 默认情况下，前一个全局配置文件并不存在。如果你创建了前一个文件，那你可能也想在其中 source 后一个文件（如果你想获得它提供的功能：允许 Neovim 使用通过 pacman 安装的 Vim 包）。

Neovim 兼容 Vim 大多数的选项，不过有一些选项特定于 Neovim。完整的 Neovim 选项列表参见 Neovim 的帮助文件。

Neovim 的数据目录位于 `~/.local/share/nvim/`，包含打开文件的交换文件、ShaDa（共享数据）文件和插件的站点目录。

从 Neovim 0.5 版本开始，可以使用 Lua 设置 Neovim（默认是 `~/.config/nvim/init.lua`），API 仍然很年轻，不过常见的配置开箱即用，无需过多配置。关于转换现有的配置的建议，参见 。当前，与 `init.vim` 相比，使用 `init.lua` 没有太多的优势，不过在使用得当时，Lua 可以小幅改善启动时间，并且由于配置简单，在使用以 Lua 编写的插件时会很有用。

从 Vim 迁移

要将现有的 Vim 配置迁移到 Neovim，只需将 `~/.vimrc` 复制到 `~/.config/nvim/init.vim`。如果需要的话，将 `~/.vim/autoload/` 的内容复制到 `~/.local/share/nvim/site/autoload/`。

在 Vim 和 Neovim 间共享配置

Neovim 将 `$XDG_CONFIG_HOME/nvim` 而不是 `~/.vim` 作为主要配置目录，并使用 `$XDG_CONFIG_HOME/nvim/init.vim` 而不是 `~/.vimrc` 作为主要配置文件。

如果要继续使用 Vim，又希望在 Neovim 中 source 现有的 Vim 配置，参见 nvim-from-vim 或 `:help nvim-from-vim` Neovim 命令。

加载插件

`/etc/xdg/nvim/sysinit.vim` 会自动 source 从官方软件仓库或 AUR 中安装的 Vim 或 Neovim 插件，因此无需额外设置。大量的插件都可以在两个地方找到，但最好通过插件管理器添加插件，最常用的是 vim-plug（可用于 Vim 和 Neovim）和

~packer（使用 Lua 编写，只能用于 Neovim 0.5 或更新的版本）~，packer已停止维护，建议使用lazy.nvim或pckr.nvim。这些插件管理器都可接受在 GitHub 分支到运行时命令这一范围内的深度配置。

大多数为 Vim 编写的插件无需花费很多精力即可在 Neovim 上工作，不过不是每个为 Neovim 编写的插件都适用于 Vim，因此，如果你的目标是确保配置兼容两个版本，坚持使用 `init.vim` 或 `.vimrc`.

提示与技巧

将 vi 和 vim 替换为 neovim

大部分情况下设置 `$VISUAL` 和 `$EDITOR` 环境变量应该就足够了。

一些应用程序可能硬编码 vi 或 vim 为默认编辑器，对于这种情况，安装 neovim-symlinksAUR 或 neovim-drop-inAUR。

将 init.vim 设置为指向 .vimrc 的符号连接

因为 Neovim 与标准 Vim 大部分兼容，可以将 `nvim/init.vim` 设置为指向原来 `.vimrc` 的符号连接，从而保留旧的配置选项：

```
$ ln -s ~/.vimrc ~/.config/nvim/init.vim
```

如果想要某几行特定于每个版本，可以使用 `if` 块：

```
.vimrc
if has('nvim')
    " 特定于 Neovim 的命令
else
    " 特定于标准 Vim 的命令
endif
```

真彩色支持

这个项目的 `README` 文件解释了如何为语法高亮添加 24 位“真彩色”支持，以及如何使用取色器实时查看效果。包含作者为 C++ 提供的语法高亮（如果安装的话）。

Lastplace cursor support

如果你想保留光标的最后位置，lastplace.lua 非常有用。只需将其放置在 `~/.config/nvim/plugin/` 或系统目录 `/usr/share/nvim/runtime/plugin/` 中即可。

Language Server Protocol

Neovim 内置 Language Server Protocol（LSP）客户端，nvim-lspconfig 插件提供常见的配置。

参见语言服务器协议。

Use as a pager

你可以用 `:Man` 命令打开手册页。要使用 Neovim 打开所有手册页，请将 `MANPAGER` environment variable 设置为 `nvim +Man!`.

对其他帮助阅读程序的支持请安装 nvimpagerAUR 或 nvimpager-gitAUR 包并将 `PAGER` environment variable 设置为 `nvimpager`.

也可以试试 page, 打包在 page-gitAUR 中。

问题解决

退出后光标没有恢复到之前的状态

如果退出 Neovim 后光标仍闪烁，参见 neovim FAQ 中的解决方案。

参见

- GitHub 仓库
- GitHub wiki

5.6 Spacemacs

难度 高级 | **篇幅** 21.1 KB | **所属** 文本编辑器

本节将讲到

- 安装
- 基础使用
- 配置
- 故障排除

Spacemacs 是一款基于 Emacs 构建、采用 Vim 按键绑定方案的可扩展文本编辑器。该项目旨在融合 Vim 与 Emacs 编辑器的优势，取其精华。Spacemacs 的发行版基于社区驱动的 Emacs 配置，不仅大幅扩展了 Emacs 的默认行为，更增添了诸多额外功能。

安装

安装 Emacs

Spacemacs 是基于 Emacs 构建的，因此我们需要先安装 Emacs。

- 安装 Emacs 包

备份旧的 Emacs 配置 (可选)

若您曾使用过Emacs，请务必备份之前的配置文件。

```
$ mv ~/.emacs.d ~/.emacs.d.bak && mv ~/.emacs ~/.emacs.bak
```

安装 Spacemacs

要安装 Spacemacs，我们需要从 GitHub 克隆一个实际配置，并完全替换 Emacs 的配置。

```
$ git clone https://github.com/syl20bnr/spacemacs ~/.emacs.d
```

注意：此命令应从您的用户账户运行。

安装 Adobe Source Pro fonts (可选)

Spacemacs 默认使用的字体是 Source Code Pro by Adobe. 若需使用该字体，建议在系统中安装。

- 安装 adobe-source-code-pro-fonts包

如果未找到指定字体，则回落到一个可用字体。

移除 Emacs 配置文件

（如果需要）备份并删除 ~/.emacs 文件，否则，Spacemacs 将无法加载，因为该文件会阻止 Emacs 加载正确的初始化文件。你可以通过以下方式重命名它：

```
$ mv ~/.emacs ~/.emacs.bak
```

你也可以不备份就删除它：

```
$ rm ~/.emacs
```

第一次运行 Spacemacs

下面首次运行Emacs:

```
$ emacs
```

第一次你应该被要求选择要安装的功能。所有选项都是可替代的，所以无论如何都必须选择某个选项。这些选择会影响一些 Spacemacs 的行为和快捷键。建议选择默认值，只需按 **Enter** 键。默认值已经相当优化，你之后可以随时更改它们。

提示： 大多数默认插件比替代方案功能更丰富。但在部分场景下（如性能优化、特殊功能需求），其它变体插件反而会更实用。

完成配置问答后，Spacemacs 会自动下载并安装所有依赖插件，整个过程可能需要几分钟。期间 Spacemacs 界面看似卡住，属于正常情况。

启动 Spacemacs

直接执行以下命令即可启动：

```
$ emacs
```

当底部状态栏不再显示 ... 类加载进度提示时，就代表 Spacemacs 已就绪可用。

注意： 若启动耗时超过 10 秒，请参考下方故障排查章节。

守护进程模式

Spacemacs 支持以守护进程模式运行。守护进程模式只会初始化一次编辑器，后续可直接连接进程，无需重复读取配置文件。适合配置文件体量很大的场景，仅需完成一次初始化，之后随时都能快速连接使用。

以守护进程模式启动 Spacemacs：

```
$ emacs --daemon=instance1
```

后续可通过 **emacsclient** 连接 `instance1` 实例：

```
$ emacsclient -nc -s instance1
```

提示：你可以创建多个不同名称的守护进程实例。

systemd 自启配置

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Written like a blog, violates Help:Style#Language register.（在Talk:Spacemacs讨论）

你可以创建 systemd 服务来常驻运行 Emacs 守护进程。出于社区 ELPA 插件安全等因素考量，**推荐以普通用户权限配置 systemd 用户级服务**，配置方式如下。

新建 Emacs 用户系统服务文件：

```
.config/systemd/user/emacs.service
[Unit]
Description=Emacs text editor
Documentation=info:emacs man:emacs(1) https://gnu.org/software/emacs/

[Service]
Type=forking
ExecStart=/usr/bin/emacs --daemon=instance1
ExecStop=/usr/bin/emacsclient --eval "(kill-emacs)"
Restart=on-failure

[Install]
WantedBy=default.target
```

按以下内容修改 Emacs/Spacemacs 桌面入口文件，重点修改 `Exec` 字段，其余配置仅为界面美化：

```
/usr/share/applications/emacs.desktop
[Desktop Entry]
Name=Spacemacs
GenericName=Text Editor
Comment=Edit text
MimeType=text/english;text/plain;text/x-makefile;text/x-c++hdr;text/x-c++src;text/x-
chdr;text/x-csrc;text/x-java;text/x-moc;text/x-pascal;text/x-tcl;text/x-
tex;application/x-shellscript;text/x-c;text/x-c++;
Exec=emacsclient -nc -s instance1 %F
Icon=/home/[!!! YOUR USER NAME HERE !!!]/.emacs.d/assets/spacemacs.svg
Type=Application
Terminal=false
Categories=Development;TextEditor;
StartupWMClass=Emacs
Keywords=Text;Editor;
```

执行守护进程配置重载，再启动 `emacs.service` 用户服务单元。

检查是否出现运行报错。如果你之前手动配置过 Emacs 守护进程且存在异常，建议启用刚创建的用户服务后重启系统，多数问题都能解决。也可以查看 `emacs.service` 单元运行状态排查故障。确认正常后，启用该用户服务实现开机自启。

配置完成后，可通过桌面环境启动器打开 Spacemacs，加载速度会大幅提升；即便在高性能设备上，也依旧无法做到瞬时秒开。你可以为连接命令设置别名，`instance1` 可自定义命名，但必须和服务文件内的守护进程名称保持一致。

```
$ emacsclient -nc -s instance1
```

基础使用

初次上手 Spacemacs 会有一定门槛，纯新手尤为明显，但熟悉后收益极高。只需掌握少量核心概念，就能完成日常基础编辑工作。

随时输入 `:q` 按下回车键，即可退出 Spacemacs。

内置教程

在 Spacemacs 界面中按下 `SPC h T`，可打开官方内置入门教程。

基础核心概念

前置准备

为讲解基础操作，我们先生成一段测试文本用于实操。暂时不用理解当下的快捷键，只需跟着步骤操作即可：

1. 启动 Spacemacs 按下 `SPC b N` 新建空白缓冲区 按下 `9 SPC i l l` 自动生成测试文本

执行后会生成九行示例文本，后续所有命令练习都可以基于这段内容操作。

注意：用 SPC 组合键时需要依次按顺序敲击：先按空格键，再按 b，再按 N。大写字母需要配合 Shift 输入，例如 N 对应 Shift+n，完整按键顺序为：空格 → b → Shift+n

提示：按下空格键后，底部会弹出帮助菜单，可查看所有可用命令。

接下来介绍 Spacemacs 核心的**编辑模式**概念。

编辑模式

Spacemacs 与普通文本编辑器最大的区别就是**编辑模式**。不同模式下编辑器的按键行为完全不同：比如**插入模式**可以直接输入文字（和普通编辑器一致），**普通模式**下所有按键都会被识别为编辑命令，不会输入任何字符。

同一时间只能激活一种模式，熟练切换各类模式是用好 Spacemacs 的核心技能。

当前编辑模式会显示在界面左下角，默认以带数字「1」的彩色矩形标识，颜色对应不同模式。模式种类很多，日常只需掌握常用三种：

- 橙色：**普通模式**，用于输入命令、光标导航
- 绿色：**插入模式**，用于文字输入编辑
- 灰色：**可视模式**，用于选中文本、批量操作也可以通过光标颜色判断当前所处模式。

> **注意**：使用 Spacemacs 至少需要掌握普通模式和插入模式。

普通模式

普通模式为默认模式，标识颜色为**橙色**，主要用于光标导航和执行编辑命令，无法直接输入文字，适合快速移动光标、修改文本内容。

若不小心退出普通模式，随时按下 `ESC` 键 或 连续按 `fd` 即可快速返回。

注意：下方仅列出部分常用命令，实际指令远不止这些，可查阅拓展文档按需学习。

注意：无需一次性记完所有命令，循序渐进掌握一小部分常用指令，就足够入门使用。

导航

对于基础导航，可以使用以下按键：

- `h` - 光标向左移动一个符号
- `j` - 光标向下移动一行
- `k` - 光标向上移动一行
- `l` - 光标向右移动一个符号

你也可以使用单键在单词甚至句子之间进行导航：

- `w` - 移动到下一个单词（词首）
- `b` - 移动到上一个单词（词首）
- `(` - 移动到当前句子的句首
- `)` - 移动到下一个句子的句首
- `^` - 移动到行首
- `$` - 移动到行尾

要滚动页面，可以使用以下命令：

- `Ctrl+f` - 向下翻一页
- `Ctrl+b` - 向上翻一页

- `gg` - 跳转到文档的第一行
- `G` - 跳转到文档的最后一行

你还可以将数字与命令结合使用，使其重复 **n** 次：

- `5j` - 将光标向下移动五行
- `7w` - 将光标向前移动七个单词
- `3 Ctrl+f` - 向下翻三页
- `20gg` - 将光标移动到第 20 行

提示：数字参数是 Spacemacs 世界中广泛使用的技巧。

这里还有很多未被发掘的命令。得益于类似于 **Vim** 的操作流，基本上你可以在 Spacemacs 中到处导航。请查阅附加资源以获取详细信息。

文本操作

你可以使用以下命令修改文本：

- `x` - 剪切光标下的符号
- `dw` - 剪切光标下的单词
- `dd` - 剪切光标所在的行
- `yw` - 复制（拉取）光标下的单词
- `yd` - 复制（拉取）光标所在的行
- `p` - 粘贴已复制/剪切的文本
- `r*a*` - 将光标下的符号替换为 *a*

你也可以在此处使用数字参数。

撤销/重做

你可以使用以下命令撤销和重做更改：

- `u` - 撤销上次更改
- `Ctrl+r` - 重做上次更改

插入模式

插入模式用于文本输入。它的行为与普通编辑器非常接近。然而，其修改文本的能力有限。你需要切换回**普通模式**才能进行修正。插入模式的颜色为**绿色**。

进入

要从**普通模式**进入插入模式，请按 `i`。你的光标将变为一条绿色的细线。现在你可以输入内容了。完成后，只需按下 `ESC` 键或 `fd` 键序列即可退出插入模式。

进入插入模式的方法有很多种。区别在于初始光标位置的不同。初次使用时，只需知道 `i` 快捷键就足够了。但当你熟练掌握后，以下其他快捷键也会非常有用：

- `i` - 在光标前进入插入模式
- `a` - 在光标后进入插入模式
- `I` - 在行首进入插入模式
- `A` - 在行尾进入插入模式
- `o` - 在下一行进入插入模式
- `O` - 在上一行进入插入模式

退出

要离开插入模式，请按 `ESC` 键或 `fd` 键序列。你将返回**普通模式**，光标将变为橙色。

可视模式

此模式用于可视化文本选择。它允许选择文本块并剪切/复制它们。该模式的颜色为**灰色**。

要从**普通模式**进入可视模式，请按 `v` 快捷键。然后你可以使用普通模式的快捷键进行导航，唯一的区别是：文本选择。基于初始光标位置，光标的移动会选中文本，之后你可以使用 `y`ank（复制）或 `d`elete（删除）它们。记住，你可以使用 `ve` 或 `v(` 等命令快速选择单词或句子。请参阅 普通模式：导航 章节以了解详情。

你也可以按 `V` 快速选中整行。

可视块模式

可视块模式是可视模式的增强版本。它允许按列选择文本。这类似于常规编辑器和 IDE 中的多光标概念。可以通过按下 `Ctrl+v` 快捷键进入此模式。然后你可以使用 `h j k l` 键进行导航以查看区别。

在可视块模式下可以做很多事情。相关信息请参阅附加资源。该功能在 Vim 术语中被称为**vim 可视块模式**。

缓冲区

Spacemacs 中的文本位于称为缓冲区的区域中。它们与常规编辑器的标签页非常相似。你可以在缓冲区之间切换并创建新的缓冲区。编辑器本身也使用缓冲区来存储一些可供稍后检查的信息。

导航

要显示当前缓冲区列表，请按 `SPC b b`。你将在底部看到一个新窗口。这是你可以检查、筛选和导航缓冲区的地方。那里已经存在一些缓冲区，如 *Messages*\ 和 *scratch*\。它们由编辑器创建，包含一些有用的信息。

你在底部窗口可以做的第一件事是在 `pattern`（模式）字段中输入任何内容。这将筛选缓冲区。如果筛选后没有剩余的缓冲区，你可以在输入后直接按“Enter”创建一个新缓冲区。新缓冲区将被创建并打开。

你也可以手动打开任何缓冲区。按 `Ctrl+**j**` 或 `Ctrl+**k**` 在行之间导航。然后按 `Ctrl+**l**` 或 `Enter` 确认选择。选中的缓冲区将被打开。

提示：还记得 `h j k l` 键吗？它们被广泛用于导航。在某些情况下，我们需要使用像 `Ctrl` 这样的修饰键。这样可以同时进行输入和导航。

你也可以使用**普通模式**中的一些快捷键来控制缓冲区：

- `SPC b b` - 列出缓冲区
- `SPC TAB` - 切换到上次查看的缓冲区
- `SPC b n` - 切换到下一个缓冲区（向前一个）
- `SPC b p` - 切换到上一个缓冲区（向后一个）
- `SPC f s` - 将当前缓冲区保存到文件
- `SPC b d` - 关闭当前缓冲区

提示：如果你想保存一个新缓冲区，你应该为其选择一个文件。详情请参阅下一节。

文件

Spacemacs 提供了两种文件导航选项：内联导航和内置文件管理器。内联导航用于 Spacemacs 的确认对话框，与 Shell 非常相似。内置文件管理器更加用户友好，允许检查文件详细信息。掌握每种方式的基础是精通 Spacemacs 的关键。

也有高级选项可用，例如更强大的文件管理器和文件树。这些内容包含在 高级 章节中。

内联

使用 `SPC f f` 快捷键可使用内联导航。它使用的窗口与缓冲区导航窗口非常相似。你可以在那里筛选和选择文件。只需输入任何内容以缩小结果范围，或按 `Ctrl+j` 或 `Ctrl+k` 上下移动行。按 `Ctrl+l` 打开文件或目录，按 `Ctrl+h` 返回上一级。按 `TAB` 自动补全输入。

文件管理器

如果你需要更可视化的方法，请按 `SPC a d Enter` 运行内置文件管理器。你可以使用 `Ctrl+h j k l` 键进行导航，并按 `Enter` 进入目录和打开文件。

有一些可用的快捷键（更多请参考 dired 文档）：

- `q` - 退出 dired
- `R` - 重命名文件
- `C` - 复制文件
- `+` - 创建新目录

提示：如果你需要更强大的文件管理器，请查看 高级 章节中的 Ranger。它提供了更多功能，一旦你掌握了它，它可以成为 Dired 的最佳替代品。

退出

可以通过 `SPC q` 退出编辑器，这将显示多种退出方法。

一些可用的方法包括：

- `SPC qf` 退出当前帧 - 适用于 emacsclient 帧
- `SPC qq` 退出 Emacs (Kill-emacs)
- `SPC qr` 退出 Emacs 并重启（恢复布局）
- `SPC qR` 退出 Emacs 并重启
- `SPC qd` 退出 Emacs 并重启（附带 --debug-init 参数）

高级概念

在这一步，你已经能够成功打开文件、进行修改并保存。路已走完一半，现在你可以选择接下来要掌握什么。以下是一些你可能感兴趣的章节。

层

层是 Spacemacs 最强大的功能之一。层是一组软件包和配置选项，能以某种方式极大地扩展编辑器功能。例如，有针对不同编程语言的层，或者提供额外工具的层（如 IRC 消息或集成网页浏览器）。层的完整列表可以在 层 文档页面找到。

有些层已经随 Spacemacs 一起提供，其他的可以手动添加。为此，打开 Spacemacs 配置文件（`SPC f e d`），找到其中的 `dotspacemacs-configuration-layers` 部分。然后只需将选定的层添加到列表中并重启 Spacemacs。它会在下次启动时下载所有必需的文件。

当你打开一个已知扩展名的文件时，Spacemacs 也会建议你安装一个新的层。例如，如果你打开一个 `.html` 文件，系统会建议安装 `html` 层。

你可以通过在 Spacemacs 配置文件中覆盖某些特定于层的变量来自定义层的行为。请查看相应的层文档以获取详细信息。

文件导航

还有一些额外的文件导航工具。它们可能会极大地改变你日常使用 Spacemacs 的方式。

文件树

你可以按 `SPC f t` 运行文件树。新窗口将打开，可通过 `SPC 0` 访问。那里可以使用标准的 `h j k l` 导航。你可以使用 `R` 更改根文件夹，使用 `s` 切换隐藏文件。使用 `c` 创建新文件，使用 `r` 重命名旧文件。详情请查看 Neotree 文档。

提示：如果你需要将根目录更改为更高级别的目录，只需在当前根路径（窗口的第一行）上按 `R`。内联文件导航将打开，只需根据需要按 `H` 返回上一级，然后选择 `.` 目录即可。

文件管理器

如果你需要一个全功能的文件管理器，那么 Ranger 可能是最佳选择。那里有许多有用的功能，如即时的 `h j k l` 导航、内联文件预览以及操作文件的能力。它还稍微改进了默认的 Dired 行为（`SPC a d`）。安装 `ranger` 层并使用 `SPC a r` 运行它。详情请查看 Ranger 文档。除了自定义选项外，还有许多有用的快捷键。

注意：如果你打开 Ranger 有问题，请尝试先关闭 Neotree

窗口

Spacemacs 允许你将屏幕分割成独立的窗口。每个窗口都有一个个人编号，可以通过 `SPC *n*` 快捷键访问，其中 `*n*` 是选定的编号。窗口可以单独分割，因此可以创建复杂的布局。

下面列出了一些窗口快捷键。使用内联帮助（`SPC w`）获取更多信息。

- `SPC w 3` - 聚焦编号为 3 的窗口
- `SPC w s` - 水平分割窗口
- `SPC w v` - 垂直分割窗口
- `SPC w d` - 删除窗口
- `SPC w u` - 撤销上次窗口操作
- `SPC w m` - 切换窗口全屏
- `SPC w .` - 进入窗口瞬态状态 (transient state)

提示：瞬态状态允许你调整窗口设置，如它们的顺序和比例。只需进入该状态，所有可用选项都会显示出来。

配置

故障排除

启动缓慢

如果启动时间超过 10 秒，可能是 `exec-path-from-shell` 模块出现了问题。在 Linux 系统上可以安全地禁用它。完成以下步骤：

1. 按 `SPC f e d` 打开 Spacemacs 配置文件
2. 找到 `dotspacemacs-excluded-packages` 部分
3. 在此处添加 `exec-path-from-shell` 模块，最终条目应类似于 `dotspacemacs-excluded-packages '(exec-path-from-shell)`
4. 使用 `SPC f s` 保存更改并重启 Spacemacs

次要模式图标字体不正确

如果你看到的是 24ba 24c0 符号而不是 ⒺⓀ，或者它们太丑， 那么你必须为此类符号安装一个好的 Unicode 后备字体，或者通过将 `dotspacemacs-mode-line-unicode-symbols` 设置为 `nil` 来禁用它们。

提示：Spacemacs 的一个不错的 Unicode 后备字体是 ttf-symbolaAUR

5.7 Visual Studio Code

难度 高级 | **篇幅** 23.3 KB | **所属** 文本编辑器

本节将讲到

- 安装
- 使用
- 配置
- 疑难解答

Code 是由微软开发的跨平台文本编辑器，基于 Electron 框架构建，并且极具扩展性。可以在编辑器自带的应用商店，或者从 https://marketplace.visualstudio.com/VSCode 中安装扩展。“Visual Studio Code” 是 Code - OSS仓库的二进制分发版本，该仓库遵循 MIT 许可证，但包含了微软特定的定制，并在专有许可证下发布（有关混合授权的说明，请参阅此 GitHub 评论）。另外还有一个由社区驱动、遵循 MIT 许可证的二进制发行版本名为VSCodium，默认情况下禁用了遥测功能。

安装

下列安装包提供 VSCode:

- **Code - OSS** — Arch Linux 官方开源版本。配置有Open VSX。

: https://github.com/microsoft/vscode || code包

- **Visual Studio Code** — 微软官方版本，专有软件。

: https://code.visualstudio.com/ || visual-studio-code-binAUR

- **VSCodium** — 社区驱动的完全开源的 VSCode 版本,不带微软附属和遥测功能,同时启用了Open VSX。

: https://vscodium.com/ || vscodiumAUR

其他可用版本：

- code-gitAUR(开发中的最新开源版本)
- visual-studio-code-insiders-binAUR(微软官方版本,每日更新)
- vscodium-binAUR(社区驱动的完全开源的 VSCode 版本,不带微软附属和遥测功能,二进制包)
- vscodium-gitAUR (社区驱动的完全开源的 VSCode 版本,不带微软附属和遥测功能,最新的开发版)

这些不同的版本都是基于Code - OSS构建的, 但具有不同的许可证和默认配置。值得注意的是，只有专有版本被允许使用官方拓展市场并使用微软的专有扩展，例如OmniSharp C# Debugger。后者通过握手机制进行验证，并不能被绕过。有关开源和专有 "Visual Studio Code "构建之间差异的更多信息，请查阅Code - OSS GitHub wiki.

扩展支持

Code 的一个主要特点是其灵活的API和丰富的扩展生态(托管在Visual Studio Marketplace上)。但是，根据Code的扩展商场使用条款，拓展商场只能和微软官方版本配合使用。因此code包并没有内置一个预配置的拓展商场，虽然它支持Open VSX registry，但其并不能提供相同范围的扩展。

警告：由微软开发的某些语言扩展现强制要求使用专有的Visual Studio Code版本，且无法通过修改`product.json`文件来绕开限制。目前已知受影响扩展包括C/C++和C# Dev Kit。

已知的解决方法:

- 请求开发者将扩展上传至Open VSX registry;

- 根据你使用的版本，安装对应的包来接入官方拓展市场：code-marketplaceAUR/vscodium-bin-marketplaceAUR/vscodium-marketplaceAUR/vscodium-electron-marketplaceAUR。这些软件包包含一个 pacman 钩子，在每次更新后会根据Github comment来更新`product.json`文件。

提示：当你尝试频繁修改 `product.json` 文件时，你可以设置一个的快捷键 更为快捷地重新加载IDE

使用

对于以下版本，运行`code`或`code --no-sandbox`即可启动：

- code包
- visual-studio-code-binAUR

对于以下版本，运行`code-git`即可启动：

- code-gitAUR

对于以下版本，运行`codium`即可启动：

- vscodium-binAUR/vscodiumAUR/vscodium-gitAUR/vscodium-electronAUR

如果你想打开多个实例，可以使用`-n`选项。

配置

注意：像code-featuresAUR这样通过修改`product.json`来打补丁的软件包，可以覆盖变量 `nameShort` 和 `dataFolderName`，这可能会使下文部分的描述不准确。

- code包 将配置文件保存在 `~/.config/Code - OSS/User/settings.json`，拓展保存在`~/.vscode-oss`。
- visual-studio-code-binAUR 将配置文件保存在 `~/.config/Code/User/settings.json`，拓展保存在`~/.vscode`。
- vscodiumAUR 及其衍生版本的配置文件保存在 `~/.config/VSCodium/User/settings.json`，拓展保存在`~/.vscode-oss`。

当从Code迁移到Codium时(反之亦然), 可以直接复制或移动配置文件，因为两者共享大部分代码库，设置是兼容的。

集成终端

点击*查看* > *集成终端*或使用快捷键 ` Ctrl + ` 打开集成终端。 `[Bash](../命令行解释器/Bash.md)` 作为默认终端不带任何附加选项的方式启动。

terminal.integrated.shell.linux 可以配置默认终端，
terminal.integrated.shellArgs.linux` 可以配置启动终端时的附加参数。

例子：

```
~/.config/Code/User/settings.json
"terminal.integrated.shell.linux": "/usr/bin/fish",
"terminal.integrated.shellArgs.linux": ["-l","-d 3"]
```

您可能会收到奇怪的提示，这是由于将集成终端参数设置成了外部终端。要解决此问题，请移除该句，或换用外部终端。

外部终端

如果你使用Terminator作为Arch的默认终端，而且在Visual Studio Code中遇到一个错误`Unable to launch debugger worker process (vsdbg) through the terminal. spawn truecolor ENOENT`，可以换另一个终端（比如，gnome-terminal包）供Visual Studio Code使用。

`"terminal.external.linuxExec": "Yours alternative terminal"` 用于设置调试默认终端。

例子：

```
~/.config/Code/User/settings.json
"terminal.external.linuxExec": "gnome-terminal"
```

在Wayland下原生运行

Code使用了Electron,你可以参照Wayland#Electron设置来让其在Wayland下原生运行。请注意，某些版本的应用会使用自带的Electron(而不是系统的Electron)，这种情况下应用将不会读取全局配置文件，你需要对环境变量进行修改(而不是全局配置文件)。

使用用户配置文件时，visual-studio-code-binAUR和code包都会读取`~/.config/code-flags.conf`处的配置文件。这仅针对这些包及其衍生版本,因为它们使用修改过的加载脚本来读取。

如果使用的是vscodium-binAUR, 其会加载`~/.config/codium-flags.conf`处的配置文件。注意从终端运行时，使用`codium` (而不是`vscodium`), 否则它将不会在Wayland下原生启动。

vscodiumAUR不会读取配置文件(因为它没有使用加载脚本)。其提供了一个桌面项文件 `vscodium-wayland.desktop` 以在Wayland下原生运行，在菜单中显示为"VSCodium - Wayland"。

原生文件对话框

如果你使用KDE/Plasma, 默认情况下VS Codium将会打开GTK文件对话框。请确保您已经安装了KDE桌面门户(xdg-desktop-portal-kde包)，并将环境变量设置为`GTK_USE_PORTAL=1`来解决这个问题。

远程SSH连接

官方的"Remote - SSH"扩展无法与code包（VSCode-OSS）兼容，有两种解决方案：

- 安装code-featuresAUR
- 尝试安装Open Remote-SSH 扩展（目前存在一个PR 使其支持 VSCode-OSS）。

疑难解答

KDE/Plasma 全局菜单失效

Visual Studio Code使用DBus传递菜单，尝试安装 libdbusmenu-glib包。

也可以直接:

```
$ cp /usr/share/applications/code.desktop ~/.local/share/applications/
```

然后注销或者重启KDE桌面。

无法将内容移至回收站

默认状态下，Electron使用`gio`删除文件。如果检测到Plasma将会自动设置为使用`kioclient5`删除文件。不同的回收站实现可以通过设置`ELECTRON_TRASH`环境变量来使用。

例如，要使用trash-cli包删除文件：

```
$ ELECTRON_TRASH=trash-cli code
```

这样一来，Electron就支持了`kioclient5`，`kioclient`，`trash-cli`，`gio`（默认）和`gvfs-trash`（不推荐）。详情可在此查看documentation page。

C#无法调试

您若想调试C#.NET（使用OmniSharp扩展），则需要通过AUR安装微软专版。原因是.NET Core调试器仅授权官方微软产品使用，参见这个github上的讨论。

若使用开源软件包，调试器失效时不会有太多信息。调试台仅会提示初始信息：

```
You may only use the Microsoft .NET Core Debugger (vsdbg) with
Visual Studio Code, Visual Studio or Visual Studio for Mac software
to help you develop and test your applications.
```

要使用开源软件包进行调试，可以选择netcoredbgAUR。要在VS Code中运行，请在项目的.NET Core启动配置中添加这个配置：

```
./.vscode/launch.json
"configurations": [
    {
...
    "pipeTransport": {
        "pipeCwd": "${workspaceFolder}",
        "pipeProgram": "/usr/bin/bash",
        "pipeArgs": ["-c"],
        "debuggerPath": "/usr/bin/netcoredbg"
    }
...
```

无法使用OmniSharp server打开.csproj，Microsoft.Common.props位置无效

您必须从mono切换到合适SDK版本的props。

```
/opt/dotnet/sdk/{VERSION}/Sdks/Microsoft.NET.Sdk/Sdk/Sdk.props
$(MSBuildExtensionsPath)\$(MSBuildToolsVersion)\Microsoft.Common.props
```

把导入修改成类似这样：

```
/opt/dotnet/sdk/{VERSION}/Sdks/Microsoft.NET.Sdk/Sdk/Sdk.props
/opt/dotnet/sdk/{VERSION}/Current/Microsoft.Common.props
```

OmniSharp错误，MSBuild无法定位

OmniSharp中提到了Arch Linux用户应当安装mono-msbuild包软件包。否则，您可能收到类似以下的错误：

```
OmniSharp Log
[info]: OmniSharp.MSBuild.Discovery.MSBuildLocator
        Registered MSBuild instance: StandAlone 15.0 - "~/.vscode/extensions/ms-
vscode.csharp-1.18.0/.omnisharp/1.32.11/omnisharp/msbuild/15.0/Bin"
            MSBuildExtensionsPath = /usr/lib/mono/xbuild
            BypassFrameworkInstallChecks = true
            CscToolPath = ~/.vscode/extensions/ms-
vscode.csharp-1.18.0/.omnisharp/1.32.11/omnisharp/msbuild/15.0/Bin/Roslyn
            CscToolExe = csc.exe
            MSBuildToolsPath = ~/.vscode/extensions/ms-
vscode.csharp-1.18.0/.omnisharp/1.32.11/omnisharp/msbuild/15.0/Bin
            TargetFrameworkRootPath = /usr/lib/mono/xbuild-frameworks
System.TypeLoadException: Could not load type of field
'OmniSharp.MSBuild.ProjectManager:_queue' (13) due to: Could not load file or assembly
'System.Threading.Tasks.Dataflow, Version=4.5.24.0, Culture=neutral,
PublicKeyToken=b03f5f7f11d50a3a' or one of its dependencies.
...
```

不过您依然有可能成功组建——则很可能取决于您是否也安装了mono包。

Omnisharp自带一个mono版本,因此如果它没有找到已安装的版本, 但是您想让omnisharp去寻找安装在计算机上的全局mono版本,您可以尝试在settings.json添加:

```
settings.json
"omnisharp.useGlobalMono:"always"
```

。

使用“Retry as Sudo”保存无效

这个特性在code包软件包中无效，因为微软不支持Arch软件包的打包方式（原生的而非捆绑的Electron）。详情参见FS#61516和upstream bug report。

二进制发行版visual-studio-code-binAUR没有这个问题，该特性在此版本中有效。

键盘变体与键位映射不相符

根据GitHub上的wiki页面：

: Switching keyboard layouts under some Linux window managers does not result in a change in the low level X window APIs VS Code uses to read the current keyboard layout. This means that VS Code ends up sometimes reading one of the other configured keyboard layouts and not the current active one. PR welcome...

根据wiki页面，有两种解决方案：

1. 确保`setxkbmap -query`返回的第一个键盘布局是你想在VS Code中使用的。
2. 在设置中使用`"keyboard.dispatch": "keyCode"`，并重启VS Code。这将彻底阻止VS Code确定您的键盘布局。

指令"..."未找到

在VS Coded官方编译版本中，`product.json`文件列出了所有能够使用某些特定API的扩展。然而,`product.json`在Code - OSS与VSCodium中是缺失的，尽管这似乎不是由于许可问题。这可能导致一些插件无法正常工作(例如Live Share)。但和强制打开官方拓展商场不同的是，这个解决方案得到了微软的认可。

您可以尝试通过安装一个pacman 钩子来解决，它将会在每次包更新时修补文件：

- 对于 code包, 安装 code-featuresAUR
- 对于 vscodiumAUR, 安装 vscodium-featuresAUR
- 对于 vscodium-binAUR, 安装 vscodium-bin-featuresAUR
- 对于 vscodium-electronAUR, 安装 vscodium-electron-featuresAUR

您也可以手动将相关条目添加到`product.json`文件中的`extensionAllowedProposedApi`部分：

- 对于 code包, 编辑 `/usr/lib/code/product.json`
- 对于 vscodiumAUR, 编辑 `/usr/share/vscodium/resources/app/product.json`
- 对于 vscodium-binAUR, 编辑 `/opt/vscodium-bin/resources/app/product.json`
- 对于 vscodium-electronAUR, 编辑 `/usr/lib/vscodium/product.json`

一个使Live Share工作的手动配置示例是:

```
product.json
...
  "extensionAllowedProposedApi": [
    "ms-vsliveshare.vsliveshare",
    "ms-vscode.node-debug",
```

```
    "ms-vscode.node-debug2"
  ]
...
```

再或者，您也可以考虑使用命令行标帜来启动，就如同GitHub拉取请求中所说的。

VS Live Share缺失API

要么使用上述解决方案编辑`product.json`，要么使用如下命令打开VS Code：

```
$ code --enable-proposed-api ms-vsliveshare.vsliveshare
```

另请注意，为了使此扩展程序正常工作，您需要安装在这里列出的依赖项。

指令'remote-containers.openFolder'未找到

如FS#63374所述，打开VS Code时启用remote-containers API：

```
$ code-oss --enable-proposed-api ms-vscode-remote.remote-containers
```

指令'GitHub Pull Requests: Configure Remotes...'导致错误（指令'pr.configureRemotes'未找到）

使用如下命令打开VS Code：

```
$ code --enable-proposed-api GitHub.vscode-pull-request-github
```

错误提示'Git: ssh_askpass: exec(/usr/lib/ssh/ssh-askpass): No such file or directory'

这个错误是由于SSH密匙加密且无法使用SSH agent造成的,参见这个问题报告。您可以尝试安装一个配置解释器（dialogue provider），例如SSH keys#x11-ssh-askpass或者其他的代替方案(例如，对于KDE用户您可以尝试使用ksshaskpass包)。

对于*ksshaskpass*，为了让Code找到它，您需要将其链接到`/usr/lib/ssh/ssh-askpass`:

```
# ln /usr/bin/ksshaskpass /usr/lib/ssh/ssh-askpass
```

或者为您的shell设置以下环境变量,参见:

```
GIT_ASKPASS=ksshaskpass
SSH_ASKPASS=ksshaskpass
SSH_ASKPASS_REQUIRE=prefer
```

为了禁用Code内置的*git-askpass*, 请在以下文件中添加:

```
~/.config/Code - OSS/User/settings.json
{
    "git.useIntegratedAskPass": false
}
```

终端中的截断字符

太宽的字符可能会被截断。例如Deno堆栈跟踪中使用的斜体粗体文本。

你可以将"terminal.integrated.rendererType"设置为"experimentalWebgl"来解决这个问题。

运行在Wayland下出现字体模糊

Code默认会在Xwayland下运行,如果你使用HiDPI屏幕，可能会有字体模糊的问题。你可以尝试让Electron强制在Wayland下运行——参见#在Wayland下原生运行。

或者如你的Wayland环境提供了运行未缩放的Xwayland应用程序的选项，你可以绕过这个问题。你可以附带参数`--force-device-scale-factor=`来运行Code,手动指定其缩放比例。

例如，将缩放比例设置成2:

```
$ code --force-device-scale-factor=2
```

找不到接口"org.freedesktop.Secret.Collection"

参见Code官方文档settings-sync#_troubleshooting-keychain-issues。

使用VSCodium时无法进行Github身份验证

参照这个Github评论,当要连接到Github时,将弹出的网址中的"vscodium"改为"vscode"，随后将识别令牌（identification token）拷贝到VSCodium中。如果还是不成功,你可以尝试安装一个密钥环，例如gnome-keyring包，或者你也可以参照Code官方文档与这个Github链接来自己创建一个密钥环。

如果遇到登录账户后会自动退出的情况，你也可以尝试安装密匙环gnome-keyring包来解决这个问题，如果你使用的是KDE,可能还需要安装qtkeychain-qt5包。

找不到密钥环"An OS keyring couldn't be identified"

在一些桌面环境上(例如i3),Code可能会找不到密钥环。你需要手动指定Code使用的密匙环,例如gnome-keyring,在以下文件中添加:

```
~/.vscode-oss/argv.json
{
    ...
    "password-store": "gnome-libsecret",
}
```

这个文件地址仅适用于code包包，如果你使用的是其他版本的Code,请相应调整`argv.json`的目录。例如，如果安装的是visual-studio-code-binAUR，其地址应调整为`~/.vscode/argv.json`。

5.8 Vim

难度 高级 | **篇幅** 24.0 KB | **所属** 文本编辑器

本节将讲到

- 安装
- 用法
- 配置

- 文件合并
- 技巧和建议

Vim)是一个终端文本编辑器。作为vi的一个扩展版本，它具有以下附加功能：语法高亮，全面的帮助系统，本地脚本（Vim script），文本选择的可视模式和文件比较（vimdiff(1)）。以及有限功能的工具，如 rview(1) 和 rvim(1)。

安装

安装下面两个软件包中的一个：

- vim包——提供Python, Lua, Ruby 和 Perl解释器支持，但没有 GTK/X 支持
- gvim包——除了提供和 *vim* 一样的功能外，还提供GTK/X支持。

注意：

- *vim* 包不包含 Xorg 支持。具体而言，Vim缺失 `+clipboard` 特性，因而不能够使用 *primary* 和 *clipboard* 选择缓冲区。*gvim* 同时提供命令行版本带`+clipboard`的Vim。
- 非官方源herecura也提供数个Vim/gVim变种版本: `vim-cli`、`vim-gvim-common`、`vim-gvim-gtk3`、`vim-rt` 和 `vim-tiny`。

用法

有关如何使用Vim的基本概述，请遵循vim教程运行*vimtutor*（控制台版本）或*gvimtutor*（图形界面版本）。

Vim包含了一个广泛的帮助系统，可以用`:h *subject*`命令来访问。*subject*可以是命令，配置选项，热键绑定，插件等。使用`:h`命令（不带任何*subject*）来获取帮助系统的相关信息以及在不同的主题之间切换。

配置

Vim的用户特定配置文件位于主目录`~/.vimrc`，当前用户的Vim文件位于`~/.vim/`；全局配置文件为`/etc/vimrc`，全局Vim文件是位于`/usr/share/vim/` 的 `defaults.vim` 和 `archlinux.vim`。 对于 gVim，用户特定的配置文件为 `~/.gvimrc`，全局配置文件为 `/etc/gvimrc`。

注意：

- 常用的功能，如语法高亮在 `defaults.vim` 中启用，当没有 `~/.vimrc` 时加载。将 `let skip_defaults_vim=1` 添加到 `/etc/vimrc` 以完全禁用加载 `defaults.vim`。。或者，若要启用 `defaults.vim`，即便 `~/.vimrc` 存在，请参见 vim 中的 `:h defaults`。
- gVim 同时加载 Vim 和 gVim 的配置文件，而 Vim 只加载 Vim 的配置文件。

剪贴板

Vim命令如 `:yank` 或 `:put` 通常使用未命名寄存器，默认情况下对应于 `""` 寄存器。如果 `+clipboard` 功能可用且它的值包含 `unnamed`，那么 Vim yank, delete, change 和 put 操作（通常会转到未命名寄存器）将使用剪贴板寄存器 `"*` 替代，对应于 X 中的 `PRIMARY` 缓冲区。

要更改默认寄存器，您可以 `:set clipboard=unnamedplus` 使用 `"+` 寄存器。`"+` 剪贴板寄存器对应于X中的 `CLIPBOARD` 缓冲区。应该注意，`clipboard` 选项可以设置为逗号分隔的值。如果您 `:set clipboard=unnamedplus,unnamed`，那么除了 `"+` 寄存器之外，yank 操作还会将被拉取的文本复制到 `"*` 寄存器(但是，delete, change 和 put 操作仍然只在 `"+` 寄存器上操作)。

欲见更多信息，请参见 `:help 'clipboard'`。 可以为 `clipboard` 功能设置其他值。您可以使用 `:help clipboard-unnamed` 进入帮助主题，查找可为此功能设置的第一个有效值，然后查找所有其他有效值的帮助。

提示：

- 可以创建复制和粘贴操作的自定义快捷键。参见例如 用于绑定 `ctrl + c`, `ctrl + v` 和 `ctrl + x`。
- 当 vim 退出时，X 剪贴板会被刷新。要使 vim 选区在 X 剪贴板中持久化，您需要一个剪贴板管理器。或者，您可以将 `autocmd VimLeave * call system("echo -n $'" . escape(getreg(), "'") . "' | xsel --input --clipboard")` 添加到你的 `.vimrc`（需要 xsel包 包）。

语法高亮

启用语法高亮（Vim支持许多语言的语法高亮）：

```
:filetype plugin on
:syntax on
```

自动换行显示

`wrap`默认是开启的，这会使Vim在一行文本的长度超过窗口宽度时，自动将放不下的文本显示到下一行。`wrap`只会影响文本的显示，文本本身不会被改变。

自动换行显示一般在该行窗口能容纳下的最后一个字符发生，即使刚好是在一个单词的中间。更为智能的自动换行显示可以用`linebreak`来控制。当用`set linebreak`开启时，自动换行发生在字符串选项`breakat`中列出来的字符之后。默认情况下，`breakat`包含空格和一些标点符号（参考`:help breakat`）。

被换行的字符一般在下一行的开头开始显示，没有任何相应的缩进。breakindent 指示Vim在换行时将缩进考虑在内，因而新行将与原本要显示的文本有相同的缩进。

breakindent行为可以用breakindentopt选项来调整，比如说在Python文件中，新行将在原本缩进的基础上再缩进4个空格（更多细节参考：help breakindentopt）：

```
autocmd FileType python set breakindentopt=shift:4
```

使用鼠标

Vim可以使用鼠标，但只在某些终端上起作用：

- 基于xterm/urxvt的终端模拟器
- 带有gpm包的Linux控制台（更多细节请参阅控制台鼠标支持）
- PuTTY。

要开启这个功能，将下面这行代码加入~/.vimrc中：

```
set mouse=a
```

mouse=a 选项在 defaults.vim 中已被设置。

注意：如果可以访问X服务器，复制/粘贴将使用 "* 寄存器，参见剪贴板。按住shift key键可以使用 xterm 处理鼠标按钮。

跨行移动光标

默认情况下，在行首按←或者在行尾按→不能将光标移动至上一行或下一行。

如要改变默认行为，将set whichwrap=b,s,<,>,[,]加至你的~/.vimrc文件中。

文件合并

Vim自带了一个文件差异编辑器（一个用来显示多个文件之间的差异还可以方便的将其合并的程序）。用*vimdiff*来启动它——指定所需文件即可：vimdiff *file1* *file2*。以下是*vimdiff*\-specific命令的清单。

行为	快捷键
下一差异	]c
上一差异	[c
差异导入	do
差异导出	dp
打开折叠	zo
关闭折叠	zc
重新扫描文件	:diffupdate
窗口切换	Ctrl+w+w

技巧和建议

显示行号

使用:`set number`来显示行号。默认显示绝对行号，可用:`set relativenumber`开启相对行号。当两种显示方式同时开启时，当前行显示绝对行号，其它行显示相对行号。

使用:`*line number*` or `*line number*gg`跳转到指定行号。跳转都记录在一个跳转列表中，更多细节参考:`h jump-motions`。

拼写检查

Vim有拼写检查的功能，用下面的命令开启:

```
set spell
```

Vim默认只安装了英语字典（在 `/usr/share/vim/vim82/spell/`）。更多字典可以通过搜索 `vim-spell` 在官方软件仓库中找到。其他字典可以在Vim的FTP archive中找到。把下载的字典文件存入`~/.vim/spell/`中，并使用以下命令启用：`:setlocal spell spelllang=*en_us*`(将`*en_us*` 换成所需的字典的名称)。

行为	快捷键
下一个拼写错误	`]s`
上一个拼写错误	`[s`
拼写纠正建议	`z=`
拼写正确，添加到用户正确字典	`zg`
在会话中当作正确拼写	`zG`
拼写错误，添加到用户错误字典	`zw`
在会话中当作正确拼写	`zW`
重新进行拼写检查	`:spellr`

提示：

- 如果需要针对两种语言进行拼写检查（例如英语与德语），在`~/.vimrc`或`/etc/vimrc`中添加`set spelllang=*en,de*`并重启Vim即可。
- 您可以通过使用FileType插件和用于文件类型检测的自定义规则，为任意文件类型（例如.*txt*）启用拼写检查。 要对以.*txt*结尾的任何文件启用拼写检查，请创建文件 `/usr/share/vim/vimfiles/ftdetect/plaintext.vim`，并将 `autocmd BufRead,BufNewFile *.txt set filetype=plaintext` 插入该文件。接下来，将 `autocmd FileType plaintext setlocal spell spelllang=*en_us*` 插入到 `~/.vimrc` 或 `/etc/vimrc` 中，然后重新启动Vim。一定要编辑这一行（特别是 `*.txt`），以包括用于拼写检查的文件类型。
- 如果想只对LaTeX（或TeX）文档起用拼写检查，在`~/.vimrc`或`/etc/vimrc`添加`autocmd FileType **tex** setlocal spell spelllang=*en_us*`，重启Vim即可。至于非英语语言，替换上述语句中的`en_us`为相应语言代码即可。

保存运行状态

通常，退出 vim 会丢弃所有不重要的信息，如打开的文件、命令行历史记录、删除的文本等。可以通过以下方式配置保存这些信息。

viminfo 文件

viminfo 文件可用于存储命令行历史、查找字符串历史、输入行历史、寄存器内容、文件标记、文件内的位置标记、最后的查找/替换模式（在会话中使用 n 和 & 的查找模式）、缓冲区列表以及您可能定义的任何全局变量。要使 viminfo 模式可用，您安装的 vim 版本必须使用 +viminfo 功能进行编译。

通过在 ~/.vimrc 文件中添加特定内容可以配置保存在 viminfo 文件中的内容，以下是一个示例：

```
set viminfo='10,<100,:100,%,n~/.vim/.viminfo
```

其中每个参数前面都有一个标识符：

```
'q  : q 是记住的编辑过的文件数
<m  : m 为每个寄存器保存的行数
:p  : p 是记住的历史命令行数
%   : 保存和恢复缓冲区列表
n...: viminfo 文件的完整路径名称（注意，这是一个字面上的 "n"）
```

请参阅官方的 viminfo 文档，了解当前的 viminfo 文件当会话信息更新时如何修改它，比如从当前会话中的几个缓冲区中退出。

会话文件

会话文件可用于保存随时间变化的任意数量的特定会话的状态。一个不同的会话文件可以用于您感兴趣的每个会话或项目。要使该模式可用，您安装的vim版本必须使用+mksession功能进行编译。

在会话中，:mksession[!] [*my_session_name.vim*] 将 vim 脚本写入当前目录中的 *my_session_name.vim*，如果不提供文件名，则默认为 Session.vim。可选的 ！将删除具有相同名称和路径的已经存在的会话文件。

从终端启动 *vim* 时，可以恢复 vim 会话：

```
$ vim -S [my_session_name.vim]
```

或者在已经打开的会话缓冲区中运行 vim 命令：

```
:source my_session_name.vim
```

vim文档中详细介绍了保存的内容以及会话文件选项的其他细节。此处有注释过的示例。

记录光标位置

Vim可以记录上次打开某一文件时的光标位置，并在下次打开同一文件时将光标移动到该位置。要开启该功能，在配置文件~/.vimrc中加入以下内容：

```
augroup resCur
  autocmd!
  autocmd BufReadPost * call setpos(".", getpos("'\""))
augroup END
```

参见 Vim wiki 上的 Restore cursor to file position in previous editing session

用 vim 替代 vi

创建一个别名，如下：

```
alias vi=vim
```

或者，如果你想输入sudo vi而得到vim，安装vi-vim-symlinkAUR，它将移除vi并用一个符号链接vim代替。您还可以自己创建这个符号链接，并将其放置在 path 中比 /usr/bin 更高的位置，以使其更为优先。

DOS/Windows回车问题

打开MS-DOS或Windows下创建的文本文件时，经常会在每行行末出现一个^M。这是因为Linux使用Unix风格的换行，用一个换行符（LF）来表示一行的结束，但在Windows、MS-DOS中使用一个回车符（CR）接一个换行符（LF）来表示，因而回车符就显示为^M。

可使用下面的命令删除文件中的回车符：

```
:%s/^M//g
```

注意，^代表控制字符。输入^M的方法是按下Ctrl+v,Ctrl+m。

另一个解决方法是，安装 dos2unix包，然后执行 dos2unix <文件名>。

注意：另一个简单的方法是更改 fileformat 设置。set ff=unix 以转化DOS/Windows行尾为Unix行尾。要做到相反，只要 set ff=dos，就可以将Unix行尾转换成DOS/Windows行尾。

gVim窗口底部的空格

如果窗口管理器设置为忽略窗口大小提示，gVim会将非功能区域填充为GTK主题背景色。

解决方案是调整gVim在窗口底部保留的空间大小。将下面的代码加入 ~/.vimrc中：

```
set guiheadroom=0
```

注意：如果将其设为0，将无法看到底部的水平滚动条。

Vim 作为 pager

如果使用脚本，那么 Vim 可以用作 terminal pager，这样您就可以获得各种Vim功能，如颜色方案。

Vim 附带了 `/usr/share/vim/vim90/macros/less.sh` 脚本，您可以为其创建别名。注意，该脚本不支持 less(1) § OPTIONS 中提到的任何命令行标志。

另外，还有 vimpager包 Vim 脚本。要更改默认 pager，请导出 `PAGER` 环境变量。注意，并非所有命令行标志都受支持；GitHub上提供了支持的标志列表。

高亮搜索结果

为了突出显示在搜索中匹配的第一个字符串，请在 `~/.vimrc` 中添加以下行：

```
set incsearch
```

为了突出显示在搜索中匹配的所有字符串，在 `~/.vimrc` 中添加以下行：

```
set hlsearch
```

注意：

- 设置 `hlsearch` 将突出显示所有匹配项，直到进行更多搜索。这种行为可能是不被希望的，因此要在下次搜索之前暂时禁用高亮显示，请运行 `:nohlsearch`。如果您发现自己经常运行此命令，请考虑将其绑定到按键。
- 当在其他命令（如 `s` 或 `g`）中匹配正则表达式时，也会观察到这种行为。

插件

向Vim添加插件可以提高您的效率。 插件可以改变Vim的界面，添加新命令，代码完成支持，使用Vim集成其他程序和实用程序，添加对其他语言的支持等等。

提示：有关常用插件的列表，请参阅 Vim Awesome。

安装

使用内置包管理器

Vim 8增加了原生加载第三方插件的可能性。可以通过在 `~/.vim/pack` 中存储第三方软件包来使用此功能。这个文件夹的结构与典型的插件管理器的结构略有不同，典型的插件管理器通常每个插件只有一个目录。下面是一个典型的安装过程和目录结构 (以 Tim Pope 的 vim-surround 插件为例) :

```
$ mkdir -p ~/.vim/pack/tpope/start
```

需要注意的是，`~/.vim/pack/tpope` 是一个*包目录*，它在 Vim文档中被宽松定义为包含一个或多个插件的目录。插件存储库不应下载到此目录。包目录的名称也是任意的。您可以选择将所有插件保存在一个包目录中，或者如我们的示例中所示，使用作者的 GitHub 名称 `tpope`。

包目录可以包含以下子文件夹:

- `start` - 这个子文件夹中的插件将在 Vim 启动时自动加载。这是最常用的位置。
- `opt` - 这个子文件夹中的插件可以按需加载，方法是在 Vim 中发出 `:packadd` 命令。

现在切换到 `start` 文件夹并检出插件存储库：

```
$ cd ~/.vim/pack/tpope/start
$ git clone https://tpope.io/vim/surround.git
```

这会创建一个额外的子文件夹 `~/.vim/pack/tpope/start/surround`，插件文件就放在这个子文件夹中。

接下来，如果插件包含帮助文件，请更新帮助索引：

```
$ vim -u NONE -c "helptags surround/doc" -c q
```

启动 Vim 时，插件将自动加载。除了特定于插件的选项外，不需要对 `~/.vimrc` 进行任何更改。

使用插件管理器

插件管理器允许以类似的方式安装和管理插件，而与在何种平台上运行 Vim 无关。它本身是一个插件，其功能是作为其他 Vim 插件包管理器。

- Vundle 是现在最流行的 Vim 插件管理器，从 vundleAUR 或 vundle-gitAUR 获取。
- Vim-plug 是一个极简的 Vim 插件管理器，有许多的特性，比如按需插件加载和并行升级，从 vim-plugAUR 或 vim-plug-gitAUR 获取。
- pathogen.vim 是一个简单的用于管理 Vim 的 runtimepath 的插件，从 vim-pathogenAUR 或 vim-pathogen-gitAUR 获取。
- Dein.vim 是一个替代 NeoBundle 的插件管理器，从 vim-deinAUR 或 vim-dein-gitAUR 获取。

使用 Arch 软件库

vim-plugins包组包组下有许多插件。 使用`pacman -Sg vim-plugins`来列出可用的插件，然后你可用 pacman 安装。

著名的插件

cscope

Cscope是用于浏览项目的工具。 通过导航到字/符号/函数并调用 cscope（通常使用快捷键），它可以找到：调用函数的函数，函数定义等等。

安装 cscope包 包。

将 cscope 默认文件复制到 Vim 将自动读取的位置:

```
mkdir -p ~/.vim/plugin
wget -P ~/.vim/plugin https://cscope.sourceforge.net/cscope_maps.vim
```

注意：在 Vim 的 7.x 版本中，你可能需要在 `~/.vim/plugin/cscope_maps.vim` 中取消下列行的注释来启用 cscope 快捷键：

```
set timeoutlen=4000
set ttimeout
```

创建一个文件，该文件包含了你希望 cscope 索引的文件的清单（cscope 可以操作很多语言，下面的例子用于寻找 C/C++ 中的 *.c*、*.cpp*和*.h* 文件）：

```
cd /path/to/projectfolder/
find . -type f -print | grep -E '\.(c(pp)?|h)$' > cscope.files
```

创建 cscope 将读取的数据文件：

```
cscope -bq
```

注意：必须从当前路径浏览工程文件，也可以设置 `$CSCOPE_DB` 变量指向 `cscope.out` 文件，并导出。

默认快捷键：

```
 Ctrl-\ and
      c: 查找调用指定函数的函数
      d: 查找指定函数调用的函数
      e: 查找指定 egrep 模式
      f: 查找指定文件
      g: 查找指定定义
      i: 查找指定文件 #including 的文件
      s: 查找指定 C 符号
      t: 查找指定的字符串
```

可随意改变这些快捷键。

```
#Maps ctrl-c to find functions calling the function
nnoremap <C-c> :cs find c <C-R>=expand("<cword>")<CR><CR>
```

Taglist

Taglist 提供源码文件的结构概览，使你能更高效的浏览不同语言的源文件。

安装 vim-taglistAUR 包。

将下列设置添入文件`~/.vimrc`:

```
let Tlist_Compact_Format = 1
let Tlist_GainFocus_On_ToggleOpen = 1
let Tlist_Close_On_Select = 1
nnoremap <C-l> :TlistToggle<CR>
```

常见问题

gVim运行缓慢

GTK 3 GUI 下 vim 可能比 GTK 2 下运行缓慢（参阅 FS#51366）。可安装gvim-gtk2AUR来避免该问题。

参阅

官方资源

- Vim主页
- Vim文档
- Vim Wiki
- Vim脚本

教程

- vim 入门教程
- 中文版《A Byte of Vim》
- vi参考指南
- vi-Vim 图解教程
- Vim 简介与教程
- Open Vim - Vim 学习工具集合
- Vim 渐进式教程
- 学习 Vim 在 2014
- 有效率进行文本编辑的七个习惯
- 基本 Vim 技巧

视频

- Vimcasts - *.ogg* 格式的视频教程。
- Vim Tutorial Videos - 从入门到精通，各种视频教程

速查表

- https://devhints.io/vim
- https://vim.rtorr.com/ - 一个移动友好的 Vim 速查表 - 源代码

游戏

- Vim Adventures
- VimGolf

配置范例

- nion's

- 来自 Amir Salihefendic 的一个详细配置
- Bart Trojanowski
- Steve Francia 的 Vim 发行版
- Vim Awesome - Vim 插件
- W4RH4WK 的 Vim 配置文件
- 来自 askapache 的快速 vimrc/色彩方案
- 基础 vimrc
- Usevim

色彩方案

- Vivify
- Vim 色彩方案定制

终端模拟器

这些终端解决同一件事：在图形会话里开个 shell 窗口，但深浅差很大——从最小配置的传统 X 终端，到以 GPU 渲染和配置文件为核心的新式终端，再到不开 X、跑在帧缓冲或 KMS 的一类。先分清是桌面还是纯控制台。

按篇幅读：Alacritty、Foot 的配置、提示与技巧、排错最全，先读；Konsole、Yakuake 按桌面挑；Rxvt-unicode 最长，要剪贴板或 Perl 插件时再读。Mlterm、Fbterm、KMSCON 可跳，除非真要离开图形会话。别当换外观的窗口——插件、字库支持各在各家终端里成立。

6.1　Mlterm

难度 入门 | **篇幅** 1.2 KB | **所属** 终端模拟器

本节将讲到

- 介绍
- 安装
- 使用
- 配置

介绍

Mlterm 是一个轻量级的, 高性能的, 多功能的终端模拟器。

安装

请安装 mlterm-gitAUR。

使用

打开 Mlterm 后, 即可使用 bash 开始工作, 若需要支持输入法, 请参见 Mlterm#配置输入法。

配置

在 Mlterm 中, 我们使用 Ctrl+鼠标Right 组合键打开配置菜单, 如果无效, 请尝试执行 /usr/libexec/mlterm/mlconfig 手动打开。

调整配置之后, 可以通过 **Apply & Exit** 临时应用, 或 **Save & Exit** 永久应用。

配置输入法

Mlterm 默认不使用输入法, 需要在配置菜单中手动设置, 一般选择 **XIM** 即可, 如果使用 iBus, 请选择 **iBus**。

配置外观

滚动条

在配置界面中, 进入 Scrollbar 标签, 可以调整滚动条位置。

背景

在配置界面中, 进入 Background 标签, 可以调整背景, 以下是各个选项简要说明:

- Image: 使用一张图片作为背景。
- Pseudo Transport: 透明, 需要注意的是, 这只能看到桌面墙纸, 不能看到其他窗口。

6.2 Konsole

难度 入门 | **篇幅** 1.4 KB | **所属** 终端模拟器

本节将讲到

- 安装
- 问题解决
- 参见

翻译状态:

- 本文（或部分内容）译自 Konsole，最近一次同步于 2023-05-27，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Konsole 是 KDE 桌面的一个终端模拟器。

安装

安装 konsole包 软件包。

问题解决

真彩程序无法正确显示

这是因为默认情况下，*konsole* 不会正确设置 `TERM` 环境变量。要设置它，编辑 *konsole* 配置或新建一个配置：

```
~/.local/share/konsole/konsole.profile
[General]
Name=konsole
Environment=TERM=konsole-direct,COLORTERM=truecolor
```

注意：在将 `TERM` 设置为 `konsole-direct` 后，一些程序可能无法正常工作。这时，使用 `konsole-256color`。

参见

- KDE UserBase Wiki 上的 Konsole

6.3 Fbterm

难度 入门 | **篇幅** 3.6 KB | **所属** 终端模拟器

本节将讲到

- 功能
- 安装
- 自定义
- 提示与技巧

Fbterm (**F**rame **b**uffer **term**inal)是内核终端的直接替代：一个没有 Xorg 也能使用的终端模拟器。

警告：Fbterm 的开发已经停止。

功能

摘自 http://code.google.com/p/fbterm/:

: FbTerm 是一个使用 frame buffer 设备或 VESA 显卡的终端模拟器，功能包括：

: - 若加速滚动开启的话，速度可和linux核心的终端媲美

- 使用fontconfig选择字体，和Qt/GTK+上用的应用一样，使用freetype2来绘制字体
- 动态创建和销毁最多10个原生运行默认shell的窗口
- 记录任何窗口的回滚历史
- 自动检测目前本地化环境下的文本内码，支持双字节脚本，比如中文、日文等
- 在线热键切换配置的额外文本内码
- 当gpm服务器运行时，可使用鼠标在窗口间复制和粘贴选择的文本
- 可改屏幕显示方式，比如：屏幕翻转
- C/S结构（客户端/服务器）的轻量级输入法框架
- 背景图片

安装

如果您仍想要使用Fbterm，您需要安装fbtermAUR。

注意：官方软件仓库已经没有 Fbterm

安装完成之后，请注意其后续说明：

```
==> 若想使用非根用户运行fbterm，需要把用户加入video组：
sudo gpasswd -a YOUR_USERNAME video
==> 若想非根用户可使用键盘快捷方式，需要：
sudo setcap 'cap_sys_tty_config+ep' /usr/bin/fbterm
或者：
sudo chmod u+s /usr/bin/fbterm
```

自定义

配置文件

Fbterm 使用 ~/.fbtermrc 来进行配置。该文件将在第一次运行 Fbterm 之后自动生成。文件内有详细的注释，可以帮助您了解如何配置 Fbterm。

字体

Fbterm 使用 fontconfig 管理字体，试一下每一个字体直到可以渲染字符。

要修改字体，从 `fc-list` 给出的列表中选定一个字体，用 `--font-names` 选项指定。

输入法支持

目前，Fbterm支持不同的输入法，通过作为一个独立的输入法服务器的客户端。Internationalization#Input methods 记录了 Arch 支持的几个程序。

中文用户可以安装AUR软件仓库中的 fcitx-fbterm-gitAUR 或 fcitx5-fbterm-gitAUR 来输入中文。在安装好软件包之后，直接启动 `fcitx-fbterm-helper` 或修改 `~/.fbtermrc` 文件中的 `input-method` 行为

```
input-method=fcitx-fbterm
```

启动 Fbterm 即可。

提示与技巧

背景图

想要使用背景图片，Fbterm 可以设置成启动时截取 frame buffer 设备的屏幕。

下面的脚本 (使用 fbvAUR 图形查看器) 是在man页面推荐的：

```
#!/bin/bash
# fbterm-bi: a wrapper script to enable background image with fbterm
# usage: fbterm-bi /path/to/image fbterm-options
echo -ne "\e[?25l" # hide cursor
fbv -ciuker "$1" << EOF
q
EOF
shift
export FBTERM_BACKGROUND_IMAGE=1
exec fbterm "$@"
```

白色字体

默认配置下，fbterm 把白色的文字显示成灰色，即使使用 -f 7 开关也不行。 可以通过在 fbterm 中运行 echo 一次来得到真正的白色，例如：

```
echo -en "\e]P7ffffff"
```

6.4 Foot

难度 入门 | **篇幅** 4.1 KB | **所属** 终端模拟器

本节将讲到

- 安装
- 配置
- 提示与技巧
- 疑难解答

foot 是一个快速、轻量和简约的 Wayland 终端模拟器。

安装

安装 foot包 软件包或 foot-gitAUR 开发版本。

此外还可以安装 foot-terminfo包 以获取增强的 terminfo。

配置

foot 加载 `$XDG_CONFIG_HOME/foot/foot.ini` 中的配置文件（默认为 `$HOME/.config/foot/foot.ini`）。配置文件模板位于 `/etc/xdg/foot/foot.ini`，复制模板到 `$XDG_CONFIG_HOME/foot/foot.ini` 并取消要修改设置的注释。重启 *foot* 以应用新设置。

手册页面 foot.ini(5) 提供了关于如何配置 *foot* 的详细信息。

颜色

您可以通过修改 `[colors]` 节自定义颜色。

主题位于 `/usr/share/foot/themes`。要应用主题，请在 `[main]` 下添加 `include` 关键字。

```
$HOME/.config/foot/foot.ini
[main]
include=/usr/share/foot/themes/主题名称
```

服务器（守护进程）模式

正常运行时，*foot* 会为每个窗口启动一个新的 *foot* 进程。

foot 也可以在服务器模式下运行。在该模式下，一个进程会托管多个窗口。所有 Wayland 通信、VT 解析和渲染都在服务器进程中完成。

注意：

- 在服务器模式下运行 *foot* 的优势包括减少内存占用和启动时间。
- 缺点是会影响性能；所有窗口的输入和输出都在同一线程中复用（但每个窗口都有自己的渲染线程）。这意味着，如果一个窗口忙于输出等工作，其他窗口就会受到影响。此外，如果服务器进程崩溃，所有窗口都将消失。

运行 *footclient* 以打开新窗口，其会一直运行到终端窗口关闭，然后以客户端进程（通常是 shell）的退出值退出。

启用/启动用户单元 `foot-server.service` 或相应的 `foot-server.socket` 以激活套接字，以便在登录时自动启动脚本服务器。

提示与技巧

GNOME

显示边框

目前，GNOME 不支持 Wayland 上的服务器端装饰。在 GNOME 上运行的 *foot* 将显示一个普通的平面窗口。您可以配置 *foot* 显示边框，以便更容易区分不同的 *foot* 窗口。

```
$HOME/.config/foot/foot.ini
[csd]
border-width=2
border-color=ff404040
```

terminfo

标准的 foot包 使用 ncurses 的 terminfo(5)。安装 foot-terminfo包 以使用 *foot* 的上游 terminfo，其包含一些功能改进， 主要针对 tmux 优化，如应用程序同步更新和 24 位颜色支持。

请参阅 foot 的 wiki 条目以获取更多信息。

疑难解答

登录后 foot-server 未启动

`foot-server.service` 单元和 `foot-server.socket` 单元需要 `graphical-session.target`，但是 Sway 不会自动设定这些，请遵循 Sway#使用 systemd 管理仅用于 Sway 的守护程序创建一个 `sway-session.target` 单元并在 Sway 启动时将其启动。

6.5 Yakuake

难度 入门 | **篇幅** 5.4 KB | **所属** 终端模拟器

本节将讲到

- 安装
- 使用
- 配置
- Yakuake 脚本
- 疑难解答

Yakuake 是一个适用于 KDE 的下拉式终端，类似于 GNOME 的 Guake、Tilda或 Quake 中使用的终端。

安装

安装 yakuake包 软件包。

使用

安装后，您可以在终端启动 yakuake：

```
$ yakuake
```

Yakuake 启动后，您可以点击“配置 Yakuake” 通过点击*菜单*按钮（界面右下中间）并选择*配置键盘快捷键*以更改收放终端的热键，默认是 `F12`。

配置

Plasma 上的透明/模糊背景

尽管大多数配置选项可以在 Yakuake GUI 更改，一些选项只能借助修改配置文件更改，例如 Plasma 下的模糊背景选项。

注意：在 Yakuake 的 Konsole 配置文件中启用 *Blur background* 选项会为 Yakuake 的窗口启用模糊，但将显示警告如 `Konsole` 在桌面特效启用前启动。您需要重启 `Konsole` 来显示透明背景。 见 有关此警告的上游漏洞报告。此警告有一定误导性，因为 Yakuake 使用不支持模糊/透明的 KonsolePart，但 Yakuake 本身可以对其窗口启用此效果。

要为 Yakuake 启用模糊背景，编辑以下文件：

```
~/.config/yakuakerc
[Appearance]
Blur=true
Translucency=true
```

然后重启 Yakuake 以应用更改。

Yakuake 脚本

如 Guake 一样，Yakuake 允许运行时发送 D-Bus 信息以控制它。因此，该特性可被用于在用户定义的会话启动 Yakuake。您可以创建标签页，为其指定名称，并在任何打开的标签页中要求运行任何特定命令，或者只是显示/隐藏 Yakuake 窗口：在终端中手动操作或通过为其创建自定义脚本。

下面是该脚本的一个示例。这包括打开标签页，重命名标签页，拆分 shell 以及运行命令。

```
#!/bin/bash
# 基于用户首选项启动 Yakuake。这些信息基于 https://forums.gentoo.org/viewtopic-t-873915-start-0.html
# 前一个网站中添加会话部分已损坏，故采用此：https://koston.pl/blog/sublime-text-3-cheatsheet-modules-web-develpment/

# 在 Yakuake 不接受 fcitx 输入时需要此行
/usr/bin/yakuake --im /usr/bin/fcitx --inputstyle onthespot &

# 在发送 dbus 命令前让 Yakuake 等待 2 秒
sleep 2

# 在 'user' 标签页启动 htop 然后向右拆分 'user' 终端，并在新终端运行 iotop
TERMINAL_ID_0=$(qdbus org.kde.yakuake /yakuake/sessions
org.kde.yakuake.terminalIdsForSessionId 0)
qdbus org.kde.yakuake /yakuake/tabs setTabTitle 0 "user"
qdbus org.kde.yakuake /yakuake/sessions runCommandInTerminal 0 "htop"
qdbus org.kde.yakuake /yakuake/sessions org.kde.yakuake.splitTerminalLeftRight
"$TERMINAL_ID_0"
qdbus org.kde.yakuake /yakuake/sessions runCommandInTerminal 1 "iotop

# 上下拆分并启动根会话（提示输入密码）
SESSION_ID_1=$(qdbus org.kde.yakuake /yakuake/sessions org.kde.yakuake.addSession)
TERMINAL_ID_1=$(qdbus org.kde.yakuake /yakuake/sessions
org.kde.yakuake.terminalIdsForSessionId "$SESSION_ID_1")
qdbus org.kde.yakuake /yakuake/tabs setTabTitle 1 "root"
qdbus org.kde.yakuake /yakuake/sessions runCommandInTerminal 2 "su"
qdbus org.kde.yakuake /yakuake/sessions org.kde.yakuake.splitTerminalTopBottom
"$TERMINAL_ID_1"
qdbus org.kde.yakuake /yakuake/sessions runCommandInTerminal 3 "su"

# 在其自己的标签页启动 irssi
qdbus org.kde.yakuake /yakuake/sessions org.kde.yakuake.addSession
qdbus org.kde.yakuake /yakuake/tabs setTabTitle 2 "irssi"
qdbus org.kde.yakuake /yakuake/sessions runCommandInTerminal 4 "ssh home -t 'tmux
attach -t irssi; bash -l'"

# 在其自己的标签页启动 ssh shell，并左右拆分
SESSION_ID_2=$(qdbus org.kde.yakuake /yakuake/sessions org.kde.yakuake.addSession)
TERMINAL_ID_2=$(qdbus org.kde.yakuake /yakuake/sessions
org.kde.yakuake.terminalIdsForSessionId "$SESSION_ID_2")
qdbus org.kde.yakuake /yakuake/tabs setTabTitle 3 "work server"
qdbus org.kde.yakuake /yakuake/sessions runCommandInTerminal 5 "ssh work"
qdbus org.kde.yakuake /yakuake/sessions org.kde.yakuake.splitTerminalLeftRight
"$TERMINAL_ID_2"
qdbus org.kde.yakuake /yakuake/sessions runCommandInTerminal 6 "ssh work"
```

用 dbus-send 替代 qdbus

您可用更常见的 *dbus-send*替换与 Qt 捆绑的 *qdbus*。例如，要显示/隐藏 Yakuake：

```
$ dbus-send --type=method_call --dest=org.kde.yakuake /yakuake/window
org.kde.yakuake.toggleWindowState
```

疑难解答

真彩程序不正确显示

见 Konsole#真彩程序无法正确显示。

参见

- Yakuake 脚本于 coderwall.com

6.6 KMSCON

难度 进阶 | **篇幅** 6.3 KB | **所属** 终端模拟器

本节将讲到

- 特性
- 安装
- 中日韩文字支持
- 疑难解答
- 参考资料

Kmscon 是一个简单的、基于 Linux 内核级显示模式设置的终端模拟器，尝试用用户空间终端代替内核的虚拟终端（VT）。

特性

KMSCON 能作为 Linux 内核内置终端的一个完整替代，它具有以下功能：

- 完整的 vt220 to vt510 实现。
- 完整的国际化支持：
- Kmscon 支持打印全部 Unicode 字符，包括中日韩文字。
- Kmscon 通过 libxkbcommon 对国际键盘布局提供支持，所以人们可以使用 X 支持的所有键盘布局。
- 硬件加速渲染。
- 支持 multi-seat。

注意：要允许 root 用户通过 kmscon 登录，你需要移除或注释 `/etc/pam.d/login` 中相应行来停用 `pam_securetty` 模块。

安装

虽然名字里带有 KMS，kmscon 并非硬性依赖 KMS。Kmscon 支持的视频后端如下：drm3d（Linux DRM 硬解后端）、fbdev（Linux fbdev 视频后端）。请确保你的系统支持其中之一。

安装 kmscon包，或者安装最新开发版 kmscon-gitAUR。

在 tty1 上一般有特殊的 systemd 配置。出于保守的策略，你可以继续在 tty1 上使用传统的 getty 而只在其他虚拟终端上运行 kmscon ，或者你可以用 kmscon 替换 getty。

要在 tty1 上启用 kmscon：

```
# systemctl disable getty@tty1.service
# systemctl enable kmsconvt@tty1.service
```

重新启动即可生效。

要在所有的虚拟终端上启用 kmscon：

```
# ln -s /usr/lib/systemd/system/kmsconvt\@.service /etc/systemd/system/autovt\@.service
```

这使 systemd包 在每个虚拟终端上启动 kmscon 而不是 agetty，同时使 *systemd-logind* 使用 `kmsconvt@.service` 而不是 `getty@.service` 打开新的虚拟终端。不过使用 `getty@.service` 的 systemd 单元不受影响。

如果 *kmscon* 无法启动，它会尝试启动 `getty@.service` ，此外没有虚拟终端可用时这个单元不会启动。

警告：如果你在所有的终端替换了 getty ，请在重新启动之前确认 *kmscon* 可用！ 不然你只能用 Live CD 一类的介质恢复系统了。

中日韩文字支持

Kmscon 通过默认的字体引擎 pango包 支持渲染中日韩文字。但是， 必须为 fontconfig包 设置全局配置，来将等宽字体映射到合适的中日韩字体上。我们为中文用户提供如下配置模板。此模板可以满足中文字体渲染要求：

```
/etc/fonts/conf.d/99-kmscon.conf
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "fonts.dtd">
<fontconfig>
<match>
        <test name="family"><string>monospace</string></test>
        <edit name="family" mode="prepend" binding="strong">
                <string>DejaVu Sans Mono</string>
                <string>WenQuanYi Micro Hei Mono</string>
        </edit>
</match>
</fontconfig>
```

还可以将以下行加入 `/etc/kmscon/kmscon.conf` 来配置 kmscon 全局使用这些字体：

```
/etc/kmscon/kmscon.conf
font-name=DejaVu Sans Mono, WenQuanYi Micro Hei Mono
font-size=14
```

你需要安装 ttf-dejavu包 和 wqy-microhei包，它们都可以在官方仓库中找到。

疑难解答

在切换 Xorg 和 kmscon 时遇到问题

如果你在切换 Xorg 和 kmscon 时遇到问题，试着把 `hwaccel` 添加到 `/etc/kmscon/kmscon.conf` 中。 这个文件和目录不在包内，因此你需要手动创建它们，或者你可以编辑 systemd 单元文件。

```
ExecStart=/usr/bin/kmscon "--vt=%I" --seats=seat0 --no-switchvt --font-name Terminus --font-size 12 --hwaccel --drm
```

无法控制声音

在版本 7 中，如果你不能控制声音，把你的用户添加到 `audio` 用户组中。不过要注意这么做的缺点。

Vim 无法清空终端输出

Vim 可能会在启动时不清除终端输出，这时仍然可以编辑文件，但只有更改文本后才能看到文本。解决方法是，尝试设置环境变量 `TERM = vt220`。 另外，其它类似于 vim 的编辑器（例如 vi包 或 Neovim）可能会正常工作。

自动登录

你可以在 `/etc/kmscon/kmscon.conf`中添加以下内容来自动登录用户（不需要输入密码）：

```
# 你可以按需修改以下命令

# 例：以 root 用户直接登录（非 systemd 会话）
login=/bin/bash --login

# 例：以指定用户直接登录（正常 systemd 会话）
login=/usr/bin/machinectl -q shell 你的用户名@
```

HiDPI 支持

使用 `Ctrl++`、`Ctrl+Shift+=`、`Ctrl+-` 快捷键可以实时更改字体大小。也可以在 `/etc/kmscon/kmscon.conf` 里设置 font-dpi 和 font-size 选项，例如 `font-dpi=288`。288 = 96 * 3，也就是三倍大小。96 是默认值。

参考资料

1. ↑ 来自该项目代码库的介绍

6.7 Alacritty

难度 进阶 | **篇幅** 8.5 KB | **所属** 终端模拟器

本节将讲到

- 安装
- 配置
- 提示和技巧
- 故障排除

Alacritty，一个用 Rust 语言编写的，简洁、具有 GPU 加速功能的终端模拟器。它支持回滚、24 位颜色（zhwp:色彩深度）、复制粘贴、点击 URL 和自定义按键绑定。

安装

安装 alacritty包 包或者开发版本 alacritty-gitAUR。

配置

Alacritty 按照以下顺序搜索配置文件：

- `$XDG_CONFIG_HOME/alacritty/alacritty.toml`
- `$XDG_CONFIG_HOME/alacritty.toml`
- `$HOME/.config/alacritty/alacritty.toml`
- `$HOME/.alacritty.toml`

0.13.0 以前的版本使用 YAML 配置文件。旧的 YAML 配置文件可通过运行 `alacritty migrate` 转换为 TOML。然而，自动迁移会丢弃所有注释。

Alacritty 默认不提供配置文件。配置选项可从项目首页中找到。如果启用了 `live_config_reload` 选项（默认启用），大部分设置将会在配置文件保存后立即生效。

颜色

有关可用配色方案的列表，请参阅 alacritty 主题仓库。如果您喜欢的配色方案在列表中，请将提供的代码粘贴到您的配置文件中。

字体

如果不想使用系统默认字体，可以通过更改以下行来指定不同的字体：

```
[font]
size = 12.0

[font.bold]
family = "monospace"
style = "Bold"

[font.bold_italic]
```

```
family = "monospace"
style = "Bold Italic"

[font.italic]
family = "monospace"
style = "Italic"

[font.normal]
family = "monospace"
style = "Regular"
```

将 `monospace` 替换为以下输出中的某个字体名称：

```
$ fc-list : family style
```

请注意，某些字体不提供 `Italic` 而提供 `Oblique` 样式。

提示和技巧

在同一目录中生成新实例

将以下内容添加到配置文件中，之后，通过按下 `Ctrl+Shift+Enter` 使 *Alacritty* 在当前工作目录中生成一个新的实例：

```
[keyboard]
bindings = [
   { key = "Return", mods = "Control|Shift", action = "SpawnNewInstance" }
]
```

Vi 模式和复制粘贴

Vi 模式允许使用键盘在 Alacritty 的视口中移动和回滚。默认使用 `Ctrl+Shift+Space` 切换该模式。要复制内容，可使用鼠标选择并按 `Ctrl+Shift+c`，或进入 Vi 模式，用 `v` 开始选择，然后像在 vim 中那样用 `hjkl` 移动，最后按 `y` 复制选择的内容。要粘贴内容，可按 `Ctrl+Shift+v`。要从 X 剪切板中复制或是往其中粘贴内容，可使用鼠标选择进行复制，然后用鼠标中键进行粘贴。

提示（Hints)

终端提示被用于在终端的可视部分寻找文本或超链接，并通过管道将其传至其他应用程序。默认情况下，Alacritty 通过快捷键 `Ctrl+Shift+O` 启用网址链接（URL）提示并使用 xdg-open 将其打开。参见 Alacritty TOML 配置手册的 `HINTS` 小节以获取细节信息。

要使如 `*filename*.rs:*line*:*character*` 的文件提示（例如 `my_crate/src/server.rs:181:49`）能够被鼠标点击并能使用 Visual Studio Code 打开，可在 Alacritty 的 TOML 配置中添加以下小节：

```
alacritty.toml
[[hints.enabled]]
regex = "[^ ]+\\.rs:\\d+:\\d+"
command = { program = "code", args = [ "--goto" ] }
mouse = { enabled = true }
```

基于正则表达式的多种提示类型可通过添加多个 `[[hints.enabled]]` 小节实现。

随时切换主题

要切换主题，例如通过 ssh 连接到一个服务器时，可以使用如下命令：

```
$ alacritty msg config "$(cat ~/path/to/theme.toml)"
```

故障排除

鼠标在 Vim 中无法正常工作

将 `set ttymouse=sgr` 和 `set mouse=a` 添加至 `.vimrc` 中，或换用 Neovim。也可以参阅这个议题。

边框在 dwm 中变成透明

Alacritty 的边框在 dwm 中变得透明。将如下那行添加到 dwm 源代码目录的 `drw.c` 中并重新编译可修复此问题：

```
if (!XftColorAllocName(...))
    die("error, cannot allocate color '%s'", clrname); /* 找到这行代码 */
dest->pixel |= 0xff << 24;                             /* 添加这行代码 */
```

终端功能在远程 shell 中不可用

当使用 Alacritty 终端连接到远程系统（例如 SSH）时，可能会发生此问题，系统的 terminfo 数据库中可能没有 Alacritty 的条目 (`/usr/share/terminfo/a/alacritty*`)。 因此，所有交互式终端功能都不起作用。如 termite#Terminal issues with SSH（英语：termite#Terminal issues with SSH） 中所述，将 Alacritty 的 terminfo 复制到远程服务器可解决该问题。

在本地主机上，使用 Alacritty：

```
$ infocmp > alacritty.terminfo                # 导出 Alacritty 的 Terminfo
$ scp alacritty.terminfo user@remote-host:~/  # 或者使用其他方法复制到远程主机上
```

在远程主机上，在与先前复制 `alacritty.terminfo` 文件的相同目录中：

```
$ tic -x alacritty.terminfo  # 为当前用户导入 Terminfo
$ rm alacritty.terminfo      # 可选操作：删除 Terminfo
```

上述过程可简化为以下一行：

```
$ infocmp | ssh "$user@$host" 'tic -x /dev/stdin'
```

注意：在此之后，您将需要启动一个新的 SSH 会话以使远程 shell 加载新的 Terminfo。

或者，可以将配置中的 `TERM` 设置为 `xterm-256color` 而不是默认的 `alacritty`：

```
[env]
TERM = "xterm-256color"
```

Wayland GNOME 上没有标题栏

当使用 Wayland GNOME时，标题栏是空的并且有奇怪的图标。有关详细信息，请参阅 https://github.com/alacritty/alacritty/issues/4739 以获取详细信息。

一种解决方案是设置一个空的 `WAYLAND_DISPLAY` 环境变量，使 Alacritty 以 Xwayland 启动，而不是原生 Wayland。

多显示器上字体大小不同

默认情况下，Alacritty 会在每个显示器上尝试根据 `Device pixel ratio` 将字体缩放到合适的大小。在某些有多个显示屏的设置中，这会使字体的物理尺寸相差巨大，如 和 。

要查看每个监视器的现有设备像素比率值，请运行 `alacritty -v`，将子窗口移动到每个监视器，并注意父窗口中报告的 `Device pixel ratio`。

使用环境变量 `WINIT_X11_SCALE_FACTOR` 强制设置一个固定的设备像素比应当足以解决字体大小不同的问题：

```
$ WINIT_X11_SCALE_FACTOR=1.66 alacritty
```

也可以在配置文件中设置 `WINIT_X11_SCALE_FACTOR` 的值：

```
[env]
WINIT_X11_SCALE_FACTOR = "1.66"
```

无法恢复到先前用 pywal 设置的配色方案

可将以下代码添加至 shell 的执行命令（`.bashrc`）中：

```
if command -v wal > /dev/null 2>&1 && [ "$TERM" = "alacritty" ]; then
    wal -Rqe
fi
```

这比简单地添加 `wal -R` 更好，因为：

1. 只需在终端模拟器窗口中执行此操作。
2. `wal -R` 非常慢，并且不需要在每个子 shell 中执行。
3. 使用了 `-q` 选项，不需要查看标准输出（StdOut）。
4. 不需要让桌面的其他组件（例如 gtk、xrdb、polybar、i3）也重新加载颜色。这由 `-e` 标志完成。

6.8 Rxvt-unicode

难度 进阶 | **篇幅** 16.4 KB | **所属** 终端模拟器

本节将讲到

- 安装
- 配置
- 复制和粘贴
- Perl插件
- 故障排除

rxvt-unicode 是一个从 rxvt 分支出的可定制终端。rxvt-unicode 的特性包括：通过 Unicode 实现的多语言支持、透明度支持、多字体显示和 Perl 插件支持。

安装

安装 rxvt-unicode包 包，或 rxvt-unicode-truecolorAUR 以获得 24 位真彩色支持。

配置

浏览 urxvt(1) 和 urxvt(7) 以获得可用的设置。

Xresources

Rxvt-unicode 用命令行参数或者Xresources来控制。命令行参数会覆盖Xresource设置，详情参见X resources。

`urxvt --help` 在标准错误输出 输出所有的 *rxvt* 设置. 帮助页(man page)有每一个设置的完整描述。

回滚位置

当终端内容过多而无法显示时，旧内容会存放到缓冲区。 默认情况下，当shell有新的输出时，会自动跳到最底部。如果你想看以往的输出内容(而不跳到最底部)，设置如下 `~/.Xresources`:

```
! 输出时不滚动
URxvt*scrollTtyOutput: false

! scroll in relation to buffer (with mouse scroll or Shift+Page Up)
URxvt*scrollWithBuffer: true

! 按键按下时回到最底部
URxvt*scrollTtyKeypress: true
```

回滚辅助屏幕

当你滚动一个有*辅助屏幕*的页面(例如： 不带`**-X**`参数的`less`，禁用滚动条而让这个页面*自己*处理滚动会更好:这在konsole终端和基于vte的终端都是默认且不可更改的。

在 urxvt 中, 关闭回滚,启用*第二屏幕*:

```
URxvt.secondaryScreen: 1
URxvt.secondaryScroll: 0
```

除了使用鼠标滚轮滚动之外，上述配置正常生效。当您使用鼠标滚轮在“辅助屏幕”中滚动页面，同时在回滚缓冲区中有一些内容。这时，回滚缓冲区会滚动，而不是页面。为了解决这个问题，有必要在rxvt-unicode中引入一个新选项。在AUR中有一个修补过的rxvt-unicode，名为 rxvt-unicode-better-wheel-scrollingAUR。安装后，将以下内容添加到配置文件:

```
URxvt.secondaryWheel: 1
```
注意：避免在使用这个参数时使用urxvt-vtwheelAUR perl插件, 二者冲突。

字体设置方法

```
URxvt.font: 9x15
```
等同于:

```
URxvt.font: -misc-fixed-medium-r-normal--15-140-75-75-c-90-iso8859-1
```
设置粗体

```
URxvt.font: 9x15bold
```
等同于:

```
URxvt.font: -misc-fixed-bold-r-normal--15-140-75-75-c-90-iso8859-1
```
在 `/usr/share/fonts/misc/fonts.alias` 可以找到完整的字体(X core font)简称列表(在`/usr/share/fonts/`的某些子目录下也有其他的fonts.alias文件,但因为他们分别来自其他字体包，所以可能包含你实际上未安装的字体)。值得注意的是，这些别名选择的是ISO-8859-1版本的字体，而不是ISO-10646-1 (Unicode)版本、是75 DPI而不是100 DPI版本，因此您最好避免使用它们，而是使用完整的长名称来选择字体。

注意：上面的方法只适用于位图字体。其他字体可以用下面的方法通过 Xft 设置:

```
URxvt.font: xft:monaco:size=10
```
或者

```
URxvt.font: xft:monaco:bold:size=10
```
注意：如果Xft字体名称中包含连词符(-), 则必须用反斜杠(\\)转义两次。这与使用 urxvt 的 -fn参数和fc-list返回的结果不同，其中的反斜杠只出现于一次。

在提交配置之前，在实时终端中测试字体的一个好方法是在终端中打印转义代码，例如:

```
$ printf '\e]710;%s\007' "xft:Terminus:pixelsize=12"
```
字体间距

默认情况下，字符之间的距离可能会感觉太宽。间距可以减少一个像素，如:

```
~/.Xresources
URxvt.letterSpace: -1
```
这是一些关于*urxvt*如何计算字体宽度的讨论 。 rxvt-unicode-patchedAUR 改变了计算方法, 使字体更紧凑 (通常情况下)。

颜色

rxvt-unicode在编译时就加入了颜色支持。除了默认的前景色和背景色外，rxvt最多可以显示256种颜色(加上高辨识度的粗体/闪烁/下划线和任何组合)。

还可以将前景(foreground)、背景(background)、游标颜色(cursorColor)、cursorColor2、colorBD、colorUL的颜色值指定为数字0-15，作为引用color0-color15的颜色名称的方便快捷方式。详情参见 #Xresources.

注意：默认情况下 `urxvt` 使用与Xterm相同的颜色,除了一个. 添加 `URxvt.color12: rgb:5c/5c/ff` 到 Xresources 来改变它.

打印

默认情况下，当按下`PrintScreen`时，rxvt-unicode将通过lpr打印出一个屏幕转储。使用`Ctrl+PrintScreen`或`Shift-PrintScreen`也会在打印输出中包含终端的回滚。根据个人喜好和需要，可以更改或完全禁用此行为。

```
~/.Xresources
! 当按下PrintScreen时，禁用打印终端内容。
URxvt.print-pipe: "cat > /dev/null"
```

复制和粘贴

Rxvt-unicode默认使用`PRIMARY`作为剪切缓冲区。详情参见 Selecting and pasting text。

可以通过绑定{{ic|ALT-CTRL-c}和`ALT-CTRL-v`到复制和粘贴来访问`CLIPBOARD`。

注意：选中文本会自动复制到 `PRIMARY`。 `selection-to-clipboard` 从rxvt-unicode包 9.20开始的Perl插件也会拷贝到 `CLIPBOARD`。

如果你想复制到`PRIMARY`选区，并确保你的 `CLIPBOARD`选区更新了相同的内容，你可以添加以下内容:

```
URxvt.perl-ext-common:  ...,clipboard,...
```

以及

```
URxvt.clipboard.autocopy: true
URxvt.keysym.M-c: perl:clipboard:copy
URxvt.keysym.M-v: perl:clipboard:paste
```

参见 Clipboard#Managers.

Perl插件

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：侧重于介绍url-select,因为matcher和url-select基本上是等价的而且手册页存在重复（在Talk:Rxvt-unicode讨论）

我们可以通过包含以下行来启用URxvt perl扩展:

```
 URxvt.perl-ext-common: extension_name_1,extension_name_2,...
```

请注意：在多个插件名之间**不应该**有任何的间隔

Clickable URLs

您可以使用matcher扩展使终端中的url可单击。例如，要使用鼠标左键在默认的web浏览器中打开链接，请将以下内容添加到`.Xresources`中:

```
URxvt.perl-ext-common: default,matcher
URxvt.url-launcher: /usr/bin/xdg-open
URxvt.matcher.button: 1
```

rxvt-unicode 9.14 可以使用matcher通过键盘打开和列出最近(目前限制为10个)的url:

```
URxvt.keysym.C-Delete: perl:matcher:last
URxvt.keysym.M-Delete: perl:matcher:list
```

匹配的链接可以指定前景或背景色，例如蓝色:

```
URxvt.matcher.rend.0: Uline Bold fg5
```

或者使用`colorUL`设置#RRGGBB颜色。然而，这会将颜色应用到所有下划线的文本，而不是只有URL链接:

```
URxvt.colorUL: #4682B4
```

Yankable URLs (no mouse)

此外，你可以在web浏览器中选择和打开url，而无需使用鼠标。 从 official repositories 安装urxvt-perls包 包，修改你的 `.Xresources` 。 下面是一个例子：

```
URxvt.perl-ext: default,url-select
URxvt.keysym.M-u: perl:url-select:select_next
URxvt.url-select.launcher: /usr/bin/xdg-open
URxvt.url-select.underline: true
```

注意：这个扩展代替了上面提到的可点击url扩展，因此`matcher`可以从`URxvt`中删除。`perl-ext`

key commands:

Key	描述
Alt+u	进入选择模式。您屏幕上的最后一个URL将被选中。您可以重复`Alt+u`来向上选择下一个的URL。
k	向上选择下一个URL。
j	向下选择下一个的URL
Return	在浏览器中打开选定的URL并退出选择模式
o	在浏览器中打开选定的URL，不退出选择模式
y	复制选定的URL并退出选择模式
Esc	退出选择模式

Simple tabs

要将选项卡(tabs)添加到urxvt，请将以下内容添加到您的`~/.Xresources`:

```
URxvt.perl-ext-common: ...,tabbed,...
```

使用如下命令控制选项卡(tabs):

Key	描述
Shift+Down	新建选项卡
Shift+Left	转到左边的选项卡
Shift+Right	转到右边的选项卡
Ctrl+Left	向左移动当前选项卡
Ctrl+Right	向右移动当前选项卡
Ctrl+d	

Key	描述
	关闭选项卡

你可以通过如下方式改变标签的颜色:

```
URxvt.tabbed.tabbar-fg: 2
URxvt.tabbed.tabbar-bg: 0
URxvt.tabbed.tab-fg: 3
URxvt.tabbed.tab-bg: 0
```

全屏

您可以安装AUR包urxvt-fullscreenAUR，然后绑定一个键来启用和关闭urxvt全屏。

```
~/.Xresources
...
URxvt.perl-ext-common: ...,fullscreen,...
URxvt.keysym.F11: perl:fullscreen:switch
...
```

动态更改字体大小

从AUR安装urxvt-resize-font-gitAUR，将它添加到您的Perl扩展`~/.Xresources`中

```
URxvt.perl-ext-common:  ...,resize-font,...
```

默认快捷键是：

- `Ctrl++` (or `Ctrl+Shift+=`) 增大字体
- `Ctrl+-` 减小字体
- `Ctrl+=` 恢复(reset)字体大小
- `Ctrl+?` 查看当前字体大小

你也可以改变快捷键, 像这样:

```
URxvt.keysym.C-Down:  resize-font:smaller
URxvt.keysym.C-Up:    resize-font:bigger
```

要使Ctrl+Shift绑定生效，需要禁用默认绑定(参见讨论):

```
URxvt.iso14755: false
URxvt.iso14755_52: false
```

禁用Perl插件

如果不使用Perl扩展特性，可以通过完全禁用Perl扩展来提高安全性和速度。

```
URxvt.perl-ext:
URxvt.perl-ext-common:
```

注意：如果您使用多个Perl扩展特性，您可以依次列出它们，用逗号分隔: `urxwt.Perl -ext-common:default,matcher,tabbed`

故障排除

升级到v9.09后，透明度不生效

rxvt-unicode开发人员删除了很多非标准壁纸设置程序(wallpaper setter)的兼容性代码。使用不兼容的壁纸设置程序会破坏对透明度的支持。建议壁纸设置程序:

- feh

- hsetroot
- esetroot

要使透明真正的起效，请确保注释掉 URxvt.tintColor 和URxvt.inheritPixmap。

远程主机

如果您正在登录到远程主机，在rxvt-unicode下运行文本模式程序时可能会遇到问题。可以通过在远程主机上安装rxvt-unicode包或将`/usr/share/terminfo/r/rxvt-unicode`从您的本地机器复制到您的主机`~/.terminfo/r/rxvt-unicode`来解决这个问题;rxvt-unicode-256color 也使用相同的方法。

有些远程系统不会自动更改标题，除非您指定TERM=xterm。要修复此问题，请在远程机器上的.bashrc中添加以下代码:

```
PROMPT_COMMAND='echo -ne "\033]0;${USER}@${HOSTNAME}:${PWD}\007"'
```

使用 rxvt-unicode 作为 gmrun 终端

与其他一些终端不同，urxvt希望`-e`的参数是单独给出的，而不是与引号组合在一起。这将导致gmrun出现问题，因为它规定了相反的行为。这可以通过在`.gmrunrc`中gmrun的“Terminal”变量前加上一个“eval”来解决:

```
Terminal = eval urxvt
TermExec = ${Terminal} -e
```

(gmrun使用`/bin/sh`执行命令，因此这里可以理解*eval*。)*eval*的副作用是将参数*分解*为`-e`，就像`$@`在Bash中所做的那样，使urxvt能够理解命令。

我的数字键盘行为怪异，产生不同的输出? (e.g. 在vim中)

一些Debian GNU/Linux用户似乎有这个问题，尽管到目前为止还没有具体的细节报告。这可能是由于错误的TERM环境变量设置造成的，尽管关于是否会发生这种情况以及如何发生的细节还不清楚，但TERM=rxvt应该会提供一个兼容的键映射。

然而，使用*xmodmap*程序(xorg-xmodmap包)，你可以将你的数字键盘重新映射回来。

1\. 使用`xev`程序检查数字键盘(numpad)生成的keycode。

- 启动`xev` 程序
- 按下数字键盘的按键 并且 在 `xev`的输出中寻找 ... *keycode xxx* ...。 例如, numpad 1 在某些键盘中是 "End" 键,所以你得到的keycode是'**keycode 87**'.

2\. 创建或修改你的xmodmap文件，通常是`~/.Xmodmap`，其中的内容表示你的键码。

带有数字键盘键码的xmodmap文件示例:

```
keycode 63 = KP_Multiply
keycode 79 = Home KP_7
keycode 80 = Up KP_8
keycode 81 = Prior KP_9
keycode 82 = KP_Subtract
keycode 83 = Left KP_4
keycode 84 = KP_5
keycode 85 = Right KP_6
keycode 86 = KP_Add
keycode 87 = End KP_1
keycode 88 = Down KP_2
keycode 89 = Next KP_3
keycode 90 = Insert KP_0
keycode 91 = Delete KP_Decimal
keycode 112 = Prior
keycode 117 = Next
```

3\. 在 X 会话启动时载入你的xmodmap文件。

例如, 在 `~/.xinitrc`文件中添加:

```
...
xmodmap ~/.Xmodmap
...
```

组合键不工作

参见在终端中启用Alt键.

绘制符号时性能低下

有些程序，如alsamixer和xprop，在某些图形驱动程序上表现不佳，因此重绘非常缓慢。选项“skipBuiltinGlyphs”为`~/。Xresources`或命令行选项`-sbg`可以修复此错误。一种可能的解决方案是在`~/.Xresources`中添加以下内容:

```
URxvt*skipBuiltinGlyphs:    true
```

非常多的行会导致速度减慢

`matcher`插件可能是这里的罪魁祸首。它必须在每次行更新时对行匹配regex，如果您有一个很大的`saveLines`值，那么允许非常大的最多行可能会加剧这个问题。

这里有一些简单的解决方法:

- 减少 `saveLines`
- 关闭 `matcher` 插件

如果这两个选项都不合适，你可以设置超过某个截止点之后禁用URL匹配:

1. 复制 `/usr/lib/urxvt/perl/matcher` 到 `~/.urxvt/ext/` (如果目录不存在则创建)
2. 修改 `~/.urxvt/ext/matcher`, 找到 `my ($self, $row) = @_;` 行的 `on_line_update` 部分。它应该在270行。

3. 那行之后, 插入新行： `return () if $row < -100;`. 这将在终端顶部后面超过100行的任何行上禁用URL匹配。

参见

- rxvt-unicode - 官方网站
- Source Code - 可浏览的 CVS
- rxvt-unicode FAQ - 官方 FAQ
- rxvt-unicode Reference - 官方手册页
- urxvtperl - 官方 Perl 扩展参考

第 7 章　共 6 节　108 KB

命令行

命令行是每天真正动手的地方，也最容易停在「会用几个命令」上。本章补这一层：终端怎么用（复制文本、行内编辑与补全）、遇事怎么靠 man 手册自查，以及权限、ACL、umask 与文件属性的区别、dd 这类直接写设备的工具和核心工具的取舍。

阅读上，文件权限与属性、Man 手册必读：前者是后续操作的前提，后者决定你能否自己查证；核心工具通读建立印象即可。Readline 与从终端复制文本按需翻阅，Dd 只在克隆硬盘或备份 MBR 时打开。最易犯的错是以为权限改一次就永久生效，忽略 ACL、umask 与文件属性的共同作用；以及把 dd 当普通复制命令，它对块设备的写入不可撤销，动手前须确认目标设备并先备份。

7.1　从终端复制文本

难度 进阶 | **篇幅** 6.1 KB | **所属** 命令行

本节将讲到

- 一般方法
- 常见终端仿真器比较
- 特殊情况

大多数成熟的终端模拟器允许用户复制或保存其内容。

一般方法

在图形化终端模拟器中，内容通常可以通过鼠标选择，然后通过右键菜单、编辑菜单或快捷键组合（例如 `Ctrl+Shift+c`）进行复制。

不原生支持 CLIPBOARD 选区的终端

Xorg

有些终端仿真器不原生支持 CLIPBOARD 选区，而是将数据复制到 PRIMARY 选取。对于这些终端，可以使用 xclip包：

```
$ xclip -o | xclip -selection clipboard -i
```

上述命令将从 PRIMARY 选区读取数据并写入 CLIPBOARD 选择。

其他剪贴板管理器（如autocutselAUR）提供选择缓冲区之间的自动同步。

Wayland

像 wl-clipboard包 和 clipboardAUR 这样的工具可以将数据复制到Wayland剪贴板：

```
$ command 2>&1 | wl-copy
```

截取命令输出

使用 tee（英语：tee） 截取命令的输出。

```
$ command 2>&1 | tee output-file
```

在执行 `command` 之后，`output-file` 将包含其输出，同时显示输出。

访问 Linux 终端历史

可以通过 `/dev/vcs*N*` 访问名为 `/dev/tty*N*` 的本地终端的历史。因此，如果在 `/dev/tty1` 中工作，以下代码段将把历史存储到文件`output-file`中：

```
# cat /dev/vcs1 > output-file
```

常见终端仿真器比较

**本文或本章节的事实准确性存在争议。**

原因： 此表中的一些“否”项可能不准确。（在 Talk:从终端复制文本#常见终端仿真器备忘单：“否”项的准确性 中讨论）

除非“快捷键组合”列另有说明，否则快捷键组合为`Ctrl+Shift+c`。

仿真器	选择到 PRIMARY	CLIPBOARD			
快捷键组合	上下文菜单	窗口菜单	选择		
Alacritty	是	是	否	否	否
atermAUR	是	否	否	否	否
etermAUR	是	否	否	否	否
foot	是	是	否	否	可选
germinalAUR	是	是	是	否	否
Guake（英语：Guake）	是	是	是	否	否
Konsole	是	是	是	是	可选
lilyterm-gitAUR	是	是 `Ctrl+Delete`	是	否	否
lxterminal包	是	是	是	是	否
mate-terminal包	是	是	是	是	否
mlterm	是	是	否	否	是
pantheon-terminal包	是	是	是	否	否
PuTTY（英语：PuTTY）	是	否	否	否	否
qterminal包	是	是	是	是	否
roxtermAUR	是	是	是	是	否
rxvt-unicode	是	是 `Ctrl+Alt+c`	否	否	可选
sakura包	是	是	是	是	否
st	是	是	否	否	否
Terminator（英语：Terminator）	是	是	是	否	否
terminology包	是	是	是	否	否
Termite（英语：Termite）	是	是	否	否	否
Tilda（英语：Tilda）	是	是	是	否	否

仿真器	选择到 PRIMARY	CLIPBOARD			
xfce4-terminal 包	是	是	是	是	否
xterm（英语：xterm）	是	可选	否	否	是
Yakuake	是	是	是	否	可选

特殊情况

putty

xclip 方法适用于 *putty*: 只需记住，*xclip* 命令应该在本地计算机上（在另一个终端中）执行，而不是在与 *putty* 连接的远程计算机上执行。

urxvt

将文本选择到 CLIPBOARD 选区需要 `selection-to-clipboard` Perl 扩展。有关详细信息，请参见 rxvt-unicode#复制和粘贴。

xterm

访问 *xterm* 中的 CLIPBOARD 选区需要额外的步骤（英语：xterm#PRIMARY or CLIPBOARD）。

mlterm

除了 `Ctrl+Shift+c`，如果不想意外杀死进程，还可以使用 `Ctrl+Insert`，同时，一些版本的 MIterm 会在选择时自动实时复制选取。

7.2 Readline

难度 进阶 | **篇幅** 8.4 KB | **所属** 命令行

本节将讲到

- 安装
- 编辑模式
- Fast word movement
- 历史
- Faster completion

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 翻译未完成（在 Talk:Readline# 中讨论）

Readline 是来自 GNU Project 的库，Bash 和其他命令行界面的程序使用它在命令行编辑和交互。详情请参阅 readline(3)。

安装

readline包 包很可能已作为 Bash 的依赖安装。

编辑模式

Readline 默认使用 Emacs 风格的快捷键与命令行交互。不过，vi 风格的编辑界面也受支持，但需要将以下内容添加到 `~/.inputrc`：

```
~/.inputrc
set editing-mode vi
```

或者，要只为 Bash 设置，你可将以下内容添加到 `~/.bashrc`：

```
~/.bashrc
set -o vi
```

提示中的模式指示符

Vi 风格的编辑有两种模式：命令模式和插入模式。你可通过添加以下命令显示当前的模式：

```
~/.inputrc
set show-mode-in-prompt on
```

这将在你的提示中显示模式（默认是 `(cmd)/(ins)`），显示的内容可通过 `vi-ins-mode-string` 和 `vi-cmd-mode-string` 变量自定义。

为每个模式指定不同的光标样式

你可以使用"\\1 .. \\2" 转义符为每个模式指定不同的光标样式：

```
~/.inputrc
set vi-ins-mode-string \1\e[6 q\2
set vi-cmd-mode-string \1\e[2 q\2
```

这将在命令模式中显示方块形状的光标，在插入模式中显示竖线形状的光标。注意，你必须先启用模式指示符（见#提示中的模式指示符）。

Virtual Console 使用不同的转义代码，因此你应先检查使用的是哪种终端：

```
~/.inputrc
$if term=linux
        set vi-ins-mode-string \1\e[?0c\2
        set vi-cmd-mode-string \1\e[?8c\2
$else
        set vi-ins-mode-string \1\e[6 q\2
        set vi-cmd-mode-string \1\e[2 q\2
$endif
```

详情请参阅 software cursor for VGA。

Fast word movement

Xterm 默认支持使用 `Ctrl+Left` 和 `Ctrl+Right` 在单词间移动。要在其他终端模拟器中实现这一效果，先找到正确的终端代码，然后在 `~/.inputrc` 中绑定到 `backward-word` 和 `forward-word`。

例如，对于 urxvt：

```
~/.inputrc
"\e[1;5D": backward-word
"\e[1;5C": forward-word
```

历史

通常，无论你输入了什么，按向上箭头都会出现上一次输入的命令。然而，用户可能会觉得只显示匹配刚刚输入的内容的命令更实用。

例如，假设输入了以下命令：

- `ls /usr/src/linux-2.6.15-ARCH/kernel/power/Kconfig`
- `who`
- `mount`
- `man mount`

此时，当输入 `ls` 并按向上箭头，输入将会被替换为 `man mount`。如果你使用了历史搜索功能，那么只有过去以 `ls`（当前的输入）开头的命令会被显示，在这里就是 `ls /usr/src/linux-2.6.15-ARCH/kernel/power/Kconfig`。

要启用历史搜索功能，只需把以下内容加入到 `/etc/inputrc` 或者 `~/.inputrc`：

```
"\e[A": history-search-backward
"\e[B": history-search-forward
```

如果你使用 vi 模式，添加以下内容到 `~/.inputrc`（来自这个帖子）：

```
set editing-mode vi
$if mode=vi
set keymap vi-command
# these are for vi-command mode
"\e[A": history-search-backward
"\e[B": history-search-forward
j: history-search-forward
k: history-search-backward
set keymap vi-insert
# these are for vi-insert mode
"\e[A": history-search-backward
"\e[B": history-search-forward
$endif
```

如果你选择将这些内容添加到 `~/.inputrc`，那建议你同时在文件的开始添加以下内容，以避免像这样的怪事：

```
$include /etc/inputrc
```

Alternatively, one can use reverse-search-history (incremental search) by pressing `Ctrl+R`, which does not search based on previous input but instead jumps backwards in the history buffer as commands are typed in a search term. Pressing `Ctrl+R` again during this mode will display the previous line in the buffer that matches the current search term, while pressing `Ctrl+G` (abort) will cancel the search and restore the current input line. So in order to search through all previous

`mount` commands, press `Ctrl+R`, type 'mount' and keep pressing `Ctrl+R` until the desired line is found.

The forward equivalent to this mode is called forward-search-history and is bound to `Ctrl+S` by default. Beware that most terminals override `Ctrl+S` to suspend execution until `Ctrl+Q` is entered. (This is called XON/XOFF flow control). For activating forward-search-history, either disable flow control by issuing:

```
$ stty -ixon
```

或者在 `inputrc` 中使用不同的键。例如，要使用默认未绑定的 `Alt+S`：

```
"\es": forward-search-history
```

Faster completion

When performing tab completion, a single tab attempts to partially complete the current word. If no partial completions are possible, a double tab shows all possible completions.

The double tab can be changed to a single tab by setting:

```
~/.inputrc
set show-all-if-unmodified on
```

Or you can set it such that a single tab will perform both steps: partially complete the word *and* show all possible completions if it is still ambiguous:

```
~/.inputrc
set show-all-if-ambiguous on
```

Colorized completion

You can enable coloring of completion of filenames with the `colored-stats` option. You can also color the identical prefix of completion-lists with `colored-completion-prefix`. For example:

```
~/.inputrc
# Color files by types
# Note that this may cause completion text blink in some terminals (e.g. xterm).
set colored-stats On
# Append char to indicate type
set visible-stats On
# Mark symlinked directories
set mark-symlinked-directories On
# Color the common prefix
set colored-completion-prefix On
# Color the common prefix in menu-complete
set menu-complete-display-prefix On
```

宏

Readline also supports binding keys to keyboard macros. For simple example, run this command in Bash:

```
bind '"\ew": "\C-e # macro"'
```

or add the part within single quotes to inputrc:

```
"\ew": "\C-e # macro"
```

Now type a line and press `Alt+W`. Readline will act as though `Ctrl+E` (end-of-line) had been pressed, appended with ' `# macro`'.

Use any of the existing keybindings within a readline macro, which can be quite useful to automate frequently used idioms. For example, this one makes `Ctrl+Alt+L` append "| less" to the line and run it (`Ctrl+M` is equivalent to `Enter`):

```
"\e\C-l": "\C-e | less\C-m"
```

The next one prefixes the line with 'yes |' when pressing `Ctrl+Alt+Y`, confirming any yes/no question the command might ask:

```
"\e\C-y": "\C-ayes | \C-m"
```

This example wraps the line in `su -c ''`, if `Alt+S` is pressed:

```
"\es": "\C-a su -c '\C-e'\C-m"
```

This example prefixes the line with `sudo` , if `Alt+S` is pressed. It is safer because it will not input the `Enter` key.

```
"\es": "\C-asudo \C-e"
```

As a last example, quickly send a command in the background with `Ctrl+Alt+B`, discarding all of its output:

```
"\e\C-b": "\C-e > /dev/null 2>&1 &\C-m"
```

禁用 control 回显

Readline 使得终端在按下 `Ctrl+C` 后回显 `^C`。要禁用，只需将以下内容添加到 `~/.inputrc`：

```
set echo-control-characters off
```

参阅

- vi readline 编辑速查表
- emacs readline 编辑速查表

7.3 Man 手册

难度 进阶 | **篇幅** 14.9 KB | **所属** 命令行

本节将讲到

- 安装
- 阅读手册页
- 搜索手册页
- 页面宽度
- 阅读本地man页面

man 手册页，指 "manual pages"，是类 UNIX 系统的标准手册工具，包括Arch Linux。这个命令常常被显示为 man(1)。

man手册页被设计成独立的文档，不能引用其它手册页面，于是这限制了它们在讨论相似话题时指向其它的手册页面。这与支持超链接的 Info 文档形成鲜明对比，GNU正在将 man 手册替换成 info 文档。

安装

man-db包 提供了 *man* 命令，less 是 *man* 的默认分页器。mandoc包 同样可供使用。

man-pages包 既提供Linux和POSIX.1的man页面。

中文用户可以选择安装中文版本，其余语言版本见英文wiki:

- man-pages-zh_cn包 简体中文版本
- man-pages-zh_tw包 繁体中文版本

你也可以搜索所有可用的本地man页面在官方仓库和AUR

阅读手册页

通过以下命令阅读man手册页：

```
$ man 手册名
```

man手册页分为很多段落。每个段落有一段介绍，像intro(1), intro(2)等等。完整的列表可以参考:man-pages(7) § Sections of the manual pages .

man手册页通过名称和所属分类标识。有些不同分类的man手册页名字可能相同，比如 man(1) 和 man(7)，这时需要额外指明分类以访问需要的手册。例如：

```
$ man 5 passwd
```

会显示有关文件`/etc/passwd`，而非命令 `passwd`，的内容。

相同地，我们也可以使用man页面后用一'.'分开，并追加一段落号:

```
$ man passwd.5
```

搜索手册页

如果用户压根儿不知道要查阅的手册的名称，该怎么办呢？没事，通过 `-k` 或者 `--apropos` 参数就可以按给定关键词搜索相关手册。例如，要查阅有关密码的手册（“password”）：

关键词搜索特性是从一个专用的缓存生成的。默认情况下你没有这个缓存，所以无论你搜什么，都会提示你*nothing appropriate*。你可以通过下面的命令来生成这个缓存：

```
# mandb
```

每当你安装新的manpage之后都需要运行这个命令，缓存才会更新。

现在你可以开始搜索了。例如，要查阅有关密码的手册（“password”）,可以使用下面的命令:

```
$ man -k password
$ man --apropos password
$ apropos password
```

关键字可以使用正则表达式。

如果你想全文搜索的话，你可以用`-K`选项：

```
$ man -K password
```

通过`whatis`命令，可以只显示需要的man手册页的简要信息。如果只是想获取对命令 ls 的简要说明，使用以下命令：

```
$ whatis ls
ls (1p)              - list directory contents
ls (1)               - list directory contents
```

页面宽度

man页面宽度由环境变量`MANWIDTH` 来控制。

如果终端中的列数很小（比如窗口宽度很小），换行可能会出现错误。这会很影响你阅读。你可以通过设置在`man`调用时设置`MANWIDTH`来修复。如Bash：

```
~/.bashrc
man() {
    local width=$(tput cols)
    [ $width -gt $MANWIDTH ] && width=$MANWIDTH
    env MANWIDTH=$width \
    man "$@"
}
```

阅读本地man页面

你可以通过以下软件来查看man面：

- **Emacs** — The extensible and self-documenting editor can also be used to read man pages with the built-in M-x `man` command.

: https://www.gnu.org/software/emacs/ || emacs包

- **GNOME Help** — Help viewer for GNOME. Part of gnome包组. It can show man pages via `yelp man:<name>` or the undocumented `Ctrl+L` keybinding from an existing window.

: https://apps.gnome.org/Yelp/ || yelp包

- **KHelpCenter** — Application to show KDE Applications' documentation. Part of kde-system包组. Man pages are in *UNIX manual pages* or by running `khelpcenter man:<name>`.

: https://apps.kde.org/khelpcenter/ || khelpcenter包

- **Konqueror** — KDE file manager and web browser. It can show man pages via `man:<name>`.

: https://konqueror.org/ || konqueror包

- **neovim** — The editor can be used to read man pages using the built-in `:Man *name*` command, or configured as a man pager with `export MANPAGER='nvim +Man!'`. Supports highlighting and navigation between command-line flags, keywords, and other man pages. Automatically generates an outline using section headers, command-line flags, and keywords as entries (available via `gO`).

: https://neovim.io/ || neovim包

- **xman** — Provides a categorized look at man pages.

: https://xorg.freedesktop.org/ || xorg-xman包

使用如 lynx包 and firefox包 等浏览器来查看man页面可以使用户获得如阅读`info`页面的超链接优势，你可以使用以下替代方案：

转换为 HTML

mandoc

安装 mandoc包来转换页面，如 `free(1)`：

```
$ mandoc -Thtml -Ostyle=style.css /usr/share/man/man1/free.1.gz > free.html
```

然后在你的浏览器中打开`free.html`

man2html

首先，安装软件包man2html包。

然后使用它转换man手册页：

```
$ man free | man2html -compress -cgiurl man$section/$title.$section$subsection.html > ~/man/free.html
```

此外，`man2html`还可以把man页转换为便于打印的文本文件：

```
$ man free | man2html -bare > ~/free.txt
```

man -H

man-db包提供的man也具有浏览器阅读功能：

```
$ man -H free
```

由`BROWSER`环境变量决定使用的浏览器。也可以使用 `man -Hlynx`（H后无空格）这样的形式手动设置浏览器。

roffit

安装 roffitAUR。

转化一个man页面：

```
$ gunzip -c /usr/share/man/man1/free.1.gz | roffit > free.html
```

转换为 PDF

man pages 是可以打印的，遵循 troff(1) 格式，本来就是一种打印设置语言。因此，你可以很容易地通过**groff**将man页面转换为输出设备支持的任意格式，这也被man-db包 所使用。通过groff(1) (如果你使用mandoc包，请使用mandoc(1)）的`-T`选项来列出所有输出设备。

如下命令将会为你导出PDF文件：

$ man -Tpdf "manpage" > "filename"

注意，这里只能使用Times字体的硬编码大小，一些man页面是特地为终端查看而设计的，并且不会在PDF或PS格式中正确被渲染。

Qman

这是一个提供查看man页面的界面的替代软件，它支持像超链接和历史记录等现代的功能，安装qman-gitAUR。然后你可以使用`qman`来代替`man`了：

```
$ qman ls    # 显示ls的man页面
$ qman -k ls # 'ls'上执行apropos命令
```

查看这个项目的 GitHub页面以了解更多

可选配置

Qman 的配置文件在 `~/.config/qman.conf` (用户的) or `/etc/xdg/qman.conf` (全局的).

如果你正在使用一个支持256色和Unicode的现代终端，你可能想要使用一个自定义配置来为这个软件提供一个更现代化的界面

默认的*Qman*是通过`xdg-open`来打开超链接和电子邮件链接。你可以通过在你的配置文件中添加如下行来更改：

```
[misc]
browser_path=/path/to/your/browser
mailer_path=/path/to/your/email/client
```

使用在线手册页

警告：一些发行版与Arch Linux不同的man页面，这可能是补丁后或过时版本的页面

许多网站提供在线man手册页，详细列表参见：

- Arch Linux manual pages—contains man pages from Arch Linux packages. Used for man page links from the wiki. You can also use the `!archman` bang with search engines like DuckDuckGo or Brave to search through the Arch manual pages directly.
- man7.org—The Linux man-pages project. Upstream of the man-pages包 package. The online pages currently show an outdated version of man-pages (5.13, released in 2021).
- manned.org—collection from various Linux distributions, BSD, etc., with multiple package versions
- linux.die.net
- man.cx—Man pages extracted from Debian testing.
- Debian man pages
- Ubuntu man pages
- DragonFlyBSD man pages
- FreeBSD man pages
- NetBSD man pages
- OpenBSD man pages
- Plan 9 Manual — Volume 1
- Inferno Manual — Volume 1
- The UNIX and Linux forums man page repository

注意， man-pages包提供的是POSIX.1-2017 (见)的man页面，一个官方的在线参考也存在于：

值得注意的man页面

这列举了一些可能帮助你更深度理解很多事情的页面，但这并不是很详尽。它们中的一些可能给你一个很好的参考（如ASCII表）

- ascii(7)
- boot(7)
- charsets(7)
- chmod(1)
- credentials(7)
- fstab(5)
- file-hierarchy(7)
- systemd(1)
- locale(1p), locale(5), locale(7)
- printf(3)
- proc(5)
- regex(7)
- signal(7)
- term(5), term(7)
- termcap(5)

- terminfo(5)
- utf-8(7)

你也可以看：category 7 (miscellaneous) pages:

```
$ man -s 7 -k ".*"
```

Arch Linux 的特定页面：

- alpm-hooks(5)
- libalpm(3)
- makepkg(8)
- makepkg.conf(5)
- makepkg-template(1)
- mkinitcpio(8)
- pacman(8)
- pacman.conf(5)
- pacman-conf(8)
- pacman-key(8)

7.4 文件权限与属性

难度 高级 | **篇幅** 22.0 KB | **所属** 命令行

本节将讲到

- 查看权限
- 修改权限
- 修改所有者
- 访问控制列表（ACL）
- Umask

文件系统用权限和属性来规范系统进程与文件和目录之间的交互。

警告：权限和属性设置只能防范来自已经启动了的当前系统的攻击。为了防止数据被能物理访问该机器的人的接触，必须同时使用磁盘加密.

查看权限

使用ls命令的-l选项查看为目录内容设置的权限（或文件模式），例如：

```
$ ls -l /path/to/directory
total 128
drwxr-xr-x 2 archie users  4096 Jul  5 21:03 Desktop
drwxr-xr-x 6 archie users  4096 Jul  5 17:37 Documents
drwxr-xr-x 2 archie users  4096 Jul  5 13:45 Downloads
-rw-rw-r-- 1 archie users  5120 Jun 27 08:28 customers.ods
-rw-r--r-- 1 archie users  3339 Jun 27 08:28 todo
-rwxr-xr-x 1 archie users  2048 Jul  6 12:56 myscript.sh
```

第一栏是我们必须关注的内容。 以 `drwxrwxrwx +`为例，每个字符的含义在下表中说明：

<table class="wikitable"><tbody><tr style="text-align:center;"><td><code>d</code></td><td><code>rwx</code></td><td><code>rwx</code></td><td><code>rwx</code></td><td><code>+</code></td></tr><tr><td>文件类型，技术上不属于其权限。 有关可能值的说明，请参阅 <code>info ls -n "What information is listed"</code> 命令的输出。</td><td>所有者对文件的权限，如下所述。</td><td>该组对该文件的权限，如下所述。</td><td>所有其他用户对该文件的权限，如下所述。</td><td>一个指定备用访问方法是否适用于该文件的字符。 当此字符是空格时，没有备用访问方法。 <code>.</code>字符表示具有安全上下文的文件，但没有其他备用访问方法。 具有任何其他备用访问方法的文件用<code>+</code>字符标记，例如在使用访问控制列表 (ACL) 的情况下.</td></tr></tbody></table>

权限三元组（也就是上面例子中的`rwx`）可以由以下字符（权限位）构成：

	字符	文件效果	目录效果
读权限（第一个字符）	-	不能读取该文件	不显示目录里的内容
r	可以读取该文件	能显示目录里的内容	
写权限（第二个字符）	-	不能修改该文件	不能修改目录中的内容
w	可以修改该文件	能修改目录中的内容（创建、重命名或删除文件或子目录）；需要同时设置执行权限，否则本权限无效。	
执行权限（第三个字符）	-	不能执行该文件	无法使用 cd 命令访问该目录
x	可以执行该文件	能使用 cd 命令访问该目录。这是实践中唯一一个算是可以从父目录*继承*而来的权限。实际上，如果某文件或目录路径中的任何一个目录没有设置 *x* 权限， 就算最终访问的文件或目录设置了对应的权限也是打不开的。请参考 path_resolution(7) 来了解更多。	
s	当出现在用户（所有者）权限三元组时，代表 setuid 位； 当出现在组权限三元组时，代表 **setgid** 位；不会出现在对其他用户权限的三元组中；同时意味着设置了 x 权限。		
S	与 s 相同，但是未设置 x 权限；普通文件中很少		

	字符	文件效果	目录效果
	见，并且对于目录是无效的。		
t	粘滞位，只存在于对其他用户权限的三元组中；同时意味着设置了 x 权限。		
T	与 t 相同，但是未设置 x 权限 ；普通文件中很少见。		

请参考 `info Coreutils -n "Mode Structure"` 和 chmod(1) 以获得更详细的说明。

提示：您可以用 `namei -l *path*` 命令查看文件或者目录沿路的权限

案例

让我们用一些案例来解释说明：

```
drwx------ 6 archie archie  4096 Jul  5 17:37 Documents
```

Archie 具有对 `Document` 目录的完全访问权限。它可以列出、创建和重命名或删除 Documents 中的任何文件，无论其中文件的权限是什么。而它访问文件的能力取决于具体文件的权限。

```
dr-x------ 6 archie archie  4096 Jul  5 17:37 Documents
```

Archie 拥有完全访问权限，但他们不能创建、重命名或删除任何文件。他们可以列出文件，并且（如果文件的权限允许）可以访问文档中的现有文件。

```
d-wx------ 6 archie archie  4096 Jul  5 17:37 Documents
```

Archie 不能在 `files` 目录中执行 `ls` ，但是如果它知道现有文件的名称，就可以列出、重命名、删除或(如果文件的权限允许)访问该文件。此外，它们还能够创建新文件。

```
d--x------ 6 archie archie  4096 Jul  5 17:37 Documents
```

Archie 只能够在文件权限允许的情况下访问已知的文件。它不能列出目录中的文件，也无法创建、重命名或者删除任何文件。

要记得，我们在此介绍的是目录权限，这与单个文件权限无关。创建一个新文件时，修改的是目录。这就是为什么您需要有目录的写权限。

接下来我们看一些文件权限方面的例子：

```
-rw-r--r-- 1 archie web  5120 Jun 27 08:28 foobar
```

在这里，我们可以看到第一个字母不是 `d` 而是 `-`。所以我们知道它是一个文件，而不是一个目录。接下来，所有者的权限是`rw-`。所有者并没有拥有全部的三个权限可能很奇怪，但这个文件作为一个文本文件而不是一个可执行文件时，并不需要`x` 权限（但如果它包含 python 代码之类的内容，那么它很可能是可执行文件），我们可以用

Gedit、EMACS 等文本编辑器或者 R 之类的软件来读取它。该文件的组权限为 `r--`，因此该组的成员可以读取文件，但不能以任何方式对其进行写入/编辑，实际上就是把它设置为只读。最后是对其他用户的权限，被设置为与组权限一致，也是只读。

修改权限

chmod 是 Linux 和其他 Unix-like 操作系统中用来修改文件或者目录权限的命令。

以字符形式修改权限

想要修改文件的权限（或者叫*访问模式*），请在终端里使用 *chmod* 命令。以下是该命令的基本结构：

```
chmod who=permissions filename
```

其中的 `*who*` 是一个或者多个字母，代表要指定权限的对象，可以为：

- `u`: 拥有该文件的用户
- `g`: 该文件所属的用户组 .
- `o`: 所有其他用户
- `a`: 以上所有，等效于 `ugo`.

权限和上面 #查看权限所述的相同 (`r`, `w` 和 `x`).

现在看一下使用这个命令的一些示例。假设您对“文档”目录进行了严格的保护，并希望拒绝除您之外的所有人在其中读取、写入和执行(或在本例中是拒绝搜索或查看)的权限。

修改前：`drwxr-xr-x 6 archie web 4096 Jul 5 17:37 Documents`

```
$ chmod g= Documents
$ chmod o= Documents
```

修改后：`drwx------ 6 archie web 4096 Jul 6 17:32 Documents`

在这里，因为您想要禁止所有权限，所以不要在 = 后面输入任何内容。如此修改后，除了文件所有者的权限为 `rwx` 之外，其他权限都是 `-`。

您可以使用如下命令来恢复原来的状态：

再次修改前：`drwx------ 6 archie web 4096 Jul 6 17:32 Documents`

```
$ chmod g=rx Documents
$ chmod o=rx Documents
```

再次修改后：`drwxr-xr-x 6 archie web 4096 Jul 6 17:32 Documents`

在下一个例子中，您想给组及其他用户读取和执行权限，所以您可以在 = 后面输入相应的权限 r 和x，记得不要带空格。

您可以在 *who* 的地方输入多个字母，来同时给多个对象设置权限，如：

```
$ chmod go=rx Documents
```

注意：chmod 命令中的 *who* 字母或者权限位（）字符的顺序是无关紧要的。chmod go=rx file 和 chmod og=xr file 这两个命令的含义完全相同

接下来我们考虑第二个例子，假设您想让自己和 web 用户组中的用户（可能是您的同事）有 foobar 这个文件的读写权限， 但其他人只能读取它。那么操作如下：

修改前： -rw-r--r-- 1 archie web 5120 Jun 27 08:28 foobar

```
$ chmod g=rw foobar
```

修改后：-rw-rw-r-- 1 archie web 5120 Jun 27 08:28 foobar

这与第一个示例完全相同，但使用的是文件，而不是目录，并且您授予的是写入权限（这只是为了给出授予每个权限的示例）。

字符形式的捷径

chmod 命令允许使用 + 或 - 而不是=从现有集合中添加和减去权限。这与上面的命令不同，后者实质上是重设权限(例如，要将权限从r--更改为rw-， 您仍然需要在 *chmod* 命令的=之后包含 r 和 w。如果遗漏了r， 那么在使用=重设权限时，它将取消r的权限。使用+和-可以通过添加或删除当前的权限集来避免这种情况)。

让我们用 + 和 - 的方式复现一下前面给组添加写权限的例子：

修改前： -rw-r--r-- 1 archie web 5120 Jun 27 08:28 foobar

```
$ chmod g+w foobar
```

修改后：-rw-rw-r-- 1 archie web 5120 Jun 27 08:28 foobar

另一个例子，取消所有用户和组的写入权限。

修改前： -rw-rw-r-- 1 archie web 5120 Jun 27 08:28 foobar

```
$ chmod a-w foobar
```

修改后： -r--r--r-- 1 archie web 5120 Jun 27 08:28 foobar

另一个不同的捷径是特殊的 X 模式：实际上不存在这个模式，但它通常与 -R 选项（递归）一起使用，以设置只针对目录的执行权限，而不修改常规文件的权限，例如:

```
$ chmod -R a+rX ./data/
```

复制权限

您可以让 *chmod* 把某对象（比如所有者）的权限复制给其他对象（比如组或者所有人）。想要做到这一点，在 = 后面不要跟权限（r、w和x），而是输入另一个 *who* 字母。比如：

修改前 `-rw-r--r-- 1 archie web 5120 Jun 27 08:28 foobar`

```
$ chmod g=u foobar
```

修改后 `-rw-rw-r-- 1 archie web 5120 Jun 27 08:28 foobar`

这个命令的意思就是「把对组(g=)的权限设置为与所有者(=u)的权限相同」。请注意，您不能复制一组权限，也不能授予新的权限，以下是错误示范，*chmod* 会报错：

```
$ chmod g=wu foobar
```

数字形式

chmod 也可以用数字来设置权限。

使用数字是另一种方法，它允许您同时编辑所有者、组和其他人的权限，以及setuid、setgid和粘滞位。命令的基本结构如下：

```
$ chmod xxx filename
```

xxx 部分是一个三位数，其中的每一位可以是0到7之间的任何数字。第一位数字指所有者的权限，第二位数字指组的权限，而第三位数字指所有其他人的权限。

在这个数字表示法中，权限 r、w和x所对应的数值是:

```
r=4
w=2
x=1
```

要得到这个三位数，您要考虑为所有者、组和所有其他人设置哪些权限，然后将其对应的数值累加起来。例如，如果希望授予目录的所有者读写和执行权限，并希望 group 和其他所有人只有读写权限，那么通过计算可以得到如下数值：

- 所有者: rwx\=4+2+1=7
- 组: r-x\=4+0+1=5
- 其他人: r-x\=4+0+1=5

```
$ chmod 755 filename
```

这就相当于使用如下命令：

```
$ chmod u=rwx filename
$ chmod go=rx filename
```

若要以数字形式查看文件或目录的现有权限，请使用stat(1)命令：

```
$ stat -c %a filename
```

%a 指定以数字形式输出文件权限。

大多数目录被设置为 755 以允许所有者读取、写入和执行，但拒绝被其他所有人写入；文件通常是644 以允许所有者读取和写入，但允许其他所有人读取；请参考最后一个关于不可执行文件缺少x权限的说明，此处同理。

为了用实例来说明这一点，我们用数字方式复现一下前面举过的例子：

修改前：-rw-r--r-- 1 archie web 5120 Jun 27 08:28 foobar

```
$ chmod 664 foobar
```

修改后： -rw-rw-r-- 1 archie web 5120 Jun 27 08:28 foobar

如果这是一个可执行文件，同时您想向所有者和组授予可执行权限，则权限数字为774。或者，如果您希望每个人都只具有读取权限，则数字将为444。将r视为4，w视为2，以及x视为1，这可能是计算chmod xxx *文件名*的权限数值最简单的方法。

但还有一种二进制方法，其中每个权限都代表一个二进制数，最后再将其转换为一个数字。它有点复杂，但为了完整起见，在此也介绍该方法。

对于以下权限组合：

```
-rwxrwxr-x
```

如果您在每个授予的权限下写1，而在每个未授予的权限下写0，则结果如下：

```
-rwxrwxr-x
 111111101
```

然后把它当成二进制数进行转换：

```
000=0       100=4
001=1       101=5
010=2       110=6
011=3       111=7
```

因此，上述值为775。

如果我们想从组中删除写权限：

```
-rwxr-xr-x
 111101101
```

因此，该值将为755，您可以使用 chmod 755 *文件名*删除可写权限。无论使用哪种方法，您最终都会得到相同的三位数。使用文本还是数字取决于个人偏好和打字速度。如果您希望将目录或文件还原为默认权限（例如所有者有读写和执行权限，但拒绝其他所有人的写权限），则使用chmod 755/644 *文件名*可能会更快。但如果您要将权限更改为比较少见的组合，则使用文本方法可能会更简单、更快，而不是尝试将其转换为数字，这可能会导致错误。对于只需要偶尔使用*chmod*的用户来说，这两种方法的速度都没有显著的差异。

您也可以用数字形式设置setuid, setgid 和 sticky 权限位。

```
setuid=4
```

```
setgid=2
sticky=1
```
比如 `chmod 2777 *filename*` 将为每个人设置读/写/可执行位，并启用`setgid`位。

批量 chmod

通常，目录和文件不应具有相同的权限。如果需要批量修改目录树，请使用find选择性地修改其中一个或另一个。

若要只把目录权限修改为 755：

```
$ find directory -type d -exec chmod 755 {} +
```
若要只把文件权限修改为 644：

```
$ find directory -type f -exec chmod 644 {} +
```
修改所有者

chown 能够修改文件或者目录的所有者，这在某些情况下会更简单快捷。

比如，您要使用GParted为备份数据创建一个新分区。Gparted以root身份执行所有操作，因此默认情况下所有内容都属于root。这一切都很好，但当要向挂载的分区写入数据时，常规用户会没有权限。

```
brw-rw---- 1 root disk 8,    9 Jul  6 16:02 sda9
drwxr-xr-x 5 root root    4096 Jul  6 16:01 Backup
```
若您所见，`/dev`是由root所有的，而挂载点(`/media/Backup`也是一样。如果我们要修改挂载点的所有者，您可以执行如下命令：

修改前：`drwxr-xr-x 5 root root 4096 Jul 6 16:01 Backup`

```
# chown archie /media/Backup
```
修改后：`drwxr-xr-x 5 archie root 4096 Jul 6 16:01 Backup`

现在在无需调整权限的情况下，这个分区就可以由新的所有者 archie 写入了。（作为所有者，它本身就拥有 `rwx` 权限）

注意：

- `chown` 总是会清除 setuid 和 setgid 权限位
- 非 root 用户不能使用 `chown` 来把他们自己拥有的文件「送给」其他用户

访问控制列表（ACL）

Access Control Lists 通过允许为任何用户或组设置对任何文件的权限，为文件系统提供了额外的、更灵活的权限机制。

Umask

umask 实用程序用于控制文件创建模式掩码，该掩码确定了新创建文件的权限位的初始值。

文件属性

除了控制用户和组读、写和执行权限的文件权限之外，还有几个文件系统支持文件属性，这些属性可以进一步自定义允许的文件操作。

警告：默认情况下， cp、rsync 和其他类似的程序不会保留文件属性

e2fsprogs包 软件包内有 lsattr(1) 和 chattr(1) 这两个可以列出和修改文件属性的工具。

以下是一些有用的属性，但并非所有的文件系统都支持每个属性。

- `a` - 仅追加（append only）：文件只能以追加的方式打开。
- `c` - 压缩（compressed）：为文件启用文件系统级的压缩。
- `i` - 不变（immutable）：无法被修改、删除、重命名或创建指向它的链接。只能由root用户设置。
- `j` - 数据日志（data journaling）：为文件的写入和元数据启用日志。
- `m` - 不压缩（no compression）：为文件禁用文件系统级的压缩。
- `A` - 禁用atim更新（no atime update）：文件的 atime （访问时间）将不会被修改。
- `C` - 禁用写时拷贝（no copy on write）：对于支持写时拷贝的文件系统，禁用写时拷贝。

查看 chattr(1) 以获得更详细的文件属性列表以及它们的作用。

比如，如果您想给某个文件设置不变（immutable）属性，使用如下命令：

```
# chattr +i /path/to/file
```

要删除文件上的属性，只需把 `+` 改为 `-`。

扩展属性

根据 xattr(7)：「扩展属性是与文件和目录永久关联的键值对」。有四个扩展属性类: security、 system、 trust 和 user。

警告：默认情况下，cp、rsync 和其他类似的程序不会保留文件属性。请参考 #保留扩展属性.

扩展属性还用于设置能力（Capabilities）.

用户扩展属性

用户扩展属性可用于存储有关文件的任意信息，比如：

```
$ setfattr -n user.checksum -v "3baf9ebce4c664ca8d9e5f6314fb47fb" foo.txt
```

使用getfattr来显示扩展属性：

```
$ getfattr -d foo.txt
# file: foo.txt
user.checksum="3baf9ebce4c664ca8d9e5f6314fb47fb"
```

如果要删除扩展属性：

```
$ setfattr -x user.checksum foo.txt
```

保留扩展属性

命令	所需的选项
cp	--preserve=mode,ownership,timestamps,xattr
mv	默认会保留[\[注 1\]](#cite_note-1)
tar	创建tar档时用 --xattrs ，解压时用 --xattrs-include='*'
bsdtar	解压时用 -p
rsync	--xattrs

1. ↑ 当目标文件系统不支持扩展属性时，mv 会静默地丢弃这些扩展属性。

要在使用文本编辑器的时候保留扩展属性，您需要把他们配置为在保存时截断（truncate）文件而非使用 rename(2).

技巧和窍门

Preserve root

使用 --preserve-root 选项来阻止 chmod 递归地从 / 修改文件权限。 这可以防止在系统范围内删除可执行位，从而破坏系统。要每次使用此标志，请在alias中设置它。参见 .

参见

- wikipedia:Chattr
- Linux 文件权限的困惑
- Linux 文件权限的困惑第二部分
- wikipedia:Extended file attributes#Linux
- 扩展属性：好，不太好，坏。
- 在 Linux 中备份和恢复文件权限
- 为什么 “chmod -R 777 /” 具有破坏性?
- Unix 中用户私有组的方式和原因

7.5 Dd

难度 高级 | **篇幅** 26.9 KB | **所属** 命令行

本节将讲到

- 安装
- 硬盘克隆与恢复
- 备份与恢复MBR
- 除磁盘操作外的额外用法
- 故障排除

dd) 是一个核心工具，其主要用途是复制文件，并可在复制过程中选择性地对文件进行转换。

与cp类似，默认情况下dd会按位复制文件，但还提供了更底层的 I/O 流控制功能。

更多信息参见dd(1)或完整文档。

提示：默认情况下，dd在执行复制操作过程中不会有任何输出。要监控操作进度，可添加status=progress选项。

警告：使用dd等磁盘操作类命令时应格外小心，错误使用可能会不可逆转地破坏数据。

安装

dd是GNU coreutils包的一部分。该软件包中的其它工具，可参见核心工具。

硬盘克隆与恢复

dd是一个简单而又功能多样的强大工具。其可逐块从源（source）到目标（destination）复制内容，而不关心源或目标的文件系统类型、操作系统。可通过live环境（如 Live CD）便捷的在任何计算机上使用dd。

警告：作为磁盘操作类命令，使用dd时应格外小心，避免破坏其它数据。使用时必须注意输入文件（if=）、输出文件（of=）的顺序，切勿颠倒！务必确保目标设备或分区（of=）的大于等于源（if=）的大小。

克隆分区

若要将硬盘/dev/sda上的第一个分区克隆到硬盘/dev/sdb的第一个分区上：

```
# dd if=/dev/sda1 of=/dev/sdb1 bs=64K conv=noerror,sync status=progress
```

注意：请确保of=中指定的输出分区（sdb1）存在。否则，dd会在根文件系统下创建同名文件并写入，而不是写入到目标分区中。

克隆整个硬盘

若要将硬盘/dev/sda上的所有内容克隆到另一个硬盘/dev/sdb上：

```
# dd if=/dev/sda of=/dev/sdb bs=64K conv=noerror,sync status=progress
```

这将克隆整个硬盘，包括分区表、引导加载程序、所有分区、UUID 和数据。

- `bs=`用于设置块大小，默认为512字节。512字节是自20世纪80年代初以来的硬盘的“经典”块大小。但在现代硬盘上，这并非最佳设置。建议使用64K、128K等更大的值。另外，请详细阅读下文的说明，该选项不仅设置“块大小”，还影响读取错误的处理方式。详情请参阅、。
- `noerror`表示当遇到读取错误时，忽略错误继续操作。在默认情况下，`dd`将在遇到任何错误时停止。
- `sync`表示当遇到读取错误时，用零填充输入块的末尾，以保持数据偏移量同步（若认为输入设备可能产生读取错误，建议阅读下文关于sync在遇到读取错误时的行为的详细解释）。
- `status=progress`显示传输统计信息，可用于估算操作完成时间。

注意：块大小设置会影响读取错误的处理方式，详见下文。若正在进行数据恢复，请使用ddrescue。

实际上，`dd`存在“输入块大小”（IBS）和“输出块大小”（OBS）。

当设置`bs`时，将同时设置IBS和OBS。在通常情况下，若设置块大小为1MiB，`dd`将从输入文件读取1024×1024字节，并写入输出文件。但若发生读取错误，则读取会提前终止。

在使用`noerror,sync`选项时，一种常见的错误看法是，设置`bs`为1MiB，若读取过程中发生错误，`dd`只会将发生错误的磁盘块（例如，大小可能为512字节）用零填充，而1MiB块的其它部分仍然包含正确的数据；但实际上，根据文档，`dd`会先尝试从输入文件读取IBS个字节，一旦发生读取错误，读取就立刻停止，之后用零将输入块填充到OBS个字节，再写入输出文件。这意味着，对于磁盘而言，若在当前1MiB数据块的头部便发生读取错误（例如，1个512字节的扇区无法读取），则将用零填充整个1MiB数据块，而不是只填充发生错误的扇区。若用ERROR表示错误，则12ERROR89会变成128900000，而不是120000089。

若确定磁盘没有错误，则可以使用更大的块大小以大幅提高复制速度。例如，在一个普通的 Celeron 2.7 GHz系统上，将`bs`从512改为64K，复制速度从35 MB/s提高到了120 MB/s。但需要注意，若发生读取错误，源磁盘上一个扇区的读取错误最终会成为目标磁盘上一整个块的错误，即一个512字节的读取错误可能会导致整个64 KiB的输出块被填充零。

注意：

- 在克隆后，若要为ext2/3/4文件系统重新生成UUID，以确保UUID的唯一性，可对每个需要调整的分区执行`tune2fs /dev/sd*XY* -U random`。对于交换分区，需要执行`mkswap -U random /dev/sd*XY*`。
- 在克隆GPT磁盘后，可使用sgdisk来随机化磁盘和分区的GUID，以保证其唯一性。

- 使用dd克隆磁盘导致的分区表更改不会自动被内核注册。要在不重启的情况下通知内核检查更改，可使用类似partprobe（GNU Parted 的一部分）的工具。

备份分区表

参见Fdisk#备份和恢复分区表、GPT_fdisk#备份和恢复分区表。

创建磁盘镜像

首先，从 live 介质启动，并确保要备份的源硬盘上没有任何分区被挂载。

然后，挂载用于备份的外部硬盘，备份源硬盘到镜像中：

```
# dd if=/dev/sda conv=sync,noerror bs=64M status=progress | lz4 -z
> /path/to/backup.img.lz4
```

如果需要（例如，当生成的文件将存储在FAT32等单文件大小限制较小的文件系统上时），可将磁盘镜像分割成多个部分（另见split(1)）：

```
# dd if=/dev/sda conv=sync,noerror bs=64M status=progress | lz4 -z | split -a3 -b2G
- /path/to/backup.img.lz4
```

如果本地没有足够的磁盘空间用于存储镜像文件，可以通过ssh发送镜像：

```
# dd if=/dev/sda conv=sync,noerror bs=64M status=progress | lz4 -z | ssh user@local dd
of=backup.img.lz4
```

最后，保存有关源磁盘物理结构的额外信息（其中最重要的是扇区大小）。这些信息可用于解析镜像中的分区表。

```
# fdisk -l /dev/sda > /path/to/list_fdisk.info
```

提示：

- 可设置块大小（bs=）为要备份的硬盘的缓存容量。例如，若源硬盘的缓存为512 MiB，可使用bs=512M。上文使用的64 MiB块大小的备份速度已经比默认的bs=512字节快，使用更大的块大小可能进一步提升备份速度。
- 上例使用lz4(1)进行压缩，其支持多线程，也可使用其它压缩工具。

恢复系统

要从镜像文件中恢复系统：

```
# lz4 -dc /path/to/backup.img.lz4 | dd of=/dev/sda status=progress
```

若镜像文件使用分卷方式储存，改用以下命令：

```
# cat /path/to/backup.img.lz4* | lz4 -dc | dd of=/dev/sda status=progress
```

备份与恢复MBR

在对磁盘进行更改之前，可能希望备份设备的分区表和分区方案。此外，还可以通过备份与恢复的方式将相同的分区布局复制到多个设备。

MBR存储在磁盘的前512字节中，由4个部分组成：

1. 前440字节包含引导加载程序代码（引导加载程序）。

2. 接下来的6字节包含磁盘签名。
3. 再接下来的64字节包含分区表（4个条目，每个16字节，每个条目对应一个主分区）。
4. 最后2字节包含引导签名。

要将MBR保存到`mbr_file.img`镜像中：

```
# dd if=/dev/sdX of=/path/to/mbr_file.img bs=512 count=1
```

此外，也可以从完整磁盘备份镜像中提取MBR：

```
# dd if=/path/to/disk.img of=/path/to/mbr_file.img bs=512 count=1
```

要将备份的MBR恢复到磁盘上（注意，这将销毁现有的分区表，导致磁盘上所有数据无法访问）：

```
# dd if=/path/to/mbr_file.img of=/dev/sdX bs=512 count=1
```

警告：若恢复的MBR与磁盘上的实际分区布局不匹配，则由于分区识别错误，磁盘上的数据将无法读取，且很难修复。如果只是需要重新安装引导加载程序，不应通过恢复MBR的方式进行，因为除了MBR区域外，引导加载程序还可能需要使用DOS 兼容区域，参见：GRUB、Syslinux。

若只需要恢复引导加载程序代码部分，而不恢复主分区表，则只需恢复MBR的前 440 字节：

```
# dd if=/path/to/mbr_file.img of=/dev/sdX bs=440 count=1
```

若要只恢复主分区表，不恢复引导加载程序代码部分，使用：

```
# dd if=/path/to/mbr_file.img of=/dev/sdX bs=1 skip=446 count=64
```

移除引导加载程序

要清除MBR中引导程序加载程序的代码（可能在需要完全重新安装另一操作系统时有用），只需将前440字节清零：

```
# dd if=/dev/zero of=/dev/sdX bs=440 count=1
```

除磁盘操作外的额外用法

虽然dd(1)支持一些其他常用工具中不支持的独特功能，但其与其它核心工具的命令行语法差距较大，许多默认行为并不符合预期，且在应用于特定场景时容易出错。因此，若可能，推荐使用在特定方面更易使用的其它工具。

不过，由于`dd`是核心工具，默认安装于Arch和其它众多系统上。因此，对于不便在系统上安装新软件包的情况，可能仍需使用`dd`进行操作，而不能使用其它更好的替代品。

为了同时涵盖上述两个方面，本节聚焦于dd(1)的独特功能。这些功能在其它常用工具中很少见。本节的形式类似于Pacman/各软件包管理器命令对应关系，同时给出使用

dd实现与使用其它工具实现的示例，但减少了其它工具的示例的数量，以聚焦于dd的功能。其它工具的示例将在“提示”框中通过实际指令或伪代码体现。

注意：为保持本节简洁，只考虑官方仓库中的替代工具，并着重展示dd具有明显优势的情况。如有必要，我们会详细解释其优势。 更多替代方案，可参见核心工具#dd_替代品。

原地逐块修补二进制文件

在自动化的shell脚本中，经常使用dd作为二进制文件修补程序。尽管其功能相比专用工具有限，但支持：

- 通过seek选项，在输出文件特定偏移量处开始写入。
- 覆盖输出文件中的特定部分，但不将输出文件截断到写入结束的位置（使用conv=notrunc选项）。

在下例中，使用dd修改了符合cpio(5) § Portable ASCII Format格式的归档文件中第一个成员的时间戳，该时间戳起始于文件的第49（十六进制0x30）字节：

```
$ touch a-randomly-chosen-file
$ bsdtar -cf example-modify-ts.cpio --format odc -- a-randomly-chosen-file
$ printf '%011o' "$(date -d "2019-12-21 00:00:00" +%s)" | dd conv=notrunc of=example-modify-ts.cpio seek=48 oflag=seek_bytes
```

注意：在上例中，使用了尚未写入文档的C0 输出标志，用于以字节而不是块为单位，指定写入输出文件的偏移量。

提示：若要将命令行中输入的十六进制表示转换为二进制字节流，可使用basenc(1) § base16或printf(1)。

提示：若需要在输出文件指定偏移量处开始写入数据，且不截断输出文件，除了使用dd外，还可以考虑使用支持在shell打开的文件描述符上调用lseek(2)的shell。若有以下需求：

- 输入文件是一个管道，其连接到一个使用splice(2)系统调用的程序，且希望避免dd产生的不必要的用户空间I/O，以获得更好的性能
- 为避免在shell脚本的循环中频繁fork(2)，以减少性能开销

那么需要让shell先打开文件描述符，在文件描述符上执行定位操作，并最终连接到相应程序的输出端。“相应程序”可以是上面提到的使用splice(2)系统调用的工具、不进行fork的shell内建命令（例如：zshmodules(1) § sysseek）：

```
$ zsh
$ local +xr openToWriteFD
$ zmodload zsh/system
$ sysopen -wu openToWriteFD example-modify-ts.cpio
$ sysseek -u $openToWriteFD 48
$ printf '%011o' "$(date -d "2019-12-21 00:00:00" +%s)" >&${openToWriteFD}
...
$ : finally close the fd {openToWriteFD}>&-
```

警告：若不能确定产生输出的相应程序是否使用了splice(2)，请避免使用此方法。因为这意味着对于该程序而言，不应在其输出上执行任何类型的定位或截断操作。 一些程序可能会自行在输入/输出文件描述符上进行定位/截断，即使未在命令行参数中要求该行为。程序执行的定位/截断操作将导致shell的lseek(3)调用无效，或导致打开的文件描述被意外截断。

输出VFAT文件系统镜像的卷标

提示：对于该需求，更好的选择是使用file包。

VFAT文件系统的卷标位于偏移量`0x047`处，长度为11字节，卷标长度小于11字节的，由ASCII空格填充到11字节。要读取存储在镜像文件中的VFAT文件系统的卷标：

```
$ truncate -s 36M empty-hole.img
$ mkfs.fat -F 32 -n LabelMe empty-hole.img
$ dd iflag=skip_bytes,count_bytes count=11 skip=$((0x047)) if=empty-hole.img | sed -e
's% *$%%'
```

注意：使用到的两个输入标志当前均未写入文档：

- `skip_bytes`：在开始从输入文件read(2)之前，先定位到指定偏移量，偏移量以字节数而不是块数指定。若输入不支持定位操作，则跳过该输入文件。
- `count_bytes`：以字节而不是块为单位指定从输入文件复制的数据量。注意，即使使用了该标志，仍可能遇到“部分读取”问题。其实际行为类似C1。

提示：在shell脚本中，若要实现从输入文件指定偏移量开始，读取指定长度的数据作为输出的功能，也可以考虑使用curl(1) § r,。

注意：若输入文件是设备/管道，`curl`不支持进行定位/跳过操作。此时可使用socat(1)，其支持对输入文件（包括块设备，但不包括管道和字符设备）进行这些定位操作，但不如`curl`常用：

```
$ socat -u -,seek=$((0x047)),readbytes=11 - < empty-hole.img | sed -e 's% *$%%'
```

在管道命令之间充当缓冲（sponge）

提示：对于该需求，也可使用sponge(1)，其先读取所有输入数据，将输入数据写入到临时目录（通过`$TMPDIR`环境变量指定；若未指定，则为`/tmp`），再一次性写入到输出中。从而实现原子写入。

在下例中，若输出端长时间阻塞，则输入端将长时间等待，导致建立的TCP连接持续时间过长。此时可以在两个命令之间“拼接”一个`dd`。`dd`的输出块大小应明显大于输入，同时需要小于可用内存容量：

```
$ curl -qgsfL http://example.org/mirrors/ftp.archlinux.org/mirrored.md5deep | dd
ibs=128k obs=200M | <一个逐行对输入文件中存储的路径信息执行镜像，但不会先缓冲整个输入文件的低效镜像脚本>
```

警告：这绝不应被视为sponge(1)的通用替代方案，因为`dd`在开始复制操作之前会截断输出文件。

传输有大小限制的数据

在数据流式 shell 脚本中，可以使用`dd`来限制管道命令可能消耗的数据总长度。例如，以流式方式使用shell脚本函数检查ustar头部块tar(5) § POSIX ustar Archives）：

注意：在`count`选项的参数中使用`B`后缀是GNU coreutils v9.1中新引入的功能，其效果与count_bytes输入标志相同。注意，在不使用`B`后缀时，`count`指定的是块数量，而不是字节数量，即使附加了其它后缀（例如，`k`）。例如，`count=256k`表示复制262144个输入块而不是262144字节。

```
hexdump-field() {
  set -o pipefail
  printf '%s[%d]:\n' $1 $2
  dd count=${2}B status=none | hexdump -e $2'/1 "%3.2x"' -e '" " '$2'/1 "%_p" "\n"'
}
```

```
inspect-tar-header-block() {
  local -a hdrstack=(
    name 100
    mode 8
    uid 8
    gid 8
    size 12
    mtime 12
    checksum 8
    typeflag 1
    linkname 100
    magic 6
    version 2
    uname 32
    gname 32
    devmajor 8
    devminor 8
    prefix 155
    pad 12
  )
  set - ${hdrstack[@]}
  while test $# -gt 0; do
    hexdump-field $1 $2 || return
    shift 2
  done
}
$ bsdtar -cf - /dev/tty /dev/null 2>&- | dd count=1 skip=1 status=none | inspect-tar-header-block
```

提示：为实现“使用流式方式，将给定长度的数据从输入传输到输出”的功能，可使用pv(1) § S,，其支持splice(2)系统调用。

注意：另一个替代方案是head(1) § c。不过，若在glibc上使用GNU coreutils之外的C0实现，则读取的输入数据量可能比指定的更多，导致在流式shell脚本中出现数据错位问题。

提示：除上述介绍外，如果输入文件在流式传输前需要lseek(2)到特定偏移量，并且dd的输出端是一个管道，且该管道连接到的程序使用splice(2)。那么，作为替代方案，可以考虑使用：

- 内置lseek(2)定位功能的shell（见前之前中的替代方案）。
- 使用类Bourne shell（例如bash），通过使用xxd(1) § s在shell打开的文件描述符上进行一次性lseek(2)操作，

并使用pv(1) § S,（上文已提及）。例如，下面的示例使用bash（假设文件描述符最初未被shell分配。在bash中，可通过`ls -l /proc/self/fd`查看已分配的文件描述符）：

```
$ bash
$ exec 9<dummy-but-rather-large.img
$ xxd -g 0 -l 0 -s $((0x47ffff)) <&9
$ pv -qSs 104857601200 <&9 | <一个处理大量数据但不会按需限制读取长度，也不支持按偏移量读取的程序>
$ exec 9<&-
```

注意：如coreutils测试套件中的一个示例所示，上例中的xxd(1) § s可使用dd并配合`count=0`、`skip`选项代替。虽然这种用法与POSIX标准和一些非GNU实现不兼容。

将可引导的磁盘镜像写入块设备，并按需显示进度信息

不推荐直接将dd用于该用途。其它常用工具参见U_盘安装介质#使用基本命令行工具。

提示：若要将文件内容写入块设备，并同时显示进度信息，相比直接使用`dd`，建议使用dd_rescue(1) § W替代。若块设备中已经存储了旧版本的镜像内容，在写入新版本的镜像时，其能避免不必要的写入操作。

故障排除

部分读取：复制的数据量小于要求的大小

若`dd`在尝试进行读取操作时，一个完整的输入块暂时不可用，则输出文件的最终的大小可能比要求复制的大小要小。如文档所述：

: 此外，如果未指定`conv`数据转换操作数（即本文中提到过的选项），从输入读取到的内容会立即被复制到输出，即使读取到的数据量小于块大小。

在Linux上，底层的read(2)系统调用在从管道（pipe(7)）或某些设备文件（例如`/dev/urandom`、`/dev/random`，由于底层内核设备驱动程序硬编码的限制）读取时，可能会在读取到部分数据，数据量还未达到参数指定的值时便提前返回（即"部分读取"）。这导致在结合使用`bs`和`count=*n*`选项时，复制的数据总大小小于预期。其中`count=*n*`指定了要复制到输出的输入块的数量，但每个输入块的大小可能小于`bs`字节。

在发生部分读取时，`dd`可能显示警告，但不保证一定会显示警告：

```
dd: warning: partial read (X bytes); suggest iflag=fullblock
```

解决方案是依照警告的提示，在`dd`命令中添加`iflag=fullblock`输入选项。例如，要创建一个大小为40MiB、填充了随机数据的新文件：

```
$ dd if=/dev/urandom of=new-file-filled-by-urandom.bin bs=40M count=1 iflag=fullblock
```

注意：当需要从管道或特殊设备（详见下文）中复制特定长度的一段内容时（通过`count=*n*`选项指定），强烈建议添加`iflag=fullblock`选项，特别是在擦除特定文件或设备的一部分时。

当从管道读取时，`iflag=fullblock`的一个替代方案是将`bs`设置为linux/limits.h中定义的`PIPE_BUF`常量值，以使得pipe(7) I/O成为原子操作。例如，若要创建总长度为5MiB，填充了随机字母数字字符串的文本文件：

```
$ LC_ALL=C tr -dc '[:alnum:]' < /dev/urandom | dd of=passtext-5m.txt bs=4k count=1280
```

若输出文件不是管道，可使用`ibs`、`obs`选项分别为输入管道和输出文件设置块大小。例如，由于输出文件存储在磁盘上，可设置更大的块大小，以提高效率：

```
$ LC_ALL=C tr -dc '[:alnum:]' < /dev/urandom | dd of=passtext-5m.txt ibs=4k obs=64k
count=1280
```

提示：在某些情况下，保持输出块大小与输入块大小相同且等于`PIPE_BUF`常量定义的值，可能就已经是最佳的了。

总传输字节数读数错误

如果在写入输出时遇到错误（例如收到SIGPIPE信号、发生介质故障或网络块设备意外断开），命令显示的总传输字节数可能会大于实际值（“部分写入”）。如下例所示，其中第二个`dd`从输入中读取的字节数显然不会超过512200字节，但第一个`dd`实例仍然报告了复制了512400字节：

```
$ yes 'x' | dd bs=4096 count=512400B | dd ibs=1 count=512200 status=none >/dev/null
125+1 records in
125+1 records out
512400 bytes (512 kB, 500 KiB) copied, 10.7137 s, 47.8 kB/s
```

对于类似上述例子的情况，当要恢复中断的传输时，建议以已经完整复制的输出块的数量为依据（`125+1 records out`中“+”号前面的数字，即125）。

注意：若即使添加`iflag=fullblock`选项后，部分I/O块数（“+”号后面的数字仍大于1，则说明部分I/O发生了不止一次。在这种情况下，为可靠地恢复传输进度，建议：

- 改用ddrescue重新进行传输，以更灵活地处理可能存在故障的介质上发生的部分读取。
- 若网络连接不稳定且需要写入网络设备，使用dd_rescue(1)以直接I/O方式重新进行传输。
- 避免向可能发生故障的介质写入。

另见

- dd(1p): POSIX specification of dd core utility in manpage form

7.6 核心工具

难度 高级 | **篇幅** 30.1 KB | **所属** 命令行

本节将讲到

- 基础
- 非关键
- 替代品
- 参阅

*核心工具*是 GNU/Linux 系统基本、基础的工具。本文提供对它们不完整的概览、链接文档并描述有用的替代品。本文的范围包括，但不限于 GNU coreutils。大多数核心工具都是传统的 Unix 工具，它们中的许多已由 POSIX 标准化，但已进一步发展，从而提供更多功能。

大多数命令行界面的文档都在 man 手册中，来自 GNU 项目的工具的文档主要在 Info 手册中，一些 shell 为内置命令提供了 `help` 命令。另外，大多数命令在使用 `--help` 标志运行时会显示自身的使用说明。

基础

下表列出了一些 Arch Linux 用户应当熟悉的重要的工具。另请参见 intro(1)。

包	工具	描述	文档	替代品
shell 内置	cd	变更目录	cd(1p)	#cd 替代品
GNU coreutils包	ls	列出目录	ls(1)、lsGNU 文档	tree包, #ls 替代品
cat	将文件拼接到 stdout	cat(1)、catGNU 文档	tac(1), #cat 替代品	
mkdir	创建目录	mkdir(1)、mkdirGNU 文档		
rmdir	删除空目录	rmdir(1)、rmdirGNU 文档		
rm	删除文件或目录	rm(1)、rmGNU 文档	shred unlink(1)	
cp	复制文件或目录	cp(1)、cpGNU 文档	#cp 替代品	
mv	移动文件或目录	mv(1), info		
ln	创建硬链接或符号链接	ln(1), info	sln(8) （soname 修复）	
chown	改变文件所有者和组	chown(1), info	chgrp(1)	
chmod	改变文件权限	chmod(1), info		
dd	转换并复制文件	dd(1), info	#dd 替代品	
df	报告文件系统磁盘空间使用情况	df(1), info	#df 替代品	
du	估算文件和文件夹占用的空间	du(1), info	#du 替代品	
GNU tar包	tar	tar 归档程序	tar(1), info	归档与压缩
GNU less包	less	终端分页程序	less(1)	终端翻页器
GNU findutils包	find	搜索文件或目录	find(1), info, GregsWiki	#find 替代品
GNU diffutils包	diff	逐行比对文件	diff(1), info	#diff 替代品
GNU grep包	grep	打印匹配模式的行	grep(1), info	#grep 替代品
GNU sed包	sed	流编辑器	sed(1), info, one-liners\[失效链接 2025-08-15 ⓘ\]	sad包, sd包
GNU AWK (gawk包)	awk	模式扫描和处理语言	gawk(1), info	AWK#替代方案
util-linux包	dmesg	print or control the kernel ring buffer	dmesg(1)	systemd 日志
lsblk	列出块设备	lsblk(8)		
mount	挂载文件系统	mount(8)		
umount	卸载文件系统	umount(8)		
su	替换用户	su(1)	sudo, doas	
kill	中止进程	kill(1)	pkill(1), killall(1)	
procps-ng包	pgrep	根据名称或属性查找进程	pgrep(1)	pidof(1)
ps	显示进程信息	ps(1)	top(1), 任务管理器	
free	显示已用和可用的内存	free(1)		

防止数据损失

`rm`、`mv`、`cp` 和 shell 重定向无需确认即可删除或覆盖文件。`rm`、`mv` 和 `cp` 都支持 `-i` 标志以在删除/覆盖前提醒用户。一些用户喜欢通过 alias 默认启用 `-i`。依赖这些 shell 选项可能很危险，因为你会习惯于它们，从而在你使用没有它们的系统或用户时导致潜在的数据损失。防止数据损失的最好办法是创建备份。

非关键

此表格列出了常常很有用的核心工具。

包	工具	描述	文档	替代品
shell 内置	alias	定义或显示别名	alias(1p)	
type	打印命令的类型	type(1p)	command(1p), whereis(1), which(1)	
time	记录命令执行时间	time(1p)		
GNU coreutils包	tee	读取 stdin，写入 stdout 和文件	tee(1), info	pee(1)
mktemp	创建临时文件或目录	mktemp(1), info		
mknod	创建命名管道或设备节点	mknod(1), mkfifo(1), info		
truncate	将文件缩小或扩展到指定的大小	truncate(1), info	fallocate(1)	
basenc	编码数据并输出	basenc(1), base64(1), info		
cut	打印选中的行	cut(1), info	colrm(1), hck包, choose包	
tr	转换或删除字符	tr(1), info	uconv(1)	
od	以 8 进制或其他格式转储文件	od(1), info	hexdump(1), vim's xxd(1)	
sort	给行排序	sort(1), info		
uniq	报告或忽略重复的行	uniq(1), info	anewer包, runiqAUR, huniq-gitAUR	
comm	逐行比对两个有序的文件	comm(1), info	zetAUR	
head	输出文件的前一部分	head(1), info		
join	将两个输入中有相同栏位的行连接起来	join(1), info	combine(1) zetAUR	
md5sum	根据密码学计算输入的哈希值并输出	sha256sum(1), sha512sum(1), info	shasum(1), hashdeep(1), rhash(1)	
tail	输出文件的末尾部分，或者动态跟踪显示文件末尾内容	tail(1), info		
wc	输出换行、单词和字节数	wc(1), info		
GNU binutils包	strings	打印二进制文件中可打印的字符	strings(1), info	stringsextAUR

包	工具	描述	文档	替代品
util-linux包	column	转化文件为列格式，也可按表格方式美观地打印	column(1)	paste(1), csviewAUR
GNU findutils包	xargs	从标准输入组合或按模板构建外部命令并执行	xargs(1)	parallel(1) (parallel_alternatives(7))
GNU glibc包	iconv	转换字符编码	iconv(1)	recode包, uconv(1)
GNU sharutils包	uudecode	将文件编码为电子邮件友好的文本格式	uuencode(1), uudecode(1), info	uudeview(1)
file包	file	猜测文件类型	file(1)	

moreutils包 包提供了 GNU coreutils 中没有的有用的工具，例如 sponge(1)。

替代品

核心工具的替代品由以下包提供：

- **BusyBox** — 用于救援系统和嵌入式系统的核心工具。

: https://busybox.net || busybox包

- **Toybox** — 一体化的 Linux 命令行。

: https://landley.net/toybox || toyboxAUR

- **uutils** — Rust 重构的跨平台的 GNU 核心工具。

: https://github.com/uutils/coreutils || uutils-coreutils包

- **Heirloom Toolchest** — 标准 Unix 工具的传统实现。

: https://heirloom.sourceforge.net || heirloom-shAUR, heirloom-doctoolsAUR

- **9base** — 原版 Plan9 工具在 Unix 的移植。

: https://tools.suckless.org/9base || 9base包

- **sbase** — *nix 核心工具的一个极简变体。

: https://core.suckless.org/sbase || sbase-gitAUR

- **ubase** — sbase 工具的一个扩展。

: https://core.suckless.org/ubase || ubase-gitAUR

cat 替代品

- **bat** — 支持语法高亮和 Git 集成的 cat 克隆。

: https://github.com/sharkdp/bat || bat包

cd 替代品

- **zoxide** — 一个智能化的 cd 命令，可以学习用户的习惯，只需按几个键即可随处导航。

: https://github.com/ajeetdsouza/zoxide || zoxide包

- **autojump** — 在命令行中导航到文件的快速方式。

: https://github.com/wting/autojump || autojumpAUR

参阅 Bash#输入路径自动添加"cd" 和 Zsh#记住最近的目录。

cp 替代品

使用 rsync#作为 cp/mv 的替代可以断点续传、显示传输状态、跳过已存在的文件并使用校验和确保目标文件的完整性。

ls 替代品

- **broot** — 一种在目录间浏览和导航的新方法。

: https://github.com/Canop/broot || broot包

- **clifm** — 一个像 ls(1) 一样列出文件的文件管理器，加上了图标和 RGB 颜色支持。

: https://github.com/leo-arch/clifm/wiki/Advanced#files-lister-ls-mode || clifmAUR

- **eza** — 另一个 ls 的替代品，具有颜色支持、树视图、git 集成和其他功能。基于停止支持的 exa 开发而来。

: https://github.com/eza-community/eza || eza包

- **lsd** — ls 的现代版，具有很多漂亮的颜色和美观的图标。

: https://github.com/Peltoche/lsd || lsd包

find 替代品

- **fd** — find 的简单、快速且易用的替代品。默认忽略隐藏文件和 `.gitignore` 文件。

: https://github.com/sharkdp/fd || fd包

- **fuzzy-find** — 模糊查找文件。

: https://github.com/silentbicycle/ff || ff-gitAUR

- **plocate** — 一个更快速的 locate 命令。

: https://plocate.sesse.net/ || plocate包

- **rawhide** — 使用 C 语言样式的表达式查找文件。

: https://raf.org/rawhide/ || rawhideAUR

图形界面的文件搜索工具请参阅应用程序列表/工具#文件搜索。

diff 替代品

diffutils包 不提供单词级别的文本比较，一些其他程序提供了这一功能：

- git diff 可以加上 `--color-words` 参数进行单词级别的比较，使用 `--no-index` 可以比较 Git 工作树外的文件。
- **cwdiff** — 一个对 GNU wdiff 的包装，对输出进行了着色。

: https://github.com/junghans/cwdiff || cwdiffAUR

- **git-delta** — 一个带语法高亮功能的分页器，作用于 git、diff 和 grep 的输出。

: https://dandavison.github.io/delta/ || git-delta包

- **dwdiff** — diff 程序加上单词比较功能的前端，支持着色。

: https://os.ghalkes.nl/dwdiff.html || dwdiffAUR

- **icdiff** — 带着色的、Python 编写的 diff 工具。“改进版彩色 diff”用于扩充 diff 的用法。

: https://github.com/jeffkaufman/icdiff || icdiffAUR

- **wdiff** — GNU diff 加上单词比较功能的实现，不支持着色。

: https://www.gnu.org/software/wdiff/ || wdiff包

参阅应用程序列表/工具#比较，差异，合并。

grep 替代品

- **mgrep** — 多行 grep。

: https://sourceforge.net/projects/multiline-grep/ || mgrepAUR

- **pdfgrep** — 在 PDF 文件中搜索文本的工具。

: https://pdfgrep.org/ || pdfgrep包

- **ripgrep-all** — 搜索纯文本，包括在 PDF、电子书、Office 文档、zip、tar.gz 中的文本。

: https://github.com/phiresky/ripgrep-all || ripgrep-all包

代码搜索工具

以下 3 个工具旨在在代码搜索方面替代 grep。它们默认执行递归搜索、跳过二进制文件并遵守 `.gitignore`。

- **ack** — 基于 Perl 的 grep 替代品，为搜索大量异构的源码树而生。

: https://beyondgrep.com/ || ack包

- **ripgrep (rg)** — 结合了 ag 的易用性和 grep 的速度。

: https://github.com/BurntSushi/ripgrep || ripgrep包

- **The Silver Searcher (ag)** — 类似 Ack 的代码搜索工具，不过更快。

: https://github.com/ggreer/the_silver_searcher || the_silver_searcher包

- **ugrep (ug)** — 超快速的 grep，具备交互式用户界面、模糊搜索、布尔查询、十六进制转储等功能。

: https://github.com/Genivia/ugrep || ugrep包

参阅 cscope包。

交互式过滤器

- **fzf** — 通用的命令行模糊查找工具，默认由 find 提供支持。

: https://github.com/junegunn/fzf || fzf包

- **fzy** — 一种简单快速的文本模糊选择工具，具有先进的评分算法。

: https://github.com/jhawthorn/fzy || fzy包

- **peco** — 简单的交互式过滤工具。

: https://github.com/peco/peco || peco包

- **percol** — 为传统的 UNIX shell 管道概念加上了交互式过滤功能。

: https://github.com/mooz/percol || percolAUR

- **skim** — 用 Rust 编写的模糊查找工具，与 fzf 类似。

: https://github.com/lotabout/skim || skim包

dd 替代品

参阅 dd 和 ddrescue。

dd 的替代实现

本小节列举了 *dd* 的其他实现，它们的接口和默认行为基本符合 dd(1p) 的 POSIX 规范。

- **ddpt** — 由 Linux 内核的 SCSI 子系统维护者对 sg_dd(8) 进行的可移植重写，具有可选但非常专业的硬件 I/O（SCSI 命令集）支持，还有许多其他功能。

: http://sg.danny.cz/sg/ddpt.html || ddptAUR

- **sdd** — 可以跨 UNIX 环境使用的 dd 实现，由 Joerg Schilling 开发，可以对复制的数据进行校验并重新读取损坏数据。

: https://schilytools.sourceforge.net/ || schily-tools-sddAUR

GNU dd 的衍生版

核心工具中的 GNU 实现的 dd 的也符合 POSIX。本小节列出了它的衍生版。

- **dcfldd#dcfldd)** — GNU dd 的功能增强衍生版，用于取证和安全场景，包括即时哈希、灵活擦除、写入验证、同时输出到多个目标、拆分和管道输出。

: https://dcfldd.sourceforge.net || dcflddAUR

- **dc3dd#dc3dd)** — GNU dd 的另一个打了补丁的版本，来自美国国防部网络犯罪中心 (DC3)，与 dcfldd 具有相似的目标和功能。

: https://sourceforge.net/projects/dc3dd/ || dc3ddAUR

现代化的 dd 仿照版

本小节列举的 *dd* 替代方案，在类 JCL 命令行语法和默认行为方面是不符合 POSIX 的。

- **dd_rescue** — 功能丰富的现代化 dd 仿照版，适用于日常脚本编写、磁盘克隆和数据恢复。

: http://www.garloff.de/kurt/linux/ddrescue/ || dd_rescue包

- **rw** — 小型且可移植的 *dd* 仿照版，具有传统的命令行参数。

: https://sortix.org/rw/ || rwAUR

buffer 的衍生版

本小节列举了 bufferAUR 的衍生版，buffer 是一个类似于 *dd* 的通用 I/O 缓冲程序，但具有动态大小的缓冲区。它支持区块为单位的 I/O，可以用来对 LTO 磁带进行进/出转储，以避免擦鞋 (Shoe Shining) 效应。

- **mbuffer** — *buffer* 程序的延续，具有多线程等功能。

: https://www.maier-komor.de/mbuffer.html || mbuffer包

df 替代品

- **duf** — 一个查看磁盘使用情况/剩余空间的工具。

: https://github.com/muesli/duf || duf包

du 替代品

- **cdu** — du 的包装，带有着色和漂亮的直方图。

: http://arsunik.free.fr/prog/cdu.html || cduAUR

- **dust** — du 的更直观版本，用 Rust 编写。

: https://github.com/bootandy/dust || dust包

- **ncdu** — 基于 ncurses 的简单磁盘使用分析器。

: https://dev.yorhel.nl/ncdu || ncdu包

- **gdu** — 带控制台界面的磁盘使用分析器，用 Go 编写。

: https://github.com/Dundee/gdu || gdu包

参阅应用程序列表/工具#磁盘使用情况显示。

参阅

- coreutils 手册GNU 文档
- GNU Coreutils FAQ
- Coreutils Gotchas: GNU 核心工具的维护者写的关于核心组件一些奇怪行为的注释
- POSIX 工具

Git 网络界面

想把提交历史摆到浏览器里，或者干脆自建一套协作平台，就得在这几个工具里挑一个。四节由轻到重排开：《Gitweb》最轻，只多一道网络服务器的配置；《Gitea》居中，安装、配置、用法、技巧与问题解决俱全；《GitLab》最重，篇幅近半，另多出启动测试与每次更新时的数据库升级；《SonarQube》另成一类，安装、配置、升级、问题解决单列。

读法上，《Gitea》必读，自建平台照它走；《Gitweb》可速读；《GitLab》按需展开，功能最全，但要连带照看数据库与后续配置；只想浏览仓库，《SonarQube》可以跳过。最容易踩的坑是把升级当成换个包：《GitLab》每次更新都要单独处理数据库，漏掉这步服务就起不来。同样别以为装上即可访问，《Gitweb》的前置是先把网络服务器配好。

8.1　SonarQube

难度 入门 | **篇幅** 4.8 KB | **所属** Git 网络界面

本节将讲到

- 安装
- 配置
- 升级
- 用法
- 问题解决

翻译状态：

- 本文（或部分内容）译自 SonarQube，最近一次同步于 2024-8-1，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

SonarQube 是用于改进代码的代码质量工具，以 LGPL3 许可发布。

安装

安装 sonarqube-binAUR 软件包。

SonarQube 默认使用 H2 数据库（不推荐）。 或需要使用数据库后端，支持以下数据库：

- MariaDB/MySQL
- PostgreSQL
- Oracle
- MSSQL

配置

用户配置文件位于 `/etc/webapps/sonarqube/sonar.properties` 中。

更多配置示例请参阅 SonarQube docs。

PostgreSQL

安装并配置 PostgreSQL。

SonarQube 似乎只支持 TCP Socket 。

使用 TCP 套接字

连接 postgresql：

```
[postgres]$ psql
```

以 `postgres` 用户身份连接服务器时创建新用户（系统会提示输入新用户密码）：

```
postgres=# CREATE USER sonarqube WITH PASSWORD password;
```

创建 Gitea 数据库，由 `gitea` 用户拥有：

```
postgres=# CREATE DATABASE sonarqube OWNER sonarqube;
postgres=#GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonarqube;
```

配置 PostgreSQL，使其可从远程主机访问

验证其工作状态：

```
$ psql --host=ip_address --dbname=sonarqube --username=sonarqube --password
```

通过更新 `sonar.properties` 配置 SonarQube：

```
/etc/webapps/sonarqube/sonar.properties
sonar.jdbc.url=jdbc:postgresql://localhost/sonarqube
sonar.jdbc.username=sonarqube
sonar.jdbc.password=password
sonar.web.javaOpts=-Xmx512m -Xms128m -XX:+HeapDumpOnOutOfMemoryError
sonar.web.javaAdditionalOpts=-server
sonar.web.host=0.0.0.0
sonar.web.port=9000
sonar.log.level=INFO
sonar.path.logs=logs
```

升级

升级 SonarQube 后，请按照以下步骤操作：

- 更新 `sonar.properties` 文件以匹配新版本。
- SonarQube 运行后，会出现一个页面提示您升级。详细说明请访问升级向导。
- 导航至 SonarQube 服务器的设置页面 `https://*ip*:*port*/setup` 并点击升级按钮。
- 升级过程完成后，将显示正常的登录页面。

用法

启动/启用 `sonarqube.service`，网络接口应在 `http://localhost:9000` 上监听。

默认证书为 `admin/admin`。

注意：

- 您可能需要配置一个反向代理进行远程访问，例如 nginx。
- 如果不想让 Sonarqube 监听所有接口，请在 `/etc/webapps/sonarqube/sonar.properties` 中将 `sonar.web.host` 设置为 127.0.0.1 等。

与 maven 一起使用

```
$ mvn clean verify sonar:sonar -Dsonar.projectKey=project key -
Dsonar.projectName=project -Dsonar.host.url=http://localhost:9000 -Dsonar.token=token
```

注意：*project key*、*project* 和 *token* 的值可在网络界面创建。

与 sonar-scanner 一起使用

有一个软件包 sonar-scannerAUR。用法如下

```
$ /opt/sonar-scanner/bin/sonar-scanner -Dsonar.projectKey=project key -Dsonar.sources=.
-Dsonar.host.url=http://localhost:9000 -Dsonar.token=token
```

注意：*project key* 和 *token* 的值可在网络界面创建。

问题解决

您可以查看单元状态或 journal。 还有 sonarqube 日志：

```
# /var/log/sonarqube/
```

如果 sonarqube 未激活，其中一个日志将包含错误信息。

参见

- Sonarqube Documentation
- sonarqube 安装手册

8.2 Gitweb

难度 入门 | **篇幅** 5.8 KB | **所属** Git 网络界面

本节将讲到

- 安装
- 配置网络服务器
- 配置 Gitweb
- 参见

Gitweb 是 Git 自带的默认网络界面。

Gitweb 实际上原生支持 FCGI，因此无需将其包装成 CGI 脚本。

安装

要安装 Gitweb，首先要安装 git包 和网络服务器。如果想快速测试，请参阅 `git instaweb` 的帮助。否则，如果你想要一个全面的设置，请继续阅读。

以下所有示例都需要安装 perl-cgi包 软件包。（FS#45431）。

配置网络服务器

Apache

将以下内容添加到 `/etc/httpd/conf/httpd.conf` 的末尾：

```
Alias /gitweb "/usr/share/gitweb"
<Directory "/usr/share/gitweb">
   DirectoryIndex gitweb.cgi
   Options ExecCGI
   Require all granted
   <Files gitweb.cgi>
   SetHandler cgi-script
   </Files>
   SetEnv  GITWEB_CONFIG  /etc/gitweb.conf
</Directory>
```

或者将其添加到另一个文件中，如 `etc/httpd/conf/extra/gitweb.conf` 然后在 `httpd.conf` 末尾添加以下内容：

```
# gitweb configuration
Include conf/extra/gitweb.conf
```

如果 Apache 拒绝显示 Gitweb，而是打印 perl 脚本的纯源代码，则很可能是 Apache 的配置不允许执行 CGI。请确保 `httpd.conf` 中的以下内容未被注释：

```
<IfModule !mpm_prefork_module>
   LoadModule cgid_module modules/mod_cgid.so
</IfModule>
<IfModule mpm_prefork_module>
   LoadModule cgi_module modules/mod_cgi.so
</IfModule>
```

然后重启 `httpd.service` 来应用这些更改。有关使用 Apache 执行 CGI 的更多详情，请参阅 https://httpd.apache.org/docs/2.4/howto/cgi.html。

Lighttpd

在 `/etc/lighttpd/lighttpd.conf` 中添加以下内容：

```
server.modules += ( "mod_alias", "mod_cgi", "mod_redirect", "mod_setenv" )
url.redirect += ( "^/gitweb$" => "/gitweb/" )
alias.url += ( "/gitweb/" => "/usr/share/gitweb/" )
$HTTP["url"] =~ "^/gitweb/" {
       setenv.add-environment = (
               "GITWEB_CONFIG" => "/etc/gitweb.conf",
               "PATH" => env.PATH
       )
       cgi.assign = ( ".cgi" => "" )
       server.indexfiles = ( "gitweb.cgi" )
}
```

然后重启 `lighttpd.service` 来应用这些更改。 您可能还需要在 `mimetype.assign` 行添加 `".css" => "text/css"` 以便 GitWeb 正常显示。

Nginx

将此位置添加到 nginx 配置中（可能需要更改位置）：

```
/etc/nginx/nginx.conf
location /gitweb.cgi {
    include fastcgi_params;
    gzip off;
    fastcgi_param   SCRIPT_FILENAME  /usr/share/gitweb/gitweb.cgi;
    fastcgi_param   GITWEB_CONFIG  /etc/gitweb.conf;
    fastcgi_pass    unix:/run/fcgiwrap.sock;
}

location / {
    root /usr/share/gitweb;
    index gitweb.cgi;
}
```

如果您遵循 Nginx#CGI 实现，请尝试将`include fastcgi_params;`替换为 `include fastcgi.conf;`。

配置 Gitweb

所有配置选项的列表，请参见 gitweb.conf(5) 。

基本配置

打开（如果不存在，则创建）文件 `/etc/gitweb.conf`，并将此内容放入其中：

```
/etc/gitweb.conf
# The directories where your projects are. Must not end with a slash.
our $projectroot = "/path/to/your/repositories";

# Base URLs for links displayed in the web interface.
our @git_base_url_list = qw(git://<your_server> http://git@<your_server>);
```

要启用 "blame" 视图（显示源文件中每一行的作者），请添加以下一行：

```
$feature{'blame'}{'default'} = ;
```

配置完成后，重启网络服务器。

添加仓库

要添加版本库，请进入版本库文件夹，像这样创建版本库：

```
$ mkdir my_repository.git
$ git init --bare my_repository.git/
$ cd my_repository.git/
$ touch git-daemon-export-ok
$ echo "Short project's description" > description
```

接下来打开 `config` 文件并添加以下内容：

```
config
[gitweb]
owner = Your Name
```

这将填写 Gitweb 中的 "Owner" 字段。这不是必填项。

这里假定您希望将此仓库作为 "central" 仓库，将您的提交推送至此，因此 git-daemon-export-ok 和 --bare 的作用是将开销降至最低，并允许在此仓库上使用 git 守护进程。

这就是创建版本库的全部过程。现在你可以在 http://localhost/gitweb 上看到它了（假设一切顺利）。由于 Gitweb CGI 脚本只需读取版本库文件夹，因此无需为新版本库重启 Apache。

语法高亮

要在 Gitweb 上启用语法高亮，必须先安装 highlight包 软件包：

安装 highlight 后，只需在 `gitweb.conf` 中添加此行即可：

```
$feature{'highlight'}{'default'} = ;
```

保存文件后，高亮显示就会启用。

参见

- How To Install A Public Git Repository On A Debian Server——HowtoForge 页面用作本文的主要资料来源。

8.3 Gitea

难度 进阶 | **篇幅** 13.3 KB | **所属** Git 网络界面

本节将讲到

- 安装
- 配置
- 用法
- 技巧和窍门
- 问题解决

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 需要翻译。（在 Talk:Gitea# 中讨论）

翻译状态：

- 本文（或部分内容）译自 Gitea，最近一次同步于 2024-04-12，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

Gitea 是 Gogs 的社区管理分支，Gogs 是用 Go 编写并在 MIT 许可下发布的轻量级代码托管解决方案。

安装

安装 gitea包 或 gitea-gitAUR 软件包。还有一个 gitea fork forgejo包 软件包。

Gitea 需要使用数据库后端，支持以下数据库：

- MariaDB/MySQL
- PostgreSQL
- SQLite
- MSSQL

配置

用户配置文件位于 `/etc/gitea/app.ini`。

有关更多配置示例，参见 Gitea 文档。

PostgreSQL

安装并配置 PostgreSQL.

在 TCP 或 UNIX 套接字之间进行选择，然后跳转到相应的部分。

注意：当 Gitea 和 PostgreSQL 在同一台机器上时，您应该使用 Unix 套接字，因为它更快、更安全。

用 TCP socket

以 `postgres` 用户身份连接服务器时创建新用户（系统会提示输入新用户的密码）：

```
[postgres]$ createuser -P gitea
```

创建用户 `gitea` 拥有的 Gitea 数据库：

```
[postgres]$ createdb -O gitea gitea
```

PostgreSQL#Configure PostgreSQL to be accessible from remote hosts

验证它是否有效：

```
$ psql --host=ip_address --dbname=gitea --username=gitea --password
```

通过首次运行安装程序或更新 `app.ini` 配置 Gitea：

```
/etc/gitea/app.ini
DB_TYPE              = postgres
HOST                 = hostadress:port
NAME                 = gitea
USER                 = gitea
; Use PASSWD = `your password` for quoting if you use special characters in the
password.
PASSWD               = password
```

用 Unix socket

以 `postgres` 用户身份连接服务器时创建新用户（系统会提示输入新用户的密码）：

```
[postgres]$ createuser gitea
```

创建用户 `gitea` 拥有的 Gitea 数据库：

```
[postgres]$ createdb -O gitea gitea
```

通过将以下行添加到 `/var/lib/postgres/data/pg_hba.conf` 来设置 Unix 套接字：

```
/var/lib/postgres/data/pg_hba.conf
local    gitea           gitea           peer
```

重新启动 `postgresql.service`.

验证它是否有效：

```
[gitea]$ psql --dbname=gitea --username=gitea
```

通过首次运行安装程序或更新 `app.ini` 配置 Gitea：

```
/etc/gitea/app.ini
DB_TYPE             = postgres
HOST                = /run/postgresql/
NAME                = gitea
USER                = gitea
PASSWD              =
```

MariaDB/MySQL

注意：使用`/var/run/mysqld/mysqld.sock` 作为监听参数，即可启用 MySQL 套接字支持。

以下是设置MariaDB的示例，设置所需的密码：

```
$ mysql -u root -p
mysql> CREATE DATABASE `gitea` DEFAULT CHARACTER SET `utf8mb4` COLLATE
`utf8mb4_unicode_ci`;
mysql> CREATE USER `gitea`@'localhost' IDENTIFIED BY 'password';
mysql> GRANT ALL PRIVILEGES ON `gitea`.* TO `gitea`@`localhost`;
mysql> FLUSH PRIVILEGES;
mysql> \q
```

尝试使用新用户连接到新数据库：

```
$ mysql -u gitea -p -D gitea
```

通过首次运行安装程序或更新 `app.ini` 配置MariaDB：

```
/etc/gitea/app.ini
DB_TYPE  = mysql
HOST     = 127.0.0.1:3306 ; or /var/run/mysqld/mysqld.sock
NAME     = gitea
USER     = gitea
PASSWD   = password
```

用法

启动/启用 `gitea.service`, 网络接口应在 `http://localhost:3000` 上监听。

首次运行 Gitea 时，应该会重定向到 `http://localhost:3000/install`。

注意：

- 您可能希望配置反向代理以进行远程访问，例如 nginx.
- 如果希望 Gitea 监听所有接口，请在 `/etc/gitea/app.ini` 中设置 `HTTP_ADDR = 0.0.0.0`。

技巧和窍门

本地 shell 客户端 (tea)

通过 tea包，您可以使用 gitea 的官方客户端。更多信息请访问 https://gitea.com/gitea/tea 。

启用 SSH 支持

确保 SSH 已正确配置并运行。

设置域名

您可能希望设置`SSH_DOMAIN`, 例如

```
/etc/gitea/app.ini
SSH_DOMAIN                = git.domain.tld
```

注意： 如果将 `PROTOCOL` 设置为 `unix`，则需要取消设置 `LOCAL_ROOT_URL` 或将其设置为 `http://unix/`。参见 comment。

配置 SSH

默认情况下，Gitea 将以用户 `gitea` 的身份运行；该账户也将用于 ssh 版本库访问。要使 ssh 访问正常，必须启用 PAM。或者，也可以解锁服务账户。

```
/etc/ssh/sshd_config
...
UsePAM yes
...
```

如果在 SSH 配置中使用 `AllowUsers`，请在其中添加 `AllowUsers gitea`，例如：

```
/etc/ssh/sshd_config
...
AllowUsers archie gitea
...
```

如果使用 `sshd.service`，请重新启动该服务（如果使用 `sshd.socket`，则无需重启）。

禁用 HTTP 协议

默认情况下，已启用通过 HTTP 协议与版本库交互的功能。 如果使用SSH，可以将 `DISABLE_HTTP_GIT` 设置为 `true`，禁用 HTTP 支持。

绑定受限端口

如果使用内置 SSH 服务器并希望 Gitea 将其绑定到 22 端口，或希望直接将 Gitea 网络服务器绑定到 80/443 端口（即在不使用代理的设置中），则需要添加 drop-in systemd unit override：

```
/etc/systemd/system/gitea.service.d/override.conf
[Service]
AmbientCapabilities=CAP_NET_BIND_SERVICE
```

```
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
PrivateUsers=false
```

启用 Dark 主题

在*ui*部分，可以将 `DEFAULT_THEME` 设置为 `arc-green`，使网页界面使用深色背景。

自定义 Gitea 的外观

更多详情，请参阅 Gitea 文档。

gitea 的外观可使用Go模板进行高度定制。 首先，创建 `/var/lib/gitea/custom` 目录。 可通过编辑 `/var/lib/gitea/custom/templates` 中的文件覆盖模板。 默认模板可在 `templates` 目录下的 Gitea 源代码中找到。例如，要自定义主页，可将源代码中的 `templates/home.tmpl` 复制到 `/var/lib/gitea/custom/templates/home.tmpl` 并根据需要编辑模板。

此外，还可以通过创建以下文件更改徽标和图标： `/var/lib/gitea/custom/public/img/logo.svg` 和 `/var/lib/gitea/custom/public/img/favicon.svg`

更改这些文件后，需要重新启动 Gitea。

配置反向代理

有关更多信息和示例，请参阅 Gitea 文档网站 上的反向代理部分。

nginx

The following is an example of using nginx as reverse proxy for Gitea over unix socket (you need to provide the SSL certificate):

```
/etc/nginx/servers-available/gitea.conf
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name git.domain.tld;

    ssl_certificate /path/to/fullchain.pem;
    ssl_certificate_key /path/to/privkey.pem;

    location / {
        client_max_body_size 512M;
        proxy_pass http://unix:/run/gitea/gitea.socket;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Update the `[server]` and `[session]` section of `app.ini`:

```
/etc/gitea/app.ini
[server]
PROTOCOL                  = unix
```

```
DOMAIN                       = git.domain.tld
ROOT_URL                     = https://git.domain.tld
HTTP_ADDR                    = /run/gitea/gitea.socket
LOCAL_ROOT_URL               =

[session]
COOKIE_SECURE                = true
```

注意：You do not need to activate any SSL certificate options in `/etc/gitea/app.ini`.

Apache HTTP Server

The following is an example of using the Apache HTTP Server as reverse proxy for Gitea over unix socket. To forward `domain.tld` to the gitea server, use

```
/etc/httpd/conf/httpd.conf
ProxyPreserveHost On
ProxyRequests off
AllowEncodedSlashes NoDecode
Proxypass / unix:/run/gitea/gitea.socket|http://domain.tld nocanon
ProxypassReverse / unix:/run/gitea/gitea.socket|http://domain.tld nocanon
```

where `domain.tld` should be replaced by your domain name (this entry is only passed as a header to the proxy, and does not seem to matter for this setup).

```
/etc/gitea/app.ini
[server]
PROTOCOL                     = unix
DOMAIN                       = domain.tld
ROOT_URL                     = https://domain.tld
HTTP_ADDR                    = /run/gitea/gitea.socket
LOCAL_ROOT_URL               =
```

To forward a subpath such as `domain.tld/git` to the gitea server, use

```
/etc/httpd/conf/httpd.conf
<Proxy *>
  Order allow,deny
  Allow from all
</Proxy>
AllowEncodedSlashes NoDecode
Proxypass /git unix:/run/gitea/gitea.socket|http://domain.tld nocanon
ProxypassReverse /git unix:/run/gitea/gitea.socket|http://domain.tld nocanon
/etc/gitea/app.ini
[server]
PROTOCOL                     = unix
DOMAIN                       = domain.tld
ROOT_URL                     = https://git.domain.tld
HTTP_ADDR                    = /run/gitea/gitea.socket
LOCAL_ROOT_URL               =
```

Setup for custom data directory

As of now, you cannot use a custom path like `/srv/gitea` as your server home, since the shipped `gitea.service` unit file marks everything read-only.

To enable these custom paths, create a drop-in snippet with your server home directory as a new `ReadWriteDirectories` directive:

```
/etc/systemd/system/gitea.service.d/data-directory.conf
[Service]
ReadWriteDirectories=/srv/gitea
```

Then do a daemon-reload and restart `gitea.service` for the changes to take effect.

问题解决

升级到 1.5.0 后启动数据库时出现错误

升级到 1.5.0 后可能会出现问题。服务无法启动，日志中出现以下错误：

```
/var/log/gitea/gitea.log
2018/08/21 16:11:12 [...itea/routers/init.go:60 GlobalInit()] [E] Failed to initialize
ORM engine: migrate: do migrate: Sync2: Error 1071: Specified key was too long; max key
length is 767 bytes
```

要解决这个问题，请在 MySQL/MariaDB 服务器上以 `\root\` 用户身份运行以下命令

```
$ mysql -u root -p
MariaDB> set global innodb_large_prefix = `ON`;
```

gitea should stop complaining about key size and startup properly.

Service failing with permission denied

如果手动创建 `gitea` 用户，并将其主页文件夹设置为 `/home/gitea`，gitea 服务将无法启动，并输出类似于：

```
Sep 04 04:44:32 systemd: gitea.service: Failed with result 'exit-code'.
Sep 04 04:44:32 systemd: gitea.service: Main process exited, code=exited,
status=200/CHDIR
Sep 04 04:44:32 (gitea): gitea.service: Failed at step CHDIR spawning /usr/bin/gitea:
Permission denied
Sep 04 04:44:32 (gitea): gitea.service: Changing to the requested working directory
failed: Permission denied
```

该服务需要将用户的主文件夹作为 gitea 的主文件夹，默认文件夹为 `/var/lib/gitea` ：

```
$ usermod -d /var/lib/gitea gitea
```

参见

- Gitea Documentation

8.4 GitLab

难度 高级 | **篇幅** 26.8 KB | **所属** Git 网络界面

本节将讲到

- 安装
- 配置
- 开始并测试Gitlab
- 每次更新时升级数据库
- 更多配置

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： No updates since 2018（在 Talk:GitLab# 中讨论）

翻译状态：

- 本文（或部分内容）译自 Gitlab，最近一次同步于 2024-2-23，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

这是来自 Gitlab的主页:

: GitLab 提供 git 仓储管理、代码查阅、问题追踪、动态监控和维基. 企业在本地安装Gitlab并用LDAP和活动目录服务器连接以进行安全的身份验证和授权。单个GitLab服务器可以处理超过25,000个用户，但也可以使用多个活动服务器创建高可用性设置。

你可以在 GitLab.com找到实时版本的例子.

安装

注意：这篇文章优先覆盖非https安装和配置Gitlab. 如果需要，查阅#更多配置 来设置SSL.

Gitlab需要 Redis 和数据库后端. 如果你计划在同一个机器运行的话, 首先得安装 PostgreSQL (简体中文).

Install gitlab包 包.

为了接受邮件通知, 一个邮件服务器必须被安装和配置. 查阅 Category:Mail server 获取更多细节.

配置

初步说明

Gitlab包括多种组件, 可查阅架构概览页.

gitlab包 包会以更接近标准Linux公约的方式安装Gitlab:

描述	GitLab's Official	gitlab包
配置文件 GitShell	`/home/git/gitlab-shell/config.yml`	`/etc/webapps/gitlab-shell/config.yml`
配置文件 GitLab	`/home/git/gitlab/config/gitlab.yml`	`/etc/webapps/gitlab/gitlab.yml`
用户 (家目录)	`git` (`/home/git`)	`gitlab` (`/var/lib/gitlab`)

提示：如果你熟悉 Arch Build System 你可以编辑PKGBUILD和相关文件来改变gitlab的家目录到你喜欢的位置.

GitLab

编辑 `/etc/webapps/gitlab/gitlab.yml` 并至少设置下面的参数:

注意： `hostname` 和 `port` 被用于 `git clone [http://hostname:port] (http://hostname:port)` 的例子中.

Hostname: 在 `gitlab:` 区里设置 `host:` - 把 `localhost` 换成 `yourdomain.com` (**注意:** 不要加 'http://' 或者后面的斜线) - 到你的完全合格的域名.

Port: `port:` 可能会让你很困扰. 这不是Gitlab服务器运行的端口(unicorn) ; 这是用户通过浏览器初次连接的端口. 基本上, 如果你有意让用户通过他们的浏览器访问 'yourdomain.com' , 而且不用附加端口号, 把 `port:` 设成 `80`. 如果你想让用户输入比如 'yourdomain.com:3425' 这样的东西到浏览器, 那么你就得设置 `port:` 成 `3425`. 你也会得**设置你的网络服务器**来监听那个端口.

Timezone (时区，可选): `time_zone:` 参数是可选的, 但是它可能对强制Gitlab应用的区域有用.

最后设置正确的 permissions 到 *uploads*（上传） 目录:

```
# chmod 700 /var/lib/gitlab/uploads
```

为Puma自定义端口

GitLab Puma 是处理大多数用户请求的主要组件. 默认的, Unix套接字 /run/gitlab/gitlab.socket，可以通过 `/etc/webapps/gitlab/puma.rb` 文件来改变:

```
/etc/webapps/gitlab/puma.rb
bind 'unix:///run/gitlab/gitlab.socket'
bind 'tcp://127.0.0.1:8080'
```

如果 Puma 地址被改变, 其它与Puma交流的组件的配置也需要被更新:

- 对于 GitLab Shell, 在 `/etc/webapps/gitlab-shell/config.yml`更新 `gitlab_url` 变量.

: > 提示：根据注释里的配置文件, UNIX 套接字能通过 URL转义斜杠(/) (即. `http+unix://%2Frun%2Fgitlab%2Fgitlab.socket` 对应默认的 `/run/gitlab/gitlab.socket`). 额外的, 未转义斜杠能用来指定 相关URL根路径 (例如. `/gitlab`).

- 对于 GitLab Workhorse, edit `gitlab-workhorse.service` 并更新 `-authBackend` 选项.

机密字符串

确定 `/etc/webapps/gitlab/secret` 和 `/etc/webapps/gitlab-shell/secret` 文件包含了些什么. 它们的内容应该被保密因为它们会被用作生成认证令牌等.

比如, 能够通过下面的命令生成随机字符串:

```
# hexdump -v -n 64 -e '1/1 "%02x"' /dev/urandom > /etc/webapps/gitlab/secret
# chmod 640 /etc/webapps/gitlab/secret
# hexdump -v -n 64 -e '1/1 "%02x"' /dev/urandom > /etc/webapps/gitlab-shell/secret
# chmod 640 /etc/webapps/gitlab-shell/secret
```

同时需要填写以下新的机密字符串

```
/etc/webapps/gitlab/secrets.yml
production:
  secret_key_base: secret
  db_key_base: secret
  otp_key_base: secret
  openid_connect_signing_key: secret
```

Redis

为了提供足够的性能，你可能需要一个缓存数据库. 安装并配置一个redis实例, 注意专用于通过套接字监听的部分.

- 添加 `gitlab` 用户到 `redis` user group.
- 更新配置文件:

```
/etc/webapps/gitlab/resque.yml
development:
  url: unix:/run/redis/redis.sock
test:
  url: unix:/run/redis/redis.sock
production:
  url: unix:/run/redis/redis.sock
/etc/webapps/gitlab-shell/config.yml
# Redis settings used for pushing commit notices to gitlab
redis:
  bin: /usr/bin/redis-cli
  host: 127.0.0.1
  port: 6379
  # pass: redispass # Allows you to specify the password for Redis
  database: 5 # Use different database, default up to 16
  socket: /run/redis/redis.sock # uncomment this line
  namespace: resque:gitlab
```

数据库后端

在Gitlab运行之前需要一个\[PostgreSQL\]数据库.

PostgreSQL

登录 PostgreSQL 并创造 `gitlabhq_production` 数据库和它的用户一起. 记得改变 `your_username_here` 和 `your_password_here` 到你的真正的值:

```
# psql -d template1
template1=# CREATE USER your_username_here WITH PASSWORD 'your_password_here';
template1=# ALTER USER your_username_here SUPERUSER;
template1=# CREATE DATABASE gitlabhq_production OWNER your_username_here;
template1=# \q
```

注意：创建超级用户的目的是Gitlab在视图变得"智能"并安装扩展(并不仅仅在它的命名区间创建).并且它只被Postgresql的超级用户允许.

用新用户连接到新数据库来确定它有用:

```
# psql -d gitlabhq_production
```

打开新的 `/etc/webapps/gitlab/database.yml` 并设置 `username:` 和 `password:`的值. 比如:

```
/etc/webapps/gitlab/database.yml
#
# PRODUCTION
#
production:
  main:
    adapter: postgresql
    encoding: unicode
    database: gitlabhq_production
    username: your_username_here
    password: "your_password_here"
    # host: localhost
    # port: 5432
    socket: /run/postgresql/.s.PGSQL.5432
  ci:
    adapter: postgresql
    encoding: unicode
    database: gitlabhq_production
    database_tasks: false
    username: your_username_here
    password: "your_password_here"
    # host: localhost
    # port: 5432
    socket: /run/postgresql/.s.PGSQL.5432
...
```

对于我们的目的 (除非你知道你在干什么),你不必担心配置列在 `/etc/webapps/gitlab/database.yml`的其它数据库. 我们只需要设置生产的数据库来让Gitlab工作.

注意：注意：自 GitLab 15.9 起，数据库.yml 中只有 main: 部分的做法已被淘汰。在 GitLab 17.0 及更高版本中，你必须在 database.yml 中包含两个部分：main: 和 ci:. ci:必须与 main: 连接到同一个数据库。

防火墙

如果你想通过Iptables (简体中文) 防火墙给予Gitlab安装过程直接权限, 你可能需要调整端口和网络地址:

```
# iptables -A tcp_inbound -p TCP -s 192.168.1.0/24 --destination-port 80 -j ACCEPT
```

启用 API-access:

```
# iptables -A tcp_inbound -p TCP -s 192.168.1.0/24 --destination-port 8080 -j ACCEPT
```

如果你在一个路由器后面, 别忘了转发这个端口到Gitlab服务器端口, 如果你想运行WAN-access的话.

初始化Gitlab数据库

在初始化数据库之前开启 Redis 服务器和 `gitlab-gitaly.service` .

初始化数据库并激活更多特性:

```
$ cd /usr/share/webapps/gitlab
$ sudo -u gitlab $(cat environment | xargs) bundle exec rake gitlab:setup
GITLAB_ROOT_PASSWORD=yourpassword GITLAB_ROOT_EMAIL=youremail
GITLAB_LICENSE_FILE=/path/to/license
```

最后运行下面的命令来检查你的安装:

```
$ sudo -u gitlab $(cat environment | xargs) bundle exec rake gitlab:env:info
$ sudo -u gitlab $(cat environment | xargs) bundle exec rake gitlab:check
```

注意：

- *gitlab:env:info* 和 *gitlab:check* 命令会显示一个和Git相关的严重的错误. 这是正常的.
- *gitlab:check* 会显示缺少初始化脚本. 这也没什么好担心的, 因为 systemd 服务文件会被相应使用 (而这Gitlab无法识别).

调整修改位

(如果用户和组所有权没有正确配置的话Gitlab检查不会通过)

```
# chmod -R ug+rwX,o-rwx /var/lib/gitlab/repositories/
# chmod -R ug-s /var/lib/gitlab/repositories
# find /var/lib/gitlab/repositories/ -type d -print0 | xargs -0 chmod g+s
```

开始并测试Gitlab

确定 PostgreSQL (简体中文) 和 Redis 运行和设置正确.

然后 start/enable `gitlab.target`.

现在你可以通过访问 http://localhost:8080 或者 http://yourdomain.com来测试你的 Gitlab\[失效链接 2020-08-02 ⓘ\], 你可能会被提示创建密码:

```
username: root
password: You'll be prompted to create one on your first visit.
```

查阅#故障排除和在 `/usr/share/webapps/gitlab/log/` 目录下的日志文件来 排除故障.

每次更新时升级数据库

手动操作

在更新 gitlab包 包后, 需要升级数据库:

```
$ cd /usr/share/webapps/gitlab
$ sudo -u gitlab $(cat environment | xargs) bundle exec rake db:migrate
```

之后, 之后重载并重启Gitlab相关服务: `gitlab-sidekiq.service`, `gitlab-puma.service`, `gitlab-workhorse.service` 和 `gitlab-gitaly.service`

自动操作

你可以创建 pacman 钩子来自动升级 GitLab 软件包更新的数据库。创建以下三个文件，不要忘记使 shell 脚本可执行：

```
/etc/pacman.d/hooks/05-gitlab-pre.hook
[Trigger]
Operation = Upgrade
Type = Package
Target = gitlab

[Action]
Description = Stopping gitlab services
When = PreTransaction
Exec = /usr/bin/systemctl stop gitlab-gitaly.service gitlab-mailroom.service gitlab-puma.service gitlab-sidekiq.service gitlab-workhorse.service
```

```
/etc/pacman.d/hooks/99-gitlab-post.hook
[Trigger]
Operation = Upgrade
Type = Package
Target = gitlab

[Action]
Description = Migrating GitLab database and starting services
When = PostTransaction
Exec = /etc/pacman.d/scripts/gitlab-migrate-database.sh
/etc/pacman.d/scripts/gitlab-migrate-database.sh
#!/bin/sh

cd "/usr/share/webapps/gitlab"
sudo -u gitlab $(cat environment | xargs) bundle exec rake db:migrate

# The hook runs after 30-systemd-daemon-reload.hook so another systemctl daemon-reload
is not necessary.
systemctl start gitlab.target
```

更多配置

基本的 SSH

在完成了基本安装后, 为用户设置ssh权限. 安全外壳协议的配置会在下面描述. 其它SSH客户端和服务器需要不同的调整.

获取添加用户SSH秘钥的建议，过程在GitLab 网站里描述的很好了. 你可以在 `/var/lib/gitlab/log/gitlab-shell.log`检查管理员日志来确认用户SSH秘钥被正确提交了. 在这些事件之后, GitLab 添加这些秘钥到*authorized_keys* 文件里，它在 `/var/lib/gitlab/.ssh/authorized_keys`.

测试秘钥的常见方法 (比如: `$ ssh -T git@*YOUR_SERVER*`) 需要一点额外配置才能正常工作. 在 `/etc/webapps/gitlab/gitlab.yml` (默认用户: `gitlab`)里配置的用户必须添加到服务器的sshd配置文件,除此之外还有几个其它改变 :

```
/etc/ssh/sshd_config
PubkeyAuthentication   yes
AuthorizedKeysFile     %h/.ssh/authorized_keys
```

更新配置文件之后, 重启ssh守护进程:

```
# systemctl restart sshd
```

测试用户SSH米哟啊 (可选添加 -v 来查看额外信息):

```
$ ssh -T gitlab@YOUR_SERVER
```

自定义SSH连接

如果你在一个非标准端口运行SSH，你必须改变Gitlab用户的SSH配置:

```
/var/lib/gitlab/.ssh/config
host localhost      # Give your setup a name (here: override localhost)
user gitlab         # Your remote git user
port 2222           # Your port number
hostname 127.0.0.1; # Your server name or IP
```

你还必须在 `/etc/webapps/gitlab/gitlab.yml` 文件里改变相应的选项 (比如. ssh_user, ssh_host, admin_uri) .

HTTPS/SSL

改变Gitlab配置

修改 `/etc/webapps/gitlab/shell.yml` 那样到你的Gitlab站点的URL就会以 `https://`开头. 修改 `/etc/webapps/gitlab/gitlab.yml` 那样 `https:` 设置就会被设为 `true`.

查阅 Apache HTTP Server#TLS 和 Let's Encrypt.

Let's Encrypt(让我们加密吧)

验证你的URL, Let's Encrypt的过程会试图用像 `https://gitlab.*YOUR_SERVER_FQDN*/.well-known/acme-challenge/*A_LONG_ID*`的东西连接你的Gitlab服务器. 但是, 因为Gitlab配置, 每个到 `gitlab.*YOUR_SERVER_FQDN*` 的请求会被重定向到一个代理 (gitlab-workhorse) 而它无法处理这个URL.

为了绕过这个问题, 你可以使用 Let's Encrypt的webroot配置,
在 `/srv/http/letsencrypt/`设置webroot.

除此之外, 强迫到Gitlab的Let's Encrypt请求重定向到这个webroot可通过添加下面的:

```
/etc/http/conf/extra/gitlab.conf
Alias "/.well-known"  "/srv/http/letsencrypt/.well-known"
RewriteCond   %{REQUEST_URI}  !/\.well-known/.*
```

Web服务器配置

如果你想把Gitlab集成进一个运行的服务器而不是用它的内置http服务器Unicorn，那么按照这些说明操作.

Node.js

你可以轻松在443端口设置http代理来代理到8080端口的Gitlab程序的流量，通过为Node.js使用http-master. 在你创建你的域名的 OpenSSL 秘钥并获取你的CA 证书 (或自己设置的)后, 然后去 https://github.com/CodeCharmLtd/http-master 来学习使用https代理到Gitlab的请求多容易. http-master 建立在 node-http-proxy之上.

Nginx

查阅 Nginx#Configuration 获取基本的 *nginx* 配置信息和 Nginx#TLS 来启用 HTTPS. 在这个部分的例子假设服务器区块是用 Nginx#Managing server entries管理的.

创建和编辑基于下面代码的配置. 查阅 upstream GitLab repository获取更多例子.

```
/etc/nginx/servers-available/gitlab
upstream gitlab-workhorse {
  server unix:/run/gitlab/gitlab-workhorse.socket fail_timeout=0;
}
```

```
server {
  listen 80;
  #listen 443 ssl; # uncomment to enable ssl
  server_name example.com

  #ssl_certificate ssl/example.com.crt;
  #ssl_certificate_key ssl/example.com.key;

  location / {
      # unlimited upload size in nginx (so the setting in GitLab applies)
      client_max_body_size 0;

      # proxy timeout should match the timeout value set
in /etc/webapps/gitlab/unicorn.rb
      proxy_read_timeout 60;
      proxy_connect_timeout 60;
      proxy_redirect off;

      proxy_set_header Host $http_host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-Ssl on;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto $scheme;

      proxy_pass http://gitlab-workhorse;
  }

  error_page 404 /404.html;
  error_page 422 /422.html;
  error_page 500 /500.html;
  error_page 502 /502.html;
  error_page 503 /503.html;
  location ~ ^/(404|422|500|502|503)\.html$ {
    root /usr/share/webapps/gitlab/public;
    internal;
  }
}
```

Apache

安装并配置 Apache HTTP Server (简体中文). 你可以使用这些 upstream recipes 来开始Gitlab虚拟主机的配置文件.

对于SSL配置查阅 Apache HTTP Server#TLS. 如果你不需要它, 移除它. 注意到SSL虚拟主机需要特定IP而不是通用IP. 同样如果你为Unicorn设置了自定义端口, 不要忘了在 `BalanceMember` 行也设置它.

Gitlab-workhorse

**这一章节正在考虑移除。**

原因: gitlab-workhorse包 组件现在是有必要的. 默认设置能应对普遍的情况所以这个部分能被移除. (在 Talk:GitLab 讨论)

Gitlab8.0以后使用分开的http服务器gitlab-workhorse包来应对大量的http请求比如 Git 推送/拉取. 如果你想使用这个而不是 SSH, 安装 gitlab-workhorse包 包, 启用 `gitlab-workhorse.service` 并为它配置web服务器. gitlab-workhorse包 优先级应高于 `gitlab-unicorn` 根据 GitLab 的建议: https://gitlab.com/gitlab-org/gitlab-ce/issues/22528#note_16036216

注意：Unicorn还是需要所以不要disable或者stop它 `gitlab-unicorn.service`.

默认 gitlab-workhorse包 监听 `/run/gitlab/gitlab-workhorse.socket`. 你可以 edit `gitlab-workhorse.service` 并改变参数 `-listenAddr` 来让它监听一个地址, 例如 `-listenAddr 127.0.0.1:8181`. 如果你监听一个地址的话你也要把网络类型设置为 `-listenNetwork tcp`

当你使用nginx时记得修改你的nginx配置文件. 从gitlab-unicorn 转换到 gitlab-workhorse 相应的编辑以下两个设置

```
/etc/nginx/servers-available/gitlab
upstream gitlab {
   server unix:/run/gitlab/gitlab-workhorse.socket fail_timeout=0;
}

...

      proxy_pass http://unix:/run/gitlab/gitlab-workhorse.socket;
  }
}
```

有用的建议

隐藏选项

到Gitlab的家目录:

```
# cd /usr/share/webapps/gitlab
```

并运行:

```
# rake -T | grep gitlab
rake gitlab:app:check                             # GITLAB | Check the configuration of the
GitLab Rails app
rake gitlab:backup:create                         # GITLAB | Create a backup of the GitLab
system
rake gitlab:backup:restore                        # GITLAB | Restore a previously created
backup
rake gitlab:check                                 # GITLAB | Check the configuration of
GitLab and its environment
rake gitlab:cleanup:block_removed_ldap_users      # GITLAB | Cleanup | Block users that
have been removed in LDAP
rake gitlab:cleanup:dirs                          # GITLAB | Cleanup | Clean namespaces
rake gitlab:cleanup:repos                         # GITLAB | Cleanup | Clean repositories
rake gitlab:env:check                             # GITLAB | Check the configuration of the
environment
rake gitlab:env:info                              # GITLAB | Show information about GitLab
and its environment
rake gitlab:generate_docs                         # GITLAB | Generate sdocs for project
rake gitlab:gitlab_shell:check                    # GITLAB | Check the configuration of
GitLab Shell
rake gitlab:import:all_users_to_all_groups        # GITLAB | Add all users to all groups
(admin users are added as owners)
rake gitlab:import:all_users_to_all_projects      # GITLAB | Add all users to all projects
(admin users are added as masters)
rake gitlab:import:repos                          # GITLAB | Import bare repositories from
gitlab_shell -> repos_path into GitLab project instance
rake gitlab:import:user_to_groups[email]          # GITLAB | Add a specific user to all
groups (as a developer)
rake gitlab:import:user_to_projects[email]        # GITLAB | Add a specific user to all
projects (as a developer)
rake gitlab:satellites:create                     # GITLAB | Create satellite repos
rake gitlab:setup                                 # GITLAB | Setup production application
```

```
rake gitlab:shell:build_missing_projects      # GITLAB | Build missing projects
rake gitlab:shell:install[tag,repo]           # GITLAB | Install or upgrade gitlab-
shell
rake gitlab:shell:setup                       # GITLAB | Setup gitlab-shell
rake gitlab:sidekiq:check                     # GITLAB | Check the configuration of
Sidekiq
rake gitlab:test                              # GITLAB | Run all tests
rake gitlab:web_hook:add                      # GITLAB | Adds a web hook to the
projects
rake gitlab:web_hook:list                     # GITLAB | List web hooks
rake gitlab:web_hook:rm                       # GITLAB | Remove a web hook from the
projects
rake setup                                    # GITLAB | Setup gitlab db
```

备份和恢复

创建Gitlab系统的备份:

```
# sudo -u gitlab -H rake RAILS_ENV=production gitlab:backup:create
```

从之前创建的备份文件恢复 `/home/gitlab/gitlab/tmp/backups/20130125_11h35_1359131740_gitlab_backup.tar`:

```
# sudo -u gitlab -H rake RAILS_ENV=production gitlab:backup:restore
BACKUP=/home/gitlab/gitlab/tmp/backups/20130125_11h35_1359131740
```

注意： 备份文件夹在 `config/gitlab.yml`设置. GitLab 备份和恢复记录在 这里.

通过SMTP从Gitlab发送邮件

你可能想用 gmail (或者其它邮件服务) 从你的Gitlab寄邮件. 这能避免在Gitlab服务器上安装邮件守护进程的需要.

根据你的邮件服务器设置调整 `smtp_settings.rb`：

```
/usr/share/webapps/gitlab/config/initializers/smtp_settings.rb
if Rails.env.production?
  Gitlab::Application.config.action_mailer.delivery_method = :smtp

  ActionMailer::Base.delivery_method = :smtp
  ActionMailer::Base.smtp_settings = {
    address:              'smtp.gmail.com',
    port:                 587,
    domain:               'gmail.com',
    user_name:            'username@gmail.com',
    password:             'application password',
    authentication:       'plain',
    enable_starttls_auto: true
  }
end
```

Gmail 会拒绝这样接受的邮件 (并会邮寄给你一个邮件它拒绝了). 你需要关闭安全认证 (按照拒绝邮件中的链接) 来解决这个问题. 更安全的办法是为username@gmail.com开启双因素认证并为这个配置文件设置应用密码.

故障排除

HTTPS不是绿色 (gravatar不使用 https)

Redis 缓存了 gravatar 图像, 如果你用http拜访 GitLab, 那么启用 https, gravatar 会加载不安全的图像. 你可以清楚这些缓存通过执行以下命令

```
cd /usr/share/webapps/gitlab
RAILS_ENV=production bundle-2.3 exec rake cache:clear
```

以Gitlab用户的身份.

更新后的错误

从AUR更新安装包后, 数据库迁移和资源更新有时候会失败. 这些步骤可以解决这些问题, 如果简单的开关机不行的话.

首先, 移动到Gitlab安装目录.

```
# cd /usr/share/webapps/gitlab
```

如果每个Gitlab页面都给了一个500页面, 那么数据库迁移和资源可能太陈旧了. 如果不是的话, 跳过这步.

```
# su - gitlab -s /bin/sh -c "cd '/usr/share/webapps/gitlab'; bundle-2.3 exec rake db:migrate RAILS_ENV=production"
```

如果Gitlab一直在等待布置完成的话, 那么资源可能没有被重新编译.

```
# su - gitlab -s /bin/sh -c "cd '/usr/share/webapps/gitlab'; bundle-2.3 exec rake gitlab:assets:clean gitlab:assets:compile cache:clear RAILS_ENV=production"
```

最后, 重启Gitlab服务再测试你的网站.

```
# systemctl restart gitlab-unicorn gitlab-sidekiq gitlab-workhorse
```

Gitlab-Unicorn 无法访问非默认的从库目录

如果自定义的仓库存储目录被设置在 `/home`, 在`gitlab-unicorn.service`禁用 `ProtectHome=true` 参数 (查阅 Systemd#Drop-in files 和 github.com上的相关论坛帖子).

无法连接到 Gitaly

有时候, Gitaly服务无法开始, 让gitlab无法连接到gitlay. 解决办法很简单:

```
# systemctl start gitlab-gitaly
```

也可查阅

- Official installation documentation
- GitLab recipes with further documentation on running it with several web servers
- GitLab source code

Linux 控制台

虚拟控制台是登录与排障的底层入口，默认配置往往只够登录。这一章处理登录与周边：用 Getty 管理登录和虚拟控制台数量，用键盘配置调整布局、按键延时和重复频率，再往下是字体、光标、HiDPI 与提示音。

顺序按 Getty、键盘配置、Linux 控制台推进，前两节必读，否则登录和输入都难保证；Gpm 管控制台内的鼠标，用不上跳过。常见误解：以为自动登录只拨一个开关，实际还要把虚拟控制台一并安排；以为引导消息会自己留在 tty1，其实它和 Getty 争同一块屏幕；以及把按键延时、重复频率的调整当成键盘映射生效。

9.1　Gpm

难度 入门 | **篇幅** 3.8 KB | **所属** Linux 控制台

本节将讲到

- 安装
- 配置
- 参见

GPM（General Purpose Mouse，通用鼠标）是为 Linux 虚拟控制台（TTY）提供鼠标支持的守护进程。

安装

安装 gpm包 软件包。有关笔记本电脑上的触摸板支持，请参阅笔记本电脑#触摸板。

配置

`-m` 参数用于声明要使用的鼠标。`-t` 参数用于定义鼠标类型。要获取 `-t` 选项的可用类型列表，请使用 `-t help` 运行 `gpm`。

```
# gpm -m /dev/input/mice -t help
```

gpm包 软件包需要使用一些参数启动。这些参数可以通过创建文件 `/etc/conf.d/gpm` 记录，也可以在直接运行 *gpm* 时使用。`gpm.service` 默认包含 USB 鼠标的参数（`ExecStart=/usr/bin/gpm -m /dev/input/mice -t imps2`）。

显然，如果需要为其他鼠标类型使用该服务，应该对其进行编辑，最好以 systemd 友好的方式。

- PS/2 鼠标：

```
    -m /dev/psaux -t ps2
```

- IBM Trackpoint（“小红点”）：

```
    -m /dev/input/mice -t ps2
```

注意：如果鼠标只有 2 个按钮（无中键），则将 `-2` 传递给 `GPM_ARGS`，第二个按钮将执行粘贴功能。

找到合适的配置后，启动并启用 `gpm.service`。

更多信息见 gpm(8)。

QEMU 或 VirtualBox

QEMU 和 VirtualBox 默认模拟的鼠标在使用 *gpm* 和 X 时存在严重的定位和点击问题。鼠标的定位与主机失去同步，因此无法在不反复退出并重新进入窗口的情况下悬停于某些区域。点击的位置与光标显示的位置不符。

QEMU 和 VirtualBox 通过提供 USB 平板设备的模拟来解决此问题，该模拟提供了绝对定位功能。（libvirt包 自动使用这种方式。）

然而，*gpm* 只知道如何以相对定位模式使用模拟的鼠标，因此这些问题依然存在。尝试通过 `-t` 使用其他类型也均无法正确工作。

gpm-vmAUR 包含了一个已有数年的拉取请求，用于添加 VirtualBox 的 USB 平板支持（在 QEMU 下也能工作），并修改了 `gpm.service` 文件以默认启用它。

你可能需要更改所使用的事件。给 *gpm* 原始的 `-m /dev/input/mice` 不会起作用。默认情况下：

```
/etc/gpm-vm.conf
event="/dev/input/event2"
```

你可以通过安装 evtest包 并运行以下命令来确定要使用的事件：

```
# evtest
...
/dev/input/event2:      QEMU QEMU USB Tablet
...
```

如果需要给 *gpm* 添加额外的选项，可以在 `/etc/gpm-vm.conf` 中设置 `additional_args`。

一旦找到合适的配置，启动并启用 `gpm.service`。

参见

- Gentoo:GPM
- consolationAUR 基于 libinput 的替代品

9.2 Getty

难度 进阶 | **篇幅** 6.3 KB | **所属** Linux 控制台

本节将讲到

- 安装
- 添加额外的虚拟控制台
- 自动登录到虚拟控制台
- 将引导消息保留在 tty1 上
- 参考资料

getty) 是管理终端线路及其所连终端的程序的通用名称。其目的是保护系统，防止未经授权的访问。通常，每个 getty 进程由 systemd 启动，一个进程管理一条终端线路。

安装

agetty 是 Arch Linux 中默认的 getty 程序，它是 util-linux包 包的一部分。它在等待登录时修改 TTY 设置，使得换行符不会转换为 CR-LF，否则会使打印到控制台的消息产生“阶梯效应”。Agetty 管理着虚拟控制台，Arch Linux 中默认提供六个虚拟控制台。一般按 `Ctrl+Alt+F1` 到 `Ctrl+Alt+F6` 来访问它们。

其他可选替代包括：

- **mingetty** — 一个允许自动登录的最小化 getty。

: mingettyAUR || mingettyAUR

- **fbgetty** — 类似于 mingetty，支持帧缓冲。

: http://projects.meuh.org/fbgetty/\[失效链接 2022-09-18 ⓘ\] || fbgettyAUR

- **mgetty** — 在 Unix 下处理调制解调器各个方面功能的程序。

: http://mgetty.greenie.net/ || mgettyAUR

添加额外的虚拟控制台

打开 `/etc/systemd/logind.conf` 文件并将 **NAutoVTs=6** 设置为你想要在启动时得到的虚拟控制台数量。

如果你想临时获取一个控制台，可以为所需的 TTY 启动一个 getty 服务，执行：

```
$ systemctl start getty@ttyN.service
```

自动登录到虚拟控制台

配置自动登录要使用 systemd 的附加代码片段 (drop-in snippet) 功能来重写传递给 *agetty* 的默认参数。

虚拟控制台和串口控制台的配置是不同的。大多数情况下，你应该是想在虚拟控制台下设置自动登录（这种控制台的设备名称为 `tty*N*`，其中 `*N*` 是一个数字）。串口控制台的自动登录配置稍有不同，它们的设备名称类似于 `ttyS*N*`，其中 `*N*` 是一个数字。

虚拟控制台

注意：根据用户报告这种方法可能会影响系统休眠。

要修改现存单元文件，可以手动创建下列附加文件，或执行 `systemctl edit getty@tty1` 并输入附加代码片段 (drop-in snippet) 的内容：

```
/etc/systemd/system/getty@tty1.service.d/override.conf
[Service]
ExecStart=
ExecStart=-/usr/bin/agetty --autologin username --noclear %I $TERM
```

提示：默认 `getty@.service` 中的 `Type=idle` 选项将会推迟该服务的启动时间，直到所有任务（该单元的前置任务）已经完成，防止启动信息淹没了登录提示符。当 自动启动 X 时，可以通过添加 `Type=simple` 到 附加代码片段 (drop-in snippet) 来立即启动 `getty@tty1.service`，因为此时 init 进程和 *startx* 都被 屏蔽 了输出，避免残留启动时的信息。

如果你想用 *tty* 而不是 *tty1*，请参阅 Systemd 常见问题。

串口控制台

创建以下文件（及目录）：

```
/etc/systemd/system/serial-getty@ttyS0.service.d/autologin.conf
[Service]
ExecStart=
ExecStart=-/usr/bin/agetty --autologin username -s %I 115200,38400,9600 vt102
```

Nspawn 控制台

要为 systemd-nspawn 容器配置自动登录，需要重写 *console-getty* 服务：

```
/etc/systemd/system/console-getty.service.d/override.conf
[Service]
ExecStart=
ExecStart=-/sbin/agetty --noclear --autologin username --keep-baud console
115200,38400,9600 $TERM
```

将引导消息保留在 tty1 上

默认情况下，Arch 会启动 `getty@tty1` 服务。该服务单元文件已经写入了 `--noclear` 参数，它可以阻止 agetty 清空屏幕。但是 systemd 会在启动该服务之前清空屏幕。要关闭这项特性，请创建 `/etc/systemd/system/getty@tty1.service.d/noclear.conf` 文件：

```
/etc/systemd/system/getty@tty1.service.d/noclear.conf
[Service]
TTYVTDisallocate=no
```

这将仅改写 TTY1 上的 *agetty* 的 `TTYVTDisallocate` 参数，并保持全局服务文件 `/usr/lib/systemd/system/getty@.service` 不变。可参阅 Systemd#修改现存单元文件。

注意：

- 确保从 内核参数 中移除了 `quiet`。
- KMS 晚启动可能会造成部分早期启动信息丢失。请参阅 Kernel mode setting#KMS 早启动 或 Kernel mode setting#禁用 KMS。

参考资料

- Systemd#更改开机默认启动目标
- The TTY demystified
- Wikipedia:Tty (unix))

9.3 Linux 控制台/键盘配置

难度 进阶 | **篇幅** 10.4 KB | **所属** Linux 控制台

本节将讲到

- 查看键盘设置
- 键盘映射
- 修改按键延时和重复频率

用于 Linux 控制台的键盘映射（keymaps）、控制台字体和控制台映射由 kbd包 包（systemd 的依赖）提供，它还提供了许多管理文本控制台的低级工具。此外，*systemd* 还提供了 *localectl* 工具，可以控制控制台和 Xorg 的系统区域和键盘布局设置。

查看键盘设置

使用 `localectl status` 查看当前的键盘配置。

键盘映射

键盘映射文件存储在 `/usr/share/kbd/keymaps/` 目录树中。通常一个键盘映射文件对应一个键盘布局（`include` 语句可用于共享通用的部分，一个键盘映射文件可以包含多个布局，其中一些按键组合用于切换）。更多细节请参见 keymaps(5)。

列出键盘映射

控制台键盘映射的命名约定有些随意，但它们通常基于：

- 语言代码：语言代码与国家代码相同（例如 `de` 表示德语，`fr` 表示法语）。

- 国家代码：不同国家使用相同语言的不同变体（例如，`uk` 用于英国英语， `us` 用于美国英语）；国家代码列表也可以在 wikipedia:ISO 3166-1#Officially assigned code elements 中找到。
- 键盘布局：布局与国家或语言无关（例如 `dvorak` 用于 Dvorak 键盘布局）。

使用以下命令查看可用键盘映射的列表：

```
$ localectl list-keymaps
```

要搜索键盘映射，请使用以下命令，将 `*search_term*` 替换为您的语言、国家或布局代码：

```
$ localectl list-keymaps | grep -i search_term
```

或者，使用 find：

```
$ find /usr/share/kbd/keymaps/ -type f -name "*search_term*"
```

Loadkeys

可以只为当前会话设置一个键盘映射。这在测试不同的键盘映射、解决问题等的时候非常有用。

loadkeys 工具用于此目的，它由 systemd 在加载 `/etc/vconsole.conf` 中配置的键盘映射时在内部使用。用法很简单：

```
# loadkeys 键盘映射
```

详情请参阅 loadkeys(1)。

持久化设置

持久键盘映射可以在`/etc/vconsole.conf`中设置，systemd 可以在启动时加载。`KEYMAP` 变量来明确定义键盘映射。如果此变量是空的或没有设置， `us`键盘映射作为默认值。参见 vconsole.conf(5) 详细设置。 例如:

```
/etc/vconsole.conf
KEYMAP=uk
...
```

localectl 可以方便设置终端键盘映射。既可以改变在`/etc/vconsole.conf`里 `KEYMAP` 变量值也可以为当前会话设置临时值。

```
# localectl set-keymap --no-convert keymap
```

`--no-convert` 选项可以阻止 `localectl` 自动的安装最近似的适配改变 `Xorg keymap` （不会影响xwindows下键盘布局）。参见 `localectl(1)` 获得更多细节。

如果有需求，可以在早期用户空间时，使用 `keymap` 钩子从 `/etc/vconsole.conf` 设置中读取键盘映射 mkinitcpio hook。

提示：如果将自定义的键盘映射保存在 `/usr/share/kbd/keymaps/` 以外的目录中，在设置`vconsole.conf`或使用上面提到的 *localectl*命令时使用完整的绝对路径。

创建自定义键盘映射

当使用终端时,你可以使用热键输出特殊字符。然而我们还可以输出一系列字符和一些转义字符。如果我们输出一系列字符组成命令，之后转义字符换一个新行，这个命令将被执行。

编辑键盘映射文件可以实现上述方法。但是，由于更新系统原因键盘映射文件会被重写，不鼓励编辑上述键盘映射文件。一个更好的解决方案是整合现存的键盘映射文件和个人设置的键盘映射文件，`loadkeys` 命令工具可以做到这一点。

首先，创建一个键盘映射文件，这个键盘映射文件可以存放在任何地方，但是推荐的方法是在`/usr/local`: create里面模拟目录结构：创建/usr`/usr/local/share/kbd/keymaps`目录，编辑 edit `/usr/local/share/kbd/keymaps/personal.map`文件。

PS：值得注意的是，这样的个人键盘映射文件对于重新定义已经是默认的键盘映射的按键行为也非常有用:当使用`loadkeys` 指令时，默认的键盘映射和新的指令相冲突时，默认键盘映射被取代，反之则保留。这种方法仅仅会改变在个人键盘映射里面必须被改变的键盘映射。

提示：

- 你也可以编辑一个现存的键盘映射，它在`/usr/share/kbd/keymaps/` 目录里面。键盘映射文件的扩展名是.map.gz，比如 *us.map.gz* 是美国键盘映射。复制这个键盘映射到 `/usr/local/share/kbd/keymaps/personal.map.gz` 并且使用gunzip解压缩之。
- 通过设置 `/etc/vconsole.conf` 文件自定义的个人键盘映射可以持久化。鉴于此，如果你在mkinitcpio hook中使用的`sd-vconsole` 来代替 `keymap`，你应该把你的自定义键盘映射文件放入`/usr/share/kbd/keymaps/`中。这种方法它的来自 `/usr/share/kbd/keymaps`的依赖关系会被钩子自动的加载到初始内存镜像中去。另一方面，如果你把自定义的键盘映射放在了 `/usr/local/` 下，它的依赖需要在 `mkinitcpio.conf` 文件中 `FILES` 行手工明确加载。

增加映射命令

个人键盘映射包括2种指令。首先，keycode 命令，这和默认键盘映射看到的格式相匹配。这指令把 keycode 和 keysym 联系在一起。keysyms 代表键盘的动作。这个可用的动作包括：输出字符码或转义字符，切换终端和键盘映射，启动机器等许多其他动作。全部的当前活动的键盘映射可以从以下命令获得:

```
# dumpkeys -l
```

大部分的键盘动作都很直观。例如，设置 112键输出“e”，这个命令是：

```
keycode 112  = e
```

设置112输出 euro符号，这个命令是：

```
keycode 112 = euro
```

一些键盘动作不能立即和某个键盘按键联系起来。尤其是键盘动作前缀是大写的F后面是1-3位数字，F1-F246构成大于30的总是空白的。定义一个热键输出一系列字符或其他动作是非常有用：

```
keycode 112 = F70
```

此时F70可以绑定输出特定的字符串：

```
string F70 = "Hello"
```

当112键被按下时，会输出F70的内容。为了在终端执行可以输出的命令，换行符必须附加到命令字符串的后面。例如：输出系统休眠命令enter a system into hibernation ，加入下面的键盘映射。

```
string F70 = "sudo systemctl hibernate\n"
```

其他例子

- 对于 emacs 用右侧的 Alt 行使和左侧 Alt 同样的功能，用下面的命令在你的键盘映射里。它将会包含 `/usr/share/kbd/keymaps/i386/include/linux-with-two-alt-keys.inc` ，可以查看它的实现细节。

```
include "linux-with-two-alt-keys"
```

- 切换 Caps_lock 和 Escape 键（为vim），重新映射相应的 keycodes。

```
keycode 1 = Caps_Lock
keycode 58 = Escape
```

- 切换 Caps_lock 作为额外的 Control 键，重新映射相应的 keycodes。

```
keycode 58 = Control
```

- 互换 Caps_lock 和左侧 Control 键，重新映射相应的 keycodes。

```
keycode 29 = Caps_Lock
keycode 58 = Control
```

储存更改

为了使用个人键盘映射，必须使用 *loadkeys*。

```
# loadkeys /usr/local/share/kbd/keymaps/personal.map
```

然而，这个键盘映射只能在当前会话被激活。为了在启动时使用这个键盘映射，需要在 /etc/vconsole.conf 中设置`KEYMAP` ，并且明确写出个人键盘映射文件绝对路径。这个文件不必想被 kbd包 提供的官方的键盘映射一样使用gunzip压缩。

KEYMAP=/usr/local/share/kbd/keymaps/personal.map

修改按键延时和重复频率

按键延时是只长按一个按键多少时间（通常以毫秒为单位）才会开始重复这个按键。开始重复过程后，字符会以一定频率重复（通常以赫兹单位），也就是**重复频率**。这些值可以通过 *kbdrate* 设置。请注意，这些设置需要为控制台和 Xorg 分别配置。

```
# kbdrate [-d delay] [-r rate]
```

例如，要将延时设置为 200 毫秒，将重复频率设置为 30Hz，请使用以下命令：

```
# kbdrate -d 200 -r 30
```

若不指定延时和重复频率，延时和重复频率将被重置为各自的默认值：延时 250ms，重复频率 11Hz：

```
# kbdrate
```

Systemd 服务

可以使用 systemd 服务设置重复频率。例如：

```
/etc/systemd/system/kbdrate.service
[Unit]
Description=Keyboard repeat rate in tty.

[Service]
Type=oneshot
RemainAfterExit=yes
StandardInput=tty
StandardOutput=tty
ExecStart=/usr/bin/kbdrate --silent --delay 250 --rate 30

[Install]
WantedBy=multi-user.target
```

然后启动/启用 `kbdrate.service` systemd 服务。

9.4 Linux 控制台

难度 进阶 | **篇幅** 15.1 KB | **所属** Linux 控制台

本节将讲到

- 实现
- 键盘快捷键
- 字体
- 光标外观
- HiDPI

根据维基百科：

: **Linux 控制台**是 Linux 内核内部的一个系统控制台。Linux 控制台为内核和其他进程提供了一种向用户发送文本输出并接收用户文本输入的方式。用户通常用计算机键盘输入文本，并在计算机显示器上阅读输出的文本。Linux 内核支持虚拟控制台——在逻辑上独立的控制台，但它们访问相同的物理键盘和显示器。

本文介绍了 Linux 控制台的基础知识以及如何配置字体显示。/Keyboard configuration 子页面中描述了键盘配置。对于提供更多功能的替代控制台解决方案（完整的 Unicode 字体、现代图形适配器等），请参阅 KMSCON 或类似项目。

实现

与大多数直接与用户交互的服务不同，控制台是在内核中实现的。这与终端仿真软件形成鲜明对比，如 Xterm，它是一个在用户空间实现的普通的应用程序。控制台一直是发布的 Linux 内核的一部分，但在其历史上经历了一些变化，最明显的是过渡到使用 framebuffer 以及对 Unicode 的支持。

虽然对控制台进行了许多改进，但它对旧硬件完全的向后兼容性意味着与图形终端仿真器相比它是受限的。Linux 控制台与图形终端仿真器的主要区别在于：Linux 控制台中的 shell 直接连接到 TTY 设备（`/dev/tty*`），而图形终端仿真器中的 shell 则连接到伪 TTY（`/dev/pty*`）。

此外，图形终端仿真器可以拥有比 Linux 控制台更多的功能，包括更丰富的可用字体集、多标签页/窗口、分屏视图、回滚缓冲区/滑块、背景颜色/图像（可选透明）等。其中一些功能可以通过终端多路复用器（如 Tmux 或 GNU Screen）在 Linux 控制台中使用，或通过依赖 ncurses 等库的文本用户界面程序（TUI）实现，例如 Vim、nano 或 Emacs。这些功能也可在图形终端仿真器中使用。

虚拟控制台

控制台以一系列虚拟控制台的形式呈现给用户。这些虚拟控制台给人的印象是几个独立的终端在同时运行；每个虚拟控制台可以用不同的用户登录，运行自己的 shell，并有自己的字体设置。每个虚拟控制台都使用一个 `/dev/ttyX` 设备，可以通过按 `Alt+F*x*`（其中 `*x*` 是虚拟控制台编号，从 1 开始）在它们之间切换。`/dev/console` 设备会被自动映射到活动的虚拟控制台。

另请参阅 chvt(1)、openvt(1) 和 deallocvt(1)。

文本模式

由于 Linux 最初是适用于 PC 硬件的内核，所以控制台是使用标准 IBM CGA/EGA/VGA图形开发的，当时所有 PC 都支持这种图形。图形在 VGA 文本模式下运行，它提供了一个简单的 16 色 80x25 字符显示。这种传统模式类似于专用文本终端的功能，例如 DEC VT100 系列。如果系统硬件支持的话，仍可以通过内核参数（`vga=0 nomodeset`）以文本模式启动，但几乎所有的现代发行版（包括 Arch Linux）都使用 framebuffer 控制台来代替。

Framebuffer 控制台

随着 Linux 被移植到其他非 PC 架构上，需要一个更好的解决方案，因为其他架构不使用兼容 VGA 的图形适配器，而且可能根本不支持文本模式。framebuffer 控制台的

实现是为了在所有平台上提供一个标准的控制台，因此无论底层的图形硬件如何，都会呈现相同的 VGA 风格界面。因此，Linux 控制台不是一个终端模拟器，它本身就是一个终端。它使用终端类型 `linux`，并与 VT100 基本兼容。

键盘快捷键

键盘快捷键	描述
`Ctrl+Alt+Del`	重新启动系统（由符号链接 `/usr/lib/systemd/system/ctrl-alt-del.target` 指定）
`Alt+F1、F2、F3.....`	切换到第 *n* 个虚拟控制台（不同于 Xorg 和 Wayland 中使用的 `Ctrl+Alt+F*n*`）
`Alt+左箭头`	切换到上一个虚拟控制台
`Alt+右箭头`	切换到下一个虚拟控制台
`Scroll Lock`	当 Scroll Lock 被激活时，输入/输出被锁定
`Ctrl+c`	杀死当前任务
`Ctrl+d`	插入 EOF
`Ctrl+z`	暂停当前任务

参见 console_codes(4)。

字体

Linux 控制台默认使用 UTF-8 编码，但由于使用的是标准的兼容 VGA 的 framebuffer，终端字体限定为 256 或 512 个字形。如果字体超出了 256 个字形，那么颜色的数量就会从 16 减少到 8。为了为给定的 Unicode 值分配正确的符号来显示，需要一个特殊的翻译映射，通常称为 *unimap*。现在，大多数控制台字体都内置了 "unimap"；在历史上，它必须被单独加载。

默认情况下，虚拟控制台使用内核的内置字体，其包含 CP437 字符集，但这个设置很容易改变。内核提供了约 15 种内置字体可供选择，其中官方支持的内核提供两种：VGA 8x16 字体（`CONFIG_FONT_8x16`）和 Terminus 16x32 字体（`CONFIG_FONT_TER16x32`）。内核根据屏幕分辨率的评估结果选择使用的字体。可以通过内核参数强制使用其他内置字体，例如 `fbcon=font:TER16x32`。

kbd包 包提供改变虚拟控制台字体和字体映射的工具。可用的字体存储在 `/usr/share/kbd/consolefonts/` 目录中，以 *.psfu* 或 *.psfu.gz* 结尾的字体内置 Unicode 翻译映射。

键盘映射是按键和计算机使用的字符的对应关系，位于 `/usr/share/kbd/keymaps/` 的子目录中，详情请参阅 /Keyboard configuration。

注意：替换字体可能会导致需要标准 VGA 样式字体的程序出现问题，例如使用线条绘图图形的程序。

提示：对于用拉丁/希腊字母书写的欧洲语言，可以使用 `eurlatgr` 字体，它包括广泛的拉丁/希腊字母变化以及特殊字符。

预览和测试

提示：这里有一个整理过的预览影像数据库：Linux 控制台字体截图。

```
$ showconsolefont
```

显示字体可用字形与字符的列表。

如果指定（`/usr/share/kbd/consolefonts/` 中的）字体名称，`setfont` 将暂时改变字体：

```
$ setfont lat2-16 -m 8859-2
```

字体名称区分大小写。如果不加参数，`setfont` 会将控制台字体重设为默认字体。

因此，要使用一个**小的 8x8** 字体，在安装字体后，使用类似如下所示的命令：

```
$ setfont -h8 /usr/share/kbd/consolefonts/drdos8x8.psfu.gz
```

如果要使用**更大**的字体，Terminus 字体 (terminus-font包) 有多种尺寸可供选择，例如 `ter-132b` 就很大。

你也可以添加 `-d` 选项来使用双倍大小。这将使用 64x64 字体：

```
$ setfont -d ter-132n
```

提示：

- 所有更改字体的命令都可以“盲打”。
- Terminus 字体名称（`ter-*`）的解释见 `/usr/share/terminus-font/README`。

注意：*setfont* 只作用于当前正在使用的控制台。其它终端无论活跃与否都不受影响。

持久性配置

`/etc/vconsole.conf` 中的 `FONT` 变量可用于在启动时设置字体，这对于所有控制台都具有持久性作用。详情请参见 vconsole.conf(5)。

要显示*Č, ž, đ, š*或*Ł, ę, ą, ś*之类的字符，请使用 `lat2-16.psfu.gz` 字体：

```
/etc/vconsole.conf
...
FONT=lat2-16
FONT_MAP=8859-2
```

这代表使用 ISO/IEC 8859 字符的第二部分，尺寸设置为 16。可以使用其它值来更改字体尺寸（如 `lat2-08`）。可以在 zhwp:ISO/IEC 8859#各种ISO 8859字符集查询 8859 规范定义的区域。

自 mkinitcpio v33 起，`/etc/vconsole.conf` 中指定的字体默认通过 `consolefont` 钩子在早期用户空间自动加载，该钩子将字体添加到 initramfs。详情请参见 Mkinitcpio#钩子(HOOKS))。

更改 `/etc/vconsole.conf` 后，你可能还需要重启 `systemd-vconsole-setup.service`。

如果启动时字体没有变化，或只暂时变化，则最可能是因为初始化图形驱动时字体被复位，然后控制台切换到 framebuffer。默认情况下，所有树内内核驱动程序都会提前加载。NVIDIA 用户应参阅 NVIDIA#Early loading 以在应用 `/etc/vconsole.conf` 之前加载图形驱动程序。

光标外观

此主题文档较少，建议阅读以下文章：

- Software cursor for VGA（VGA 的软件光标）
- Cursor Appearance in the Linux Console（Linux 控制台中的光标外观）
- Disable Cursor Blinking on Linux Console（在 Linux 控制台上禁用光标闪烁）

<table class="wikitable"><caption>光标类型</caption><tbody><tr><th></th><th>硬件</th><th>软件</th></tr><tr><th>形状</th><td><dl><dd>(0) 默认<dl><dd>应用 (2) 下划线</dd></dl></dd><dd>(1) 不可见</dd><dd>(2) 下划线</dd><dd>(3) 下三分之一</dd><dd>(4) 下半部</dd><dd>(5) 下三分之二</dd><dd>(6) 全方块</dd></dl></td><td><dl><dd>(16) 全方块<dl><dd>形状无法更改，但当背景色和前景色相同时，光标实际上不可见</dd></dl></dd></dl></td></tr><tr><th>闪烁</th><td><dl><dd>闪烁</dd></dl></td><td><dl><dd>取决于驱动程序<dl><dd>背景色的最高位可解释为"高亮"或"闪烁"</dd></dl></dd></dl></td></tr><tr><th>颜色</th><td><dl><dd>与控制台文本颜色相同（通常为白色/灰色）<dl><dd>无法单独设置</dd></dl></dd></dl></td><td><dl><dd>可由用户设置</dd></dl></td></tr></tbody></table>

控制台光标可通过**设备属性（DA）**控制函数调整。参数序列**必须**以单个问号开头（尽管 console_codes(4) 有相反的说法）。

以下是一个全方块、不闪烁、绿色光标（光标下的符号为黑色）的示例：

```
$ printf "\x1b\x5b?16;$((8+4+2+1));$((32+0+8+4+2+1))\x63"
```

也可以使用八进制和字符代替十六进制代码表达：

```
$ printf '\033[?16;15;47c'
```

注意：控制函数（转义序列）的完整指南请参阅标准 ECMA-48 — Control Functions for Coded Character Sets。

相同的配置可通过内核参数永久应用：

```
vt.cur_default=0x2f0f10
```

注意：转义序列参数（**16, 15, 47**）以十六进制（**10, 0f, 2f**）按**相反顺序**写入。

光标大小

第一个参数名为*光标大小*，数字 **16**（内核参数最右侧两位十六进制数字为 **10**）表示"*使用软件光标*"。

注意：你可能需要使用 **48** (32+16)、**80** (64+16) 或 **112** (64+32+16) 代替 **16**。

如果想更改硬件光标形状，使用对应数字（从 **0** 到 **6**，见上表）。

注意：*光标大小*是唯一适用于*硬件*光标的参数。

切换掩码

第二个参数称为*切换掩码*，用于翻转颜色的对应位。

<table class="wikitable" style="text-align: center;"><caption>字符属性位
</caption><tbody><tr><th></th><th colspan="4">背景 （光标块）
</th><th colspan="4">前景 （光标下的符号）
</th></tr><tr><th></th><td>高亮 或闪烁 </td><td>红
</td><td>绿</td><td>蓝</td><td>高亮 或闪烁
</td><td>红</td><td>绿</td><td>蓝</td></tr><tr><th>
十进制
</th><td>128</td><td>64</td><td>32</td><td>16</td><td>8</td><td>4</td><td>2</
td><td>1</td></tr><tr><th>十六进制
</th><td>80</td><td>40</td><td>20</td><td>10</td><td>8</td><td>4</td><td>2</t
d><td>1</td></tr></tbody></table>

在我们的例子中，第二个参数是 **15**（内核参数中间两位十六进制数字为 **0f**），因此所有四个前景（符号）位都将被翻转。最重要的规则是：切换（第二个参数）在设置（第三个参数）之后应用。

注意：如果某个位被第三个参数设置且被第二个参数切换——它实际上会被清除。

设置掩码

第三个参数称为*设置掩码*。它设置相应的字符属性位。我们在示例中使用 **47**（内核参数最左侧两位十六进制数字为 **2f**），这表示两点：

- (32) 为光标块使用纯绿色
- (8+4+2+1) 设置所有四个前景（符号颜色）位。这些位将被第二个参数切换，因此光标下的符号颜色将为黑色（0000）。

HiDPI

参见 HiDPI#Linux 控制台。

声音提示

参见 PC 扬声器#beep。

参见

- The TTY demystified – Linus Åkesson

第 10 章　共 4 节　24 KB

框架

本章处理在 Arch 上安装、跑通并排查自带运行时或包源的框架：Faction、Mono、Electron 与 .NET。它们装完并不等于可用，难处在用法、配置与故障排除。

建议把 .NET 当必读主线——篇幅最长，安装、遥测、Tab 自动补全与故障排除齐全；Mono 次之，运行应用程序与问题解决两节最实用。Electron 仅在需要桌面端方案时细看，提示与技巧、故障排除比安装更值；Faction 最短，可作速查跳过。最常见的误解是把 Mono 与 .NET 当成同一套东西，或忽略遥测、Tab 自动补全这类默认行为。

10.1　Faction

难度 入门 | **篇幅** 3.2 KB | **所属** 框架

本节将讲到

- 安装
- 用法
- 示例
- 模式
- 参见

Faction 是一个用于测试驱动软件开发的 C 语言库。

安装

安装 libfactionAUR 软件包。

用法

该库提供了几个 C 宏，使编写测试变得更快。

- FI 表示 Faction Initialization
- FT 表示 Faction Test
- FC 表示 Faction Close

使用 FT 宏时，需要填写三个字段。

- AUTHORS() takes a comma-separated list of double-quotation surrounded author names
- SPEC() takes a single double-quotation surrounded test specification description
- A C boolean expression (just like when using C assert macros)

按照惯例，Faction Test 应写在包含测试代码的源文件的底部。Test 应由 FACTION 宏保护（见下面的示例）包围，以便在编译时启用或禁用。C 编译器（如 GNU C 编译器 (GCC)）m提供了一种在命令行上启用宏的方法（即 `-D` 标志）。

示例

```
/* 这就是要测试的函数 */
int
increment(int input)
{
   return (input + 1);
}

#ifdef FACTION
#include <faction.h>
#include <limits.h>
FI

  FT(
    AUTHORS( "timetoplatypus" ),
    SPEC( "increment() returns 1 when given 0" ),
    increment(0) == 1
  );

  FT(
    AUTHORS( "timetoplatypus" ),
    SPEC( "increment() returns 0 when given the largest integer value" ),
    increment(INT_MAX) == 0
  );

FC
#endif
```

可以使用 `gcc *filename.c* -D FACTION` 对其进行编译。

模式

Faction 有两种编译模式：*最小*模式和*扩展*模式。

上述示例以最小模式编译 Faction。最小化编译只依赖三个库：stdlib、stdio 和 getopt。扩展编译则需要额外的依赖库，包括一些只能通过 GNU 功能测试宏使用的函数。

因此，要在扩展模式下编译，只需在文件顶部定义 GNU 功能测试宏即可。例如，将前面的示例修改为在扩展模式下编译，就会变成这样：

```
#ifdef FACTION
#define _GNU_SOURCE
#endif
/* 这就是要测试的函数 */
increment(int input)
{
  return (input + 1);
}

#ifdef FACTION
#include <faction.h>
#include <limits.h>
FI
```

```
  FT(
    AUTHORS( "timetoplatypus" ),
    SPEC( "increment() returns 1 when given 0" ),
    increment(0) == 1
  );

  FT(
    AUTHORS( "timetoplatypus" ),
    SPEC( "increment() returns 0 when given the largest integer value" ),
    increment(INT_MAX) == 0
  );

FC
#endif
```

扩展模式功能

在扩展模式下，

- 可以在运行时使用 `-l` 标记将输出镜像到用户指定的日志文件。
- 结果表将根据所使用终端的宽度动态调整大小。否则，默认宽度为 78 个字符。

参见

- Faction 主页

10.2 Mono

难度 入门 | **篇幅** 4.7 KB | **所属** 框架

本节将讲到

- 安装
- 运行 Mono 应用程序
- 测试 Mono
- 开发
- 问题解决

摘自 Wikipedia)：

: Mono is a \[...\] project to create a .NET Framework\-compatible set of tools including, among others, a C#) compiler and a Common Language Runtime.

安装

安装 mono包 软件包。

如果需要 VisualBasic.Net 支持，则必须使用 mono-basicAUR 安装 VisualBasic.Net 解释器。

注意：安装软件包后，证书颁发机构 会存储在 `/usr/share/.mono/certs/Trust/` 中，但删除软件包并不会删除它们。

运行 Mono 应用程序

您可以手动调用 `mono` 来执行 Mono 二进制文件：

```
$ mono programsname.exe
```

您也可以直接执行 Mono 二进制文件，就像本地二进制文件一样：

```
$ chmod 755 exefile.exe
$ ./exefile.exe
```

测试 Mono

新建一个文件：

```
test.cs
using System;

public class Test {
 public static void Main(string[] args) {
  Console.WriteLine("Hello World!");
 }
}
```

然后运行：

```
$ mcs test.cs
$ mono test.exe
Hello world!
```

开发

OmniSharp 为多个编辑器提供 .NET/Mono 开发插件/集成，包括 Vim、Emacs 和 Visual Studio Code。

或者，您也可以安装 riderAUR 集成开发环境。如果不是从 AUR 安装 Rider，则需要安装 mono-msbuild包，因为最近的 Rider 版本放弃了对 xbuild 的支持，转而使用来自 net-core 的 MSBuild。

如果需要 API 文档浏览器以及一些测试和开发工具，则必须安装 mono-tools包。

问题解决

当我尝试直接运行 Mono 二进制文件时，我遇到了一个错误：“cannot execute binary file”

Mono的 binfmt_misc 处理程序尚未建立，Mono Project website 上有详细说明。

要解决这个问题，restart `systemd-binfmt` 服务。

我收到一个 TLS 握手（或类似的基于证书的）错误

造成这种情况的原因可能是 Mono 证书存储区中的证书丢失，或者 Mono 证书存储区中仍然存在过期的破损证书。

如果可能，请运行 `curl -vI` 或类似程序，在 Mono 外部复制失败请求，以确保系统证书存储处于良好状态。

- `cert-sync /etc/ssl/certs/ca-certificates.crt` 将 mono 存储与系统存储同步，并添加缺失的证书。
- 要删除已损坏的证书（即，如果上述操作无效），请删除 `/usr/share/.mono` 目录，然后重新运行 `cert-sync /etc/ssl/certs/ca-certificates.crt` 。
- 最后，旧版工具 `mozroots --import --ask-remove` 可绕过系统证书存储区，直接下载 Mozilla 的信任存储区。这同样无法删除损坏的证书，而且如果依赖于私有 CA，还可能导致其他问题。

`cert-sync`和 `mozroots` 都是 mono 软件包的一部分。

参见

- Mono 官方网站
- Mono 手册
- Mono 的应用程序接口参考
- ECMA-334: C# Language Specification
- ECMA-335: Common Language Infrastructure (CLI)
- 运行 Mono 应用程序的说明

10.3 Electron

难度 入门 | **篇幅** 5.7 KB | **所属** 框架

本节将讲到

- 安装
- 提示与技巧
- 故障排除

Electron 是由 GitHub 开发的应用程序框架，用于使用 Web 技术构建跨平台桌面应用程序。它使用 Chromium 浏览器引擎进行渲染，并使用 Node.js 运行时环境作为后端。

安装

安装 electron包 软件包以获取最新版本。

某些应用程序需要较旧版本的 Electron。你可以在安装最新版本的同时并行安装旧版本。对应的软件包名称会附带版本号后缀，例如 electron30AUR。

提示与技巧

配置文件

`electron` 命令会从 `$XDG_CONFIG_HOME/electron*XX*-flags.conf` 文件中读取命令行参数，其中 *XX* 为 Electron 的版本号。如果该文件不存在，则会回退到 `$XDG_CONFIG_HOME/electron-flags.conf`。（如果未设置 `[$XDG_CONFIG_HOME](../../01_安装与包管理/配置文件/XDG%20基本目录.md#用户目录)`，则默认为 `~/.config`。）

注意：

- 这些配置文件仅会被 Arch Linux 提供的 electron包 读取；对于自行捆绑 Electron 构建版本的软件包（例如 slack-desktopAUR），这些配置文件不起作用。有时可以使用替代软件包，例如 slack-electronAUR。
- 某些应用程序（例如 vesktopAUR）可能支持设置应用程序专属的命令行参数。对于此类应用程序，可以在 `~/.config/*application_name*-flags.conf` 创建专属的参数配置文件，其中 `*application_name*` 是需要设置参数的应用程序名称。例如，要为 spotifyAUR 设置参数，可以将其添加到 `~/.config/spotify-flags.conf` 中。

以井号 `#` 开头的行会被视为注释并被忽略，而其他行则会作为命令行参数传递给实际的 Electron 可执行文件，每行对应一个参数（不会根据空白字符拆分行！）。这些参数会在传递给 `electron` 命令本身的选项之前添加。

例如，这可以用于全局启用 Wayland。

示例：

```
~/.config/electron-flags.conf
--enable-features=WaylandWindowDecorations,AllowQt
# --ozone-platform-hint=wayland (已在 Electron 38 中移除)
--ozone-platform=wayland
# GTK4 在 Wayland 上表现更好。
--gtk-version=4
```

确定应用程序使用的 Electron 版本

如果应用程序是通过软件包管理器安装的，可以查询软件包的依赖关系，查看它是否依赖于 electron38AUR 等软件包。如果是，则说明它使用 Electron 38。（如果它依赖于 electron包：该软件包始终提供最新版本的 Electron；要了解当前的具体版本，可以查询该软件包的依赖关系。）

也可以通过运行中应用程序的开发者工具确定其使用的 Electron 版本；参见 。

Secret Service API

Electron 提供了 safeStorage API，用于与兼容 FreeDesktop.org Secret Service API 的密钥环进行交互。受支持的密钥环包括通过 GNOME libsecret 提供支持的 GNOME/Keyring、KDE Wallet 和 KeePass。

运行 Electron 应用程序时，可以通过 `--password-store` 命令行参数选择后端。例如，要让基于 Electron 构建的 `electron-desktop` 通过 `libsecret` 或 KeePass 进行交互，可以运行：

```
$ electron-desktop --password-store="gnome-libsecret"
```

将任意网站转换为应用程序

有时你可能需要将某个特定网站作为独立应用程序使用。通过 webapp-manager包，可以将任意网站转换为 Electron 应用程序。

警告：nativefier 已被弃用，不再推荐使用这种方式！参见 issue #1577。

启用 Wayland

参见 Wayland#Electron。

强制使用 X11

要强制应用程序使用 X11（XWayland）显示服务器后端，而不是默认使用原生 Wayland，可以添加 `--ozone-platform=X11` 命令行参数。

故障排除

由于 Electron 基于 Chromium，因此 Chromium#疑难解答 中的内容可能有所帮助。

10.4 .NET

难度 进阶 | **篇幅** 10.4 KB | **所属** 框架

本节将讲到

- 安装
- 遥测
- Tab 自动补全
- 故障排除
- 参阅

.NET（旧称 .NET Core）是 Microsoft 开发的开源软件框架，支持 C#, Visual Basic 和 F#。和之前的 .NET Framework 相比，它支持跨平台开发，设计上更加模块化，面向现代程序开发。

.NET 源码位于存放在 Github 上的 dotnet/dotnet。

安装

如果要运行 .NET 管理的程序，请安装 dotnet-runtime包。

要使用 .NET 编译程序，还需要安装 dotnet-sdk包。

要使用 ASP.NET Core 建立动态网站，应用和服务，安装 aspnet-runtime包.

微软推荐使用 Visual Studio Code 编译和调试 .NET 程序，它是微软开发的基于 Electron 的开源 IDE。

提示：将 `~/.dotnet/tools` 加入 `PATH`，否则 dotnet 工具无法在 shell 中启动。

如果要使用 .NET 6.0，将上述包加上“-6.0”后缀即可（例如 dotnet-runtime-6.0包, dotnet-sdk-6.0包 和 aspnet-runtime-6.0包）

版本区别

.NET SDK 有多个版本，只有 `1xx` 版本可从源码构建，并在官方仓库中提供。如果需要其它版本，可以在 AUR 中查找对应的 `*-bin` 包。

手动安装多个版本

你可以通过执行 .NET 官方提供的 dotnet-install.sh 脚本来**同时**安装 .NET SDK 或 .NET 运行时的多个版本。另外，你可以在此链接中找到关于这个脚本的说明。

例如，该命令将安装“STS”（标准期限支持）频道的最新版本到 `/usr/share/dotnet`：

```
# ./dotnet-install.sh --install-dir /usr/share/dotnet -channel STS -version latest
```

你可以先使用 `-Dryrun` 来模拟安装。

脚本执行完毕后，你可以通过以下命令来验证是否安装成功：

```
$ dotnet --list-sdks
2.2.108 [/usr/share/dotnet/sdk]
3.0.103 [/usr/share/dotnet/sdk]
$ dotnet --version
3.0.103
```

手动卸载已安装版本

官方提供的卸载工具 .NET Uninstall Tool 尚无 Linux 支持，如果你想卸载以前通过 `dotnet-install.sh` 脚本安装的过时版本，需要手动操作。

```
$ dotnet --list-sdks
5.0.100 [/usr/share/dotnet/sdk]
5.0.102 [/usr/share/dotnet/sdk]
```

解除特定版本的安装：

```
$ SDK_VERSION="5.0.100"
$ DOTNET_UNINSTALL_PATH="/usr/share/dotnet"
# rm -rf $DOTNET_UNINSTALL_PATH/sdk/$SDK_VERSION
```

`dotnet-install.sh` 同时会安装 dotnet host 及共享包，根据版本不同可能需要单独进行移除。

你可能会安装了一些额外的依赖，通过下面的命令移除他们：

```
$ SDK_VERSION="5.0.100"
$ DOTNET_VERSION="5.0.0"
$ DOTNET_UNINSTALL_PATH="/usr/share/dotnet"
# rm -rf $DOTNET_UNINSTALL_PATH/sdk/$SDK_VERSION
# rm -rf $DOTNET_UNINSTALL_PATH/shared/Microsoft.NETCore.App/$DOTNET_VERSION
# rm -rf $DOTNET_UNINSTALL_PATH/shared/Microsoft.AspNetCore.All/$DOTNET_VERSION
# rm -rf $DOTNET_UNINSTALL_PATH/shared/Microsoft.AspNetCore.App/$DOTNET_VERSION
# rm -rf $DOTNET_UNINSTALL_PATH/host/fxr/$DOTNET_VERSION
```

通过 AUR 安装多个版本

有些 AUR dotnet 包可以被同时安装。只需要安装 (dotnet-host-binAUR 或 dotnet-host包) 获取命令行工具，就可以安装任意大版本的最新 SDK 和运行时。以下为兼容的包：

- dotnet-host-binAUR, dotnet-runtime-binAUR, aspnet-runtime-binAUR, dotnet-sdk-binAUR
- dotnet-host-preview-binAUR, dotnet-runtime-preview-binAUR, aspnet-runtime-preview-binAUR, dotnet-sdk-preview-binAUR
- dotnet-runtime-7.0-binAUR, aspnet-runtime-7.0-binAUR, dotnet-sdk-7.0-binAUR,
- dotnet-runtime-3.0AUR, aspnet-runtime-3.0AUR, dotnet-sdk-3.0AUR,
- dotnet-runtime-2.2AUR, aspnet-runtime-2.2AUR, dotnet-sdk-2.2AUR, dotnet-sdk-2.2-vs2017AUR
- dotnet-runtime-2.1AUR, aspnet-runtime-2.1AUR, dotnet-sdk-2.1AUR

安装 PowerShell Core

你可以将 PowerShell Core 安装为“全局”工具

```
# dotnet tool install --global PowerShell
```

按如下方法将其升级到最新版本

```
# dotnet tool update --global PowerShell
```

另请参考 PowerShell。

遥测

Microsoft 构建的 .NET SDK 默认启用遥测。AUR .NET SDK 包（`*-bin` 变体）基于 Microsoft 构建的 .NET。.NET 运行时组件在任何情况下都不收集遥测信息。

社区构建版本（包括 Arch；从 .NET 7 开始）不收集遥测数据，这是由 Red Hat 对 .NET SDK 做出的更改。

可以通过设置环境变量 `DOTNET_CLI_TELEMETRY_OPTOUT=1` 来关闭遥测。

Tab 自动补全

所有使用 `System.CommandLine.Parser` 解析命令行选项的 dotnet 程序都支持自动补全。要启用该功能，需按照文档在 `.bashrc` / `.zshrc` 中添加数行内容。 对于独立的二进制文件，需参考这里。

故障排除

It was not possible to find any compatible framework version

如果在运行新创建的项目时出现了如下报错，不需要按各种 GitHub issue 中的描述设置 `DOTNET_ROOT` 变量，Arch 提供的 dotnet 包（从 3.1 版本开始）会安装到微软建议的 `/usr/share/dotnet` 位置。

```
$ dotnet run
It was not possible to find any compatible framework version
The framework 'Microsoft.AspNetCore.App', version '3.1.0' was not found.
  - No frameworks were found.

You can resolve the problem by installing the specified framework and/or SDK.

The specified framework can be found at:
  - https://aka.ms/dotnet-core-applaunch?
framework=Microsoft.AspNetCore.App&framework_version=3.1.0&arch=x64&rid=arch-x64
```

该问题的产生原因是 Arch 将运行时拆分到了单独的软件包，你需要确保同时安装了 aspnet-runtime包。

"the required library libhostfxr.so could not be found"

有些 dotnet SDK 工具（如 *libman*、*dotnet-watch* 等）可能会要求配置 `DOTNET_ROOT` 环境变量。如果没有配置该变量，就会出现如下报错：

```
A fatal error occurred, the required library libhostfxr.so could not be found.
If this is a self-contained application, that library should exist in
[/home/my_user/.dotnet/tools/.store/microsoft.web.librarymanager.cli/1.0.172/microsoft.
web.librarymanager.cli/1.0.172/tools/netcoreapp2.1/any/].
If this is a framework-dependent application, install the runtime in the default
location [/usr/share/dotnet] or use the DOTNET_ROOT environment variable to specify the
runtime location.
```

解决方案是手动在终端环境导出 `DOTNET_ROOT`：

```
~/.bashrc
export DOTNET_ROOT=/opt/dotnet
```

Error MSB4019: The imported project "/usr/share/dotnet/sdk/.../Sdks/Microsoft.NET.Sdk/targets/Microsoft.NET.Sdk.Common.targets" was not found. Confirm that the expression in the Import declaration ... is correct, and that the file exists on disk.

这是由更新导致的，当前的 shell 或登录会话中环境变量储存的 dotnet SDK 版本与已安装版本不同。重新打开 shell 或重新登录可以解决该问题。

无法找到指定的 SDK

这被认为是 Mono 和 MSBuild SDK 库及 dotnet 库冲突了。要修正该问题，需参考如下方式手动在 shell 中设置路径（按需将示例中的版本号替换为实际安装的版本）：

```
~/.bashrc
export MSBuildSDKsPath=$( echo /usr/share/dotnet/sdk/3.*/Sdks );
```

仍已安装 dotnet 命令

已安装的包不会卸载 dotnet-host，需要手动卸载 dotnet-host包。

参阅

- .NET 主页
- .NET 基金会
- .NET API 浏览器

Lisp 方言

本章解决 Lisp 在 Arch 上真正跑起来的问题：选哪门方言、用哪种实现、运行时怎么装、依赖怎么管、REPL 与开发环境怎么配，而不是讲语言本身。

建议先读 Common Lisp：篇幅最长，安装、依赖管理、开发环境、提示与疑难解答俱全，最值得完整读完。Scheme 讲实现选型、包管理与开发，宜通读；Clojure 只有安装与 REPL 两项，走一遍即可。最常见的误解是把 Lisp 当成"一种语言一个实现"——Scheme 单列实现一节，正说明要先挑实现再谈包管理；包管理与依赖管理也分属两节，经验不能跨方言平移。疑难解答只出现在 Common Lisp 一节，另两门方言遇坑需回看各自的开发部分。

11.1　Clojure

难度 入门 | **篇幅** 1.6 KB | **所属** Lisp 方言

本节将讲到

- 安装
- REPL
- 参见

Clojure 是 Lisp 的方言，它是一种针对 Java 虚拟机，CLR 和 JavaScript 的动态，功能齐全的通用编程语言。

安装

安装 clojure包 包，或安装 clojure-gitAUR 开发版本。

REPL

要运行 REPL，请从 leiningen包 安装 leiningen。然后在终端运行

```
lein repl
```

m2 仓库路径

要配置本地 m2 仓库路径，请修改 profiles.clj：

```
{:user {:local-repo #=(eval (str (System/getenv "XDG_CACHE_HOME") "/m2"))
        :repositories  {"local" {:url #=(eval (str "file://" (System/getenv
"XDG_DATA_HOME") "/m2"))
                                 :releases {:checksum :ignore}}}
        }}
```

参见

- Wikipedia:Clojure

11.2　Scheme

难度 入门 | **篇幅** 4.8 KB | **所属** Lisp 方言

本节将讲到

- 实现
- 包管理
- 开发
- 参见

根据 Scheme.org：

: Scheme 是 Lisp 语言家族中的经典编程语言。它强调函数式编程和领域特定语言（DSL），同时也能够适应其他编程风格。Scheme 以简洁而极简的设计著称，是最长寿且研究最为深入的动态编程语言之一，拥有众多快速且可移植的实现方案。

实现

- **Bigloo** — 快速的 Scheme 编译器

: https://www-sop.inria.fr/mimosa/fp/Bigloo/ || bigloo包

- **Chez** — Chez Scheme 是针对 Revised^6 Report on Scheme (R6RS) 语言的编译器和运行时系统，具有许多扩展。

: https://cisco.github.io/ChezScheme/ || chez-schemeAUR

- **CHICKEN** — 功能丰富的 R5RS/R7RS 编译器和解释器

: https://call-cc.org/ || chicken包

- **Gambit Scheme** — 高效率的 Scheme 实现

: https://gambitscheme.org/ || gambit-cAUR

- **Gauche** — R7RS Scheme 实现（包含 gosh）

: https://practical-scheme.net/gauche/ || gaucheAUR

- **GNU Guile** — 用 C 编写的可移植、可嵌入 Scheme 实现

: https://www.gnu.org/software/guile/ || guile包、guile2.0AUR、guile1.8AUR

- **Kawa** — 针对 JVM 的 Scheme 实现和框架

: https://www.gnu.org/software/kawa/ || kawaAUR

- **Larceny** — Larceny 是 Scheme 编程语言的一种简单有效的实现。

: https://larcenists.org/ || larceny-binAUR

- **MIT/GNU Scheme** — MIT/GNU Scheme

: https://www.gnu.org/software/mit-scheme/ || mit-schemeAUR

- **Racket** — 使用 DrRacket IDE 的执行各种任务的语言。前身为 PLT Scheme。

: https://racket-lang.org/ || racket包、racket-minimal包

- **Scheme48** — 实验性的 Scheme 解释器。

: https://s48.org/ || scheme48AUR

包管理

Akku.scm 是 Scheme 语言的一个包管理器。可通过安装 akkuAUR 获取。

开发

学习

- Harold Abelson、Gerald Jay Sussman 与 Julie Sussman 所著的《计算机程序的构造与解释（第二版）》（1996）（另提供 EPUB3 和 PDF 格式版本）
- Programming Languages: Application and Interpretation（PLAI），lotuc 和 MrMathematica 提供了简体中文版本。

语言服务器

Scheme-langserver 是 Scheme 语言的语言服务器。可通过 Akku.scm 获取。

参见 Emacs-China 上的讨论。

参见

- Scheme.org
- Scheme 语言标准
- Scheme 实现提案（SRFI）
- Scheme 与 Common Lisp 的对比

11.3 Common Lisp

难度 进阶 | **篇幅** 9.6 KB | **所属** Lisp 方言

本节将讲到

- 安装
- 依赖管理
- 开发环境
- 提示
- 疑难解答

Common Lisp 是一种高度动态的多范例语言，强调交互性和性能。完全标准化后，其有许多独立的实现可供选择。

安装

sbcl包 (http://www.sbcl.org/) 是最受欢迎的 FOSS 实现，总体跨软件生态兼容度最高。其编译器可生成原生的机器码。该项目每月发布更新。

配置文件是 `~/.sbclrc`，使用 Lisp 语言编写。但是，基于您选择的依赖管理策略，您可能用不上它。

替代实现

有许多可用的替代实现。除下述外，有两大商用 Lisp 实现：Allegro 和 LispWorks，但它们附有严格的许可条款，也未打包到 Arch Linux 生态中。

活跃

这些功能与现代工具可无缝配合，可用于大型项目。

- **ABCL** — Armed Bear Common Lisp：在 Java 虚拟机上运行。

: https://common-lisp.net/project/armedbear/ || abclAUR

- **CCL** — Clozure Common Lisp：基于 Open Machintosh Common Lisp，以极快的编译速度闻名。

: https://ccl.clozure.com/ || cclAUR

- **Clasp** — Clasp：一种较新的 Common Lisp 实现，可与 C++ 库互操作，并借助 LLVM 编译至机器码。

: https://clasp-developers.github.io/ || clasp-clAUR

- **ECL** — Embeddable Common Lisp：可编译至 C 代码，从而提供良好的 C 集成和嵌入性。

: https://common-lisp.net/project/ecl/ || ecl包

历史

尽管有相应软件包，但库和工具的兼容性令人怀疑，而且有些实现已不再受到积极维护。

- **CLISP** — ANSI Common Lisp 解释器、编译器及调试器：提供良好的 C 集成和嵌入性。

: https://www.gnu.org/software/clisp/ || clisp包

- **CMUCL** — CMU Common Lisp：一个仅 POSIX 的实现，最初在卡内基梅隆大学开发。

: https://gitlab.common-lisp.net/cmucl/cmucl/-/wikis/home || cmucl包

- **GCL** — GNU Common Lisp：80 年代 Kyoto Common Lisp 的后代，ECL 的兄弟。

: https://www.gnu.org/software/gcl/ || gclAUR

- **MKCL** — Mankai Common Lisp：也是 Kyoto Common Lisp 的后代。

: https://mkcl.common-lisp.dev/ || mkcl-gitAUR

依赖管理

Quicklisp

Quicklisp 是这样一个系统：获取库，并将其加载到 Common Lisp 程序中。默认情况下，本机所有程序共用单一全局软件包缓存。而且，Quicklisp 从一个保守更新的仓库中提取软件包，该仓库也叫做 Quicklisp（可能引起混淆）。

安装 quicklisp包 包后，可通过以下方式，将其作为一个特定编译器注册：

```
> sbcl --load /usr/share/quicklisp/quicklisp.lisp
* (quicklisp-quickstart:install)
* (ql:add-to-init-file)
```

在此之后，可以在以后所有的 REPL 会话中使用 `(ql:quickload "foo")` 来加载一个依赖项，必要时会下载该依赖项。

要更新通过 Quicklisp 安装的所有包，在 REPL 中运行以下命令：

```
(ql:update-all-dists)
```

注意：一般来说，Quicklisp 假定您创建的所有 Common Lisp 项目都在 `~/common-lisp/` 目录下。

Ultralisp

Ultralisp 是一个替代 Quicklisp 的仓库，提供所有包的滚动更新，原则上与 Github 最新发布保持一致。

要将其注册，在 REPL 中运行：

```
(ql-dist:install-dist "http://dist.ultralisp.org/" :prompt nil)
```

如果您 `ql:quickload` 的包在 Ultralisp 中有较新版本，该包将从 Ultralisp 而非 Quicklisp 加载。

Qlot

或者，若您需要项目本地依赖，可以采用 Qlot (qlotAUR)。

安装后，项目仓库可以通过以下命令初始化：

```
qlot init
```

自定义依赖

使用 Qlot、所有自定义依赖的位置都记录在 `qlfile` 中。例如，要声明使用 Ultralisp，只需添加：

```
dist http://dist.ultralisp.org
```

或指定本地依赖：

```
local foobar /home/you/code/common-lisp/foobar
```

参见 Qlot 的 README 以了解更多选项。

要安装已经声明的依赖，运行：

```
qlot install
```

启用 REPL

要在当前 Qlot 环境中加载 REPL，请运行：

```
qlot exec sbcl
```

使用 Sly/Slime 的 Emacs 用户，考虑这样配置编辑器内 REPL 的启动方式：

```
(setq sly-default-lisp 'qlot-sbcl
      sly-lisp-implementations '((qlot-sbcl ("qlot" "exec" "sbcl") :coding-system
utf-8-unix)))
```

Slime 类似，进行必要的调整即可。

开发环境

Emacs

Common Lisp 开发常常通过 Emacs 完成：借助 slime 或较新的 sly。两者都被社区广泛采用且使用方法类似。

Lem

Lem (lem-editor-gitAUR) 是一个较新的 Emacs 风格编辑器，但是完全由 Common Lisp 编写和配置。其拥有终端和图形化前端，并支持许多语言。

提示

管理 Init 文件

库的作者经常使用多个编译器实现来测试他们的代码。但是，每个实现都有各自不同的 init 文件，例如 `.sbclrc` 或 `.eclrc`。但这些文件的内容往往是相同的：与其为每个编译器手写这些文件，不如创建一个主文件，并将其他文件链接到该文件。例如，如果您考虑 `.sbclrc` 成为您的 "主配置"，然后：

```
ln -s /home/you/.sbclrc .eclrc
ln -s /home/you/.sbclrc .abclrc
```

等等。这些主要 init 文件是：

```
~/.sbclrc           # for SBCL
~/.abclrc           # for ABCL
~/.ccl-init.lisp    # for CCL
~/.clasprc          # for CLASP
~/.eclrc            # for ECL
~/.clisprc.lisp     # for CLISP
~/.cmucl-init.lisp # for CMUCL
~/.mkclrc           # for MKCL
~/.clinit.cl        # for Allegro
```

疑难解答

Quicklisp 不能加载本地项目

`(ql:quickload "...")` 用于加载外部依赖项和本地项目。但若本地项目不位于 `~/common-lisp/`，将无法加载。项目甚至可能被从在线平台上删除，如果已经发布。

实际上，Quicklisp 只是获取和整理软件包。它内部使用 ASDF 构建系统，以实际加载这些包。也可以通过 `(asdf：load-system "foo")`，默认情况下，ASDF 只在 `~/common-lisp/` 中寻找本地项目。

虽然可以配置，但更简单的方法是使用符号链接：

```
ln -s /home/you/code/common-lisp/ common-lisp
```

注意：如果使用 Qlot，则不需要上述符号链接。只要您执行 `qlot install`，然后 `(asdf：load-package "foo")` 即可。

什么是项目、系统、软件包?

本文其余部分使用的“包”一词是其他编程语言通常使用的“库 ”的同义词。不过，虽然 Common Lisp 在 90 年代中期就已标准化，但其最早的形式却是在 80 年代出现的，因此涉及项目管理的术语也有所不同。实质上：

- 项目：一组系统。有时在其他语言中被称为“命名空间”。
- 系统：一组包。既可代表库，也可代表 “可执行文件”。
- 软件包：一组函数和类型定义。在其他语言中通常称为“模块”，但可跨多个文件。

如您所见，在其他地方通常被称为库的东西，在 Common Lisp 中被称为“系统”。因此有 `asdf:load-system` 的命名。参见此处以获得相互依赖、多系统项目的一个例子。

参见

- The Common Lisp 使用手册：许多问题的解决方法。

- 社区规范：在线 stdlib 文档。
- Awesome CL：生态概览。
- Quickdocs：Quicklisp 文档查询。
- Portability：编译器实现的兼容性列表。
- Wikipedia:Common Lisp
- Cliki：Common Lisp 维基

TeX

把 TeX 源文件编译成成品，靠的不是一个软件，而是发行版、引擎与编辑器三层：TeX Live 提供完整的宏包与工具集，Tectonic 走按需下载的轻量路线，TeXstudio 只是套在外面的编辑器，三者并非都得装。建议先读《TeX Live》，篇幅最长也最要紧，其中 Arch 打包版与原生安装的对比能帮你决定用 pacman 还是官方安装脚本；再看《Tectonic》，适合不愿维护庞大数据目录、可以联网取包的场合；《TeXstudio》可按需跳过，装上即用。

最容易踩的坑是把编辑器当成编译器：只装 TeXstudio 并不能编译文档，引擎仍需自行准备。另一处误解是两套 TeX Live 可以随意共存，两者目录与更新方式各自独立，混装会让版本与路径互相覆盖。

12.1　TeXstudio

难度 入门 | **篇幅** 1.6 KB | **所属** TeX

本节将讲到

- 安装

是一款免费的 LaTeX 编辑器。它提供多种功能，可轻松输入特殊字符、编辑表格、预览数学模式片段和语言检查。

安装

安装 texstudio包 软件包。

语言检查

如果想启用语言检查功能，请安装 hunspell包 和至少一个对应的词典。TeXstudio 中的语言选择将与所安装的 hunspell 词典相匹配。如果没有安装词典，则无法在应用程序中选择语言。

您也可以安装 languagetool包 来进行更高级的语言检查，或者连接到远程 LanguageTool 服务器。无论如何，您都需要为所需语言安装 hunspell 词典，否则即使语言工具包或服务器支持该词典，TeXstudio 也不会让您选择该词典。

12.2　Tectonic

难度 入门 | **篇幅** 2.0 KB | **所属** TeX

本节将讲到

- 安装
- 用法
- 参见

翻译状态：

- 本文（或部分内容）译自 Tectonic，最近一次同步于 2024-12-22，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Tectonic 是一个基于 XeTeX 的 TeX 发行版。

它由TeX Live 驱动，按需下载编译文档所需的 TeX 软件包。与传统的 TeX 工具不同，Tectonic 的一个优点是在编译文档时避免在工作目录中加入不必要的文件。

不过，Tectonic 基于的 XeTeX 引擎并没有得到维护，与 pdfTeX 或 LuaTeX 等其他引擎（均可通过标准 TeX Live 安装）相比，可能存在兼容性问题和功能缺失。

安装

Tectonic 引擎可通过 tectonic包 软件包安装。

用法

Tectonic 的基本用法是：

```
$ tectonic main.tex
```

只需运行一次命令，就能将其编译成完全处理过的 pdf 文件。

编译时，tectonic 会自动下载必要的 LaTeX 软件包，无需进行配置。

V2 界面

与默认的命令行界面相比，使用 `-X` 标志访问的新界面拥有更多功能。更多相关信息，请参阅官方 Tectonic Book。

参见

- Tectonic 网站
- Tectonic Book

12.3 TeX Live

难度 进阶 | **篇幅** 17.8 KB | **所属** TeX

本节将讲到

- Arch-packaged TeX Live 与原生 TeX Live 的对比
- 用法
- 窍门与技巧
- 相关页面

根据Wikipedia:

: **TeX Live** 是一种自由的的 TeX 排版系统发行版，包含主要的 TeX 相关程序、宏包和字体。

TeX Live 包括：

- 多种 TeX 引擎，例如：
- pdfTeX
- XeTeX
- LuaTeX
- 基本宏包（在 TeX 术语中称为“格式”），例如：
- LaTeX
- ConTeXt
- 许多用户制作的包

TeX Live 的替代方案包括 MikTeX 和 Tectonic，它们采用最小化安装方式，并在运行时按需下载所需的包。

Arch-packaged TeX Live 与原生 TeX Live 的对比

安装 TeX Live 有多种方式。虽然 TeX Live 的一个快照应该可以通过 Arch Linux 包仓库获得（如下所述），但该快照大多数情况下每年只更新一次。相比之下，TeX Live 本身是一个滚动发布的发行版，基于用户每年必须升级一次的模型，但每个软件包可以在子日级别进行更新。

拥有原生且最新的 TeX Live 安装的优点包括可以访问最新的功能和 bug 修复，以及相关的文档。尽管在没有原生 TeX Live 的情况下（请参见 `TEXMFLOCAL` 和 `TEXMFHOME`），仍然可以升级或安装单独的软件包，但这需要额外的注意——必须确保所有相关软件包的版本彼此兼容。

与原生 TeX Live 相比，Arch 仓库中的软件包通常不包括文档，这是一个问题，因为来自 CTAN 的文档通常仅适用于 TeX 包的最新版本。

注意：多个 TeX Live 安装可以在单一系统上和平共存。

原生 TeX Live

安装

请参见 quickinstall 文档，并根据需要查看 full 文档。安装过程如下：

1. 下载 netinstall 压缩包
2. 解压压缩包并进入新创建的目录

3. 运行安装脚本，`perl ./install-tl`，启动一个交互式安装程序（文本界面）
4. 设置所需的安装路径
5. 可选地选择 TeX Live 提供的某些“集合”包
6. 可选地选择 Letter 纸张尺寸作为默认，而不是 A4
7. 等待所有文件下载完毕
8. 安装脚本会在过程结束时通知您，需要将 TeX Live 安装路径添加到系统可执行文件和文档路径中：`PATH`、`MANPATH`、`INFOPATH`。安装脚本会输出在退出前所需的具体路径。

或者，也可以安装 texlive-installerAUR 包，然后按照上述步骤运行 `install-tl` 。

更新

请参阅 tlmgr 文档，`tlmgr update --all` 将升级安装时选择的所有包集合。它还会安装已添加到安装的 TeX Live 包集合中的新包。

每年一次，当 TeX Live 发布新版本时，`tlmgr update --all` 会失败，这时需要重新安装 TeX Live。

Arch 打包的 TeX Live

- texlive包组 组包含大多数 TeX Live 包，按上游集合分类（见 ）。
- texlive-basic包，基础安装，基于上游的“中等”安装方案。该包包括 pacman hook 自动化操作 *mktexlsr*、*fmtutil* 和 *updmap*。
- texlive-latex包 包含必要的 LaTeX 包。（例如，如果您打算使用 `pdflatex`，则需要此包）。
- texlive-latexrecommended包 和 texlive-latexextra包 包含许多有用的 LaTeX 包，如 `polyglossia`、`amsmath` 和 `graphicx`。
- texlive-fontsrecommended包 包含必需的字体（包括默认的 Latin Modern）。
- texlive-fontsextra包 包含附加字体，可在 LaTeX 字体目录查看。
- texlive-xetex包 和 texlive-luatex包 分别包含 XeTeX 和 LuaTeX 的包。
- texlive-bibtexextra包 包含 BibLaTeX 包和额外的 BibTeX 样式及参考文献数据库。
- texlive-mathscience包 包含数学、自然科学和计算机科学所需的基本包。
- texlive-lang包组 组包含为非拉丁字符语言提供字符集和功能的包。
- biber包 提供 BibLaTeX 的替代参考文献处理后端。

请注意，texlive-binextra包 中的某些工具有可选依赖项，未自动安装。例如，*latexindent* 依赖于 perl-yaml-tiny包 和 perl-file-homedir包。

提示：如果缺少特定的 *.sty* 文件，可以运行 `pacman -F` 来查找提供该文件的 Arch 包：

```
$ pacman -F soul.sty
extra/texlive-plaingeneric 2023.66594-19 (texlive)
usr/share/texmf-dist/tex/generic/soul/soul.sty
```

如果需要安装一个可用的 CTAN TeX 包，请运行以下命令来确定它是否包含在 Arch 的 *texlive-* 包中：

```
$ tlmgr info ctan_package_name | grep collection
```

这将列出与 Arch *texlive-* 包对应的 TeX Live 包集合名称（也可以检查 texlive包组 组以确认是否存在此包）。例如，输出 `collection: collection-plaingeneric` 表示该 TeX 包包含在 texlive-plaingeneric包 中。

另外，可以使用 `tlmgr` 手动安装单个 TeX 包（见下文）。

注意：某些 TeX Live 包中包含的工具和实用程序依赖于 ghostscript包、perl包、python包 或 ruby包。有关详细信息，请查看每个包的可选依赖项。

tllocalmgr

警告：有建议认为 tllocalmgr 将不再工作。请参阅 。

tllocalmgr 工具由 tllocalmgrAUR 提供，允许您将来自 CTAN 的包作为 pacman 包安装（和更新）。请参见使用方法（`-h`）了解详细信息。

tlmgr

使用 texlive-basic包 时，`tlmgr` 工具应当开箱即用。

默认情况下，其需要root权限才可运行。

```
# tlmgr install package_name
```

如您想在不使用 root 权限的情况下运行程序，请使用内置的用户模式。默认情况下，该功能会将 CTAN 软件包安装到 `~/texmf` 目录。 `tlmgr` 必须先对此目录进行初始化，因此请运行：

```
$ tlmgr init-usertree
```

现在，你可以使用用户模式：

```
$ tlmgr --usermode install package_name
```

要更改包的安装位置，请更改 `TEXMFHOME` 这个环境变量。

```
$ export TEXMFHOME="$HOME/.local/texmf"
$ tlmgr init-usertree
$ ...
```

`tlmgr` 会自行选择一个合适的镜像。不过，你也可以自行设置首选镜像网站。需要你在镜像路径末尾添加 `/systems/texlive/tlnet`。例如：

```
$ tlmgr option repository http://mirrors.rit.edu/CTAN/systems/texlive/tlnet
```

包文档

官方仓库中的包不包含字体/宏包的文档或源文件。

要使用`texdoc` 离线访问文档，请安装 texlive-doc包 包，它包含完整的 TeX Live 文档和源文件。

您也可以在线访问文档：

- https://tug.org/texlive/Contents/live/doc.html
- https://ctan.org/ – TeX 相关材料的中心
- https://texdoc.org/ (`http://texdoc.org/pkg/*packagename*` 可直接获取相关 PDF)

用法

请参见以下资源：

- Wikibooks:LaTeX
- 《LaTeX 2ε 简明介绍》
- 学习 LaTeX – Andrew Roberts
- TeX FAQ
- 《GNU TeX for the Impatient》（中文译本：《TeX 急就帖》）

有些人使用 TeX 编辑器来创建文档。

编译文档的高级封装器

通常来说，使用 latexmk(1) 等高级工具，或者使用 arara(1)，比直接调用 pdflatex(1) 更有意义，主要是因为较低级的工具通常需要多次运行才能完全编译一个文档。

texmf 树和 Kpathsea

texmf 树（*texmf* 代表 TeX 和 Metafont）应遵循 TeX 目录结构，否则可能无法找到文件。

TeX Live 使用 Kpathsea 库来通过文件名查找路径，跨越多个 texmf 树和当前工作目录。

Kpathsea 按逆序搜索以下变量（较晚的树会覆盖较早的树）。

变量	Arch 默认值 1)	由 使用
TEXMFDIST	/usr/share/texmf-dist	原始分发文件
TEXMFLOCAL	/usr/local/share/texmf:/usr/share/texmf	系统管理员用于全局安装附加或更新的宏、字体等

变量	Arch 默认值 1)	由 使用
TEXMFSYSVAR	/var/lib/texmf	updmap 和 fmtutil（用户模式）存储（缓存）运行时数据
TEXMFSYSCONFIG	/etc/texmf	updmap 和 fmtutil（用户模式）存储修改后的配置数据
TEXMFHOME	~/texmf	用户用于个别安装附加或更新的宏、字体等
TEXMFVAR	~/.texlive/texmf-var	updmap 和 fmtutil（系统模式）存储（缓存）运行时数据
TEXMFCONFIG	~/.texlive/texmf-config	用户模式的配置数据
TEXMFCACHE	$TEXMFSYSVAR; $TEXMFVAR	

注意：默认值在/etc/texmf/web2c/texmf.cnf 中定义；它们可以通过环境变量进行覆盖。

Kpathsea 提供了 kpsewhich(1) 命令来查找路径。当使用 `--var-brace-value=*VARIABLE*` 参数运行时，它还可以打印变量的值。

Kpathsea 使用文件名数据库（`ls-R`）来加速在系统级 texmf 树（通过 `TEXMFDBS` 变量配置）中的搜索。这意味着当系统级文件树发生更改时，需要以 root 运行 mktexlsr(1) 或 `texhash`（符号链接）。texlive-basic包 通过 pacman hook 实现了自动化，该 hook 针对所有默认的系统级 texmf 树，但 `/usr/local/share/texmf`。因此，只要您通过 pacman 安装系统级软件包，就完全不需要运行 *mktexlsr* 或 *texhash* 。

提示：要设置本地存储库，您可以创建~/texmf/目录结构{format} 通常是latex），其中自定义类位于 ./{format}文件夹的根目录中，其他本地文件则放在同名文件夹中（例如，mycustompackage.sty放在./{formats}/mycustompackage/mycustompackage.sty文件夹中），然后运行texhash来更新用户数据库。

窍门与技巧

更改默认纸张尺寸

**本文或本章节的事实准确性存在争议。**

原因： 可以通过直接编辑配置文件来实现。（在 Talk:TeX Live 中讨论）

目前无法使用 texlive 工具设置默认页面大小，因为它们与标准的 Arch 软件包不兼容。

通常情况下，你会运行 `texconfig` 或 `tlmgr` ，它们还可以更改其他有用的设置。

您可以按如下方式编辑配置文件。

- 要为 *pdftex* 系列命令（ *pdftex* 、 *pdflatex* 等）设置纸张大小，请编辑 `/etc/texmf/tex/generic/tex-ini-files/pdftexconfig.tex` 文件。

也可以通过运行 `kpsewhich pdftexconfig.tex` 来发现。修改指定 `pdfpageheight` 和 `pdfpagewidth` 的两行。例如，要使用信纸大小，请修改

```
\pdfpageheight = 297 true mm
\pdfpagewidth  = 210 true mm
```

到

```
\pdfpageheight = 11 true in
\pdfpagewidth  = 8.5 true in
```

遗憾的是，这只有在 *.fmt* 二进制文件重新构建后才会生效。您可以使用 `fmtutil-sys` 来完成此操作。使用 pacman 重新安装 texlive-bin 也能达到同样的效果。

- 对于 *dvips* ，您可以使用 `-t` 选项指定纸张尺寸： `dvips -t letter foo.dvi` 。要更改默认值，请编辑 `/etc/texmf/dvips/config/config.ps` 文件。

也可以使用 `kpsewhich config.ps` 命令来查找。该文件的末尾列出了 *dvips* 已知的所有纸张尺寸。列表中第一个纸张尺寸将作为默认值。只需将您希望设为默认值的纸张尺寸移动到列表顶部即可。一旦该文件发生更改， *dvips* 的行为就会受到影响。

让字体在Fontconfig下识别

默认情况下，各种 TeX Live 宏包自带的字体不会自动被 Fontconfig 识别 。如果您想在 XeTeX 或 LibreOffice 等软件中使用这些字体，最简单的方法是在字体目录中创建一个指向用户字体路径下子目录的符号链接。对于 OpenType 字体，请运行以下命令：

```
$ ln -s /usr/share/texmf-dist/fonts/opentype/public/some_fonts_you_want ~/font_path/OTF/
```

要使 fontconfig 能够访问它们，请运行：

```
$ fc-cache ~/font_path
$ mkfontscale ~/font_path/OTF
$ mkfontdir ~/font_path/OTF
```

TrueType 字体和 Type 1 字体的操作步骤类似。在上面的几行代码中，将 `opentype` 替换为 `truetype` 或 `type1` ，将 `OTF` 替换为 `TTF` 或 `Type1` 。

或者， texlive-basic 包含文件 `/usr/share/fontconfig/conf.avail/09-texlive-fonts.conf` ，其中包含 TeX Live 使用的字体目录列表。您可以将此文件与以下命令一起使用：

```
# ln -s /usr/share/fontconfig/conf.avail/09-texlive-fonts.conf /etc/fonts/conf.d/09-texlive-fonts.conf
```

然后更新系统配置

```
$ fc-cache && mkfontscale && mkfontdir
```

注意：如果 TeX 和 Fontconfig 分别使用了相同的字体，即搜索路径上有多个相同的字体副本，则可能会与 XeTeX/XeLaTeX 发生冲突。 LibreOffice 5.3 及更高版本将不再提供 Type 1 字体。

更新babelbib语言定义

本文或本章节的语言、语法或风格需要改进。参考：帮助:风格

原因：您不应该更改 `/usr/share/texmf-dist/` 。（在Talk:TeX Live#Talk:TeX_Live讨论）

如果您遇到 babelbib 没有您需要的最新语言定义这一特殊问题，并且不想重新编译所有内容，您可以从 https://www.tug.org/texlive/devsrc/Master/texmf-dist/tex/latex/babelbib/ 手动获取这些定义，并将其放入 `/usr/share/texmf-dist/tex/latex/babelbib/` 中。例如：

```
$ cd /usr/share/texmf-dist/tex/latex/babelbib/
# wget https://www.tug.org/texlive/devsrc/Master/texmf-dist/tex/latex/babelbib/romanian.bdf
# wget [...all-other-language-files...]
# wget https://www.tug.org/texlive/devsrc/Master/texmf-dist/tex/latex/babelbib/babelbib.sty
```

之后，您需要运行 `texhash` 来更新 TeX 数据库：

```
# texhash
```

相关页面

- LaTeX
- TeX
- TeX Live
- pacman

控件工具集

本章处理 Qt 与 GTK 两套控件工具集在同一台 Arch 上的共存。两者各带一套主题、主题引擎与配置逻辑，装完常出现深色模式只对一半程序生效、配置改了却不生效的割裂外观。

建议先读《统一 Qt 和 GTK 应用程序的外观》，通用主题与主题引擎是后两节共同的底座，必读；再按程序属于哪一边翻到 Qt 或 GTK 一节，查安装、配置与疑难解答。不做图形程序开发的话，这两节里的开发内容可以跳过。最容易踩的坑是以为存在一处开关统管全部外观：通用主题覆盖不到的角落，仍要回到各框架自己的配置、GDK 后端与主题引擎细节里逐个调。

13.1　统一 Qt 和 GTK 应用程序的外观

难度 进阶 | **篇幅** 14.7 KB | **所属** 控件工具集

本节将讲到

- 介绍
- Qt 和 GTK 通用的主题
- 主题引擎
- Tips and tricks
- 故障排查

基于 Qt 和 GTK 的程序使用不同的图形库来渲染图形化用户界面，各自具有不同的主题、风格和图标，所以观感明显有所不同。这篇文章将使你的 Qt 和 GTK 程序看起来更相似，以此获得更加现代化、集成化的桌面体验。

介绍

要想在不同的图形库之间获得相似的外观，你通常需要修改以下内容：

- **主题**——应用程序和组件等元素的自定义外观。通常包含一套风格、图标主题和颜色主题。
- **风格**——组件的布局和样式。
- **图标主题**——全局图标。
- **颜色主题**——与风格相适应的全局颜色样式。

你可以选择下面的不同方法：

- 使用下面列出的，适用于各个图形库的工具分别修改 Qt 和 GTK 的主题，以选择外观相似的主题（样式，颜色，图标，光标，字体）。
- 使用特殊的主题引擎，该引擎作为中间人修改其他图形库的主题，以匹配您的主要图形库。

Qt 和 GTK 通用的主题

下面列出了几种主题，Qt 和 GTK 都可使用，并且风格统一。这些主题适配了 Qt 和 GTK 的主要版本。有了这些主题，你就可以在使用不同的图形库构建的应用程序之间得到相同的外观了。

提示：如果希望将用户定义的主题应用于 root 用户的应用, 参考GTK#主题没有应用到 root 用户的应用程序上和Qt#主题没有应用到 root 用户的应用程序上。

注意：GTK 3 从 3.16 版本开始不再支持非 CSS 主题，因此，以前的解决方案，例如 Oxygen-Gtk，已经不再可用。

Breeze

Breeze 是 KDE Plasma 的默认 Qt 主题。它可以通过适用于 Qt 5 的 breeze包，适用于 Qt 4 的 breeze-kde4AUR 以及适用于 GTK 2 和 GTK 3 的 breeze-gtk包 软件包安装。

安装后，您可以使用GTK 配置工具更改 GTK 主题。

如果当前运行的是 KDE Plasma，请安装 kde-gtk-config包，然后从命令行运行之；或者注销后重新登录，接着转到*系统设置* > *外观* > *应用程序风格* > *配置 GNOME/GTK 应用程序风格…*来配置 GTK 程序的主题。“系统设置”中设置的字体、图标主题、光标和组件样式将自动应用至 GTK，仅 GTK 主题需要用前面提到的 GTK 配置工具手动更改。

Adwaita

Adwaita是 GNOME 默认的主题。GTK 3 版本包含在 gtk3包 软件包中，而 GTK 2 版本包含在 gnome-themes-extra包 中。adwaita-qt是 Adwaita 主题的 Qt 移植版本。与模仿 GTK 2 主题的 #QGtkStyle 不同，它提供了与 GTK 3 Adwaita 相似的原生 Qt 样式。可以通过适用于 Qt 4 的 adwaita-qt4AUR、适用于 Qt 5 的 adwaita-qt5包 和适用于 Qt 6 的 adwaita-qt6包 软件包来安装此主题。

要将 Qt 样式设置为默认值：

- 对于 Qt 4，可以使用*Qt 配置*（`qtconfig-qt4`）启用它，在*外观* > *GUI 样式*中选择*adwaita*。或者，编辑 `/etc/xdg/Trolltech.conf`（系统范围）或 `~/.config/Trolltech.conf`（用户范围）文件：

```
~/.config/Trolltech.conf
...
[Qt]
style=adwaita
...
```

- 对于 Qt 5，可以通过以下环境变量来启用主题：`QT_STYLE_OVERRIDE=adwaita`。或者，使用 qt5ct包 软件包。更多详细说明，参考 Qt#在KDE Plasma以外的环境下配置Qt5应用程序。

主题引擎

主题引擎可以认为是一个简单的 API，用于从一个或多个图形库转译主题（图标除外）。这些引擎为进程添加额外的代码，这种解决方案不如使用原生主题那样优雅可行。

Kvantum

Kvantum（kvantum包）是一个用于 Qt 5 的，基于 SVG 的可定制主题引擎，具有多种内置样式，包含一些版本的流行 GTK 主题，例如 Adapta、Arc、Ambiance 和 Materia。

Kvantum 被视为一种风格而不是主题。要通过环境变量的方式为所有 Qt 应用设置 Kvantum，使用`QT_STYLE_OVERRIDE=kvantum`

主题配置

要为 Kvantum 设置一个主题变体，如 KvLibadwaita，像这样编辑配置文件：

```
/etc/xdg/Kvantum/kvantum.kvconfig（或 ~/.config/Kvantum/kvantum.kvconfig）
...
theme=KvLibadwaita
...
```

QGtkStyle

注意：QGtkStyle 已从 qt5-base包 5.7.0 中移除并添加到 qt5-styleluginsAUR 中。

警告：取决于 GTK 2 的主题，此样式可能会导致显示问题，例如透明的字体或不一致的部件。

此 Qt 样式使用 GTK 2 渲染所有组件以便于与 GNOME 和类似基于 GTK 的环境相协调。从 Qt 4.5 开始，此样式包含在 Qt 中。它要求安装 gtk2包 并进行配置。

这是 Cinnamon、GNOME 和 Xfce 中的默认 Qt 4 样式，也是 Cinnamon、GNOME、MATE、LXDE 和 Xfce 中的默认 Qt 5 样式。至于其他环境：

- 对于Qt4，可以使用*Qt 配置*（`qtconfig-qt4`）启用它，在*外观* > *GUI 样式*下选择 *GTK*。或者，编辑 `/etc/xdg/Trolltech.conf`（系统范围）或 `~/.config/Trolltech.conf`（用户范围）：

```
~/.config/Trolltech.conf
...
[Qt]
style=GTK+
...
```

- 对于 Qt 5，可以通过安装 qt5-stylepluginsAUR 并设置以下环境变量来启用它：`QT_QPA_PLATFORMTHEME=gtk2`
- 对于 Qt 6，可以通过安装 qt6gtk2AUR 并在 qt6ct包 中选择 *qt6gtk2* 样式，或设置以下环境变量：`QT_QPA_PLATFORMTHEME=gtk2`。

为了完全统一，请确保配置的 GTK 主题同时支持 GTK 2 和 GTK 3。如果你的首选主题在将 Qt 配置为使用 GTK 2 后出现了不一致的渲染效果，请安装 gtk-theme-switch2AUR，然后选择一个主题。

QAdwaitaDecorations

Qt 装饰插件实现类似 GNOME Adwaita 的客户端装饰，适用于Qt 6 和 QT 5 平台。

安装 qadwaitadecorations-qt5AUR 和 qadwaitadecorations-qt6包 软件包。

可以通过设置环境变量 `QT_WAYLAND_DECORATION=adwaita` 开启。

QWhiteSurGtkDecorations

QWhiteSurGtkDecorations是一个 Qt 装饰插件，可以在Gnome wayland上实现类似 GNOME WhiteSur-gtk 主题风格的客户端装饰，适用于Qt 6 和 QT 5 平台。

需要安装 qwhitesurgtkdecorations-qt5AUR 和 qwhitesurgtkdecorations-qt6包 软件包。

可以通过设置环境变量 `QT_WAYLAND_DECORATION=whitesur-gtk` 启用它。

Tips and tricks

在Qt应用程序中使用GTK图标主题

如果您正在运行Plasma，请安装kde-gtk-config包，然后在*系统设置* > *应用程序风格* > *应用样式* > *配置GNOME/GTK应用样式*下选择图标主题。

如果您使用的是GNOME，请首先检查是否安装了dconf-editor包。

然后，运行`dconf-editor`，然后在*org*> *gnome* > *desktop* > *interface* 下查找`icon-theme`键，并将其更改为您的首选图标主题。

如果您使用的不是GNOME，例如，如果您使用的是i3-wm包，请首先安装dconf-editor包。

然后，运行`dconf-editor`，然后在 *org* > *gnome* > *desktop*> *interface*下查找`icon-theme`键，并将其更改为您的图标主题。

由于您使用的不是GNOME，因此可能需要在个人的配置文件中设置DESKTOP_SESSION的值。 为此，请在终端中执行以下代码，然后重新启动系统。

```
$ echo 'export DESKTOP_SESSION=gnome' >> /etc/profile
```

或者

将export DESKTOP_SESSION=gnome设置在~/.xinitrc中的某个位置，或者在Xprofile中使用一个Display manager。

注意：如果未应用图标主题，则可能要检查您输入的首选主题名称是否格式正确。例如，如果要将当前活动的图标主题应用于QT应用程序，则可以使用以下命令找到其名称的正确格式：

```
$ awk -F= '/icon-theme/ {print $2}' ~/.gtkrc-2.0
```

添加标题栏和框架到KDE Plasma下的GTK3应用程序

要使Gnome/GTK应用程序显示KDE/Plasma的标题栏和框架，请安装gtk-nocsd-gitAUR并重新启动窗口管理器以加载更新的库路径。

您还可以使用它运行Gtk应用程序：

```
$ gtk3-nocsd gedit
```

在KDE Plasma下改善GTK应用程序的亚像素渲染

See Font configuration#LCD filter.

一致的文件对话框

为了具有相同的文件对话框，可以使用XDG Portal。

安装 xdg-desktop-portal包和xdg-desktop-portal-kde包，并在环境变量environment variable中设置GTK_USE_PORTAL=1。

故障排查

主题对GTK程序无效

如果你安装的风格或主题引擎在某些GTK程序不能显示，很可能你的GTK设置文件因某些原因不能被加载。你可以检查你的系统找到那些文件作如下设置：

```
export | grep gtk
```

通常那些文件设置在 ~/.gtkrc （GTK1）, ~/.gtkrc2.0 或 ~/.gtkrc2.0-kde （GTK2）

新版gtk-qt-engine 使用 ~/.gtkrc2.0-kde 和 ~/.kde/env/gtk-qt-engine.rc.sh 设置输出变量 如果你最近移除了gtk-qt-engine然后试图设置GTK主题，你必有要移除 ~/.kde/env/gtk-qt-engine.rc.sh 然后重启。这样做会使GTK外观使用标准的设置 ~/.gtkrc2.0来代替 ~/.gtkrc2.0-kde

系统升级后GTK应用程序不使用svg（breeze）图标

尝试运行下面命令来解决问题：

```
# gdk-pixbuf-query-loaders --update-cache
```

Flatpak Qt应用程序不使用Gnome Adwaita黑暗主题

如果将主题切换为Adwaita-dark后，Flatpak Qt应用程序仍使用精简版，请安装所需的KStyle：

```
# flatpak install flathub org.kde.KStyle.Adwaita
```

即使在设置Qt主题后，在GNOME Wayland上运行的Qt应用也有不匹配的窗口装饰外观

安装 qadwaitadecorations-qt5AUR 和 qadwaitadecorations-qt6AUR，并设置以下环境变量environment variable:`export QT_WAYLAND_DECORATION=adwaita`。

GTK 应用程序不能完全使用 KDE 系统设置

为了进一步整合 GTK 应用程序上的 Plasma 设置，可能需要安装 gnome-settings-daemon包、gsettings-desktop-schemas包 和 gsettings-qt包，这将为 GTK 提供适当的 Qt 绑定。

安装kde-gtk-config后，没有“系统设置>应用程序样式> GTK”菜单

使用kde-gtk-config包时，可以选择lxappearance包之类的GTK configuration tools来配置GTK 2和GTK 3样式。

即使它来自LXDE项目，它也是与桌面无关的（它不需要LXDE桌面的其他部分）

Dolphin主题与Nautilus匹配效果不佳

参考 arch:Dolphin#Mismatched_folder_view_background_colors 小节以解决颜色奇怪的问题

13.2 Qt

难度 高级 | **篇幅** 23.1 KB | **所属** 控件工具集

本节将讲到

- 安装
- 默认 Qt 库
- 配置
- 开发
- 解决问题

Qt 是一个跨平台的应用程序和图形组件工具包，它使用标准C++，但同时对C++作出了功能拓展，通过使用特别的代码生成器（称为 Meta Object Compiler ，元对象编译器，简称 *moc* ）以及数个宏来扩展语言的功能。它还包括以下几个更重要的特性：

- 支持各种主流桌面平台和部分手机平台。
- 完善的国际化支持。
- 提供 SQL 数据访问、XML 解析、线程管理、网络支持和统一的跨平台的文件处理API。

Qt 框架是 KDE 软件社区和其它一些重要开源和闭源应用的基石，例如 VLC、VirtualBox、Mathematica 等等。

安装

可以从官方仓库安装 Qt 6.x 和 5.x 。旧版本的Qt （4.x 和 3.x）可以从 AUR 安装。具体通过如下的软件包安装：

- **Qt 6.x**：软件包 qt6-base包，文档包 qt6-doc包。
- **Qt 5.x**：软件包 qt5-base包，文档包 qt5-doc包。
- **Qt 4.x**：软件包 qt4AUR，文档包 qt4-docAUR。
- **Qt 3.x**：软件包 qt3AUR，文档包 qt3-docAUR。

默认 Qt 库

安装 qtchooserAUR 可以改变 `/usr/bin` Qt二进制文件（例如， *qmake* ）的指向（它们默认指向Qt5版本的程序，例如 *qmake-qt5* ），让它们指向旧版本的程序（例如，*qmake-qt4* 或者 *qmake-qt3* ）。

注意：现在 qtchooserAUR 与 qt5-base包 相冲突，可以自行编译并安装到`/usr/local`。Arch 官方不支持这种方式FS#51308。

修改环境变量

可以通过 `QT_SELECT` 环境变量设置默认的 QT. 例如要使用 Qt4，可以设置 `export QT_SELECT=4`。

使用配置文件

创建 `~/.config/qtchooser/default.conf` 软链接，链接到`/etc/xdg/qtchooser/`目录中需要的 *.conf* 文件上。例如要使用 Qt4，将 `/etc/xdg/qtchooser/4.conf` 软链接到 `~/.config/qtchooser/default.conf`。

```
$ ln -s /etc/xdg/qtchooser/4.conf ~/.config/qtchooser/default.conf
```

配置

Qt5 样式

Qt5基于当前使用的桌面环境来决定所使用的样式：

- 在 KDE Plasma 桌面环境中，呈现实际选择的Qt风格。可以在 *KDE System Settings* （ *systemsettings* ）中更改，这项设置的具体位置是*外观* > *应用程序风格*。
- 在 Cinnamon、GNOME、MATE、LXDE、Xfce 等桌面环境中，呈现GTK风格 (QGtkStyle)。
- 在其他桌面环境中，呈现 Fusion 风格。

如果要强制指定一种样式，你可以设置 `QT_STYLE_OVERRIDE` 环境变量。特别的，如果你想要使用GTK主题，把它设置成`gtk2`（注意：你将需要安装在下文中提到的Qt样式插件来获取GTK样式）。Qt5应用同时也支持`-style`标志，你可以用它来使用指定的样式运行一个Qt5应用程序。

Qt5中自带两种样式: *Fusion*、*Windows*。其他的可以通过官方仓库安装:

- **Breeze** — 来自Plasma桌面的Breeze视觉样式，包括了绘画、样式和素材。

: https://invent.kde.org/plasma/breeze || breeze包

- **Oxygen** — KDE 的 Oxygen 轻氧风格。

: https://invent.kde.org/plasma/oxygen || oxygen包

- **Lightly** — Lightly是breeze主题的fork，它致力于现代风和极简风格。

: https://github.com/Luwx/Lightly || lightly-gitAUR

- **QtCurve** — 为KDE和Gtk编写的可配置的小部件样式。

: https://invent.kde.org/system/qtcurve || qtcurve-qt5包

- **Adwaita-Qt** — 让Qt应用程序看上去有GNOME风格。

: https://github.com/MartinBriza/adwaita-qt || adwaita-qt5包

- **Qt style plugins** — Qt 5的附加样式，包含了 *GTK*、*Cleanlooks*、*Motif*、*Plastique*。

: https://code.qt.io/cgit/qt/qtstyleplugins.git || qt5-styleplugins AUR

- **Kvantum** — 可定制的基于 SVG 的主题引擎，具有多种内置样式，包括对一些流行的 GTK 主题的模仿，例如 *Adapta*、*Arc*、*Ambiance*。

: https://github.com/tsujan/Kvantum/tree/master/Kvantum || kvantum包

Qt4 样式

Qt4 应用程序会尝试模仿所运行的桌面环境的行为，除非碰到了某些问题或者进行了强制配置。

- 在 KDE Plasma 桌面环境中，呈现实际选择的Qt风格。可以在 *KDE System Settings* （ *systemsettings* ）中更改，这项设置的具体位置是*外观* > *应用程序风格*。
- 在 Cinnamon、GNOME、Xfce 等桌面环境中，呈现GTK风格 (QGtkStyle)。
- 在其他桌面环境中，呈现 Windows 风格。

要修改 Qt4 程序的外观，可以使用 qt4AUR 提供的 Qt 配置工具*qtconfig-qt4*。这个程序可以配置 Qt4 程序的样式、颜色、字体等。

注意：如果使用 *GTK* 样式，将忽略颜色和字体设置，直接使用 GTK2 的值。

Qt将所有的配置信息保存在`/etc/xdg/Trolltech.conf` (系统级别) 或者`~/.config/Trolltech.conf` (只适用于特定用户)。这个文件很难浏览，因为它还包含许多与外观无关的信息，但是要更改它，您只需添加到文件末尾并覆盖任何以前的值（注意要把修改添加在\[Qt\]标题下）。

例如要将主题更改为 QtCurve，请添加：

```
~/.config/Trolltech.conf
...
[Qt]
style=QtCurve
```

Qt4 已经包含数种样式，例如 GTK 样式、Windows 样式、CDE 样式等，其它的主题（大多数为 KDE Plasma 桌面编写）可以从官方源或者 AUR 中安装：

Qt4 自带这些样式: *CDE*、*Cleanlooks*、*GTK*、*Motif*、*Plastique*、*Windows*。其它的主题（大多数为 KDE Plasma 桌面编写）可以单独安装：

- **Breeze** — 来自Plasma桌面的Breeze视觉样式，包括了绘画、样式和素材。

: https://invent.kde.org/plasma/breeze || breeze-kde4AUR

- **Adwaita-Qt** — 让Qt应用程序看上去有GNOME风格。

: https://github.com/MartinBriza/adwaita-qt || adwaita-qt4AUR

Qt 样式表

定制Qt程序的外观有一个有趣的方式，那就是通过Qt样式表，它们只是简单的CSS文件而已。通过样式表，你可以修改程序中所有组件的外观。

想要用不同的风格运行程序，只需要执行：

```
$ qt_application -stylesheet style.qss
```

要了解关于Qt样式表的更多信息，请浏览官方文档或者其他的教程。Qt样式表的示例：修改Dolphin的外观。

GTK+ 和 Qt

如果你有 GTK 和 Qt 应用程序，它们的外观可能无法融合到一起。如果你希望使 GTK 风格与 Qt 风格匹配，请阅读统一 GTK 和 Qt 应用程序外观.

在KDE Plasma以外的环境下配置Qt5应用程序

不像Qt4，Qt5并没有提供一个qtconfig实用程序来配置字体、图标或者风格。相应的，它会尝试使用桌面环境提供的配置。在KDE Plasma和GNOME下这个功能运行的很好，但对于其他的比较小众的桌面环境或者窗口管理器而言，这可能导致Qt5程序缺少图标。解决方案之一是通过设置变量 `XDG_CURRENT_DESKTOP=KDE` 或 `GNOME` 来假装当前运行着的桌面环境，然后再使用相应的配置程序来配置想要的图标。

另一个解决方案是安装 qt5ct包，它提供了一个独立于桌面环境的 Qt5 QPA 和一个配置实用程序。安装完成后，运行 `qt5ct` 以设置图标主题，然后修改环境变量 `QT_QPA_PLATFORMTHEME=qt5ct`，这样做出的配置才会被Qt程序读取。或者不修改环境变量，使用 `--platformtheme qt5ct` 作为Qt5程序的参数也可以达到目的。

qt5ct-kdeAUR 提供了一个修改过的 *qt5ct*，它更好的整合了KDE程序，包括KDE QML程序。

如果遇到了下列错误，并且一些图标依然不在一些应用程序中出现，安装oxygen包和oxygen-icons包：

```
Icon theme "oxygen" not found.
Icon theme "oxygen" not found.
Error: standard icon theme "oxygen" not found!
```

开发

支持的平台

Qt支持如今的绝大多数平台，甚至包括一些十分冷门的平台，而且每隔一段时间就会出现更多的移植。更加完整的平台列表请查阅维基百科上的Qt#Platforms)。

Android

注意：Qt Creator 4.12或更高版本可以自动设置SDK工具、NDK和必要的软件包。有关更多信息，请参阅Qt Creator:指定Android设备设置

首先从 AUR 或使用 Android Studio 安装一个 Android SDK 和 NDK。

SDK 也需要 OpenJDK，版本要求也有不同，请查阅这里。

接下来你需要安装Qt 5 for Android。你可以按下面描述的包从 AUR 安装，也可以自行构建。构建需要用到的命令可以从 Qt wiki 找到。

如果遇到了问题，你可以需要查阅已知问题。

- android-armv7a-eabi-qt5AUR - armeabi-v7a
- android-aarch64-qt5AUR - aarch64
- android-x86-qt5AUR - x86
- android-x86-64-qt5AUR - x86_64

或者你也可以使用官方 Qt 安装程序进行安装。

工具

以下是官方的Qt工具：

- **Qt Creator** — 专为Qt设计的跨平台IDE，支持Qt所有的特性。

: https://doc.qt.io/qtcreator/ || qtcreator包

- **Qt Linguist** — Qt程序的多语言支持工具集。

: https://doc.qt.io/qt-5/qtlinguist-index.html || Qt 5: qt5-tools包, Qt 4: qt4AUR

- **Qt Assistant** — 可配置、可再分发的Qt *qch* 文件阅读器。

: https://doc.qt.io/qt-5/qtassistant-index.html || Qt 5: qt5-tools包, Qt 4: qt4AUR

- **Qt Designer** — 为Qt Widget程序编写的跨平台GUI设计、窗体构建工具。

: https://doc.qt.io/qt-5/qtdesigner-manual.html || Qt 5: qt5-tools包, Qt 4: qt4AUR

- **Qt Quick Designer** — QML文件的可视化编辑器，支持WYSIWYG（所见即所得）。它能让你从零开始快速构建一个Qt Quick程序。

: https://doc.qt.io/qtcreator/creator-using-qt-quick-designer.html\[失效链接 2022-09-22 ⓘ\] || qtcreator包

- **qmlscene** — 可以用来加载QML文档的工具。这让开发和调试QML程序变得很快。

: https://doc.qt.io/qt-5/qtquick-qmlscene.html || Qt 5: qt5-declarative包, Qt 4 QML Viewer: qt4AUR

- **qmake** — 用来简化跨平台程序的构建过程的工具，与 cmake 相似，但提供的选项更少，同时是为Qt程序定制的。

: https://doc.qt.io/qt-5/qmake-manual.html || Qt 5: qt5-base包, Qt 4: qt4AUR

- **uic** — 读取 \.ui* XML文件并且生成相应的C++文件的工具。

: https://doc.qt.io/qt-5/uic.html || Qt 5: qt5-base包, Qt 4: qt4AUR

- **rcc** — 用于在构建过程中将资源（例如图片）嵌入到Qt应用程序中的工具。它生成包含在 Qt 资源 (.qrc) 文件中指定的数据的 C++ 源文件。

: https://doc.qt.io/qt-5/rcc.html || Qt 5: qt5-base包, Qt 4: qt4AUR

- **moc** — 用来处理Qt的C++语言功能拓展的工具（例如信号槽技术、运行时类型信息、动态属性系统等等）。

: https://doc.qt.io/qt-5/moc.html || Qt 5: qt5-base包, Qt 4: qt4AUR

语言支持

Qt支持许多流行的编程语言，查阅 https://wiki.qt.io/Language_Bindings 以获取完整列表。

下面的例子会在一个窗口中显示 'Hello world!'。

C++

- 包：qt5-base包
- 网站： https://www.qt.io/developers/
- 构建：`g++ $(pkg-config --cflags --libs Qt5Widgets) -fPIC -o hello hello.cpp`
- 运行：`./hello`

```
hello.cpp
#include <QApplication>
#include <QLabel>

int main(int argc, char **argv)
{
    QApplication app(argc, argv);
    QLabel hello("Hello world!");

    hello.show();
```

```
    return app.exec();
}
```

QML

- 包：qt5-declarative包.
- 网站： https://doc.qt.io/qt-5/qtquick-qmlscene.html
- 运行：`qmlscene hello.qml`

```
hello.qml
import QtQuick 2.3

Rectangle {
    id: page
    width: 400; height: 100
    color: "lightgray"

    Text {
        id: helloText
        text: "Hello world!"
        anchors.horizontalCenter: page.horizontalCenter
        anchors.verticalCenter: page.verticalCenter
        font.pointSize: 24; font.bold: true
    }
}
```

Python (PyQt)

- 包：python-pyqt5包 - Python 3 bindings
- 网站： https://riverbankcomputing.com/software/pyqt/intro
- 运行：`python hello-pyqt.py`.

```
hello-pyqt.py
import sys
from PyQt5 import QtWidgets

app = QtWidgets.QApplication(sys.argv)
label = QtWidgets.QLabel("Hello world!")

label.show()
sys.exit(app.exec_())
```

Python (PySide2)

- 包：pyside2包
- 网站： https://wiki.qt.io/Qt_for_Python
- 运行：`python hello-pyside.py`

```
hello-pyside2.py
import sys
from PySide2.QtWidgets import QApplication, QLabel

app = QApplication(sys.argv)
label = QLabel("Hello world!")

label.show()
sys.exit(app.exec_())
```

C#

查看 QtSharp。

解决问题

禁用/更改 Qt 日志行为

当使用 KDE 和/或者任何其他Qt 桌面环境时，调试信息可能会频繁的输出到 systemd 日志中。

将 `QT_LOGGING_RULES` 设为环境变量来修改这个行为。例如，完全禁用所有日志：

```
/etc/environment
QT_LOGGING_RULES='*=false'
```

若只想禁用调试信息的日志，请使用 `QT_LOGGING_RULES="*.debug=false"`。

图标主题没有生效

从Qt 5.1开始，SVG支持被移动到了其他的模块。由于 qt5-base包 不依赖于 qt5-svg 包，可能会出现 qt5-base包 安装上了，但未安装 qt5-svg包 的情况。这会导致欺骗性的图标行为。由于SVG不受支持，图标的绘制过程被安静地跳过了，因此图标主题可能处于未使用状态。手动安装 qt5-svg包 可以解决这个问题。

主题对root下运行的程序无效

由于用户的主题配置文件 (`$XDG_CONFIG_HOME/Trolltech.conf`) 不被其他的用户读取，你选择的主题不会对 root下运行的X程序生效。以下是几个可能解决方案：

- 创建符号链接，例如

```
# ln -s /home/[username]/.config/Trolltech.conf /etc/xdg/Trolltech.conf
```

- 配置系统级别的主题文件：`/etc/xdg/Trolltech.conf`
- 将主题调整为root

Qt 4 风格未被考虑

如果纯Qt4程序（非KDE程序）不遵循你所选择的Qt4风格，那么你可能需要告诉Qt4如何找到KDE风格（Oxygen等）。只需要设置环境变量 `QT_PLUGIN_PATH`即可。例如：

```
QT_PLUGIN_PATH=$HOME/.kde4/lib/kde4/plugins/:/usr/lib/kde4/plugins/
```

`qtconfig-qt4` 应该能够找到你的kde主题，从而解决这些问题。

或者你也可以将Qt4风格文件夹软链接到KDE4风格文件夹：

```
# ln -s /usr/lib/{kde,qt}4/plugins/styles/theme_name
```

Qt5升级后所有基于Qt5的程序无法启动

如果你得到大概类似以下的报错信息：

```
Qt FATAL: Cannot mix incompatible Qt library (version 0x50900) with this library (version 0x50901)
```

那么很可能是因为你正使用着没有针对最新Qt5重新编译的Qt5主题或风格插件。它们通常使用Qt私有头文件，这意味着它们依赖于特定的Qt版本，而不仅仅是匹配的soname。通过检查 `QT_STYLE_OVERRIDE` 和 `QT_QPA_PLATFORMTHEME` 环境变量找出您正在使用的主题/样式，并重新构建提供它的AUR包。

QXcbConnection: XCB error: 2 (BadValue)

创建含有如下内容的文件：

```
/etc/xdg/QtProject/qtlogging.ini
[Rules]
qt.qpa.xcb.xcberror=false
```

图形未正确对齐或缩放不正确

查看 HiDPI#Qt 5。

死键在Qt程序中不工作

如果你进行了正确的键盘配置，并且死键在GTK应用程序（或其他小部件工具包）中工作，但在KDE或任何Qt应用程序中不起作用，那么你可能没有在Xorg中加载正确的文件。

去确认的一种方法：

1. launch a Qt app with `qt.xkb.compose.debug` logging rule enabled, eg. launching qtqr包: `QT_LOGGING_RULES=qt.xkb.compose.debug=true qtqr`
2. then try to write a character using a dead key, eg. `<dead_circumflex> <e>` for ê (LATIN SMALL LETTER E WITH CIRCUMFLEX)
3. if you encounter `qt.xkb.compose: failed to create compose table` then you probably have this issue.

To fix this, first identify your locale. Then, if your locale doesn't have its own folder in `/usr/share/X11/locale/`, eg. `fr_FR.UTF-8`, look for it in the `compose.dir` mapping file to find the corresponding compose file (eg. `en_US.UTF-8/Compose`):

```
$ grep fr_FR.UTF-8 /usr/share/X11/locale/compose.dir
en_US.UTF-8/Compose             fr_FR.UTF-8
en_US.UTF-8/Compose:            fr_FR.UTF-8
```

Now create or edit `~/.XCompose` to include this compose file:

```
~/.XCompose
include "%S/en_US.UTF-8/Compose"
```

最后，重启Qt 软件, dead keys should be working and `qt.xkb.compose: failed to create compose table` error should have disappeared whenever you debug with `QT_LOGGING_RULES=qt.xkb.compose.debug=true`.

参阅

- Qt官网
- Qt文档
- Planet Qt

13.3 GTK

难度 高级 | **篇幅** 30.1 KB | **所属** 控件工具集

本节将讲到

- 安装
- 主题
- 配置工具
- 配置
- GDK 后端

摘自 GTK 官方网站：

: GTK，或称 GIMP Toolkit，是一个跨平台的图形界面开发库。该库提供一套完整的窗口部件，从一次性工具到大型应用都可使用。

GTK，即 GIMP Toolkit，最初由 GNU 项目为 GIMP 开发，但现在它已经是一个非常流行的工具包，绑定了很多语言。本文将探讨 GTK 主题、风格、图标、字体和字号的配置工具，也会详细介绍手动配置。

安装

目前在官方软件仓库中有三个版本的 GTK。它们可以通过以下软件包来安装：

- **GTK 4.x** 可以通过 gtk4包 包获得。
- **GTK 3.x** 可以通过 gtk3包 包获得。
- **GTK 2.x** 可以通过 gtk2包 包获得。

GTK 1.x 可以通过 AUR 上的 gtkAUR 包获得。

主题

GTK 3 和 GTK 4

GTK 3 和 GTK 4 的默认主题是 Adwaita，但也包含了 HighContrast 和 HighContrastInverse 主题。

要设置特定主题，请通过 dconf 编辑器设置 `org.gnome.desktop.interface` 命名空间下的 `gtk-theme` 属性:

```
$ gsettings set org.gnome.desktop.interface gtk-theme Adwaita
```

注意： 对基于 libadwaita 的 GTK 4 应用程序，需要选择具有 特殊支持 的主题，且需要设置 `GTK_THEME` 环境变量 以强行应用，或者使用打过补丁的libadwaita版本：libadwaita-without-adwaita-gitAUR。

若不使用 dconf 属性，则可以使用 `GTK_THEME` 设置 GTK 3 和 GTK 4 主题。例如将 GNOME 计算器的主题设为 Adwaita 的暗色变体：

```
$ GTK_THEME=Adwaita:dark gnome-calculator
```

提示：要将上述更改应用于桌面快捷方式（或启动器），请参阅 桌面项#编辑环境变量。

注意：如果不使用对应的 XDG_桌面门户 ，则可能需要设置 环境变量 `ADW_DISABLE_PORTAL=1` 方可正确载入通过 GSettings 设置的主题。参见 https://gitlab.gnome.org/GNOME/libadwaita/-/commit/e715fae6a509db006a805af816f9d163f81011ef

GTK 2

GTK 2 的默认主题是 Raleigh，但 Arch Linux 位于 `/usr/share/gtk-2.0/gtkrc` 的自定义配置文件将默认主题设为了 Adwaita。不过，这需要安装 gnome-themes-extra-gtk2AUR 包才能正常工作，否则 GTK 2 应用程序将继续显示 Raleigh 主题。

若要修改 GTK 2 的主题，可以通过配置环境变量 `GTK2_RC_FILES` 来实现。例如将 GIMP 的主题设为 Raleigh：

```
$ GTK2_RC_FILES=/usr/share/themes/Raleigh/gtk-2.0/gtkrc gimp
```

提示：`gtkrc` 也可以是由 #配置工具 在家目录 (/home) 下创建的自定义文件。参见 #示例。

在官方软件仓库或 AUR 可以安装更多主题。可以在 `~/.themes/` 或 `~/.local/share/themes/` 目录中手动提取主题。

GTK 2 与 GTK 3 的主题支持：

- **Adapta** — 基于 Material Design 准则的自适应 GTK 主题。包括: Adapta, Adapta-Eta, Adapta-Nokto, Adapta-Nokto-Eta

: https://github.com/tista500/Adapta || adapta-gtk-theme包

- **Arc** — 具有现代外观和透明元素的平面主题。包括: Arc, Arc-Dark, Arc-Darker

: https://github.com/jnsh/arc-theme || 透明: arc-gtk-themeAUR, 不透明: arc-solid-gtk-themeAUR

- **Bluebird** — Xfce 的蓝色桌面套件。

: https://github.com/shimmerproject/Bluebird || xfce-theme-bluebirdAUR

- **Breeze** — KDE 默认小部件主题的 GTK 版本。包括: Breeze, Breeze-Dark

: https://invent.kde.org/plasma/breeze-gtk || breeze-gtk包

- **Deepin** — 深度桌面的默认主题。包括: deepin, deepin-dark

: https://github.com/linuxdeepin/deepin-gtk-theme || deepin-gtk-theme包

- **GNOME Extra Themes** — GNOME 桌面的附加主题。包括: Adwaita, Adwaita-dark, HighContrast

: https://gitlab.gnome.org/GNOME/gnome-themes-extra || gnome-themes-extra包

- **Greybird** — 灰色和蓝色的 Xfce 主题，在 Xubuntu 12.04 中默认使用。

: https://github.com/shimmerproject/Greybird || xfce-theme-greybirdAUR

- **Materia** — GTK3、GTK2 和 GNOME Shell 的类 Material Design 平面主题。

: https://github.com/nana-4/materia-theme || materia-gtk-theme包

- **MATE Themes** — MATE 桌面的默认主题。包括: BlackMATE, Blue-Submarine, BlueMenta, ContrastHighInverse, Green-Submarine, GreenLaguna, Menta, TraditionalGreen, TraditionalOk

: https://github.com/mate-desktop/mate-themes || mate-themes包

- **Numix** — 具有现代外观的简约轻盈的主题 (GNOME, Openbox, Unity, Xfce)。包括：Numix

: https://github.com/numixproject/numix-gtk-theme || numix-gtk-theme-gitAUR

- **Vertex** — GTK 3、GTK 2、Gnome-Shell 以及 Cinnamon 的主题。

: https://github.com/horst3180/vertex-theme || vertex-themesAUR

- **Zuki** — GTK、gnome-shell 等的主题。

: https://github.com/lassekongo83/zuki-themes || zuki-themesAUR

AUR 中还有很多 GTK 主题，例如：搜索 gtk-theme。

GTK 与 QT

同时在系统上安装 GTK 和 QT（通常是 KDE 组件）程序的人都知道，两者的外观并不怎么协调。关于两者外观统一的问题，参见：统一 Qt 和 GTK 应用程序的外观。

配置工具

大部分主流桌面环境都提供配置 GTK 主题、图标、字体和字体尺寸的工具，并使用 XSettings 管理这些设置：

- 如果使用 Cinnamon，使用主题工具 (*cinnamon-settings themes*): 前往*系统设置* > *主题*.

- 如果使用 Enlightenment：前往*设置* > *全部* > *外观* > *应用程序主题*.
- 如果使用 GNOME，使用 GNOME Tweaks (*gnome-tweaks*): 安装 gnome-tweaks包.
- 如果使用 MATE，使用“外观首选项”工具 (*mate-appearance-properties*): 前往*系统* > *设置* > *外观*.
- 如果使用 Xfce，使用“外观”工具: 前往*设置* > *外观*.

其他 GUI 工具通常会覆写配置文件。

同时支持 GTK 2 与 GTK 3 的：

- **KDE GTK Configurator** — 允许您更改 GTK 2 和 GTK 3 应用程序样式和字体的应用程序。

: https://invent.kde.org/plasma/kde-gtk-config || kde-gtk-config包 : 安装完成后, 在*系统设置* > *外观* > *Application Style* > *Configure GNOME/GTK Application Style* 中能找到 `kde-gtk-config`。

- **LXAppearance** — LXDE 项目中独立于桌面的 GTK 2 和 GTK 3 风格配置工具（不需要 LXDE 桌面的其他部分）。

: https://wiki.lxde.org/en/LXAppearance\[失效链接 2022-09-18 ⓘ\] || lxappearance-gtk3包

- **Oo-mox** — 一款图形应用程序，用于生成 Numix 和 Flat Plat 主题（GTK 2 和 3）、Archdroid 和 Gnome 颜色图标主题的不同颜色变体，也能为 HiDPI 显示生成预缩放的 GTK 2 主题。

: https://github.com/actionless/oomox || themix-full-gitAUR

只支持 GTK 2 的：

- **GTK Change Theme** — 能改变 GTK 2.0 主题的小程序（被认为是 *switch2* 之外的更好选择）。

: http://plasmasturm.org/code/gtk-chtheme/ || gtk-chtheme包

- **GTK Preference Tool** — GTK 主题选择器和字体切换器。

: https://gtk-win.sourceforge.io/home/index.php/Main/GTKPreferenceTool || gtk2_prefsAUR

- **GTK Theme Switch** — 简易 GTK 主题切换器。

: http://muhri.net/nav.php3?node=gts || gtk-theme-switch2AUR

配置

GTK 设置可以在配置文件中手动指定，但桌面环境和应用程序可以覆盖这些设置。根据 GTK 版本，这些文件位于：

- GTK 2 用户特定设置: `~/.gtkrc-2.0`
- GTK 2 系统全局设置: `/etc/gtk-2.0/gtkrc`
- GTK 3 用户特定设置: `$XDG_CONFIG_HOME/gtk-3.0/settings.ini` (如果 `$XDG_CONFIG_HOME` 未设置，则为 `$HOME/.config/gtk-3.0/settings.ini`)
- GTK 3 系统全局设置: `/etc/gtk-3.0/settings.ini`

注意：

- 有关当前支持的 GTK 配置选项的完整列表，请参阅 GTK 编程参考手册中的 GTK4 和 GTK3 *GtkSettings* 属性（以及 GTK 2 属性）。
- 自 GTK 3.10 以来，以下描述的一些设置 (例如 `gtk-icon-sizes`) 已弃用并被忽略。
- 如果编辑 GTK 配置文件，只有新启动的应用程序才会显示更改。

基本主题配置

要手动改变 GTK 主题、图标、字体和字体大小，请在配置文件中添加以下内容：

- GTK 2:

```
~/.gtkrc-2.0
gtk-icon-theme-name = "Adwaita"
gtk-theme-name = "Adwaita"
gtk-font-name = "DejaVu Sans 11"
```

- GTK 3:

```
$XDG_CONFIG_HOME/gtk-3.0/settings.ini
[Settings]
gtk-icon-theme-name = Adwaita
gtk-theme-name = Adwaita
gtk-font-name = DejaVu Sans 11
```

如果主题没有应用于 GTK 3，请使用 `gsettings`:

```
$ gsettings set org.gnome.desktop.interface gtk-theme Pop
```

注意：图标主题名称是其目录的名称，而不是其 index.theme 中的 name 属性。

深色版主题

有些 GTK 3 的主题包含深色版本, 但只有应用程序明确要求时才会使用. 要在所有 GTK 3 程序中应用深色模式, 请使用以下配置:

```
gtk-application-prefer-dark-theme = true
```

对于 GTK 4，请使用以下命令：

```
$ gsettings set org.gnome.desktop.interface color-scheme prefer-dark
```

键盘快捷键

把鼠标放在某个菜单项，然后按下某个按键组合，即可修改该项目的快捷键。不过，该功能默认是关闭的。启用方法是使用以下配置：

```
gtk-can-change-accels = 1
```

Emacs 按键绑定

要在 GTK 应用程序中拥有类似 Emacs 的键绑定，请添加以下内容：

```
~/.gtkrc-2.0
gtk-key-theme-name = "Emacs"
~/.config/gtk-3.0/settings.ini
[Settings]
gtk-key-theme-name = Emacs
```

对于 GTK3 还需要运行：

```
$ gsettings set org.gnome.desktop.interface gtk-key-theme "Emacs"
```

Xfce 也需要类似的设置：

```
$ xfconf-query -c xsettings -p /Gtk/KeyThemeName -s Emacs
```

`/usr/share/themes/Emacs/` 中的配置文件决定了 Emacs 的键位绑定，并且可以更改。您也可以把其中的内容复制到用户目录下的 `~/.gtkrc-2.0` 文件中来为个别用户进行自定义。

GNOME 菜单延迟

鼠标指向菜单与菜单显示之间有一定延迟，通过以下设置调整延迟时间（以毫秒为单位）：

```
gtk-menu-popup-delay = 0
```

缩小窗口部件

如果屏幕很小，不喜欢过大的图标和窗口部件，可以调整其尺寸。

若要让工具栏仅显示图标而不显示文字（valid values）：

```
gtk-toolbar-style = GTK_TOOLBAR_ICONS
```

若要显示小图标：

```
gtk-icon-sizes = "panel-menu=16,16:panel=16,16:gtk-menu=16,16:gtk-large-toolbar=16,16\
:gtk-small-toolbar=16,16:gtk-button=16,16"
```

若要移除按钮上的图标：

```
gtk-button-images = 0
```

若要移除菜单上的图标：

```
gtk-menu-images = 0
```

另见 和 .

隐藏 CSD 按钮

若要删除 *gtk3* 窗口的客户端装饰 (CSD) 最小化和最大化按钮：

```
gtk-decoration-layout=menu:close
```

参见 GTK 文档。

禁用鼠标中键粘贴

若要关闭鼠标中键（主键）粘贴：

```
gtk-enable-primary-paste=false
```

文件选择器启动时的初始位置

在**当前工作目录**而不是**最近**位置内打开文件选择器。通常来说，**当前工作目录**是家 *(home)* 目录。

GTK 3

通过如下命令，更改设置：

```
$ gsettings set org.gtk.Settings.FileChooser startup-mode cwd
```

GTK 2

将下面内容加入 `~/.config/gtk-2.0/gtkfilechooser.ini`：

```
StartupMode=cwd
```

传统滚动操作

注意：并非所有的 GTK 应用程序都遵从这个设置。

提示：只需要用右键代替左键，就能可靠地实现传统的滚动操作。

在 GTK 3.6 之前，点击滚动条中滑块的任何一边都会使滚动条向点击的方向滚动大约一页。从 GTK 3.6 开始，滑块会直接移动到点击的位置。通过创建包含以下内容的文件，可以在一些应用程序中恢复这种操作：

```
~/.config/gtk-3.0/settings.ini
[Settings]
gtk-primary-button-warps-slider = false
```

Disable overlay scrollbars

Since GTK 3.15, overlay scrollbars are enabled by default, meaning that scrollbars will be shown only on mouseover in GTK 3 applications. This behavior can be reverted by setting the following environment variable: `GTK_OVERLAY_SCROLLING=0`. See Environment variables#Graphical environment.

Alternatively, overlay scrollbars can be disabled in the GTK 3 settings since GTK 3.24.9. To do so, the value of gtk-overlay-scrolling has to be set to false in the \[Settings\] section of the settings file:

```
~/.config/gtk-3.0/settings.ini
[Settings]
gtk-overlay-scrolling = false
```

GTK 4 will no longer support `GTK_OVERLAY_SCROLLING`. It has already been dropped from master. As of GTK 4, the overlay nature of the scrollbars is part of the toolkit. The blanket toggle has been removed to prevent developers from breaking

applications that have not been tested with both combinations. To allow application developers to decide what their applications should look like, the toolkit instead provides a mechanism to opt-out or add a setting for users. The function gtk_scrolled_window_set_overlay_scrolling() can be used to enable/disable overlay scrolling on a *per-application* basis. Application developers can optionally use GSettings to have a user setting bound to the property.

Remove overlay scroll indicators

The positions of the overlay scrollbars are indicated by thin dashed lines in the application window. These dashed lines will be present even when overlay scrolling is disabled using the environment variable discussed in the section above. To remove the indicator lines, create the following file:

```
~/.config/gtk-3.0/gtk.css
/* Remove dotted lines from GTK 3 applications */
undershoot.top, undershoot.right, undershoot.bottom, undershoot.left { background-image: none; }
```

示例

GTK example configurations:

注意：May be ignored by some desktop environments (e.g. GNOME).

```
~/.gtkrc-2.0
gtk-theme-name="Arc-Dark"
gtk-icon-theme-name="breeze-dark"
gtk-font-name="Sans 11"
gtk-cursor-theme-name="Breeze_Amber"
gtk-cursor-theme-size=0
gtk-toolbar-style=GTK_TOOLBAR_BOTH_HORIZ
gtk-toolbar-icon-size=GTK_ICON_SIZE_SMALL_TOOLBAR
gtk-button-images=0
gtk-menu-images=0
gtk-enable-event-sounds=0
gtk-enable-input-feedback-sounds=0
gtk-xft-antialias=1
gtk-xft-hinting=1
gtk-xft-hintstyle="hintslight"
gtk-xft-rgba="rgb"
~/.config/gtk-3.0/settings.ini
[Settings]
gtk-theme-name=Arc-Dark
gtk-icon-theme-name=breeze-dark
gtk-font-name=Sans 11
gtk-cursor-theme-name=Breeze_Amber
gtk-cursor-theme-size=0
gtk-toolbar-style=GTK_TOOLBAR_BOTH_HORIZ
gtk-toolbar-icon-size=GTK_ICON_SIZE_SMALL_TOOLBAR
gtk-button-images=0
gtk-menu-images=0
gtk-enable-event-sounds=0
gtk-enable-input-feedback-sounds=0
gtk-xft-antialias=1
gtk-xft-hinting=1
gtk-xft-hintstyle=hintslight
gtk-xft-rgba=rgb
# gtk-decoration-layout=menu:close
# gtk-application-prefer-dark-theme=1
```

GDK 后端

GDK（GTK 的底层抽象层）支持多个后端显示 GTK 应用。

Wayland 后端

GDK Wayland 后端仅由 gtk3包 支持，并且是使用 Wayland 显示服务器时的默认后端。

使用 gtk3包 之前的 GTK 版本的应用程序不支持 Wayland，并且需要通过 Xwayland 才能使用 *X11* 后端在 Wayland 会话上运行。

使用 Wayland 后端时，一些变量不会从 `settings.ini` 读取。 Any key that is present in the GSettings schema `org.gnome.desktop.interface` are read from there instead of `settings.ini`.

An example of such variables are `gtk-color-scheme` and `icon-theme`, which must have their keys set with GSettings in order to theme GTK Applications under Wayland. 或者，如果只需要自定义主题，可以设置环境变量 `GTK_THEME`。

Xorg 后端

如果正在使用 Xorg 显示服务器，则后端自动默认为 *x11*。

通过设置环境变量 `GDK_BACKEND=x11`，可以强制 Wayland 会话的 GTK3 应用程序通过 Xwayland 使用 *X11* 后端。

Broadway 后端

The GDK Broadway backend provides support for displaying GTK applications in a web browser, using HTML5 and web sockets.

When using *broadwayd*, specify the display number to use, prefixed with a colon, similar to X. The default display number is 0 (zero).

```
$ display_number=:5
```

Start it.

```
$ broadwayd $display_number
```

Port used by default

```
port = 8080 + $display_number
```

Point your browser to http://127.0.0.1:port

To Start applications

```
$ GDK_BACKEND=broadway BROADWAY_DISPLAY=$display_number <<application>>
```

Alternatively can set address and port

```
$ broadwayd --port $port_number --address $address $display_number
```

疑难解答

Different themes between GTK 2 and GTK 3 applications

In general, if a selected theme has support for both GTK 2 and GTK 3, the theme will be applied to all GTK 2 and GTK 3 applications. If a selected theme has support for only GTK 2, it will be used for GTK 2 applications and the default GTK theme will be used for GTK 3 applications. If the selected theme has support for only GTK 3, it will be used for GTK 3 applications and the default GTK theme will be used for GTK 2 applications. Thus for application theme consistency, it is best to use a theme which has support for both GTK 2 and GTK 3.

You could find what themes installed on your system have both an GTK 2 and GTK 3 version by using this command (does not work with names containing spaces):

```
find $(find ~/.themes /usr/share/themes/ -wholename "*/gtk-3.0" | sed -e "s/
^\(.*\)\/gtk-3.0$/\1/") -wholename "*/gtk-2.0" | sed -e "s/.*\/\(.*\)\/gtk-2.0/\1"/
```

Theme not applied to root applications

As user theme files (`$XDG_CONFIG_HOME/gtk-3.0/settings.ini`, `~/.gtkrc-2.0`) are not read by other accounts, the selected theme will not apply to X applications run as root. Possible solutions include:

- Create symlinks, e.g

```
# ln -s /home/[username]/.gtkrc-2.0 /etc/gtk-2.0/gtkrc
# ln -s /home/[username]/.config/gtk-3.0/settings.ini /etc/gtk-3.0/settings.ini
```

- Configure system-wide theme files: `/etc/gtk-3.0/settings.ini` (GTK 3) or `/etc/gtk-2.0/gtkrc` (GTK 2)
- Adjust the theme as root

```
# lxappearance
```

- Use a settings daemon (this is what most desktop environments do). A desktop-agnostic variant using XSettings is available in the AUR under xsettingsd-gitAUR.

Client-side decorations

GTK 3.12 introduced client-side decorations, which move the title-bar away from the window manager. This may present issues such as double title-bars, no title-bar at all, double shadows with compositing enabled, or being unable to move a frozen application.

To remove the shadow and gap around windows (for example in combination with a tiling window manager), create the following file:

```
~/.config/gtk-3.0/gtk.css
.window-frame, .window-frame:backdrop {
 box-shadow: 0 0 0 black;
 border-style: none;
 margin: 0;
 border-radius: 0;
}

.titlebar {
```

```
 border-radius: 0;
}

.window-frame.csd.popup {
  box-shadow: 0 1px 2px rgba(0, 0, 0, 0.2), 0 0 0 1px rgba(0, 0, 0, 0.13);
}

.header-bar {
  background-image: none;
  background-color: #ededed;
  box-shadow: none;
}
/* You may want to use this if you don't like the double title.
GtkLabel.title {
    opacity: 0;
}*/
```

Note that if visual problems persist, you may want to use the GTK Inspector to find the offending elements as explained here .

To adjust the buttons in the header bar, use the `gtk-decoration-layout` setting. The below examples removes all buttons:

```
~/.config/gtk-3.0/settings.ini
gtk-decoration-layout=menu:
```

To remove client-side decorations altogether, it's possible to use a patched library like gtk3-classicAUR or gtk3-nocsd-gitAUR.

cedilla ç/Ç instead of ć/Ć

See , and for a workaround using Xcompose (US international layout).

Suppress warning about accessibility bus

If you do not use any Gnome Accessibility features, you may receive warnings like:

```
WARNING **: Couldn't connect to accessibility bus:
```

To suppress these warnings, execute programs with `NO_AT_BRIDGE=1` or set that as a global environment variable.

Titlebar background color mismatch

If you are using a window manager which uses a window decoration theme that mimics the GTK theme background color, you may find that the titlebar color no longer completely matches the application color in some GTK 3 applications. As a workaround, create the following file:

```
~/.config/gtk-3.0/gtk.css
/* Always use background color */
GtkWindow {
    background-color: @theme_bg_color;
}

/* Fix tooltip background override */
.tooltip {
    background-color: rgba(0, 0, 0, 0.8);
}

.tooltip * {
    background-color: transparent;
}

/* Fix Nautilus desktop window background override */
NautilusWindow {
```

```
    background-color: transparent;
}
```

Wrong focus events with tiling window managers

注意：This disables smooth scrolling and touchscreen support for GTK3 applications.

Define `GDK_CORE_DEVICE_EVENTS=1` to use GTK2 style input, instead of xinput2.

Thumbnail support for GTK file dialog

Install gtk2-patched-filechooser-icon-viewAUR and gtk3-patched-filechooser-icon-viewAUR to have the option to view files as thumbnails instead of list in the GTK file chooser.

Button and menu icons

本文或本章节的事实准确性存在争议。

原因： Explain what the issue is. GNOME ignores `settings.ini` if GDM is used.（在 Talk:GTK 中讨论）

For some applications in GNOME's Wayland session. Your `~/.config/gtk-3.0/settings.ini` file is misconfigured. This can happen if you try other GTK based desktop environments. These are the offending values:

```
~/.config/gtk-3.0/settings.ini
[Settings]
gtk-button-images=1
gtk-menu-images=1
```

Simply set them to 0 or remove the whole file to use GNOME defaults.

GTK 3 without polkit

GTK3 depends on polkit through colord, which is required for printing. However printing works fine without polkit installed; at least with a monochrome printer and package versions gtk3-print-backends=3.22.19-2 and colord=1.4.1-1.

Some GTK 2 themes only change the UI color palette

Depending on the theme of choice's support for GTK 2, UI controls may still have the default Raleigh appearance, possibly with a different color palette. This is due to these themes requiring the GTK 2 Murrine engine, which is missing (GTK 2 programs should complain about it on their standard error output). Install the gtk-engine-murrine包 package.

Patching GTK file chooser to use regular type ahead

GTK file chooser uses the same type-ahead-find feature as GNOME/Files. This can be very jarring and does not fit in very well with other desktop enviroments.

Some applications support XDG-desktop-portal which allows application to use the native file chooser. If that does not work you can restore type-ahead functionality by using a patched GTK, for example gtk3-classicAUR.

Text in GTK 4 applications is blurry or renders incorrectly

GTK 4 switched to grayscale antialiasing without hinting when rendering fonts. A setting is available that will restore some of the GTK 3 behavior . Subpixel antialiasing is not available.

```
~/.config/gtk-4.0/settings.ini
[Settings]
gtk-hint-font-metrics=1
```

参见

- GTK 官方网站
- 维基百科对 GTK 的介绍

终端多路复用器

并行跑编译、翻日志、调远端时，反复开终端容易丢会话；本章给出三条不同层次的路线。建议精读 Tmux：篇幅最长，配置、会话初始化、剪贴板集成与默认 TERM 都在其中，是主力方案。Abduco 仅 1.7KB，只想要后台挂着、断开再回来的读者读它即可。Kitty 可按需查阅，用它当终端的人看安装与配置两节，其余可跳。最容易踩的坑是把三者当成一层：Abduco 没有分屏和配置体系，指望它有 Tmux 式窗格会失望；Tmux 的默认 TERM 配错会让远端颜色和快捷键失灵；剪贴板集成同样要单独配置。

14.1　Abduco

难度 入门 | **篇幅** 1.7 KB | **所属** 终端多路复用器

本节将讲到

- 安装
- 会话管理
- 限制

abduco 是一个轻量级工具，可让程序独立于控制终端运行。在这方面，它类似于 tmux 和 GNU Screen，但它没有多路复用功能。

安装

安装 abduco包 软件包。

会话管理

要在名为 `*session_name*` 的会话中启动 `*command...*`（稍后可能会引用该会话），请运行

```
$ abduco -c session_name command...
```

要退出进程，请按 `Ctrl+\` 或关闭控制终端。在这两种情况下，进程都将保持存活。

要重新连接到 `*session_name*`，请运行

```
$ abduco -a session_name
```

在不带任何选项的情况下调用 `abduco` 会打印会话列表。

限制

在 *abduco* 会话中启动的进程会保留其调用的 `TERM` 变量。重新连接时，必须使用兼容的终端类型。

abduco 不提供多窗口和多路复用功能。如果需要，可使用 dvtm、*tmux* 或 GNU Screen。

14.2 Kitty

难度 进阶 | **篇幅** 6.5 KB | **所属** 终端多路复用器

本节将讲到

- 安装
- 用法
- 配置
- 提示和技巧
- 疑难解答

kitty 是一个基于 OpenGL 的可编程终端模拟器，具有真彩色，连字支持，键盘输入和图像渲染的协议扩展。它还提供了类似于 GNU Screen 或 tmux 的平铺功能。

安装

安装 kitty包；若要获取开发版本，请安装 kitty-gitAUR。

用法

可以通过各种 `Ctrl+Shift` 快捷键来创建新的选项卡和窗口并调整其大小。布局可通过 `Ctrl+Shift+l` 切换，并可以保存/恢复。

*全键盘模式*可区分 `Ctrl+i` 和 `Tab` 等歧义键。此外，新的文本效果（例如，波浪下划线）也可用于支持它的应用程序。

Kitten

kitty 有一个用于创建子程序的框架，称为 kitten（小猫）。所有 kitten 命令均以 `kitty +kitten` 为前缀，很方便作为 shell 别名。

icat

这个 kitten 基于 kitty 图形协议。其依赖 ImageMagick。要在终端内显示图像：

```
$ kitty +kitten icat image.jpg
```

其也可以在终端内显示 gif 动图。除了图片文件，您也可以传递目录或图片 URL。这也可以在远程服务器上通过 ssh 显示图片。一些应用程序像 ranger 和 neofetchAUR 使用该协议在终端内显示图片。关于更多信息，请参阅官方文档。

diff

这个 kitten 需要 git 或 diffutils包 被安装。此外，您也可以安装 python-pygments包 来获得语法高亮效果。要显示两个文件的差异：

```
$ kitty +kitten diff file1 file2
```

这个 kitten 显示了图形或文本的差异，也可以通过 ssh 使用，您可以用一个目录代替文件来显示递归差异。关于更多信息，参阅官方文档。

clipboard

这个 kitten 用于读写系统剪贴板，可以跨 ssh 使用。如果要将标准输入复制到剪贴板：

```
$ echo "Hello" | kitty +kitten clipboard
```

将当前剪贴板内容输出到标准输出：

```
$ kitty +kitten clipboard --get-clipboard
```

默认情况下，这条命令会提示请求权限，要禁用该提示，请在配置文件中编辑 clipboard_control 选项：

```
~/.config/kitty/kitty.conf
clipboard_control write-clipboard read-clipboard
```

关于更多信息，请参阅官方文档。

配置

kitty 在 `~/.config/kitty/kitty.conf` 中存储配置，默认配置位于 `/usr/share/doc/kitty/kitty.conf`。可以调整字体，颜色，光标和回滚行为。您可以在官方文档或 kitty.conf(5) 中查看所有可用的选项。

注意：若您正在使用 vim 编辑默认配置文件，所有的节开始都会被折叠，每个节默认都可以用 `zo` 展开。

提示：

- 位于 `/usr/share/doc/kitty/kitty.conf` 的默认配置文件自带每个选项的说明。
- 官方文档在本地具有副本，位于 `/usr/share/doc/kitty/html/index.html`。

提示和技巧

输入法兼容

Kitty 默认情况下关闭了 IBus 框架。

请在环境变量中启用：

```
GLFW_IM_MODULE=ibus
```

由于兼容问题，该环境变量也可解决 kitty 对 fcitx5 的兼容问题。

单实例模式

这类似于守护进程模式，当以 `--single-instance` 或 `-1` 选项启动 kitty 时，仅会运行一个 kitty 实例，随后以相同选项启动 kitty 时会创建已经存在的 kitty 实例的新窗口。这可以降低内存占用，因为共享了 GPU 缓存，而且会减少启动时间。您可以用 `--instance group *name*` 选项创建多组 kitty 实例。参阅 kitty(1) § single 以获取更多信息。

疑难解答

SSH 连接故障

当 kitty 用于 ssh 连接到没有其 terminfo 的远程主机时，可能会出现各种问题。解决方案通常是复制 terminfo。Kitty 有一个 ssh 小工具来自动化这一过程。

```
$ kitty +kitten ssh user@host
```

您可以将其设置为 ssh 的别名。实现这一点的一种方法是检测用户是否在使用 Kitty，如果是，则为 ssh 命令设置别名。为此，您需要将以下行附加到您的 `~/.bashrc` 或 `~/.zshrc` 文件中：

```
[ "$TERM" = "xterm-kitty" ] && alias ssh="kitty +kitten ssh"
```

背景色在 vim 中消失

在 vim 中使用带背景色的配色方案后，滚动时背景色可能会消失或闪烁。欲修复，请确保 `TERM` 环境变量依然被设为 `xterm-kitty`，然后添加以下行到您的 `.vimrc` 文件：

```
~/.vimrc
let &t_ut=''
```

相关 bug 报告：Github issue #108，kitty FAQ。

无法识别位图字体

因为 kitty 的基本特性是以任意大小显示字体，而位图字体不适合缩放，故 kitty 不支持位图字体，参阅 Github issue #97。

参见

- 官方网站
- GitHub 仓库

14.3 Tmux

难度 高级 | **篇幅** 31.4 KB | **所属** 终端多路复用器

本节将讲到

- 安装
- 配置
- 会话初始化
- 剪贴板集成
- 提示与技巧

tmux 是终端多路复用器，可在一个屏幕中创建、访问并控制多个终端（或窗口），每个终端或窗口内都可以运行独立的程序。将 tmux 从当前窗口分离（tmux detach）后，tmux 依然可以在后台运行，直到恢复会话（tmux attach）。 使用 ISC 协议的

tmux 是 GNU Screen 的替代品。尽管两者相似，但依然有许多不同之处（参考 tmux FAQ page）。

安装

安装 tmux包 软件包。 可选择安装 tmux-bash-completion-gitAUR 软件包用于提供 bash 补全功能。

配置

自 3.2 版本起，tmux 默认从 `$XDG_CONFIG_HOME/tmux/tmux.conf`，然后是 `~/.config/tmux/tmux.conf` 寻找用户配置文件。全局配置文件位于 `/etc/tmux.conf`，但 Archlinux 并无此文件。

快捷键绑定

命令快捷键的前缀默认是 `Ctrl+b`。

提示：可以使用默认的帮助快捷键 `Ctrl+b ?` 查询所有可用的快捷键绑定。

要垂直分割一个窗口，请按下 `Ctrl+b %`。

将窗口分割成多个面板（panes）后，通过按下命令前缀（例如 `Ctrl+b`），然后按住 `Ctrl` 不放的同时，按下方向键 `Left`、`Right`、`Up` 或 `Down` 调整面板大小。交换不同的面板与此类似，只需将调整大小环节中所按的方向键改为 `o` 键即可。

按键绑定可以在 `tmux.conf` 配置文件中通过 bind 与 unbind 命令修改。例如，将下面的示例配置添加到用户配置文件中可将默认的前缀绑定 `Ctrl+b` 更改为 `Ctrl+a`：

```
unbind C-b
set -g prefix C-a
bind C-a send-prefix
```

提示：使用特殊字符作为前缀时，请用 `Alt`（也被称为 Meta）替换 `Ctrl`，例如：`set -g prefix m-'\'`。

使用 `Ctrl+b c` 创建一个新的窗口（window），通过快捷键 `Ctrl+b n` 与 `Ctrl+b p` 切换到下一个或上一个窗口。

下面还有一些用于在窗口间移动的其他方法：

```
Ctrl+b l（移动到上一个使用的窗口）
Ctrl+b w（列出所有窗口与窗口编号）
Ctrl+b <窗口编号>（移动到指定编号的窗口，默认编号是 0 到 9）
Ctrl+b q（显示面板编号，在面板编号显示的同时按下对应编号可以切换到该面板）
```

tmux 有窗口搜索选项与快捷键绑定，便于在多个窗口之间切换：

```
Ctrl+b f <窗口名称>（按名称搜索窗口）
Ctrl+b w（从交互式窗口列表中选择窗口）
```

复制模式

tmux 中的窗口可能会处于几种模式中的一种。默认的模式下可以直接访问窗口中的终端。另一种模式是“复制模式”。在复制模式下用户可以浏览缓冲区、滚动浏览历史。复制模式下所使用的快捷键是 vi 或是 emacs 风格的。除非用户在 VISUAL 或是 EDITOR 变量中设置了 vi，否则复制模式中默认的快捷键绑定是 emacs 风格。

按如下快捷键进入复制模式：

```
Ctrl+b [
```

然后就可以像在自己的编辑器里一样浏览缓冲区内容了。

使用如下之一的快捷键退出复制模式：

vi 模式下请使用：

```
q
```

emacs 模式下则是：

```
Esc
```

打开 URL

安装并配置好 urlviewAUR 后才能在 tmux 中打开 URL。以下是配置示例：

在一个新的终端中：

```
bind-key u capture-pane \; save-buffer /tmp/tmux-buffer \; run-shell "$TERMINAL -e urlview /tmp/tmux-buffer"
```

在一个新的 tmux 窗口中（无需打开新的终端）：

```
bind-key u capture-pane \; save-buffer /tmp/tmux-buffer \; new-window -n "urlview" '$SHELL -c "urlview < /tmp/tmux-buffer"'
```

设置正确的终端

256 色

如果用户使用的是 256 色终端，应在 tmux 中设置正确的终端如：`tmux` 或 `tmux-256color`：

```
tmux.conf
set -g default-terminal "tmux-256color"
```

也可以使用 `tmux -2` 强制让 tmux 认定终端支持 256 色防止显示混乱。

24 位深颜色

tmux 支持 24 位深颜色。如果用户使用的终端支持这一特性（请参考这个链接），请将它添加至 `terminal-features`。

例如，对于 Alacritty 终端，请添加以下内容：

```
set -as terminal-features ",alacritty*:RGB"
```

对于其他终端，请使用 $TERM 的值替换上述的 alacritty。

关于 RGB 的详细信息，请查阅 tmux(1)。

xterm-keys

将下面这一行内容添加至配置文件以启用 xterm-keys：

```
tmux.conf
set-option -g xterm-keys on
```

注意，如果在 tmux.conf 中启用了 xterm-keys，用户需要自行构建一个自定义的 terminfo 用于声明新的转译码，否则应用程序将识别不到。使用 tic 编译以下内容之后，就可以将“xterm-screen-256color”作为 TERM 使用了：

```
# 一个以 screen- 前缀命名的 TERMINFO，
# 用于声明由 tmux 配置“set-window-option -g xterm-keys”所启用的转译序列。
#
# 以 xterm- 前缀命名，因为一些应用程序除了检查终端功能，
# 还会检查 TERM 名称。
xterm-screen-256color|GNU Screen with 256 colors bce and tmux xterm-keys,

# As of Nov'11, the below keys are picked up by
# .../tmux/blob/master/trunk/xterm-keys.c:
       kDC=\E[3;2~, kEND=\E[1;2F, kHOM=\E[1;2H,
       kIC=\E[2;2~, kLFT=\E[1;2D, kNXT=\E[6;2~, kPRV=\E[5;2~,
       kRIT=\E[1;2C,

# 如果所用的终端不支持 bce，
# 请将下面的内容改为 screen-256color：
       use=screen-256color-bce,
```

使用 tic -c 检查所使用的终端是否支持 bce：

```
$ tic -c xterm-screen-256color
"xterm-screen-256color", line 16, terminal 'xterm-screen-256color': resolution of
use=screen-256color-bce failed
```

将上述内容保存后使用 tic 编译：

```
$ tic xterm-screen-256color
```

文件编译完成后会被保存到 $HOME/.terminfo（若以 root 运行，则会保存到 /usr/share/terminfo/ 中，可全局使用）。

定制主题

可以为 tmux 定制主题，但首先需要知道颜色代码。在终端中执行以下命令，将输出颜色代码与其对应的颜色示例：

```
$ for i in {0..255}; do printf "\x1b[38;5;${i}mcolor${i} - ██████████\n"; done
```

上述命令所输出的颜色代码可用于修改 tmux 的颜色主题。下面是一个修改状态栏颜色的示例：

```
# 状态栏颜色
set -g status-bg "color4"          # 蓝色背景
set -g status-fg "color7"          # 灰色文字
set -g status-right "%l:%M %p"     # 时间格式
```

```
set-window-option -g window-status-current-style "bg=color75,fg=color231 bold"    # 当前窗口的背景和前景色
```

面板边框也可以通过以下的方式设置主题颜色：

```
# 边框颜色
set -g pane-border-style        fg="colour255"
set -g pane-active-border-style fg="colour33"
```

其他设置

限制滚动缓冲区上限至最多 10000 行：

```
set -g history-limit 10000
```

切换鼠标支持：

```
bind-key m set-option -g mouse \; display "Mouse: #{?mouse,ON,OFF}"
```

随 systemd 启动

开机启动 tmux 有如下的好处。例如，创建一个新的 tmux 会话时，开机启动的 tmux 服务能缩减启动延迟。

除此以外，如有特殊需求（例如需要加载大量的 tmux 配置项或是有共享的用户会话），开机启动 tmux 能够在用户不登录的情况下恢复并持续运行这些会话。

下面这个服务为特定用户（例如，启动或启用 `tmux@*username*.service`）启动 tmux：

```
/etc/systemd/system/tmux@.service
[Unit]
Description=tmux session for user %I

[Service]
Type=forking
User=%I
# 警告：这里请使用 %I 而不要使用 %u，在系统服务中 %u=root。
ExecStart=/usr/bin/tmux new-session -s %I -d
ExecStop=/usr/bin/tmux kill-session -t %I

[Install]
WantedBy=multi-user.target
```

提示：

- 可以添加 `WorkingDirectory=*自定义路径*` 以自定义工作目录。若设置为 `~`，将使用 `User=` 中指定的用户家目录。
- 添加 `-v` 参数可在工作目录中生成 tmux 客户端和服务端的日志。
- 若希望引入图形会话的相关环境变量，例如 `DISPLAY`、`WAYLAND_DISPLAY` 或 `SESSION_MANAGER`，请将 `WantedBy` 选项修改为 `graphical-session.target`，并在 `[Unit]` 下面添加 `After=graphical-session.target`。
- 也可以创建一个在桌面环境（也可以是窗口管理器或者 Wayland 混成器）启动后自动执行的脚本，并使用 `tmux setenv -g name [value]` 加载并引入相关的环境变量。

也可以将此文件存放至 systemd/用户目录中（此时不需要 `User=%I`，同时还需要将 `WantedBy` 中的 `multi-user.target` 修改为 `default.target`），例如

`~/.config/systemd/user/tmux.service`。此时 tmux 服务会在用户登录时启动，并且由于这个原因，用户服务会保持运行状态，登出并不会停止服务。除此以外还可以设置随系统自动启动_systemd_用户实例。

会话初始化

将下面的内容添加到 `~/.tmux.conf` 中可在打开一个新的 tmux 会话时含有预加载的窗口：

```
new  -n WindowName Command
neww -n WindowName Command
neww -n WindowName Command
```

要让打开的新会话包含多个分割窗口，请将 `splitw` 命令添加至需要分割的 `neww` 下方，像这样：

```
new  -s SessionName -n WindowName Command
neww -n foo/bar foo
splitw -v -p 50 -t 0 bar
selectw -t 1
selectp -t 0
```

将会打开两个窗口，其中第二个窗口的名称是 foo/bar，且会被垂直平均分割成 foo 和 bar 上下两个面板，foo 在 bar 的上方。窗口焦点会在窗口 2（foo/bar）内上方的面板（foo）中。

注意：会话、窗口和面板的序号都是从 0 开始，除非用户在 `.conf` 中指定序号从 1 开始。

在配置文件中 source 不同的会话文件可以实现多个会话的管理：

```
# 初始化多个会话
bind F source-file ~/.tmux/foo
bind B source-file ~/.tmux/bar
```

剪贴板集成

提示：tmux 插件 tmux-yank 提供了类似的功能。

可以将 tmux 中选择的内容复制到显示服务的剪贴板（both primary/secondary selections），再粘贴回 tmux。下面的配置文件片段提供了将 X11 和 Wayland 剪贴板与当前的 tmux 集成的方法：

注意：下面的 Vim 风格示例中，需要使用 `unbind p` 解除“切换至上一个窗口”的快捷键的绑定，才能正常使用粘贴功能。

与 Xorg 集成

首先可以考虑使用 xsel包：

Emacs 风格:

```
bind-key -T copy-mode y send-keys -X copy-pipe-and-cancel "xsel -i -p && xsel -o -p | xsel -i -b"
bind-key C-y run "xsel -o | tmux load-buffer - ; tmux paste-buffer"
```

Vim 风格:

```
bind-key -T copy-mode-vi y send-keys -X copy-pipe-and-cancel "xsel -i -p && xsel -o -p
| xsel -i -b"
bind-key p run "xsel -o | tmux load-buffer - ; tmux paste-buffer"
```

也可以使用 xclip包 达到相同的目的。与 xsel 不同，xclip 在处理与目前区域设置（locale）不兼容的原始比特流时更好一些。尽管如此，使用 xsel 比 xclip 更干净整洁些，这是因为 xclip 在读取 tmux 缓冲区内容后并不会关闭 `STDOUT`。这样一来，tmux 就无法知道复制是否完成，并一直等待 xclip 终止，看起来就像是 tmux 不响应了一样。解决方法是将 `STDOUT` 重定向到 `/dev/null`：

Emacs 风格:

```
bind-key -T copy-mode y send-keys -X copy-pipe-and-cancel "xclip -i -sel clip
> /dev/null"
bind-key C-y run "xclip -o -sel clip | tmux load-buffer - ; tmux paste-buffer"
```

Vim 风格:

```
bind-key -T copy-mode-vi y send-keys -X copy-pipe-and-cancel "xclip -i -sel clip
> /dev/null"
bind-key p run "xclip -o -sel clip | tmux load-buffer - ; tmux paste-buffer"
```

与 Wayland 集成

首先请确保安装了 wl-clipboard包。

Emacs 风格:

```
bind-key -T copy-mode y send-keys -X copy-pipe-and-cancel "wl-copy && wl-paste -n | wl-
copy -p"
bind-key C-y run "wl-paste -n | tmux load-buffer - ; tmux paste-buffer"
```

Vim 风格:

```
bind-key -T copy-mode-vi y send-keys -X copy-pipe-and-cancel "wl-copy && wl-paste -n |
wl-copy -p"
bind-key p run "wl-paste -n | tmux load-buffer - ; tmux paste-buffer"
```

在 Urxvt 中使用中键粘贴

注意：启用鼠标支持后才能使用此功能。

tmux 有一个非官方的 perl 扩展（在 tmux 的官方 FAQ 中有提到），可在 urxvt 中通过鼠标中键粘贴剪贴板内容。

首先，下载 perl 脚本并将其保存到 urxvt 的 perl 库中：

```
wget http://anti.teamidiot.de/static/nei/*/Code/urxvt/osc-xterm-clipboard
mv osc-xterm-clipboard /usr/lib/urxvt/perl/
```

同时还需要在 .Xdefaults 中启用 perl 脚本：

```
~/.Xdefaults
...
*URxvt.perl-ext-common:           osc-xterm-clipboard
...
```

接下来，需要让 tmux 知道这个新功能，并启用鼠标支持（如果还未启用）：

```
~/.tmux.conf
...
set-option -ga terminal-override ',rxvt-uni*:XT:Ms=\E]52;%p1%s;%p2%s\007'
set -g mouse on
...
```

这样就完成了。请结束所有 tmux 进程后再尝试新添加的中键功能。

在 tmux 中，`Shift+MiddleMouseClick` 会粘贴剪贴板内容，而仅用鼠标中键会粘贴 tmux 缓冲区中的剪贴板内容。

在 tmux 外部，仅用鼠标中键会粘贴 tmux 缓冲区内容，而标准 `Ctrl+c`用于复制。

提示与技巧

以默认的会话布局启动 tmux

像 *tmuxinator* 和 tmuxp 这样的会话管理器能简化通常的会话配置。

要使用 *tmuxinator*，请安装 tmuxinatorAUR。安装完成后尝试运行以下命令测试安装是否成功：

```
$ tmuxinator doctor
```

获取默认布局参数

首先，根据个人的喜好启动并配置 tmux 窗口和面板。完成后，在当前会话内，执行下面的命令获取当前的布局参数值：

```
tmux list-windows
```

命令的输出可能会像下面这样（比如两个窗口，分别有 3 个和 2 个面板的布局）：

```
0: default* (3 panes) [274x83] [layout
20a0,274x83,0,0{137x83,0,0,3,136x83,138,0[136x41,138,0,5,136x41,138,42,6]}] @2 (active)
1: remote- (2 panes) [274x83] [layout e3d3,274x83,0,0[274x41,0,0,4,274x41,0,42,7]] @3
```

这其中需要复制并在这之后使用到的部分，是从 **\[layout...** 之后开始到 **... \] @2 (active)** 之前的部分。对于上面的例子，第一个窗口的布局对应的内容是 **20a0,274x83,0,0{137x83,0,0,3,136x83,138,0\[136x41,138,0,5,136x41,138,42,6\]}**。

定义默认的 tmux 布局

在上一节中获取到需要的布局信息后，就可以退出当前的 tmux 会话了。然后编辑 tmuxinator 的配置文件来创建自定义的默认 tmux 会话布局（不要复制示例，用上一节获取到的布局信息）：

```
~/.tmuxinator/default.yml
name: default
root: ~/
windows:
  - default:
      layout:
20a0,274x83,0,0{137x83,0,0,3,136x83,138,0[136x41,138,0,5,136x41,138,42,6]}
      panes:
        - clear
        - vim
        - clear && emacs -nw
```

```
  - remote:
      layout: 24ab,274x83,0,0{137x83,0,0,3,136x83,138,0,4}
      panes:
        -
        -
```

上面的示例定义了两个窗口（名称分别是“default”和“remote”）和用户定义的布局参数值。每个面板设置都需要至少一行 -。tmux 启动时，第一个窗口中的第一个面板中执行了“clear”，第二个面板中是“vim”，第三个面板中执行了“clear && emacs -nw”。第二个窗口中的两个面板在启动时都没有执行任何命令。

使用如下命令测试新的默认布局：

```
tmuxinator default
```

令 tmux 自启动时使用默认布局

如果希望自启动 tmux 会话时使用自定义的默认布局，请按下面的示例编辑：

```
~/.bashrc
 if [ -z "$TMUX" ]; then
   tmuxinator default
 fi
```

使用默认会话的其他方法

除了以上的方法，也可以直接编写一个在执行时创建并附加默认会话的 bash 脚本。

然后就能在终端执行并获得一个预先配置好的 tmux 会话了：

```
#!/bin/bash
tmux new-session -d -n WindowName Command
tmux new-window -n NewWindowName
tmux split-window -v
tmux selectp -t 1
tmux split-window -h
tmux selectw -t 1
tmux -2 attach-session -d
```

在 Urxvt 中启动 tmux

使用下面的命令启动一个带 tmux 会话的 urxvt，可以将这段命令保存至 .ratpoisonrc 文件中：

```
urxvt -e bash -c "tmux -q has-session && exec tmux attach-session -d || exec tmux new-session -n$USER -s$USER@$HOSTNAME"
```

在每次登录时启动 tmux

```
if [ -x "$(command -v tmux)" ] && [ -n "${DISPLAY}" ] && [ -z "${TMUX}" ]; then
    exec tmux new-session -A -s ${USER} >/dev/null 2>&1
fi
```

上面的片段完成了这些操作：

1. 测试 tmux 是否是可执行的命令，
2. 并且图形化会话是否在运行（若希望让 tmux 在任意登录 shell 中启动，请**删除这个条件**，但这可能会干扰登录时自动启动 X），
3. 并且目前不在某个 tmux 会话中，

4. 然后尝试加入（attach）已有会话，如果失败，就创建一个新的会话。

如果已配置 systemd 保留会话，可以用下面的命令替换上面 if 语句块中的内容，这样 tmux 会加入被保留的会话并退出其他所有已加入该会话的客户端。

```
if ! systemctl --user is-active --quiet tmux.service; then
    systemctl --user start tmux.service
fi
exec tmux attach-session -d -t "${USER}" >/dev/null 2>&1
```

启动一个非登录 shell

tmux 默认启动一个登录 shell，这有可能产生许多负面影响：

- fortune) 用户可能会注意到在创建新的面板时，引号也被输出到了屏幕上。
- 每次创建新面板时，为登录 shell 所配置的文件例如 `~/.profile` 都会被重新解析，这也导致了一些本应在会话初始化时才需要执行的命令在创建新面板时再次执行（比如设置音量）。

将下面的内容添加到 `~/.tmux.conf` 中以禁用 tmux 默认启动登录 shell 的行为：

```
set -g default-command "${SHELL}"
```

将 tmux 窗口用作选项卡

将下面的内容添加到 `~/.tmux.conf` 后就可以像选项卡一样使用 tmux 的窗口了，具体请参考 urxvt's tabbing 扩展所提供的参考快捷键。这样做的优点是，这些虚拟的“选项卡”是独立于终端模拟器的。

```
#urxvt tab like window switching (-n: no prior escape seq)
bind -n S-down new-window
bind -n S-left prev
bind -n S-right next
bind -n C-left swap-window -t -1
bind -n C-right swap-window -t +1
```

但要注意这些快捷键不应覆盖其他应用程序的快捷键。例如终端，虽然终端上可能未激活这些快捷键。

同样也可以很方便地提供用于 tmux detach 所对应的 EOT 快捷键 `Ctrl+d`：

```
bind-key -n C-j detach
```

客户端同时与会话中的多个窗口交互

Brandur Leach 在 Practical Tmux 中写了这么一段内容：

: Screen and tmux's behaviour for when multiple clients are attached to one session differs slightly. In Screen, each client can be connected to the session but view different windows within it, but in tmux, all clients connected to one session must view the same window. : This problem can be solved in tmux by spawning two separate sessions and synchronizing the second one to the windows of the first, then pointing a second new session to the first.

翻译内容如下：

: Screen 和 tmux 在多个客户端加入同一个会话时的行为稍微有些不同。Screen 中，每个客户端在连接到会话后，可以在各个客户端中浏览不同的窗口，而在 tmux 中，所有连接到同一会话的客户端都只能浏览同一个窗口（即不同的客户端所显示的内容都是一样的）。 : 这个问题的解决方法是在 tmux 中创建两个独立的会话，将第二个会话同步到第一个会话的窗口，然后将第二个新会话指向第一个会话。

下面的 `tmx` 脚本实现了上面所说的解决方法。这里的版本稍微进行了一些调整，假设第二个参数是 `1`，那么执行 `tmux new-window`。脚本中调用的 `tmx *base_session_name*` 用于启动所必需的基础会话。否则，将一个新的“客户端”会话连接到前置会话（可选添加新窗口并加入），并设置其一旦变成僵尸进程就终止。最后记得为脚本添加可执行权限。

```
~/bin/tmx
#!/bin/bash
# Modified TMUX start script from:
#     http://forums.gentoo.org/viewtopic-t-836006-start-0.html

# Works because bash automatically trims by assigning to variables and by passing
arguments
trim() { echo $1; }

if [[ -z "$1" ]]; then
    echo "Specify session name as the first argument"
    exit
fi

# Only because I often issue `ls` to this script by accident
if [[ "$1" == "ls" ]]; then
    tmux ls
    exit
fi

base_session="$1"
# This actually works without the trim() on all systems except OSX
tmux_nb=$(trim `tmux ls | grep "^$base_session" | wc -l`)
if [[ "$tmux_nb" == "0" ]]; then
    echo "Launching tmux base session $base_session ..."
    tmux new-session -s $base_session
else
    # Make sure we are not already in a tmux session
    if [[ -z "$TMUX" ]]; then
        echo "Launching copy of base session $base_session ..."
        # Session id is date and time to prevent conflict
        session_id=`date +%Y%m%d%H%M%S`
        # Create a new session (without attaching it) and link to base session
        # to share windows
        tmux new-session -d -t $base_session -s $session_id
        if [[ "$2" == "1" ]]; then
              # Create a new window in that session
              tmux new-window
       fi
        # Attach to the new session & kill it once orphaned
       tmux attach-session -t $session_id \; set-option destroy-unattached
    fi
fi
```

对此，有用的设置是：

```
setw -g aggressive-resize on
```

将上面这个添加到 `~/.tmux.conf`。可以让 tmux 基于最小的客户端实际大小来调整窗口大小，而非整个会话中最小的。

另一种做法是把下面的内容添加到 `~/.bashrc`：

```
~/.bashrc
function rsc() {
  CLIENTID=$1.`date +%S`
  tmux new-session -d -t $1 -s $CLIENTID \; set-option destroy-unattached \; attach-session -t $CLIENTID
}

function mksc() {
  tmux new-session -d -s $1
  rsc $1
}
```

引用作者：

: "mksc foo" creates a always detached permanent client named "foo". It also calls "rsc foo" to create a client to newly created session. "rsc foo" creates a new client grouped by "foo" name. It has destroy-unattached turned on so when I leave it, it kills client. : Therefore, when my computer looses network connectivity, all "foo.something" clients are killed while "foo" remains. I can then call "rsc foo" to continue work from where I stopped.

根据终端类型调整正确的 TERM 变量值

除了可以在 tmux 中为 TERM 变量设置固定值，还可以根据终端模拟器的类型设置正确的 TERM（`screen` 或 `screen-256color`）：

```
~/.tmux.conf
## set the default TERM
set -g default-terminal screen

## update the TERM variable of terminal emulator when creating a new session or attaching a existing session
set -g update-environment 'DISPLAY SSH_ASKPASS SSH_AGENT_PID SSH_CONNECTION WINDOWID XAUTHORITY TERM'
## determine if we should enable 256-colour support
if "[[ ${TERM} =~ 256color || ${TERM} == fbterm ]]" 'set -g default-terminal screen-256color'
~/.zshrc
## workaround for handling TERM variable in multiple tmux sessions properly (by Nicholas Marriott)
if [[ -n ${TMUX} && -n ${commands[tmux]} ]];then
        case $(tmux showenv TERM 2>/dev/null) in
                *256color) ;&
                TERM=fbterm)
                        TERM=screen-256color ;;
                *)
                        TERM=screen
        esac
fi
```

重新加载更新过的配置但无需重启 tmux

tmux 默认只在启动时读取 `~/.tmux.conf`。要让 tmux 在启动后加载配置文件，执行下面的命令：

```
tmux source-file 配置文件路径
```

这条命令也可以添加到 `~/.tmux.conf` 中：

```
~/.tmux.conf
 bind r source-file 文件路径
```

然后执行下面的命令，效果同上：

```
source .tmux.conf
```

在新会话或加入会话时运行程序的脚本模板

下面的脚本会检查某个程序是否是已经运行的状态，如果找到该程序，就将它附加到会话并选择该窗口。如果没有找到，那就创建一个新的 tmux 会话，同时创建一个用该程序命名的新窗口，并在该窗口中运行此程序。

```
#!/bin/bash

PID=$(pidof $1)

if [ -z "$PID" ]; then
    tmux new-session -d -s main ;
    tmux new-window -t main -n $1 "$*" ;
fi
    tmux attach-session -d -t main ;
    tmux select-window -t $1 ;
exit 0
```

这里有一个由此派生出的版本，用于配合 *nicklist* 运行 *irssi*。

终端模拟器的窗口标题

如果用户是通过 SSH 登录进的 tmux 窗口，终端模拟器的窗口标题依然会显示 `user@localhost` 而非 `user@server`。要让标题栏自适应显示所连接的远程主机名，请在 `~/.tmux.conf` 中设置以下内容：

```
set -g set-titles on
set -g set-titles-string "#T"
```

对于 `set-titles-string`，`#T` 会显示为 `user@host:~` 并根据用户所连接到的不同主机显示对应的主机名称。

自动设置布局

在创建或销毁分割窗口时，并不会应用当前所选定的布局。将下面的按键绑定添加到配置文件中可以解决这个问题：

```
bind-key -n M-c kill-pane \; select-layout
bind-key -n M-n split-window \; select-layout
```

提示：tmux-xpanesAUR 可以简化窗口布局管理和 SSH 连接，感兴趣可以尝试。

Vim 颜色主题未能加载

如果 tmux 中的 vim 颜色主题没有正常加载，请参阅和。

对 Vim 用户友好的配置

参考。

更好的面板分割

默认的分割面板快捷键分别是，垂直分割：`Ctrl+b %`，水平分割：`Ctrl+b "`。对于有些键盘布局而言较难按下，而且也不便于记忆。

一种对用户而言更友好且方便记忆的的快捷键绑定是使用 `Ctrl+b h` 水平分割，以及用 `Ctrl+b v` 垂直分割面板。

将下面的内容添加到 `~/.tmux.conf` 中可以实现快捷键更改：

```
# 更好的面板分割快捷键：
bind-key h split-window -h
bind-key v split-window -v
```

防止系统休眠

如果 tmux 所连接到的主机因为进入睡眠模式而挂起，请使用 inhibition lock 运行会话：

```
tmux new-session -A "systemd-inhibit --what=idle $SHELL"
```

故障排除

缓冲区滚动问题

如果在终端内使用 `Shift+Page Up` 或 `Shift+Page Down` 滚动时需要问题，下面的配置项可以禁用任意终端的 *smcup* 和 *rmcup* 功能（这些会将所有操作反馈前都添加 `xterm`）：

```
set -ga terminal-overrides ',xterm*:smcup@:rmcup@'
```

这样的技巧会使终端模拟器认为 tmux 是一个全屏的应用程序，就像 pico 或 mutt 一样（参考）。这时滚动操作就会被正确记录下来。但要注意，这样做之后，在窗口或面板间切换时可能会引起一些显示混乱。所以请考虑使用 tmux 原生的滚动历史功能。

鼠标滚动

注意：这会干扰剪贴板复制粘贴功能。要复制或粘贴剪贴板，请按住 Shift 键。

要使用鼠标滚轮滚动缓冲区内容，请先确保启用了鼠标：

```
set -g mouse on
```

设置最大可回滚历史记录数：

```
set -g history-limit 30000
```

自 tmux 2.1 版本开始，请将下面的内容之一或全部添加到 ~/.tmux.conf 中：

```
   bind -T root WheelUpPane   if-shell -F -t = "#{alternate_on}" "send-keys -M"
"select-pane -t =; copy-mode -e; send-keys -M"
   bind -T root WheelDownPane if-shell -F -t = "#{alternate_on}" "send-keys -M"
"select-pane -t =; send-keys -M"
```

由于上面的配置每次只能滚动一行，添加下面的配置能让每次滚动时滚动一整页：

```
bind -t vi-copy    WheelUpPane   page-up
bind -t vi-copy    WheelDownPane page-down
bind -t emacs-copy WheelUpPane   page-up
bind -t emacs-copy WheelDownPane page-down
```

终端模拟器不支持 UTF-8 鼠标事件

已设置 `mouse on` 选项，且终端模拟器不支持 UTF-8 鼠标事件时，在终端窗口中左键点击可能会粘贴如 `[M#` 或 `[Ma` 的字符串。

配置以下内容可以解决这个问题：

```
set -g mouse-utf8 off
```

Midnight Commander 中 Shift+F6 无效

请参阅 Midnight_Commander#快捷键失效。

另见

- BBS topic
- Screen and tmux feature comparison
- powerline，tmux 动态状态栏。
- Plugins for tmux
- Oh My Tmux!
- tmux Wiki

教程

- Practical Tmux
- 手册页 tmux(1)
- Tmux tutorial Part 1 和 Part 2
- *The Tao of tmux*，Tony Narlock 所著的电子书，同时也是 tmuxp 和 libtmux 的作者

问题跟踪系统

项目一旦多人协作，缺陷和需求就需要可追踪的流水线。这一章给出三种自建方案：Flyspray 只到安装为止；Bugzilla 在安装之外还有配置和参见；Trac 从安装讲到快速入门与后续步骤。

建议这样读：Flyspray 的安装和 Trac 的快速入门必读，一个最短、便于验证环境，一个决定装完往哪走；Bugzilla 的配置和 Trac 的后续步骤可以先跳，真正投入使用再看。最容易踩的坑是把三者挨个装一遍，或在安装还没跑通时就先去调 Bugzilla 的配置——顺序颠倒只会让问题混在一起。

15.1　Flyspray

难度 入门 | **篇幅** 2.3 KB | **所属** 问题跟踪系统

本节将讲到

- 安装

Flyspray 是一个用 PHP 编写的错误跟踪系统。尤为值得一提的是，Arch Linux 发行版自身就曾使用过 FlySpray 该系统。

安装

安装 flyspray包 软件包。Flyspray 需要 Web 服务器（例如带有 PHP 的 Apache HTTP Server）和 SQL 服务器（例如 MySQL 或 PostgreSQL）。

Apache 配置

注意：您需要将 Apache HTTP Server 配置为与 PHP。查看 Apache HTTP Server#PHP 以获得说明。确保取消注释 `/etc/php/php.ini` 中的 `extension=mysqli`。

您将需要为 apache 创建一个配置文件以找到您的 Flyspray 安装。创建以下文件：

```
/etc/httpd/conf/extra/flyspray.conf
Alias /flyspray "/usr/share/webapps/flyspray"
<Directory "/usr/share/webapps/flyspray">
        AllowOverride All
        Options FollowSymlinks
        Require all granted
        php_admin_value open_basedir "/srv/http/:/tmp/:/usr/share/webapps/flyspray"
</Directory>
```

然后，您将需要编辑 `/etc/webapps/flyspray/.htaccess` 并将 `deny from all` 更改为 `allow from all`。现在，您应该能够导航到 flyspray 界面（例如 http://localhost/flyspray ），它将显示一个预安装检查页面。此处的所有问题都应先解决，然后再继续。

15.2　Bugzilla

难度 入门 | **篇幅** 2.7 KB | **所属** 问题跟踪系统

本节将讲到

- 安装
- 配置
- 参见

Bugzilla 是一个设计为帮助你管理软件开发的服务器软件。

安装

安装 bugzilla包 软件包。

配置

模块依赖

首先执行模块检查:

```
# cd /srv/http/bugzilla
# ./checksetup.pl --check-modules
```

通过输出查看哪些模块是必须的,哪些模块是可选的。用于安装缺失模块的 Shell 命令会显示出来。

使用一下命令安装所有必须和可选的模块:

```
# perl install-module.pl --all
```

最终模块检查

接下来的更多配置会让 BugZilla 连接到 MySQL 并在其中创建初始化表。

再次运行 `checksetup.pl`,这次不带 `-check-modules`选项:

```
# ./checksetup.pl
```

如果一切正常,会创建一个 `localconfig` 文件。然后,修改文件中的一些参数:

```
$webservergroup = 'http';
$db_driver = 'DATABASE_TO_USE_HERE';
$db_name = 'DATABASE_NAME_HERE';
$db_user = 'DATABASE_USER_HERE';
$db_pass = 'YOUR_PASSWORD_HERE';
```

Apache

最后,配置 Apache HTTP Server 来使用 mod_cgi (也可以使用 mod_perl 来配置;详情请参考此节) 运行 Bugzilla。

首先在 `/etc/httpd/conf/httpd.conf` 中将下列行取消注释:

```
LoadModule cgi_module modules/mod_cgi.so
```

然后把下列行加入 `/etc/httpd/conf/httpd.conf`:

```
<Directory /srv/http/bugzilla>
  AddHandler cgi-script .cgi
  Options +ExecCGI
  DirectoryIndex index.cgi
```

```
  AllowOverride All
</Directory>
```

现在重启 Apache 和 需要的模块。

使用你的网页浏览器访问 `http://*server_domain_or_ip*/bugzilla/`。

参见

- bugzilla 文档
- https://web.archive.org/web/20200111080219/http://blog.samsonis.me/2009/04/bugzilla-on-archlinux/

15.3 Trac

难度 入门 | **篇幅** 3.0 KB | **所属** 问题跟踪系统

本节将讲到

- 安装
- 快速入门
- 后续步骤

从项目网页:

: Trac is an enhanced wiki and issue tracking system for software development projects. Trac uses a minimalistic approach to web-based software project management. Our mission is to help developers write great software while staying out of the way. Trac should impose as little as possible on a team's established development process and policies.

安装

安装 tracAUR 包。配置是基于每个环境进行的。请参阅下文，了解如何创建环境。有关详细说明，请访问 https://trac.edgewall.org/wiki/TracGuide 。

快速入门

创建和初始化环境

初始化环境

```
# cd /srv/;
# mkdir tracenv;
# trac-admin /srv/tracenv initenv;
```

环境配置可以在 `/srv/tracenv/conf/trac.ini` 找到。

配置 systemd 服务文件

默认服务文件位于 `/usr/lib/systemd/system/tracd.service`。将此文件复制到 `/etc/systemd/system/tracd.service`，然后编辑它以指向您的新环境。该 `ExecStart` 条目应如下所示：

```
ExecStart=/usr/bin/tracd -b localhost -p 8080 /srv/tracenv
```

查看 Web 服务器

之后启动（并可选地启用）服务（或直接运行 `/usr/bin/tracd`）之后，可以使用 Web 浏览器在 `[http://localhost:8080](http://localhost:8080)` 上查看 Web 界面。

后续步骤

Trac 用户

专门为 trac 服务创建用户是一个好主意。创建该用户后，您可以使用该用户创建环境：

```
# cd /srv/;
# mkdir tracenv;
# chown trac:trac tracenv;
# sudo -u trac trac-admin /srv/tracenv initenv;
```

将以下内容添加到 systemd Unit 文件中，以确保它以 `trac` 用户身份启动：

```
[Service]
User=trac
Group=trac
```

Trac中的用户和权限

（本节涉及在 trac 环境中创建用户，而不是 GNU/Linux 用户。）

接下来，您将要添加用户并向这些用户授予权限。要添加用户，请参阅 https://trac.edgewall.org/wiki/TracStandalone#UsingAuthentication （您必须更改 `.service` 文件以引用您选择的身份验证机制）。要向用户授予权限，请在 trac 服务器上运行以下命令：

```
# trac-admin /srv/tracenv permission add <username> TRAC_ADMIN
```

第 16 章　共 2 节　39 KB

版本控制系统

把工作目录纳入版本控制，是 Arch 上开发与维护绕不开的一环。《Git》篇幅最重，安装、配置、基本用法、技巧与扩展依次展开，建议完整读完；《Subversion》先讲基于 Apache 服务器的安装，再讲 svnserve 的设置、备份与还原以及客户端，只在接手旧仓库或必须沿用既有工作流时精读，其余可略过。

最容易踩的坑是把装包当成装好：《Subversion》要在 Apache 与 svnserve 两种方式里选定一种并配通，备份与还原另有专门步骤；《Git》若跳过配置直接上手，后面往往反复返工。

16.1　Subversion

难度 进阶 | **篇幅** 9.2 KB | **所属** 版本控制系统

本节将讲到

- 用于Subversion安装的Apache服务器
- 设置 Svnserve
- Subversion backup and restore
- Subversion clients
- See also

"Apache Subversion 是一套功能全面的版本控制系统，最初被设计为CVS的改进版本。其后Subversion的发展大大超出了取代CVS的原始目标，但它的基本模型、设计和接口仍然受到了这一目标的深刻影响。"

本文主要介绍架设svn服务器的方法。有两种流行的svn服务器，内建的`svnserve`以及更高级的选择——结合了svn插件的Apache HTTP Server。

用于Subversion安装的Apache服务器

目标

这篇指南的目标是结合Apache安装Subversion。选用Apache是因为其提供了单机`svnserve`不具备的诸多特性。

- 支持 HTTPS，比 svnserve 使用的 md5 认证更加安全。
- 细粒度的控制权。可以使用 Apache 的认证限制目录的访问权限。可以允许所有文件可读，但仅允许提交 trunk，同时对另一组用户赋予提交 tags 和 branches 的权限。
- 一个自由的源码库查看器
- Subversion 团队正在进行无缝 webdav 集成的工作。不久你就能用任何 webdav 接口更新源码库中的文件。

安装

安装 Apache HTTP Server。除了 apache 之外，还需要安装 subversion包。

配置 Subversion

创建一个目录

```
# mkdir -p /home/svn/repositories
```

编辑 httpd.conf

请确认下列模块加载指令在文件中列出。如果没有请添加它们(通常你只需要添加后两行)，保持先后顺序：

```
/etc/httpd/conf/httpd.conf
LoadModule dav_module           modules/mod_dav.so
 LoadModule dav_fs_module        modules/mod_dav_fs.so
 LoadModule dav_svn_module       modules/mod_dav_svn.so
 LoadModule authz_svn_module     modules/mod_authz_svn.so
```

用不用SSL？

SSL允许用户使用Apache的AuthType Basic而不必担心有人嗅探密码。

生成证书：

```
# cd /etc/httpd/conf/
# openssl req -new -x509 -keyout server.key -out server.crt -days 365 -nodes
```

然后添加下面的配置到`/etc/httpd/conf/extra/httpd-ssl.conf`，以便在虚拟主机配置指令中包含它们。

```
<Location /svn>
   DAV svn
   SVNParentPath /home/svn/repositories
   AuthzSVNAccessFile /home/svn/.svn-policy-file
   AuthName "SVN Repositories"
   AuthType Basic
   AuthUserFile /home/svn/.svn-auth-file
   Satisfy Any
   Require valid-user
</Location>
```

为了确保SSL设置已加载，取消`/etc/httpd/conf/httpd.conf`中SSL配置行的注释：

```
Include /etc/httpd/conf/extra/httpd-ssl.conf
LoadModule ssl_module modules/mod_ssl.so
LoadModule socache_shmcb_module modules/mod_socache_shmcb.so
```

创建/home/svn/.svn-policy-file

```
[/]
* = r

[REPO_NAME:/]
USER_NAME = rw
```

/部分中的*用来匹配匿名用户。对除只读以外的任何访问Apache AuthType Basic都会提示输入用户名和密码。REPO_NAME:/一节继承了之前的权限设置，于是匿名用户

对其有只读权限。最后一项设置为用户USER_NAME授予来REPO_NAME源码库的读写权限。

创建/home/svn/.svn-auth-file

这个文件可以用htpasswd或htdigest创建。这里使用了htpasswd。同样，因为SSL，不用过多担心密码嗅探。htdigest甚至会对嗅探提供更好的安全特性。

```
# htpasswd -cs /home/svn/.svn-auth-file USER_NAME
```

以上创建了文件(-c)并使用SHA1保存密码(-s)用户USER_NAME被创建。要添加其他用户，可以去掉 -c 选项：

```
# htpasswd -s /home/svn/.svn-auth-file OTHER_USER_NAME
```

创建源码库

```
# svnadmin create /home/svn/repositories/REPO_NAME
```

设置权限

对Apache用户设置新源码库的权限：

```
# chown -R http.http /home/svn/repositories/REPO_NAME
```

创建项目

项目的目录结构

创建 `branches tags trunk` 临时目录结构：

```
$ cd /path/to/directoryofchoice
$ mkdir -p ~/svn-import/{branches,tags,trunk}
```

将源码添加到目录

将源代码文件放入创建的 trunk 目录：

```
$ cp -R /my/existing/project/* ~/svn-import/trunk
```

导入项目

```
$ svn import -m "Initial import" ~/svn-import https://yourdomain.net/svn/REPO_NAME/
```

测试SVN检出

```
$ svn checkout https://yourdomain.net/svn/REPO_NAME/ /my/svn/working/copy
```

如果以上所有配置都成功，你应该能得到一个受版本控制的新源码库的副本。

设置 Svnserve

安装软件包

安装 subversion包。

创建源码库

创建你的源码库

```
mkdir /path/to/repos/
svnadmin create /path/to/repos/repo1
```

初始源码库是空的，如果想导入文件，使用以下命令：

```
svn import ~/code/project1 file:///path/to/repos/repo1 --message 'Initial repository layout'
```

设置访问策略

编辑文件/path/to/repos/repo1/conf/svnserve.conf，在\[general\]中取消以下行的注释或者添加之：

```
password-db = passwd
```

你也许想改变对匿名用户的默认设置

```
anon-access = read
```

对允许任何人提交的源码库，替换"read"为"write"，或者将其改为"none"来禁止所有匿名访问。

编辑文件/path/to/repos/repo1/conf/passwd

```
[users]
harry = foopassword
sally = barpassword
```

以上定义了用户harry和sally，分别使用密码foopassword和barpassword，可以按需修改。

启动服务器守护进程

在启动服务器之前，编辑配置文件

```
/etc/conf.d/svnserve
SVNSERVE_ARGS="--root=/path/to/repos"
```

The `--root=/path/to/repos` option set the root of repository tree. If you have multiple repositories use `--root=/path-to/reposparent`. Then access independent repositories by passing in repository name in the URL: `svn://host/repo1`. make sure that the user has read/write access to the repository files)

Optionally add a `--listen-port` if you want a different port, or other options.

By default, the service runs as root. If you want to change that, add a drop-in:

```
/etc/systemd/system/svnserve.service.d/50-custom.conf
[Service]
User=svn
```

Now start the *svnserve.service* daemon.

svn+ssh

To use svn+ssh://, we have to have a wrapper written for svnserve.

check where the svnserve binary is located:

```
# which svnserve
/usr/local/bin/svnserve
```

Our wrapper is going to have to fall in PATH prior to this location...

Create a wrapper with the following content:

```
/usr/bin/svnserve
#!/bin/sh
# wrapper script for svnserve
umask 007
/usr/local/bin/svnserve -r /path/to "$@"
```

Then, make it executable.

`-r /path/to` is what makes use of the svn co svn+ssh://server.domain.com:/reponame instead of `:/path/to/reponame`.

Start svnserve with new wrapper script like so:

```
# /usr/bin/svnserve -d  ( start daemon mode )
```

we can also check the perms for remote users like this:

```
$ svn ls svn+ssh://server.domain.com:/reponame
++server.domain.com++
dev/
qa/
release/
```

Subversion backup and restore

To back up your subversion repositories, do this for each repository you have.

```
$ svnadmin dump /path/to/repo > /tmp/repo.dump
```

To restore the backup, create the corresponding repositories first:

```
$ svnadmin create /path/to/repo
```

Then load svn dump into new repo:

```
$ svnadmin load /path/to/repo < /tmp/repo.dump
```

Setting permissions:

```
$ chown -R svn:svnusers /path/to/repo
$ chmod -R g+w /path/to/repo/db/
```

These repositories should now be all setup.

Subversion clients

See also Wikipedia:Comparison of Subversion clients.

- **kdesvn** — Subversion client for KDE.

: https://invent.kde.org/sdk/kdesvn || kdesvn包

- **RabbitVCS** — Set of graphical tools written to provide simple and straightforward access to the version control systems you use.

: http://rabbitvcs.org/ || rabbitvcsAUR

- **RapidSVN** — GUI front-end for the Subversion revision system written in C++ using the wxWidgets framework.

: http://rapidsvn.tigris.org/ || rapidsvnAUR

See also

- https://svnbook.red-bean.com/en/1.1/svn-book.html#svn-ch-9-sect-2.2-re-load
- https://subversion.apache.org/

16.2 Git

难度 高级 | **篇幅** 30.1 KB | **所属** 版本控制系统

本节将讲到

- 安装
- 配置
- 基本用法
- 提示与技巧
- 扩展

: "我遇到过一些人，他们认为 Git 是 GitHub 的前端。他们错了，Git 其实是 AUR 的前端。"—— Linus Torvalds

Git 是由 Linux 内核作者 Linus Torvalds 设计并开发的版本控制系统（VCS），现在被用来维护 AUR 软件包以及数以千计的其他项目，其中包括 Linux 内核。

安装

安装 git包 软件包。要使用开发版本，请安装 git-gitAUR 软件包。当使用 *git svn*、*git gui* 和 *gitk* 等工具时请检查可选依赖项是否安装。

图形化前端

参考 git GUI Clients。

- **Commit** — 帮助编写更好的 Git 和 Mercurial 提交消息的编辑器。

: https://apps.gnome.org/Commit/ || commit包

- **Forge Sparks** — 支持 Github、GitLab、Gitea 和 Forgejo 的简单通知应用。

: https://apps.gnome.org/ForgeSparks/ || forge-sparks包

- **Giggle** — 用于 git 的 GTK+ 前端。

: https://wiki.gnome.org/Apps/giggle/ || giggle包

- **GitAhead** — 一个包含内置合并工具的 git 前端。

: https://gitahead.github.io/gitahead.com/ || gitaheadAUR

- **GitButler** — 基于 Git 的版本控制客户端，使用 Tauri/Rust/Svelte 构建。

: https://gitbutler.com/ || gitbutlerAUR

- **Git Cola** — 用 Python 编写的丝滑而强大的 git 图形前端。

: https://git-cola.github.io/ || git-colaAUR

- **Git Extensions** — 允许用户不使用命令行就可以完成 git 各项操作的图形前端。

: https://gitextensions.github.io/ || gitextensionsAUR

- **gitg** — 用于查看 git 仓库的 GNOME GUI 客户端。

: https://wiki.gnome.org/Apps/Gitg || gitg包

- **git-gui** — Tcl/Tk 库编写的可移植 git 图形前端。

: https://git-scm.com/docs/git-gui || git包 + tk包 : > **注意**：要打开 *git-gui* 的拼写检查功能，请安装 aspell包，同时还需要与 `LC_MESSAGES` 环境变量 相对应的字典文件。参阅 FS#28181 和 aspell。

- **GitHub Desktop** — 由 GitHub 开发的一个基于 Electron 的 GitHub 客户端。

: https://github.com/desktop/desktop || github-desktopAUR github-desktop-binAUR

- **gitk** — Tcl/Tk 库编写的 Git 仓库查看器。

: https://git-scm.com/docs/gitk || git包 + tk包

- **Gittyup** — 基于 Qt 的 Git 客户端。

: https://github.com/Murmele/Gttyup || gittyupAUR

- **Guitar** — 一个 Git 的图形化前端。

: https://github.com/soramimi/Guitar || guitarAUR

- **gitui** — 用 Rust 编写的快速的 Git 终端用户界面。

: https://github.com/extrawurst/gitui || gitui包

- **Kommit** — KDE 的 Git GUI 客户端。

: https://apps.kde.org/kommit/ || kommit包

- **lazygit** — 一个简洁的 Git TUI 工具。

: https://github.com/jesseduffield/lazygit || lazygit包

- **QGit** — 可图形化地按照不同开发分支显示修订历史记录、查阅补丁内容、查看被修改文件的 Git GUI 查看器。

: https://github.com/tibirna/qgit || qgit包

- **RabbitVCS** — 一组图形化工具，用于轻松、直接地访问您使用的版本控制系统。

: http://rabbitvcs.org/ || rabbitvcsAUR

- **Sublime Merge** — 由 Sublime Text 开发商开发的 Git 前端。

: https://www.sublimemerge.com/ || sublime-mergeAUR

- **Tig** — 基于 ncurses 的 git 字符模式前端。

: https://jonas.github.io/tig/ || tig包

- **ungit** — 在不牺牲 git 各种功能的情况下使其变得更加友好。

: https://github.com/FredrikNoren/ungit || nodejs-ungitAUR

配置

你至少需要设置好姓名和邮箱之后才能开始使用 Git：

```
$ git config --global user.name  "John Doe"
$ git config --global user.email "johndoe@example.com"
```

参阅起步 - 初次运行 Git 前的配置。

更多设置选项可参阅#提示与技巧。

基本用法

一个 Git 版本库包含在一个名为 `.git` 的目录内，该目录包含了修订历史以及其他元数据。版本库所跟踪的目录（默认为父目录）称为工作目录。在工作树进行的更改在被提交 (commit) 前需要先暂存 (stage) 起来。Git 还可以让你恢复以前提交的工作树文件。

参见起步。

获取 Git 仓库

参见获取 Git 仓库 - Git 基础

记录更改

参见记录每次更新到仓库 - Git 基础

查看提交记录

参见查看提交历史 - Git 基础

撤销修改

参见撤消操作 - Git 基础

远程操作

参见远程仓库的使用 - Git 基础

分支 (branch)

参见分支简介 - Git 分支

分支与合并基础

参见分支的新建与合并 - Git 分支

分支管理

参见分支管理 - Git 分支

分支开发工作流

参见分支开发工作流 - Git 分支

远程分支

参见远程分支 - Git 分支

变基

参见变基 - Git 分支

多人合作

分布式工作流

参见分布式工作流程 - 分布式 Git

为已有项目贡献

参见向一个项目贡献 - 分布式 Git

维护自己的项目

参见维护项目 - 分布式 Git

Git 工具

选择修订版本

参见选择修订版本 - Git 工具

交互式暂存

参见交互式暂存 - Git 工具

贮藏与清理

参见贮藏与清理 - Git 工具

签署工作

参见签署工作 - Git 工具

搜索

参见搜索 - Git 工具

重写历史

参见重写历史 - Git 工具

重置揭密

参见重置揭密 - Git 工具

高级合并

参见高级合并 - Git 工具

Rerere

参见Rerere - Git 工具

使用 Git 调试

参见使用 Git 调试 - Git 工具

子模块

参见子模块 - Git 工具

打包

参见打包 - Git 工具

替换

参见替换 - Git 工具

凭证存储

参见凭证存储 - Git 工具

提示与技巧

使用 git-config

Git 从 4 个 ini 类型的配置文件里读取配置：

- `/etc/gitconfig` 是应用于整个系统的默认配置文件
- `~/.gitconfig` 和 `~/.config/git/config` （自 1.7.12 起）是应用于特定用户的配置文件
- `.git/config` 是应用于特定仓库的配置文件

这些文件可以直接编辑，但是更常用的方法是使用 *git config*，下面是一些示范。

列出当前已配置的变量：

```
$ git config list {--local,--global,--system}
```

将默认文本编辑器从 vim 改成 nano：

```
$ git config --global core.editor "nano -w"
```

设置默认的推送 (push) 行为：

```
$ git config --global push.default simple
```

设置不同的 *git difftool* 工具（默认是 *meld*）：

```
$ git config --global diff.tool vimdiff
```

更多信息请参阅 git-config(1) 和配置 Git。

包含单独的配置文件

自 2012 年的 v1.7.10 版本起，Git 能够使用 `include` 关键字在 `gitconfig` 文件中构建分割成多个配置文件的配置。

提示：当使用 `--local` 或 `--global` 标志时，默认禁用加载包含的配置文件。要启用它，请将 `--includes` 标志与 `git config get` 和 `git config list` 命令一起使用。

保持良好的礼仪

- 当你想为一个现有的项目贡献时，请先阅读并理解这个项目的许可，因为它可能会过度限制你更改代码的权力。有些许可会在代码的所有权方面引起争议。
- 理解这个项目的社区，以及你可以融入其中的程度。要了解项目的主要方向，可以阅读所有文档甚至是代码库的 log。
- 当发起一个合并请求，或者提交一个补丁时，保证它是小改动并且有完善的文档；这将有助于项目维护者理解你的改动，并决定是否合并这些改动或是让你再改一下。
- 如果贡献被拒绝，不要气馁，毕竟这是他们的项目。如果它很重要，请尽可能清楚和耐心地讨论这次贡献的理由，最终可能通过这种方法解决问题。

加快身份验证

每次向 Git 服务器推送时都要认证身份，你可能会想要避免这种麻烦。

- 如果你是用 SSH 密钥来认证的，请使用 SSH agents。参阅 OpenSSH#Speeding up SSH 和 OpenSSH#保活。
- 如果你是用账号和密码来认证的，在服务器支持 SSH 的情况下请切换至 SSH 密钥，否则请尝试 git-credential-cache 或 git-credential-store。

使用 git-credential-libsecret 作为凭证帮助程序

Git 可能会从 org.freedesktop.secrets 兼容密钥环（如 GNOME Keyring、KeePassXC（英语：KeePass#Secret Service） 或 KDE Wallet）获取您的凭据。因此，设置一个兼容的密钥环，并使用以下命令检查密钥环是否已注册到 dbus：

```
$ dbus-send --session --print-reply --dest=org.freedesktop.DBus / \
    org.freedesktop.DBus.GetConnectionUnixProcessID \
    string:org.freedesktop.secrets
```

然后运行

```
$ git config --global credential.helper /usr/lib/git-core/git-credential-libsecret
```

设置 git。

使用 git-credential-netrc 作为凭证帮助程序

Git 可以读取 netrc 文件来访问凭据。首先，将 Git 指向 netrc 帮助程序脚本：

```
$ git config --global credential.helper /usr/share/git/credential/netrc/git-credential-netrc.perl
```

然后，创建 `.netrc` 文件：

```
~/.netrc
machine git-host
login username
password password
```

如果您想保护您的机密安全，凭证帮助程序还支持 gpg 加密文件 (`~/.netrc.gpg`)。

默认通讯协议

如果你正在使用一个上述那种复用的 SSH 连接，让 Git 使用 SSH 可能比使用 HTTPS 更快。同时，一些服务器（比如 AUR）只允许通过 SSH 推送更改。例如，像下面这样配置可以使得 Git 通过 SSH 访问 AUR 上的任何仓库。

```
~/.gitconfig
[url "ssh://aur@aur.archlinux.org/"]
       insteadOf = https://aur.archlinux.org/
       insteadOf = http://aur.archlinux.org/
       insteadOf = git://aur.archlinux.org/
```

Bash 自动补全

要启用 Bash 的自动补全，请在 Bash 启动文件里用 source 加载 `/usr/share/git/completion/git-completion.bash` 文件。或者也可以安装 bash-completion包。

Git 提示符

Git 包带有一个提示符脚本。要启用它，请用 source 加载 `/usr/share/git/completion/git-prompt.sh` 脚本，然后使用 `%s` 参数设置一个自定义 shell 提示符：

- Bash 用户： `PS1='[\u@\h \W$(__git_ps1 " (%s)")]\$ '`
- Zsh 用户： `setopt PROMPT_SUBST ; PS1='[%n@%m %c$(__git_ps1 " (%s)")]\$ '`

> **提示**：Zsh 提供了 `vcs_info` 函数作为替代方案。参见 zshcontrib(1) § GATHERING INFORMATION FROM VERSION CONTROL SYSTEMS。

注意命令替换必须被转义，详情见 Bash/Prompt customization#Embedding commands。持久化配置见命令行解释器#配置文件。

当切换至一个 Git 仓库所在目录时，shell 提示符会变成所在分支名称。也可以配置提示符来显示其他信息：

<table class="wikitable"><caption></caption><tbody><tr><th>Shell 变量</th><th>信息</th></tr><tr><td>GIT_PS1_SHOWDIRTYSTATE</td><td>已暂存 (staged) 显示 <code>+</code>，未暂存 (unstaged) 显示 <code>*</code>。
</td></tr><tr><td>GIT_PS1_SHOWSTASHSTATE</td><td>已储藏 (stashed) 显示 <code>$</code>。
</td></tr><tr><td>GIT_PS1_SHOWUNTRACKEDFILES</td><td>有未跟踪文件时显示 <code>%</code>。
</td></tr><tr><td>GIT_PS1_SHOWUPSTREAM</td><td><code><</code>, <code>></code>, <code><></code> 分别表示落后、领先或偏离上游。
</td></tr><tr><td>GIT_PS1_STATESEPARATOR</td><td>分支名称和状态符号之间的分隔符</td></tr><tr><td>GIT_PS1_DESCRIBE_STYLE</td><td>分离 HEAD 时，相对于标签/分支显示提交
</td></tr><tr><td>GIT_PS1_SHOWCOLORHINTS</td><td>显示颜色
</td></tr></tbody></table>

环境变量的完整文档可在脚本的注释中找到。

注意：

- 如果发生了 `$(__git_ps1)` 返回 `((unknown))` 的情况，是因为有一个 `.git` 文件夹在你当前的文件夹里面，但却不包含任何存储库，因此 Git 不认识它。例如，将 Git 在`~/.gitconfig` 的配置文件误认为在 `~/.git/config` 就会发生这种情况。
- 如果提示符在遇到非常大的存储库时延迟，很可能是由于 `GIT_PS1_SHOWUNTRACKEDFILES` 选项，该选项每次都会触发完整的目录树扫描以检测新文件，从而导致明显的性能影响。要为这些存储库本地禁用此选项，您可以使用命令 `git config --local bash.showUntrackedFiles false`。

你也可以使用来自 AUR 的自定义 git shell 提示符软件包，例如 bash-git-promptAUR 或 gittifyAUR。

可视化显示

要了解已经完成了多少工作：

```
$ git diff --stat
```

带有 fork 显示的 *git log*：

```
$ git log --graph --oneline --decorate
```

给图形化的 *git log* 做一个别名（使用 *git graph* 即可显示经过修饰的 log）：

```
$ git config --global alias.graph 'log --graph --oneline --decorate'
```

关于提交 (commit) 的小提示

重置为以前的提交（非常危险，这会擦除所有跟踪文件到指定提交）：

```
$ git reset --hard HEAD~
```

如果远程仓库的地址发生变化，可以这样更新它的位置：

```
$ git remote set-url origin git@address:user/repo.git
```

自动附加签名行到提交（将某个 姓名-电邮 签名添加到提交中，某些项目会要求这样做）：

```
$ git commit -s
```

自动附加签名到补丁（使用 `git format-patch *commit*` 时生效）：

```
$ git config --local format.signoff true
```

提交已更改文件的特定部分。如果有大量更改时，最好拆分成多个提交，这种情况下这个命令通常很有用：

```
$ git add -p
```

对提交 (commit) 签名

Git 允许使用 GnuPG 对提交和标签进行签名，请参见签署工作。

注意： 如果是借助 pinentry包 来进行 GPG 签名，请确保 `export GPG_TTY=$(tty)`（或者使用 pinentry-tty），否则当 GPG 处于锁定状态时签名这一步会失败（因为它无法在 shell 提示符里询问 pin 码）。

配置 Git 使它自动对提交进行签名：

```
$ git config --global commit.gpgSign true
```

在非主分支上工作

偶尔项目维护人员会要求你在其他分支上完成工作。这些分支通常被称为 `devel` 或 `testing`。首先要克隆存储库。

要进入不是主分支的分支（*git clone* 只会显示主分支，但其他分支其实也是存在的，用 `git branch -a` 可以显示出来）：

```
$ git checkout -b branch origin/branch
```

然后就可以像平常一样编辑文件，但是要使得整个仓库都保持同步，下面这两个命令都要用：

```
$ git pull --all
$ git push --all
```

直接将补丁发送至邮件列表

如果你想直接将补丁发送至一个邮件列表，需要安装以下软件包：perl-authen-sasl包和 perl-io-socket-ssl包。

确保你已经配置了用户名和邮件地址（参见#配置）。

配置你的邮箱设置：

```
$ git config --global sendemail.smtpserver smtp.example.com
$ git config --global sendemail.smtpserverport 587
$ git config --global sendemail.smtpencryption ssl
$ git config --global sendemail.smtpuser foobar@example.com
```

现在你应该可以将补丁发送至某个邮件列表了（可参阅OpenEmbedded: 通过电子邮件发送补丁和 git-send-email.io）：

```
$ git add filename
$ git commit -s
$ git send-email --to=openembedded-core@lists.openembedded.org --confirm=always -M -1
```

远程库很大时的注意事项

使用大型远程存储库时，必须获取大量数据。以下示例使用 Linux 内核来说明如何使用此类代码库。

接收整个仓库

最简单的解决方案是获取整个存储库：

```
$ git clone git://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git
```

注意：`git clone` 不支持断点续传。

可以通过 `git pull` 更新存储库。

部分接收

为了将本地存储库限制为原存储库的较小子集，例如在 v4.14 之后分离出错误，请使用浅克隆 (shallow clone)：

```
$ git clone --shallow-exclude v4.13
git://git.kernel.org/pub/scm/linux/kernel/git/stable/linux-stable.git
```

将获取 v4.14 及更高版本，但不会获取 v4.13 及更早版本。

如果您只想要最新的快照，则可以忽略所有历史记录。（如果有可用且足够完整的 tarball，请选择它。从 git 存储库下载需要更大的带宽。）您可以使用以下命令获取它：

```
$ git clone --depth 1 git://git.kernel.org/pub/scm/linux/kernel/git/stable/linux-
stable.git
```

可以在之后获取较旧的提交，如以下两个示例：

```
$ git fetch --tags --shallow-exclude v4.1
$ git fetch --tags --shallow-since 2016-01-01
```

注意：如果不指定 `--tags`，就不会获取标签。

使用 Scalar

Scalar，前身是 Virtual File System for Git (VFS for Git)，允许无需本地实例访问 git 仓库。

参见 scalar(1)。

获取其他分支

在上面的示例中，本地的仓库仅跟踪主线内核，即"最新开发"的内核。假设你想获取最近的 LTS 内核，比如最新的 4.14 分支，可以这么做：

```
$ git remote set-branches --add origin linux-4.14.y
$ git fetch --shallow-exclude v4.14
$ git branch --track linux-4.14.y origin/linux-4.14.y
```

最后一行不是必须的，但你应该需要执行它。（要获取需要的那个分支的具体名称，没有什么通用的方法，或许可以靠 Gitweb 页面的"ref"链接来猜测）

如果需要 linux-4.14.y 的快照，这样做：

```
$ git checkout linux-4.14.y
```

或者将它解压到其他目录里：

```
$ mkdir /path/to/src-4.14; cd /path/to/src-4.14
$ git clone --no-local --depth 1 -b linux-4.14.y ../linux-stable
```

然后像平常一样，执行 `git pull` 来更新你的快照。

过滤机密信息

有时，软件可能会将纯文本密码保存在配置文件中，而不是挂接到密钥环中。在这些情况下，git clean-filters 可能很方便，可以避免意外提交机密信息。例如，以下文件将过滤器分配给文件"some-dotfile"：

```
.gitattributes
some-dotfile filter=remove-pass
```

每当文件"some-dotfile"添加到 git 时，git 都会在添加之前对文件调用过滤器"remove-pass"。必须在 git 配置文件中定义过滤器，例如：

```
.git/config
[filter "remove-pass"]
clean = "sed -e 's/^password=.*/#password=TODO/'"
```

注意：在这种情况下，转义 sed 表达式的特殊字符可能是一项棘手的任务。请记住，git 会将两个反斜杠变成一个，而 git 调用来运行命令的 shell 会再次将两个反斜杠变成一个。有关更多详细信息，请参阅Git 过滤器和 sed 争夺 \C0（英文）。

HTML 帮助文件

通过安装 git-htmldocsAUR，可以以 HTML 形式获取 `git help` 文档。安装后，可以通过传递 `-w` 标志来访问 HTML 文档。例如：

```
$ git help -w merge
```

可以通过设置 `git config` 选项默认加载 HTML 文档：

```
$ git config --global help.format html
```

扩展

助手和工具

- **git-extras** — Git 实用工具集合（仓库摘要、repl、变更日志、作者提交百分比等）

: https://github.com/tj/git-extras || git-extrasAUR - 如果使用 oh-my-zsh，还可以启用 git-extras 插件

- **gitflow-cjs** — 使用 Vincent Driessen 的分支模型扩展 Git。CJS 版本是一个积极维护的分支。

: https://github.com/CJ-Systems/gitflow-cjs || gitflow-cjsAUR

- **gitmoji-cli** — 一个 gitmoji 交互式 NodeJS 客户端，用于在提交消息中使用 gitmojis。

: https://github.com/carloscuesta/gitmoji-cli || nodejs-gitmoji-cliAUR

大文件同步支持

- **git-annex** — 用 Haskell 编写的分布式文件同步系统，允许使用 Git 管理大数据。

: https://git-annex.branchable.com || git-annex包

多 Git 仓库管理

- **git-bulk** — 用 Bash 编写的命令行工具，用于管理多个 Git 仓库。

: https://github.com/tj/git-extras || git-extrasAUR

- **mani** — 用 Go 编写的命令行工具和 TUI，用于管理多个 Git 仓库。

: https://github.com/alajmo/mani || maniAUR

- **gita** — 用 Python 编写的命令行工具，用于管理多个 Git 仓库。

: https://github.com/nosarthur/gita || gitaAUR

- **mu-repo** — 用 Python 编写的命令行工具，用于管理多个 Git 仓库。

: https://github.com/fabioz/mu-repo || mu-repoAUR

- **myrepos (mr)** — 用 Perl 编写的命令行工具，用于管理多个 Git 仓库。

: https://myrepos.branchable.com/ || myrepos包

参见

- Git 手册页，见 git(1)
- Pro Git book
- GitHub 的 Git Reference
- Git workflow: Forks, remotes, and pull requests
- VideoLAN wiki article
- Git Protocols - Git on the Server
- \[https://gun.io/blog/how-to-github-fork-b

电子设计自动化

EDA 的难点往往不在设计本身，而在让厂商的闭源套件在一个滚动更新的系统上跑起来。本章只有一节，围绕 Xilinx Vivado 的安装、提示和故障排除展开：安装部分绕不开，必须按顺序读完；提示和技巧可以先粗读，需要时回来查；故障排除不必通读，等真出问题再按症状翻。

最容易踩的坑，是把 Vivado 当成一件普通的发行版软件——它并非从仓库里装出来的东西，装完也不等于立刻可用。另一类误解是忽略系统库与套件自带库之间的版本错配，等启动失败了才回头找原因，白白多绕一圈。

17.1　Xilinx Vivado

难度 进阶 | **篇幅** 15.4 KB | **所属** 电子设计自动化

本节将讲到

- 安装
- 提示和技巧
- 故障排除

Arch Linux 并不受 Vivado 的正式支持，但是从 Xilinx ISE WebPACK 的使用体验上看，只需要一点修改便能正常使用所有功能。

安装

可以从官方网址下载Xilinx Vivado。我们建议您下载“Vivado HLx <year>.<version>: All OS installer Single-File Download”。别心急，因为这个文件(写作时最新版本为2020.3)有将近50GB。

AUR包

vivadoAUR AUR 包可以由pacman管理安装. 由于vivado安装程序需要登录后提交个人信息才能下载，因此需要像上面那样手动下载，并将其放置在与 PKGBUILD 相同的目录中. 该软件包只构建最新的主要版本(< 年份 >.< 版本 >) , 而不是较小的更新(< 年份 >.< 版本 >.<更新版本>) 。如果需要，请手动安装 Vivado。

手动安装

依赖

安装程序需要 ncurses5 库，并且不能使用官方仓库里的 ncurses6。您可以通过安装 Arch User Repository 中的 ncurses5-compat-libsAUR 来解决这个问题。您可能需要安装 libpng12包 与 lib32-libpng12包 以便使用 Xilinx Document Navigator。

SDK 2018.3（以及其他可能的版本）需要 gtk2 库。如果您打算使用 SDK，您可能需要安装 gtk2包.Vitis需要安装 xorg-xlsclients包 来正常工作.

可以使用xilinx-vivado-dummyAUR\[损坏的链接：package not found\] 进行替代安装所有这些依赖项。

在一些平铺式窗口管理器上 (例如 dwm 和 Xmonad), 您可能需要在启动 `xsetup` 之前设置这个环境变量(否则安装 GUI 将不呈现).

```
$ export _JAVA_AWT_WM_NONREPARENTING=1
```

您必须安装主程序包，并且还建议安装最新的更新补丁。

默认的字体可能难以阅读，并导致一些 UI 元素被切断错误渲染。通过安装 noto-fonts 包 来修复。

PetaLinux Tools的依赖

在启动 petalinux-tools 之前，请安装:

git包 diffstat包 unzip包 texinfo包 python包 chrpath包 wget包 xterm包 sdlAUR rpcsvc-proto包 socat包 cpio包 inetutils包 python2包 net-tools包 tftp-hpa包 python-virtualenv包 xorg-server-xvfb包 bison包 flex包 gnupg包 ncurses包 autoconf包 libtool包 tar包 gcc包 sdlAUR sdl2包 glib2包 screen包 pax包 pax-utils包 libstdc++5AUR python-django包 iproute2包 lib32-zlib包 openssl包 gawk包 python-pexpect包 python-pip包 python-gitpython包 python-jinja包 xz包 iputils包 python-pylint包

和AUR中的

pod2manAUR libselinuxAUR debianutilsAUR

启用 multilib 库 并安装multilib-devel包组 包组

启用 tftp-hpa包

```
$ sudo systemctl enable tftpd.service
$ sudo systemctl start tftpd.service
```

Vivado 与 SDK

建议在默认位置 `/opt/Xilinx` 安装该套件，因为本页中的进一步说明将假定该套件安装在那里。

警告：2018.1及以后版本不再需要以下补丁。请直接解压后执行 xsetup 文件。

Once downloaded and unpacked the tarball, the install script must be patched to be able to properly detect the machine architecture. You can do it by going to the directory where installer is extracted and running:

```
$ sed -i.original 's/uname -i/uname -m/' xsetup
```
Install script will be patched and original will be backed up as `xsetup.original`, just in case you need to restore it later. Once patched, just run the script; it should work perfect and install the suite without a problem:

```
# ./xsetup
```
It is recommended to install the suite at the default location `/opt/Xilinx`, as further instructions in this page will assume the suite is installed there.

更新补丁

警告：2018.1 及以后版本不再需要以下补丁。

It is recommended to install the latest update patch, and repeat the process each time a new patch is released. Note that update patches cannot be applied to WebPACK installs. If you installed Vivado WebPACK, skip this section.

To install the update, repeat the same hack used to install the suite. Once downloaded and unpacked, go to the directory containing the extracted tarball, patch the install script and run it:

```
$ sed -i.original 's/uname -i/uname -m/' xsetup
# ./xsetup
```

证书

警告：2018.1及以后的 Webpack 版本不再需要手动下载证书文件。

If you already have a license file, you can load it using Vivado License Manager. Unfortunately, if you want to obtain a WebPack license, further steps are needed. Vivado installs old stdc++ libraries, causing problems when spawning programs not included with Vivado Suite (like your default browser). To fix this, do the following steps:

```
# cd /opt/Xilinx/Vivado/2015.4/lib/lnx64.o/
# mv libstdc++.so.6 libstdc++.so.6.orig
# ln -s /usr/lib/libstdc++.so.6
```
Close any running Vivado Suite program, and launch license manager:

```
$ /opt/Xilinx/Vivado/2015.4/bin/vlm
```
If you try obtaining a WebPack license, your default browser should open, and the license should be generated normally. If Vivado License Manager fails to automatically load the generated license, download the .lic file, and manually load it.

Digilent USB-JTAG 驱动

要使用来自 Vivado 的 Digilent Adept USB-JTAG 适配器(例如内置在 ZedBoard 上的 JTAG 适配器)，你需要安装 Digilent Adept Runtime。

确保您已经从 AUR 安装了 fxloadAUR。

要安装 Digilent Adept Runtime，建议从 AUR 安装 digilent.adept.runtimeAUR 。

此外，安装 digilent.adept.utilitiesAUR 可以很好地配置你的开发板。

桌面与应用程序菜单快捷方式

安装过程必须作为 root 用户运行，以便对/opt/Xilinx 进行写访问。如果需要应用程序菜单中的快捷方式，则必须将它们从根帐户移动到/usr/share。桌面快捷方式应该移动到用户桌面。

以下命令假定只有 Xilinx 已经安装到根帐户，并且用户具有用户名.

复制应用程序菜单快捷方式:

```
# mv /root/.local/share/applications/* /usr/share/applications/
# mv /root/.local/share/desktop-directories/* /usr/share/desktop-directories/
# mv /root/.config/menus/applications-merged/* /etc/xdg/menus/applications-merged/
```

复制桌面快捷方式:

```
# chown username /root/Desktop/*
# mv /root/Desktop/* /home/username/Desktop/
```

Linux cable 驱动

以 root 权限运行以下脚本：

```
$
{vivado_install_dir}/data/xicom/cable_drivers/lin64/install_script/install_drivers/inst
all_drivers
```

提示和技巧

启用屏幕缩放功能

启动 Vivado，然后按照以下方式启用屏幕缩放功能：

```
Tools -> Setting -> Display -> Scaling
```

禁用WebTalk

免费的WebPACK许可证不允许你禁用这一功能，该功能在生成比特流时将使用数据上传到赛灵思的服务器，但即使连接失败，综合仍会顺利完成。一个简单的方法是为Vivado工具设置一个无效的HTTPS代理，使其无法链接到服务器。

```
/opt/Xilinx/Vivado/<version>/bin/setupEnv.sh
...

export HTTPS_PROXY=localhost
```

这个方法不会污染您的工作环境，只是在工具启动时配置的临时环境，它不会破坏其他任何东西。

故障排除

Synthesis segfaults

See https://support.xilinx.com/s/feed/0D52E00006hpUycSAE?language=en_US

You will need to recompile glibc (just take the PKGBUILD from the abs) with `--disable-lock-elision`. Instead of patching the system libc in /usr/lib, copy the

newly compiled `libpthread-2.25.so` and `libc-2.25.so` to `/opt/Xilinx/Vivado/2016.4/ids_lite/ISE/lib/lin64` Do not forget to repeat this when glibc gets upgraded.

xsct segfault

xsct might crash with message:

```
 Xilinx/SDK/2018.1/bin/xsct: line 141:  5611 Segmentation fault      (core dumped)
"$RDI_BINROOT"/unwrapped/"$RDI_PLATFORM$RDI_OPT_EXT"/rlwrap -rc -f
"$RDI_APPROOT"/scripts/xsdb/xsdb/cmdlist -H "$HOME"/.xsctcmdhistory
"$RDI_BINROOT"/loader -exec rdi_xsct "${RDI_ARGS[@]}"
```

This is a problem with the rlwrap version bundled with vivado, probably due the lack of legacy vsyscall emulation in Arch Linux. To fix this issue either drop rlwrap altogether (losing history and autocompletion in xsct) or install rlwrap包 from the official repo, and edit the corresponding line in the xsct startup script:

```
../Xilinx/SDK/2018.1/bin/xsct
# Use rlwrap to invoke XSCT
/usr/bin/rlwrap -rc -f "$RDI_APPROOT"/scripts/xsdb/xsdb/cmdlist -H
"$HOME"/.xsctcmdhistory "$RDI_BINROOT"/loader -exec rdi_xsct "${RDI_ARGS[@]}"
# OR run xsct without rlwrap
#"$RDI_BINROOT"/loader -exec rdi_xsct "${RDI_ARGS[@]}"
```

Vivado HLS testbench error with GCC

Vivado requires an older version of glibc (2.26 as of vivado 2018.1).

The solution proposed in this thread\[失效链接 2021-11-19 ⓘ\] from Xilinx forums suggests to update the fixed headers shipped by Xilinx.

For vivado version 2016, run:

```
 # /opt/Xilinx/Vivado_HLS/<version>/lnx64/tools/gcc/libexec/gcc/x86_64-unknown-linux-
gnu/4.6.3/install-tools/mkheaders /opt/Xilinx/Vivado_HLS/<version>/lnx64/tools/gcc/
```

For vivado 2017 and newer, run:

```
 # /opt/Xilinx/Vivado/2018.1/lnx64/tools/gcc/libexec/gcc/x86_64-unknown-linux-
gnu/4.6.3/install-tools/mkheaders /opt/Xilinx/Vivado/2018.1/lnx64/tools/gcc
```

Vivado Crashes with Wayland

If Vivado crashes and the error file contains something similar to this:

```
hs_err_pid*.log
#
# An unexpected error has occurred (11)
#
Stack:
/opt/Xilinx/Vivado/2018.2/tps/lnx64/jre/lib/amd64/server/libjvm.so(+0x923da9)
[0x7fced0c19da9]
/opt/Xilinx/Vivado/2018.2/tps/lnx64/jre/lib/amd64/server/libjvm.so(JVM_handle_linux_sig
nal+0xb6) [0x7fced0c203f6]
/opt/Xilinx/Vivado/2018.2/tps/lnx64/jre/lib/amd64/server/libjvm.so(+0x9209d3)
[0x7fced0c169d3]
/usr/lib/libc.so.6(+0x368f0) [0x7fcf0ea408f0]
/opt/Xilinx/Vivado/2018.2/tps/lnx64/jre/lib/amd64/libawt_xawt.so(+0x42028)
[0x7fceb1a20028]
/opt/Xilinx/Vivado/2018.2/tps/lnx64/jre/lib/amd64/libawt_xawt.so(+0x42288)
[0x7fceb1a20288]
/opt/Xilinx/Vivado/2018.2/tps/lnx64/jre/lib/amd64/libawt_xawt.so(Java_sun_awt_X11_XRobo
tPeer_getRGBPixelsImpl+0x17c) [0x7fceb1a1867c]
[0x7fcec0cb94cf]
```

Switch to using Xorg instead of Wayland. The version of Java Vivado uses has compatibility problems with Wayland.

集成开发环境

把 KDevelop 装上并不等于就能顺手用起来。本章处理的是让这款集成开发环境在 Arch 上真正跑通：安装、功能、为它构建插件，以及出了问题怎么查，目标是从"能启动"走到"能干活"。

只有一节。安装与功能介绍必读，先把环境和主要能力弄清楚；故障排除可以当案头手册，遇到报错再翻；构建其他插件属于扩展开发，不打算写插件就跳过。

最容易踩的坑有两个。一是把"构建其他插件"读成替别的项目做构建，它讲的其实是给 KDevelop 本身编写并编译插件。二是以为启动成功就万事大吉，缺了东西时功能会静默失效，照着故障排除一节逐步排查，比反复重装快得多。

18.1　KDevelop

难度 入门 | **篇幅** 4.7 KB | **所属** 集成开发环境

本节将讲到

- 安装
- 功能
- 构建其他插件
- 故障排除

KDevelop 网站上的介绍：

: KDevelop 是自由和开源软件（FOSS），是一个由 KDE 社区提供的集成开发环境。它支持多种编程语言的编辑、浏览和调试功能，提供自动纠错、代码建议和自动化构建集成工具，支持许多版本控制系统（例如 git）。同时，基于插件的架构提供了足以满足个人需求的功能扩展和定制能力。

KDevelop 5 具有支持 C、C++、Objective-C、OpenCL 和 JavaScript/QML 的解析器，在插件的加持下还支持 PHP、Python3 和 Ruby。尽管不支持其他的源码和标记语言的语义分析，但 KDevelop 依然支持基本的语法高亮和代码折叠。

KDevelop 是 KDE 项目的一部分，基于 KDE 框架和 Qt。C/C++ 后端使用 Clang、clang-tidy 和 heaptrack 来提供准确的信息，应对非常复杂的代码库也没有问题。

安装

安装 kdevelop包 软件包即可开始使用。

功能

KDevelop 通过 KParts 框架使用嵌入式文本编辑器组件，默认编辑器是 KDE Advanced Text Editor（Kate），可以选择将其替换为基于 Qt Designer 的编辑器。

下面的列表重点介绍 KDevelop 本身的功能。

- 支持语法高亮和自动缩进功能的源代码编辑器（Kate）。
- 自 KDevelop 5.0 起使用 Clang 后端支持 C/C++。
- 支持不同项目类型的管理，例如 Automake、CMake、qmake（用于基于 Qt 的项目）和 Ant（用于基于 Java 的项目）。
- 类查看器。
- GUI 设计工具。
- GCC 和 GDB 的前端。
- 用于生成和更新类定义和应用程序框架的向导。
- 自动代码补全（C/C++）。
- 内置 Doxygen 支持。
- 修订控制（也称为 SCM）支持。支持的系统包括 CVS、Subversion、Perforce、ClearCase、Git、Mercurial 和 Bazaar。

KDevelop 4 是一个完全基于插件的架构。当开发人员进行更改时，他们只需要编译插件。

代码补全适用于 C 和 C++，符号保存在 Berkeley DB 文件中，以便快速查找而无需重新解析。

KDevelop 还提供了一个开发者框架，旨在帮助编写其他编程语言的新解析器。

集成的调试器通过图形化方式进行各种调试，包括断点和回溯。不同于命令行式的 GDB，它可以与动态加载的插件一起工作。

Quick Open 可在文件之间快速导航。

插件

目前可用的插件大约有 50 到 100 个。 主要的插件可提供整个项目范围的持久化代码书签保留、支持快速扩展文本的代码缩写、保存文件时进行格式化、正则表达式搜索以及有助于重构代码的项目范围内的代码搜索和替换等功能。

安装插件以提供自动补全和其他特定语言的功能：

- 对于 PHP ，安装 kdevelop-php包 软件包。

- 对于 Python，安装 kdevelop-python包 软件包。
- 对于 C++， 安装 gcc包 或 clang包，两个软件包都安装的情况下也没有影响。选择其他编译器也是可行的。

构建其他插件

KDevelop 解析器生成器（kdevelop-pg-qt包）是构建额外插件所必需的。如果未事先安装此包，插件将无法编译。

故障排除

KDevCMakeManager

如遇到 `Could not load project management plugin KDevCMakeManager` 的错误，请安装 cmake包。

使用 GDB 进行调试

需要安装 okteta包 并重启 KDevelop 后才能使用 gdb包 的调试选项。